book 8

python 3.14

a complete language manual

Raeliosol

version 4.0, compiled 10 2026

Contents

1. toolchain: cpython 3.14 on windows 8
   1.1. the interpreter and the launchers 8
   1.2. running code five ways 10
   1.3. the verify gate 11
   1.4. utf-8 mode and the windows console 11
2. lexical structure 14
   2.1. lines, indentation, blocks 14
   2.2. the token stream 15
   2.3. literals 17
   2.4. names, keywords, and the walrus 19
3. the object model 22
   3.1. names, objects, references 22
   3.2. mutability and copies 23
   3.3. representation and hashing 26
   3.4. gc and weakref 28
4. builtin collections, iteration, and generators 31
   4.1. list, tuple, dict, set internals 31
   4.2. the iterator and iterable protocols 33
   4.3. comprehensions and generators 34
   4.4. heapq, bisect, counter, deque 36
5. strings, bytes, and text 39
   5.1. str and the unicode model 39
   5.2. bytes, bytearray, memoryview 40
   5.3. re in practice 42
   5.4. formatting 43
6. functions, scopes, and decorators 46
   6.1. scopes and closures 46
   6.2. arguments 48
   6.3. decorators 50
   6.4. the functools toolkit 54
7. classes and inheritance 58
   7.1. classes as namespaces and factories 58
   7.2. dunders 59
   7.3. inheritance and the mro 61
   7.4. slots, properties, descriptors 63
8. the typing system 66
   8.1. annotations at runtime 66
   8.2. generics 68
   8.3. the typing toolbox 70
   8.4. protocols 71
9. dataclasses, enums, and pattern matching 75
   9.1. dataclasses 75
   9.2. post-init and ordering 76
   9.3. enums 78
   9.4. match statements 80
10. exceptions and context managers 82
   10.1. the exception tree 82
   10.2. try semantics 84

10.3. exception groups 85
10.4. context managers 88
11. modules, packages, and imports 92
11.1. modules and packages 92
11.2. the import system 93
11.3. project layout 96
11.4. environments and pins 98
12. how cpython runs your code 100
12.1. source to bytecode 100
12.2. frames and cells 102
12.3. the eval loop and specialization 104
12.4. tier 2 and the jit 106
13. pymalloc: arenas, freelists, and tracemalloc 108
13.1. why a private heap 108
13.2. arenas, pools, and blocks 109
13.3. freelists and the 512-byte boundary 110
13.4. what getsizeof measures 111
13.5. tracemalloc attribution 113
13.6. pythonmalloc and the mimalloc option 114
13.7. refcounts and the collector recap 114
13.8. the particle kernel: numpy versus tuples 115
14. profiling: cprofile, sys.monitoring, and p99 119
14.1. cprofile and pstats 119
14.2. clocks: timeit versus perf_counter 120
14.3. percentiles by hand 121
14.4. sys.monitoring, pep 669 121
14.5. the free-threaded sidebar 123
14.6. the particle kernel measured 124
14.7. the row kernel measured 124
14.8. across the six books 126
15. files and serialization 128
15.1. pathlib 128
15.2. open, text vs binary 130
15.3. csv and json 133
15.4. pickle and tempfile 136
16. os, subprocess, and signals 140
16.1. subprocess run and check 140
16.2. popen and pipes 142
16.3. os interfaces 145
16.4. signals on windows 148
17. threads and the gil 152
17.1. threads and the interpreter 152
17.2. the gil 153
17.3. locks and queues 155
17.4. races demonstrated safely 159
18. multiprocessing and process patterns 163
18.1. processes on windows 163
18.2. pools and executors 165
18.3. shared state 167
18.4. process patterns 169
19. operational plumbing 172

19.1. argparse ... 172
19.2. logging ... 173
19.3. datetime and zoneinfo ... 175
19.4. env, secrets, hashing ... 177
20. testing with unittest ... 180
20.1. unittest anatomy ... 180
20.2. mocks ... 182
20.3. doctest and parametrics ... 183
20.4. the suite in the gate ... 185
21. asyncio ... 187
21.1. the event loop ... 187
21.2. tasks and groups ... 189
21.3. queues and producers ... 193
21.4. streams, subprocess, and threads ... 196
22. fastapi ... 201
22.1. the asgi model ... 201
22.2. validation and dependency injection ... 204
22.3. async endpoints, lifespan, and the testclient ... 208
22.4. boundaries ... 211
23. stdlib numerics: math, statistics, decimal, random ... 215
23.1. math and cmath ... 215
23.2. statistics ... 218
23.3. decimal and fractions ... 221
23.4. random ... 224
24. numpy ... 228
24.1. the ndarray ... 228
24.2. vectorization and broadcasting ... 230
24.3. fancy indexing and masks ... 232
24.4. dtypes and promotion ... 234
25. pandas ... 238
25.1. series and dataframe ... 238
25.2. io and copy-on-write ... 240
25.3. reshape and join ... 242
25.4. time series and missing ... 245
26. pydantic ... 248
26.1. basemodel and validation ... 248
26.2. serializers and json ... 251
26.3. composed models ... 252
26.4. boundaries and pins ... 255
27. http kernel ... 258
27.1. the handler shape on http.server ... 258
27.2. the route table, hand rolled ... 260
27.3. the error envelope ... 261
27.4. json in and out ... 262
27.5. the request id ... 262
27.6. testing the kernel without a socket ... 263
27.7. the app skeleton and its wiring table ... 264
28. middleware ... 266
28.1. the chain over the kernel's shape ... 266
28.2. recover ... 266
28.3. request id at the edge ... 267

28.4. the access log ... 268
28.5. the deadline over the injected clock ... 269
28.6. wiring order and why ... 270
28.7. testing the stack ... 271
29. the user resource ... 273
29.1. the representation ... 273
29.2. decoding with strictness ... 274
29.3. validation from first principles ... 275
29.4. patch merge semantics ... 276
29.5. keyset pagination ... 276
29.6. the status contract ... 277
29.7. golden handler tests ... 278
30. authn ... 280
30.1. password storage with pbkdf2 ... 280
30.2. the phc string and the memory-hard alternative ... 280
30.3. login and timing flattening ... 281
30.4. server-side sessions ... 282
30.5. cookie mechanics ... 283
30.6. jwt from scratch with hmac ... 284
30.7. the asymmetric variant in prose ... 285
30.8. the refresh rotation and reuse detection ... 286
30.9. testing auth without walls ... 286
31. authz and rbac ... 288
31.1. authentication versus authorization ... 288
31.2. roles and the grant table ... 288
31.3. policies as functions ... 289
31.4. 401 versus 403 ... 290
31.5. object-level authorization ... 290
31.6. deny by default with a route audit ... 291
31.7. the rbac matrix test ... 292
32. store ... 294
32.1. db-api, the bundled engine, and the connection budget ... 294
32.2. migrations test driven ... 295
32.3. queries with parameters every time ... 296
32.4. the transaction idiom ... 297
32.5. isolation levels and what sqlite actually gives you ... 298
32.6. the lost update demo and the cas answer ... 299
32.7. analytics with window functions ... 300
32.8. store tests without docker ... 300
33. concurrency control ... 302
33.1. the lost update prevented ... 302
33.2. if-none-match and 304 ... 303
33.3. idempotency keys ... 303
33.4. singleflight ... 305
33.5. the primitives revisited ... 305
33.6. bounded fan-out with first error ... 306
33.7. the race canary without a detector ... 307
34. caching ... 309
34.1. what caching promises ... 309
34.2. http caching semantics for private apis ... 310
34.3. lru from first principles ... 310

34.4. ttl beside lru ... 311
34.5. the stampede and its guard ... 312
34.6. invalidation on write ... 313
34.7. what not to cache ... 314
35. rate limiting ... 315
35.1. why limits exist ... 315
35.2. the token bucket from first principles ... 316
35.3. the stdlib has no rate limiter ... 316
35.4. the 429 contract ... 317
35.5. what to key on ... 318
35.6. limits under test ... 319
35.7. admission preview ... 320
36. observability ... 322
36.1. logging the real setup ... 322
36.2. structured beats interpolated ... 323
36.3. the request context in contextvars ... 323
36.4. tracing from scratch ... 324
36.5. metrics and the prometheus text format ... 325
36.6. p99 done honestly ... 326
36.7. one request, every signal ... 327
37. load and p99 ... 329
37.1. admission control ... 329
37.2. the drain contract ... 330
37.3. the exit code reports the drain ... 331
37.4. testing the drain without sleeps ... 331
37.5. the load harness ... 332
37.6. the report and honest percentiles ... 333
37.7. the thread census ... 333
38. ship it ... 335
38.1. the image ... 335
38.2. compose, one command ... 337
38.3. the docker lane as a gate ... 338
38.4. the shipped checklist ... 340
39. the test suite ... 343
39.1. what the suite holds ... 343
39.2. the unittest surface ... 344
39.3. doubles and unittest.mock ... 344
39.4. clocks, events, zero sleeps ... 345
39.5. properties as seeded loops ... 346
39.6. dated and declined ... 347
39.7. coverage, opt in ... 348
39.8. what the suite proves ... 349
40. capstone: ingest and analysis pipeline, part 1 ... 351
40.1. the capstone contract ... 351
40.2. sources and fetch ... 352
40.3. pydantic at the boundary ... 356
40.4. the queue pipeline ... 357
40.5. profiled: the ingest pipeline ... 360
40.6. what part 1 proves ... 361
41. capstone: ingest and analysis pipeline, part 2 ... 364
41.1. analytics and the pool ... 364

41.2. the cli and logging 366
41.3. the golden report 368
41.4. the served api 371
41.5. version pins asserted 373
41.6. profiled, and why 374
41.7. honest boundaries 375
41.8. the service part after the capstone 376
42. appendices: coverage and sources 377
42.1. topic matrix 377
42.2. sources ledger 377
42.3. the version drift rule 378

1. toolchain: cpython 3.14 on windows

Every claim in this book runs before it is printed. This chapter pins the interpreter that makes that sentence true: cpython 3.14.7, the current patch of the 3.14 line, installed per user and reached through the py launcher. Another python, 3.13.15 and also current, sits earlier on PATH and never runs a sample. Ruff 0.16.7 plays the role clang-format plays in the c book: the formatter and linter every sample must satisfy before it counts as green, pinned tools-only in the same requirements file as the libraries.

1.1. the interpreter and the launchers

Windows installs are per user by default, and this machine follows the default: the 3.14 line lives under `%LOCALAPPDATA%\Programs\Python\Python314`, and the py launcher lists it as `-V:3.14`. The launcher is the honest entry point precisely because bare `python` is not: on this box `python` resolves to a 3.13.15 in the same tree, one major line older. The gate in `tools/run-py-samples.ps1` therefore resolves the interpreter itself, trying `py -3.14` first, then PATH candidates that report 3.14, then the per-user fallback path, and refuses to run at all when none of them answers.

The gate also owns a venv under `tools\python\build\`, its directory name derived from the requirements hash, created with the resolved interpreter and filled from `books/python/requirements.txt`: pydantic 2.13.4, pandas 3.0.5, numpy 2.5.3, fastapi 0.141.1, and ruff as the tools-only formatter, plus httpx2 2.12.0 for the test suites that drive fastapi's TestClient on the current starlette line, never imported by a sample, and starlette 1.6.0 itself, pinned so the venv reproduces the exact asgi base chapter 22 asserts. The sha256 of the requirements file names the venv directory: a changed pin builds a fresh venv alongside the old one instead of deleting a venv another run may still be using, and an unchanged hash reuses the built one. The first sample asserts the whole chain from inside:

```python
# chapter 1: the exact pin and the interpreter's own identity
import os
import platform
import sys
from pathlib import Path

checks = 0


def ok(condition, name):
    global checks
    checks += 1
    if not condition:
        raise SystemExit(f"FAIL {name} (check {checks})")
    print(f"ok {checks} {name}")


def main():
    ok(platform.python_version() == "3.14.7", "cpython 3.14.7 exactly")
    ok(sys.version_info[:3] == (3, 14, 7), "sys.version_info agrees")
    ok(sys.implementation.name == "cpython", "implementation is cpython")
    ok(sys.implementation.version[:3] == (3, 14, 7), "sys.implementation agrees")
    ok(platform.python_implementation() == "CPython", "platform agrees")
    ok(os.name == "nt" and sys.platform == "win32", "windows target")


def venv():
    ok(sys.prefix != sys.base_prefix, "running inside a venv")
    exe = Path(sys.executable).resolve()
    ok(
        exe.parent.name == "Scripts"
        and any(part.startswith("venv-") for part in exe.parts),
        "the harness venv",
    )
    base = Path(sys._base_executable).resolve()
    ok(base.parent.name == "Python314", "cloned from the per-user 3.14")


if __name__ == "__main__":
    main()
    venv()
```

listing 1: the check contract every sample carries, the exact 3.14.7 pin, and the venv asserts (source: python/samples/src/Ch01/version.py, lines 1-42)

The `ok` function is the whole testing convention of this book: a check prints one `ok N name` line and moves on, or raises `SystemExit` with a `FAIL` message and a nonzero exit. The harness counts the `ok` lines, fails on any nonzero exit, and the chapter's closing callout quotes the count. The version checks are deliberately exact: a 3.14.8 that arrives tomorrow turns this sample red until the book re-pins, and that is the point. Nothing is tested by eyeball.

NOTE: why samples assert the venv too

The last three checks pin the launch chain, not just the version: `sys.prefix` differing from `sys.base_prefix` proves a venv is active, the executable living under `Scripts` proves the gate's venv is the one running, and `sys._base_executable` still pointing into `Python314` proves the resolution ended at the per-user 3.14, never at the PATH python. A wrong interpreter is a loud red before any behavioral claim runs.

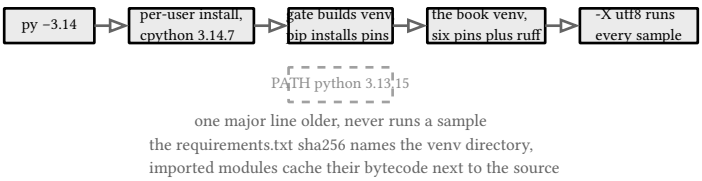


figure 1: the launch chain, from the py launcher to the venv that runs every sample

1.2. running code five ways

The interpreter has five everyday entrances, and each one answers the same three questions differently: what leads `sys.path[0]`, what sits in `sys.argv[0]`, and which module owns the name `__main__`. A script run by path puts its own directory first on the path and its own path in `argv[0]`. A module run with `-m` puts the cwd first and the module file's full path in `argv[0]`. A string run with `-c` puts an empty path entry that reads as the cwd and the literal `-c` in `argv[0]`, probed on this machine. The repl reads from stdin with no file and an empty `argv[0]`. A console entry point, the exe a pinned library can install, is a generated wrapper whose target is the package's main hook. The gate uses the first way exclusively: absolute script path from a foreign working directory, so a sample that wants to know where it stands must ask, not assume:

```
# chapter 1: how the interpreter was invoked, visible from inside
import sys
from pathlib import Path

checks = 0


def ok(condition, name):
    global checks
    checks += 1
    if not condition:
        raise SystemExit(f"FAIL {name} (check {checks})")
    print(f"ok {checks} {name}")


def main():
    ok(__name__ == "__main__", "the script is the main module")
    ok(sys.flags.interactive == 0, "not the repl")
    ok(Path(sys.argv[0]).resolve() == Path(__file__).resolve(), "argv[0] is the script")
    here = Path(sys.path[0]).resolve()
    ok(here == Path(__file__).resolve().parent, "sys.path[0] is the script dir")
    ok(Path.cwd().resolve() != here, "cwd is a fresh directory elsewhere")


if __name__ == "__main__":
    main()
```

listing 2: invocation facts checked from inside: main module, argv, path head, foreign cwd (source: python/samples/src/Ch01/invocation.py, lines 1-26)

The five checks are the ones a harness can see. `__name__` equal to `__main__` is the fact the guard at the bottom tests for: it lets the same file serve as a script and as an importable module without re-running its checks, and it is the guard multiprocessing needs on windows, which chapter 18 relies on. The `interactive` flag being 0 separates this run from the repl. The last two checks encode the gate's promise: the script's directory leads the path, and the cwd is somewhere else entirely, a fresh temp directory per sample, so no sample can accidentally read a file it did not create.

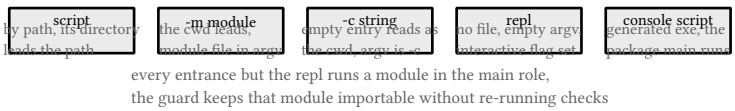


figure 2: the five entrances and what each one sets

1.3. the verify gate

`make verify-py` runs the harness in `tools/run-py-samples.ps1`. It resolves the 3.14 interpreter, then walks every chapter's samples through six legs: toolchain, venv, run, format, expect-dis, capstone. The run leg executes each sample under the venv python with `-X utf8` from a fresh temp directory and counts its `ok` lines. The format leg runs ruff check and ruff format over the whole book directory against `books/python/ruff.toml`: 88 columns, py314, a lint select frozen to the classic error set, because ruff 0.16 grew its default rules from 59 to 413 and a minor upgrade must not redden a pinned book. The expect-dis leg reads pipe-separated rows from each chapter's `expect-dis.txt` and asserts the named substring appears in the named function's bytecode, through `tools/py-dis-check.py`, which compiles the source without executing it and matches nested code objects by qualname. Only stable base opnames may be asserted, because cpython 3.14 specializes instructions as they warm up. The capstone leg runs `unittest discover` over the capstone suite under `books/python/capstone`, 78 tests across 10 files, and counts them in the same report line; if that suite were ever absent the leg would say so and pass with zero tests, stated rather than skipped silently. Any leg failing turns the target red, and `make verify` fails with it.

VERIFY: the gate was written test first

While proving the harness, `version.py` briefly asserted 3.14.6. The run leg answered `FAIL cpython 3.14.6 expected (check 1)` and exited 1, turning the whole gate red. Restoring the exact pin turned it green again. That red-then-green pair is the evidence the harness reports failures rather than rubber-stamping output.

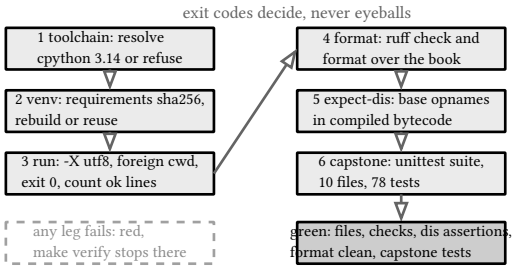


figure 3: one sample through the six legs of the verify gate

1.4. utf-8 mode and the windows console

Windows python already does the right thing where the user looks: an interactive console reads and writes through the wide-character api, utf-8 end to end, with or without any flag. The hazards live one layer down, in what the locale decides when no encoding is named. Probed on this machine: `locale.getpreferredencoding(False)` answers cp1252 in a plain run, and that answer is what `open()` uses when no `encoding=` is passed and what redirected output falls back to. The command line flag `-X utf8`, or the environment variable `PYTHONUTF8` which is the same switch, flips the locale answer to utf-8, and the gate passes the flag to every sample run:

```python
# chapter 1: utf-8 mode, what -X utf8 changes on windows
import locale
import sys
from pathlib import Path

checks = 0


def ok(condition, name):
    global checks
    checks += 1
    if not condition:
        raise SystemExit(f"FAIL {name} (check {checks})")
    print(f"ok {checks} {name}")


def main():
    ok(sys.flags.utf8_mode == 1, "the -X utf8 flag is set")
    ok(locale.getpreferredencoding(False) == "utf-8", "preferred encoding")
    ok(sys.stdout.encoding == "utf-8", "piped stdout is utf-8")
    text = "héllo wörld ∧ 世界\n"
    probe = Path.cwd() / "utf8-probe.txt"
    probe.write_text(text)  # no encoding=, the flag carries it
    ok(probe.read_text() == text, "default open encoding round trips")
    probe.unlink()


if __name__ == "__main__":
    main()
```

listing 3: utf-8 mode from inside: the flag, the locale answer, and a default-encoding round trip (source: python/samples/src/Ch01/utf8.py, lines 1-29)

The probe check is the one that would hurt without the flag: the text contains characters cp1252 cannot encode, so `write_text` with no encoding argument would raise on this box instead of round tripping. One wrinkle the probe surfaced: this machine exports `PYTHONIOENCODING=utf-8:surrogateescape`, which already covers pipes by itself, so the piped-stdout check proves the gate's guarantee rather than the flag alone. Chapter 15 makes the habit explicit anyway: `encoding=` on every open, flag or no flag. And the boundary is moving: pep 686 makes utf-8 mode the default in 3.15, at which point the left column of the figure below becomes the historical one.

PITFALL: the console is not where utf-8 mode bites

Interactive stdout and stderr on windows use the console's wide-character api and are utf-8 regardless of any flag. What the flag changes is the locale-preferred encoding that `open()`, pipes, and redirected streams fall back to when nothing names an encoding. Testing by printing to a terminal proves nothing; the round-trip check writes a file, which is where cp1252 actually bites.

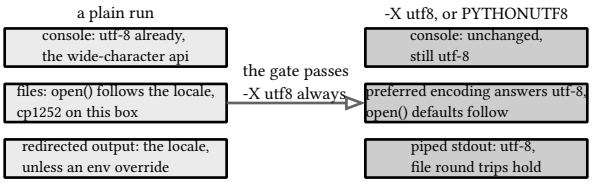


figure 4: what changes with -X utf8: the console never did, files and pipes do

VERIFY: what a green run proves

> A scoped run, `pwsh tools/run-py-samples.ps1 -Chapter Ch01`, walks this chapter's three samples and reports: 3 files, 18 checks, 0 dis assertions, format clean, 0 capstone tests. The dis leg is fully live but chapter 1 pins no bytecode rows; chapter 2 starts asserting base opnames. The capstone leg belongs to no chapter, so the scoped run skips it with a stated line and reports zero; the unscoped `make verify-py` runs the suite itself, 78 tests across 10 files.

sources: docs.python.org/using/windows (per-user installs, the py launcher), docs.python.org/using/cmdline (-X utf8, PYTHONUTF8, -c, -m), docs.python.org/library/sys (flags, implementation, path, argv), docs.python.org/library/venv, docs.python.org/library/locale (getpreferredencoding), peps.python.org/pep-0686 (utf-8 mode by default in 3.15), accessed 2026-09-12. Interpreter paths, encodings, and the PYTHONIOENCODING export probed on this machine the same day. Sample behavior verified by `make verify-py`, 18 checks in chapter 1 of the samples suite.

2. lexical structure

Before a program has objects or functions it is text, and cpython walks a fixed path from text to bytecode: physical lines become logical lines, logical lines become a stream of tokens, and the parser consumes that stream. This chapter covers that front half, plus the two ways source text names things: identifiers against the keyword lists, and the assignment expression `:=`. Every claim is pinned by one of the 41 checks across the four samples, run on cpython 3.14.7 through the gate from book 8, ch 1 (python 3.14, toolchain: cpython 3.14 on windows), or by a syntax diagnostic captured on this machine.

2.1. lines, indentation, blocks

A physical line ends at a newline character. A logical line is what the tokenizer hands the parser, and most of the time the two coincide. Two constructs join physical lines without ending the logical one: a backslash as the last character, and anything inside `(`, `[`, or `{`. A third rule empties lines out: a line that is blank or holds only a comment produces no logical line at all. The sample pins all of it, and because the book formats its samples with ruff, which rewrites backslash joins into bracketed joins and deletes blank lines inside brackets, the join facts are probed through `compile()` strings so the listings stay stable:

```
IMPLICIT = [
    1,
    2,
]

ns_join = {}
exec(compile('JOINED = "ab" \\\n    "cd"', "<join>", "exec"), ns_join)
check(
    ns_join["JOINED"] == "abcd",
    "a backslash join glues two physical lines into one string",
)
check(IMPLICIT == [1, 2], "newlines inside brackets join lines implicitly")
check(
    eval(compile("[1,\n\n 2]", "<blank>", "eval")) == [1, 2],
    "blank lines inside brackets are ignored",
)
```

listing 4: bracket joins carry newlines, backslash joins and blank lines survive as compile probes (source: python/samples/src/Ch02/lines.py, lines 18-33)

The bracket join on line 18 is the spelling the formatter itself emits. The two probes under it compile source held in strings: a backslash join produces one 4 character string, and a blank physical line between the elements of a bracketed list is simply ignored. The backslash is the more fragile of the two joins because nothing may follow it, not even a comment:

```
try:
    compile("y = 1 + \\ # comment\n2\n", "<join>", "exec")
    check(False, "comment after continuation")
except SyntaxError as exc:
    check(
        "unexpected character after line continuation" in str(exc),
        "a comment after the continuation backslash is a syntax error",
    )
```

listing 5: a comment after the continuation backslash is a hard error (source: python/samples/src/Ch02/lines.py, lines 35-42)

The diagnostic on this build reads `unexpected character after line continuation character` (probed 2026-09-12). Indentation is the other half of line structure. The reference fixes no width: any consistent step works, tabs measure as one to eight spaces advancing to the next multiple of 8, and the interpreter rejects a layout whose meaning depends on the worth of a tab with a `TabError`:

```
TABS = "def f():\n\tif 1:\n\t\treturn 7\n\treturn 0\n"
ns = {}
exec(compile(TABS, "<tabs>", "exec"), ns)
check(ns["f"]() == 7, "consistent tab indentation compiles and runs")

TWO_SPACE = "def g():\n  return 5\n"
ns2 = {}
exec(compile(TWO_SPACE, "<two>", "exec"), ns2)
check(ns2["g"]() == 5, "indentation width is a choice, consistency is not")

MIXED = "def f():\n        if 1:\n\t\treturn 7\n        return 0\n"
try:
    compile(MIXED, "<mixed>", "exec")
    check(False, "mixed indentation accepted")
except TabError:
    check(True, "same column via different tab space mixes raises TabError")
```

listing 6: tab consistent and two space layouts compile, the mixed layout cannot (source: python/samples/src/Ch02/lines.py, lines 44-59)

The first source uses tabs throughout and returns 7. The second indents with 2 spaces, proving width is a project choice. The third interleaves them: the `if` body sits at column 8 via spaces, the nested block at column 16 via tabs, and the closing `return` at column 8 via spaces again, so the same column is reached by different mixes and the stack comparison turns ambiguous.

PITFALL: the formatter and the join spellings

Running `ruff format` on a file that uses a backslash join or a blank line inside brackets rewrites both: the backslash becomes a bracketed join, the blank line disappears (probed with ruff 0.16.7 on 2026-09-24). That is why the samples pin these facts through `compile()` probes instead of executable spellings, and why a project should pick the bracket join and never mix tabs into indentation.

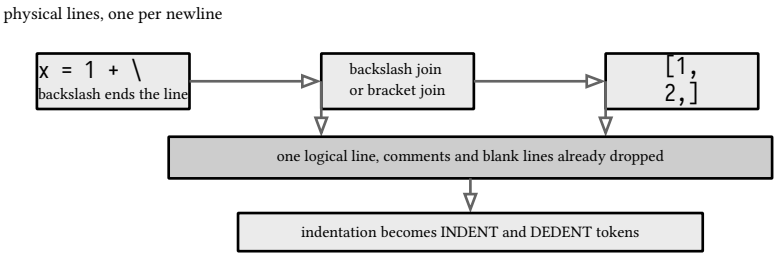


figure 5: physical lines to logical lines, where each joining rule fires

2.2. the token stream

The `tokenize` module exposes the lexer's actual output. Every lexeme lands in a category, comments and newlines are tokens like any other, and the surprising entries are the ones people reach for last:

```
SOURCE = "x = 1  # note\nif x:\n    y = x\n"
names = [
    token.tok_name[t.type] for t in tokenize.generate_tokens(StringIO(SOURCE).readline)
]
strings = [t.string for t in tokenize.generate_tokens(StringIO(SOURCE).readline)]
check(
    "NAME" in names and "NUMBER" in names and "OP" in names,
    "every lexeme lands in a category: NAME, NUMBER, OP",
)
check(strings[names.index("COMMENT")] == "# note", "comments are tokens")
check(
    names.count("NEWLINE") == 3, "each logical line, headers included, ends in NEWLINE"
)
```

listing 7: categories, the comment token, and the NEWLINE count, headers included (source: python/samples/src/Ch02/tokens.py, lines 22-34)

The count is 3 because `if x:` is itself a logical line: the compound statement header ends in `NEWLINE` like any other. Keywords are not a token category of their own, `if` arrives as a `NAME`, and the soft keyword distinction happens later, at the parser. Non-logical newlines get their own category, and indentation is delivered as paired tokens:

```
NOISE = "x = 1\n\n# only a comment\ny = 2\n"
noise = [
    token.tok_name[t.type] for t in tokenize.generate_tokens(StringIO(NOISE).readline)
]
check(
    "NL" in noise and noise.count("NEWLINE") == 2,
    "blank and comment only lines end in NL, not NEWLINE",
)

BLOCK = "if 1:\n    a = 1\n    if 2:\n        b = 2\n"
block = [
    token.tok_name[t.type] for t in tokenize.generate_tokens(StringIO(BLOCK).readline)
]
check(
    block.count("INDENT") == 2 and block.count("DEDENT") == 2,
    "indentation arrives as INDENT and DEDENT tokens",
)
```

listing 8: NL against NEWLINE, and INDENT/DEDENT pairs for the nested block (source: python/samples/src/Ch02/tokens.py, lines 36-52)

Blank and comment-only lines end in `NL`, never `NEWLINE`, which is the tokenizer's way of saying they belong to no logical line. The block probe counts 2 `INDENT` and 2 `DEDENT` for two nested levels, the stack the reference describes. Since 3.12 the tokenizer also owns f-strings (PEP 701), and they arrive as three token kinds, not one string:

```
FSTRING = 'a = f"{x} tail"\n'
fs = [
    token.tok_name[t.type] for t in tokenize.generate_tokens(StringIO(FSTRING).readline)
]
check(
    "FSTRING_START" in fs and "FSTRING_MIDDLE" in fs and "FSTRING_END" in fs,
    "an f-string is start middle end tokens, not one STRING",
)
```

listing 9: an f-string lexes to start, middle, and end tokens (source: python/samples/src/Ch02/tokens.py, lines 54-61)

Before 3.12 the whole literal was pre-digested into one `STRING` token with the formatting work done by a separate pass. The PEP 701 world means an f-string can contain quotes matching its own delimiters and nest arbitrarily, a fact the formatting chapter puts to work. The keyword lists close the section:

```
check(len(keyword.kwlist) == 35, "35 hard keywords cannot be used as names")
check(
    "match" not in keyword.kwlist and "type" not in keyword.kwlist,
    "match and type are not on that list",
)
check(
    list(keyword.softkwlist) == ["_", "case", "match", "type"],
    "the soft keywords are exactly _, case, match, type",
)

ns = {}
exec(compile("match = 1\ntype = match + 1\n", "<soft>", "exec"), ns)
check(ns["type"] == 2, "soft keywords bind as ordinary names")
```

listing 10: 35 hard keywords, 4 soft ones, and soft keywords binding as names (source: python/samples/src/Ch02/tokens.py, lines 63-75)

NOTE: soft keywords are parser decisions, not token categories

`match`, `case`, `_`, and `type` act as keywords only inside match and type statements. The tokenizer still emits them as `NAME` tokens, which is exactly why the compile probe at the bottom can bind `match` and `type` as variables. The hard list has 35 entries, and `match` joined the soft side when the match statement landed in 3.10, `type` in 3.12.

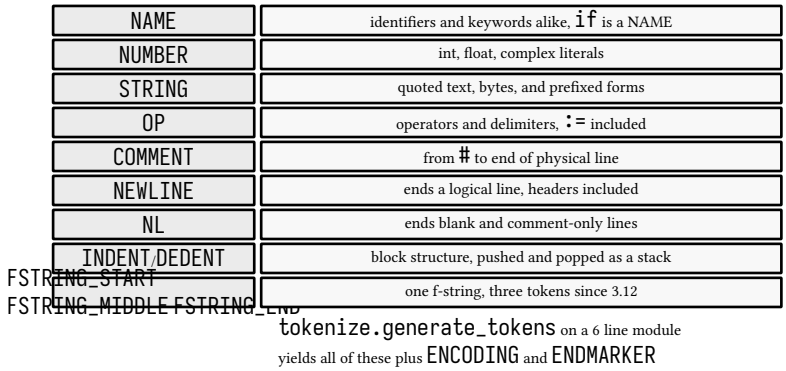


figure 6: the token categories of one small module, with the f-string split open

2.3. literals

Numeric literals carry grouping underscores (PEP 515): single underscores between digits, and, since the grammar was written down this way, one underscore may also follow a base prefix. The reference's own examples are `0x_1f` valid against `0_x1f` and `0x__1f` rejected:

```
check(1_000_000 == 10**6, "underscores group integer digits for the reader")
check(0x_FF == 255 and 0b_101 == 5, "an underscore may follow the base prefix")
check(1_000.5 == 1000.5 and 1_0.25 == 10.25, "floats group the same way")

rejected = []
for bad in ("10_", "1__0", "1_.5"):
    try:
        eval(bad)
    except SyntaxError:
        rejected.append(bad)
check(len(rejected) == 3, "doubled, trailing, and leading dot separators are rejected")

check(2j**2 == -4 and (3 + 4j).imag == 4.0, "the j suffix makes a complex literal")
```

listing 11: legal grouping, the base prefix rule, three rejected placements, and the j suffix (source: python/samples/src/Ch02/literals.py, lines 19-31)

Strings carry prefixes, and the prefixes compose: `r` keeps backslashes as text, `b` makes bytes, `f` interpolates, `rb` combines the first two, and 3.14 adds `t`, which builds a template instead of a string (PEP 750):

```
check(len(r"\n") == 2 and len(rb"\n") == 2, "r and rb keep the backslash as text")
check(len(b"\n") == 1 and b"\x41" == b"A", "bytes literals hold byte values")
check(f"{2 + 2}" == "4", "the f prefix interpolates an expression")

template = t"a {1 + 1} b"
check(type(template).__name__ == "Template", "the t prefix makes a template, pep 750")
check(
    type(template).__module__ == "string.templatelib",
    "templates come from string.templatelib",
)
```

listing 12: prefix behavior, and the new t prefix landing in string.templatelib (source: python/samples/src/Ch02/literals.py, lines 33-42)

Escapes and adjacency follow. `\xNN` names a byte of text, `\N{...}` names a character by its unicode name, triple quotes span physical lines, and two adjacent literals concatenate at compile time, which the sample again proves through a `compile()` probe so the formatter cannot merge the demonstration away. Unknown escapes are a warning, not an error, because they may one day mean something:

```
check("\x41" == "A" and "\N{BULLET}" == "•", "escapes name code and character")
ns_pair = {}
exec(compile('pair = "a" "b"', "<adjacent>", "exec"), ns_pair)
check(ns_pair["pair"] == "ab", "adjacent string literals concatenate at compile time")
TRIPLE = """ab
cd"""
check(len(TRIPLE) == 5, "triple quotes span physical lines")

with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    compile("'\\q'", "<esc>", "eval")
check(
    caught and caught[0].category is SyntaxWarning,
    "unknown escapes warn, they do not error",
)
```

listing 13: escapes, adjacency, triple quotes, and the SyntaxWarning for `\q` (source: python/samples/src/Ch02/literals.py, lines 44-58)

The warning text on this build reads `"\q" is an invalid escape sequence.`
`Such sequences will not work in the future` (probed 2026-09-12). Treating it as noise is how code breaks on a future upgrade, the same discipline the c book applies to its deprecation diagnostics.

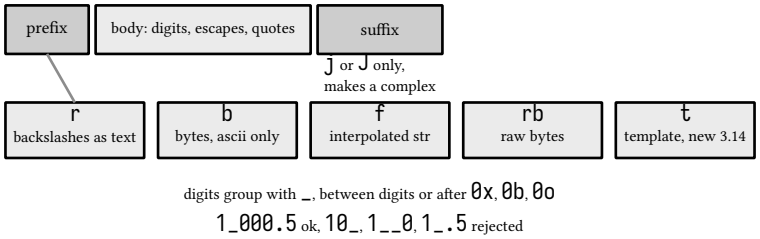


figure 7: the anatomy of a literal: prefix, body, and what each prefix produces

2.4. names, keywords, and the walrus

Identifiers follow unicode: any name that reads as an identifier in the source's script works, so π and é bind, while `2x` dies with `invalid`
`decimal literal` because the lexer claims it as a number first (probed 2026-09-12). The walrus operator `:=` (PEP 572) is assignment that lives inside an expression, and its grammar fences are steep enough that the sample probes them all through `compile()`:

```
DATA = [10, 20, 30, 40]


def walrus_limit(data):
    if (n := len(data)) > 3:
        return n
    return 0


check(walrus_limit(DATA) == 4, ":= assigns and tests inside the if condition")

chunks = ["a", "b", None, "c"]
seen = []
stream = iter(chunks)
while (item := next(stream)) is not None:
    seen.append(item)
check(seen == ["a", "b"], "the while next idiom stops at the sentinel")
```

listing 14: the condition form and the while next idiom, the two everyday uses (source: python/samples/src/Ch02/walrus.py, lines 18-34)

Both uses do the same job: name a value at the point where it is produced so the condition and the body share it without a second call. In a comprehension the walrus binds in the enclosing function scope, not the comprehension's own scope:

```
def filtered(values):
    picked = [y for x in values if (y := x * x) > 100]
    return picked, y


picked, bound = filtered([4, 12, 5])
check(
    picked == [144] and bound == 25, "a walrus in a comprehension binds in the function"
)
```

listing 15: the filter's walrus leaves `y` in `filtered`'s scope, alive after the comprehension (source: python/samples/src/Ch02/walrus.py, lines 37-45)

That rule is the deliberate exception to comprehension scoping, and it is why the technique pairs with a filter that wants to reuse the computed value. The fences:

```
def diagnose(src):
    try:
        compile(src, "<walrus>", "exec")
        return None
    except SyntaxError as exc:
        return str(exc)


check(diagnose("n := 1") is not None, "a bare walrus statement is a syntax error")
check("attribute" in diagnose("a.b := 1"), "attributes cannot be walrus targets")
check(diagnose("x := y := 0") is not None, "unparenthesized chains are rejected")
check(diagnose("def f(a := 2): pass") is not None, "no walrus in a parameter list")
check(
    "__debug__" in diagnose("assert (__debug__ := 1)"),
    "__debug__ refuses walrus assignment, new in 3.14",
)
```

listing 16: five rejections, each a compile error with a stable message (source: python/samples/src/Ch02/walrus.py, lines 48-63)

Four of the five are old PEP 572 rules: no bare walrus statement, no attribute target, no unparenthesized chaining, no walrus in a parameter list. The fifth is new in 3.14: assigning to `__debug__` through a walrus is now a `SyntaxError` with the message `cannot assign to __debug__` regardless of the `-O` flag, closing a hole where optimized builds accepted it. What survives:

```
ns = {}
exec(compile("z = (y := (w := 0) + 1)", "<chain>", "exec"), ns)
check(ns["z"] == 1 and ns["y"] == 1 and ns["w"] == 0, "parenthesized chains nest fine")

ns2 = {}
exec(compile("f = lambda: (k := 5)\nresult = f()\n", "<lam>", "exec"), ns2)
check(ns2["result"] == 5, "a lambda body may carry a parenthesized walrus")
```

listing 17: parenthesized chains and a lambda body walrus both compile (source: python/samples/src/Ch02/walrus.py, lines 65-71)

The bytecode view backs the source view. The expect-dis file beside the samples pins five rows against `walrus_limit`, and the assignment compiles to a `COPY` of the computed length followed by `STORE_FAST`, which is the whole operator in two opcodes. The rows assert base opnames only, because specialization warms up per run.

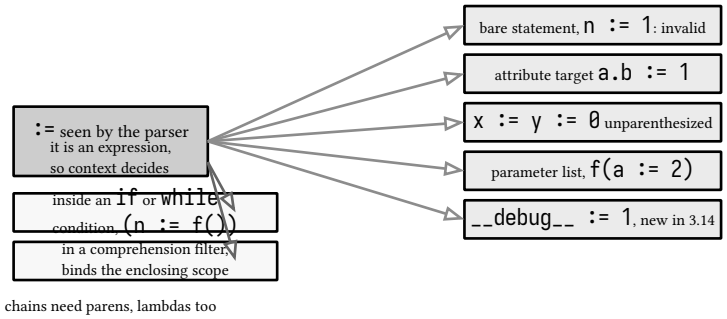


figure 8: where a walrus parses and where it is refused

VERIFY: 41 checks, and two red drafts behind them

The first run of the token sample failed its own NEWLINE count: the check expected 2 and the interpreter reported 3, because the `if x:` header is itself a logical line. The first formatting pass was the second red: `ruff format` joined the backslash-continued string and the adjacent literals into single literals, silently deleting the very constructs the checks demonstrate, so those demonstrations moved into `compile()` probes. Both failures are the gate doing its job. The expect-dis rows for `walrus_limit` were verified with `dis.get_instructions` before landing. Sample behavior verified by `make verify-py`, 41 checks in chapter 2 of the samples suite.

sources: docs.python.org/3/reference/lexical_analysis.html (line structure, indentation and the TabError rule, numeric literal grammar including `0x_1f`, keywords and soft keywords, f-string parenthesization) and docs.python.org/3/reference/expressions.html (assignment expression grammar), docs.python.org/3/library/tokenize.html, library/token.html, library/keyword.html, docs.python.org/3/whatsnew/3.14.html (t prefix, `__debug__` refusal, incompatible prefix diagnostics), peps.python.org/pep-0515/, pep-0572/, pep-0701/, and pep-0750/, all accessed 2026-09-12; the diagnostics for the continuation comment, the mixed indentation TabError, `2x`, the `\q` warning, and the walrus rejections were probed on this machine the same day. Sample behavior verified by `make verify-py`, 41 checks in chapter 2.

3. the object model

Everything a python program touches at runtime is an object, and every name is only a binding to one. This chapter walks the model in four steps: names and references, mutability and copying, the representation and hashing protocols that decide what may sit in a dict key, and the memory story, reference counting first, the generational collector second, weak references third. Every claim is pinned by one of the 42 checks in the four samples, run on cpython 3.14.7, or stated as a measured fact of this build.

3.1. names, objects, references

Assignment never copies. `a = []` creates the object and then binds the name `a` to it, so a second assignment of the same object shares, while `list(a)` builds a new one that only compares equal:

```
bucket = ["empty"]
alias = bucket
materialized = list(bucket)
check(alias is bucket, "assignment binds a second name to the same object")
check(
    materialized == bucket and materialized is not bucket,
    "== compares values, is compares identity",
)

fresh = []
leftover = fresh
fresh = []
check(leftover == [], "rebinding one name leaves the other object alone")
```

listing 18: sharing a binding against materializing a copy, and rebinding leaving the old object alone (source: python/samples/src/Ch03/refs.py, lines 18-30)

The `is` operator answers identity, `==` answers equality, and only the first is safe against two equal-but-distinct objects. `None` is the boundary case: its type builds the same singleton, which is why `None` checks are identity checks. `del` closes the section, because it does less than people expect:

```
check(type(None)() is None, "None is a singleton, its type builds the same object")

bound = 5
check("bound" in globals() and bound == 5, "a module level name lives in globals")
del bound
check("bound" not in globals(), "del removes the binding, not the value behind it")
```

listing 19: None builds itself, and del removes only the name (source: python/samples/src/Ch03/refs.py, lines 32-37)

The binding disappears from its namespace, and whether the object dies is a reference count question. That count is observable:

```
obj = object()
base = sys.getrefcount(obj)
holder = obj
extra = sys.getrefcount(obj) - base
del holder
check(
    extra == 1 and sys.getrefcount(obj) == base,
    "each binding adds exactly one reference",
)
```

listing 20: each alias adds exactly one reference, and getrefcount itself adds one (source: python/samples/src/Ch03/refs.py, lines 39-47)

`sys.getrefcount` reports one extra because the call holds its own temporary reference to the argument. The model's last clause is the one with consequences: functions, classes, types, and `None` are objects in the same sense as numbers, and types themselves have a type:

```
check(
    isinstance(1, object)
    and isinstance(None, object)
    and isinstance(len, object)
    and isinstance(int, object),
    "numbers, none, functions, and types are all objects",
)
check(
    type(int) is type and type(True) is bool and isinstance(True, int),
    "types are objects too, True is a bool and an int",
)
```

listing 21: the object test applied across numbers, functions, and types (source: python/samples/src/Ch03/refs.py, lines 49-59)

NOTE: names have no type, objects do

A name is a binding, so binding it to something else changes what the name refers to, never the object. Type checks like `type(True) is bool` and `isinstance(True, int)` both hold, because `bool` is a subclass of `int` and the instance answers both questions. The samples use `is` only where identity is the claim being pinned.

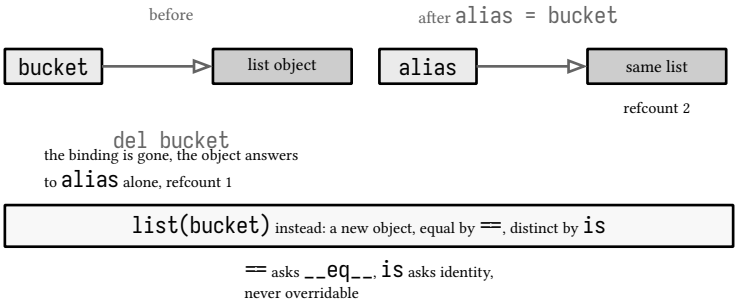


figure 9: one binding added, one unbound, the objects stay what they are

3.2. mutability and copies

Immutable objects reject item assignment, mutable ones accept it, and the split predicts everything downstream, from copying cost to the default argument trap:

```
def rejects_item_assignment(value):
    try:
        value[0] = 99
        return False
    except TypeError:
        return True


check(
    rejects_item_assignment("abc") and rejects_item_assignment((1, 2)),
    "str and tuple reject item assignment",
)
row = [1, 2]
row[0] = 9
check(row == [9, 2], "list item assignment mutates the one object")
```

listing 22: str and tuple refuse item assignment, a list takes it in place (source: python/samples/src/Ch03/copies.py, lines 19-33)

Copying splits the same way. `copy.copy` reproduces the outer container and shares everything inside it, `copy.deepcopy` walks the graph, and its memo is what keeps aliasing intact inside one copy:

```
nested = [[1], [2]]
shallow = copy.copy(nested)
deep = copy.deepcopy(nested)
check(shallow[0] is nested[0], "a shallow copy shares the inner objects")
check(
    deep[0] is not nested[0] and deep == nested, "a deep copy reproduces the whole tree"
)

shared = [7]
two_refs = [shared, shared]
deep_two = copy.deepcopy(two_refs)
check(deep_two[0] is deep_two[1], "deepcopy preserves aliasing inside one object")
```

listing 23: shallow shares the inner lists, deep rebuilds the tree but preserves the alias (source: python/samples/src/Ch03/copies.py, lines 35-46)

`deep_two[0] is deep_two[1]` holds because the memo records the first copy of `shared` and returns it for the second reference. The default argument trap is mutability's most visited accident:

```
def collect(item, target=[]):
    target.append(item)
    return target


check(
    collect(1) == [1] and collect(2) == [1, 2],
    "the default list is created once, at def time",
)


def collect_fresh(item, target=None):
    if target is None:
        target = []
    target.append(item)
    return target


collect_fresh(1)
check(collect_fresh(2) == [2], "the None sentinel builds a fresh list per call")
```

listing 24: the default list built once at def time, and the None sentinel fix (source: python/samples/src/Ch03/copies.py, lines 48-68)

Defaults are evaluated when `def` runs, not per call, so every call without `target` appends into one shared list. The `None` sentinel builds a fresh list inside the call, which is the standard fix and the reason `None` appears so often in signatures. The section closes with the two interning facts, one for ints, one for strings:

```
low_a, low_b = int("256"), int("256")
high_a, high_b = int("257"), int("257")
check(low_a is low_b, "small ints come from the cache on this build")
check(high_a is not high_b, "large ints are fresh objects")

part = "hel"
runtime = part + "lo"
canonical = "hello"
check(runtime is not canonical, "runtime concatenation makes a fresh string")
check(sys.intern(runtime) is canonical, "sys.intern returns the canonical string")
```

listing 25: the small int cache against fresh large ints, and sys.intern returning the canonical string (source: python/samples/src/Ch03/copies.py, lines 70-79)

PITFALL: interning is a cpython optimization, not a language rule

The small int cache and string interning are implementation facts, not promises. The samples construct values at runtime through `int("257")` and `part + "lo"` precisely so the checks do not lean on compile time constant folding, and they pin this build's behavior. Equality is the contract, `is` on numbers and computed strings is not.

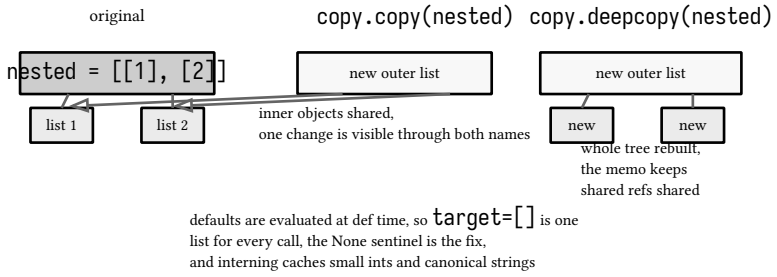


figure 10: one nested structure through a shallow copy and a deep copy

3.3. representation and hashing

`repr` is for developers, `str` is for users, and when a class defines only `__repr__` everything else follows it:

```
check(repr("\n") == "'\\n'" and str("\n") == "\n", "repr escapes, str does not")


class Plain:
    pass


class Shown:
    def __repr__(self):
        return "Shown()"


default_repr = repr(Plain())
shown = Shown()
check(
    default_repr.startswith("<") and "Plain object" in default_repr,
    "the default repr names the class and the address",
)
check(
    str(shown) == "Shown()" and f"{shown}" == "Shown()" and f"{shown!r}" == "Shown()",
    "str and f-strings fall back to __repr__",
)
```

listing 26: repr escapes where str does not, the default form, and the fallback (source: python/samples/src/Ch03/reprs.py, lines 18-39)

The default `repr` carries the class name and the address, `<Plain object at 0x...>`, good enough to tell two instances apart and nothing else. `format` is the third channel, and 3.14 keeps the rule that made it honest: a class that does not override `__format__` gets a `TypeError` for any non-empty spec, rather than silently ignoring it:

```
class Formatted:
    def __format__(self, spec):
        return f"Formatted[{spec}]"


check(
    format(Formatted(), "raw") == "Formatted[raw]"
    and f"{Formatted():>9}" == "Formatted[>9]",
    "__format__ receives the spec verbatim",
)

try:
    format(Plain(), "spec")
    check(False, "object format with a spec accepted")
except TypeError:
    check(True, "object.__format__ rejects non-empty specs")
```

listing 27: a custom `__format__` receives the spec verbatim, the default one rejects it (source: python/samples/src/Ch03/reprs.py, lines 42-57)

Hashing is the contract behind dict keys and set membership: equal objects must hash equal, `1`, `1.0`, and `True` all do, and the dict keeps a single entry when they collide:

```
check(hash(1) == hash(1.0) == hash(True), "equal objects must hash equal")
merged = {1: "first"}
merged[1.0] = "second"
check(
    list(merged) == [1] and merged[1] == "second",
    "equal keys collapse, last write wins",
)


def unhashable(container):
    try:
        {container: 1}
        return False
    except TypeError:
        return True


check(unhashable([1]) and unhashable({"a": 1}), "list and dict cannot be keys")
check(unhashable(([1], 2)), "a tuple is only as hashable as its contents")
```

listing 28: the contract, one key for three equal spellings, and the unhashable kinds (source: python/samples/src/Ch03/reprs.py, lines 59-77)

The last two checks carry the load-bearing details: a tuple is exactly as hashable as its contents, and defining `__eq__` without `__hash__` sets the hash to `None`, making instances unusable as keys, which is the language forcing the contract:

```
class EqOnly:
    def __eq__(self, other):
        return True


try:
    hash(EqOnly())
    check(False, "eq without hash still hashable")
except TypeError:
    check(True, "defining __eq__ without __hash__ makes instances unhashable")

nan = float("nan")
check(
    nan != nan and len({nan, nan}) == 1 and isinstance(hash(nan), int),
    "nan is unequal to itself yet one set element",
)
```

listing 29: eq without hash refuses, and nan's one element set (source: python/samples/src/Ch03/reprs.py, lines 80-95)

`nan != nan` yet a set holds one nan, because containment checks try identity before equality. Chapter 9's dataclasses generate exactly this pair, `__eq__` with a matching `__hash__` or none, see book 8, ch 9 (python 3.14, dataclasses, enums, and pattern matching).

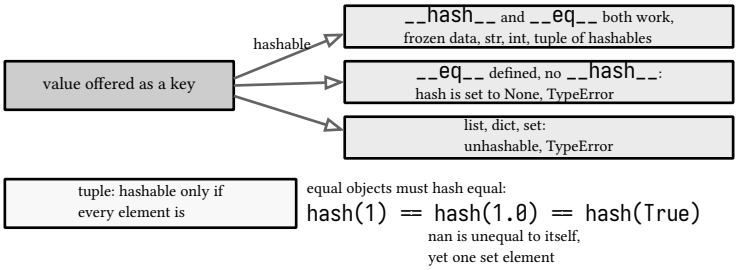


figure 11: may this value live in a dict key

3.4. gc and weakref

Cpython frees most objects without any collector: each object carries a reference count, and the count hitting zero frees it immediately:

```
sample = Node("sample")
base = sys.getrefcount(sample)
holder = sample
check(
    sys.getrefcount(sample) == base + 1,
    "getrefcount reports one extra for its own argument",
)
del holder

guarded = Node("guarded")
weak = weakref.ref(guarded)
del guarded
check(weak() is None, "refcounting frees the object the moment the last name goes")
```

listing 30: the count arithmetic, and the weak reference flipping the moment the last name goes (source: python/samples/src/Ch03/gcweak.py, lines 26-38)

The weak reference returning `None` right after `del` is the proof no collector pass was involved. Cycles are the exception refcounting cannot handle, and the generational collector exists for them:

```
gc.collect()
left = Node("left")
right = Node("right")
left.peer = right
right.peer = left
watcher = weakref.ref(left)
del left, right
check(watcher() is not None, "a cycle keeps itself alive without any name")
collected = gc.collect()
check(
    collected ≥ 1 and watcher() is None, "the generational collector reclaims cycles"
)
```

listing 31: a two node cycle survives both deletions, then one collect clears it (source: python/samples/src/Ch03/gcweak.py, lines 40-51)

The collector tracks objects in generations, promotes survivors, and triggers on allocation counts. The thresholds are not documented values, so the sample pins them as a fact of this build, `(2000, 10, 10)`:

```
check(
    tuple(gc.get_threshold()) == (2000, 10, 10),
    "the allocation thresholds on this build",
)

cache = weakref.WeakValueDictionary()
value = Node("value")
cache["key"] = value
del value
check(list(cache) == [], "a WeakValueDictionary entry dies with its value")
```

listing 32: the measured thresholds, and a weak valued mapping dropping its entry (source: python/samples/src/Ch03/gcweak.py, lines 53-62)

Weak references power caches and registries that must not keep things alive, and `finalize` puts a callback on the same hook:

```
events = []
target = Node("target")
final = weakref.finalize(target, events.append, "target")
check(final.alive, "finalize registers a callback while the object lives")
del target
gc.collect()
check(events == ["target"], "the finalizer runs at collection time")

target2 = Node("target2")
final2 = weakref.finalize(target2, events.append, "target2")
final2.detach()
del target2
gc.collect()
check(events == ["target"], "a detached finalizer never runs")
```

listing 33: a finalizer firing at collection, and detach cancelling it (source: python/samples/src/Ch03/gcweak.py, lines 64-77)

NOTE: 3.14 shipped two garbage collectors

3.14.0 through 3.14.4 ran an incremental collector with two generations and much smaller pauses. Reports of memory pressure in production led to a revert, and 3.14.5 restored the 3.13 generational collector, which is what 3.14.7 runs here. The thresholds check would read differently on a 3.14.2 interpreter. The lesson generalizes: memory behavior is versioned behavior, and the whatsnew is part of the spec.

The last sample block answers the two timing questions people get wrong: weak references only attach to types that opt in, and `__del__` is a finalizer callback, not a destructor with guaranteed timing:

```
try:
    weakref.ref(1)
    check(False, "int weakrefable")
except TypeError:
    check(True, "weak references need a weakref able type")

record = []


class Cleaner:
    def __del__(self):
        record.append("del")


cycle = Cleaner()
cycle.self_ref = cycle
del cycle
collected2 = gc.collect()
check(
    "del" in record and collected2 ≥ 1,
    "__del__ runs at collection, not at scope exit",
)
```

listing 34: int refuses weakrefs, and `__del__` runs at collection, not at scope exit (source: python/samples/src/Ch03/gcweak.py, lines 79-100)

Since PEP 442 even cycles containing `__del__` methods are collected, the callback list runs once, and a self referencing object still dies at `gc.collect`. Code that needs deterministic cleanup uses context managers, the subject of book 8, ch 10 (python 3.14, exceptions and context managers), not `__del__`.

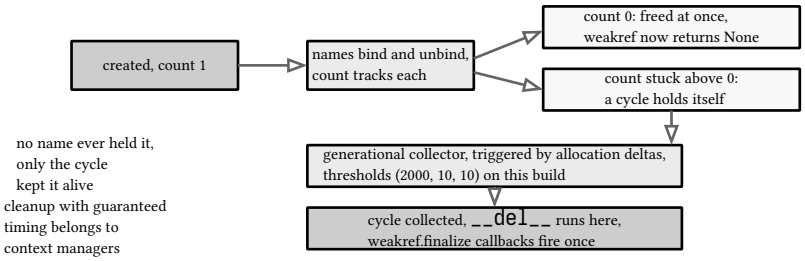


figure 12: the life of an object, refcounting first, the collector only for cycles

VERIFY: 42 checks, one red draft behind them

The first run of the names sample failed its last check: it asserted `type(True) is int`, and the interpreter answered that `type(True)` is `bool`, a subclass of `int`. The corrected check pins both facts separately. The cycle, weakref, and finalizer checks all passed on the first run, and the thresholds check is a measured build fact rather than a documented default, the gc page documents the shape of the tuple only. Sample behavior verified by `make verify-py`, 42 checks in chapter 3 of the samples suite.

sources: docs.python.org/3/reference/datamodel.html (objects, values, types, `del`, the `__eq__`/`__hash__` contract, `__repr__` and `__str__`, `__format__`, `__del__`), docs.python.org/3/library/stdtypes.html (hashability of dict keys, NoneType), docs.python.org/3/library/copy.html (shallow against deep, the memo), docs.python.org/3/library/gc.html (generations, thresholds, freeze), docs.python.org/3/library/weakref.html (ref, WeakValueDictionary, finalize and detach), docs.python.org/3/library/sys.html (getrefcount, intern), docs.python.org/3/whatsnew/3.14.html (the incremental gc revert in 3.14.5), and peps.python.org/pep-0442/ (safe object finalization), all accessed 2026-09-12; the refcount values, thresholds, and interning behavior were probed on this machine the same day. Sample behavior verified by `make verify-py`, 42 checks in chapter 3.

4. builtin collections, iteration, and generators

The builtin containers are not interchangeable: each one makes a different promise about order, cost, and identity, and the interpreter enforces those promises with concrete layouts. This chapter reads list, tuple, dict, and set from their memory behavior outward, then the iterator protocol that makes them interchangeable at the call site, then comprehensions and generators, and closes on the four utility collections that replace hand written loops. Every claim is pinned by one of the 45 checks in the four samples on cpython 3.14.7, with the size numbers stated as facts of this build.

4.1. list, tuple, dict, set internals

A python list is a C array of object pointers with spare capacity, and that spare capacity is measurable. The sample records `sys.getsizeof` before each append:

```
sizes = []
lst = []
for i in range(18):
    sizes.append(sys.getsizeof(lst))
    lst.append(i)
check(
    sizes[0] == 56 and sizes[1] == 88,
    "an empty list is 56 bytes here, the first append over-allocates to 88",
)
check(len(set(sizes)) < len(sizes), "capacity grows in steps, so appends are amortized")
check(sizes == sorted(sizes) and sizes[-1] == 248, "the measured ladder never shrinks")
```

listing 35: the empty list, the first over-allocation, and the growth ladder (source: python/samples/src/Ch04/internals.py, lines 18-28)

The ladder runs 56, 88, 120, 184, 248 bytes over 17 appends on this build: capacity grows in steps, which is why append is documented as amortized constant time. A dict is a hash table in the compact layout it has had since 3.6, an indices array pointing into a dense entries array, and only the indices array is searched:

```
ladder = []
table = {}
for i in range(9):
    table[i] = i
    ladder.append(sys.getsizeof(table))
check(
    ladder[0] == 224 and ladder[5] == 352,
    "the compact dict resizes between 5 and 6 entries on this build",
)
```

listing 36: the compact dict reserving its entries array between 5 and 6 entries here (source: python/samples/src/Ch04/internals.py, lines 30-38)

Insertion order falls out of that layout, and the language has promised it since 3.7. The sample exercises the promise from both ends:

```
order = {}
for word in ("later", "sooner", "never"):
    order[word] = len(word)
check(list(order) == ["later", "sooner", "never"], "dicts preserve insertion order")
check(order.popitem() == ("never", 5), "popitem removes the newest pair")
check(list(reversed(order)) == ["sooner", "later"], "reversed walks newest first")

before = sys.getsizeof(order)
for word in ("later", "sooner"):
    del order[word]
check(
    sys.getsizeof(order) == before and order == {},
    "deleting keys does not shrink the table",
)
```

listing 37: order preserved, popitem LIFO, reversed backwards, and no shrink on delete (source: python/samples/src/Ch04/internals.py, lines 40-53)

Deleting keys leaves the table size untouched, 184 bytes before and after, because removal marks entries empty rather than rebuilding. That string-keyed table rides a slimmer entry row than the integer-keyed ladder of chapter 13, whose same-size dict reads 224. Sets are dicts without values, which makes membership the whole story:

```
check({1, 1.0, True} == {1}, "equal values collapse to one set element")


class Key:
    def __init__(self, tag):
        self.tag = tag

    def __hash__(self):
        return 7

    def __eq__(self, other):
        return isinstance(other, Key) and self.tag == other.tag


colliders = {Key("a"), Key("b"), Key("c")}
check(
    len(colliders) == 3 and Key("a") in colliders, "collisions are probed, not dropped"
)
```

listing 38: equal values collapsing to one element, and collisions resolved by probing (source: python/samples/src/Ch04/internals.py, lines 55-72)

Three objects with the same hash and different equality all survive in the set: open addressing walks the probe sequence, so a bad `__hash__` costs lookups but never correctness. The merge operators close the section, `|` builds a new dict since 3.9 (PEP 584) and `|=` updates in place, see lines 74 through 80 of the sample.

NOTE: sizes are build facts, order is a language fact

`sys.getsizeof` ladders are pinned to this interpreter binary and would read differently on another build. Insertion order, `popitem` removing the last pair, and equal values collapsing in a set are language guarantees, and those are the checks that survive a version bump unchanged.

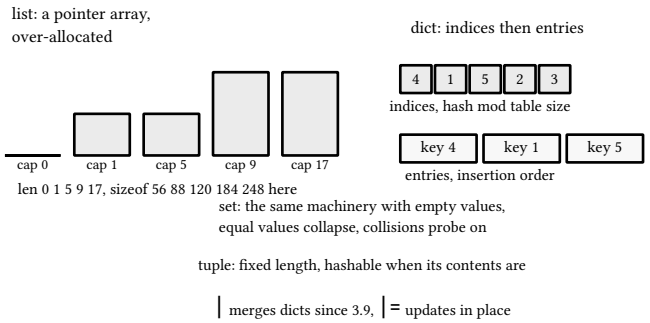


figure 13: how the four containers spend their memory

4.2. the iterator and iterable protocols

`for` is one protocol. It calls `iter()` on the object, then `next()` on the result until `StopIteration`, and both halves are reachable by hand:

```
class Countdown:
    def __init__(self, start):
        self.left = start

    def __iter__(self):
        return self

    def __next__(self):
        if self.left ≤ 0:
            raise StopIteration
        self.left -= 1
        return self.left


check(list(Countdown(3)) == [2, 1, 0], "for drives iter then next until StopIteration")

manual = iter(Countdown(2))
check(next(manual) == 1 and next(manual) == 0, "next drives an iterator by hand")
```

listing 39: the two dunders, the for loop over them, and manual driving (source: python/samples/src/Ch04/iterproto.py, lines 18-35)

Two facts fall out of the manual half. Exhaustion is permanent, a spent iterator yields nothing forever, and `next` takes a default that swallows the stop. The protocol also has a legacy door: an object with only `__getitem__` still iterates, because `iter()` falls back to the old sequence protocol, indexes from 0, and stops at `IndexError`:

```
class Sequence:
    def __getitem__(self, index):
        if index > 2:
            raise IndexError
        return index * 10


check(list(Sequence()) == [0, 10, 20], "iter falls back to __getitem__")
```

listing 40: no `__iter__`, yet fully iterable (source: python/samples/src/Ch04/iterproto.py, lines 43-50)

That fallback is why strings and old sequence classes work in for loops, and its edges are sharp: `reversed()` refuses the same object unless it also defines `__len__`, the diagnostic on this build being `object of type 'Headless' has no len()` (probed 2026-09-12), see lines 53 through 62. The helpers built on the protocol finish the section:

```
check(list(zip([1, 2], "abc")) == [(1, "a"), (2, "b")], "zip truncates at the shortest")
try:
    list(zip([1, 2], "abc", strict=True))
    check(False, "strict zip accepted")
except ValueError as exc:
    check(
        "longer than" in str(exc), "strict=True turns a length mismatch into an error"
    )
try:
    list(map(lambda a, b: (a, b), [1, 2], "x", strict=True))
    check(False, "strict map accepted")
except ValueError as exc:
    check("shorter than" in str(exc), "map grew the same strict flag in 3.14")
check(
    list(enumerate("ab", 1)) == [(1, "a"), (2, "b")],
    "enumerate numbers from any start",
)
```

listing 41: zip truncating silently, both strict flags, and enumerate numbering from 1 (source: python/samples/src/Ch04/iterproto.py, lines 64-80)

`zip` stops at the shortest input unless told otherwise, a silent truncation that has eaten real data, and `strict=True` turns the mismatch into a `ValueError` naming the longer argument. New in 3.14, `map` grew the same flag with the same semantics.

PITFALL: zip truncates by default

`list(zip([1, 2], "abc"))` returns 2 pairs and no complaint. When uneven inputs are a bug, `strict=True` is the assertion, and since 3.14 `map(..., strict=True)` covers the mapped case. Both error messages name the longer argument, which is usually the one with the bug in it.

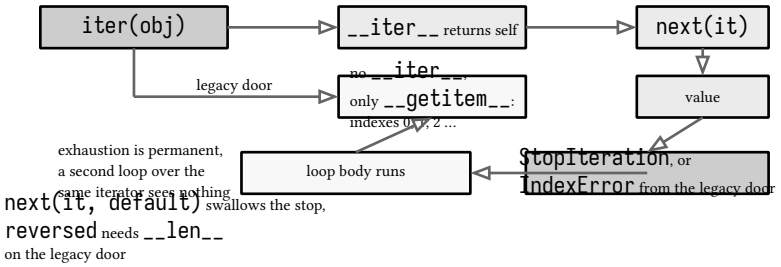


figure 14: the for loop protocol, with the legacy sequence door

4.3. comprehensions and generators

A comprehension is an eager loop spelled inline, and since python 3 it runs in its own scope: the loop variable never leaks. A generator expression looks almost identical and behaves almost oppositely:

```
source = [1, 2]
lazy = (x * x for x in source)
source.append(3)
check(list(lazy) == [1, 4, 9], "a genexp is lazy, it sees later source changes")
check(list(lazy) == [], "a genexp is one shot")
```

listing 42: laziness proven by appending to the source, and the spent genexp (source: python/samples/src/Ch04/gens.py, lines 18-22)

Nothing runs until the first `next`, so the appended 3 is part of the result, and once consumed the genexp is finished. Generator functions are the full form, with a two way channel:

```
def counter():
    got = yield 1
    yield got


stepper = counter()
next(stepper)
check(stepper.send(41) == 41, "send injects the value the yield evaluates to")

finished = counter()
next(finished)
finished.close()
try:
    next(finished)
    check(False, "closed generator resumed")
except StopIteration:
    check(True, "close finishes the generator for good")
```

listing 43: send injecting the value a yield evaluates to, and close ending the generator (source: python/samples/src/Ch04/gens.py, lines 25-41)

The `send` check is the one worth sitting with: `got = yield 1` makes 41 the value of the expression the paused generator resumes into. `return` inside a generator is not an error, it rides out as the value of the final `StopIteration`, and `yield from` delegates and captures it:

```
def reporter():
    yield 1
    return "payload"


done = reporter()
next(done)
try:
    next(done)
    check(False, "return ignored")
except StopIteration as exc:
    check(exc.value == "payload", "a return rides out on StopIteration")


def delegate():
    result = yield from reporter()
    yield result


check(list(delegate()) == [1, "payload"], "yield from captures the subgenerator return")
```

listing 44: the payload on StopIteration, and yield from delivering it (source: python/samples/src/Ch04/gens.py, lines 44-63)

Comprehension scoping closes the loop the lexical chapter opened: the comprehension variable stays inside, and a walrus in a comprehension filter binds in the enclosing scope, the one deliberate exception:

```
check([x * x for x in range(4)] == [0, 1, 4, 9], "a list comprehension builds eagerly")


def leak_check():
    [item for item in range(3)]
    return "item" in locals()


check(leak_check() is False, "the comprehension variable never leaks out")


def walrous(values):
    picked = [y for x in values if (y := x * x) > 1]
    return picked, y


picked, bound = walrous([1, 5])
check(
    picked == [25] and bound == 25, "a walrus in a filter binds in the enclosing scope"
)
```

listing 45: no leak, the walrus exception, and nested comprehensions reading like nested for loops (source: python/samples/src/Ch04/gens.py, lines 65-84)

The eager and lazy forms are the same pipeline wearing different clothes, and the async world lifts the whole design: `yield` becomes `await`, the generator protocol becomes coroutines, see book 8, ch 21 (python 3.14, asyncio).

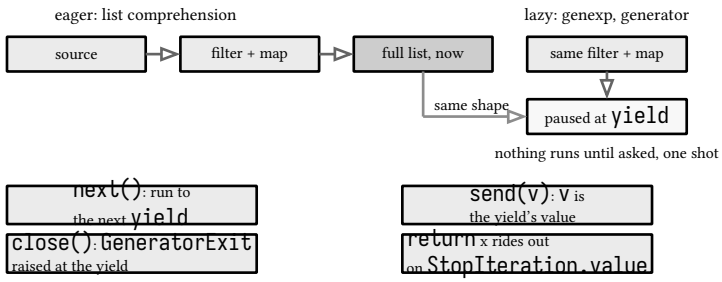


figure 15: one pipeline, two evaluations, and the two way generator channel

4.4. heapq, bisect, counter, deque

Four utility collections replace the loops people write by hand. `heapq` keeps a list satisfying the heap invariant, parent no greater than either child, with the minimum at index 0:

```
def heap_ok(data):
    for parent in range(len(data)):
        for child in (2 * parent + 1, 2 * parent + 2):
            if child < len(data) and data[parent] > data[child]:
                return False
    return True


heap = [5, 3, 8, 1]
heapq.heapify(heap)
check(heap_ok(heap), "heapify establishes parent ≤ children in the array")
drained = [heapq.heappop(heap) for _ in range(4)]
check(drained == [1, 3, 5, 8], "heappop yields ascending order")
heapq.heappush(heap, 2)
heapq.heappush(heap, 6)
check(heap[0] == 2 and heap_ok(heap), "the smallest element sits at index 0")
```

listing 46: the invariant checked directly, pops in ascending order, and the top of the heap (source: python/samples/src/Ch04/utils.py, lines 21-36)

`nlargest` and `nsmallest` (lines 38 through 42) read the same heap for the fixed size case without paying a full sort. `bisect` maintains a sorted list by finding the insertion point with binary search:

```
ordered = []
for value in (3, 1, 4, 1, 5):
    bisect.insort(ordered, value)
check(ordered == [1, 1, 3, 4, 5], "insort keeps the list sorted on insert")
check(
    bisect.bisect_left(ordered, 1) == 0 and bisect.bisect_right(ordered, 1) == 2,
    "left lands before the equal run, right after it",
)
```

listing 47: insort keeping order on insert, and left against right around the equal run (source: python/samples/src/Ch04/utils.py, lines 44-51)

`deque` is a doubly linked block structure with cheap ends, a `maxlen` that turns it into a ring, and a `rotate` that walks it:

```
window = deque(maxlen=3)
for value in (1, 2, 3, 4):
    window.append(value)
check(list(window) == [2, 3, 4], "a bounded deque evicts from the opposite end")
window.rotate(1)
check(list(window) == [4, 2, 3], "rotate walks the ring right for positive n")
window.appendleft(9)
check(window.popleft() == 9 and list(window) == [4, 2], "both ends push and pop")
```

listing 48: bounded eviction from the opposite end, rotation, and both ends cheap (source: python/samples/src/Ch04/utils.py, lines 53-60)

`Counter` is a dict subclass that counts, and because it is a dict its order rules apply: ties in `most_common` keep first-seen order:

```
counts = Counter("aabcc")
check(
    counts.most_common() == [("a", 2), ("c", 2), ("b", 1)],
    "ties in most_common keep first seen order",
)
check(counts + Counter("a") == Counter({"a": 3, "c": 2, "b": 1}), "counters add")
check(
    counts - Counter("abbc") == Counter({"a": 1, "c": 1}), "subtraction drops empties"
)
```

listing 49: tie order, and the + and - arithmetic with empty results dropped (source: python/samples/src/Ch04/utils.py, lines 62-70)

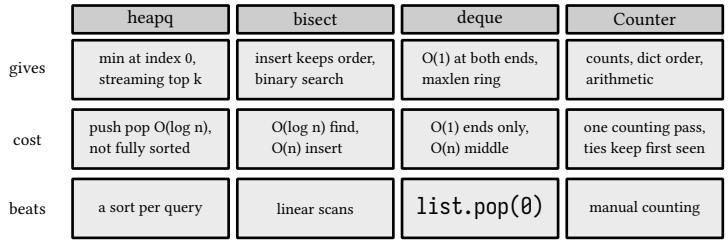


figure 16: what each utility buys, and what it costs

VERIFY: 45 checks, three red drafts behind them

The collections sample went red on its own expectations three times before it went green, and all three were instruction, not bug. `|=` merges only the new operand, so the expected dict had one key, not three. The first `send` draft yielded 2 instead of 41 because the assignment target has to be the first `yield` the generator pauses at. And counter subtraction keeps keys the other operand never mentions, so the fixture needed a b count in it. The gate caught all three. Sample behavior verified by `make verify-py`, 45 checks in chapter 4 of the samples suite.

sources: docs.python.org/3/library/stdtypes.html (list append as amortized constant, dict insertion order, set membership), docs.python.org/3/library/functions.html (iter including the `__getitem__` fallback, next, zip and its strict flag, map and its new strict flag, enumerate, reversed), docs.python.org/3/reference/expressions.html (comprehension scope, yield, yield from, generator methods), docs.python.org/3/library/heapq.html, library/bisect.html, library/collections.html (deque and Counter), docs.python.org/3/whatsnew/3.10.html (zip strict), docs.python.org/3/whatsnew/3.14.html (map strict), and peps.python.org/pep-0584/ (dict merge operators), all accessed 2026-09-12; the sizeof ladders and the heap array were probed on this machine the same day. Sample behavior verified by `make verify-py`, 45 checks in chapter 4.

5. strings, bytes, and text

Text and bytes are different types with different jobs, and most encoding bugs are a missing conversion between them. This chapter builds the model from the bottom: `str` as code points, `bytes`, `bytearray`, and `memoryview` as byte sequences with the encode/decode discipline between them, the `re` module over text, and the four formatting mechanisms python already carries, and the 3.14 addition. Every claim is pinned by one of the 44 checks in the four samples on cpython 3.14.7.

5.1. str and the unicode model

A `str` is a sequence of unicode code points, not bytes and not graphemes. The same character can be spelled several ways, and normalization is the discipline that makes comparisons honest:

```
composed = "é"
decomposed = unicodedata.normalize("NFD", composed)
check(len(composed) == 1 and len(decomposed) == 2, "str counts code points, not bytes")
check(
    decomposed[0] == "e" and unicodedata.combining(decomposed[1]) != 0,
    "NFD splits the base letter from the combining mark",
)
check(unicodedata.normalize("NFC", decomposed) == composed, "NFC re-composes the pair")
```

listing 50: one code point composed, two decomposed, NFC reuniting them (source: python/samples/src/Ch05/strmodel.py, lines 19-26)

"é" composed is length 1, its NFD form is a base letter plus a combining mark, length 2, and neither == nor `len` treats them alike. Indexing follows the same unit:

```
emoji = chr(0x1F600)
check(len(emoji) == 1 and emoji[0] == emoji, "one astral code point is one element")

text = "abcdef"
check(
    text[1:4] == "bcd" and text[::-1] == "fedcba" and text[-1] == "f",
    "indexing and slicing step by code point",
)
check(
    ord("é") == 233 and chr(233) == "é", "ord and chr move between text and code point"
)
```

listing 51: one astral code point is one element, slices step by code point, ord and chr (source: python/samples/src/Ch05/strmodel.py, lines 28-38)

An emoji from the astral planes is one `str` element here, which is the practical difference from utf-16 languages where such characters split across two units. Ordering is by code point, so uppercase sorts before lowercase, and the type is immutable, so every method returns a new string:

```
check(
    "Z" < "a" and sorted(["b", "A", "Z", "a"]) == ["A", "Z", "a", "b"],
    "ordering is by code point, uppercase first",
)

shout = "abc".upper()
check("abc" == "abc" and shout == "ABC", "methods return new strings, never mutate")
check(
    "c" in "abcd" and "abcd".startswith(("x", "ab")) and "abcd".endswith("cd"),
    "in and startswith and endswith accept tuples",
)
```

listing 52: code point ordering, methods building new strings, and tuple membership tests (source: python/samples/src/Ch05/strmodel.py, lines 40-50)

PITFALL: code points are not graphemes

`len` counts code points, so `"é"` NFD reports 2 while a user sees one character, and flag-plus-skin-tone emoji count as several. Comparisons across spellings need `unicodedata.normalize` first, and text that must round trip through another system should agree on NFC or NFD up front. Byte counts are a third number again, and belong to the next section.

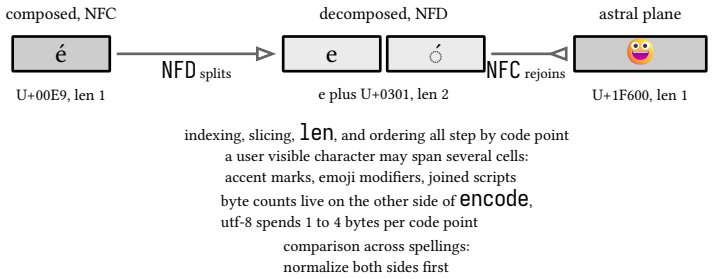


figure 17: the same character, three spellings, and what indexing sees

5.2. bytes, bytearray, memoryview

`bytes` literals look like strings with a `b`, but they hold byte values and accept only ascii text or escapes:

```
check(b"\xff"[0] == 255 and len(b"\n") == 1, "bytes literals hold byte values")

try:
    compile("b'é'", "<bytes>", "eval")
    check(False, "non ascii bytes literal accepted")
except SyntaxError as exc:
    check("ASCII" in str(exc), "bytes literals take ascii text or escapes only")
```

listing 53: byte values, and the ascii rule enforced at compile time (source: python/samples/src/Ch05/bytesio.py, lines 19-25)

Encoding is the bridge, and it is a choice, not a property of the text. The same `é` becomes two bytes in utf-8 and one in latin-1:

```
text = "héllo"
check(
    text.encode("utf-8") == b"h\xc3\xa9llo" and text.encode("latin-1") == b"h\xe9llo",
    "the encoding, not the text, picks the bytes",
)
```

listing 54: the encoding picks the bytes (source: python/samples/src/Ch05/bytesio.py, lines 27-31)

Decoding is where the errors surface, and `errors=` is the policy dial. Strict is the default and the right default:

```
try:
    b"\x80".decode("utf-8")
    check(False, "strict decode accepted")
except UnicodeDecodeError:
    check(True, "strict is the default error mode")

check(b"\x80".decode("utf-8", "replace") == "\ufffd", "replace substitutes one character")
check(b"ab\x80cd".decode("utf-8", "ignore") == "abcd", "ignore drops the bad bytes")
raw = b"ab\x80cd"
check(
    raw.decode("utf-8", "surrogateescape").encode("utf-8", "surrogateescape") == raw,
    "surrogateescape round trips unknown bytes",
)
```

listing 55: strict refusing, replace substituting, ignore dropping, surrogateescape round tripping (source: python/samples/src/Ch05/bytesio.py, lines 33-45)

`surrogateescape` is the handler for bytes that are not valid utf-8 but must survive a round trip, filenames on unix being the classic case. The mutable side and the zero copy view follow:

```
buffer = bytearray(b"abcd")
view = memoryview(buffer)
view[0] = 88
check(buffer == bytearray(b"Xbcd"), "a memoryview shares memory with its owner")
check(bytes(view[1:3]) == b"bc", "slicing a view copies no buffer")

frozen = b"abc"
try:
    frozen[0] = 88
    check(False, "bytes item assignment accepted")
except TypeError:
    check(True, "bytes is immutable, bytearray is the mutable flavor")
buffer[1] = 89
check(buffer == bytearray(b"XYcd"), "bytearray assigns items as ints")
```

listing 56: memoryview sharing the buffer, bytes refusing mutation, bytearray accepting it (source: python/samples/src/Ch05/bytesio.py, lines 47-60)

`memoryview` is the read or write window over another object's buffer without copying, the buffer protocol that c extensions and `struct` speak. `struct` closes the section by making byte order explicit instead of inherited:

```
check(
    struct.pack("<I", 1) == b"\x01\x00\x00\x00", "struct pins the byte order explicitly"
)
check(
    sys.byteorder == "little" and struct.unpack("<H", b"\x01\x00") == (1,),
    "this machine is little endian, matching the pinned order",
)
```

listing 57: pinned little endian packing, cross checked against the machine's order (source: python/samples/src/Ch05/bytesio.py, lines 62-68)

WARNING: never let the encoding be ambient

Every `open()`, `encode`, and `decode` in this book names its encoding, because the alternative is a platform default, and windows and the container this code ships to may disagree. The `errors=` policy deserves the same

> explicitness: strict unless the bytes are somebody else's problem, `surrogateescape` when they must round trip. The files chapter makes `encoding=` a mandatory argument in every example, book 8, ch 15 (python 3.14, files and serialization).

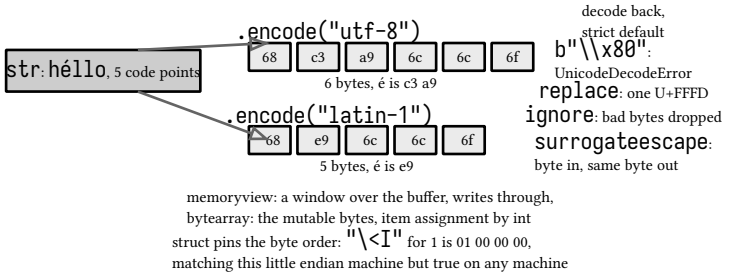


figure 18: one word through encode and decode, and where each error policy lands

5.3. re in practice

Three entry points anchor every regex question: `match` anchors at the start, `search` scans, `fullmatch` demands the whole text:

```
pattern = re.compile(r"(\d{4})-(\d{2})")
match = pattern.match("2026-09-12")
check(
    match is not None and match.group(1) == "2026",
    "compile once, match anchors at the start",
)
check(
    re.match(r"\d+", "abc") is None and re.search(r"\d+", "abc123").group() == "123",
    "match anchors, search scans",
)
check(
    re.fullmatch(r"\d+", "123") is not None and re.fullmatch(r"\d+", "123x") is None,
    "fullmatch must span the whole text",
)
```

listing 58: one pattern compiled once, and the three anchoring answers (source: python/samples/src/Ch05/rex.py, lines 19-32)

Compiling once matters when the pattern is reused, both for the cache and for readability. Groups come positional and named, and `finditer` walks every match with its spans:

```
named = re.match(r"(?P<year>\d{4})-(?P<month>\d{2})", "2026-09")
check(
    named.group("year") == "2026"
    and named.groupdict() == {"year": "2026", "month": "09"},
    "named groups read back by word",
)

spans = [found.span() for found in re.finditer(r"\d", "a1b22c333")]
check(
    spans == [(1, 2), (3, 4), (4, 5), (6, 7), (7, 8), (8, 9)],
    "finditer yields every match with its span",
)
```

listing 59: named groups reading back by word, and finditer's span list (source: python/samples/src/Ch05/rex.py, lines 34-45)

Substitution accepts a function run per match, which is how casing, lookups, and escaping decisions get made per hit:

```
check(
    re.sub(r"\b\w+\b", lambda m: m.group().title(), "hello big world")
    == "Hello Big World",
    "sub accepts a function run per match",
)
```

listing 60: a function title casing each word (source: python/samples/src/Ch05/rex.py, lines 47-51)

The quantifier temperament is the classic regex trap, and raw strings are the discipline that keeps backslashes sane:

```
greedy = re.search(r"<.*>", "<a><b>")
lazy = re.search(r"<.*?>", "<a><b>")
check(
    greedy.group() == "<a><b>" and lazy.group() == "<a>",
    "greedy takes the most, lazy the least",
)

check(
    r"\d" == "\\d" and re.fullmatch(r"\d", "7") is not None,
    "raw strings spare the backslash",
)

escaped = re.escape("a.b*")
check(
    escaped == "a\\.b\\*" and re.fullmatch(escaped, "a.b*") is not None,
    "re.escape neutralizes metacharacters in user text",
)
```

listing 61: greedy against lazy on the same input, raw patterns, and re.escape for foreign text (source: python/samples/src/Ch05/rex.py, lines 53-69)

`<.*>` eats both tags because `.` and `*` are greedy by default, `.*?` stops at the first `>`. In a plain string literal `"\\d"` is the only way to hand the engine `\d`, while `r"\d"` hands it over directly, which is why regexes are written raw. When the pattern must contain user text, `re.escape` quotes its metacharacters.

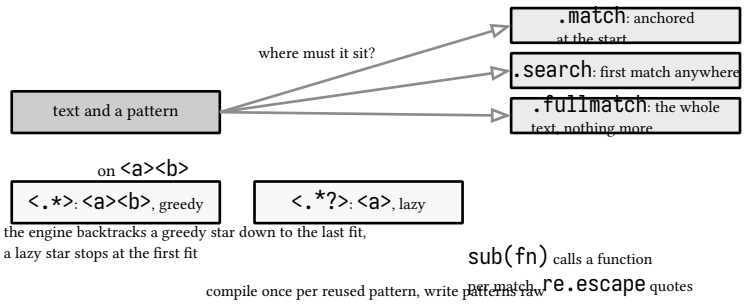


figure 19: choosing the matcher, and the two quantifier temperaments

5.4. formatting

Four mechanisms coexist, each a generation. F-strings are the working default, and their `=` specifier is the debugging form:

```
value = 3
check(
    f"{value=}" == "value=3" and f"{value = }" == "value = 3",
    "the = specifier shows the expression and the value",
)
check(f"{value!r}" == "3" and f"{'a'!r}" == "'a'", "!r calls repr, the default is str")

check(f"{7:03}" == "007" and f"{255:x}" == "ff", "numeric specs pad and pick a radix")
check(
    f"{1234.5:,.1f}" == "1,234.5" and f"{0.25:.1%}" == "25.0%", "grouping and percent"
)
```

listing 62: the = specifier keeping the expression text, !r, and the numeric specs (source: python/samples/src/Ch05/fmt.py, lines 19-29)

The space around = survives into the output, which is the point. Since 3.12 the tokenizer owns f-strings (PEP 701), so a nested field may reuse the outer quotes:

```
nest_ns = {"table": {"k": 9}}
exec(compile('result = f"outer {table["k"]}"', "<nest>", "exec"), nest_ns)
check(nest_ns["result"] == "outer 9", "pep 701 lets the same quote nest")
```

listing 63: the same quote nesting inside the field (source: python/samples/src/Ch05/fmt.py, lines 31-33)

That check compiles its source through a string, because the formatter normalizes the nested quote spelling when it appears as a literal, the same listing stability discipline chapter 2 applies to line joins. The format spec mini language is shared by every mechanism, and a class hooks it through `__format__`:

```
class Formatted:
    def __format__(self, spec):
        return f"Formatted[{spec}]"


check(f"{Formatted():x>4}" == "Formatted[x>4]", "a class sees its spec in __format__")

check(
    "{} and {name}".format(1, name="n") == "1 and n",
    "str.format fills positional and named fields",
)

check("%s-%03d" % ("a", 7) == "a-007" and "%.2f" % 3.14159 == "3.14", "% still works")
```

listing 64: a class receiving its spec verbatim, and str.format filling both field kinds (source: python/samples/src/Ch05/fmt.py, lines 36-48)

The older pair, printf style `%` and `string.Template`, still pays rent in old code and in user supplied templates:

```
slot = Template("hi $who")
check(slot.substitute({"who": "x"}) == "hi x", "Template substitutes named slots")
check(
    Template("hi $missing").safe_substitute() == "hi $missing",
    "safe_substitute leaves unknown slots alone",
)
```

listing 65: % still working, and safe_substitute leaving unknown slots alone (source: python/samples/src/Ch05/fmt.py, lines 50-55)

3.14 adds a fifth mechanism, the template string (PEP 750). A `t` prefix builds a `Template` object instead of a `str`, holding the literal parts and the unevaluated interpolations side by side:

```
template = t"a {1 + 1} b"
interpolation = template.interpolations[0]
check(
    type(template) is not str and type(template).__name__ == "Template",
    "t-strings build templates, not strings, pep 750",
)
check(
    template.strings == ("a ", " b") and interpolation.value == 2,
    "literal text and values stay separate",
)
check(
    interpolation.expression == "1 + 1",
    "the expression source survives for the consumer",
)
```

listing 66: t-strings separating literal text from values, with the expression source intact (source: python/samples/src/Ch05/fmt.py, lines 57-70)

The value is evaluated, but no formatting decision has been made yet: the consumer of the template chooses how to render, which is what makes sql and html escaping possible. An f-string is a finished string, a t-string is a request.

f-string	evaluated now, the default for display	f"{x:03}"
format spec	fill, align, sign, width, precision, type	f"{1234.5:,.1f}"
__format__	a class answers the spec itself	format(obj, "x")
str.format	fields filled by position or name	"{0} {name}"
%	printf style, legacy glue	"%s-%03d" % t
Template	user editable templates, safe substitute	"hi $who"
t-string	3.14, structure before rendering	t"hi {name}"

an f-string renders immediately, a t-string hands the pieces to code that decides

figure 20: five formatting mechanisms, one per generation

VERIFY: 44 checks, and one formatter fight

The samples ran green on their first execution, but not on their first formatting pass: `ruff format` collapsed the nested same-quote f-string into single-quoted inner keys and would have silently deleted the PEP 701 demonstration. The check moved into a `compile()` probe, the same treatment the lexical chapter gives its line joins, and the pinned listing shows the mechanism instead of the formatter's preference. Sample behavior verified by `make verify-py`, 44 checks in chapter 5 of the samples suite.

sources: docs.python.org/3/library/stdtypes.html (str, bytes, bytearray, memoryview and the buffer protocol), docs.python.org/3/howto/unicode.html (code points, normalization practice), docs.python.org/3/library/unicodedata.html (normalize, combining), docs.python.org/3/library/re.html (match, search, fullmatch, groups, finditer, sub, escaping, greedy against lazy), docs.python.org/3/reference/lexical_analysis.html (f-strings, format specs, the t prefix), docs.python.org/3/library/string.html (Template, the format spec mini language), docs.python.org/3/library/struct.html (byte order characters), docs.python.org/3/whatsnew/3.14.html (t-strings, PEP 750), peps.python.org/pep-0701/ and pep-0750/, all accessed 2026-09-12; the ascii literal rejection, the decode handler outputs, the struct byte order, and the formatter behavior were probed on this machine the same day. Sample behavior verified by `make verify-py`, 44 checks in chapter 5.

6. functions, scopes, and decorators

A function is an object that carries its own namespace, its own defaults, and, when it closes over names, a set of cells that outlive the call that created them. This chapter walks the four load bearing parts: scope resolution and closures, the parameter grammar with its two markers, the decorator protocol that makes rebinding a function a one line spelling, and the functools toolkit built on top. Every claim is pinned by one of the 37 checks in the four samples on cpython 3.14.7, and the closure claims are additionally pinned in bytecode by this chapter's expect-dis file.

6.1. scopes and closures

Name resolution walks one fixed chain: local, then enclosing functions, then module globals, then builtins. One function in the sample touches every layer with a single return:

```
module_level = "module"


def read_chain():
    enclosed = "enclosed"

    def inner():
        local = "local"
        return local, enclosed, module_level, len

    return inner()


check(
    read_chain() == ("local", "enclosed", "module", len),
    "resolution walks local, enclosing, global, builtins",
)
```

listing 67: four names, four layers, one tuple proving the walk order (source: python/samples/src/Ch06/scopes.py, lines 18-34)

A name anywhere in a function body makes it local for the whole function unless a declaration says otherwise, and the two declarations exist because rebinding is what needs permission, reading is not:

```
counter_total = 0


def bump_global():
    global counter_total
    counter_total += 1
    return counter_total


bump_global()
check(bump_global() == 2, "global rebinds the module level name")
```

listing 68: global reaching the module name from inside (source: python/samples/src/Ch06/scopes.py, lines 36-46)

Closures are the enclosing layer's machinery. The inner function keeps a reference to the cell holding `total`, the cell outlives `make_adder`'s frame, and mutation through `nonlocal` is visible to everyone sharing the cell:

```
def make_adder():
    total = 0

    def add(x):
        nonlocal total
        total += x
        return total

    return add


adder = make_adder()
check(adder(1) == 1 and adder(10) == 11, "a closure keeps its cell alive after return")
check(
    adder.__closure__[0].cell_contents == 11, "the captured cell holds the live value"
)
```

listing 69: capture by reference, the cell readable from outside (source: python/samples/src/Ch06/scopes.py, lines 49-64)

`locals()` inside a function answers a snapshot, not a live view, a rule PEP 667 made consistent in 3.13:

```
def snapshot():
    seen = "first"
    frame_locals = locals()
    seen = "second"
    return frame_locals["seen"]


check(snapshot() == "first", "locals() in a function is a snapshot, pep 667")
```

listing 70: the snapshot ignoring the later assignment (source: python/samples/src/Ch06/scopes.py, lines 67-74)

The late binding trap and its two fixes close the section, and both fixes are the same idea, give each function its own binding moment:

```
late = [lambda: index for index in range(3)]
check(
    [fn() for fn in late] == [2, 2, 2], "loop variables are captured late, by reference"
)

early = [lambda index=index: index for index in range(3)]
check([fn() for fn in early] == [0, 1, 2], "a default argument binds the value early")


def make_one(index):
    return lambda: index


per_call = [make_one(index) for index in range(3)]
check(
    [fn() for fn in per_call] == [0, 1, 2], "a factory gives each closure its own cell"
)
```

listing 71: three lambdas sharing one cell, the default argument binding early, the factory giving each closure its own (source: python/samples/src/Ch06/scopes.py, lines 76-92)

The counter at lines 95 through 109 is the pattern the expect-dis file pins: `bump` reads and writes `count` through `LOAD_DEREF` and `STORE_DEREF`, which is the whole closure mechanism at the bytecode level. Frames and cells from the interpreter's side are book 8, ch 12 (python 3.14, how cpython runs your code).

PITFALL: loops bind names, not values

A lambda written in a loop captures the loop variable's cell, and the cell holds the last value once the loop ends, so `[fn() for fn in late]` reads `[2, 2, 2]`. The default argument `index=index` evaluates at def time, and the factory function creates a fresh cell per call. Both are the same fix: change when the binding happens.

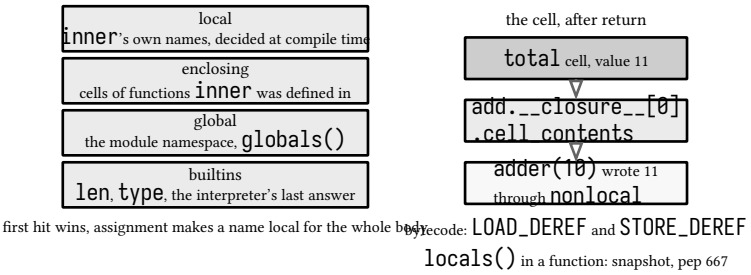


figure 21: the legb stack, and the cell a closure carries

6.2. arguments

The parameter list has an order and two markers that divide it: / ends the positional-only zone, * starts the keyword-only zone:

```
def mixed(a, b=1, /, c=2, *, d=4):
    return (a, b, c, d)


check(mixed(0) == (0, 1, 2, 4), "defaults fill every flavor of parameter")
try:
    mixed(a=0)
    check(False, "positional only accepted as keyword")
except TypeError as exc:
    check("positional-only" in str(exc), "/ ends the positional-only parameters")
```

listing 72: one signature with every flavor, and the positional-only refusal (source: python/samples/src/Ch06/args.py, lines 19-28)

The error messages are worth reading on this build: `mixed() got some positional-only arguments passed as keyword arguments: 'a'`. The * marker's refusal lands the same way:

```
def keyed(*, key):
    return key


try:
    keyed()
    check(False, "keyword only accepted as positional")
except TypeError as exc:
    check("keyword-only" in str(exc), "* starts the keyword-only parameters")
```

listing 73: a required keyword-only argument and its diagnostic (source: python/samples/src/Ch06/args.py, lines 31-39)

Packing and unpacking are mirror spellings of the same two markers, one pair at the def site, the other at the call site:

```
def gather(*args, **kwargs):
    return args, kwargs


check(
    gather(1, 2, flag=True) == ((1, 2), {"flag": True}),
    "* and ** pack the extras at the def site",
)


def spread(a, b, c):
    return a + b + c


pair = (1, 2)
keywords = {"c": 3}
check(spread(*pair, **keywords) == 6, "call sites unpack with * and **")
```

listing 74: packing the extras into args and kwargs, then unpacking a tuple and a dict into a call (source: python/samples/src/Ch06/args.py, lines 42-58)

The function object carries its defaults, split by zone, and `inspect` renders the whole contract back as source-shaped text:

```
check(
    mixed.__defaults__ == (1, 2) and mixed.__kwdefaults__ == {"d": 4},
    "defaults live on the function object",
)
check(
    str(inspect.signature(mixed)) == "(a, b=1, /, c=2, *, d=4)",
    "inspect renders the markers",
)
```

listing 75: `__defaults__` and `__kwdefaults__`, and the signature string with both markers (source: python/samples/src/Ch06/args.py, lines 60-67)

One argument cannot arrive twice, the last classic:

```
def duplicate(a):
    return a


try:
    duplicate(1, a=1)
    check(False, "duplicate accepted")
except TypeError as exc:
    check("multiple values" in str(exc), "one argument cannot arrive twice")
```

listing 76: positional and keyword naming the same parameter is a TypeError (source: python/samples/src/Ch06/args.py, lines 70-78)

The mutable default trap closes the section, the same accident chapter 3 demonstrated from the object side:

```
def collect(item, target=[]):
    target.append(item)
    return target


check(
    collect(1) == [1] and collect(2) == [1, 2],
    "the default list is built once, at def time",
)


def collect_fresh(item, target=None):
    if target is None:
        target = []
    target.append(item)
    return target


collect_fresh(1)
check(collect_fresh(2) == [2], "the None sentinel is the standard fix")
```

listing 77: the default list built once, and the None sentinel per call (source: python/samples/src/Ch06/args.py, lines 81-100)

NOTE: defaults are def time state, and 3.14 added a reserved slot

`__defaults__` is a plain tuple on the function object, built when `def` runs, which is why every call without the argument shares one list. New in 3.14, `functools.Placeholder` is a sentinel for reserving positional slots in `partial` objects, the modern spelling of the pass-through-then-fill pattern. Chapter 3's copies sample shows the object side of the same trap, see book 8, ch 3 (python 3.14, the object model).

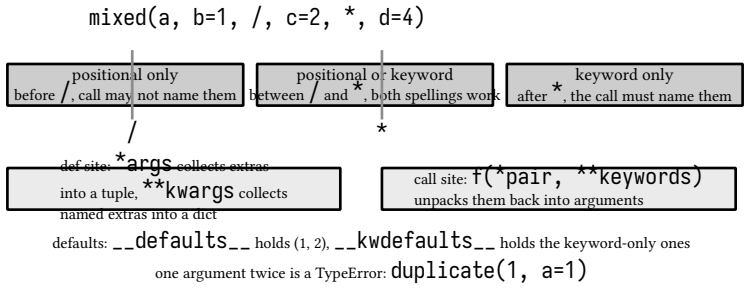


figure 22: the parameter zones of one signature

6.3. decorators

`@` is sugar. The expression above the `def` runs once, at definition time, receives the fresh function, and whatever it returns is what the name binds to:

```
registry = {}


def register(fn):
    registry[fn.__name__] = fn
    return fn


@register
def signed_up():
    return "original"


check(
    registry["signed_up"]() == "original", "a decorator can register instead of replace"
)


def shout(fn):
    def wrapped(name):
        return fn(name).upper()

    return wrapped


def plain_greet(name):
    return f"hi {name}"


manual = shout(plain_greet)
check(manual("x") == "HI X", "@ is sugar for passing the function and rebinding")
```

listing 78: a decorator that only registers, and the same wrapping done by hand (source: python/samples/src/Ch06/decos.py, lines 19-49)

The manual `shout(plain_greet)` line is the whole protocol with the sugar removed. Stacking reads bottom up, the decorator nearest the `def` applies first, and both run at definition time, before any call:

```
order = []


def first(fn):
    order.append("first")
    return fn


def second(fn):
    order.append("second")
    return fn


@first
@second
def stacked():
    return 1


check(order == ["second", "first"], "the decorator nearest the def applies first")
```

listing 79: the order list proving bottom up application (source: python/samples/src/Ch06/decos.py, lines 51-70)

Wrapping costs identity, and `functools.wraps` is the standard repair, copying `__name__`, `__doc__`, and the rest onto the wrapper and keeping the original reachable as `__wrapped__`:

```
def shout_wrapped(fn):
    @wraps(fn)
    def wrapped(name):
        return fn(name).upper()

    return wrapped


@shout_wrapped
def greet(name):
    "say hi"
    return f"hi {name}"


check(
    greet("x") == "HI X" and greet.__name__ == "greet" and greet.__doc__ == "say hi",
    "wraps preserves the wrapped function's identity",
)
check(greet.__wrapped__("x") == "hi x", "__wrapped__ still reaches the original")
```

listing 80: wraps preserving the identity, and `__wrapped__` reaching the original (source: python/samples/src/Ch06/decos.py, lines 73-91)

A decorator with arguments is one more closure level: the outer function takes the arguments, returns the decorator, the decorator returns the wrapper. A stateful decorator keeps its counter in a cell the same way a closure does:

```
def repeat(times):
    def deco(fn):
        def call(*args, **kwargs):
            return [fn(*args, **kwargs) for _ in range(times)]

        return call

    return deco


@repeat(3)
def ping():
    return "p"


check(ping() == ["p", "p", "p"], "a decorator with arguments adds one closure level")


def counting(fn):
    count = 0

    @wraps(fn)
    def call():
        nonlocal count
        count += 1
        return fn(), count

    return call


@counting
def tick():
    return "t"


tick()
tick()
check(tick() == ("t", 3), "a stateful decorator keeps its counter in a cell")
```

listing 81: repeat(3) as a three level closure, and a counting decorator (source: python/samples/src/Ch06/decos.py, lines 94-131)

Nothing requires the decorator to be a function. Any callable works, and a class instance carrying `__call__` gives the decorator somewhere to keep state as attributes:

```
class UpperBound:
    def __init__(self, fn):
        self.fn = fn
        self.calls = 0

    def __call__(self, *args, **kwargs):
        self.calls += 1
        return self.fn(*args, **kwargs).upper()


@UpperBound
def yell(word):
    return word


check(
    yell("hey") == "HEY" and yell.calls == 1, "any callable can decorate, classes too"
)
```

listing 82: a class as decorator, counting calls in an attribute (source: python/samples/src/Ch06/decos.py, lines 134-151)

The class machinery behind method calls is book 8, ch 7 (python 3.14, classes and inheritance), and the two decorator ecosystems built on this protocol, dataclasses and properties, get their own chapters.

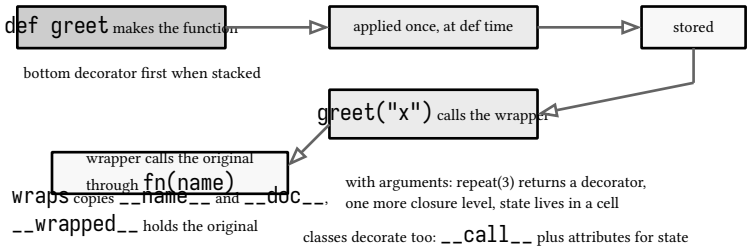


figure 23: a decorated function, from def to call

6.4. the functools toolkit

`partial` freezes a function's arguments into a new callable, positional or keyword:

```
def volume(length, width, height):
    return length * width * height


footprint = partial(volume, height=1)
check(footprint(2, 3) == 6, "partial freezes positional and keyword arguments")
```

listing 83: one keyword frozen, the rest supplied at call time (source: python/samples/src/Ch06/functool.py, lines 19-24)

`lru_cache` memoizes by arguments and answers with counts, and the maxsize eviction is visible in the same numbers:

```
@lru_cache(maxsize=2)
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)


check(
    fib(10) == 55 and fib.cache_info().currsize == 2,
    "lru_cache caches by arguments and evicts at maxsize",
)
fib(10)
check(fib.cache_info().hits ≥ 1, "the second call for the same arguments is a hit")
```

listing 84: a size 2 cache computing fib(10) correctly while evicting (source: python/samples/src/Ch06/functool.py, lines 27-37)

`fib(10)` recurses through many repeated calls, the cache holds only the 2 most recent arguments at each level, and `cache_info` reports the hits that the eviction allowed. `cached_property` is the instance level sibling, computing once and storing the result in the instance dict, where deleting it forces a recompute:

```
class Rectangle:
    def __init__(self, side):
        self.side = side

    @cached_property
    def area(self):
        return self.side * self.side


shape = Rectangle(4)
check(
    shape.area == 16 and "area" in shape.__dict__,
    "cached_property stores the value on the instance",
)
del shape.__dict__["area"]
check(shape.area == 16, "deleting the stored value recomputes")
```

listing 85: the value landing in `__dict__`, and deletion recomputing (source: python/samples/src/Ch06/functool.py, lines 40-55)

`singledispatch` routes on the runtime type of the first argument, the most specific registration wins, and unregistered types fall to the default:

```
@singledispatch
def describe(value):
    return "something"


@describe.register
def _(value: int):
    return f"int {value}"


@describe.register
def _(value: bool):
    return "bool"


check(
    describe(1) == "int 1" and describe("x") == "something",
    "dispatch picks the registration for the runtime type",
)
check(describe(True) == "bool", "the most specific registration wins")
```

listing 86: int and bool registrations, with bool winning over its own base class (source: python/samples/src/Ch06/functool.py, lines 58-77)

`reduce` folds left with a seed, refuses an empty iterable without one, and is almost never what you want because a builtin or a comprehension says it better:

```
check(
    reduce(lambda acc, n: acc + n, [1, 2, 3], 0) == 6, "reduce folds left with a seed"
)
try:
    reduce(lambda acc, n: acc + n, [])
    check(False, "empty fold accepted")
except TypeError:
    check(True, "an empty fold without a seed is an error")
check(sum([1, 2, 3]) == 6, "sum is the builtin answer for the common case")
```

listing 87: the fold, the empty refusal, and sum being the answer (source: python/samples/src/Ch06/functool.py, lines 79-87)

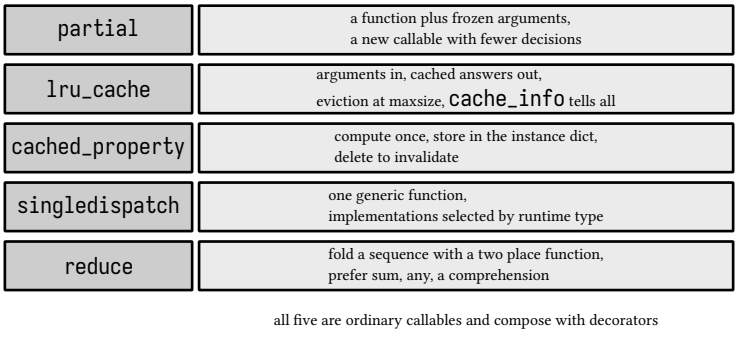


figure 24: the toolkit, what each tool holds and when it pays

VERIFY: 37 checks, first run green, bytecode pinned

All four samples in this chapter ran green on their first execution, after the underlying behaviors were probed in scratch scripts, and that green is the weakest evidence in the book, which is why the closure claims carry a second witness: the expect-dis rows beside the samples assert that `make_counter.<locals>.bump` compiles to `LOAD_DEREF`, `BINARY_OP`, `STORE_DEREF`, and `RETURN_VALUE`, verified against `dis.get_instructions` on this interpreter.

> If the compiler changed how closures are built, that file goes red before any prose drifts. Sample behavior verified by `make verify-py`, 37 checks in chapter 6 of the samples suite.

sources: docs.python.org/3/reference/compound_stmts.html (the def statement, parameter grammar with / and *, decorator application), docs.python.org/3/reference/simple_stmts.html (global, nonlocal, del), docs.python.org/3/reference/expressions.html (lambda, calls, argument unpacking), docs.python.org/3/library/functools.html (partial and the new Placeholder, lru_cache, cached_property, singledispatch, reduce), docs.python.org/3/library/inspect.html (signature rendering), docs.python.org/3/library/stdtypes.html (function attributes `__defaults__`, `__kwdefaults__`, `__wrapped__`), docs.python.org/3/whatsnew/3.13.html (PEP 667 locals semantics), docs.python.org/3/whatsnew/3.14.html (functools.Placeholder), and peps.python.org/pep-0570/ and pep-0667/, all accessed 2026-09-12; the error message texts and the dis opnames were probed on this machine the same day. Sample behavior verified by `make verify-py`, 37 checks in chapter 6.

7. classes and inheritance

Chapter 3, book 8, ch 3 (python 3.14, the object model), established that a name is a binding to an object and that every object has a type. A class ties those two facts together: it is itself an object, a namespace that executes once, and the factory that makes instances carrying that namespace as their type. This chapter walks the surface in four stops: attribute lookup and the class body, the dunder hooks the runtime calls on your behalf, inheritance and the method resolution order, and the descriptor machinery that sits under every attribute access. Every behavioral claim below is a check in the four samples, 56 in total, run under cpython 3.14.7 with `-X utf8`, and every byte count is a fact about this build.

7.1. classes as namespaces and factories

The `class` statement does 3 things. It executes the body top to bottom in a fresh namespace, that namespace becomes the class `__dict__`, and the resulting class object is bound to the name. Nothing about that requires an instance: the body runs at definition time, exactly once, whether or not you ever call the class. Calling it is a separate act, routed through `type`, which is the class of every class:

```
body_log = []
body_reads = 0


class Stack:
    """one class object, many instance objects, one shared namespace."""

    limit = 8  # class attribute, shared by every instance

    body_log.append("once")  # reads resolve outward, to the module
    body_reads += 1  # but assignment stores into the class namespace

    def __init__(self):
        self.items = []  # instance attribute, private to one object

    def push(self, value):
        self.items.append(value)
        return self

    def read_limit(self):
        return self.limit  # falls back to the class when self has none

    def read_broken(self):
        return limit  # noqa: F821, class body names are not in method scope
```

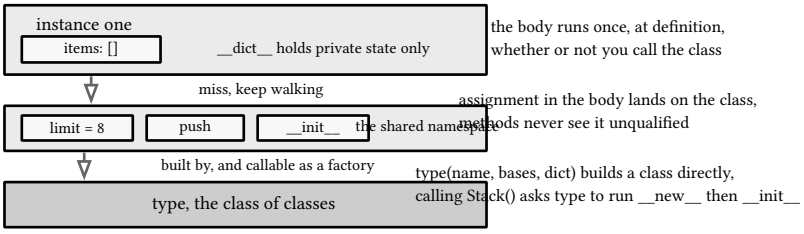
listing 88: the body mechanics, the shared namespace, and the one deliberate bug (source: python/samples/src/Ch07/lookup.py, lines 18-41)

Lines 27-28 are the two facts worth pinning early. `body_log.append` reads outward and finds the module-level list, because reading a name in a class body falls back to enclosing scopes. `body_reads += 1` looks outward too, but assignment stores into the class namespace, so `Stack.body_reads` is 1 while the module-level `body_reads` stays 0. The same asymmetry explains line 41: inside a method, the class namespace is simply not in scope, and `read_broken` raises `NameError` when called, while `read_limit` at 37-38 qualifies through `self` and works. The lookup that `read_limit` relies on is the whole section in one rule, instance first, then class:

```
Stack.limit = 16
check(one.read_limit() == 16, "rebinding it on the class moves every instance")
one.limit = 3
check(
    one.read_limit() == 3 and two.read_limit() == 16,
    "an instance attribute shadows the class one for that instance only",
)
del one.limit
check(one.read_limit() == 16, "deleting the shadow restores the class attribute")
check(
    one.items is not two.items and one.items == two.items == [],
    "each instance gets its own mutable state",
)
check(
    vars(one) == {"items": []} and vars(one) is one.__dict__,
    "vars of an instance is its own attribute dictionary",
)
check("push" not in vars(one) and "push" in vars(Stack), "methods live on the class")
check(one.push.__self__ is one, "attribute access on an instance binds the method")
```

listing 89: rebind on the class, shadow on the instance, delete to reveal (source: python/samples/src/Ch07/lookup.py, lines 57-75)

Line 57 rebinds `Stack.limit` and both instances see 16, because neither has a `limit` of its own. Line 59 gives `one` a private `limit`, line 64 deletes it again, and the shared value resurfaces. Lines 66-73 pin where private state lives: `vars(one)` is `one.__dict__`, each instance owns one, and the `items` lists of the two instances are distinct objects. Lines 74-75 carry the consequence for methods: `push` lives in the class dict, never in an instance dict, and attribute access on an instance binds it, `one.push.__self__ is one`. The binding is not a copy, it is descriptor machinery, section 4′s subject.



the object model of chapter 3 is the floor under this: a class is just an object with a useful type

figure 25: attribute lookup is a walk down a stack of namespaces

NOTE: one rule, applied recursively

Instance first, then class, is also the whole inheritance story: when the class namespace misses, the walk continues through the classes of the method resolution order, section 3. Nothing in this lookup is special to `class`, it is the same binding machinery chapter 3 traced for plain names, extended one level.

7.2. dunders

Dunder methods are the operator table of the language, and the runtime, not your code, decides when to dial one. Define `__len__` and `len` calls it, define `__eq__` and == calls it, define `__getitem__` and indexing, slicing, iteration, and membership all call it. The first listing is the comparison pair, with the two returns that make the protocol composable:

```
class Kelv:
    def __init__(self, k):
        self.k = k

    def __repr__(self):
        return f"Kelv({self.k})"

    def __eq__(self, other):
        if not isinstance(other, Kelv):
            return NotImplemented
        return self.k == other.k

    def __lt__(self, other):
        if not isinstance(other, Kelv):
            return NotImplemented
        return self.k < other.k


class Shelf:
    def __init__(self, titles):
        self.titles = list(titles)

    def __len__(self):
        return len(self.titles)

    def __getitem__(self, index):
        return self.titles[index]
```

listing 90: the comparison pair and the container surface (source: python/samples/src/Ch07/dunders.py, lines 18-44)

`NotImplemented` at lines 27 and 32 is a signal, not an exception: it tells the runtime to try the other operand's reflected hook and fall back to identity, which is why `a == 5` at line 73 is `False` rather than an error. Line 74 pins the price of defining `__eq__` alone, the class becomes unhashable, because equal objects must hash equal and the runtime cannot know that your `__eq__` agrees with the inherited hash. `sorted` at line 81 uses `__lt__` and nothing else, one method is the whole ordering protocol. The `Shelf` pair shows one method buying four operations:

```
class Vec:
    def __init__(self, xs):
        self.xs = list(xs)

    def __repr__(self):
        return f"Vec({self.xs})"

    def __add__(self, other):
        return Vec(self.xs + other.xs)

    def __radd__(self, other):
        return Vec(list(other) + self.xs)

    def __mul__(self, n):
        return Vec(self.xs * n)

    def __rmul__(self, n):
        return Vec(self.xs * n)
```

listing 91: the arithmetic surface, forward and reflected (source: python/samples/src/Ch07/dunders.py, lines 47-64)

The arithmetic rules follow one pattern: for `a + b`, the runtime tries `a.__add__` then `b.__radd__`. Lines 90-93 pin both lanes, `[5] + Vec` succeeds only because `list.__add__` refuses a `Vec` and the reflected `__radd__` runs. Lines 96-98 close the loop: without `__iadd__`, `v += w` computes `v + w` and rebinds the name, visible as a changed `id`.

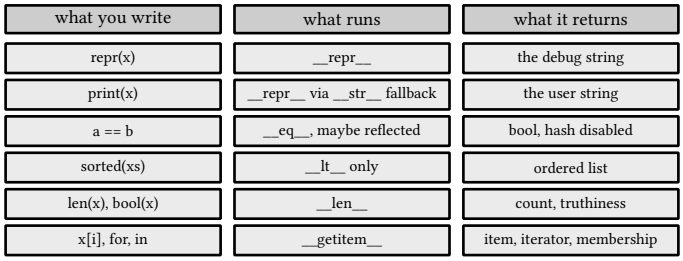


figure 26: what you call versus what the runtime calls

7.3. inheritance and the mro

Multiple inheritance needs a rule for ordering the namespaces, and python's is c3 linearization, computed once when the class object is built and stored as `__mro__`. The diamond is the canonical case:

```
class Base:
    def __init__(self, **kwargs):
        self.attrs = dict(kwargs)

    def who(self):
        return "a"


class B(Base):
    def who(self):
        return "b+" + super().who()


class C(Base):
    def who(self):
        return "c+" + super().who()


class D(B, C):
    def who(self):
        return "d+" + super().who()
```

listing 92: the diamond: four classes, one super call each (source: python/samples/src/Ch07/mro.py, lines 18-38)

`super()` is the sentence everyone misreads. It does not mean "my parent class", it means "the next class after me in this instance's mro". In a `D` instance the order is D, B, C, Base, object, so `D.who` calls `B.who`, which calls `C.who`, not `Base.who`, and the result is the string `d+b+c+a`. The same code outside the diamond, `C().who`, gives `c+a`, because a `C` instance's mro never mentions `B`. That single mechanism is what makes cooperative multiple inheritance work:

```
class Reader(Root):
    def __init__(self, order, *, tag="", **kwargs):
        order.append("reader")
        out = super().__init__(order, **kwargs)
        self.tag = tag
        return out


class Writer(Root):
    def __init__(self, order, *, mode="w", **kwargs):
        order.append("writer")
        out = super().__init__(order, **kwargs)
        self.mode = mode
        return out


class File(Reader, Writer):
    def __init__(self, order, **kwargs):
        order.append("file")
        super().__init__(order, **kwargs)


order = []
f = File(order, tag="csv", mode="a")
check(
    order == ["file", "reader", "writer", "root"],
    "cooperative inits run once, in mro order",
)
check(
    f.tag == "csv" and f.mode == "a", "each mixin consumes its keyword, passes the rest"
)
check(isinstance(f, Reader) and isinstance(f, Writer), "isinstance walks the mro")
```

listing 93: cooperative inits: each class consumes its keywords, forwards the rest (source: python/samples/src/Ch07/mro.py, lines 56-87)

Every `__init__` in the chain takes `**kwargs`, pulls out only the keywords it owns, and forwards the remainder. The recorded order is file, reader, writer, root, each initializer ran exactly once, and `Reader`'s `super().__init__` dispatched to `Writer`, a class it has never heard of. The contract for entering that chain is cheap: accept and forward what you do not consume. C3 also refuses to be fooled:

```
try:
    type("Broken", (Left, Right), {})
except TypeError as exc:
    check(
        "Cannot create a consistent method resolution order" in str(exc),
        "an impossible order is rejected at class creation",
    )
else:
    check(False, "an impossible order is rejected at class creation")
check(
    [k.__name__ for k in Right.__mro__] == ["Right", "Left", "object"],
    "the mro is computed when the class object is built",
)
check(super(B, D).who(D()) == "c+a", "super takes explicit start and target classes")
```

listing 94: an inconsistent request is refused at class creation, not at call time (source: python/samples/src/Ch07/mro.py, lines 100-113)

Asking for bases `(Left, Right)` when `Right` already inherits `Left` cannot be linearized, and the `type` call raises `TypeError` with “Cannot create a consistent method resolution order” before any instance exists.

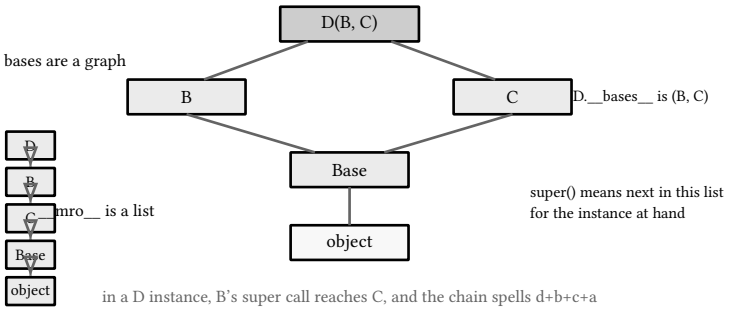


figure 27: the diamond and the linear order c3 derives from it

7.4. slots, properties, descriptors

The instance `__dict__` of section 1 is flexible and costs memory, and `__slots__` is the opt-out. Declaring slots tells the class to store attributes in fixed positions instead:

```
class WithDict:
    def __init__(self):
        self.x = 1
        self.y = 2


class WithSlots:
    __slots__ = ("x", "y")

    def __init__(self):
        self.x = 1
        self.y = 2


plain = WithDict()
slim = WithSlots()
check(hasattr(slim, "__dict__") is False, "slots remove the per instance dictionary")
try:
    slim.z = 3
except AttributeError as exc:
    check(
        "no __dict__ for setting new attributes" in str(exc),
        "new attributes are refused",
    )
else:
    check(False, "new attributes are refused")
check(
    sys.getsizeof(plain) == 48 and sys.getsizeof(slim) == 48,
    "both instances are 48 bytes on this build",
)
check(
    sys.getsizeof(plain.__dict__) == 296,
    "the dictionary version pays 296 more bytes on this build",
)
```

listing 95: the storage trade, measured on this build (source: python/samples/src/Ch07/descriptors.py, lines 18-51)

The numbers on this build: both instances are 48 bytes, but the dictionary version carries a 296-byte `__dict__` beside it, and the slots version refuses new attributes outright with "no `__dict__` for setting new attributes". Slots are inherited by declaring new ones, lines 55-64, and a subclass without its own `__slots__` quietly reintroduces a `__dict__`, the trap the sample sidesteps by declaring `("z",)`. The attribute you access every day is usually not a plain value but a protocol participant:

```
class Temp:
    def __init__(self):
        self._c = 0

    @property
    def c(self):
        return self._c

    @c.setter
    def c(self, value):
        self._c = value * 9

    @c.deleter
    def c(self):
        del self._c


t = Temp()
t.c = 5
check(t.c == 45, "property routes reads and writes through your functions")
check(
    type(Temp.__dict__["c"]) is property,
    "a property is an object in the class namespace",
)
check(
    hasattr(property, "__get__") and hasattr(property, "__set__"),
    "property carries the full descriptor protocol",
)
del t.c
check("_c" not in vars(t) and "_c" in vars(Temp()), "deleter support is opt in")
```

listing 96: property is a descriptor living in the class namespace (source: python/samples/src/Ch07/descriptors.py, lines 67-96)

`Temp.__dict__["c"]` is a `property` object, and `property` carries `__get__` and `__set__`. Reads and writes on the instance route through your functions, `t.c = 5` lands as `_c = 45`, and class access returns the property itself. The last listing is the same protocol written by hand, which is what `property`, plain functions, `staticmethod`, and `classmethod` all are:

```
class Twice:
    def __set_name__(self, owner, name):
        self.internal = "_" + name

    def __get__(self, obj, objtype=None):
        if obj is None:
            return self
        return getattr(obj, self.internal)

    def __set__(self, obj, value):
        setattr(obj, self.internal, value * 2)


class Holder:
    n = Twice()
```

listing 97: a hand written descriptor, with the hook that names it (source: python/samples/src/Ch07/descriptors.py, lines 99-113)

`__set_name__` runs at class creation and hands the descriptor its attribute name, so one `Twice` class serves many attributes. The instance stores `_n`, the descriptor stores the doubling policy, and `h.n = 21` reads back as 42. Lines 122-125 are the punchline that close the chapter: a plain function has `__get__`, which is exactly the machinery behind section 1's observation that methods live on the class and bind on access. Bound methods were descriptors all along.

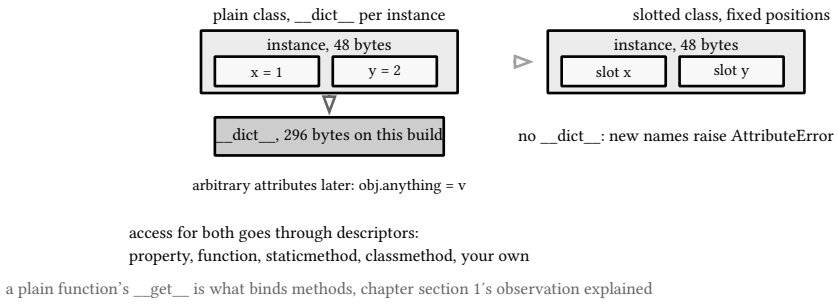


figure 28: where attributes live, with and without slots

VERIFY: 56 checks behind this chapter

The four samples ran green under cpython 3.14.7 with `-X utf8`: `lookup.py` pins the namespace and factory facts, 13 checks, `dunders.py` the operator table, 19, `mro.py` linearization and cooperative init, 10, and `descriptors.py` storage and the protocol,
14. Three drafts went red on the way: the class-body counter stored into the class instead of the module, `5 + Vec` cannot iterate an int, and the first inconsistent-mro probe used a consistent order. Each red draft is now a check that would catch the same mistake.

sources: docs.python.org/3/reference/datamodel.html, special method names, custom classes, and the descriptor protocol sections, accessed 2026-09-12; docs.python.org/3/library/functions.htmlsuper, accessed 2026-09-12; docs.python.org/3/reference/expressions.html, binary arithmetic operations and value comparisons, accessed 2026-09-12; the 48 and 296 byte figures and every lookup, binding, and linearization order probed on this machine the same day. Sample behavior verified by `make verify-py`, 56 checks in chapter 7 of the samples suite.

8. the typing system

Python's type annotations began life as a runtime eagerly evaluated dictionary, grew a string-quote escape hatch for forward references, and in 3.14 became lazy by default. That history matters because the new behavior changes what `__annotations__` means and adds a new module, `annotationlib`, as the sanctioned way to read annotations. This chapter walks annotations at runtime first, then the PEP 695 generic syntax, then the toolbox of unions, `Literal`, `TypedDict`, and `Never`, and closes on `Protocol`, structural typing. One boundary up front: the annotations themselves never enforce anything at runtime. Enforcement belongs to static checkers and to libraries like the one book 8, ch 26 (python 3.14, pydantic) covers, and this chapter states that limit rather than blurring it. Every claim below is one of the 56 checks in the four samples, run under cpython 3.14.7 with `-X utf8`.

8.1. annotations at runtime

Before 3.14, `def later(x: Undefined)` raised `NameError` at definition time unless `Undefined` already existed, and the escape hatch was a quoted string, `"Missing"`, which then had to be evaluated by hand. PEP 649 changed the default: the compiler now emits an `__annotate__` function per annotated object and calls it only when annotations are accessed. PEP 749 added the public reading API, `annotationlib`, with three formats that name the whole design:

```
def later(x: Undefined, y: "Missing") -> None: ...


class Node:
    nxt: Node | None  # unquoted, self referential, fine in 3.14
    n: int = 0


module_probe: "list[int]"
```

listing 98: an unquoted forward reference, a quoted one, and a self referential class (source: python/samples/src/Ch08/annotate.py, lines 22-30)

The `Node` body at lines 25-27 is the headline demo: `nxt: Node | None` names the class currently being defined, unquoted, and it works because nothing evaluates until `Node` exists. The three formats read the same objects three ways:

```
try:
    annotationlib.get_annotations(later)
except NameError as exc:
    check(str(exc) == "name 'Undefined' is not defined", "value format fails at access")
else:
    check(False, "value format fails at access")
check(
    annotationlib.get_annotations(later, format=Format.STRING)
    == {"x": "Undefined", "y": "Missing", "return": "None"},
    "string format returns the source text",
)
fr = annotationlib.get_annotations(later, format=Format.FORWARDREF)
check(
    isinstance(fr["x"], annotationlib.ForwardRef)
    and fr["x"].__forward_arg__ == "Undefined"
    and fr["y"] == "Missing",
    "forwardref proxies undefined names, quoted ones stay strings",
)
```

listing 99: value raises at access, string gives source text, forwardref gives proxies (source: python/samples/src/Ch08/annotate.py, lines 33-50)

VALUE is the format `__annotations__` uses, so it raises `NameError` exactly when the name is still missing, line 34, the old failure moved from definition time to access time. STRING returns the source text, `"Undefined"` and `"None"`, and cannot fail. FORWARDREF is the diagnostic middle: real values where they resolve, `ForwardRef` proxy objects where they do not, and line 48 pins the boundary inside one dict, the unquoted name became a proxy, the quoted one stayed a string. Access order is the other half of the story:

```
class Undefined:
    pass


class Missing:
    pass


check(
    annotationlib.get_annotations(later)
    == {"x": Undefined, "y": "Missing", "return": None},
    "after definition, values resolve, quoted ones stay strings",
)
check(
    annotationlib.get_annotations(later, eval_str=True)["y"] is Missing,
    "eval_str evaluates quoted annotations too",
)
check(
    later.__annotations__["x"] is Undefined,
    "the __annotations__ view uses value format",
)
```

listing 100: define the names late, then read values, strings, or evaluated strings (source: python/samples/src/Ch08/annotate.py, lines 53-73)

Once `Undefined` and `Missing` exist, lines 61-65 read real values, with the quoted annotation still a string, and `eval_str=True` at 66-69 evaluates even those. `__annotations__` at 70-73 is the VALUE view. The `__annotate__` function itself is one more lazy object, with its own protocol rule:

```
try:
    later.__annotate__(Format.STRING)
except NotImplementedError:
    check(True, "compiler annotate functions support only value when called directly")
else:
    check(False, "compiler annotate functions support only value when called directly")
check(
    annotationlib.call_annotate_function(later.__annotate__, Format.STRING)["x"]
    == "Undefined",
    "call_annotate_function derives the other formats",
)
check(
    fr["x"].evaluate() is Undefined,
    "a forwardref can be evaluated once the name exists",
)
check(ForwardRef is annotationlib.ForwardRef, "typing forwardref is an alias now")
```

listing 101: annotate functions, the direct-call restriction, and the sanctioned wrapper (source: python/samples/src/Ch08/annotate.py, lines 85-100)

The compiler's own `__annotate__` supports only VALUE when called directly and raises `NotImplementedError` for the rest, lines 85-90, which is why `call_annotate_function` exists, lines 91-95. A held `ForwardRef` stays useful after the fact, line 96-99: `evaluate()` resolves against the scope it came from. And `typing.ForwardRef` is now an alias of `annotationlib.ForwardRef`, line 100, the PEP 749 cleanup. `typing.get_type_hints` still works and still normalizes a `None` return to `NoneType`, lines 109-112.

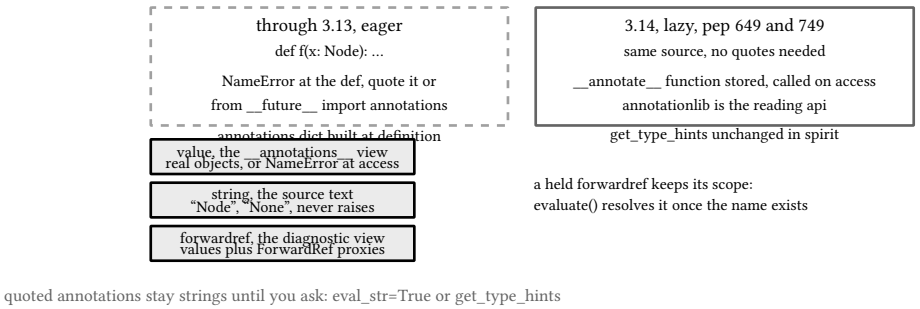


figure 29: the same annotation, read before and after 3.14, and in the 3 formats

WARNING: string format is not source recovery

STRING returns normalized text, not the exact characters you typed: `None` comes back quoted, constants fold, and expressions python cannot replay, like `a if b else c`, come back wrong or raise. Use it to render annotations, never to reconstruct code. The `from __future__ import annotations` import still exists in 3.14 and still forces the all-strings world, but new code should simply drop it, the default now defers correctly.

8.2. generics

PEP 695 replaced the typing-module ceremony with syntax. `type` statements build aliases, square brackets on `def` and `class` declare type parameters, and the parameters are scoped to the definition instead of leaking into the module:

```
type Vector[T] = list[T]
type Number = int | float
type Late = NotDefinedHere  # noqa: F821, the deferral is the demo
type IntFunc[**P] = Callable[P, int]
type Spread[*Ts] = tuple[Ts, ...]


def first[T](items: list[T]) -> T:
    return items[0]


class Box[T]:
    def __init__(self, item: T):
        self.item = item


class Bounded[S: LaterBound]:
    pass


class Choice[A: (str, bytes)]:
    pass
```

listing 102: aliases, a generic function, a generic class, a bound, and a constraint (source: python/samples/src/Ch08/generics.py, lines 23-44)

Line 23 builds a `TypeAliasType` whose `__value__` is `list[T]`, lines 30-36 attach parameters to a function and a class, and lines 39-44 spell the two restrictions a parameter can carry: a bound, `[S: LaterBound]`, means at least that, a constraint tuple, `[A: (str, bytes)]`, means exactly one of those. The laziness of section 1 extends to all of it:

```
try:
    Late.__value__
except NameError as exc:
    check(
        str(exc) == "name 'NotDefinedHere' is not defined",
        "an alias to nothing defers the error",
    )
else:
    check(False, "an alias to nothing defers the error")
check(
    annotationlib.call_evaluate_function(Late.evaluate_value, Format.STRING)
    == "NotDefinedHere",
    "evaluate_value gives the string without raising",
)
```

listing 103: an alias to an undefined name defers its own error (source: python/samples/src/Ch08/generics.py, lines 59-72)

`type Late = NotDefinedHere` at line 25 succeeds quietly, and the `NameError` fires only when `Late.__value__` is read, lines 59-67. The `evaluate_value` hook gives the STRING view without raising, lines 68-72, the same three-format contract annotations use. Bounds behave identically, lines 89-104 of the sample: the bound of `Bounded`'s parameter raises until `LaterBound` exists, then resolves.

```
b = Box[int]("not an int")
check(
    b.item == "not an int" and isinstance(b, Box),
    "parametrization documents, it does not check",
)
check(
    Box[int] == Box[int] and Box[int] != Box[str],
    "generic aliases compare structurally",
)
OldT = TypeVar("OldT")


class OldBox(Generic[OldT]):
    def __init__(self, item: OldT):
        self.item = item


old = OldBox[str]("hi")
check(
    old.item == "hi" and OldBox.__parameters__ == (OldT,),
    "the 3.11 spelling still works",
)
check(
    OldBox.__type_params__ == () and Box.__type_params__ != (),
    "old classes expose parameters, new ones expose type params",
)
check(
    hasattr(OldT, "__covariant__") and OldT.__covariant__ is False,
    "old typevars carry variance flags, new ones infer",
)
```

listing 104: parametrization documents, and the old spelling side by side (source: python/samples/src/Ch08/generics.py, lines 105-134)

Line 105 is the boundary restated as a check: `Box[int]("not an int")` runs, because the parameter is documentation. `Box[int] == Box[int]` and `≠ Box[str]` at 110-113, generic aliases compare structurally. Lines 114-134 keep the 3.11 spelling honest, `TypeVar` plus an explicit `Generic` base still works, but the old class exposes `__parameters__` while the new one exposes `__type_params__`, and the old typevar carries `__covariant__` flags that the new syntax expects checkers to infer.

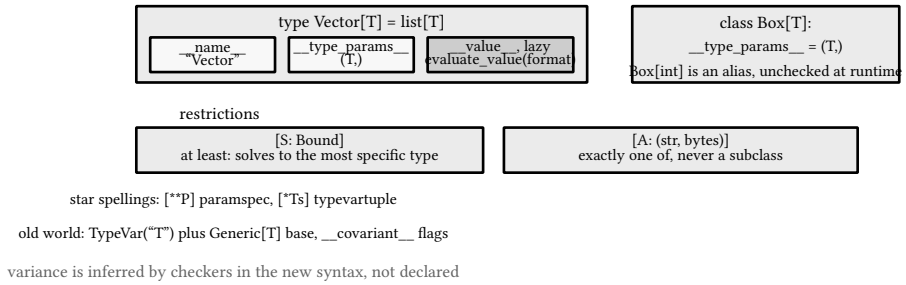


figure 30: what pep 695 builds, and where laziness sits

8.3. the typing toolbox

Five constructs cover most signatures, and the runtime story differs per construct, which is exactly what the sample pins:

```python
type MaybeInt = int | None


def user(uid: int | str, nick: MaybeInt = None) -> Literal["ok", "reject"]:
    if nick is None:
        return "reject"
    return "ok"


class Page(TypedDict):
    url: str
    title: NotRequired[str]


def kind(code: int) -> str:
    match code:
        case 0:
            return "zero"
        case 1:
            return "one"
        case _:
            assert_never(code)
```

listing 105: aliases, a literal return, a typeddict, and an exhaustive match (source: python/samples/src/Ch08/toolbox.py, lines 22-44)

Unions are real objects, `types.UnionType`, and uniquely among the constructs they work in `isinstance` at lines 47-51 of the sample. `Literal` at line 26 annotates the return of `user`, and lines 60-64 pin that the annotation object survives intact while the returned strings are just strings. `TypedDict` is the struct: a dict subclass at runtime that tracks `__required_keys__` and `__optional_keys__`, lines 69-72, and checks nothing on construction, lines 73-80, missing keys, extra keys, wrong value types all pass. `Callable` is runtime checkable. `Never` and `assert_never` close the toolbox, lines 91-100: the `match` handles every real input, and the unreachable branch calls `assert_never`, which fires as an `AssertionError`, "Expected code to be unreachable", if a case was missed. Exhaustiveness becomes a test.

construct	spelling	runtime behavior
union	int | None	isinstance works, real object
literal	Literal["a", "b"]	annotation kept, values unchecked
typeddict	class P(TypedDict)	dict, keys tracked, no checks
callable	Callable[[int], str]	isinstance works
never	assert_never(x)	assertionerror if reached

one rule for all five: annotations document, checkers enforce

figure 31: the toolbox, what each construct promises at runtime

PITFALL: typeddict is not validation

The sample constructs `Page(url="x", extra=1)` and reads back a `title` set to an int, both without complaint. A `TypedDict` is a dict with metadata for checkers. When a boundary needs runtime enforcement of shape, that is a job for data validation, book 8, ch 26 (python 3.14, pydantic)'s territory, or for plain checks you write yourself.

8.4. protocols

Nominal typing asks "what do you inherit from". Structural typing asks "what can you do". A `Protocol` class describes a shape, and any class with that shape matches, no inheritance required:

```
@runtime_checkable
class Quacks(Protocol):
    def quack(self) -> str: ...


class Silent(Protocol):
    def ping(self): ...


@runtime_checkable
class HasFields(Protocol):
    size: int
```

listing 106: three protocols: a method shape, an unmarked one, a data shape (source: python/samples/src/Ch08/protocols.py, lines 19-30)

```
class Duck:
    def quack(self):
        return "quack"


class LoudDuck(Quacks):
    def quack(self):
        return "loud"


class Cow:
    def moo(self):
        return "moo"


class Fake:
    quack = 42  # right name, wrong kind, no method at all


class Blob:
    def __init__(self, size):
        self.size = size


def announce(thing: Quacks) -> str:
    return thing.quack().upper()
```

listing 107: the implementations: related by shape, by blood, and not at all (source: python/samples/src/Ch08/protocols.py, lines 33-58)

`Duck` never mentions `Quacks` yet `isinstance(Duck(), Quacks)` is true, and `Quacks` appears nowhere in `Duck.__mro__`, while `LoudDuck`, which does inherit, gains both the check and the mro entry. The runtime checks have 3 limits the sample pins exactly:

```
try:
    isinstance(Cow(), Silent)
except TypeError as exc:
    check(
        str(exc) == "Instance and class checks can only be used with"
        " @runtime_checkable protocols",
        "runtime checks are opt in per protocol",
    )
else:
    check(False, "runtime checks are opt in per protocol")
check(isinstance(Fake(), Quacks), "runtime checks look for names, not signatures")
check(issubclass(Duck, Quacks), "issubclass works for method only protocols")
try:
    issubclass(Duck, HasFields)
except TypeError as exc:
    check(
        "non-method members don't support issubclass" in str(exc),
        "issubclass refuses protocols with data members",
    )
else:
    check(False, "issubclass refuses protocols with data members")
check(isinstance(Blob(3), HasFields), "isinstance still checks data members")
check(
    not hasattr(Cow(), "quack") and isinstance(Duck(), Quacks),
    "the match is presence of members, nothing is injected",
)
```

listing 108: the opt in, the name-only check, and the data restriction (source: python/samples/src/Ch08/protocols.py, lines 69-94)

First, runtime checks are opt in: `isinstance` against an undecorated protocol raises `TypeError`, lines 69-78. Second, `runtime_checkable` checks the presence of names only, so `Fake` with `quack = 42`, not a method at all, passes, line 79, signatures are a checker's business. Third, `issubclass` refuses protocols with non-method members, lines 81-89, while `isinstance` still works on instances, line 90. The choice between the two worlds is the closing decision:

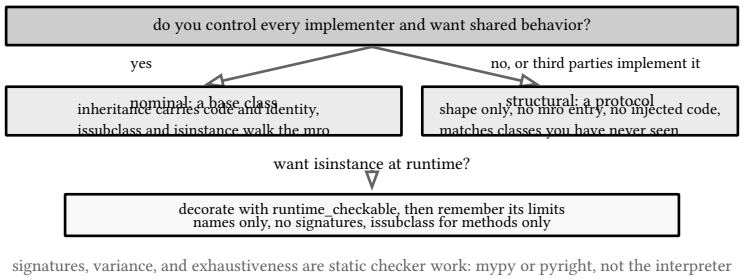


figure 32: nominal or structural, the decision

VERIFY: 56 checks behind this chapter

`annotate.py` pins the lazy evaluation contract, 15 checks, `generics.py` the PEP 695 surface and its deferral, 16, `toolbox.py` the five constructs and their runtime honesty, 14, and `protocols.py` structural matching with its 3 stated limits, 11. Two drafts went red on the way: the first read `__annotations__` before the names existed and met the VALUE `NameError`, and the first STRING comparison expected the quoted annotation to have become a `ForwardRef`. Both redrafts became checks in the final sample.

sources: docs.python.org/3/library/annotationlib.html, accessed 2026-09-12; peps.python.org/pep-0649 and peps.python.org/pep-0749, accessed 2026-09-12; docs.python.org/3/library/typing.html, generic syntax, `TypeAliasType`, `Protocol`, `Never`, and `assert_never`, accessed 2026-09-12; docs.python.org/3/library/enum.html and docs.python.org/3/library/dataclasses.html for the chapters either side of this one, accessed 2026-09-12; every

format, alias, bound, and protocol limit probed on this machine the same day. Sample behavior verified by `make verify-py`, 56 checks in chapter 8 of the samples suite.

9. dataclasses, enums, and pattern matching

Chapter 7 closed with descriptors doing the attribute work. Most classes you write are records, bags of named fields that want an `__init__`, a `__repr__`, an `__eq__`, and nothing else, and writing those by hand is exactly the boilerplate chapter 7 made you read through. This chapter covers the 3 tools that remove it: `@dataclass` for records, the `enum` family for closed sets, and `match` for dispatching on structure. Each tool is honest about what it removes and what it keeps: dataclasses generate methods, enums are real classes with real members, and `match` is a binding construct, not a switch with different spelling. Every claim below is one of the 57 checks in the four samples, run under cpython 3.14.7 with `-X utf8`.

9.1. dataclasses

The decorator reads the annotated class body and writes the methods for you, and the first thing it refuses to do is share mutable default state between instances:

```
try:

    @dataclass
    class Bad:
        xs: list = []

except ValueError as exc:
    check(
        "use default_factory" in str(exc),
        "a shared mutable default is refused at decoration",
    )
else:
    check(False, "a shared mutable default is refused at decoration")
```

listing 109: the one default the decorator will not write for you (source: python/samples/src/Ch09/fields.py, lines 19-31)

`xs: list = []` looks like the trap chapter 3 demonstrated with default arguments, and the decorator treats it as the same bug: a list built once at class creation and shared by every instance. The message names the fix, `default_factory`. The two classes that follow show what gets generated:

```
@dataclass(frozen=True)
class Key:
    a: int
    b: str = "none"


@dataclass
class Record:
    tag: str
    items: list = field(default_factory=list)
    weight: float = field(default=1.0, compare=False)
```

listing 110: a frozen key and a record with a factory and an ignored weight (source: python/samples/src/Ch09/fields.py, lines 34-44)

From 8 lines of annotations, `Record` gains an `__init__` taking 3 arguments, a `__repr__` printing all 3, and an `__eq__` comparing 2, because `weight` at line 44 opts out with `compare=False`. `Key` adds `frozen=True` on top, and frozen plus the generated `__eq__` gives a consistent `__hash__`, so `Key` instances work as dict keys, sample lines 54-57. Line 43 is the mutable discipline, `default_factory=list` constructs a fresh list per instance, and lines 51-53 pin that a mutation through one record leaves the other alone. The copying helpers have a depth policy worth pinning:

```
check(Conn(host="db").port == 5432, "kw_only forces keyword construction")
try:
    Conn("db")
except TypeError:
    check(True, "positional construction is refused")
else:
    check(False, "positional construction is refused")


@dataclass
class Inner:
    xs: list


@dataclass
class Outer:
    inner: Inner


o = Outer(Inner([1, 2]))
d = asdict(o)
d["inner"]["xs"].append(3)
check(o.inner.xs == [1, 2] and d["inner"]["xs"] == [1, 2, 3], "asdict deep copies")
plain = {"inner": o.inner}
plain["inner"].xs.append(9)
check(o.inner.xs == [1, 2, 9], "a plain dict aliases instead")
check(astuple(Key(1, "x")) == (1, "x"), "astuple strips to positionals")
```

listing 111: kw_only construction, and the copy depth of asdict against a plain dict (source: python/samples/src/Ch09/fields.py, lines 81-107)

`kw_only=True` at lines 75-78 moves every field out of positional construction, and lines 82-87 pin that `Conn("db")` now raises. `asdict` at lines 100-103 recurses: the nested `Inner` becomes a nested dict, and mutating that copy leaves the original alone. A plain `dict(...)` at 104-106 aliases the same `Inner` object, and the mutation lands. `astuple` strips to positionals, line 107. The cost side of the trade is recursion itself, `asdict` deep-copies everything, which is why the standard advice is to reach for it at boundaries, not in loops.

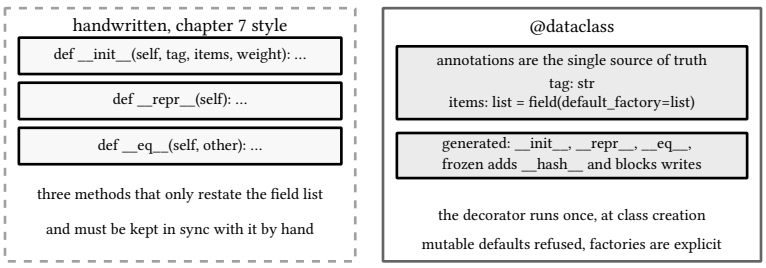


figure 33: the same record, handwritten and decorated

9.2. post-init and ordering

Generation has seams, and the seams are the interesting part. A field marked `init=False` is not a constructor argument, and a `__post_init__` hook is where derived state belongs:

```
LIMIT: ClassVar[int] = 100  # class variables are not fields


@dataclass(order=True)
class Task:
    id: float = field(compare=False)  # identity only, never compared
    name: str
    prio: int
    stamp: int = field(default=0, init=False)  # derived, not a constructor arg

    def __post_init__(self):
        self.stamp = self.prio * 10 + 1


@dataclass(frozen=True, slots=True)
class Pixel:
    x: int
    y: int
    label: str = field(init=False)  # derived, replace recomputes it
    notes: list = field(default_factory=list)

    def __post_init__(self):
        object.__setattr__(self, "label", f"pixel {self.x},{self.y}")
```

listing 112: ordering with an ignored field, and frozen slots with derived state (source: python/samples/src/Ch09/postinit.py, lines 20-42)

`order=True` on `Task` generates the full comparison set from the declared field order, skipping `id` because it is `compare=False`, so sorting at sample lines 51-55 orders by name then prio and the two tasks with different `id` values compare equal at line 56. `stamp` at line 28 is `init=False`, line 58-63 pins that passing it to the constructor raises `TypeError`, and `__post_init__` at 30-31 computes it. On a frozen class even your own hook cannot assign normally, so `Pixel` writes through `object.__setattr__`, line 42. The last two seams compose:

```
p = Pixel(3, 4)
check(p.label == "pixel 3,4", "frozen classes write derived state with object setattr")
check(
    hasattr(p, "__dict__") is False and Pixel.__slots__ == ("x", "y", "label", "notes"),
    "slots and frozen compose in one decorator",
)
p.notes.append("n")
check(p.notes == ["n"] and Pixel(3, 4).notes == [], "slots keep per instance defaults")
check(replace(p, x=9).label == "pixel 9,4", "replace reruns post_init on the copy")
try:
    p.x = 5
except Exception as exc:
    check(type(exc).__name__ == "FrozenInstanceError", "slots do not soften frozen")
else:
    check(False, "slots do not soften frozen")
```

listing 113: derived labels survive slots, and replace reruns post-init (source: python/samples/src/Ch09/postinit.py, lines 64-78)

`slots=True` together with `frozen=True` works in one decorator, lines 66-69, the instance has no `__dict__` and writes still raise `FrozenInstanceError`, lines 73-78. The subtle one is `replace` at line 72: it constructs a new instance from the current field values, and because `label` is `init=False` it is not carried over, `__post_init__` recomputes it from the new `x`. Had `label` been an ordinary field, `replace` would have copied the stale derivation, the trap the `init=False` marking avoids. The `ClassVar` annotation at line 20 stays out of the fields entirely, lines 47-50.

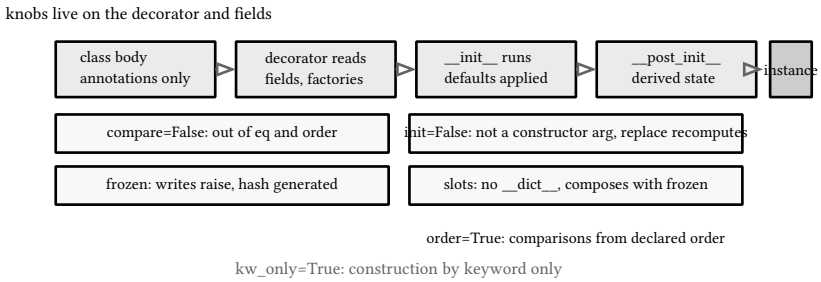


figure 34: the pipeline from decorator to instance, where each knob bites

9.3. enums

An enum is a closed set of named constants, and the family mixes in different base types with different rules. `auto()` alone has 3 policies:

```
class Color(StrEnum):
    RED = auto()
    GREEN = auto()


class Level(IntEnum):
    LOW = 1
    HIGH = 2


class Perms(Flag):
    READ = auto()
    WRITE = auto()
    EXEC = auto()


class Big(IntFlag):
    A = 1
    B = 2
```

listing 114: the family in one place: strenum, intenum, flag, intflag (source: python/samples/src/Ch09/enums.py, lines 19-37)

`StrEnum` members are strings whose `auto()` value is the lowercased member name, `IntEnum` members are integers, `Flag` and `IntFlag` members are bit sets whose `auto()` climbs powers of two. The string behavior is the one that surprises: `str(Color.RED)` is `"red"` and f-string interpolation gives the value, because `StrEnum` defines `__str__` as `str.__str__` for drop-in use as constants, while `repr` still names the member. `IntEnum` arithmetic is a one way door, sample line 63: `Level.LOW + Level.HIGH` is 3, a plain int, the membership is gone. The `Flag` rules repay attention:

```
class Build(Enum):
    DEBUG = 1
    SHIP = 2

    @classmethod
    def _missing_(cls, value):
        if isinstance(value, str):
            return cls[value.upper()]
        return None


class Period(Enum):
    FOUR = 4
    LAST = 4  # an alias, not a second member
```

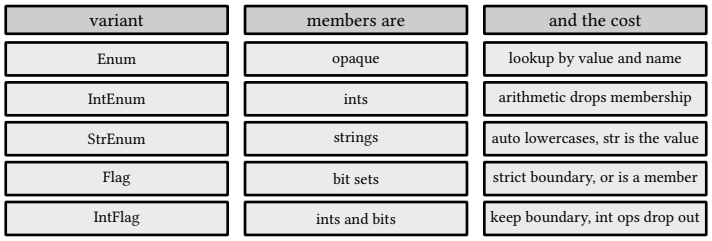
listing 115: a missing hook, and a duplicate value that aliases instead of colliding (source: python/samples/src/Ch09/enums.py, lines 40-53)

`_missing_` at 44-48 is the customization point for lookup by value: called when the constructor misses, it can map `"debug"` to `Build.DEBUG`, sample line 109. Duplicate values at 51-53 do not collide, `Period.LAST` is an alias of `Period.FOUR`, the same member object, and `len(Period)` counts 1. `@unique` turns that courtesy into an error, lines 93-103 of the sample, with the message naming the alias. Boundaries are the family's sharpest edge:

```
check([p.value for p in Perms] == [1, 2, 4], "flag auto climbs powers of two")
rw = Perms.READ | Perms.WRITE
check(
    rw.value == 3 and [p.name for p in rw] == ["READ", "WRITE"],
    "or combines, iter lists members",
)
check(
    Perms.READ in rw and bool(Perms(0)) is False and len(Perms(0)) == 0,
    "in, falsy empty, len zero",
)
kept = Big(20)
check(
    kept.value == 20 and isinstance(kept, Big),
    "intflag keeps out of range values by default",
)
try:
    Perms(20)
except ValueError as exc:
    check("invalid value 20" in str(exc), "flag rejects them, strict by default")
else:
    check(False, "flag rejects them, strict by default")
check(
    Perms._boundary_ is STRICT and Big._boundary_ is KEEP,
    "flag defaults to strict, intflag defaults to keep",
)
```

listing 116: combination arithmetic, iteration, and the two boundary defaults (source: python/samples/src/Ch09/enums.py, lines 67-91)

`READ | WRITE` is a member with value 3, iterating it yields the individual members, `in` tests membership, and the empty combination is falsy with length 0. Line 77 is the split: `Big(20)`, an `IntFlag`, keeps the out-of-range value 20, while `Perms(20)`, a `Flag`, raises `ValueError`, and lines 88-91 pin the defaults by identity, strict for `Flag`, keep for `IntFlag`.



auto: last value plus 1, powers of two for flags, lowercased names for strenum

figure 35: the enum family and what each variant gives up

9.4. match statements

`match` looks like a switch and behaves like destructuring assignment tried in order. Each `case` is a pattern, patterns nest, and a bare name inside a pattern is a capture, not a comparison:

```
class Point:
    __match_args__ = ("x", "y")

    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __repr__(self):
        return f"Point({self.x}, {self.y})"


def where(pt):
    match pt:
        case Point(0, 0):
            return "origin"
        case Point(0, y):
            return f"on y at {y}"
        case Point(x, 0):
            return f"on x at {x}"
        case Point(x, y) if x == y:
            return f"diagonal {x}"
        case Point():
            return "other point"
```

listing 117: class patterns, positional through match args, keyword by attribute name (source: python/samples/src/Ch09/match.py, lines 24-46)

`Point(0, 0)` at line 37 is a class pattern: it checks `isinstance` then matches subpatterns against attributes. Positional access needs `__match_args__`, line 25, which names which attributes the positions mean. The literals `0` compare, the bare names `x` and `y` at 39-42 capture, and line 43 adds a guard, `if x == y`, which can reject after the pattern matches. `Point()` at 45 with no arguments is just the isinstance check. The second function shows the other pattern kinds:

```
def classify(value):
    match value:
        case 0 | 1:
            return "small"
        case "stop" | "halt":
            return "halt"
        case [first, *rest]:
            return f"seq {first} {rest}"
        case {"kind": "point", "x": x, **extra}:
            return f"point {x} {sorted(extra)}"
        case Color():
            return f"color {value}"
        case _:
            return "other"
```

listing 118: literals with alternatives, sequences, mappings, and the wildcard (source: python/samples/src/Ch09/match.py, lines 49-62)

Line 51 matches either literal. Line 55 destructures any sequence with a star capture for the tail, and line 57 destructures a mapping, requiring the two keys and capturing the rest with **`extra`. Line 59 is the enum bridge, an irrefutable class pattern, and `_` at 61 always matches and binds nothing. The capture rule bites hardest at the bottom of the sample, lines 113-118: `case x` matches anything, so a lone lowercase name is a capture, while a dotted name like `Color.RED`, lines 121-132, is a value pattern that compares. That distinction is the whole syntax in one sentence, and the enum case makes exhaustiveness checkable: cover every member, keep a `case Color()` or `assert_never` arm, and a missed member announces itself in tests, book 8, ch 20 (python 3.14, testing with unittest)'s territory.

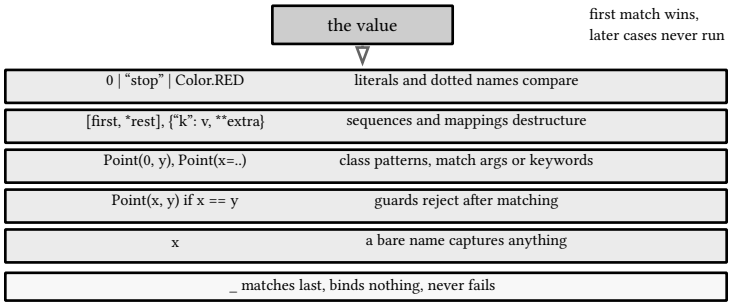


figure 36: a value enters, patterns are tried in order

VERIFY: 57 checks behind this chapter

`fields.py` pins the decorator contract, 15 checks, `postinit.py` the seams and composition, 11, `enums.py` the family and its boundaries, 17, and `match.py` the pattern kinds, 14. Two drafts went red: the first `sorted` expectation had the comparison fields backwards, and the first `replace` check expected a derived label to survive replacement, which is exactly the trap `init=False` exists to avoid.

sources: docs.python.org/3/library/dataclasses.html, accessed 2026-09-12; docs.python.org/3/library/enum.html, the variant table, `auto`, the functional api, and flag boundaries, accessed 2026-09-12; docs.python.org/3/reference/compound_stmts.html, the match statement grammar and pattern forms, accessed 2026-09-12; docs.python.org/3/howto/enum.html, accessed 2026-09-12; every generated repr, boundary default, and pattern result probed on this machine the same day. Sample behavior verified by `make verify-py`, 57 checks in chapter 9 of the samples suite.

10. exceptions and context managers

Errors are values in motion, and python gives them two vehicles: exceptions that unwind the stack looking for a handler, and context managers that wrap a block with entry and exit code so cleanup does not depend on the unwind finding one. This chapter walks the exception tree first, because knowing what derives from what decides what your `except` clauses can even see, then the exact semantics of `try`, then exception groups, the structure that lets one `raise` carry several failures at once, and closes on the `with` protocol and the `contextlib` toolkit built on it. Every claim below is one of the 53 checks in the four samples, run under cpython 3.14.7 with `-X utf8`, with the grammar refusals probed through `compile` rather than trusted.

10.1. the exception tree

Everything derives from `BaseException`, and only most of it from `Exception`. That split is the tree's load bearing wall, because `except Exception` is the conventional catch-all of application code, and it deliberately misses `KeyboardInterrupt` and `SystemExit`:

```
class Explode(BaseException):
    """outside Exception, so except Exception misses it"""


class ConfigError(Exception):
    def __init__(self, key, wanted):
        super().__init__(f"config key {key!r}: {wanted}")
        self.key = key
        self.wanted = wanted


def launch():
    raise Explode("boom")
```

listing 119: a custom error under exception, and one outside it on purpose (source: python/samples/src/Ch10/tree.py, lines 18-30)

```
check(issubclass(Exception, BaseException), "exception sits under baseexception")
check(
    issubclass(KeyboardInterrupt, BaseException)
    and not issubclass(KeyboardInterrupt, Exception)
    and issubclass(SystemExit, BaseException)
    and not issubclass(SystemExit, Exception),
    "interrupt and exit bypass exception",
)
check(
    [c.__name__ for c in ZeroDivisionError.__mro__[:4]]
    == ["ZeroDivisionError", "ArithmeticError", "Exception", "BaseException"],
    "the mro is the path to baseexception",
)
try:
    launch()
except Exception:
    check(False, "except exception misses a baseexception subclass")
except BaseException:
    check(True, "except exception misses a baseexception subclass")
```

listing 120: the tree facts, and the subclass that except exception cannot see (source: python/samples/src/Ch10/tree.py, lines 33-51)

Lines 34-40 pin the bypass by `issubclass`, and lines 46-51 pin it behaviorally: `Explode`, raised from `launch`, sails past `except Exception` and lands in `except BaseException`. Handlers match subclasses, line 52-55, `FileNotFoundError` is an `OSError`. Some builtin classes unpack their arguments into attributes, lines 56-61: `OSError(22, "invalid mode", "data.txt")` carries `errno`, `strerror`, and `filename` for free, the shape your own error classes should aim for. Chaining has two forms, explicit and implicit:

```
try:
    try:
        raise ValueError("inner")
    except ValueError as exc:
        raise RuntimeError("outer") from exc
except RuntimeError as exc:
    check(
        type(exc.__cause__) is ValueError
        and exc.__suppress_context__ is True
        and type(exc.__context__) is ValueError,
        "raise from sets cause and hides context",
    )
try:
    try:
        raise ValueError("first")
    except ValueError:
        raise RuntimeError("second")
except RuntimeError as exc:
    check(
        exc.__cause__ is None and type(exc.__context__) is ValueError,
        "unrelated raises chain implicitly",
    )
```

listing 121: raise from sets the cause, an unrelated raise chains anyway (source: python/samples/src/Ch10/tree.py, lines 62-83)

`raise ... from exc` at line 66 sets `__cause__`, marks `__suppress_context__` so tracebacks show only the causal chain, and still records the ambient `__context__`. The plain `raise` at line 78 leaves `__cause__` empty but fills `__context__` with the exception being handled, the "during handling" chain. `from None` at line 94 suppresses context entirely, and any exception can serve as a cause, lines 97-100. The custom class design in one listing: `ConfigError` formats its message once in `super().__init__`, stores structured attributes, and keeps `args` consistent with `str`, lines 22-27 and 84-92.

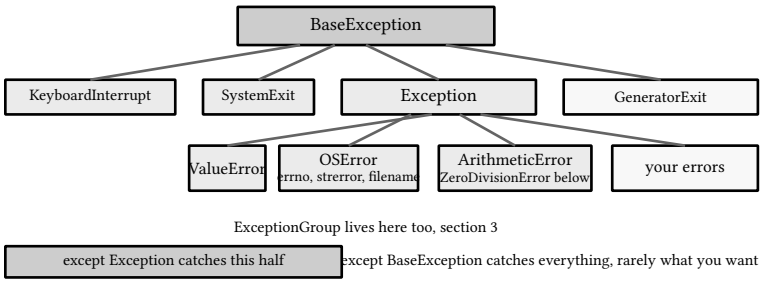


figure 37: the tree, and what a handler can see

NOTE: why the split exists

Control flow that crosses your program's boundary is not an error in it: ctrl-c, interpreter exit, generator close. Forcing those through `except Exception` would mean a library's blanket handler eats your shutdown. Catch `Exception` in application code, let the rest travel.

10.2. try semantics

The `try` statement has 4 blocks and one fixed order: the body runs, handlers are tested top to bottom on failure, `else` runs only when no failure happened, and `finally` always runs, on success, on failure, even on `return`:

```
def match_order(value):
    try:
        raise value
    except ValueError:
        return "value"
    except LookupError:
        return "lookup"


def else_runs(fail):
    try:
        if fail:
            raise KeyError("k")
    except KeyError:
        return "caught"
    else:
        return "clean"


def finally_order(fail):
    log = []
    try:
        log.append("try")
        if fail:
            raise KeyError("k")
    except KeyError:
        log.append("except")
    else:
        log.append("else")
    finally:
        log.append("finally")
    return log
```

listing 122: three probes: matching order, else, and the recorded block order (source: python/samples/src/Ch10/trysem.py, lines 19-50)

Handler matching is first match wins, not best match: lines 63-68 of the sample pin that a `KeyError` matches a `ValueError` handler not at all and a `LookupError` handler regardless of position, and an `IndexError`, which is both a `LookupError` and would match later clauses, stops at the `LookupError` line. The recorded order at lines 73-77 is the whole state machine in one list, try, except, finally when failing, try, else, finally when clean. The trap the block order enables is famous and now warned about:

```
SWALLOW = "def swallow():\n try:\n  raise ValueError('lost')\n finally:\n  return 42\n"


def reraise():
    try:
        raise ValueError("orig")
    except ValueError:
        raise
```

listing 123: the swallowing finally, and the bare raise that preserves the original (source: python/samples/src/Ch10/trysem.py, lines 53-60)

The source at line 53 is compiled under a warning recorder, and cpython 3.14 emits a `SyntaxWarning`, "'return' in a 'finally' block", lines 78-84. The compiled function still returns 42, line 86, and the `ValueError` it was unwinding is gone, no handler will ever see it, which is why the warning exists. The remaining semantics are pinned one by one: a bare `raise` inside a handler re-raises the active exception, lines 87-92, a bare `raise` with nothing active raises `RuntimeError`, "No active exception to re-raise", lines 93-101, the `as` name is deleted when the handler exits, lines 102-111, and `finally` runs while the exception is still traveling, visible as the try, finally, outer except order at lines 113-121.

A handler may name several exception types, and 3.14 changed how that is spelled. Through 3.13 the parenthesized tuple was the only form, `except (ValueError, UnicodeDecodeError):`, and PEP 758 legalizes the paren-less `except ValueError, UnicodeDecodeError:`. The parentheses return the moment an `as` clause rides the handler: the interpreter refuses `except ValueError, UnicodeDecodeError as exc:` with "multiple exception types must be parenthesized when using 'as'" while the parenthesized twin with `as` parses. Matching does not change, the list is the same tuple and first match still wins. The formatter has an opinion under a py314 target: `ruff format` rewrites the parenthesized form to the paren-less one, which is why the service part's kernel in book 8, ch 27 (python 3.14, http kernel) ships the paren-less spelling.

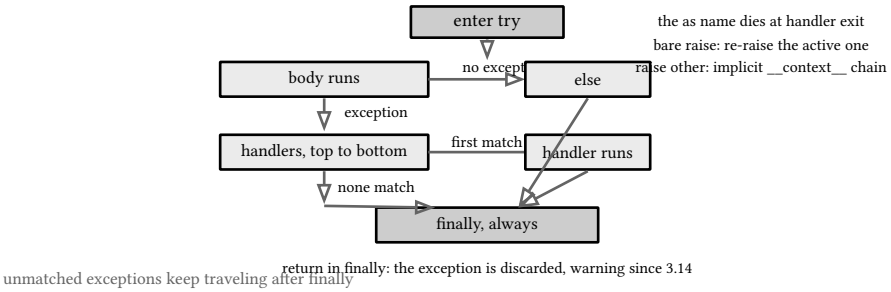


figure 38: the try statement as a state machine

10.3. exception groups

One operation can fail in several ways at once, and PEP 654 gave that a carrier: `ExceptionGroup`, a node holding a message and a tuple of leaves and other groups:

```
group = ExceptionGroup("eg", [ValueError(1), TypeError(2), ValueError(3)])
check(
    group.message == "eg" and len(group.exceptions) == 3,
    "a group is message plus a tuple",
)


def kinds(exc):
    return [(type(e).__name__, e.args) for e in exc.exceptions]
```

listing 124: a group is a message plus a tuple, and identity is not equality (source: python/samples/src/Ch10/groups.py, lines 19-28)

The `kinds` helper at 26-27 exists because exceptions compare by identity: two `ValueError(1)` instances are not equal, so the sample compares types and args. The structure operations are `split` and `subgroup`, and they recurse:

```
matched, rest = group.split(ValueError)
check(
    kinds(matched) == [("ValueError", (1,)), ("ValueError", (3,))]
    and kinds(rest) == [("TypeError", (2,))],
    "split partitions one group by type",
)
check(
    kinds(group.subgroup(ValueError)) == [("ValueError", (1,)), ("ValueError", (3,))],
    "subgroup keeps only matches",
)
nested = ExceptionGroup(
    "outer", [ValueError(1), ExceptionGroup("inner", [TypeError(2)])]
)
check(
    type(nested.exceptions[1]) is ExceptionGroup
    and kinds(nested.subgroup(TypeError).exceptions[0]) == [("TypeError", (2,))],
    "groups nest, and split recurses into them",
)
try:
    raise ExceptionGroup("bad", [KeyboardInterrupt()])
except TypeError as exc:
    check(
        "Cannot nest BaseExceptions in an ExceptionGroup" in str(exc),
        "exception is for exceptions",
    )
else:
    check(False, "exception is for exceptions")
check(
    kinds(BaseExceptionGroup("ok", [KeyboardInterrupt()]))
    == [("KeyboardInterrupt", ())],
    "baseexceptiongroup holds the rest",
)
```

listing 125: split partitions, subgroup filters, nesting is preserved, and the two container classes differ (source: python/samples/src/Ch10/groups.py, lines 30-61)

`split` at 30-35 returns a pair, matched and rest, each a group with the original shape. The nested case at 40-47 is the design point: subgrouping a tree prunes it, it does not flatten it. Containment has a rule, lines 48-61: `ExceptionGroup` refuses `BaseException` leaves with `TypeError`, and `BaseExceptionGroup` exists for when you mean it. The handler syntax is `except*`, with semantics deliberately different from `except`:

```
try:
    raise ValueError("plain")
except* ValueError as exc:
    seen.append(exc)
check(
    len(seen) == 1
    and type(seen[0]) is ExceptionGroup
    and seen[0].message == ""
    and kinds(seen[0]) == [("ValueError", ("plain",))],
    "except star wraps a plain exception in a group",
)
try:
    try:
        raise ExceptionGroup("eg", [ValueError(1), TypeError(2), OSError(3)])
    except* TypeError as exc:
        seen.append(kinds(exc))
    except* OSError as exc:
        seen.append(kinds(exc))
except ExceptionGroup as leftover:
    check(
        kinds(leftover) == [("ValueError", (1,))],
        "unmatched leaves propagate as a group",
    )
else:
    check(False, "unmatched leaves propagate as a group")
check(
    seen[1:] == [[("TypeError", (2,))], [("OSError", (3,))]],
    "every matching star clause runs, unlike plain except",
)
```

listing 126: wrapping of plain exceptions, every matching clause runs, leftovers propagate (source: python/samples/src/Ch10/groups.py, lines 63-91)

A plain exception that matches is wrapped in a group with an empty message so the `as` name is always a group, lines 63-73. Every matching clause runs, lines 74-90: the `TypeError` clause and the `OSError` clause each handle their slice, and the unmatched `ValueError` leaf leaves the statement as a new group. The grammar polices the sharp edges, lines 92-112: `except` and `except*` cannot mix in one `try`, a bare `except*` is a `SyntaxError`, and `break`, `continue`, and `return` are banned inside the star blocks. The place groups surface in real code is structured concurrency:

```
async def main():
    async with asyncio.TaskGroup() as tg:
        tg.create_task(fail_first())
        tg.create_task(fail_second())


try:
    asyncio.run(main())
except ExceptionGroup as exc:
    check(
        exc.message == "unhandled errors in a TaskGroup"
        and sorted(type(x).__name__ for x in exc.exceptions)
        == ["TypeError", "ValueError"],
        "taskgroup delivers failures as one group",
    )
else:
    check(False, "taskgroup delivers failures as one group")
```

listing 127: a taskgroup with two failing tasks delivers both failures at once (source: python/samples/src/Ch10/groups.py, lines 124-141)

The `TaskGroup` waits for every task, then raises one `ExceptionGroup` carrying both failures with the message "unhandled errors in a TaskGroup", so a partial failure cancels siblings and still reports everything. book 8, ch 21 (python 3.14, asyncio) owns the full async story, this is the delivery mechanism arriving early.

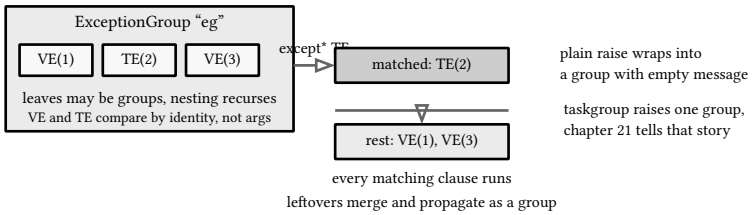


figure 39: one group, split by a handler

10.4. context managers

`try/finally` dedicates a statement to cleanup. The `with` statement dedicates an object to it, and the protocol is two methods:

```
class Probe:
    def __init__(self):
        self.exits = []

    def __enter__(self):
        self.entered = True
        return self

    def __exit__(self, et, ev, tb):
        self.exits.append((et, ev is None, tb is None))
        return False


class Mute:
    def __enter__(self):
        return self

    def __exit__(self, et, ev, tb):
        return True


class Loud:
    def __enter__(self):
        return self

    def __exit__(self, et, ev, tb):
        raise RuntimeError("exit failed")
```

listing 128: three two-method classes: a reporter, a suppressor, a failure (source: python/samples/src/Ch10/ctxmgr.py, lines 19-46)

`__enter__`'s return value is what the `as` name binds, and `__exit__` receives the triple of exception type, value, and traceback, or three `None`s on a clean exit, sample lines 49-60. The return value of `__exit__` is a verdict: falsy lets the exception continue, truthy suppresses it, sample lines 61-63, any truthy value, not just `True`. An `__exit__` that itself raises replaces the body exception and chains it as context, lines 64-73. The generator spelling covers the common case:

```
events = []


@contextlib.contextmanager
def span(name):
    events.append(f"enter {name}")
    try:
        yield f"<{name}>"
    finally:
        events.append(f"exit {name}")


with span("x") as got:
    check(got == "<x>", "the generator form yields the as value")
check(events == ["enter x", "exit x"], "code after yield runs at exit, exactly once")
cm = span("y")
with cm:
    pass
try:
    with cm:
        pass
except AttributeError as exc:
    check(
        "has no attribute 'args'" in str(exc),
        "generator managers are single use in 3.14",
    )
else:
    check(False, "generator managers are single use in 3.14")
```

listing 129: code before yield is enter, after yield is exit, and reuse is refused (source: python/samples/src/Ch10/ctxmgr.py, lines 75-102)

Everything before `yield` runs at entry, the yielded value binds to `as`, and everything after runs at exit, in a `try`/`finally` so exceptions run it too, lines 87-89. The manager object is single use, and 3.14 makes the second entry an `AttributeError`, lines 90-102, call the function again for each `with`. The toolkit rounds out the common shapes:

```
with contextlib.suppress(KeyError):
    {"a": 1}["b"]
check(True, "suppress eats only the listed types")
try:
    with contextlib.suppress(KeyError):
        raise ValueError("v")
except ValueError:
    check(True, "anything else still travels")
else:
    check(False, "anything else still travels")
```

listing 130: suppress, and the re-raise discipline it encodes (source: python/samples/src/Ch10/ctxmgr.py, lines 103-112)

`contextlib.suppress` swallows exactly the listed types and lets everything else travel, which is the re-raise discipline stated as a library: never catch broadly and quietly, catch narrowly or not at all. `closing` calls `close()` on the way out for objects that are not context managers, sample lines 115-126. `ExitStack` is the composition tool, lines 127-146: callbacks run last in first out, `pop_all` detaches the stack so an outer manager can finish the job later, and `enter_context` adopts other managers into the same unwind.

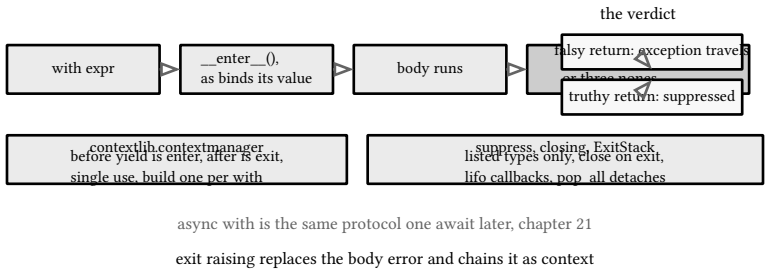


figure 40: the with statement, entry to verdict

VERIFY: 53 checks behind this chapter

`tree.py` pins the tree and both chaining forms, 12 checks, `trysem.py` block order and the swallowing finally, 11, `groups.py` the group structure and `except*` semantics with grammar refusals, 13, and `ctxmgr.py` the `with` protocol and the toolkit, 17. Two drafts went red: exception equality is identity, so the group checks compare types and args, and the module-level `return` in `except*` probe reported the wrong syntax error, the function-scoped probe gives the real one.

sources: docs.python.org/3/library/exceptions.html, the hierarchy and `OSError` attributes, accessed 2026-09-12; docs.python.org/3/reference/compound_stmts.html, the `try` statement, `except*`, and the `with` statement, accessed 2026-09-12; peps.python.org/pep-0654, exception groups and `except*`, accessed 2026-09-12; docs.python.org/3/library/contextlib.html and docs.python.org/3/library/exceptiongroup.html, accessed 2026-09-12; every chaining flag, split result, warning text, and grammar message probed on this machine the same day. Sample behavior verified by `make verify-py`, 53 checks in chapter 10 of the samples suite.

11. modules, packages, and imports

Every file this book has run so far was one module named `__main__` and the import statement never appeared. This chapter is about everything the import statement does. A module is an object, a package is a module with a `__path__`, an import is a lookup in a cache followed, on a miss, by a walk through finders and loaders that ends with compiled bytecode in `__pycache__`. The same machinery decides whether a project layout works before it is installed, and the same cache is why circular imports fail loudly instead of quietly. The verification mode is the book's gate, book 8, ch 1 (python 3.14, toolchain: cpython 3.14 on windows): the three samples under `Ch11/` run under cpython 3.14 and print 31 `ok` lines in total, and every behavioral claim below is one of those checks or a sentence from a canonical page fetched 2026-09-12.

11.1. modules and packages

The reference fixes the object model first: "Python has only one type of module object, and all modules are of this type, regardless of whether the module is implemented in Python, C, or something else." What separates the species is data, not type. A plain module carries `__name__`, `__file__`, and a `__spec__` describing how it was found and loaded. A package is the same object plus `__path__`, and a regular package is a directory whose `__init__.py` body becomes the package module's body. The sample builds one such directory at runtime, inside a `tempfile.mkdtemp()` tree, and reaches into it by putting its parent on `sys.path`:

```
    """write a small regular package into root and return its parent."""
    pkg = root / "site" / "shapes"
    pkg.mkdir(parents=True)
    (pkg / "__init__.py").write_text(
        'MARK = "init-ran"\n'
        "from .circle import AREA as CIRCLE_AREA\n"
        '__all__ = ["MARK", "CIRCLE_AREA"]\n',
        encoding="utf-8",
    )
    (pkg / "circle.py").write_text(
        "import math\n\nAREA = math.pi * 4\n",
        encoding="utf-8",
    )
    return root / "site"
```

listing 131: a regular package assembled at runtime: the `__init__` body imports from a submodule and declares `__all__` (source: python/samples/src/Ch11/modules_basics.py, lines 20-33)

The `__init__` runs once, at first import, and what it imports becomes attributes of the package. The sample asserts the whole chain: the marker lands, the relative import inside `__init__` succeeds, and `shapes.circle` is reachable both as an attribute of the package and as an entry of `sys.modules` under its dotted name. The reference states the binding rule as a rule: "When a submodule is loaded using any mechanism (e.g. importlib APIs, the import or import-from statements, or built-in `__import__()`) a binding is placed in the parent module's namespace to the submodule object." That is why `import shapes.circle` in one module makes `shapes.circle` reachable everywhere the package object travels:

```
site = build_tree(Path(tempfile.mkdtemp()))
sys.path.insert(0, str(site))
try:
    import shapes  # noqa: E402  (path set above, inside the sample)

    check(1, "import ran the package __init__", shapes.MARK == "init-ran")

    import shapes as again

    check(2, "second import is the same cached object", shapes is again)
```

listing 132: first import runs `__init__` once, the second import returns the identical object (source: python/samples/src/Ch11/modules_basics.py, lines 36-45)

The identity check is the cache at work. The sys documentation describes the dictionary: “This is a dictionary that maps module names to modules which have already been loaded. This can be manipulated to force reloading of modules and other tricks.” Evicting both entries and importing again really does rebuild the object, which the sample checks, and which is also the honest limit of the trick: the reference warns that “deleting essential items from the dictionary may cause Python to fail.” The supported spelling of a reload in running code is `importlib.import_module`, which the sample uses after the eviction:

```
    del sys.modules["shapes"]
    del sys.modules["shapes.circle"]
    shapes2 = importlib.import_module("shapes")
    check(
        10,
        "cache evicted, import_module rebuilt the object",
        shapes2 is not shapes and shapes2.MARK == "init-ran",
    )
    check(
        11,
        "spec carries origin and a loader",
        shapes2.__spec__.origin.endswith("__init__.py")
        and shapes2.__spec__.loader is not None,
    )
```

listing 133: evicting the cache entries and rebuilding the package through importlib (source: python/samples/src/Ch11/modules_basics.py, lines 83-96)

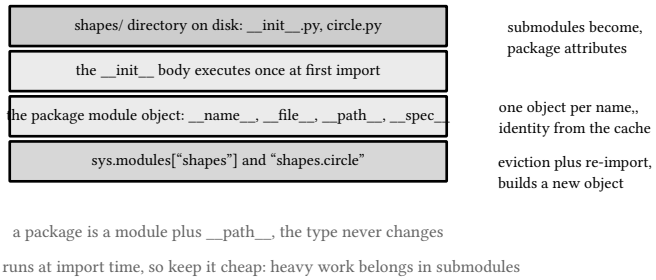


figure 41: a package as stacked layers: files on disk, one body that runs once, one object that persists

11.2. the import system

An import statement is a cache probe. On a hit in `sys.modules` the statement binds the cached object and nothing else happens, not even a stat of the file on disk. On a miss the machinery walks `sys.meta_path`, and the sample checks the order it finds there: the builtin importer, the frozen importer, then the path finder. The builtin entry explains a spec the sample reads off sys: a compiled-in module reports `origin == "built-in"` with `BuiltinImporter` as its loader, while a filesystem module like `json` reports the real path of its `__init__.py`. A name nothing can find

produces `find_spec` returning `None`, which is how the machinery says the name does not exist before the import statement turns that into `ModuleNotFoundError`:

```
# the meta path: builtin, frozen, then path finders
names = [f.__name__ for f in sys.meta_path]
check(
    4,
    "meta path order: builtin, frozen, path finder",
    names[:3] == ["BuiltinImporter", "FrozenImporter", "PathFinder"],
)
spec_sys = importlib.util.find_spec("sys")
check(
    5,
    "builtin module spec carries origin built-in",
    spec_sys.origin == "built-in" and spec_sys.loader.__name__ == "BuiltinImporter",
)
spec_json = importlib.util.find_spec("json")
check(
    6,
    "path finder resolves json to its source directory",
    spec_json.origin is not None and spec_json.origin.endswith("__init__.py"),
)
check(
    7,
    "a missing module finds no spec",
    importlib.util.find_spec("no_such_module_here") is None,
)
```

listing 134: the meta path order and three find_spec outcomes: builtin, path, missing (source: python/samples/src/Ch11/import_machinery.py, lines 67-90)

A finder returns a spec, and a loader consumes it. The full sequence is visible without any import statement at all through `importlib`: `spec_from_file_location` builds the spec for a file that is not on any path, `module_from_spec` allocates the module object, and `exec_module` runs the body. This is the programmatic form of the statement, and it is the honest answer to "how do i load a plugin from a path the user gave me":

```
# loading a file that is not on sys.path at all
lone = root / "lonely.py"
lone.write_text('VALUE = "loaded-by-spec"\n', encoding="utf-8")
spec = importlib.util.spec_from_file_location("lonely", lone)
mod = importlib.util.module_from_spec(spec)
spec.loader.exec_module(mod)
check(
    8,
    "spec_from_file_location + exec_module runs the file",
    mod.VALUE == "loaded-by-spec" and mod.__name__ == "lonely",
)
```

listing 135: loading a file that no finder would reach, spec, module, exec (source: python/samples/src/Ch11/import_machinery.py, lines 92-102)

The `__main__` guard exists because of how the same file can run. Run with `-m shapes.cli`, the module's `__name__` is `__main__` and its `__package__` is `shapes`, so the relative import inside it resolves. Run as a bare path, `python shapes\cli.py`, the package context is gone and the identical relative import dies with "attempted relative import with no known parent package". The sample runs both forms as children and asserts both outcomes:

```
r = run_child(["-m", "shapes.cli"], app)
out = r.stdout.splitlines()
check(
    1,
    "-m runs the module with __name__ __main__",
    r.returncode == 0 and out[0].split()[0] == "__main__",
)
check(
    2,
    "-m sets __package__ to the package, relative import works",
    out[0].split()[1] == "shapes" and out[1] == "guard",
)

r = run_child([str(pkg / "cli.py")], root)
check(
    3,
    "bare script path: no parent package, relative import dies",
    r.returncode != 0 and "no known parent package" in r.stderr,
)
```

listing 136: the same file both ways: -m carries the package, the bare path loses it (source: python/samples/src/Ch11/import_machinery.py, lines 47-65)

Circular imports break on the cache, not on the cycle. While `circular.a` is still executing, `sys.modules["circular.a"]` already holds the half-built module, so `from .a import A_VAL` inside `b` finds the module but not the name, and the error names the situation exactly: “cannot import name ‘A_VAL’ from partially initialized module”. The fix is not a deeper import, it is ordering: define the name before importing the other side, and import the module rather than the name so the attribute read happens after both sides exist:

```
# circular imports: import-the-name fails inside a half-built module
c1 = root / "circular"
c1.mkdir(parents=True)
(c1 / "__init__.py").write_text("", encoding="utf-8")
(c1 / "a.py").write_text("from . import b\nA_VAL = 'a-complete'\n", encoding="utf-8")
(c1 / "b.py").write_text(
    "from .a import A_VAL\nB_VAL = 'b-' + A_VAL\n", encoding="utf-8"
)
r = run_child(["-c", "import circular.a"], root)
check(
    9,
    "from .a import name dies on the partially initialized module",
    r.returncode != 0 and "partially initialized module" in r.stderr,
)

# the fix: define first, import the module object, read late
c2 = root / "circular2"
c2.mkdir()
(c2 / "__init__.py").write_text("", encoding="utf-8")
(c2 / "a.py").write_text("A_VAL = 'a-complete'\nfrom . import b\n", encoding="utf-8")
(c2 / "b.py").write_text("from . import a\nB_VAL = 'b-' + a.A_VAL\n", encoding="utf-8")
r = run_child(["-c", "import circular2.b; print(circular2.b.B_VAL)"], root)
check(
    10,
    "order plus module import untangles the cycle",
    r.returncode == 0 and r.stdout.strip() == "b-a-complete",
)
```

listing 137: the broken pair and the fixed pair, the difference is definition order plus module imports (source: python/samples/src/Ch11/import_machinery.py, lines 104-130)

PITFALL: the half-built module is already in the cache

The failing import does not leave `sys.modules` clean in the middle of the cycle. The parent under construction sits in the cache with whatever attributes it has defined so far, which is exactly why `from .a import X` fails while `from . import a` succeeds and `a.X` read later works. Retrying a broken circular import inside an `except` clause often appears to fix it because the second attempt finds the completed module. That is luck wearing a lab coat, not a design: reorder the definitions instead.

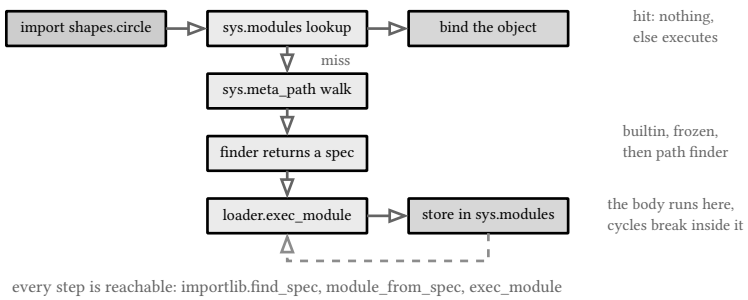


figure 42: one import statement as a state walk: cache probe, meta path, spec, execution, binding

11.3. project layout

The layout question is which directories the path finder will see and when. The flat layout puts the import packages beside `pyproject.toml` at the top of the repository. The src layout moves them down one level, and the packaging guide names the trade: “The src layout helps prevent accidental usage of the in-development copy of the code”, because “the Python interpreter includes the current working directory as the first item on the import path”, so

a test run from the repository root can import code the build would never ship. The cost is stated just as plainly: "The src layout requires installation of the project to be able to run its code, and the flat layout does not." This book's own tree is flat by necessity, the samples are run as files from their directory and never installed, and the gate pins that fact.

`pyproject.toml` is where a project states its build requirements and metadata, and this chapter treats it the way the cloud book treats terraform state: the file is real, documented, and never executed here. Nothing in this corpus builds a wheel, so every claim about what a build backend does with the file is a citation, not a check. Editable installs sit on the same boundary and earn the same sentence: "The src layout helps enforce that an editable installation is only able to import files that were meant to be importable", which is the documented reason to prefer it once installation enters the workflow at all.

Namespace packages are the layout question the filesystem answers on its own. The reference defines them: "A namespace package is a composite of various portions, where each portion contributes a subpackage to the parent package. Portions may reside in different locations on the file system." No `__init__.py` anywhere, `__file__` is absent, `__spec__.origin` is `None`, and `__path__` spans every portion. The sample assembles two portions under two separate `sys.path` roots and imports from both:

```
site1 = root / "site1"
site2 = root / "site2"
for part, mod, value in (
    (site1, "alpha", "one"),
    (site2, "beta", "two"),
):
    portion = part / "fused"
    portion.mkdir(parents=True)
    (portion / f"{mod}.py").write_text(f'VALUE = "{value}"\n', encoding="utf-8")

sys.path[:0] = [str(site1), str(site2)]
try:
    fused = importlib.import_module("fused")
    check(
        1,
        "a directory with no __init__ imports as a package",
        hasattr(fused, "__path__"),
    )
    check(
        2, "namespace package has no __file__", getattr(fused, "__file__", None) is None
    )
    check(3, "spec origin is None", fused.__spec__.origin is None)
    check(
        4,
        "loader is the NamespaceLoader",
        type(fused.__spec__.loader).__name__ == "NamespaceLoader",
    )
    portions = [Path(p).parent for p in fused.__path__]
    check(5, "__path__ spans both portions", site1 in portions and site2 in portions)
```

listing 138: two portions on two path entries, one package with no __init__ anywhere (source: python/samples/src/Ch11/namespace_pkgs.py, lines 20-48)

Precedence is the trap. The path finder scans `sys.path` in order, and a regular package on an earlier entry ends the search before any portion is collected. The sample plants a regular `fused` ahead of the namespace, evicts the cache, and the winner is the `__init__` version with a real `__file__`. Remove the regular package from the path and evict again, and the namespace comes back:

```
    # a regular package anywhere earlier on sys.path shadows the namespace
    site0 = root / "site0"
    regular = site0 / "fused"
    regular.mkdir(parents=True)
    (regular / "__init__.py").write_text('VALUE = "regular"\n', encoding="utf-8")
    sys.path.insert(0, str(site0))
    importlib.invalidate_caches()
    try:
        del sys.modules["fused"]
        winner = importlib.import_module("fused")
        check(
            7,
            "a regular package earlier on the path wins",
            winner.VALUE == "regular" and winner.__file__ is not None,
        )
    finally:
        sys.path.remove(str(site0))
        del sys.modules["fused"]

    importlib.invalidate_caches()
    restored = importlib.import_module("fused")
    restored_alpha = importlib.import_module("fused.alpha")
    check(
        8,
        "without the regular package the namespace comes back",
        getattr(restored, "__file__", None) is None and restored_alpha.VALUE == "one",
    )
```

listing 139: a regular package earlier on sys.path shadows the namespace, eviction restores it (source: python/samples/src/Ch11/namespace_pkgs.py, lines 58-84)

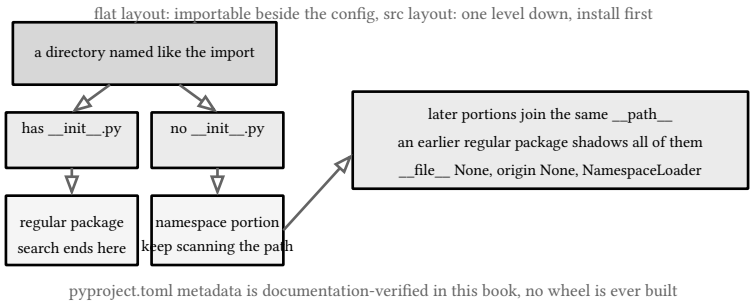


figure 43: layout as a decision: what the path finder does with a directory it is handed

11.4. environments and pins

The interpreter is the first pin. The gate resolves cpython through the py launcher, asserts the exact version before running anything, and refuses to use the 3.13 that also lives on this machine, which is why every sample in this book behaves identically wherever the tree is checked out. Above the interpreter sit the environments: a `venv` is a directory with its own `pyvenv.cfg` and its own site-packages, a place where `pip` can install without touching the per-user python, and the third-party chapters, fastapi, numpy, pandas, and pydantic, each assert the exact pinned version of their library inside the sample itself, so the run fails the moment the installed library drifts.

A requirements file is the portable form of that discipline. The pip documentation states what it is: “Requirements files serve as a list of items to be installed by pip, when using pip install.” The specifier grammar is shared with the command line, and the pin form this book uses wherever it pins at all is the double equals, ==, which the grammar reads as an exact version. The stdlib chapters, including this one, carry no requirements beyond the interpreter

because they import nothing outside it, and that absence is itself checked: every sample in `Ch11/` imports only the standard library.

NOTE: ruff formats, it does not verify

The gate runs ruff over the samples as a formatting and lint pass, not as a correctness claim. The `ok N` lines are the correctness claims, one per check, counted by the harness. Formatting and truth are different services: ruff reflowing a line can move a listing pin in this book, which is why listings name exact line ranges and the tree re-pins rather than hand-editing slices.

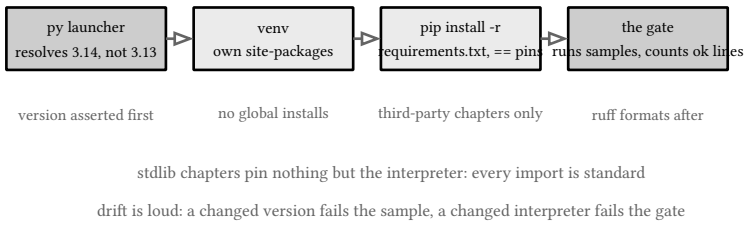


figure 44: the pin chain from interpreter to a green gate run

VERIFY: what a green run of this chapter proves

The three samples print 31 `ok` lines: module and package identity, the cache, spec and loader facts, the `-m` versus bare path split, both halves of the circular import pair, and the namespace package spanning, shadowing, and restoration. A green run proves those checks passed on cpython 3.14.7 on this machine, nothing wider.

sources: docs.python.org/3.14 reference/import (module objects, packages, namespace packages, submodule binding, meta path), library/sys (sys.modules), library/importlib (find_spec, spec_from_location, import_module), pep 420, and packaging.python.org src-layout-versus-flat-layout and requirements-file-format pages on pip.pypa.io, all accessed 2026-09-12. Sample behavior verified by `make verify-py`, 31 checks in chapter 11.

12. how cpython runs your code

A python file on disk and a running function are four different things separated by three machines: the tokenizer and parser, the compiler that emits bytecode, and the eval loop that executes it. This chapter walks the whole distance on cpython 3.14. Source becomes a code object through `compile()`, the code object becomes a frame with locals, globals, and cells when called, the frame feeds an instruction at a time to an eval loop that rewrites hot instructions in place, and above that loop sits a tier 2 jit that ships in this windows build but stays switched off until an environment variable says otherwise. The verification mode is the gate: the five samples under `Ch12/` print `51 ok` lines in total, and an `expect-dis.txt` sits beside them asserting nine stable bytecode facts, because bytecode claims should be checked the way terraform claims are, by a machine.

12.1. source to bytecode

`compile()` is the whole front end as a function: it takes source text, a filename to stamp into error messages, and a mode. The mode is a contract about shape. `exec` accepts statements and returns a module-level code object, `eval` accepts exactly one expression, and anything else is rejected by name. The return is a `types.CodeType` whose fields are the program in miniature, `co_name` and `co_qualname` for identity, `co_varnames` for the local slots in argument order, `co_consts` for the literals, `co_names` for the globals the body touches:

```
SRC = "def scale(x, factor=10):\n    return x * factor\n"

module_code = compile(SRC, "scale_demo.py", "exec")
check(
    1,
    "exec mode returns a module-level code object",
    isinstance(module_code, types.CodeType) and module_code.co_name == "<module>",
)
check(
    2,
    "co_filename is whatever string was compiled",
    module_code.co_filename == "scale_demo.py",
)

inner = inner_code(module_code)
check(
    3,
    "the function body is a nested code object in co_consts",
    inner.co_name == "scale",
)
check(4, "co_varname order: args then locals", inner.co_varnames == ("x", "factor"))
check(
    5,
    "the default rides the module consts, not the body's",
    module_code.co_consts[-1] == (10,) and inner.co_consts == (None,),
)
check(6, "co_names records global names the body touches", inner.co_names == ())
```

listing 140: compile in three modes, and the code object fields the body compiled to (source: python/samples/src/Ch12/bytecode.py, lines 25-51)

The function body is itself a nested code object inside the module's `co_consts`, and the sample's `inner_code` helper digs it out to read `co_varnames == ("x", "factor")` and the default living beside the code rather than inside it. `exec()` of the compiled module into a namespace builds the function and runs it, which is the whole import story of book 8, ch 11 (python 3.14, modules, packages, and imports) compressed into two calls: compile, then execute.

The `optimize` argument reaches the same switch `-O` flips at startup, and the sample pins its effect: with `optimize=2` the assert machinery, compare, conditional jump, and raise, is absent from the instruction stream entirely:

```
GUARD = 'def f(x):\n    assert x > 0, "bad"\n    return x\n'
dbg_inner = inner_code(compile(GUARD, "g.py", "exec"))
opt_inner = inner_code(compile(GUARD, "g.py", "exec", optimize=2))
check(
    10,
    "optimize=2 strips the assert machinery",
    "RAISE_VARARGS" in opnames(dbg_inner)
    and "RAISE_VARARGS" not in opnames(opt_inner)
    and opnames(opt_inner) == ["RESUME", "LOAD_FAST_BORROW", "RETURN_VALUE"],
)

check(
    11,
    "same source, same flags: co_code identical",
    compile(SRC, "s1.py", "exec").co_code == compile(SRC, "s2.py", "exec").co_code,
)
```

listing 141: optimize=2 compiles the assert away, the same code with and without it (source: python/samples/src/Ch12/bytecode.py, lines 69-84)

Imported modules get the same treatment with a cache in front. The first import of a `.py` file compiles it and writes the bytecode to `__pycache__` under a tag naming the interpreter, `cpython-314` here. The reference states the validation rule: “Before Python loads cached bytecode from a `.pyc` file, it checks whether the cache is up-to-date with the source `.py` file. By default, Python does this by storing the source’s last-modified timestamp and size in the cache file when writing it.” The sample imports a fresh module, finds the pyc, and reads the 16-byte header with `struct`: magic, flags, mtime, size, then asserts mtime and size equal the source’s:

```
root = Path(tempfile.mkdtemp())
(root / "fresh.py").write_text("VALUE = 'one'\n", encoding="utf-8")
sys.path.insert(0, str(root))
try:
    import fresh  # noqa: E402  (path set above, inside the sample)

    pyc = root / "__pycache__" / f"fresh.{sys.implementation.cache_tag}.pyc"
    check(
        1,
        "first import writes a tagged pyc beside the source",
        pyc.exists() and sys.implementation.cache_tag == "cpython-314",
    )

    header = pyc.read_bytes()[:16]
    magic, flags, mtime, size = struct.unpack("<4sIII", header)
    st = (root / "fresh.py").stat()
    check(2, "the pyc header stores the source mtime", mtime == int(st.st_mtime))
    check(3, "the pyc header stores the source size", size == st.st_size)
    check(4, "flags 0 means timestamp-based validation", flags == 0)
```

listing 142: the pyc path, and its header unpacked into magic, flags, mtime, size (source: python/samples/src/Ch12/pycache.py, lines 22-40)

The eviction sequence then exercises both halves of the rule. A rewrite that changes the size invalidates the pyc once the cache entry is dropped, and a rewrite that keeps the size and lands inside the same whole second does not, because both stored numbers still match. The mtime is compared to whole-second resolution, so a same-length edit inside one second is invisible to the check:

```
    # same size, same whole-second mtime: the stale pyc survives. the pin
    # copies the mtime the current pyc header stores, never the wall
    # clock, so a second boundary inside this window cannot break the setup
    (root / "fresh.py").write_text("VALUE = 'twp'\nMORE = 'x'\n", encoding="utf-8")
    pinned = struct.unpack("<4sIII", pyc.read_bytes()[:16])[2]
    os.utime(root / "fresh.py", (pinned, pinned))
    del sys.modules["fresh"]
    importlib.invalidate_caches()
    stale = importlib.import_module("fresh")
    check(7, "same size and second: the stale pyc still loads", stale.VALUE == "two")

    os.utime(root / "fresh.py", (pinned + 2, pinned + 2))
    del sys.modules["fresh"]
    importlib.invalidate_caches()
    forced = importlib.import_module("fresh")
    check(8, "a bumped mtime alone invalidates the pyc", forced.VALUE == "twp")
```

listing 143: the stale trap: same size, same second, the old bytecode still loads (source: python/samples/src/Ch12/pycache.py, lines 59-74)

PITFALL: the stale pyc is a real windows edit loop

The trap fires exactly where developers work fastest: save a small edit, rerun, see the old behavior. Same-length edits are rare by hand but routine for tools that rewrite files in place, and the one-second granularity means rapid successive saves can land in one bucket. The reference also documents the alternative, hash-based pyc files "which store a hash of the source file's contents rather than its metadata", written by `py_compile` with its invalidation modes. Default cpython runs do not use them.

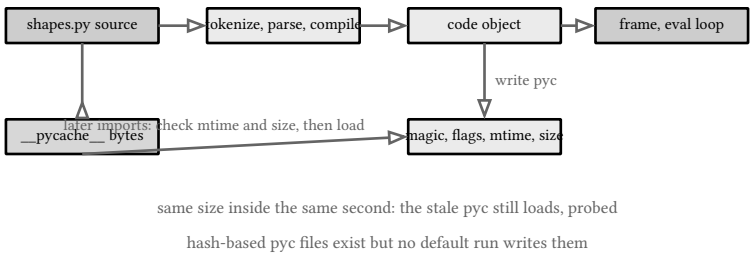


figure 45: source to running code: the two trips, first import and every later one

12.2. frames and cells

Executing a code object means building a frame around it, and the frame is where every name lookup begins. `sys._getframe()` returns the frame of the running call, and its three dictionaries are a chain: `f_locals` is the call's own slots, `f_globals` is the module dict the code was defined in, `f_builtins` is where `print` and friends resolve. A global read walks locals to globals to builtins and a global write touches only globals. The sample reads its own frame from inside a function and asserts each layer's identity:

```
def report(value):
    frame = sys._getframe()
    check(1, "_getframe is this call's own frame", frame.f_code.co_name == "report")
    check(2, "f_locals is the call's own variables", frame.f_locals.get("value") == 41)
    check(3, "f_globals is the module's dict", frame.f_globals is globals())
    check(4, "f_builtins resolves names like print", "print" in frame.f_builtins)
    check(5, "co_qualname survives rebinding", frame.f_code.co_qualname == "report")
    return value + 1


def caller_of_report():
    return report(41)


result = caller_of_report()
check(6, "frame data belongs to the running call", result == 42)
```

listing 144: one frame, its three dictionaries, and the caller one level up (source: python/samples/src/Ch12/frames.py, lines 17-32)

Closures are the same machinery pointed sideways. When `closure_factory` defines `closure_target`, the inner function's reference to `n` is not a copy, it is a cell, and `MAKE_CELL` allocates it in the factory's frame before the inner code object ever runs. The function object carries the cell in `__closure__`, names it in `co_freevars`, and reads it with `LOAD_DEREF`. The sample checks all of it, then writes through the cell and shows that every future call sees the new value:

```
def closure_factory(n):
    def closure_target(m):
        return m + n

    return closure_target


fn = closure_factory(7)
check(
    8,
    "the function object carries its cells",
    fn.__closure__ is not None and len(fn.__closure__) == 1,
)
check(9, "co_freevars names the captured variable", fn.__code__.co_freevars == ("n",))
check(
    10, "the cell holds the value at capture time", fn.__closure__[0].cell_contents == 7
)
check(11, "the cell reads through on every call", fn(1) == 8 and fn(2) == 9)

ins = [i.opname for i in dis.get_instructions(fn)]
check(
    12,
    "dis shows the cell read as LOAD_DEREF",
    "LOAD_DEREF" in ins and ins[-1] == "RETURN_VALUE",
)

fn.__closure__[0].cell_contents = 100
check(13, "writing the cell reaches every future call", fn(1) == 101)
```

listing 145: the cell: carried on the function, named in co_freevars, read with LOAD_DEREF (source: python/samples/src/Ch12/frames.py, lines 44-71)

```
outer_ins = [i.opname for i in dis.get_instructions(closure_factory)]
check(
    14,
    "the factory builds the cell and attaches it to the function",
    "MAKE_CELL" in outer_ins and "SET_FUNCTION_ATTRIBUTE" in outer_ins,
)
```

listing 146: the factory side: MAKE_CELL then SET_FUNCTION_ATTRIBUTE attach the cell (source: python/samples/src/Ch12/frames.py, lines 73-78)

This is the disassembly half of the closure argument in book 8, ch 6 (python 3.14, functions, scopes, and decorators), which made the same point about capture by reference with the tools of that chapter. Here the mechanism is the subject: a cell is one mutable slot shared by every function that closed over the variable, which is why a loop variable captured by five lambdas gives five views of one final value.

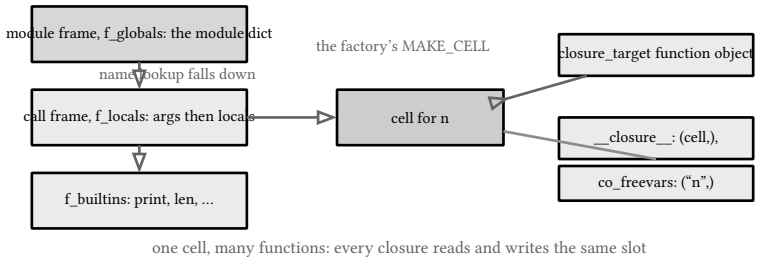


figure 46: a frame and its chain, with a cell bridging two frames

12.3. the eval loop and specialization

The eval loop reads the code object's instruction stream one entry at a time, dispatching on the opcode. In 3.14 that dispatch table is adaptive: a `BINARY_OP` that keeps seeing two ints stops being a generic arithmetic dispatch and becomes `BINARY_OP_ADD_INT` in place. This is pep 659's design, the specializing adaptive interpreter, and it is not a cache you invalidate, it is the bytecode of a live code object being rewritten as it warms up. The observable contract is in the `dis` signature: `get_instructions(x, *, first_line=None, show_caches=False, adaptive=False)`. The default view shows base opnames, the adaptive view shows what the code object currently runs, and the docs note that "Changed in version 3.13: The `show_caches` parameter is deprecated and has no effect":

```
3           RESUME_CHECK             0

4           LOAD_FAST_BORROW_LOAD_FAST_BORROW 1 (a, b)
            BINARY_OP_ADD_INT        0 (+)
            RETURN_VALUE
```

fragment, illustrative, not executed

That output is real, captured from this interpreter after 30 calls. The same function before any call disassembles to `RESUME`, `LOAD_FAST_BORROW_LOAD_FAST_BORROW`, `BINARY_OP`, `RETURN_VALUE`, and the sample asserts both states. Two more compile-time facts ride along: the doubled local load is itself a base opcode in 3.14, a fusion the compiler emits, not a warmup artifact, and small integer literals travel on `LOAD_SMALL_INT`:

```
base = [i.opname for i in dis.get_instructions(warm_target)]
check(
    1,
    "an uncalled function disassembles to base opnames",
    base
    == ["RESUME", "LOAD_FAST_BORROW_LOAD_FAST_BORROW", "BINARY_OP", "RETURN_VALUE"],
)
check(
    2,
    "RESUME opens and RETURN_VALUE closes every function",
    base[0] == "RESUME" and base[-1] == "RETURN_VALUE",
)
check(
    3,
    "the fused local load is a compile-time base opcode",
    "LOAD_FAST_BORROW" in dis.opmap
    and "LOAD_FAST_BORROW_LOAD_FAST_BORROW" in dis.opmap
    and "LOAD_SMALL_INT" in dis.opmap,
)


def small():
    return 7


check(
    4,
    "small int literals ride LOAD_SMALL_INT",
    [i.opname for i in dis.get_instructions(small)]
    == ["RESUME", "LOAD_SMALL_INT", "RETURN_VALUE"],
)
```

listing 147: the base stream, the warmed stream, and the opcodes the compiler itself emits (source: python/samples/src/Ch12/specialize.py, lines 21-51)

The discipline this forces on tooling is the reason the gate's `expect-dis.txt` exists. Base opnames like `RESUME`, `BINARY_OP`, and `RETURN_VALUE` are stable across runs, specialized names are a fact about one execution's history, so assertions name only the base forms. The chapter's file carries nine rows, among them `warm_target|BINARY_OP` and `small|LOAD_SMALL_INT`, each verified by compiling the sample source and matching the qualname's instruction stream.

Above the loop sits the monitoring plane. Pep 669 replaced trace functions with tool slots and event bit masks, `sys.monitoring` in the stdlib. The sample books the debugger slot, registers a callback for `PY_START`, and counts function entries. The event fires once per call and reads back as 0 once cleared. One machine fact came with it: on this build `import sys.monitoring` fails with "'sys' is not a package" because the module ships attached to `sys` itself, so the sample uses `from sys import monitoring`, which works:

```
# sys.monitoring: the pep 669 hook surface, counting function starts
seen = []
tool = monitoring.DEBUGGER_ID
monitoring.use_tool_id(tool, "specialize-sample")
try:
    monitoring.register_callback(
        tool,
        monitoring.events.PY_START,
        lambda code, offset: seen.append(code.co_name),
    )
    monitoring.set_events(tool, monitoring.events.PY_START)
    stable_signature(5)
    stable_signature(6)
finally:
    monitoring.set_events(tool, 0)
    monitoring.free_tool_id(tool)
check(
    10,
    "PY_START fired once per stable_signature call",
    seen.count("stable_signature") == 2,
)
check(
    11,
    "events read back as 0 once cleared",
    monitoring.get_events(monitoring.DEBUGGER_ID) == 0,
)
```

listing 148: pep 669 in use: book a tool id, register for PY_START, clear events (source: python/samples/src/Ch12/specialize.py, lines 105-130)

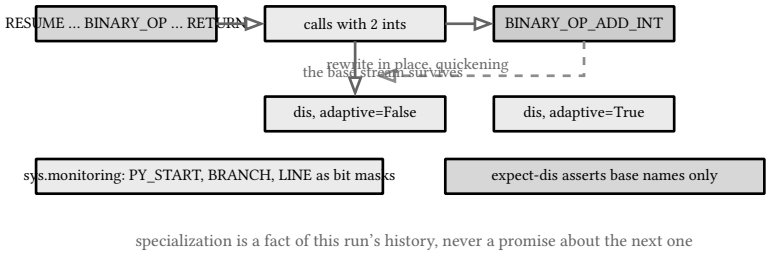


figure 47: tier 1 dispatch and its adaptive rewrites, with monitoring watching from the side

12.4. tier 2 and the jit

The whatsnew page for 3.14 states the release decision plainly: "The official macOS and Windows release binaries now include an experimental just-in-time (JIT) compiler. Although it is not recommended for production use, it can be tested by setting `PYTHON_JIT=1` as an environment variable." The same paragraph supplies the introspection: "a set of introspection functions has been provided in the `sys._jit` namespace. `sys._jit.is_available()` can be used to determine if the current executable supports JIT compilation, while `sys._jit.is_enabled()` can be used to tell if JIT compilation has been enabled for the current process."

Every one of those sentences checks on this machine, and the sample is those checks. `sys._jit` exists with exactly `is_active`, `is_available`, and `is_enabled`. `is_available()` is true on this build. `is_enabled()` is false in a default run and follows `PYTHON_JIT` when set, in both states the sample computes the expectation from the environment and asserts the agreement:

```
jit = getattr(sys, "_jit", None)
check(1, "the 3.14 windows build ships sys._jit", jit is not None)
check(
    2,
    "the introspection surface is the three functions",
    jit is not None
    and sorted(n for n in dir(jit) if not n.startswith("_"))
    == ["is_active", "is_available", "is_enabled"],
)

available = jit.is_available()
check(3, "the jit is compiled into this interpreter", available)

enabled = jit.is_enabled()
flag = os.environ.get("PYTHON_JIT")
expected = flag not in (None, "", "0")
check(4, "is_enabled follows the PYTHON_JIT environment variable", enabled == expected)
```

listing 149: the jit surface: shipped, off by default, switched by an environment variable (source: python/samples/src/Ch12/jit.py, lines 17-33)

WARNING: observed, not claimed as speed

The whatsnew page grades the jit itself: “The JIT is at an early stage and still in active development. As such, the typical performance impact of enabling it can range from 10% slower to 20% faster, depending on workload.” This chapter accordingly makes no timing claims, runs no benchmarks, and asserts nothing about `is_active`. The checks say the compiler is present and switchable on this interpreter. That is the whole claim, and it is the right size for a book whose evidence is a gate output, not a stopwatch.

The boundary below the jit is equally explicit. Tier 1’s adaptive specialization is on in every run and observable through `dis`, tier 2 exists in the binary and waits for an opt-in, and the layers above it, real threads and the gil, or processes that sidestep it, are book 8, ch 17 (python 3.14, threads and the gil) and book 8, ch 18 (python 3.14, multiprocessing and process patterns).

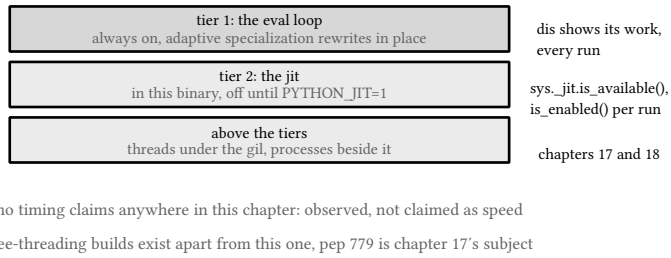


figure 48: the execution tiers of this build: what runs by default and what waits

VERIFY: what a green run of this chapter proves

The five samples print 51 `ok` lines: compile and code object fields, the pyc header and both invalidation outcomes, frame and cell structure, base versus adaptive bytecode, the monitoring event count, and the jit’s presence and switch state. The `expect-dis.txt` file adds nine bytecode assertions that recompile the sample sources to check. All of it ran on cpython 3.14.7 on this machine, and nothing here claims anything about another build’s flags.

sources: docs.python.org/3.14 library/dis (get_instructions signature, show_caches deprecation), library/sys.monitoring, whatsnew/3.14 (binary releases for the experimental just-in-time compiler), reference/import (cached bytecode invalidation), and peps.python.org pep 659, pep 744, all accessed 2026-09-12. Sample behavior verified by `make verify-py`, 51 checks in chapter 12.

13. pymalloc: arenas, freelists, and tracemalloc

Every object this book has built so far, from the tuples of chapter 3 to the arrays of book 8, ch 24 (python 3.14, numpy), came out of the same private heap, and this chapter opens it. The C API manual states the design in one sentence: "Python has a **pymalloc** allocator optimized for small objects (smaller or equal to 512 bytes) with a short lifetime", and "It falls back to `PyMem_RawMalloc()` and `PyMem_RawRealloc()` for allocations larger than 512 bytes." The plan of the chapter is to hold that sentence against the binary: the allocator prints its own census under `PYTHONMALLOCSTATS`, `id()` reveals the size classes, and `tracemalloc` attributes what survives. Everything below is either quoted from the 3.14 manual, fetched 2026-09-13, or measured on this machine under the gate interpreter, cpython 3.14.7, the same day. The chapter owns 39 ok lines across six samples, and the particle kernel it closes with is the same kernel book 8, ch 14 (python 3.14, profiling: cprofile, sys.monitoring, and p99) measures at full scale.

13.1. why a private heap

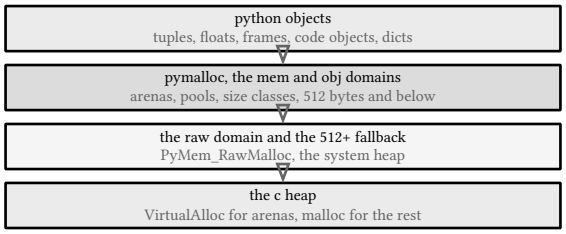
The default C heap is a general servant: it serves every allocation of every size with the same bookkeeping, and it cannot know that this program allocates and frees millions of 24-byte floats. The manual assigns the layers plainly: pymalloc "is the default allocator of the `PYMEM_DOMAIN_MEM` (ex: `PyMem_Malloc()`) and `PYMEM_DOMAIN_OBJ` (ex: `PyObject_Malloc()`) domains", while the raw domain keeps the system allocators, `malloc()` and `free()`, in every configuration. Object headers, interned strings, frames, and code objects all come through the OBJ domain, so the small-object path is the hot path of the whole interpreter, which is why it gets a private geometry instead of a general one.

```
if sys.version_info[:3] != (3, 14, 7):
    raise SystemExit(f"FAIL pinned interpreter is 3.14.7, got {sys.version}")


def census():
    """PYTHONMALLOCSTATS must be set before interpreter start, so a child
    interpreter carries the probe; the dumps land on stderr, the one at
    shutdown is the full one."""
    env = dict(os.environ, PYTHONMALLOCSTATS="1")
    proc = subprocess.run(
        [sys.executable, "-c", "xs = [i + 0.5 for i in range(20000)]"],
        capture_output=True,
        text=True,
        env=env,
        check=True,
    )
    return proc.stderr + proc.stdout
```

listing 150: the interpreter pin, then the census probe: the env var must exist before interpreter start, so a child carries it (source: python/samples/src/Ch13/arenas.py, lines 8-24)

The two tools of this chapter meet in that listing. The environment variable `PYTHONMALLOCSTATS` makes the allocator print its statistics "every time a new pymalloc object arena is created, and on shutdown", and because environment variables are read before Python code runs, a child interpreter has to carry it. `id()` is the other tool: it returns "the identity of an object", and in CPython that identity is the object's address, so the distance between two live objects is the distance between their blocks.



the small path is the interpreter's hot path: millions of 24-byte floats a day

figure 49: the allocator stack: objects ride the mem and obj domains into pymalloc, raw memory and everything above 512 bytes ride the c heap

13.2. arenas, pools, and blocks

The manual fixes the arena size for this platform: "It uses memory mappings called 'arenas' with a fixed size of either 256 KiB on 32-bit platforms or 1 MiB on 64-bit platforms", and on Windows those mappings come from "VirtualAlloc() and VirtualFree()". This is a 64-bit build, so arenas here are 1 MiB, and the binary says so itself in its census, committed as the dated capture behind this page:

```
Small block threshold = 512, in 32 size classes.

class   size   num pools   blocks in use  avail blocks
-----   ----   ---------   -------------  ------------
    0     16           1              31           990
    1     32          22           11220             0
    2     48           8            2503           217
    3     64          25            6206           169
    4     80          20            3952           128
    5     96           3             476            34
```

listing 151: the allocator's own banner and the head of its class table, captured 2026-09-13 under `PYTHONMALLOCSTATS=1` (source: python/samples/bench/captures/2026-09-13-mallocstats.txt, lines 1-10)

```


# arenas allocated total           =                  2
# arenas reclaimed                 =                  0
# arenas highwater mark            =                  2
# arenas allocated current         =                  2
2 arenas * 1048576 bytes/arena     =          2,097,152

# bytes in allocated blocks        =          1,808,192
# bytes in available blocks        =            273,968
0 unused pools * 16384 bytes       =                  0
# bytes lost to pool headers       =              6,144
# bytes lost to quantization       =              8,848
# bytes lost to arena alignment    =                  0
Total                              =          2,097,152
```

listing 152: the tail of the class table and the arena totals: 1048576 bytes an arena, 16384 a pool, 8,848 lost to quantization (source: python/samples/bench/captures/2026-09-13-mallocstats.txt, lines 37-50)

The census carries three numbers this chapter pins: 1048576 bytes per arena, 16384 bytes per pool, and the class table running 16 to 512 in 16-byte steps. The arithmetic those imply is asserted in the sample: 64 pools per arena, and after the pool's own 48-byte header, 510 blocks of the 32-byte class or 1021 of the 16-byte class. The quantization line is the tail of the pools: a 32-class pool spends its 16,384 bytes as 48 of header plus 510 blocks at 32, and the 16 left over cannot form one more block. Summed over every pool in the census those tails come to exactly 8,848, the arena total 2,097,152 minus 1,808,192 in allocated blocks, 273,968 in available ones, and 6,144 of pool

headers. The allocator never records a request size, so the census cannot report body-versus-block rounding at all: a 24-byte float in a 32-byte block pays its 8 lost bytes inside the allocated column, 160,000 of them for 20000 floats.

The `id()` measurements agree with the census to the byte. Two thousand held floats sit on a 32-byte stride, every one of them at an address that is 16 modulo 32, which is the pool header plus the class stride. Sixty thousand held floats, 1.44 MB of payload in 1.92 MB of blocks, hit all 510 distinct slot offsets a 32-class pool owns and span at least two arenas, four to five in the runs behind this page, because an arena holds 32640 floats of this class and the rest spills into the next mapping.

```
    pools_per_arena = arena // pool
    blocks32, blocks16 = (pool - header) // 32, (pool - header) // 16
    if pools_per_arena != 64 or blocks32 != 510 or blocks16 != 1021:
        raise SystemExit("FAIL the arena/pool/header arithmetic did not hold")
    print("ok 4 arithmetic: 64 pools per arena, 510 blocks of 32, 1021 of 16")

    tables = class_tables(stats)
    wanted = list(range(16, 513, 16))
    if not tables or any(t != wanted for t in tables):
        raise SystemExit(f"FAIL class tables mismatch, {[len(t) for t in tables]}")
    print(f"ok 5 classes step 16 bytes from 16 to 512, {len(tables)} dumps agree")

    ids = held_float_ids(2000)
    residues = set(i % 32 for i in ids)
    dominant = Counter(ids[k + 1] - ids[k] for k in range(len(ids) - 1)).most_common(1)[
        0
    ][0]
```

listing 153: the census arithmetic and the stride check: 64 pools, 510 blocks, every id 16 mod 32 (source: python/samples/src/Ch13/arenas.py, lines 60-76)

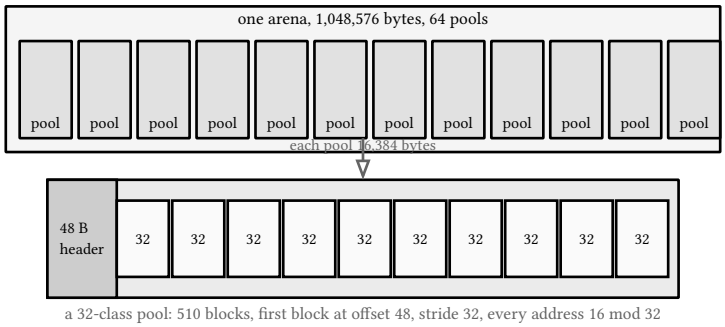


figure 50: one arena of 64 pools: each pool owns one class, the header costs 48 bytes, and blocks of the 32 class fill the rest at a fixed stride

13.3. freelists and the 512-byte boundary

A freed small object does not return its memory to the operating system or even to the arena. Its block goes back to its pool, and the next allocation of the same size class takes the cheapest block on the free list. The sample holds that to five trials per shape: a freed 4-tuple returns as the next 4-tuple, a freed float as the next float, a freed short list as the next short list, five for five each, and the extreme form is the one-line classic, `id([]) == id([])`, true here because the dropped empty list is the next free 64-class block when the second empty list asks for one.

```python
def reuse_trials(build, trials=5):
    """Free one object, immediately build a same-shape replacement."""
    outcomes = []
    for t in range(trials):
        a = build(t)
        where = id(a)
        del a
        b = build(t + 100)
        outcomes.append(id(b) == where)
        del b
    return tuple(outcomes)
```

listing 154: the reuse probe: free one object, immediately build a same-shape replacement, five trials (source: python/samples/src/Ch13/freelists.py, lines 11-22)

The boundary the manual draws is 512 bytes, and the dispatch is a size class lookup, not a search: requests of 16 through 512 bytes in 16-byte steps, 32 classes, round up to their class and take a block from that class's pool. Above 512 nothing rounds up at all, the request goes to `PyMem_RawMalloc`, and the census confirms the edge: its table stops at 512 and a 513-byte request would buy a raw block sized to the byte. The small-int cache rides the same cheap-reuse logic and stops at a fixed line: `int("256") is int("256")` holds, `int("257") is int("257")` does not, the boundary chapter 3 taught as the cached range minus 5 through 256.

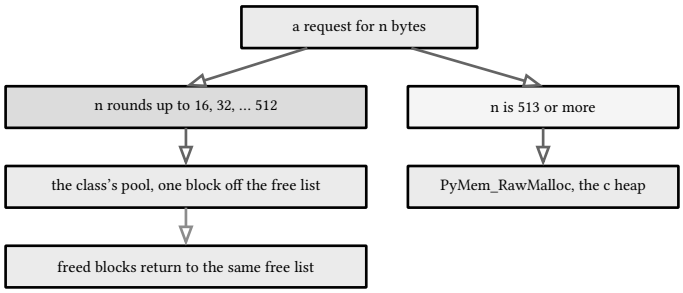


figure 51: the dispatch: a request rounds up to its class through 512, above it the raw heap takes the request unsized

13.4. what getsizeof measures

`sys.getsizeof` answers a narrow question: the storage of the object itself, header included, the objects it points to excluded. That makes it a ladder reader. The builtin containers grow in jumps rather than per element, and the jumps are the allocator's neighbors showing through: a list owns 56 bytes empty and grows by whole over-allocated segments, 72, 88, 120, 184, 312, 568, 1080, 2104 as it passes 1, 2, 4, 8, 16, 32, 64, 128, 256 elements, a dict jumps 64, 224, 352, 632, 1168, 2264, 4688 on the same ladder, and a set starts expensive, 216 bytes empty, because its table exists before its first member:

```python
def ladder(build, sizes):
    return tuple(sys.getsizeof(build(n)) for n in sizes)


def main():
    lists = ladder(lambda n: list(range(n)), (0, 1, 2, 4, 8, 16, 32, 64, 128, 256))
    if lists != (56, 72, 72, 88, 120, 184, 312, 568, 1080, 2104):
        raise SystemExit(f"FAIL list ladder {lists}")
    print(f"ok 1 list ladder {lists[:5]}..., 8 bytes a slot with over-allocation")
```

listing 155: the ladder reader: one builder, one size list, pinned per container (source: python/samples/src/Ch13/getsizeof.py, lines 9-18)

```
    if dicts != (64, 224, 224, 352, 632, 1168, 2264, 4688):
        raise SystemExit(f"FAIL dict ladder {dicts}")
    print(f"ok 2 dict ladder {dicts}, the compact layout doubles in jumps")

    sets = ladder(lambda n: set(range(n)), (0, 1, 4, 8, 16, 32, 64, 128))
    if sets != (216, 216, 216, 728, 728, 2264, 2264, 8408):
        raise SystemExit(f"FAIL set ladder {sets}")
    print(f"ok 3 set ladder {sets}, an empty set already owns 216 bytes")

    tuples = ladder(lambda n: tuple(range(n)), (0, 1, 2, 4, 8, 16, 32, 64, 128))
    if tuples != (48, 56, 64, 80, 112, 176, 304, 560, 1072):
        raise SystemExit(f"FAIL tuple ladder {tuples}")
    print(f"ok 4 tuple ladder {tuples}, exact 8-byte growth, no over-allocation")
```

listing 156: the pinned ladders: list, dict, set, tuple, each jump asserted exactly (source: python/samples/src/Ch13/getsizeof.py, lines 20-32)

The scalars carry their own arithmetic: an empty string is 41 bytes and each ASCII character adds exactly one, an int is 28 bytes at one digit and 36 at `2**60` because the digits live in a 30-bit limb array, and an empty bytes is 33. The numpy contrast is the point of the ladder: an `(8, 10000)` float64 array reports 640128 bytes, its 640000-byte buffer plus a 128-byte ndarray header, 64 bytes per particle, while the tuple representation of the same particles costs 112 bytes for the 8-slot tuple plus 8 objects of 24 bytes, 304 bytes a particle before the list slot that points at it, 4.75 times the buffer and 4.875 with the slot counted.

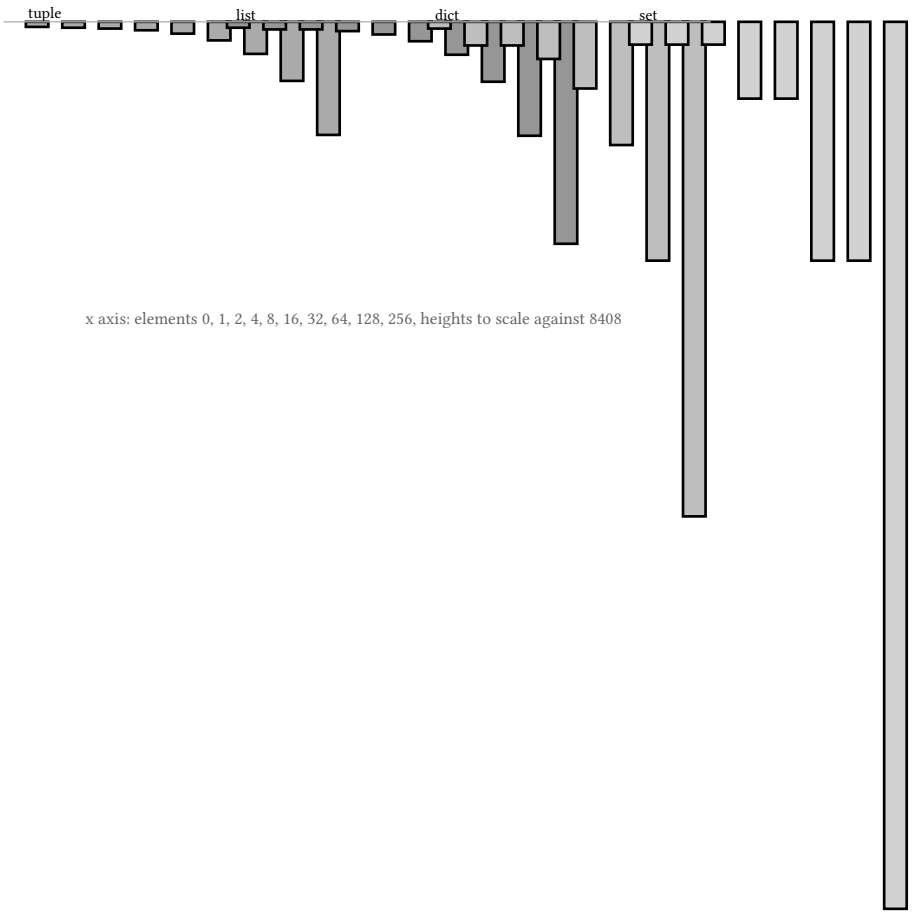


figure 52: the ladders side by side: tuple grows exactly, list over-allocates ahead of need, dict and set jump when their tables double

13.5. tracemalloc attribution

`getsizeof` reads one object at a time. "The tracemalloc module is a debug tool to trace memory blocks allocated by Python", and its snapshots answer the aggregate questions: where allocations were born and what survived. The manual defines the knobs exactly, tracing "collected tracebacks of traces will be limited to **nframe** frames", one frame by default, and warns that "Storing more frames increases the memory and CPU overhead". The statistics call groups by `key_type`, and for the lineno grouping "The result is sorted from the biggest to the smallest by: `Statistic.size`, `Statistic.count` and then by `Statistic.traceback`", so index zero is the top line by bytes, not a matter of luck.

```
def workload():
    rows = []
    for i in range(2000):
        rows.append({"id": i, "vals": [float(i) for _ in range(4)]})
    return rows
```

listing 157: the workload: one dict per row, one list per row, two distinct allocation sites (source: python/samples/src/Ch13/tracemalloc_demo.py, lines 9-15)

```
    rows = workload()
    snapshot = tracemalloc.take_snapshot()
    top = snapshot.statistics("lineno")[0]
    current, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()

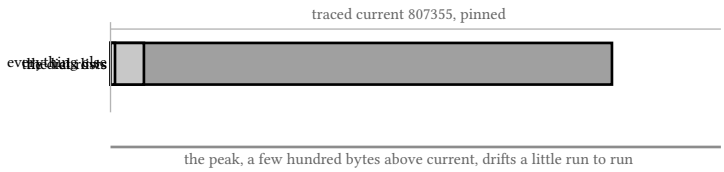
    if (
        top.traceback[0].lineno ≠ THE_VALS_LINE
        or top.traceback[0].filename ≠ __file__
    ):
        raise SystemExit(
            f"FAIL top line is {top.traceback[0].lineno}, wanted {THE_VALS_LINE}"
        )
    print(f"ok 2 the top stat names line {THE_VALS_LINE}, the vals list build")

    if (top.size, top.count) ≠ (750768, 15980):
        raise SystemExit(f"FAIL top size {top.size} count {top.count} moved")
    print(f"ok 3 pinned: top line holds {top.size} bytes in {top.count} blocks")

    if current ≠ 807355 or not current ≤ peak < 900000:
        raise SystemExit(f"FAIL current {current} peak {peak} moved")
    print(
```

listing 158: snapshot, top stat, current and peak: the vals line owns the ledger and the pins hold it (source: python/samples/src/Ch13/tracemalloc_demo.py, lines 24-45)

On the 2000-row workload the attribution is lopsided and pinned: the list-build line owns 750768 bytes across 15980 blocks, and the traced current after the build is 807355 bytes, both exact run over run, while the peak sits a few hundred bytes above 807000 and drifts a few dozen bytes between runs, because a high-water mark records whatever transient was alive at the crest. The manual's companion call, `reset_peak()`, exists for exactly that shape: it "resets the peak size of memory blocks traced by the tracemalloc module" so a second phase reports its own crest instead of inheriting the first. At kernel scale the same tool reads the in-place claim directly: traced current lands at 64777603 bytes after building the 1m-particle buffer, and after 16 steps of the chapter kernel it sits at 80777795 with a peak of 80778259, the two 8 MB scratch rows accounting for the difference to the byte and nothing else growing.



one frame by default: the traceback names the line that allocated, not the call path

reset_peak() gives a second phase its own crest instead of the first phase's

figure 53: the attribution ledger: two allocation sites, one owns the bytes, and the peak marks the crest of the transients

13.6. pythonmalloc and the mimalloc option

The allocator is selectable before the interpreter starts, through the `PYTHONMALLOC` environment variable, and the manual's table names the defaults per build: `pymalloc` for a release build, `pymalloc_debug` for a debug build, `malloc` where pymalloc is compiled out, `malloc_debug` for that build's debug flavor, and `mimalloc` for the free-threaded build. The debug hooks are byte-level and worth quoting: newly allocated memory is filled with `0xCD`, freed memory with `0xDD`, and the blocks are surrounded by forbidden bytes `0xFD` that turn an underrun or overrun into a loud crash instead of silent corruption.

The sample measures the dispatch on this binary. The gate never sets the variable, so the default rules. An unknown value is fatal before any user code: the child exits 1 at preinit with "unknown allocator". All three of `malloc`, `pymalloc`, and `mimalloc` start cleanly on this 3.14.7 Windows build, mimalloc included, because 3.14 grew the mimalloc option for the standard build. What this machine does not have is the free-threaded interpreter where mimalloc stops being an option: "In the free-threaded build, mimalloc is the default and required allocator", running "per-thread mimalloc heaps". That build is chapter 17′s boundary and book 8, ch 14 (python 3.14, profiling: cprofile, sys.monitoring, and p99)'s sidebar, and nothing in this chapter pretends to measure it.

```
def child_with(malloc):
    env = dict(os.environ, PYTHONMALLOC=malloc)
    proc = subprocess.run(
        [sys.executable, "-c", "import sys; print(sys.version_info[:3])"],
        capture_output=True,
        text=True,
        env=env,
    )
    return proc.returncode, (proc.stdout + proc.stderr).strip()
```

listing 159: the probe: one child per value, the exit code and the startup text are the measurement (source: python/samples/src/Ch13/mallocenv.py, lines 9-18)

pymalloc	the release default, this book's allocator
pymalloc_debug	release allocator plus the 0xCD, 0xDD, 0xFD hooks
malloc, malloc_debug	pymalloc compiled out, asan builds
mimalloc	free-threaded default and required, per-thread heaps
anything else	fatal at preinit, unknown allocator, exit 1

all three of malloc, pymalloc, mimalloc measured starting on this 3.14.7 binary, 2026-09-13

no free-threaded interpreter on this machine, so mimalloc under contention stays unmeasured

figure 54: the env var table: which name a build starts under, what the debug hooks fill, and where this book stands

13.7. refcounts and the collector recap

The geometry above explains why the chapter's objects are cheap to free: a decref to zero hands the block to a free list and the next same-shape build takes it back, measured five for five in the freelist sample. Reference cycles are

the case that path cannot solve, and the generational collector behind them belongs to chapter 3, which pinned its thresholds on this interpreter. This chapter only re-asserts the recap so the memory story stands complete on one page: the thresholds read `(2000, 10, 10)`, generation zero trips after 2000 net allocations minus deallocations, and the churn of 3000 built-and- dropped 4-tuples rides the freelists with at most a generation zero collection along the way, counted from `gc.get_stats()` before and after.

```
    raise SystemExit(f"FAIL tuple ladder {sizes}")
print("ok 6 tuple ladder 48 to 80 bytes: every size lands on a 16-byte class")

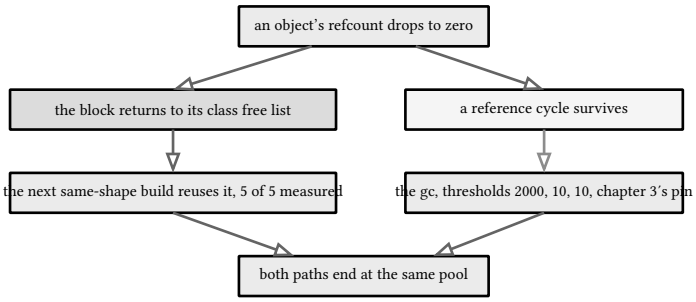
classes = range(16, 513, 16)
if len(classes) != 32 or sys.getsizeof(mk4(0)) != 80:
    raise SystemExit("FAIL class arithmetic")
print("ok 7 16 through 512 in 16-byte steps: 32 classes, the 4-tuple rides 80")

import gc

if gc.get_threshold() != (2000, 10, 10):
    raise SystemExit(f"FAIL gc thresholds {gc.get_threshold()}")
print("ok 8 recap from chapter 3: gc thresholds (2000, 10, 10) unchanged")

before = gc.get_stats()[0]["collections"]
churn = [mk4(t) for t in range(3000)]
del churn
gc.collect()
after = gc.get_stats()[0]["collections"]
if after < before:
    raise SystemExit("FAIL gen0 collection count went backwards")
print(f"ok 9 freelists absorb the churn, gen0 collections so far: {after}")
```

listing 160: the recap checks: thresholds unchanged from chapter 3, the churn absorbed, gen0 counted (source: python/samples/src/Ch13/freelists.py, lines 50-71)



the freelists absorb churn so the collector only owes the cycles

figure 55: the two reclamation paths: refcount zero returns the block now, a cycle waits for the collector, and both end at the same free list

13.8. the particle kernel: numpy versus tuples

The chapter closes on the workload the whole memory story serves, N particles of 8 float64 fields advanced by a pure arithmetic step, 16 steps, `x += vx * dt` then `vx += (q / m) * e * dt` under a constant field. The two representations of the same physics could not be more different for the allocator. The array path is one 64-bytes-a-particle buffer plus two scratch rows that the step reuses through `out=` parameters, so after the build, tracemalloc sees no growth at all. The tuple path is the ladder chapter of this book in a loop: every step rebuilds an 8-slot tuple per particle, 112 bytes plus 8 floats, and every rebuilt tuple is a freed tuple's block coming back off the free list.

```
def make_soa(n, seed=SEED):
    rng = np.random.Generator(np.random.PCG64(seed))
    out = np.empty((8, n))
    for f in range(8):
        lo, hi = RANGES[f]
        out[f] = rng.uniform(lo, hi, n)
    return out


def step_soa(p, scratch=None, scratch2=None):
    """Two scratch rows keep the step allocation-free, so the same code
    serves the 6.4 GB one-off without a doubling temporary."""
    if scratch is None:
        scratch = np.empty(p.shape[1])
    if scratch2 is None:
        scratch2 = np.empty(p.shape[1])
    np.divide(p[7], p[6], out=scratch)
    for j, e in ((0, EX), (1, EY), (2, EZ)):
        np.multiply(p[3 + j], DT, out=scratch2)
        np.add(p[j], scratch2, out=p[j])
        np.multiply(scratch, e * DT, out=scratch2)
        np.add(p[3 + j], scratch2, out=p[3 + j])
```

listing 161: the array path: one buffer, per-field rows, the step in place through two scratch rows (source: python/samples/src/Ch13/particle.py, lines 23-45)

```
def make_tuples(n, seed=SEED):
    rng = random.Random(seed)
    return [tuple(rng.uniform(lo, hi) for lo, hi in RANGES) for _ in range(n)]


def step_tuples(parts):
    # tuples are immutable: every step rebuilds the 8-slot tuple per particle
    for i, (x, y, z, vx, vy, vz, m, q) in enumerate(parts):
        qm = q / m
        x += vx * DT
        y += vy * DT
        z += vz * DT
        vx += qm * EX * DT
        vy += qm * EY * DT
        vz += qm * EZ * DT
        parts[i] = (x, y, z, vx, vy, vz, m, q)
```

listing 162: the tuple path: the same math, a rebuilt 8-slot tuple per particle per step (source: python/samples/src/Ch13/particle.py, lines 47-63)

The seeding is stated per path and honest about the difference: the array path draws from `numpy.random.Generator(np.random.PCG64(42))`, the tuple path from `random.Random(42)`, the Mersenne twister, same ranges, different streams, so the two checksums are not comparable and the sample asserts their inequality as the record of that fact. The in-suite smoke holds both at 10000 particles with pinned checksums, −51.506064417 for the array path and 60.225451136 for the tuple path.

The measured ladder belongs to the opt-in bench, `samples/bench/particle_bench.py`, run alone on 2026-09-13, best of repeats per the timeit doctrine. The crossover scan, `crossover --repeat 11`, and the ladder run, committed as the dated capture:

```
$ python particle_bench.py crossover --repeat 11
       5 tuples
      10 tuples
      20 tuples
      30 tuples
      40 tuples
      50 soa
      60 soa
      80 soa
     120 soa
     200 soa
     400 soa
     800 soa
flip between: [50]
```

listing 163: the crossover scan: the tuple path holds the lead through 40 particles, the array path takes it at 50 and never gives it back (source: python/samples/bench/captures/2026-09-13-particle-ladder.txt, lines 1-15)

```
$ python particle_bench.py ladder
python 3.14.7, numpy 2.5.3, K=16
         N  soa ns/p/step  tuples ns/p/step ratio
       100          82.94            211.0     2.5x
           checksums: soa -0.126472, tuples -24.693018
      1000          10.24            209.6    20.5x
           checksums: soa -17.634328, tuples -27.986558
     10000           3.12            227.7    73.0x
           checksums: soa 2.743116, tuples 12.732906
    100000           4.50            238.0    53.0x
           checksums: soa -71.393418, tuples 213.411720
   1000000          11.48            345.7    30.1x
           checksums: soa -353.681062, tuples 1086.925971
  10000000          16.18            345.6    21.4x
           checksums: soa -2307.074348, tuples 3499.540079
```

listing 164: the ladder, checksum line per run: soa cost dives with N then rises with cache misses, the tuple cost stays flat (source: python/samples/bench/captures/2026-09-13-particle-ladder.txt, lines 16-30)

The tuple cost sits near 210 to 240 ns a particle a step from 100 particles through 100000, then climbs to 346 at a million and holds there: past the caches, every step walks boxed floats scattered across the heap, so the per-object interpreter work gains a memory tax. The array cost dives as N amortizes the ufunc dispatches, 82.9 ns at 100 particles, 3.12 at 10000, then rises as the buffer leaves the caches behind, 11.5 at a million, 16.2 at ten million. The flip lands between 40 and 50 particles: below it the fixed cost of a dozen ufunc dispatches outweighs the work, above it the vector loop wins and never gives the lead back, 73 times faster at the 10000-particle sweet spot. The 100 million one-off, 6.4 GB of SoA, is measured alone in book 8, ch 14 (python 3.14, profiling: cprofile, sys.monitoring, and p99) the tuple path is capped at ten million by footprint, 3.1 GB of boxed doubles, and one billion is math on paper, 64 GB, not a run this machine makes.

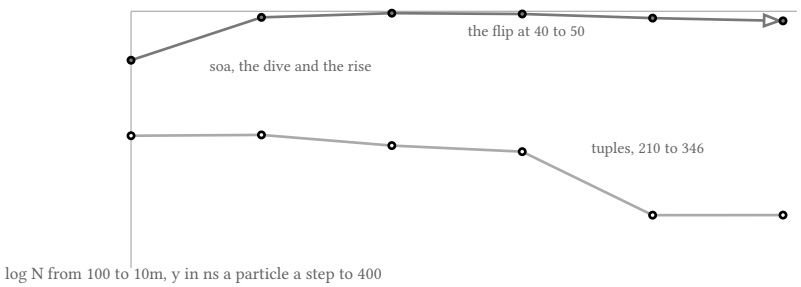


figure 56: the two cost curves against N: the tuple line near 210 to 240 ns until the caches give out, the array line diving under it by N of 50 and bottoming at 10000 before rising with cache misses

VERIFY: 39 checks, the pins asserted first

A scoped run, `pwsh tools/run-py-samples.ps1 -Chapter Ch13`, walks the six samples through the run and format legs and reports: 6 files, 39 checks, format clean. `arenas.py` carries the interpreter pin and contributes 7, the census banner, the arena and pool lines, the 64/510/1021 arithmetic, the class table, the 32-byte stride, and the 510-slot census. `freelists.py` adds 9, the five-trial reuse for tuples, floats, and lists, the `id([])` one-liner, the 256/257 cache boundary, the tuple ladder, the class arithmetic, and the gc recap with its gen0 count. `getsizeof.py` adds 7, the four container ladders, the scalar sizes, and the 64-against-304 footprint. `tracemalloc_demo.py` adds 5, the arm and disarm, the top-line attribution, and the pinned current and bounded peak. `mallocenv.py` adds 5, the unset default, the fatal unknown value, the three starting allocators, the 16-byte alignment, and the threshold edge. `particle.py` adds 6, the numpy 2.5.3 pin, the deterministic redraw, the two pinned checksums, their asserted inequality, and the buffer arithmetic. Every expected value was produced by this venv, then pinned.

sources: docs.python.org/3/c-api/memory.html (pymalloc and the 512 byte threshold, the arena sizes and VirtualAlloc, the domain defaults, the PYTHONMALLOC table, the 0xCD 0xDD 0xFD debug hooks, mimalloc as the free-threaded default, PYTHONMALLOCSTATS), docs.python.org/3/library/tracemalloc.html (the module purpose, the nframe limit and its overhead note, the statistics key_type table and its sort order, get_traced_memory and reset_peak), numpy.org/doc/2.5/reference/random/generator.html (the Generator interface behind the PCG64 seed), all accessed 2026-09-13. The `sys.getsizeof` ladder definitions, the `gc` thresholds, and the `random.Random` seeding live on pages this book already pins in chapters 1, 3, and 23, and the census capture, every id measurement, the reuse trials, and both bench runs are this machine's own, dated 2026-09-13. Sample behavior verified by `make verify-py`, 39 checks in chapter 13.

14. profiling: cprofile, sys.monitoring, and p99

Measurement is where this book stops quoting and starts counting. The interpreter chapter disclaimed timing on purpose, “no timing claims anywhere in this chapter”, and this chapter is the other half of that bargain: the tools that make a timing claim honest, the deterministic profiler, the clocks underneath it, percentiles by hand, the PEP 669 event plane, and the two kernels this corpus shares across all six language books, a particle step and a sqlite row crunch, measured on this machine under the gate interpreter, cpython 3.14.7. The chapter owns 30 ok lines across five samples, and its slow numbers live in an opt-in bench under `samples/bench/`, never in the gate.

14.1. cprofile and pstats

The profiler’s contract is stated in its first line: “cProfile and profile provide **deterministic profiling** of Python programs”, and deterministic means every event is caught, “all **function call**, **function return**, and **exception** events are monitored, and precise timings are made for the intervals between these events”. A deterministic profile of a fixed workload has a stable ranking even when its timings wobble, and the sample holds the profiler to the ranking, not the microseconds: a workload whose center of gravity is a named function must show that name at the top of `tottime`, which the manual defines as “for the total time spent in the given function (and excluding time made in calls to sub-functions)”, run after run.

```
def dominant_work(n):
    """The workload's center of gravity: pure bytecode, no helper calls."""
    total = 0
    for i in range(n):
        total = (total + i * 7) % 1_000_003
    return total


def light_work(n):
    return sum(range(n))


def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)


def profile_workload():
    prof = cProfile.Profile()
    prof.enable()
    dominant_work(40000)
    light_work(40000)
    fib(12)
    prof.disable()
    return pstats.Stats(prof)
```

listing 165: the workload: dominant_work carries the loop, light_work is the foil, fib recurses for the call-count column (source: python/samples/src/Ch14/cprofile_top.py, lines 12-35)

```
def top_local_name(stats):
    """The top local function by tottime, builtins and the profiler excluded."""
    rows = []
    for (filename, _, name), (_, _, tottime, _, _) in stats.stats.items():
        if not filename.startswith("<") and name != "profile_workload":
            rows.append((tottime, name))
    rows.sort(reverse=True)
    return rows[0]
```

listing 166: the top by tottime among local functions, the assertion the whole section stands on (source: python/samples/src/Ch14/cprofile_top.py, lines 38-45)

The pstats half reads the same table the profiler wrote. The sort is documented to descend, “all sorts on statistics are in descending order (placing most time consuming items first)”, and the recursion column is two numbers in one: “When there are two numbers in the first column (for example 3/1), it means that the function recursed. The second value is the number of primitive calls and the former is the total number of calls”. The sample pins `fib(12)` at 465 total calls over 1 primitive, which is the arithmetic of the recursion tree, not a timing, so it cannot drift with load.

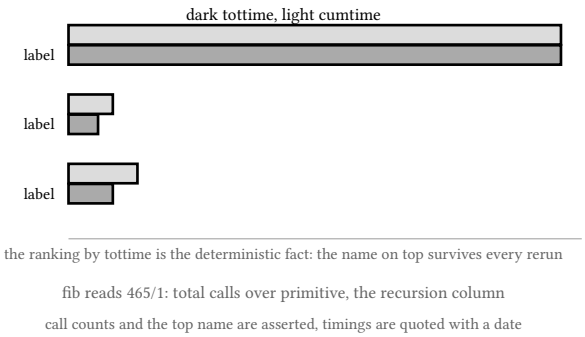


figure 57: one profile, two time columns: tottime owns the bar for dominant_work, cumtime credits the callers, and the recursion column splits 465 over 1

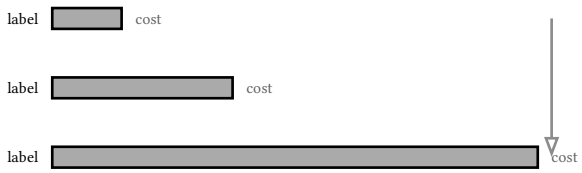
14.2. clocks: timeit versus perf_counter

Every number in this chapter rides one clock. `time.get_clock_info` names it: `perf_counter` is “QueryPerformanceCounter()” on this machine, monotonic, with a resolution of 1e-07, while the wall clock `time` reads “GetSystemTimePreciseAsFileTime()”. `timeit` rides the same counter, `timeit.default_timer` is `perf_counter` here, and the module’s promise is process, not hardware: “It avoids a number of common traps for measuring execution times”. Two of the traps are documented behavior: “By default, `timeit()` temporarily turns off garbage collection during the timing”, and the interpreter of the result vector is spelled out, “the `min()` of the result is probably the only number you should be interested in” because higher values come from interference, not from Python getting slower.

```
def cost_ns(fn):
    t0 = time.perf_counter()
    for _ in range(LOOPS):
        fn()
    return (time.perf_counter() - t0) / LOOPS * 1e9
```

listing 167: the cost probe: one loop, one closure, nanoseconds a call (source: python/samples/src/Ch14/clocks.py, lines 10-15)

The measured cost ladder on this machine, dated 2026-09-13, reads about 50 to 80 ns for a bare `perf_counter` read depending on the run, roughly 350 ns for `Timer.timeit(number=0)`, and about 42000 ns for the top-level `timeit.timeit(number=0)`, whose autocalibration machinery runs before the empty call is timed at all. The sample asserts the shape, read below method below top-level and the read under a 1000 ns pin, and the prose carries the dated values, because a nanosecond pin on a cache-warm loop would be a lie the first time Windows schedules something else.



the machine code is the same at every rung, the machinery above it is not

min of the repeats, the manual's rule, the gate's bench and this chapter both take it

figure 58: the cost ladder from the counter to the module: each step adds machinery, the top level adds autocalibration

14.3. percentiles by hand

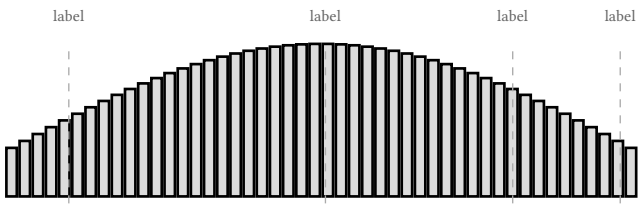
A latency claim is a distribution, and the honest summary of a distribution is its quantiles. The stdlib ships `quantiles()` in `statistics`, chapter 23's territory, and it interpolates; the cross-language contract this corpus uses is nearest rank, defined so the answer is always a sample someone waited for: rank `ceil(q/100 * n)` into the sorted copy, 1-based. On the integers 0 through 999 that is p50 499, p95 949, p99 989, p99.9 998, and on 1 through 10 it is p50 5 with everything from p95 up equal to 10.

```
def nearest_rank(samples, q):
    """Rank ceil(q/100 * n), 1-based, into the sorted copy. The p in p99 is
    'at most q percent of the samples fall below the answer'."""
    ordered = sorted(samples)
    rank = min(max(1, math.ceil(q * len(ordered) / 100)), len(ordered))
    return ordered[rank - 1]


def ladder(samples, qs=(50, 95, 99, 99.9)):
    return tuple(nearest_rank(samples, q) for q in qs)
```

listing 168: nearest rank: sort a copy, take the ceil-rank element, every answer is an actual sample (source: python/samples/src/Ch14/percentiles.py, lines 9-18)

The pinned case is 4096 gaussian draws from `random.Random(7)`, `gauss(100, 15)`: p50 100.293, p95 124.819, p99 134.594, p99.9 144.713, pinned to full precision because the seed fixes the list. The tail is the story the mean hides: the p99.9 sits almost ten standard deviations of the sample spread above the median, and no average of the two would represent either customer.



sorted draws, the height is the density, the marks are nearest-rank answers, seeded Random(7)

figure 59: the 4096 gaussian draws as a rank ladder: the four marked percentiles climb the right tail, each an actual draw

14.4. sys.monitoring, pep 669

book 8, ch 12 (python 3.14, how cpython runs your code) met the event plane from the eval loop's side. The profiler's side starts with why the plane exists, in PEP 669's own words: "Using a profiler or debugger in CPython can have a severe impact on performance. Slowdowns by an order of magnitude are common." The replacement is tool slots and event masks, "The VM can support up to 6 tools at once", and the module doc reserves the ids, debugger 0, coverage 1, profiler 2, optimizer 5. The import form this book pinned still holds on 3.14.7: `import`

`sys.monitoring` fails with "'sys' is not a package", `from sys import monitoring` is the door.

```
    seen = []
    tool = monitoring.DEBUGGER_ID
    monitoring.use_tool_id(tool, "profiling-sample")
    monitoring.register_callback(
        tool, monitoring.events.PY_START, lambda code, offset: seen.append(code.co_name)
    )
    monitoring.set_events(tool, monitoring.events.PY_START)
    alpha(10)
    alpha(11)
    monitoring.set_events(tool, 0)
    if seen.count("alpha") != 2:
        raise SystemExit(f"FAIL PY_START counted {seen.count('alpha')} of 2")
    print(
        f"ok 3 global PY_START fired exactly twice for two alpha calls, {len(seen)} total"
    )
```

listing 169: global PY_START through tool 0: two calls, two events, everything else counted too (source: python/samples/src/Ch14/monitoring.py, lines 35-50)

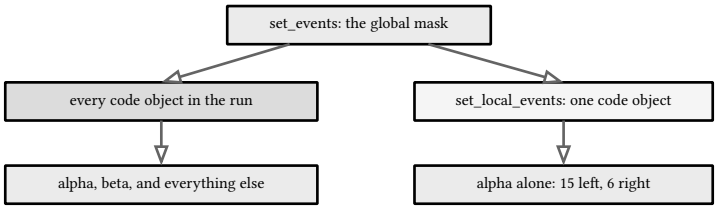
The distinction this chapter adds to the interpreter chapter's treatment is local against global. "Events can be turned on or off by setting the events either globally and/or for a particular code object", the local list covers the branch and line events, and the deprecation note points the way: "The `BRANCH` event is deprecated in 3.14. Using `BRANCH_LEFT` and `BRANCH_RIGHT` events will give much better performance as they can be disabled independently." The sample puts the mask on one function's code object and pins the census: 15 branch-left, 6 branch-right on the fixed 10-iteration loop, zero events for the function that calls it, mask 768 read back, cleared to nothing, tool freed to a `None` name.

```
    tool2 = 3
    monitoring.use_tool_id(tool2, "branch-sample")
    mask = monitoring.events.BRANCH_LEFT | monitoring.events.BRANCH_RIGHT
    monitoring.register_callback(
        tool2, monitoring.events.BRANCH_LEFT, lambda c, o, d: left.append(c.co_name)
    )
    monitoring.register_callback(
        tool2, monitoring.events.BRANCH_RIGHT, lambda c, o, d: right.append(c.co_name)
    )
    monitoring.set_local_events(tool2, alpha.__code__, mask)
    if monitoring.get_local_events(tool2, alpha.__code__) != mask:
        raise SystemExit("FAIL the local event mask did not read back")
    print("ok 4 the local mask reads back: branch left and right, 768")

    beta(10)
    if (len(left), len(right)) != (15, 6) or "beta" in left + right:
        raise SystemExit(f"FAIL branch counts {len(left)} left {len(right)} right")
    print("ok 5 local events on alpha only: 15 left, 6 right, beta fired nothing")

    try:
        monitoring.use_tool_id(tool2, "squat")
```

listing 170: local events on one code object: the mask, the census, the zero from the unmonitored caller (source: python/samples/src/Ch14/monitoring.py, lines 52-72)



an event set both ways triggers once, the manual says so and the census agrees

six tool slots, pep 669's fix for order-of-magnitude tracing slowdowns

figure 60: two scopes of the same event plane: global events fire for every code object, local events for one, and a double-set event fires once

14.5. the free-threaded sidebar

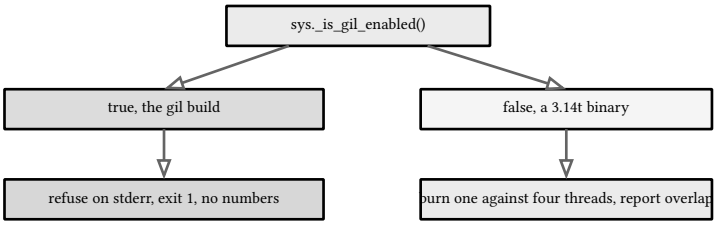
The free-threaded build is the one measurement this book refuses to make. The guard is a probe, not a flag: `sys._is_gil_enabled()` on this interpreter returns True, the machine has no 3.14t binary, `py -0p` lists 3.14 and 3.13 only, probed 2026-09-13, and so the sidebar script prints its refusal and exits 1:

```
def main():
    if sys._is_gil_enabled():
        print(
            "refusing to run: this interpreter has the GIL enabled, so it "
            "cannot say anything about free-threaded performance. this "
            "machine has no 3.14t binary (py -0p lists 3.14 and 3.13, "
            "probed 2026-09-13). install a free-threaded 3.14t and run "
            "this script under it, or read PEP 779 for the build's status.",
            file=sys.stderr,
        )
        raise SystemExit(1)
```

listing 171: the guard: on the gil build the script refuses, explains, and exits nonzero (source: python/samples/bench/free_thread_sidebar.py, lines 28-39)

NOTE: no free-threaded interpreter on this machine

`py -0p` on 2026-09-13 lists 3.14 and 3.13, neither with the free-threaded suffix. The refusal output above is the measured behavior of `free_thread_sidebar.py` under the gate interpreter, captured 2026-09-13: exit code 1, the message on stderr, nothing else. book 8, ch 17 (python 3.14, threads and the gil) carries the PEP 779 status story and book 8, ch 13 (python 3.14, pymalloc: arenas, freelists, and tracemalloc) records that mimalloc, the free-threaded build's required allocator, is compiled into this binary while the build that requires it is absent. What this book will not do is extrapolate a speedup from a build it never ran: free-threaded performance on this machine is unmeasured, and the honest number is the exit code.



this machine: the left door, measured, exit 1, captured 2026-09-13

fabricating the right door's number would be a lie with a decimal point

figure 61: the sidebar's decision: probe the gil, refuse loudly on the gil build, measure only under a real 3.14t

14.6. the particle kernel measured

The kernel is book 8, ch 13 (python 3.14, pymalloc: arenas, freelists, and tracemalloc)‘s: N particles, 8 float64 fields, 16 steps of pure arithmetic, seeded per representation. The gate smoke is 10000 particles with pinned checksums, and everything slower lives in the opt-in bench. At a million particles the array path measured 14.26 ns a particle a step as the sum of 16 individually timed steps, 2026-09-13, with a step p50 of 13.9 ms, p99 and max 18.6 ms; the best-of-repeats ladder in book 8, ch 13 (python 3.14, pymalloc: arenas, freelists, and tracemalloc) reads 11.48 ns at the same N, the gap between the two being the timer pairs and the absence of repeats, which is exactly why both are quoted with their method.

The 100 million one-off ran alone, sequential, on 2026-09-13, 6.4 GB of SoA plus two 800 MB scratch rows:

```
$ python particle_bench.py --n 100000000 --repr soa
repr soa n 100000000
build+warm 7.29s, K=16 steps total 27.01s
16.88 ns/particle/step
step latency ms p50/p99/max over K: 1680.5/1759.1/1759.1
checksum -1853.846773
```

listing 172: the 100m particle one-off: build, 16 steps, the step latency ladder, the checksum (source: python/samples/bench/captures/2026-09-13-particle-100m.txt, lines 1-6)

The per-step cost at 100 million, 16.88 ns, is the 10m ladder’s 16.18 ns with nothing left in any cache, pure DRAM streaming at 192 MB of traffic a step. One billion particles is math this machine does not run: 64 GB of buffer against the measured working set, “not measured on this machine” is the only honest sentence about it.

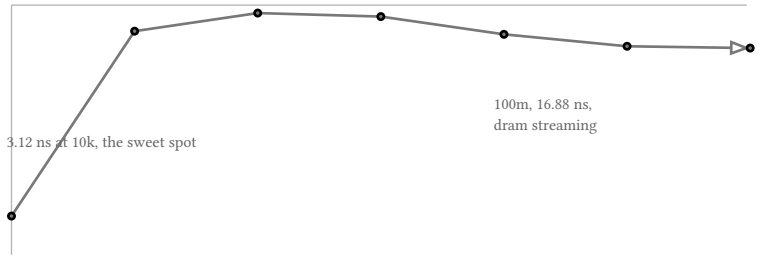


figure 62: the array path across five decades of N: the dispatch dive, the cache sweet spot at 10000, the DRAM climb through 100m

14.7. the row kernel measured

The second kernel is a sqlite table, `readings`, 412 stations `st000` through `st411`, eight REAL columns each, inserted in 10000-row batched transactions through `executemany`, the transaction rule the manual states: “The `INSERT` statement implicitly opens a transaction, which needs to be committed before changes are saved”. The engine is the interpreter’s own bundled library, `sqlite3.sqlite_version`, “Version number of the runtime SQLite library as a `string`”, 3.50.4 on this 3.14.7 build, asserted from inside the sample, and worth a sentence of drift honesty: the corpus elsewhere loads a 3.53.4 sqlite3.dll, this module carries its own older library, and both numbers are measured facts about their own binaries.

```
def build_db(path, rows, chunk):
    con = sqlite3.connect(path)
    cur = con.cursor()
    cur.execute(SCHEMA)
    for start in range(0, rows, chunk):
        cur.executemany(
            "INSERT INTO readings VALUES (?,?,?,?,?,?,?,?,?)", make_chunk(start, chunk)
        )
        con.commit()
    return con


def sql_group(cur):
    cols = ", ".join(f"min(v{i}), max(v{i}), avg(v{i})" for i in range(8))
    cur.execute(
        f"SELECT station, {cols} FROM readings GROUP BY station ORDER BY station"
    )
    return cur.fetchall()
```

listing 173: the smoke scale: 1000 seeded rows, ten committed chunks, the two aggregation passes ready (source: python/samples/src/Ch14/rowkernel.py, lines 40-57)

The 10 million row run, `row_bench.py --rows 10000000`, alone and sequential on 2026-09-13, wrote a 0.87 GB database and deleted it after:

```
$ python row_bench.py --rows 10000000
python 3.14.7, sqlite 3.50.4, rows 10000000, chunk 10000
insert: 327,703 rows/s (30.52s), db 0.87 GB
chunk commit ms p50/p95/p99/p99.9/max: 25.71/30.13/39.60/114.16/115.46 over 1000 chunks
gc gen0 collections during insert: 2753
sql group by: 536,378 rows/s (18.64s)
manual walk: 307,199 rows/s (32.55s)
min and max agree on all 412 stations, avg v0 within 1.67e-15
db deleted (10m+ one-offs leave no state behind)
```

listing 174: the 10m row kernel: insert rate, the chunk latency ladder over 1000 commits, gc counts, both aggregation passes (source: python/samples/bench/captures/2026-09-13-row-10m.txt, lines 1-10)

The commit ladder is the reason the percentile section exists: p50 25.71 ms against p99.9 114.16 ms says the median chunk tells almost nothing about the worst one in a thousand, and the tail is where a backpressure story lives. The two aggregation passes bracket the engine from both sides, the SQL GROUP BY folding at 536378 rows a second while the manual cursor walk manages 307199, and they agree: min and max exact on all 412 stations, every average within 1.67e-15 of the walk's own sum over count.

The 10m rate extrapolated sanely, 0.87 GB an order and change, so the 100 million row one-off ran too, alone, sequential, an 8.74 GB database written and deleted on 2026-09-13:

```
$ python row_bench.py --rows 100000000
python 3.14.7, sqlite 3.50.4, rows 100000000, chunk 10000
insert: 340,936 rows/s (293.31s), db 8.74 GB
chunk commit ms p50/p95/p99/p99.9/max: 24.75/28.16/42.75/112.44/138.31 over 10000 chunks
gc gen0 collections during insert: 27502
sql group by: 390,718 rows/s (255.94s)
manual walk: 332,489 rows/s (300.76s)
min and max agree on all 412 stations, avg v0 within 9.44e-16
db deleted (10m+ one-offs leave no state behind)
```

listing 175: the 100m row one-off: insert held its rate, the tail thinned by count but never disappeared, the walk caught the engine (source: python/samples/bench/captures/2026-09-13-row-100m.txt, lines 1-9)

The insert rate held at 340936 rows a second, ten times the rows in 9.6 times the seconds, and the tail tells the same story at both scales: p50 barely moves, 25.71 to 24.75 ms, while the max climbs to 138.31 ms across 10000 commits. The aggregation gap closed as the table outgrew the caches, the SQL pass falling to 390718 rows a second and the walk nearly catching it at 332489, because at 8.74 GB both passes are disk-bound and the interpreter's per-row tax shrinks to a share of the seek. One billion rows is math only, roughly 87 GB by the measured rate, "not measured on this machine".

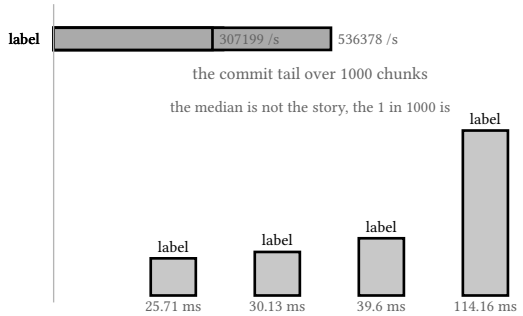


figure 63: the 10m run as two rates and a tail: insert and both aggregates in rows a second, the commit ladder underneath in milliseconds

14.7.1. the same kernel on duckdb

The corpus ran this kernel on duckdb too, from c and from go, and those chapters hold the engine-side numbers. The c twin, same 412 stations and rows bit-identical to its sqlite lane, measured the appender at 1.05 to 1.12 million rows a second against sqlite's batched 464,047, the group by at 79.6 to 83.3 million against 555,402, 143x to 150x, while the go lane appended 774,437 rows a second with its group by at 124,918,491 over the engine's own thread pool, all 2026-09-20 on duckdb 1.5.5. This book's lane stays the stdlib `sqlite3` anyway: the kernel's story is the commit ladder, percentiles over 10k-row transactions, and the module ships inside the interpreter with nothing to install. Pandas is the adjacent name: a frame converts into duckdb, and duckdb queries arrow and parquet directly. Which engine a workload wants is measured in book 13, ch 11 (infrastructure: docker, databases, and testing, duckdb: the embedded analytical engine) and chosen in book 13, ch 12 (infrastructure: docker, databases, and testing, duckdb and sqlite together: choosing the engine).

14.8. across the six books

The same two kernels ran in all six language books under one contract, N particles of 8 float64 fields stepped 16 times and 10 million readings rows of one station key plus 8 real columns, every number below quoted from the chapter that measured it on this machine, 2026-09-13. The first two columns are the particle step at 1m in each language's row-of-objects repr and its flat column repr, and they disagree on purpose: the flat repr wins 1.4x in c, 2.6x in javascript, and 21x in python, the object repr wins 2.3x in c# and 3.8x in go because the step touches all eight fields of every particle and one cache line per particle beats eight streams in flight, and lua's ffi cdata wins 11.5x over boxed tables. The last four columns are the shared sqlite contract at 10m rows, insert through 10k-row

batched transactions, the sql group by, the manual cursor walk, and the p99 of one 10k-row commit. Lua's p99 cell sits empty on purpose: its os.clock ticks at 1 ms and the chapter refuses to quote a percentile it cannot read, the boundary every book states in its own clock.

book	objects ns	flat ns	insert/s	group/s	walk/s	p99 ms
c, clang -O2	6.15 to 6.75	4.53 to 4.91	468,803	584,696	2,242,885	28.3
c#, .net 11	5.7	13.1	339,091	723,248	501,820	48.5
go 1.27	8.2	30.9	267,942	324,614	610,720	67.957
javascript, node 26	12.4	4.72	530.42k	851k	562.3k	17.914
python 3.14	345.7	16.18	327,703	536,378	307,199	39.60
lua 5.5 + luajit	114.1	9.9	504,765	573,915	580,353	

Table 1: the two kernels across the six language books: particle step ns at 1m in each book's object and flat repr, then the 10m row kernel with insert, group by, and walk in rows per second and p99 in milliseconds per 10k-row commit, every cell quoted from the chapter that measured it

The ladder above the table stays in its own book: the 100m particle one-off measured 5.768 ns flat and 7.229 ns object in c, 21.815 ns flat in c# (memory bound, soa only), 8.898 ns flat in go, 4.63 ns flat in javascript, 16.88 ns flat in python, and 8.2 ns ffi in lua, with the boxed 100m priced at 13.9 GB and refused. The 100m row one-off held its 10m insert rate within 5 percent in c, javascript, and lua, gained 4 percent in python, and paid 8 percent in go and 21 percent in c#. One billion is arithmetic in every book, 64 GB of particle state and roughly 81 to 87 GB of rows, not measured on this machine.

VERIFY: 30 checks, the boundary asserted first

A scoped run, `pwsh tools/run-py-samples.ps1 -Chapter Ch14`, walks the five samples through the run and format legs and reports: 5 files, 30 checks, format clean. `cprofile_top.py` carries the interpreter pin and contributes 5, the call totals, the top name by tottime, the ten-times margin, and the 465 over 1 recursion column. `clocks.py` adds 6, the two clock identities, the default timer, monotonicity, the read-cost pin, and the cost ladder's shape. `percentiles.py` adds 6, the two exact ladders, the sample-membership rule, the pinned gaussian percentiles, monotonicity, and the edges. `monitoring.py` adds 7, the import refusal, the tool ids, the global count, the mask readback, the 15 and 6 census, the squat refusal, and the clean teardown. `rowkernel.py` adds 6, the sqlite pin with the removed attribute, the seeded checksum, the two passes agreeing on 412 stations, the distinct count, and the cleanup. The slow numbers this chapter quotes all live in `samples/bench/`, never in the gate.

sources: docs.python.org/3/library/profile.html (deterministic profiling defined, tottime and cumtime, the sort order, the recursion column), docs.python.org/3/library/timeit.html (the common traps, gc off during timing, the min of the repeats), docs.python.org/3/library/time.html (get_clock_info and the two clock implementations), docs.python.org/3/library/sys.monitoring.html (tool ids 0 to 5, the local event list, global and local activation, the BRANCH deprecation), peps.python.org/pep-0669/ (the order-of-magnitude tracing slowdown, six tools at once, local events add to global), docs.python.org/3/library/sqlite3.html (sqlite_version defined, the implicit insert transaction, executemany's transaction handling, the version constants removed in 3.14), all accessed 2026-09-13. The gauss draws ride the random module page chapter 23 already pins, and every quoted capture, the sidebar refusal, and both 100m one-offs are this machine's own, dated 2026-09-13. Sample behavior verified by `make verify-py`, 30 checks in chapter 14.

15. files and serialization

Programs that stop when the process stops are demos. Everything in this chapter persists bytes and reads them back: paths as objects, text streams with their newline and encoding machinery, csv and json as the interchange formats, pickle as the python-native one with its protocol raised to 5 in 3.14, and tempfile as the staging layer that makes writes atomic. The windows facts are stated as windows facts: this machine writes `\r\n`, `os.linesep` says so, and the samples assert the bytes on disk rather than hoping. The gate runs the four samples under `Ch15/`, 67 `ok` lines in total, every file a scratch tree under `tempfile.mkdtemp()` that the sample removes behind itself.

15.1. pathlib

A path in python is an object with algebra. The / operator composes segments, division by a plain string works the same as division by a path, and an absolute right operand discards everything to its left, which is the composing rule people guess wrong first. The pure classes, `PurePosixPath` and `PureWindowsPath`, parse without touching the disk, and they interconvert when mixed, so windows flavour wins on the machine that runs this book:

```
win = PureWindowsPath(r"C:\Users\lavantien\report.txt")
check(
    1,
    "windows anchor and drive from the raw string",
    win.anchor == "C:\\" and win.drive == "C:",
)
check(
    2,
    "suffix and stem split the last dot",
    win.suffix == ".txt" and win.stem == "report",
)

joined = PurePosixPath("/etc") / "nginx" / "conf.d"
check(3, "the / operator composes segments", str(joined) == "/etc/nginx/conf.d")
check(
    4,
    "division by a str works the same as by a path",
    str(PurePosixPath("/etc") / "hosts") == "/etc/hosts",
)
check(
    5,
    "an absolute right operand replaces the left",
    str(PurePosixPath("/etc") / PurePosixPath("/usr")) == "/usr",
)

mix = PureWindowsPath("C:/data") / PurePosixPath("logs/app.log")
check(
    6,
    "flavour conversion when mixing pure paths",
    str(mix) == "C:\\data\\logs\\app.log",
)
```

listing 176: pure paths: anchor, drive, suffix, stem, the / operator, and the absolute right operand (source: python/samples/src/Ch15/pathlib_basics.py, lines 18-48)

The decomposition API answers the questions string splitting gets wrong. `parts` walks the drive first on windows, `suffixes` stacks `.tar` then `.gz`, and `with_name` swaps the final segment. The concrete class on this platform is `WindowsPath`, `Path.cwd()` and `Path.home()` return absolute existing paths, and the glob methods stay where they are told: `glob` reads one directory, `rglob` recurses:

```
check(
    7,
    "parts walks drive, segments, name",
    PureWindowsPath(r"C:\Users\x\demo.tar.gz").parts
    == ("C:\\", "Users", "x", "demo.tar.gz"),
)
check(
    8,
    "suffixes stack in order",
    PureWindowsPath("demo.tar.gz").suffixes == [".tar", ".gz"],
)

check(
    9,
    "on windows Path() is the windows flavour",
    type(Path(".")).__name__ == "WindowsPath",
)
here = Path.cwd()
check(10, "cwd returns an absolute path", here.is_absolute() and here.exists())
check(
    11,
    "home resolves under the user profile",
    "Users" in Path.home().parts or "Documents" in Path.home().parts,
)

with tempfile.TemporaryDirectory() as td:
    root = Path(td)
    (root / "logs").mkdir()
    (root / "logs" / "a.log").write_text("1\n", encoding="utf-8")
    (root / "logs" / "b.log").write_text("2\n", encoding="utf-8")
    (root / "logs" / "sub").mkdir()
    (root / "logs" / "sub" / "c.log").write_text("3\n", encoding="utf-8")
    (root / "notes.txt").write_text("n\n", encoding="utf-8")

    top = sorted(p.name for p in root.glob("*.txt"))
    check(12, "glob stays in one directory", top == ["notes.txt"])
    deep = sorted(p.name for p in root.rglob("*.log"))
    check(
        13, "rglob recurses through subdirectories", deep == ["a.log", "b.log", "c.log"]
    )
```

listing 177: parts and suffixes, the concrete windows flavour, glob versus rglob (source: python/samples/src/Ch15/pathlib_basics.py, lines 50-89)

Reading and writing ride the same objects. `read_text` and `write_text` open and close the file for you, and the signature page notes the newest parameter: `Path.read_text(encoding=None, errors=None, newline=None)`, with “Changed in version 3.13: The `newline` parameter was added.” The sample writes with the default and reads the bytes back to show what translation did, the subject of the next section in one check:

```
    f = root / "notes.txt"
    f.write_text("roundtrip\n", encoding="utf-8")
    check(
        14,
        "write_text then read_text round trips",
        f.read_text(encoding="utf-8") == "roundtrip\n",
    )
    check(
        15,
        "write_text translated the newline for the platform",
        f.read_bytes() == b"roundtrip" + os.linesep.encode(),
    )
    check(
        16,
        "with_name swaps the final segment",
        (f.with_name("other.txt")).name == "other.txt",
    )
    check(
        17,
        "mkdir exists_ok is idempotent",
        (root / "logs").mkdir(exist_ok=True) is None,
    )
    check(
        18,
        "os functions accept Path objects",
        os.path.isfile(f) and os.path.getsize(f) == 11,
    )
```

listing 178: read_text and write_text, the newline that reached disk, and os functions taking Path objects (source: python/samples/src/Ch15/pathlib_basics.py, lines 91-117)

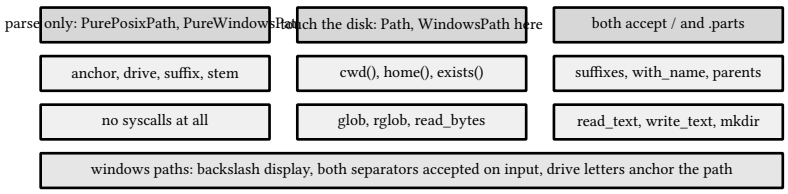


figure 64: the pathlib surface as a matrix: pure versus concrete, by operation

15.2. open, text vs binary

Text mode is a translation layer, and the `open()` page states both directions. Reading: “if `newline` is None, universal newlines mode is enabled. Lines in the input can end in `\n`, `\r`, or `\r\n`, and these are translated into `\n` before being returned to the caller. If it is ‘’, universal newlines mode is enabled, but line endings are returned to the caller untranslated.” Writing: “if `newline` is None, any `\n` characters written are translated to the system default line separator, `os.linesep`. If `newline` is ‘’ or ‘n’, no translation takes place.” On this machine `os.linesep` is `\r\n`, and the sample asserts the translation at the byte level in both directions:

```
    with open(text_file, "w", encoding="utf-8") as f:
        f.write("one\ntwo\n")
    check(1, "this machine's line separator is cr lf", os.linesep == "\r\n")
    check(
        2,
        "text mode wrote the platform separator",
        text_file.read_bytes() == b"one\r\ntwo\r\n",
    )

    with open(text_file, encoding="utf-8") as f:
        content = f.read()
    check(3, "default read translates every ending to lf", content == "one\ntwo\n")

    with open(text_file, encoding="utf-8", newline="") as f:
        raw = f.read()
    check(4, "newline empty disables translation on read", raw == "one\r\ntwo\r\n")

    classic = root / "classic.txt"
    with open(classic, "w", encoding="utf-8", newline="\n") as f:
        f.write("a\r\nb\nc\r")
    with open(classic, encoding="utf-8") as f:
        universal = f.read()
    check(5, "universal newline mode folds cr and cr lf too", universal == "a\nb\nc\n")
```

listing 179: the newline contract on windows: lf out, cr lf on disk, lf back in (source: python/samples/src/Ch15/text_binary.py, lines 22-44)

Encoding is the other half of the layer, and the habit this book enforces is passing it explicitly every time. A utf-8 file with accents is 13 bytes on disk and 11 characters in memory, the sample asserts both numbers, and an intentionally wrong `ascii` read fails with `UnicodeDecodeError` instead of guessing, which is the correct behavior for text that is not what the reader claims. The bytes side of the house is book 8, ch 5 (python 3.14, strings, bytes, and text) territory, here it is the control group: binary mode moves the exact bytes, and its `tell` and `seek` are plain byte offsets:

```
    uni = root / "unicode.txt"
    uni.write_text("héllo wörld", encoding="utf-8")
    check(
        6,
        "utf-8 encodes the accents as 2 bytes each",
        uni.read_bytes() == b"h\xc3\xa9llo w\xc3\xb6rld",
    )
    with open(uni, encoding="utf-8") as f:
        check(7, "the same encoding reads it back", f.read() == "héllo wörld")

    try:
        uni.read_text(encoding="ascii")
    except UnicodeDecodeError:
        check(8, "the wrong encoding fails loudly", True)
    else:
        check(8, "the wrong encoding fails loudly", False)

    blob = root / "blob.bin"
    with open(blob, "wb") as f:
        f.write(b"\x00\x01\xff\r\n")
    check(
        9, "binary mode does no translation", blob.read_bytes() == b"\x00\x01\xff\r\n"
    )

    with open(blob, "rb") as f:
        first = f.read(2)
        pos = f.tell()
        f.seek(0)
        check(
            10,
            "binary tell and seek are byte offsets",
            first == b"\x00\x01" and pos == 2 and f.read(1) == b"\x00",
        )

    with open(uni, encoding="utf-8") as f:
        f.seek(0)
        cookie = f.tell()
        f.readline()
        after = f.tell()
    check(
        11,
        "text tell returns bytes, not characters",
        cookie == 0 and after == len(uni.read_bytes()),
    )
```

listing 180: encoding round trips, the loud wrong-decode failure, byte offsets versus text cookies (source: python/samples/src/Ch15/text_binary.py, lines 46-89)

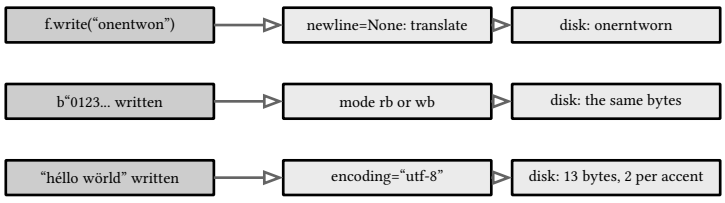
The mode string packs several facts at once and the sample covers the remaining ones: `a` appends without truncating, `w+` truncates then reads, and `buffering=0` on a binary stream hands back the raw `FileIO` object where the default wraps a `BufferedReader`. One hazard belongs beside the modes: a text stream's `tell()` returns an opaque cookie that happens to be a byte offset on this build, the sample asserts the equality to the byte length, but the docs' word for the value is a cookie and code that does arithmetic on it is code that has already decided to be fragile:

```
    append = root / "append.txt"
    append.write_text("start\n", encoding="utf-8")
    with open(append, "a", encoding="utf-8") as f:
        f.write("more\n")
    check(
        12,
        "mode a appends without truncating",
        append.read_text(encoding="utf-8") == "start\nmore\n",
    )

    with open(blob, "rb", buffering=0) as f:
        check(13, "buffering 0 returns a raw file object", type(f).__name__ == "FileIO")
    with open(blob, "rb") as f:
        check(
            14,
            "default buffering wraps a buffered reader",
            type(f).__name__ == "BufferedReader",
        )

    plus_mode = root / "rw.txt"
    with open(plus_mode, "w+", encoding="utf-8") as f:
        f.write("abc")
        f.seek(0)
        check(15, "w plus truncates then reads back", f.read() == "abc")
```

listing 181: append, w+, and the buffering levels as object types (source: python/samples/src/Ch15/text_binary.py, lines 91-114)



reading runs the same layers backwards, newline=None folds every ending to lf
csv is the odd one out: it wants newline="" so it can place its own rn

figure 65: one write, before and after the translation layers

15.3. csv and json

The csv module writes its own line endings, `\r\n` per the excel dialect, so the file must be opened with translation off. The reference says it in one clause: "If csvfile is a file object, it should be opened with `newline=''`." The sample obeys on the write side and then deliberately disobeys once, and the naive file on disk carries `\r\r\n` per row, the double translation, exactly what the docs warn about:

```
    with open(table, "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=["name", "city", "age"])
        writer.writeheader()
        writer.writerows(rows)

    raw = table.read_bytes()
    check(
        1,
        "csv writes cr lf rows and quotes separators",
        raw.startswith(b"name,city,age\r\n")
        and b'"london, uk"' in raw
        and b'"portland ""or"""' in raw,
    )

    with open(table, newline="", encoding="utf-8") as f:
        back = list(csv.DictReader(f))
    check(
        2,
        "DictReader keys on the header row",
        back[0]["name"] == "ada" and back[1]["age"] == "55",
    )
    check(
        3,
        "quoted commas and quotes survive the round trip",
        back[0]["city"] == "london, uk" and back[1]["city"] == 'portland "or"',
    )

    no_newline_arg = root / "naive.csv"
    with open(no_newline_arg, "w", encoding="utf-8") as f:
        csv.writer(f).writerows([["a", "b"], ["c", "d"]])
    check(
        4,
        "opening without newline empty writes cr cr lf",
        no_newline_arg.read_bytes() == b"a,b\r\r\nc,d\r\r\n",
    )
```

listing 182: dictwriter with newline empty, then the same write without it: cr cr lf on disk (source: python/samples/src/Ch15/csv_json.py, lines 27-61)

Dialects are parameter sets, `excel` is the default, and the sample builds one with a semicolon delimiter and `QUOTE_ALL` into a `StringIO` to show the dial turning without a disk in sight. Quoting itself is minimal by default: a comma in a field forces quotes, a quote inside doubles it, and the round trip through `DictReader` restores the originals:

```
    with open(table, newline="", encoding="utf-8") as f:
        plain = list(csv.reader(f, dialect="excel"))
    check(5, "excel is the default dialect", plain[0] == ["name", "city", "age"])

    buffer = io.StringIO()
    csv.writer(buffer, delimiter=";", quoting=csv.QUOTE_ALL).writerows(
        [["x", "y"], ["1", "2"]]
    )
    check(
        6,
        "a dialect change rewrites separators and quoting",
        buffer.getvalue() == '"x";"y"\r\n"1";"2"\r\n',
    )
```

listing 183: excel as the default dialect, and a custom one against StringIO (source: python/samples/src/Ch15/csv_json.py, lines 63-75)

Json is the interchange format and its conversion rules are where the surprises live. The sample walks the trap row by row: keys that are not strings become strings, and a dict holding both 1 and "1" serializes to a document with the same key twice, where the decoder keeps the last, tuples come back as lists, and sets are refused with `TypeError`. The escape hatch for the last case is the `default=` hook, which receives each unknown object and returns something serializable:

```
    mixed = {1: "int key", "1": "str key"}
    collided = json.loads(json.dumps(mixed))
    check(
        9,
        "non-string keys become strings and collide",
        json.dumps(mixed) == '{"1": "int key", "1": "str key"}'
        and collided == {"1": "str key"},
    )

    check(
        10,
        "tuples serialize as arrays and come back lists",
        json.dumps((1, 2)) == "[1, 2]" and type(json.loads("[1, 2]")) is list,
    )

    try:
        json.dumps({1, 2})
    except TypeError as exc:
        check(
            11, "sets are not json and fail loudly", "not JSON serializable" in str(exc)
        )

    def encode_set(value):
        if isinstance(value, set):
            return {"$set": sorted(value)}
        raise TypeError(f"unserializable {type(value).__name__}")

    tagged = json.dumps({3, 1, 2}, default=encode_set)
    check(12, "the default hook rescues unknown types", tagged == '{"$set": [1, 2, 3]}')
```

listing 184: the implicit conversions and the default hook catching a set (source: python/samples/src/Ch15/csv_json.py, lines 92-120)

PITFALL: the duplicated json key is legal and loses data

`json.dumps({1: "a", "1": "b"})` produces `{"1": "a", "1": "b"}`, a document with one key spelled twice. It is valid json and the decoder resolves it deterministically, the last value wins, so the round trip is silently lossy for the first. The check names the exact output bytes so the behavior is pinned, not folklore. The cure is upstream: keep mapping keys as strings before they reach the serializer.

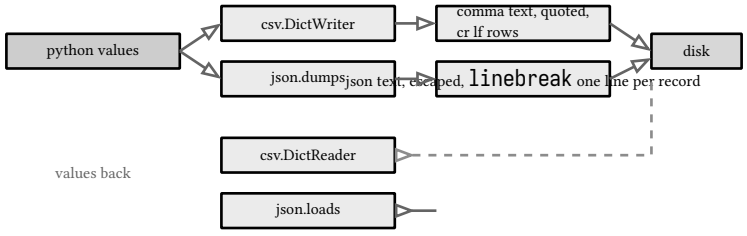


figure 66: rows through the interchange pipeline, both formats end as text on disk

15.4. pickle and tempfile

Pickle serializes python objects for python consumers, and 3.14 moved its default: the protocol page states “Currently the default protocol is 5”, the data stream section adds “Protocol version 5 … adds support for out-of-band data and speedup for in-band data. It is the default protocol starting with Python 3.14”, and the changelog rows read “Changed in version 3.8: The default protocol is 4” then “Changed in version 3.14: The default protocol is 5.” The sample pins the header byte, `\x80\x05`, and confirms that asking for 6 is refused:

```
check(1, "3.14 defaults to pickle protocol 5", pickle.DEFAULT_PROTOCOL == 5)
check(2, "the highest protocol is also 5", pickle.HIGHEST_PROTOCOL == 5)
check(
    3,
    "the protocol byte rides the stream header",
    pickle.dumps([1])[:2] == b"\x80\x05"
    and pickle.dumps([1], protocol=4)[:2] == b"\x80\x04",
)
try:
    pickle.dumps([1], protocol=6)
except ValueError as exc:
    check(4, "protocol 6 does not exist yet", "must be ≤ 5" in str(exc))
```

listing 185: the 3.14 default: protocol 5, its header byte, and the refusal of 6 (source: python/samples/src/Ch15/pickle_tempfile.py, lines 19-30)

What protocol 5 enables is the out-of-band buffer path, pep 574’s design. An object whose `__reduce__` returns a `PickleBuffer` for a large block does not get that block copied into the stream. Instead `dumps` with a `buffer_callback` hands the buffer to the caller, writes a marker into the stream, and `loads` reassembles the object only when the same buffers are passed back. The sample demonstrates the whole contract: one buffer handed over, the payload absent from the blob, the object rebuilt, loading without buffers failing, and `buffer_callback` under protocol 4 rejected:

```
class Payload:
    def __init__(self, data):
        self.data = data

    def __reduce__(self):
        return Payload, (pickle.PickleBuffer(self.data),)

    def __eq__(self, other):
        return isinstance(other, Payload) and other.data == self.data


original = Payload(bytearray(b"live-payload"))
buffers = []
blob = pickle.dumps(original, protocol=5, buffer_callback=buffers.append)
check(
    5,
    "protocol 5 hands buffers to the callback",
    len(buffers) == 1 and isinstance(buffers[0], pickle.PickleBuffer),
)
check(6, "the in-band stream carries only a marker", b"live-payload" not in blob)

restored = pickle.loads(blob, buffers=buffers)
check(
    7,
    "out-of-band data reassembles the object",
    restored == original and bytes(restored.data) == b"live-payload",
)

try:
    pickle.loads(blob)
except pickle.UnpicklingError as exc:
    check(8, "loading without the buffers fails", "out-of-band data" in str(exc))
try:
    pickle.dumps(original, protocol=4, buffer_callback=buffers.append)
except ValueError as exc:
    check(9, "buffer_callback demands protocol 5", "protocol ≥ 5" in str(exc))
```

listing 186: pep 574: buffers handed out, markers in the stream, data reassembled on load (source: python/samples/src/Ch15/pickle_tempfile.py, lines 33-68)

WARNING: unpickling runs code

The module page does not soften it: pickle "is not secure. Only unpickle data you trust. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling." The sample proves the mechanism with a benign `__reduce__` that returns a function, and the function runs on load, checked. The spawn boundary in book 8, ch 18 (python 3.14, multiprocessing and process patterns) leans on pickle, which is why that chapter treats process arguments as trusted-by-construction and everything socket-facing as json.

Staging is tempfile's job. The module page sorts the surface: the "high-level interfaces which provide automatic cleanup and can be used as context managers" against the "lower-level functions which require manual cleanup", `mkstemp()` and `mkdtemp()`. The atomic write pattern stitches the two halves together: write a complete file beside the target, then `os.replace` it over the target, which on windows replaces an existing destination where `os.rename` fails, probed on this machine. A `SpooledTemporaryFile` stays in memory under its size limit, no name on disk, and rolls over to a real file past it:

```
    with tempfile.NamedTemporaryFile(dir=root, delete=False) as tmp:
        tmp.write(b"partial")
        staged = Path(tmp.name)
    check(
        14,
        "the staged temp file sits in the target directory",
        staged.parent == root and staged.read_bytes() == b"partial",
    )
    os.replace(staged, target)
    check(
        15,
        "os.replace swaps the file in place",
        target.read_bytes() == b"partial" and not staged.exists(),
    )
    leftover = root / "leftover.txt"
    leftover.write_text("old", encoding="utf-8")
    stray = root / "leftover.txt.new"
    stray.write_text("new", encoding="utf-8")
    os.replace(stray, leftover)
    check(
        16,
        "os.replace also overwrites an existing destination",
        leftover.read_text(encoding="utf-8") == "new",
    )

    spool = tempfile.SpooledTemporaryFile(max_size=8)
    check(
        17,
        "a spooled file under its limit has no name on disk",
        (spool.name is None)
        or isinstance(spool.name, str)
        and not Path(spool.name).exists(),
    )
    spool.write(b"0123456789abcdefghij")
    spool.rollover()
    check(
        18,
        "past the limit it rolls over to a real file",
        spool.name is not None and Path(spool.name).exists(),
    )
    spool.seek(0)
    check(19, "the rolled content is intact", spool.read() == b"0123456789abcdefghij")
```

listing 187: stage beside the target, os.replace over it, spool until the limit (source: python/samples/src/Ch15/pickle_tempfile.py, lines 112-153)

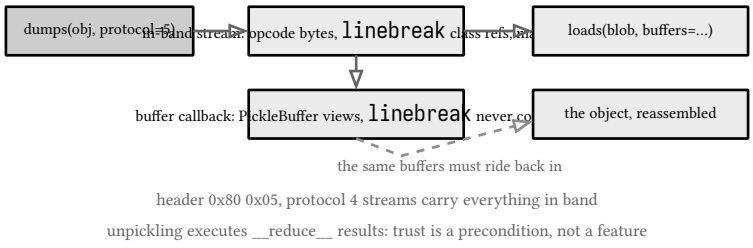


figure 67: the pickle stream as a data structure, with buffers outside the frame

VERIFY: what a green run of this chapter proves

> The four samples print 67 `ok` lines: path algebra and globs, the newline and encoding layers at the byte level, csv dialects and the doubled-row hazard, json conversions and the duplicate-key trap, pickle protocol 5 with out-of-band buffers, and the atomic write plus tempfile staging. All of it ran on cpython 3.14.7 with files under temp directories that the samples removed.

sources: docs.python.org/3.14 library/pathlib (read_text signature and newline note, glob), library/functions open (newline semantics), library/csv (newline requirement, dialects), library/json, library/ pickle (DEFAULT_PROTOCOL, data stream format, security warning), library/tempfile, and peps.python.org pep 574, all accessed 2026-09-12. Sample behavior verified by `make verify-py`, 67 checks in chapter 15.

16. os, subprocess, and signals

A python process is a process, and on windows it is a windows process. This chapter is the stdlib's process layer from the inside out: `subprocess.run` for the common case of "run this and give me the output", `Popen` and pipes for the streaming cases, the `os` surfaces for environment, arguments, directory, and exit codes, and the signal module's small windows reality, where seven signal names exist, an external kill is an unconditional `TerminateProcess`, and graceful shutdown is something a process does to itself. Every child in the samples is `sys.executable` with `-X utf8`, the same interpreter this book pins in book 8, ch 1 (python 3.14, toolchain: cpython 3.14 on windows), so the whole chapter is python watching python. The gate runs the four samples under `Ch16/`, 47 `ok` lines in total.

16.1. subprocess run and check

`run` is one function covering the common case, and the page states its contract: "Run the command described by args. Wait for command to complete, then return a CompletedProcess instance." Capture is a flag: "If `capture_output` is true, stdout and stderr will be captured", and the sample's first check pins what that means on this machine, a `bytes` stdout holding `hello\r\n`, the carriage return included, because the child wrote through a console-ish pipe and nobody translated anything. Exit status is a field, and `check` turns it into control flow, the page again: "If `check` is true, and the process exits with a non-zero exit code, a `CalledProcessError` exception will be raised":

```
r = subprocess.run(PY + ["-c", "print('hello')"], capture_output=True)
check(
    1,
    "capture_output returns bytes by default",
    r.stdout == "hello\r\n".encode() and isinstance(r.stdout, bytes),
)

check(2, "a clean exit reports returncode 0", r.returncode == 0)
check(3, "stderr is captured separately", r.stderr == b"")

code3 = subprocess.run(PY + ["-c", "import sys; sys.exit(3)"], capture_output=True)
check(4, "sys.exit(3) surfaces as returncode 3", code3.returncode == 3)

try:
    subprocess.run(PY + ["-c", "raise SystemExit(7)"], check=True)
except subprocess.CalledProcessError as exc:
    check(5, "check=True turns a nonzero exit into an exception", exc.returncode == 7)

with_output = subprocess.run(
    PY + ["-c", "import sys; sys.stderr.write('bad news')"],
    capture_output=True,
    check=False,
)
check(
    6,
    "check=False leaves the error to the caller",
    with_output.returncode == 0 and with_output.stderr == b"bad news",
)
```

listing 188: capture, the cr lf bytes on windows, returncode, and check=True raising (source: python/samples/src/Ch16/run_check.py, lines 19-46)

Timeouts are enforced, not advisory: "If the timeout expires, the child process will be killed and waited for. The `TimeoutExpired` exception will be re-raised after the child process has terminated." The sample parks a child in `time.sleep(30)` with `timeout=2` and asserts both halves, the exception with its `.timeout` intact and no orphan left behind. Text versus bytes is the other axis: `text=True` decodes with `locale.getpreferredencoding(False)` unless

`encoding` says otherwise, and the sample runs the same utf-8 child both ways, one check reading `café` and one reading the raw `\xc3\xa9` bytes:

```
utf8_child = subprocess.run(
    PY + ["-c", "print('café')"],
    capture_output=True,
    text=True,
    encoding="utf-8",
)
check(
    9,
    "text mode with encoding decodes utf-8 cleanly",
    utf8_child.stdout.strip() == "café",
)

raw = subprocess.run(PY + ["-c", "print('café')"], capture_output=True)
check(
    10,
    "bytes mode exposes the raw utf-8 on disk",
    raw.stdout == "caf\xc3\xa9\r\n".encode("latin-1"),
)

missing = subprocess.run(
    [sys.executable, "-X", "utf8", "-c", "import no_such_module_xyz"],
    capture_output=True,
    text=True,
    encoding="utf-8",
)
check(
    11,
    "the child's traceback arrives as stderr text",
    missing.returncode == 1 and "ModuleNotFoundError" in missing.stderr,
)

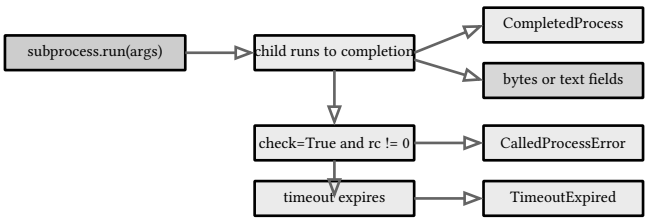
try:
    subprocess.run(PY + ["-c", "import time; time.sleep(30)"], timeout=2)
except subprocess.TimeoutExpired as exc:
    check(
        12,
        "timeout kills the child and raises",
        exc.timeout == 2 and exc.stdout is None,
    )
```

listing 189: text plus encoding against raw bytes, and the timeout that kills the child (source: python/samples/src/Ch16/run_check.py, lines 67-105)

`shell=True` deserves its own paragraph rather than a check. The security page states the default posture: “Unlike some popen functions, this library will not implicitly choose to call a system shell. This means that all characters, including shell metacharacters, can safely be passed to child processes. If the shell is invoked explicitly, via `shell=True`, it is the application’s responsibility to ensure that all whitespace and metacharacters are quoted appropriately.” On windows the 3.12 change closed the classic hunt: the shell search now uses “`%COMSPEC%` and `%SystemRoot%\System32\cmd.exe`”, so “dropping a malicious program named cmd.exe into a current directory no longer works.” The batch file caveat runs the other way: a `.bat` or `.cmd` target “may be launched by the operating system in a system shell regardless of the arguments passed to this library.” None of this book’s children use a shell, and the samples never pass `shell=True`.

PITFALL: list form, no shell, still quote nothing

> The rule this book follows: build the command as a list, pass it without a shell, and let the metacharacters be data. The moment string concatenation builds a command line, quoting becomes the program's job and the program is bad at it. The one windows exception in the docs is deliberate batch file launch with untrusted arguments, where passing `shell=True` actually lets python add the escaping. Everything else is a bug wearing a convenience.



timeout kills and waits for the child: no orphans, checked

no shell anywhere in this chapter: the command list is data

figure 68: run as a pipeline: one call, four fates for the result

16.2. popen and pipes

`run` is a wrapper, and the class under it is `Popen`. A `Popen` object exists the moment the child is created and the parent is free to keep working, which the sample checks with `poll()` returning `None` while the child has not exited. The page names the method that does the waiting: "`Interact with process`: Send data to stdin. Read data from stdout and stderr, until end-of-file is reached. Wait for process to terminate and set the `returncode` attribute", and adds the two conditions people miss: "if you want to send data to the process's stdin, you need to create the `Popen` object with `stdin=PIPE`. Similarly, to get anything other than `None` in the result tuple, you need to give `stdout=PIPE` and/or `stderr=PIPE` too":

```
proc = subprocess.Popen(
    PY + ["-c", "print('spawned'); raise SystemExit(4)"],
    stdout=subprocess.PIPE,
    stderr=subprocess.PIPE,
)
check(1, "popen returns before the child finishes", proc.poll() in (None, 4))

out, err = proc.communicate(timeout=30)
check(
    2,
    "communicate drains stdout and waits",
    out == "spawned\r\n".encode() and err == b"",
)
check(3, "communicate sets returncode", proc.returncode == 4)

feed = subprocess.Popen(
    PY + ["-c", "import sys\ntotal = sys.stdin.read()\nprint(len(total))"],
    stdin=subprocess.PIPE,
    stdout=subprocess.PIPE,
)
out, _ = feed.communicate(b"abcdef", timeout=30)
check(
    4,
    "stdin pipes carry bytes both directions",
    out.strip() == b"6" and feed.returncode == 0,
)
```

listing 190: the popen lifecycle: create ahead, communicate once, returncode set (source: python/samples/src/Ch16/pipes.py, lines 20-45)

The pipe deadlock is the reason `communicate` exists. An os pipe holds a bounded amount, on the order of kilobytes, and a parent that writes everything to a child's stdin before reading the child's stdout deadlocks in both directions: the child blocks writing a full pipe, the parent blocks writing its own. `communicate` threads the transfer, draining while it writes. The sample proves the property by volume, pushing 4 mb through a child that uppercases and returns it, far past any pipe buffer:

```
big = subprocess.Popen(
    PY
    + [
        "-c",
        "import sys\ndata = sys.stdin.buffer.read()\n"
        "sys.stdout.buffer.write(data.upper())",
    ],
    stdin=subprocess.PIPE,
    stdout=subprocess.PIPE,
)
payload = b"x" * (4 * 1024 * 1024)
out, _ = big.communicate(payload, timeout=60)
check(
    5,
    "communicate moves a 4 mb payload past the pipe buffer",
    out == b"X" * (4 * 1024 * 1024),
)

streaming = subprocess.Popen(
    PY + ["-u", "-X", "utf8", "-c", "for i in range(3):\n    print(f'line {i}')"],
    stdout=subprocess.PIPE,
    text=True,
    encoding="utf-8",
)
lines = [streaming.stdout.readline().strip() for _ in range(3)]
streaming.stdout.close()
tail = streaming.wait(timeout=30)
check(
    6,
    "line-by-line reads work while the child runs",
    lines == ["line 0", "line 1", "line 2"] and tail == 0,
)
```

listing 191: 4 mb through the pipes, and line-by-line reads while the child runs (source: python/samples/src/Ch16/pipes.py, lines 47-78)

The timeout contract differs between the two layers, and the sample pins the asymmetry. `run(timeout=...)` kills the child, quoted above. `Popen.communicate(timeout=...)` does not: "The child process is not killed if the timeout expires", so the caller cleans up, and the sample catches the expiry, asserts the child is still alive, then `kill()` and `wait()`. An explicit `env=` replaces the whole environment rather than updating it, which the sample demonstrates with a two-variable environment, one value of its own and `SYSTEMROOT`, which a python child on this machine still needs:

```
slow = subprocess.Popen(PY + ["-c", "import time; time.sleep(30)"])
try:
    slow.communicate(timeout=1)
except subprocess.TimeoutExpired:
    check(8, "a popen timeout raises without killing the child", slow.poll() is None)
finally:
    slow.kill()
    slow.wait(timeout=30)
check(9, "kill then wait settles the returncode", slow.returncode != 0)

env_child = subprocess.Popen(
    PY + ["-c", "import os; print(os.environ.get('PIPE_DEMO'))"],
    stdout=subprocess.PIPE,
    text=True,
    encoding="utf-8",
    env={"PIPE_DEMO": "inherited", "SYSTEMROOT": os.environ["SYSTEMROOT"]},
)
env_out, _ = env_child.communicate(timeout=30)
check(
    10, "an explicit env replaces the whole environment", env_out.strip() == "inherited"
)
```

listing 192: the popen timeout leaves the child alive, cleanup is kill then wait, env replaces (source: python/samples/src/Ch16/pipes.py, lines 92-112)

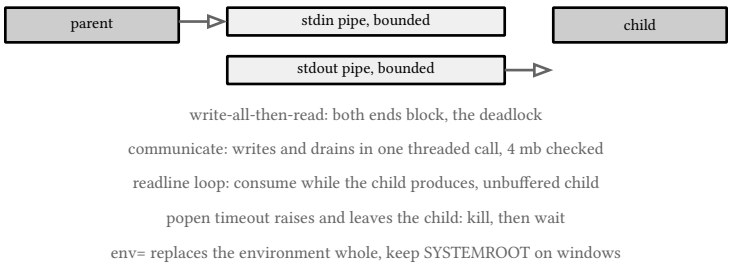


figure 69: two pipes and one bounded buffer: where the naive write blocks

16.3. os interfaces

The os module's process surfaces are the ones a program uses before and after any child exists. `os.environ` is a mapping of the live environment: assignment and deletion really change what a later child inherits, the sample sets a variable, spawns a child that prints it, then deletes it, and snapshots with `dict(os.environ)` because iteration over the live object while mutating it is exactly as fragile as it sounds. `sys.argv` belongs to the same boundary, and the sample hands a child three arguments and asserts the list it sees:

```
os.environ["OS_DEMO"] = "from-parent"
child = subprocess.run(
    PY + ["-c", "import os; print(os.environ['OS_DEMO'])"],
    capture_output=True,
    text=True,
    encoding="utf-8",
)
check(
    1,
    "a parent's environ entry reaches the child",
    child.stdout.strip() == "from-parent",
)

snapshot = dict(os.environ)
os.environ["OS_DEMO"] = "mutated"
del os.environ["OS_DEMO"]
check(
    2,
    "environ mutates in place and deletes",
    "OS_DEMO" not in os.environ and snapshot["OS_DEMO"] == "from-parent",
)
os.environ.update(snapshot)

argv_child = subprocess.run(
    PY + ["-c", "import sys; print(sys.argv[1:])", "alpha", "beta", "3"],
    capture_output=True,
    text=True,
    encoding="utf-8",
)
check(
    3,
    "argv after the program belongs to the child",
    argv_child.stdout.strip() == "['alpha', 'beta', '3']",
)
```

listing 193: environ inherited by children, mutated in place, and argv handed through (source: python/samples/src/Ch16/os_faces.py, lines 21-54)

The working directory is process state, not a parameter. `chdir` moves the whole process, every relative path and every later child with an inherited `cwd`, and the sample does the round trip inside a `try/finally` because a failed restore poisons the rest of any real program. Exit codes close the loop. `os._exit` is the abrupt form, it sets the code and skips cleanup, and the sample shows what that costs on a pipe: a child that prints without flushing loses the line entirely, while `flush=True` survives. A `SystemExit` carrying a string prints the string to stderr and codes 1, which is why error messages belong in exceptions and codes stay integers:

```
here = Path.cwd()
elsewhere = here.parent if here.parent != here else Path.cwd()
try:
    os.chdir(elsewhere)
    check(4, "chdir moves the process-wide cwd", Path.cwd() == elsewhere)
finally:
    os.chdir(here)
check(5, "chdir back restores the starting directory", Path.cwd() == here)

script_child = subprocess.run(
    PY + ["-c", "import os\nos._exit(9)"],
    capture_output=True,
)
check(
    6,
    "os._exit skips cleanup but sets the exit code",
    script_child.returncode == 9 and script_child.stdout == b"",
)

lost = subprocess.run(
    PY + ["-c", "import os\nprint('unflushed')\nos._exit(0)"],
    capture_output=True,
)
kept = subprocess.run(
    PY + ["-c", "import os\nprint('flushed', flush=True)\nos._exit(0)"],
    capture_output=True,
)
check(
    7,
    "os._exit drops unflushed stdout on the floor",
    lost.stdout == b"" and kept.stdout == b"flushed\r\n",
)

exit_child = subprocess.run(
    PY + ["-c", "raise SystemExit('boom')"],
    capture_output=True,
    text=True,
    encoding="utf-8",
)
check(
    8,
    "a string exit prints to stderr and codes 1",
    exit_child.returncode == 1 and "boom" in exit_child.stderr,
)
```

listing 194: chdir with a finally, os_exit dropping unflushed output, string exits coding 1 (source: python/samples/src/Ch16/os_faces.py, lines 56-99)

The boundary this book states rather than runs is the exec family. The functions exist on this build, the sample asserts `execv` and `spawnv` are present, but nothing here calls them: `os.exec*` replaces the calling process, which on windows interacts with console ownership and waiting in ways a gate run has no business provoking, and the posix wait macros are absent, `WIFEXITED` does not exist here, the sample checks that too. The general truth stands as a map: book 1, ch 11 (c23: the language, the machine, and the cloud, processes and program lifecycle) built the same parent verbs, wait, read the code, drop the reference, out of `CreateProcessW`, and this chapter's `Popen.wait` is the same discipline one abstraction lower.

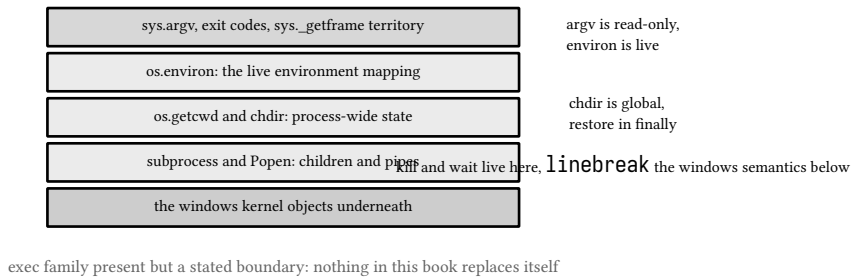


figure 70: the os layer as a stack around one running python

16.4. signals on windows

The signal module exists on windows and its surface is small. The page says which: “On Windows, `signal()` can only be called with `SIGABRT`, `SIGFPE`, `SIGILL`, `SIGINT`, `SIGSEGV`, `SIGTERM`, or `SIGBREAK`. A `ValueError` will be raised in any other case”, and `SIGBREAK` itself is the windows-only extra, “Interrupt from keyboard (CTRL + BREAK)”. The sample counts the seven names, checks the two console control event values, 0 and 1, and finds `NSIG` at 23. Defaults are real: “SIGINT is translated into a `KeyboardInterrupt` exception if the parent process has not changed it“, and `getsignal(SIGINT) is signal.default_int_handler` asserts the translator is installed:

```
accepted = [
    name
    for name in (
        "SIGABRT",
        "SIGFPE",
        "SIGILL",
        "SIGINT",
        "SIGSEGV",
        "SIGTERM",
        "SIGBREAK",
    )
    if hasattr(signal, name)
]
check(1, "seven signal names exist on windows", len(accepted) == 7)
check(
    2,
    "sigbreak is the windows-only extra",
    accepted[-1] == "SIGBREAK" and not hasattr(signal, "SIGHUP"),
)

check(
    3,
    "the console control events are the values 0 and 1",
    signal.CTRL_C_EVENT == 0 and signal.CTRL_BREAK_EVENT == 1,
)
check(4, "nsig counts one past the highest number", signal.NSIG == 23)

check(
    5,
    "sigint defaults to the KeyboardInterrupt translator",
    signal.getsignal(signal.SIGINT) is signal.default_int_handler,
)
```

listing 195: the seven names, the console event values, and the sigint default (source: python/samples/src/Ch16/signals_win.py, lines 22-53)

Then the windows truth about external kills, from the `os.kill` page: "The `signal.CTRL_C_EVENT` and `signal.CTRL_BREAK_EVENT` signals are special signals which can only be sent to console processes which share a common console window, e.g., some subprocesses. Any other value for `sig` will cause the process to be unconditionally killed by the `TerminateProcess` API, and the exit code will be set to `sig`." Every clause of that is a check. The sample arms a child with a `SIGTERM` handler, confirms it is running, kills it with `os.kill(pid, SIGTERM)`, and observes: the return code is 15, `sig` itself, and the handler never ran. There is no negotiation, because there is no signal, only termination:

```
# os.kill with any sig other than 0 or 1 is an unconditional
# TerminateProcess: exit code becomes sig, handlers never run
child = subprocess.Popen(
    PY
    + [
        "-c",
        "import signal, sys, time\n"
        "signal.signal(signal.SIGTERM, "
        "lambda s, f: print('HANDLER', flush=True))\n"
        "print('READY', flush=True)\n"
        "time.sleep(30)",
    ],
    stdout=subprocess.PIPE,
    stderr=subprocess.STDOUT,
    text=True,
    encoding="utf-8",
)
first = child.stdout.readline()
check(10, "the armed child is up and waiting", first.strip() == "READY")
time.sleep(0.5)
os.kill(child.pid, signal.SIGTERM)
try:
    rc = child.wait(timeout=10)
except subprocess.TimeoutExpired:
    child.kill()
    child.wait(timeout=5)
    raise SystemExit("FAIL armed child survived os.kill(pid, SIGTERM)")
rest = child.stdout.read()
check(
    11,
    "os.kill terminated the child with exit code sig",
    rc == signal.SIGTERM and rc == 15,
)
check(12, "the SIGTERM handler never ran", "HANDLER" not in rest)
```

listing 196: the armed child, the kill, exit code 15, and a handler that never ran (source: python/samples/src/Ch16/signals_win.py, lines 86-119)

What does work is in-process delivery. `signal.raise_signal` runs the installed handler synchronously in the main thread, the sample checks that directly, and the shutdown pattern the book teaches is built on it: install a handler that finishes work and exits with an intentional code, raise the signal from the process's own control flow, and the exit is orderly. One probed fact sharpens the boundary: in a child with piped stdio, which has no console, a `raise_signal` from a worker thread did not wake a sleeping main thread in this build, so the sample raises from the main thread and stays deterministic:

```
# the graceful pattern that works on windows: the signal is raised
# in-process, the handler runs in the main thread, exit with intent.
# run(timeout=...) kills and waits for the child if it ever stalls,
# so this probe is bounded by construction
try:
    graceful = subprocess.run(
        PY
        + [
            "-c",
            "import signal, time\n"
            "def stop(signum, frame):\n"
            "    print('graceful shutdown', flush=True)\n"
            "    raise SystemExit(3)\n"
            "signal.signal(signal.SIGTERM, stop)\n"
            "print('READY', flush=True)\n"
            "time.sleep(0.5)\n"
            "signal.raise_signal(signal.SIGTERM)\n"
            "time.sleep(10)\n",
        ],
        capture_output=True,
        text=True,
        encoding="utf-8",
        timeout=15,
    )
except subprocess.TimeoutExpired:
    raise SystemExit("FAIL graceful child did not exit on raise_signal")
check(
    13,
    "a raised sigterm runs the handler and exits cleanly",
    graceful.stdout.splitlines()[-1] == "graceful shutdown"
    and graceful.returncode == 3,
)
```

listing 197: the graceful pattern: handler in the main thread, intentional exit code (source: python/samples/src/Ch16/signals_win.py, lines 121-152)

NOTE: what ctrl+c would do to this gate

A real ctrl+c reaches every process attached to the console, the gate runner included, which is why no sample in this chapter sends `CTRL_C_EVENT`. The keyboard path is still the everyday one: sigint becomes `KeyboardInterrupt`, the exception chapters' `finally` blocks run, and the process exits non-zero. The difference from posix is the delivery vehicle: there, a kill signal can carry intent from outside, here the intent has to already be inside the process as a handler.

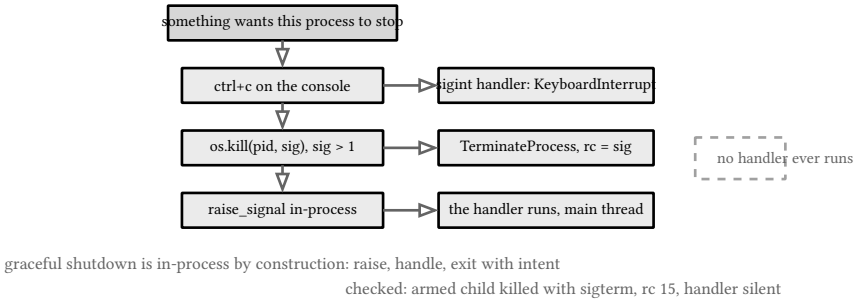


figure 71: signal delivery on windows as a decision: who sent it decides what happens

VERIFY: what a green run of this chapter proves

> The four samples print 47 `ok` lines: run's capture, exit codes, check, timeout, and encoding behavior, the popen lifecycle with a 4 mb threaded pipe transfer and the kill-then-wait cleanup, the os surfaces with the unflushed-output demonstration, and the signal truth table: seven names, the sigint default, the TerminateProcess kill with its exit code, and the in-process graceful pattern. All children were cpython 3.14.7 run with `-X utf8` on this machine.

sources: docs.python.org/3.14 library/subprocess (run, popen, communicate, frequently used arguments, security considerations, the 3.12 windows shell search change), library/os (os.kill windows paragraph), library/signal (windows signal list, SIGBREAK, default handlers), all accessed 2026-09-12. Sample behavior verified by `make verify-py`, 47 checks in chapter 16.

17. threads and the gil

Chapter 16 started separate processes and paid for every one with a spawn and a pickle. This chapter stays inside one process and starts threads, which share the address space the interpreter already built. The win and the trap come from the same fact: two threads can both touch the same list, so neither needs a copy, and neither can assume it is alone. The threading page fixes the contract in one paragraph, quoted here from the 3.14 documentation: “In CPython, due to the Global Interpreter Lock, only one thread can execute Python code at once (even though certain performance-oriented libraries might overcome this limitation).” The same paragraph ends with the trade this chapter measures instead of asserting: “threading is still an appropriate model if you want to run multiple I/O-bound tasks simultaneously.” Every behavioral claim below is a check in the 5 samples under `Ch17/`, 39 in total, run on the pinned 3.14.7 interpreter.

17.1. threads and the interpreter

A thread is `threading.Thread(target=fn, args=...)` plus `start()` and `join()`. The target runs in a new kernel thread with its own stack and its own ident, and `start()` returns immediately, so the creator needs `join()` to observe completion. The first sample keeps the whole discipline in one block:

```
    results = [None] * 4
    threads = []
    for i in range(4):
        t = threading.Thread(target=sleeper, args=(SLEEP, results, i), name=f"w{i}")
        if t.is_alive():
            raise SystemExit("FAIL an unstarted thread reports alive")
        threads.append(t)

    start = time.perf_counter()
    for t in threads:
        t.start()
    for t in threads:
        t.join()
    elapsed = time.perf_counter() - start
    if results != ["w0", "w1", "w2", "w3"]:
        raise SystemExit(f"FAIL workers recorded {results}")
    print("ok 3 four workers joined, each recorded its own name")
```

listing 198: four threads built, started, joined, each recording its own name (source: python/samples/src/Ch17/basics.py, lines 32-48)

The workers write `threading.current_thread().name` from inside the thread because the creator cannot assume it observed anything before the worker ran. After the joins, `results` is exactly `["w0", "w1", "w2", "w3"]`. The timing assertion in the same file is the io promise made concrete: 4 sleeps of 0.25 s finish in about 0.25 s of wall time, while the same 4 sleeps run sequentially in the main thread cost about 1 s:

```
    if not (SLEEP ≤ elapsed < 2 * SLEEP):
        raise SystemExit(f"FAIL the sleeps did not overlap: {elapsed:.3f}s")
    print(f"ok 4 four {SLEEP}s sleeps finished in {elapsed:.3f}s, not ~4x")

    start = time.perf_counter()
    for _ in range(4):
        time.sleep(SLEEP)
    sequential = time.perf_counter() - start
    if not (4 * SLEEP ≤ sequential < 5 * SLEEP):
        raise SystemExit(f"FAIL sequential sleeps took {sequential:.3f}s")
    if not (elapsed < sequential / 2):
        raise SystemExit(f"FAIL no speedup: {elapsed:.3f} vs {sequential:.3f}")
    print(f"ok 5 the same sleeps cost {sequential:.3f}s in one thread")
```

listing 199: overlapped waits versus sequential waits, one measured ratio (source: python/samples/src/Ch17/basics.py, lines 50-62)

The join page hands over one more contract this book leans on: “you must call `is_alive()` after `join()` to decide whether a timeout happened”. `join()` itself always returns `None`, so the sample times out a 1 s sleeper with a 0.05 s join, reads `is_alive()` as True, then joins again without a hit and requires it dead:

```
    slow = threading.Thread(target=sleeper, args=(1.0, results, 0), name="slow")
    slow.start()
    returned = slow.join(timeout=0.05)
    if returned is not None:
        raise SystemExit(f"FAIL join returned {returned!r}, not None")
    if not slow.is_alive():
        raise SystemExit("FAIL slow thread dead right after a 0.05s timeout")
    slow.join(timeout=5)
    if slow.is_alive():
        raise SystemExit("FAIL slow thread never finished")
    print("ok 6 a timed-out join left it alive, is_alive said so, join finished")
```

listing 200: a timed-out join leaves the thread alive, is_alive decides (source: python/samples/src/Ch17/basics.py, lines 64-74)

The daemon flag closes the section, because it is the most misunderstood line on the page. Two sentences carry the whole weight: “The entire Python program exits when no alive non-daemon threads are left”, and the note that follows: “Daemon threads are abruptly stopped at shutdown. Their resources (such as open files, database transactions, etc.) may not be released properly. If you want your threads to stop gracefully, make them non-daemonic and use a suitable signalling mechanism such as an `Event`.” A daemon thread is not a background worker you can forget about, it is a thread the interpreter reserves the right to kill mid-sentence. The sample only checks the flag mechanics, default False and settable at construction, and every worker in this book stays non-daemonic and exits through an `Event`, a queue shutdown, or a completed loop.

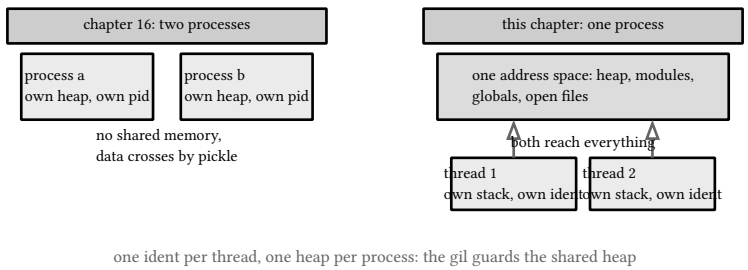


figure 72: what a process buys and what a thread buys: chapter 16’s two heaps against this chapter’s one

17.2. the gil

The Global Interpreter Lock is one mutex inside cpython that a thread must hold to run Python bytecode. The threading page states the consequence for cpu work plainly: “the GIL limits the performance gains of threading

when it comes to CPU-bound tasks, as only one thread can execute Python bytecode at a time." The interpreter does not even run its own scheduler for the handoffs, in the sys page's words: "which thread becomes scheduled at the end of the interval is the operating system's decision. The interpreter doesn't have its own scheduler." What the interpreter sells is the interval itself, `sys.setswitchinterval`, whose page reads: "This floating-point value determines the ideal duration of the 'timeslices' allocated to concurrently running Python threads", with the honest caveat that "the actual value can be higher". The default is 0.005 s, probed and asserted in the sample:

```
    if not sys._is_gil_enabled():
        raise SystemExit("FAIL this book runs the standard gil build")
    print("ok 2 the gil is enabled on this interpreter, probed")

    if sys.getswitchinterval() != 0.005:
        raise SystemExit(f"FAIL switch interval is {sys.getswitchinterval()}")
    print("ok 3 the default thread switch interval is 0.005s")

    sys.setswitchinterval(0.02)
    if sys.getswitchinterval() != 0.02:
        raise SystemExit("FAIL setswitchinterval did not take")
    sys.setswitchinterval(0.005)
    print("ok 4 setswitchinterval moved it to 0.02s and back")
```

listing 201: the gil flag on this interpreter, the default interval, one round trip (source: python/samples/src/Ch17/gil.py, lines 32-44)

`sys._is_gil_enabled()` is the probe that makes the rest of the file meaningful: everything after it runs on the standard gil build. The measurement is two loops of the same arithmetic, one in the main thread and one spread over 4 threads, each thread doing the full workload:

```
    expected = (LIMIT - 1) * LIMIT // 2
    start = time.perf_counter()
    single = burn(LIMIT)
    one_thread = time.perf_counter() - start
    if single != expected:
        raise SystemExit(f"FAIL burn computed {single}")
    print(f"ok 5 one thread ran {LIMIT} adds in {one_thread:.3f}s")

    results = [None] * 4
    threads = [
        threading.Thread(target=burner, args=(LIMIT, results, i)) for i in range(4)
    ]
    start = time.perf_counter()
    for t in threads:
        t.start()
    for t in threads:
        t.join()
    four_threads = time.perf_counter() - start
    if results != [expected] * 4:
        raise SystemExit(f"FAIL workers computed {results}")
    print("ok 6 four cpu threads each ran the loop, exact results")

    if four_threads < 2.0 * one_thread:
        raise SystemExit(
            f"FAIL cpu work overlapped: 4 x {one_thread:.3f}s finished "
            f"in {four_threads:.3f}s"
        )
    print(f"ok 7 four cpu threads took {four_threads:.3f}s: serialized")
```

listing 202: one thread versus 4 threads on identical cpu work, exact results (source: python/samples/src/Ch17/gil.py, lines 46-73)

The arithmetic is checked exactly both times, `(LIMIT-1)*LIMIT//2`, so the only variable is time: 4 threads take about 4 times as long as 1, because the bytecode cannot overlap. The guard asserts the wall time of the 4-thread run is at least twice the single-thread run, which is arithmetic over the gil's own promise. Then the same file flips the workload to `time.sleep`, and 4 sleeps of 0.25 s finish in about 0.25 s: a blocking call releases the gil, so waiting overlaps even when computing cannot.

```
    threads = [threading.Thread(target=sleeper, args=(SLEEP,)) for _ in range(4)]
    start = time.perf_counter()
    for t in threads:
        t.start()
    for t in threads:
        t.join()
    four_sleeps = time.perf_counter() - start
    if four_sleeps ≥ 2 * SLEEP:
        raise SystemExit(f"FAIL the sleeps serialized: {four_sleeps:.3f}s")
    print(f"ok 8 four {SLEEP}s sleeps took {four_sleeps:.3f}s: gil released")
```

listing 203: the same four threads sleeping instead: the gil is released (source: python/samples/src/Ch17/gil.py, lines 75-84)

NOTE: free-threaded builds exist, and this book does not run one

PEP 779, "Criteria for supported status for free-threaded Python", was accepted for 3.14: "Phase II would make the free-threaded build officially supported but still optional", following the phase I of 3.13 that made the build "available but explicitly experimental". On windows the free-threaded runtime is a separate binary with a `t` suffix, installed as `py install 3.14t` and run as `py -V:3.14t`, with `python3.14t.exe` as its alias. The threading page keeps the boundary visible: "As of Python 3.13, free-threaded builds can disable the GIL, enabling true parallel execution of threads, but this feature is not available by default (see PEP 703)." This book pins the standard build, and check 2 of `gil.py` asserts the gil is enabled before any serialization claim is made. The multiprocessing chapter, book 8, ch 18 (python 3.14, multiprocessing and process patterns), is how this book gets parallel cpu work on the standard build.

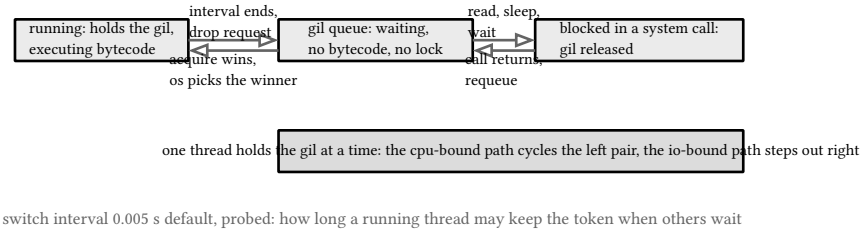


figure 73: one thread through the gil: the states and the two ways out of running

17.3. locks and queues

Shared state needs serialization, and the toolbox has exactly the pieces the samples exercise. `Lock` is the base one: one holder, `acquire` refuses everyone including the owner while held, `release` hands it back. `RLock` is the reentrant variant, the owner may acquire again and must release once per acquire. `Semaphore(n)` bounds concurrent holders to n, `Event` is a one-bit broadcast that any thread can set and any number can wait on, and `Condition` adds the predicate loop, wait until something becomes true. The mutual exclusion check runs 4 workers through 20000 locked increments each and watches the inside counter never exceed 1:

```
    lock = threading.Lock()
    if not lock.acquire(blocking=False):
        raise SystemExit("FAIL a fresh lock would not acquire")
    if lock.acquire(blocking=False):
        raise SystemExit("FAIL a held lock acquired again non-blockingly")
    lock.release()
    if not lock.acquire(blocking=False):
        raise SystemExit("FAIL the released lock would not acquire")
    lock.release()
    print("ok 2 lock: non-blocking acquire, refusal while held, release")
```

listing 204: the lock gate: one holder at a time, refused while held (source: python/samples/src/Ch17/locks.py, lines 62-71)

```
    counter = [0]
    stats = {"inside": 0, "violations": 0}
    workers = [
        threading.Thread(target=bump, args=(counter, lock, stats))
        for _ in range(WORKERS)
    ]
    for w in workers:
        w.start()
    for w in workers:
        w.join(timeout=10)
    if any(w.is_alive() for w in workers):
        raise SystemExit("FAIL a bump worker never finished")
    if stats["violations"] or stats["inside"]:
        raise SystemExit(f"FAIL mutual exclusion broken: {stats}")
    if counter[0] != WORKERS * INCREMENTS:
        raise SystemExit(f"FAIL counted {counter[0]}")
    print(f"ok 3 {WORKERS} workers x {INCREMENTS} locked adds: exact total")
```

listing 205: 80 000 locked increments, exact total, zero double entries (source: python/samples/src/Ch17/locks.py, lines 73-89)

The semaphore check runs 4 workers through a `BoundedSemaphore(2)` and asserts the observed peak of simultaneous holders never exceeds 2 while all 800 entries complete, the event check wakes a waiter with `set()` and resets it with `clear()`, and the condition check moves one payload through a one-slot handoff with `wait_for`:

```
    sem = threading.BoundedSemaphore(2)
    stat_lock = threading.Lock()
    stats = {"active": 0, "peak": 0, "done": 0}
    users = [
        threading.Thread(target=sem_user, args=(sem, stat_lock, stats, 200))
        for _ in range(WORKERS)
    ]
    for u in users:
        u.start()
    for u in users:
        u.join(timeout=10)
    if stats["done"] != WORKERS * 200:
        raise SystemExit(f"FAIL semaphore work done: {stats}")
    if stats["peak"] > 2:
        raise SystemExit(f"FAIL semaphore oversold: {stats}")
    print(f"ok 6 semaphore(2): peak holders {stats['peak']}, all work done")
```

listing 206: semaphore(2): peak holders bounded, all work done (source: python/samples/src/Ch17/locks.py, lines 123-138)

```
    cond = threading.Condition()
    slot = [None]
    out = []
    consumer = threading.Thread(target=cond_consumer, args=(cond, slot, out))
    consumer.start()
    producer = threading.Thread(target=cond_producer, args=(cond, slot))
    producer.start()
    consumer.join(timeout=5)
    producer.join(timeout=5)
    if out != [(True, "payload")]:
        raise SystemExit(f"FAIL condition handoff got {out}")
    print("ok 8 condition: wait_for handoff moved one payload")
```

listing 207: condition: wait_for the predicate, notify the other side (source: python/samples/src/Ch17/locks.py, lines 155-166)

Between threads, the safe channel is `queue.Queue`, whose page opens with the promise that matters: “The `Queue` class in this module implements all the required locking semantics.” The producer and the consumer never share a bare list, they share the queue, and the queue owns the lock. Completion has a protocol of its own: “Blocks until all items in the queue have been gotten and processed”, the `join` contract, paid back one `task_done()` per `get()`. Since 3.13 the shutdown has been first-class: “Put a `Queue` instance into a shutdown mode”, after which “Future calls to `put()` raise `ShutDown`” and a drained queue refuses `get()` the same way:

```
def drain_worker(q, counts, index):
    taken = 0
    while True:
        try:
            q.get(timeout=5)
        except queue.Empty, queue.ShutDown:
            break
        taken += 1
        q.task_done()
    counts[index] = taken
```

listing 208: the drain worker: get, task_done, exit on shutdown (source: python/samples/src/Ch17/queues.py, lines 15-24)

```
    q = queue.Queue()
    counts = [0, 0]
    workers = [
        threading.Thread(target=drain_worker, args=(q, counts, i)) for i in range(2)
    ]
    for w in workers:
        w.start()
    for i in range(ITEMS):
        q.put(i)
    q.join()
    q.shutdown()
    for w in workers:
        w.join(timeout=5)
    if any(w.is_alive() for w in workers):
        raise SystemExit("FAIL a drain worker never exited")
    if sum(counts) != ITEMS:
        raise SystemExit(f"FAIL consumed {counts}")
    print(f"ok 3 {ITEMS} items, task_done and join, shutdown released workers")

    try:
        q.put("late")
        raise SystemExit("FAIL put after shutdown did not raise")
    except queue.ShutDown:
        pass
    try:
        q.get(timeout=0.01)
        raise SystemExit("FAIL get on a shut-down empty queue did not raise")
    except queue.ShutDown:
        pass
    print("ok 4 a shut-down queue refuses put and get with queue.ShutDown")
```

listing 209: put 60, join, shutdown, then both refusals are checked (source: python/samples/src/Ch17/queues.py, lines 54-83)

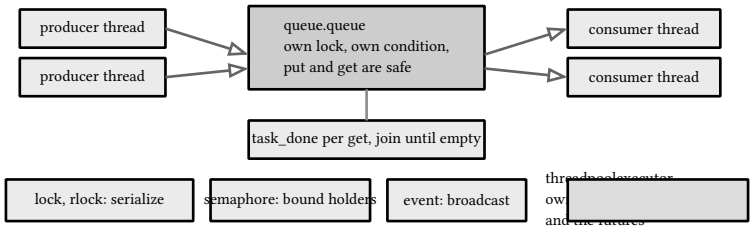
The thread pool wraps the same channel in a manager: `ThreadPoolExecutor` takes submit calls, hands each to a worker thread, and hands back a future. Its default width is documented arithmetic, "min(32, (os.process_cpu_count() or 1) + 4)" since 3.13, enough workers for io overlap without a thread per task. The sample checks ordered results from `submit`, the re-raise of a worker's exception at `result()`, and that `map` returns results in input order however the finishes interleave:

```
    with concurrent.futures.ThreadPoolExecutor(max_workers=4) as ex:
        futures = [ex.submit(sq, n) for n in range(10)]
        values = [f.result(timeout=5) for f in futures]
    if values != [n * n for n in range(10)]:
        raise SystemExit(f"FAIL submit results {values}")
    print("ok 6 threadpoolexecutor submit: 10 futures, exact results")
```

listing 210: submit and result: the future carries the value out (source: python/samples/src/Ch17/queues.py, lines 102-107)

```
    with concurrent.futures.ThreadPoolExecutor(max_workers=4) as ex:
        mapped = list(ex.map(staggered, range(10)))
    if mapped != list(range(10)):
        raise SystemExit(f"FAIL map order {mapped}")
    print("ok 8 map returned results in input order despite finish order")
```

listing 211: map keeps input order under staggered finish times (source: python/samples/src/Ch17/queues.py, lines 126-130)



the primitives guard shared state, the queue moves it, the pool schedules the workers

figure 74: the toolbox around one queue: producers, the lock inside, consumers, the pool under them

17.4. races demonstrated safely

A race needs two ingredients: a check and an act with a gap between them, and a second thread stepping into the gap. The last sample builds the gap on purpose, first as pure arithmetic with no threads at all. One thread's `counter += 1` is three micro-steps, read, add, store, and the simulator round-robins two threads one micro-step at a time:

```
def thread_steps(counter, hand):
    """one thread's rounds, each `counter += 1` as three micro-steps."""
    for _ in range(ROUNDS):
        hand["seen"] = counter[0]
        yield
        hand["seen"] += 1
        yield
        counter[0] = hand["seen"]
        yield


def locked_rounds(counter, lock, rounds):
    """one thread's rounds with the whole increment under one lock."""
    for _ in range(rounds):
        with lock:
            counter[0] += 1
        yield
```

listing 212: one increment as three yield points, the locked whole-increment variant (source: python/samples/src/Ch17/race.py, lines 16-32)

Both threads read 0, both store 1, and 6 intended adds leave 3 in the counter. The locked generator yields once per whole increment, the same scheduler interleaves, and the total is 6:

```
def simulate_unlocked():
    """round-robin the micro-steps of two threads, one step at a time."""
    counter = [0]
    left, right = {"seen": 0}, {"seen": 0}
    round_robin([thread_steps(counter, left), thread_steps(counter, right)])
    return counter[0]


def simulate_locked():
    """round-robin the same two threads at increment granularity."""
    counter = [0]
    lock = threading.Lock()
    left = locked_rounds(counter, lock, ROUNDS)
    right = locked_rounds(counter, lock, ROUNDS)
    round_robin([left, right])
    return counter[0]
```

listing 213: the same round-robin at step granularity loses 3, at increment granularity loses 0 (source: python/samples/src/Ch17/race.py, lines 48-63)

Real threads do not interleave on request, so the sample widens the gap instead of gambling on timing. The check-then-act worker reads the counter, spins a 64-iteration loop across compiler and interpreter checkpoint territory, then writes back. The tight worker keeps the whole update inside one stretch:

```
def bare_bump(counter):
    """check-then-act: read, work across loop back edges, then write."""
    for _ in range(WIDE_ROUNDS):
        seen = counter[0]
        spin = 0
        for j in range(64):
            spin += j
        counter[0] = seen + 1


def tight_bump(counter):
    """the whole update inside one stretch between loop back edges."""
    for _ in range(INCREMENTS):
        counter[0] += 1
```

listing 214: the wide window loses updates, the tight loop cannot be entered mid-update (source: python/samples/src/Ch17/race.py, lines 83-96)

With the switch interval pushed to a microsecond, the tight loop lost 0 updates out of 100000 in every run on this box, probed, while the wide window lost thousands of its 8000, a separation of three orders of magnitude that holds every run. The practical reading: absence of a crash is not correctness, it is a gap the scheduler has not stepped into yet, and the shape of the code decides the size of the gap. The fix is the lock around the whole span, and the sample prices it:

```
    old_interval = sys.getswitchinterval()
    sys.setswitchinterval(1e-6)
    try:
        expected = WORKERS * INCREMENTS
        counter = [0]
        workers = [
            threading.Thread(target=tight_bump, args=(counter,)) for _ in range(WORKERS)
        ]
        for w in workers:
            w.start()
        for w in workers:
            w.join(timeout=20)
        if any(w.is_alive() for w in workers):
            raise SystemExit("FAIL a tight worker never finished")
        if counter[0] != expected:
            raise SystemExit(f"FAIL tight loop lost {expected - counter[0]} updates")
        print(f"ok 5 the tight += 1 loop lost 0 of {expected} updates, probed")

        wide_expected = WORKERS * WIDE_ROUNDS
        counter = [0]
        workers = [
            threading.Thread(target=bare_bump, args=(counter,)) for _ in range(WORKERS)
        ]
        start = time.perf_counter()
        for w in workers:
            w.start()
        for w in workers:
            w.join(timeout=20)
        bare_elapsed = time.perf_counter() - start
    finally:
        sys.setswitchinterval(old_interval)
    if any(w.is_alive() for w in workers):
        raise SystemExit("FAIL an unlocked worker never finished")
    if counter[0] >= wide_expected:
        raise SystemExit("FAIL no updates lost with microsecond switching")
    print(
        f"ok 6 unlocked threads lost {wide_expected - counter[0]} of "
        f"{wide_expected} check-then-act adds"
    )
```

listing 215: forced switching, the two loop shapes measured, the loss asserted (source: python/samples/src/Ch17/race.py, lines 127-165)

The locked run of the wide workload completes all 8000 exactly and costs about the same wall time as the broken one, measured around 1 to 1 on this box, because 8000 lock acquisitions are cheap next to the work they guard. Correctness bought for nothing is the good trade, the one the c-os-cloud book made with its mutexes, book 1, ch 15 (c23: the language, the machine, and the cloud, threads and synchronization), and the one this chapter closes on.

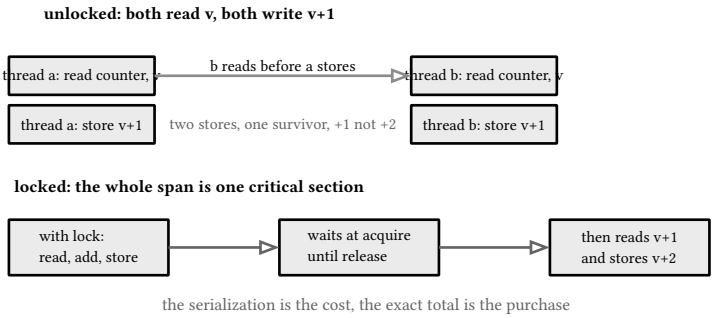


figure 75: the same two threads, unlocked and locked: where the second write lands

VERIFY: chapter 17 in the gate

The 5 samples under `python/samples/src/Ch17/` print 39 ok lines: `basics.py` 7, `gil.py` 8, `locks.py` 8, `queues.py` 9, `race.py` 7. Every timing claim is bounded on both sides, the deterministic race arithmetic is exact, and the interpreter pin asserts 3.14.7 before the first check of every file. A red line here is a real behavior change, not a flake: the lost-update check fails if the gil stops serializing, and the overlap checks fail if waits stop releasing it.

sources: docs.python.org/3.14/library/threading.html (the global interpreter lock paragraph, daemon and join semantics, the free-threading note pointing at PEP 703), docs.python.org/3.14/library/ sys.html (setswitchinterval, getswitchinterval, _is_gil_enabled), docs.python.org/3.14/library/queue.html (locking semantics, task_done and join, shutdown and the ShutDown exception, both added in 3.13), docs.python.org/3.14/library/concurrent.futures.html (threadpool executor default max_workers since 3.13), all accessed 2026-09-12; peps.python.org PEP 779 (phase II makes the free-threaded build officially supported but still optional, phase I experimental in 3.13) and docs.python.org/3.14/using/windows.html (free-threaded binaries via the t tag, python3.14t.exe alias, py -V:3.14t), accessed 2026-09-12; the 0.005 s default interval, the zero-loss tight loop, and the thousands lost through the widened window probed on this machine the same day. Sample behavior verified by `make verify-py`, 39 checks in chapter 17.

18. multiprocessing and process patterns

Chapter 17 proved the gil serializes bytecode and released the lock only for waits. The way around it on the standard build is the oldest one: more processes, each with its own interpreter and its own gil. The multiprocessing module sells that with three prices stamped on the receipt, and this chapter pays each one measured: on windows every child is spawned, never forked, every argument crosses as a pickle, and every byte of shared state needs a mechanism bought on purpose. The 4 samples under `Ch18/` carry 26 checks on the pinned 3.14.7 interpreter, and the threads-versus-processes decision at the end is measured, not asserted, book 8, ch 17 (python 3.14, threads and the gil).

18.1. processes on windows

The start-method table is short on this platform. Of the three methods the documentation lists, spawn is "The default on Windows and macOS", and only spawn exists here at all, asserted directly:

```
    if sys.platform ≠ "win32":
        raise SystemExit(f"FAIL this chapter is pinned to win32, got {sys.platform}")
    if mp.get_start_method() ≠ "spawn":
        raise SystemExit(f"FAIL start method is {mp.get_start_method()}")
    if sorted(mp.get_all_start_methods()) ≠ ["spawn"]:
        raise SystemExit(f"FAIL methods {mp.get_all_start_methods()}")
    print("ok 2 windows offers exactly one start method: spawn")
```

listing 216: windows offers exactly one start method, and it is spawn (source: python/samples/src/Ch18/spawn.py, lines 39-45)

The posix half of the table changed under this version. The fork entry carries the note: "Changed in version 3.14: This is no longer the default start method on any platform. Code that requires fork must explicitly specify that via `get_context()` or `set_start_method()`", with the reason one line up: "On POSIX platforms the default start method was changed from fork to forkserver to retain the performance but avoid common multithreaded process incompatibilities." Fork still exists on posix, but nothing defaults to it anywhere in 3.14, which makes spawn discipline a portable habit instead of a windows quirk.

Spawn means the child is a fresh interpreter that re-imports your main file before running the target. The sample watches it happen: the child reports its own pid, its name `Process-1`, and the module name it saw on import, `__mp_main__`, probed:

```
    parent_pid = os.getpid()
    q = mp.Queue()
    p = mp.Process(target=whoami, args=(q,))
    if p.exitcode is not None:
        raise SystemExit("FAIL an unstarted process already has an exitcode")
    p.start()
    child_pid, name, child_main, parent_of_child = q.get(timeout=5)
    p.join(timeout=10)
    q.close()
    q.join_thread()
    if p.exitcode != 0:
        raise SystemExit(f"FAIL clean child exitcode {p.exitcode}")
    if child_pid == parent_pid or parent_of_child != parent_pid:
        raise SystemExit(f"FAIL pid relation {(child_pid, parent_of_child)}")
    if name != "Process-1" or child_main != "__mp_main__":
        raise SystemExit(f"FAIL child identity {(name, child_main)}")
    print("ok 3 child ran with its own pid, imported this file as __mp_main__")
```

listing 217: the child's own report: new pid, parent recorded, this file re-imported (source: python/samples/src/Ch18/spawn.py, lines 47-63)

That re-import is why every sample in this chapter, and every program that starts processes on windows, guards its entry point with `if __name__ == "__main__":`. Without the guard the child's import would run the parent code again and fork the process tree recursively. The guard is not style, it is the spawn protocol, and the function the child runs must live at module level so the import can find it. The exit-code protocol follows the same discipline: a clean child leaves 0, a child that raises `SystemExit(3)` leaves 3, and a child that lets an exception escape leaves 1 with the traceback on stderr:

```
    p = mp.Process(target=exit_with, args=(3,))
    p.start()
    p.join(timeout=10)
    if p.exitcode != 3:
        raise SystemExit(f"FAIL systemexit(3) gave exitcode {p.exitcode}")
    print("ok 4 systemexit(3) in a child reads back as exitcode 3")

    p = mp.Process(target=blow_up)
    p.start()
    p.join(timeout=10)
    if p.exitcode != 1:
        raise SystemExit(f"FAIL uncaught valueerror gave exitcode {p.exitcode}")
    print("ok 5 an uncaught exception leaves the child at exitcode 1")
```

listing 218: exitcode 3 for systemexit(3), exitcode 1 for the uncaught exception (source: python/samples/src/Ch18/spawn.py, lines 65-77)

The second price is pickling. The Process page states it: "In general, all arguments to `Process` must be picklable", and the target crosses the same way, by qualified name, which is why a lambda cannot ride along while a module-level function can. The mechanics of what pickle will and will not take are chapter 15's subject, book 8, ch 15 (python 3.14, files and serialization), the boundary here is only observed:

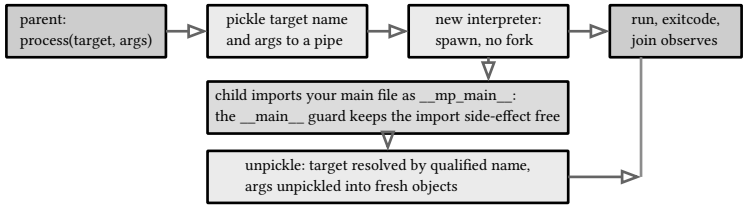
```
    try:
        pickle.dumps(lambda: None)
        raise SystemExit("FAIL a lambda pickled")
    except pickle.PicklingError, AttributeError:
        pass
    try:
        pickle.dumps(exit_with)
    except pickle.PicklingError:
        raise SystemExit("FAIL a module-level function did not pickle")
    print("ok 6 targets pickle by qualified name, a lambda cannot")
```

listing 219: the pickle boundary: a lambda refuses, a named function pickles by reference (source: python/samples/src/Ch18/spawn.py, lines 79-88)

The third price is isolation, and it cuts in the parent's favor: the child mutates the list it was handed and the parent's list is untouched, because the child unpickled its own copy:

```
    items = [1, 2, 3]
    p = mp.Process(target=mutate, args=(items,))
    p.start()
    p.join(timeout=10)
    if p.exitcode != 0:
        raise SystemExit(f"FAIL mutate child exitcode {p.exitcode}")
    if items != [1, 2, 3]:
        raise SystemExit(f"FAIL the parent list changed to {items}")
    print("ok 7 the child doubled its own copy, the parent list is untouched")
```

listing 220: the child doubled its copy, the parent's list never moved (source: python/samples/src/Ch18/spawn.py, lines 90-98)



anything in target or args that pickle refuses stops the spawn before the child runs

figure 76: what spawn does between start and run: the steps and where the guard sits

18.2. pools and executors

One process is plumbing, a pool is a schedule. The older `multiprocessing.Pool` and the newer `concurrent.futures.ProcessPoolExecutor` answer the same need, and the second is where the book and the capstone live, but the first is still everywhere in the field. The sample runs the same squaring workload through every spelling the pool offers and requires one answer:

```
    with mp.Pool(processes=2) as pool:
        plain = pool.map(square, range(20))
    if plain != [n * n for n in range(20)]:
        raise SystemExit(f"FAIL pool.map gave {plain}")
    print("ok 2 pool(2).map over 20 items, ordered results")

    with mp.Pool(processes=2) as pool:
        chunked = pool.map(square, range(20), chunksize=5)
        streamed = list(pool.imap(square, range(20)))
        paired = pool.starmap(add, [(a, 1) for a in range(10)])
    if chunked != plain or streamed != plain:
        raise SystemExit("FAIL chunked or streamed results differ")
    if paired != list(range(1, 11)):
        raise SystemExit(f"FAIL starmap gave {paired}")
    print("ok 3 chunksize 5, imap, and starmap agree with map")
```

listing 221: map, chunksize, imap, and starmap produce the same ordered list (source: python/samples/src/Ch18/pools.py, lines 31-45)

`chunksize` is the one knob with documented teeth. The executor's `map` "chops iterables into a number of chunks which it submits to the pool as separate tasks", and "For very long iterables, using a large value for chunksize can significantly improve performance compared to the default size of 1", because every task costs a pickle round trip and the default pays it per item. The executor half of the sample is the api this book uses forward:

```
    with ProcessPoolExecutor(max_workers=2) as ex:
        future = ex.submit(square, 9)
        if future.result(timeout=15) != 81 or not future.done():
            raise SystemExit("FAIL submit did not produce 81")
        ordered = list(ex.map(square, range(12), chunksize=4))
    if ordered != [n * n for n in range(12)]:
        raise SystemExit(f"FAIL executor map gave {ordered}")
    print("ok 5 executor submit and chunked map, ordered results")

    with ProcessPoolExecutor(max_workers=1) as ex:
        bad = ex.submit(fail, 3)
        try:
            bad.result(timeout=15)
            raise SystemExit("FAIL the worker exception vanished")
        except ValueError as exc:
            if str(exc) != "bad input 3":
                raise SystemExit(f"FAIL wrong exception {exc!r}")
    print("ok 6 a worker's valueerror re-raised at future.result()")
```

listing 222: submit and result, chunked map, and the worker exception re-raised in the parent (source: python/samples/src/Ch18/pools.py, lines 54-71)

The futures contract is the quiet improvement over `Pool.apply_async`: `submit` returns immediately, `result()` carries the value or re-raises the worker's exception in the parent with its message intact, and `done()` reports without waiting. The width of the pool is documented arithmetic, "If max_workers is None or not given, it will default to `os.process_cpu_count()`" since 3.13, with a windows lid: "On Windows, max_workers must be less than or equal to 61", and 0 is refused at construction. The failure mode that matters operationally is the broken pool, and it is documented behavior: "Should initializer raise an exception, all currently pending jobs will raise a `BrokenProcessPool`, as well as any attempt to submit more jobs to the pool":

```
    broken = ProcessPoolExecutor(max_workers=1, initializer=bad_initializer)
    try:
        broken.submit(square, 2).result(timeout=20)
        raise SystemExit("FAIL the broken pool produced a result")
    except BrokenProcessPool:
        pass
    finally:
        broken.shutdown()
    print("ok 8 a failing initializer surfaces as BrokenProcessPool")
```

listing 223: an initializer that raises turns every submit into BrokenProcessPool (source: python/samples/src/Ch18/pools.py, lines 80-88)

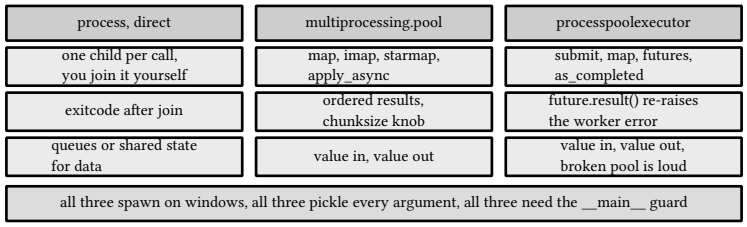


figure 77: the three ways to buy child processes, side by side

18.3. shared state

Processes start with nothing in common, so every shared number is a purchase. The cheapest is `Value` and `Array`, ctypes storage in shared memory wrapped with a lock you can take yourself: the sample's child takes `value.get_lock()` around each increment, and 2 children adding 1000 each land exactly 2000, the same mutual exclusion discipline chapter 17 proved for threads:

```
    total = mp.Value("i", 0)
    workers = [mp.Process(target=add_to, args=(total, 1000)) for _ in range(2)]
    for w in workers:
        w.start()
    for w in workers:
        w.join(timeout=15)
    if any(w.is_alive() for w in workers):
        raise SystemExit("FAIL a value worker never finished")
    if total.value != 2000:
        raise SystemExit(f"FAIL shared value holds {total.value}")
    print("ok 2 two children added 1000 each to one value, exact 2000")
```

listing 224: one shared value, two children, increments under its own lock (source: python/samples/src/Ch18/shared.py, lines 46-56)

The raw page is `multiprocessing.shared_memory`, a named block of bytes any process can attach to. Its lifecycle rule is the whole programming model: "shared memory blocks may outlive the original process that created them", so "the `close()` method should be called" when one process is done, "and the `unlink()` method should be called" exactly once, by whoever owns the deletion, when no process needs the block. Windows bends the tracker in your favor, "track is ignored on Windows, which has its own tracking and automatically deletes shared memory when all handles to it have been closed", probed here by the child attaching by name, reading the parent's bytes, and writing one back:

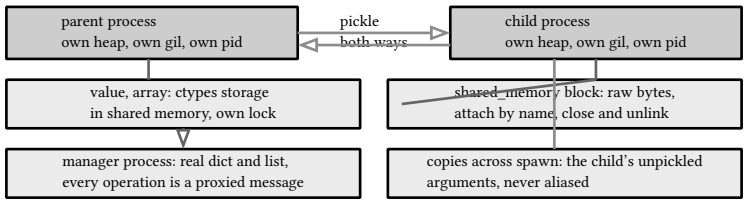
```
    block = shared_memory.SharedMemory(create=True, size=8)
    try:
        block.buf[:4] = b"pyth"
        reply_q = mp.Queue()
        w = mp.Process(target=echo_block, args=(block.name, 8, reply_q))
        w.start()
        seen = reply_q.get(timeout=10)
        w.join(timeout=15)
        reply_q.close()
        reply_q.join_thread()
        if w.exitcode != 0:
            raise SystemExit(f"FAIL memory child exitcode {w.exitcode}")
        if seen[:4] != b"pyth":
            raise SystemExit(f"FAIL the child read {seen!r}")
        if bytes(block.buf) != b"pyth\x00\x00\x00A":
            raise SystemExit(f"FAIL parent view {bytes(block.buf)!r}")
        print("ok 4 shared memory: child read parent bytes, wrote one back")
    finally:
        block.close()
        block.unlink()
```

listing 225: attach by name, read the parent's bytes, write one back, close and unlink (source: python/samples/src/Ch18/shared.py, lines 68-87)

Between the two sits `ShareableList`, fixed-length typed slots in one block, and at the top of the price range the `Manager`, a server process that owns real dict and list objects and hands out proxies that forward every operation over a pipe. A proxy behaves like the container, but every mutation is a message:

```
    with mp.Manager() as manager:
        shared_map = manager.dict()
        shared_log = manager.list()
        w = mp.Process(target=fill, args=(shared_map, shared_log))
        w.start()
        w.join(timeout=15)
        if w.exitcode != 0:
            raise SystemExit(f"FAIL manager child exitcode {w.exitcode}")
        if dict(shared_map) != {"answer": 42} or list(shared_log) != ["from-child"]:
            raise SystemExit(
                f"FAIL proxies hold {(dict(shared_map), list(shared_log))}"
            )
    print("ok 6 manager proxies: the child's writes are visible in the parent")
```

listing 226: manager proxies: the child's writes visible in the parent (source: python/samples/src/Ch18/shared.py, lines 103-115)



cheapest at the left, most general at the bottom: every step up costs a message or a syscall per access

figure 78: where each flavor of shared state lives, against the private heaps

18.4. process patterns

The patterns are short once the prices are paid. The worker pool is `map` over an input list, the offload shape the capstone of book 8, ch 40 (python 3.14, capstone: ingest and analysis pipeline, part 1) builds its analytics stage on. The queue pipeline is two processes and two queues, producer to consumer, with a `None` sentinel to stop the consumer without killing it:

```
def produce(out_q, count):
    for i in range(count):
        out_q.put(i)
    out_q.put(None)


def consume(in_q, out_q):
    taken = 0
    while True:
        item = in_q.get()
        if item is None:
            break
        out_q.put(item * item)
        taken += 1
    out_q.put(taken)
```

listing 227: produce to a sentinel, consume until it, count what passed (source: python/samples/src/Ch18/patterns.py, lines 21-35)

The wiring drains results before joining, the deadlock discipline from chapter 16's pipes applied to queues, and the sentinel keeps the consumer's exit ordinary:

```
    work_q = mp.Queue()
    done_q = mp.Queue()
    consumer = mp.Process(target=consume, args=(work_q, done_q))
    producer = mp.Process(target=produce, args=(work_q, PIPE_ITEMS))
    consumer.start()
    producer.start()
    producer.join(timeout=15)
    squares = [done_q.get(timeout=5) for _ in range(PIPE_ITEMS)]
    taken = done_q.get(timeout=5)
    consumer.join(timeout=15)
    work_q.close()
    done_q.close()
    if producer.exitcode != 0 or consumer.exitcode != 0:
        raise SystemExit("FAIL a pipeline process failed")
    if squares != [i * i for i in range(PIPE_ITEMS)] or taken != PIPE_ITEMS:
        raise SystemExit(f"FAIL pipeline results {squares[:3]}... taken {taken}")
    print(f"ok 3 queue pipeline: {PIPE_ITEMS} squares, one sentinel, drained")
```

listing 228: drain the results, then join, then close: 30 squares in order (source: python/samples/src/Ch18/patterns.py, lines 55-71)

Scatter-gather is the pool pattern with the data pre-shaped: split the input into batches, one task per batch, gather the partials in order, and the recombination is checked against the sequential answer:

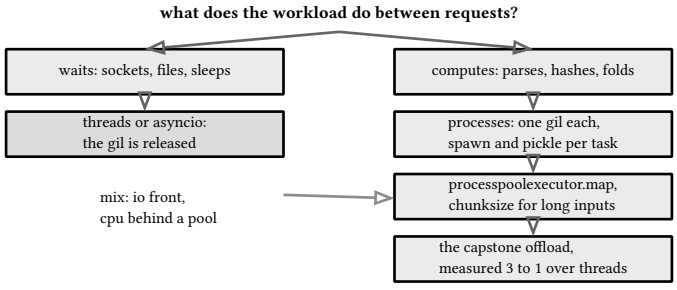
```
    data = list(range(100))
    batches = [data[i : i + 25] for i in range(0, 100, 25)]
    with ProcessPoolExecutor(max_workers=4) as ex:
        partials = list(ex.map(sum, batches))
    if partials != [sum(b) for b in batches] or sum(partials) != sum(data):
        raise SystemExit(f"FAIL scatter-gather gave {partials}")
    print("ok 4 scatter-gather: 4 batches of 25, gathered in order")
```

listing 229: 4 batches of 25, gathered order exact against the sequential sum (source: python/samples/src/Ch18/patterns.py, lines 73-79)

The decision the chapter exists for is measured last. The same burn loop from chapter 17 runs 4 times through a thread pool and 4 times through a process pool, and on this box the processes win by about 3 to 1, 1.633 s threaded against 0.548 s in processes in the recorded run, spawn cost included:

```
    tasks = [LIMIT] * 4
    burn_total = (LIMIT - 1) * LIMIT // 2
    start = time.perf_counter()
    with ThreadPoolExecutor(max_workers=4) as ex:
        threaded_results = list(ex.map(burn, tasks))
    threaded = time.perf_counter() - start
    if threaded_results != [burn_total] * 4:
        raise SystemExit("FAIL thread results differ")
    start = time.perf_counter()
    with ProcessPoolExecutor(max_workers=4) as ex:
        process_results = list(ex.map(burn, tasks))
    in_processes = time.perf_counter() - start
    if process_results != [burn_total] * 4:
        raise SystemExit("FAIL process results differ")
    if not (in_processes < threaded):
        raise SystemExit(
            f"FAIL no offload win: {threaded:.3f}s threaded vs "
            f"{in_processes:.3f}s in processes"
        )
```

listing 230: identical cpu work, threads serialized by the gil, processes parallel (source: python/samples/src/Ch18/patterns.py, lines 81-99)



measured on this box, chapter 18 check 5 of patterns.py: 1.633 s threaded, 0.548 s in processes

figure 79: the choice, and where each workload goes

VERIFY: chapter 18 in the gate

The 4 samples under `python/samples/src/Ch18/` print 26 ok lines: `spawn.py` 7, `pools.py` 8, `shared.py` 6, `patterns.py` 5. Every sample that starts processes carries the `__main__` guard and passes only module-level targets, every join has a timeout, and every shared block is closed and unlinked. The two deliberate failures, the crashing child and the broken initializer, print their tracebacks on stderr by design: their exit codes and the `BrokenProcessPool` are what the checks assert.

sources: docs.python.org/3.14/library/multiprocessing.html (start methods and the two 3.14 fork notes, picklability of Process arguments, the __main__ guard requirement), docs.python.org/3.14/library/ concurrent.futures.html (processpoolexecutor max_workers default and the windows 61 cap, chunksize guidance, the 3.14 start-method note, initializer and BrokenProcessPool), docs.python.org/3.14/library/ multiprocessing.shared_memory.html (close and unlink lifecycle, the windows tracking note, shareablelist), all accessed 2026-09-12; the __mp_main__ import name, the exitcode ladder, and the 3 to 1 thread-to-process ratio probed on this machine the same day. Sample behavior verified by `make verify-py`, 26 checks in chapter 18.

19. operational plumbing

Programs that outlive the terminal session need the same handful of parts regardless of domain: a command line worth typing, logs that survive the process, timestamps that mean the same thing tomorrow, and identifiers and digests that do not collide. This chapter assembles that kit from the standard library only: argparse for the command line, logging for the record, datetime and zoneinfo for time, os.environ for configuration, secrets, hashlib, and uuid for identity, plus the two field tools, pdb and faulthandler, that earn their one paragraph each. The 4 samples under `Ch19/` carry 28 checks on the pinned 3.14.7 interpreter, and the time-zone check documents a machine fact this book treats as a boundary rather than working around.

19.1. argparse

The pattern is one parser, subcommands, and the exit codes the parser owns. The subparsers page frames the shape: “Many programs split up their functionality into a number of subcommands”, and the construction is four calls:

```
def build_parser():
    parser = ArgumentParser(prog="tool")
    sub = parser.add_subparsers(dest="command", required=True)
    adder = sub.add_parser("add", help="add an entry")
    adder.add_argument("name")
    adder.add_argument("count", type=int)
    adder.add_argument("--tag", choices=("red", "blue"), default="red")
    server = sub.add_parser("serve", help="serve entries")
    server.add_argument("--port", type=int, default=8080)
    return parser
```

listing 231: one parser, two subcommands, a typed positional, a constrained choice, a default (source: python/samples/src/Ch19/cli.py, lines 12-21)

Three behaviors are doing the work. The `type=int` callable converts the string before you see it, and the page pins the failure contract: “If the function raises `ArgumentTypeError`, `TypeError`, or `ValueError`, the exception is caught and a nicely formatted error message is displayed.” The `choices` check runs after that conversion, “choices are checked after any type conversions have been performed”, so the tuple compares ints to ints. And the subparser was built with `required=True`, so no subcommand at all is an error too. All three failures follow one documented exit discipline: “it will print a message to `sys.stderr` and exit with a status code of 2”:

```
    bad_inputs = (
        ["add", "widget", "x"],
        ["add", "widget", "1", "--tag", "green"],
        [],
    )
    for argv in bad_inputs:
        sink = io.StringIO()
        with contextlib.redirect_stderr(sink):
            exited = "did-not-exit"
            try:
                build_parser().parse_args(argv)
            except SystemExit as exc:
                exited = exc.code
        if exited != 2 or "usage" not in sink.getvalue():
            raise SystemExit(f"FAIL {argv} gave {exited!r} and {sink.getvalue()!r}")
    print("ok 4 bad type, bad choice, missing subcommand: exit 2, usage shown")
```

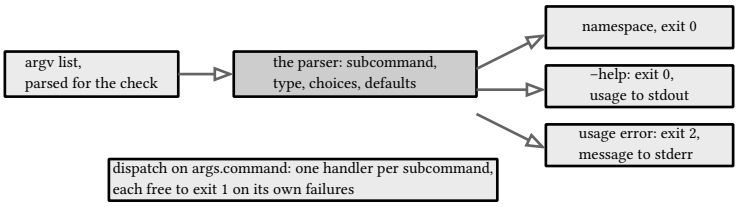
listing 232: bad type, bad choice, missing subcommand: each exits 2 with usage on stderr (source: python/samples/src/Ch19/cli.py, lines 47-62)

The help flag is free, “By default a help action is automatically added to the parser”, and it exits 0, probed here with stdout captured so the check can read the listing it printed:

```
    sink = io.StringIO()
    with contextlib.redirect_stdout(sink):
        exited = "did-not-exit"
        try:
            build_parser().parse_args(["--help"])
        except SystemExit as exc:
            exited = exc.code
    text = sink.getvalue()
    if exited != 0 or "add" not in text or "serve" not in text:
        raise SystemExit(f"FAIL help gave {exited!r}")
    print("ok 5 --help exited 0 for free, both subcommands listed")
```

listing 233: –help exits 0 and lists every subcommand it built (source: python/samples/src/Ch19/cli.py, lines 64-74)

The tool shape that falls out is a function that builds the parser, a `main` that parses and dispatches on `args.command`, and exit codes that mean one thing each: 2 for usage errors the parser caught, 1 for runtime failures the program caught, 0 for success. The verify gate of this whole book leans on exactly that convention, and the capstone’s cli chapter, book 8, ch 41 (python 3.14, capstone: ingest and analysis pipeline, part 2), builds on this sample’s shape.



2 is the parser’s, 1 is the program’s, 0 is earned

figure 80: where one argv list goes: dispatch and the three exits

19.2. logging

The module is a tree before it is anything else. “Multiple calls to `getLogger()` with the same name will always return a reference to the same Logger object”, and the names are dotted paths: “Loggers that are further down in the hierarchical list are children of loggers higher up in the list”, with every logger a descendant of the root. The recommended construction `logging.getLogger(__name__)` makes the tree match the package tree of book 8, ch 11 (python 3.14, modules, packages, and imports) for free:

```
    if logging.getLogger("app.worker") is not logging.getLogger("app.worker"):
        raise SystemExit("FAIL getLogger built two loggers for one name")
    root = logging.getLogger()
    app = logging.getLogger("app")
    worker = logging.getLogger("app.worker")
    if worker.parent is not app or app.parent is not root:
        raise SystemExit("FAIL the dot name did not build the tree")
    print("ok 2 one logger per name, app.worker hangs under app under root")
```

listing 234: one logger per name, app.worker hanging under app under root (source: python/samples/src/Ch19/logging_tree.py, lines 24-31)

A level is a number, and the ladder is short: DEBUG 10, INFO 20, WARNING 30, ERROR 40, CRITICAL 50, asserted exactly. A logger left at NOTSET borrows upward, “the hierarchy is traversed towards the root until a value other than NOTSET is found”, and the root itself starts armed at WARNING:

```
    app.setLevel(logging.INFO)
    if worker.level != logging.NOTSET:
        raise SystemExit("FAIL the child reported its own level")
    if worker.getEffectiveLevel() != logging.INFO:
        raise SystemExit("FAIL the effective level did not climb")
    if not worker.isEnabledFor(logging.INFO) or worker.isEnabledFor(logging.DEBUG):
        raise SystemExit("FAIL isEnabledFor disagreed with the ladder")
    print("ok 4 a notset child inherits info from its parent")
```

listing 235: a notset child inherits the parent's effective level (source: python/samples/src/Ch19/logging_tree.py, lines 44-51)

A record's path is decided by two different gates, and confusing them is the classic logging bug. The logger's effective level decides whether the record exists at all: `worker.debug` under an INFO parent produces nothing anywhere. Then, for records that exist, "events logged to this logger will be passed to the handlers of higher level (ancestor) loggers", and each handler applies its own level independently, so an ERROR-tinted handler drops a warning the WARNING-tinted one kept. `propagate = False` cuts the climb, which is how one subtree logs locally without duplicating to the root:

```
    cap_root = Capture()
    cap_worker = Capture()
    root.addHandler(cap_root)
    worker.addHandler(cap_worker)
    try:
        worker.warning("tree message")
        if len(cap_worker.records) != 1 or len(cap_root.records) != 1:
            raise SystemExit("FAIL the record did not reach both handlers")
        cap_root.records.clear()
        cap_worker.records.clear()
        worker.debug("dropped message")
        if cap_root.records or cap_worker.records:
            raise SystemExit("FAIL debug passed the effective info level")
        print("ok 5 one warning reached both handlers, debug was filtered")

        worker.propagate = False
        cap_root.records.clear()
        worker.warning("local only")
        if len(cap_worker.records) != 1 or cap_root.records:
            raise SystemExit("FAIL propagate=false leaked to the root")
        worker.propagate = True
        print("ok 6 propagate=false stopped the climb to the root handler")
```

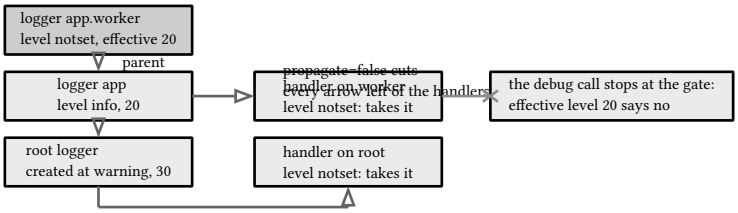
listing 236: the same warning through two handlers, the filtered debug, the stopped climb (source: python/samples/src/Ch19/logging_tree.py, lines 53-74)

The structured-field habit completes the picture. The `extra` parameter is "a dictionary which is used to populate the `__dict__` of the LogRecord created for the logging event with user-defined attributes", with one reservation worth quoting: "The keys in the dictionary passed in extra should not clash with the keys used by the logging system." The sample attaches `request_id` and `attempts` and reads them back off the record:

```
    cap = Capture()
    local = logging.getLogger("app.extra")
    local.setLevel(logging.INFO)
    local.addHandler(cap)
    try:
        local.info("request finished", extra={"request_id": "r-17", "attempts": 2})
        rec = cap.records[0]
        if rec.request_id ≠ "r-17" or rec.attempts ≠ 2:
            raise SystemExit("FAIL extra fields did not land")
        if rec.name ≠ "app.extra" or rec.levelname ≠ "INFO":
            raise SystemExit(f"FAIL record header {rec.name} {rec.levelname}")
    finally:
        local.removeHandler(cap)
    print("ok 8 extra fields landed on the logrecord with name and level")
```

listing 237: extra fields on the record, name and levelname intact (source: python/samples/src/Ch19/logging_tree.py, lines 85-98)

Formatters are the rendering half, a format string over record attributes like `%(name)s` and `%(levelname)s`. The sample keeps a capture handler instead, because an assertion wants the record, not the string.



logger levels decide existence, handler levels decide landing, propagate decides reach

figure 81: one warning's path down the tree: logger levels then handler levels then propagate

19.3. datetime and zoneinfo

The distinction that organizes everything is awareness. "Date and time objects may be categorized as 'aware' or 'naive' depending on whether or not they include time zone information", an aware object "represents a specific moment in time that is not open to interpretation", while for a naive one "whether a naive object represents Coordinated Universal Time (UTC), local time, or some other time zone is purely up to the program":

```
    naive = dt.datetime(2026, 9, 12, 10, 0, 0)
    aware = naive.replace(tzinfo=dt.timezone.utc)
    if naive.tzinfo is not None or naive.utcoffset() is not None:
        raise SystemExit("FAIL the naive datetime reports an offset")
    if aware.utcoffset() ≠ dt.timedelta(0):
        raise SystemExit("FAIL the aware datetime lost utc")
```

listing 238: naive carries no offset, replace() makes it aware (source: python/samples/src/Ch19/datetimes.py, lines 15-21)

The utc spellings were straightened in this decade. `datetime.UTC` is an alias for the singleton, added in 3.11, and `utcnow()` is the deprecated trap: "Deprecated since version 3.12: Use `datetime.now()` with `UTC` instead", because it hands back a naive object. The page's warning states the preference outright: "the recommended way to create an object representing the current time in UTC is by calling `datetime.now(timezone.utc)`":

```
if dt.UTC is not dt.timezone.utc:
    raise SystemExit("FAIL datetime.utc is not the utc singleton")
now = dt.datetime.now(dt.UTC)
if now.tzinfo is not dt.UTC:
    raise SystemExit("FAIL now(dt.UTC) came back naive")
print("ok 3 datetime.utc is the singleton, now() carries it")

with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    dt.datetime.utcnow()
if not any(issubclass(w.category, DeprecationWarning) for w in caught):
    raise SystemExit("FAIL utcnow did not warn")
```

listing 239: the utc singleton, an aware now, utcnow captured warning (source: python/samples/src/Ch19/datetimes.py, lines 23-35)

Parsing grew teeth in 3.11 and the sample leans on them: `fromisoformat` now accepts "any valid ISO 8601 format", where "Previously, this method only supported the formats that could be emitted by `date.isoformat()` or `datetime.isoformat()`". The `Z` suffix becomes real utc, `20260912T100000` parses, week dates parse, and offsets come back as offsets:

```
parsed_z = dt.datetime.fromisoformat("2026-09-12T10:00:00Z")
parsed_basic = dt.datetime.fromisoformat("20260912T100000")
parsed_week = dt.datetime.fromisoformat("2026-W37-6")
parsed_off = dt.datetime.fromisoformat("2026-09-12T10:00:00+05:30")
if parsed_z != aware or parsed_basic != naive:
    raise SystemExit("FAIL fromisoformat variants disagree")
if parsed_week != dt.datetime(2026, 9, 12):
    raise SystemExit(f"FAIL the week form parsed to {parsed_week}")
if parsed_off.utcoffset() != dt.timedelta(hours=5, minutes=30):
    raise SystemExit("FAIL the +05:30 offset did not parse")
```

listing 240: z, basic, week, and offset forms, all through fromisoformat (source: python/samples/src/Ch19/datetimes.py, lines 37-47)

Fixed-offset zones and the timestamp round trip carry the aware side, and then the chapter hits the windows boundary the docs draw. The zoneinfo module "does not directly provide time zone data, and instead pulls time zone information from the system time zone database or the first-party PyPI package tzdata, if available", and "Some systems, including notably Windows systems, do not have an IANA database available", so "it is recommended to declare a dependency on tzdata". Without either source, "all calls to `ZoneInfo` will raise `ZoneInfoNotFoundError`", and on this platform that sentence is literal: a bare windows interpreter has no database at all, so even `ZoneInfo("UTC")` raises, probed. This book's two run environments sit on opposite sides of the line. The bare interpreter has no data at all. The verify venv has tzdata, and not because anyone taught it: pandas declares tzdata a hard dependency, probed with `pip show tzdata` in that venv, version 2026.4, required-by pandas, so any environment carrying the pandas pin of book 8, ch 25 (python 3.14, pandas) carries named zones with it. A check that expected one side would be environment-dependent, the failure this book's rules ban, so the sample asks `find_spec` which world it is in and asserts that world's documented behavior:

```
    if importlib.util.find_spec("tzdata") is None:
        for key in ("UTC", "Asia/Tokyo"):
            try:
                zoneinfo.ZoneInfo(key)
                raise SystemExit(f"FAIL {key} resolved with no tz data")
            except zoneinfo.ZoneInfoNotFoundError:
                pass
        print("ok 8 bare interpreter: no iana db, no tzdata, all calls raise")
    else:
        hcm = zoneinfo.ZoneInfo("Asia/Ho_Chi_Minh")
        if hcm.utcoffset(naive) != dt.timedelta(hours=7):
            raise SystemExit(f"FAIL ho chi minh offset {hcm.utcoffset(naive)}")
        local = aware.astimezone(hcm)
        if (local.hour, local.utcoffset()) != (17, dt.timedelta(hours=7)):
            raise SystemExit(f"FAIL astimezone produced {local}")
        if local.astimezone(dt.UTC) != aware:
            raise SystemExit("FAIL the astimezone round trip drifted")
        print("ok 8 tzdata package present: ho chi minh plus 7, round trip held")
```

listing 241: the tz boundary resolved by find_spec: every call raises bare, ho chi minh plus 7 with tzdata (source: python/samples/src/Ch19/datetimes.py, lines 61-78)

Both branches are real assertions, and the ok line names which one ran. The bare branch requires the documented exception from every key it tries, and the tzdata branch pins the offset exactly, moves 10:00 utc to 17:00 plus seven, and round-trips back through utc, so a green exit always means a documented behavior was verified, never that the check closed its eyes.

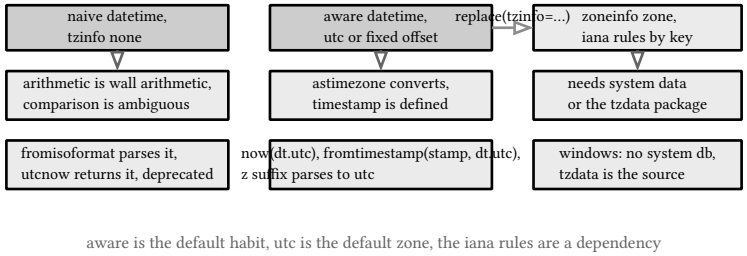


figure 82: the datetime taxonomy: what each kind knows and where it can go

19.4. env, secrets, hashing

Configuration arrives through the environment, and chapter 16 already owns the process side, book 8, ch 16 (python 3.14, os, subprocess, and signals). The operational half is small: `os.environ` is a live mapping, reads through `os.getenv`, and on windows `os.path.expandvars` expands `%NAME%` spellings, probed:

```
    key = "PYBOOK_CH17_PROBE"
    os.environ[key] = "42"
    if os.getenv(key) != "42" or os.environ[key] != "42":
        raise SystemExit("FAIL the environment variable did not land")
    if os.path.expandvars(f"value=%{key}%") != "value=42":
        raise SystemExit("FAIL percent expansion failed")
    del os.environ[key]
    if os.getenv(key) is not None:
        raise SystemExit("FAIL the variable survived deletion")
    print("ok 2 environ get, set, percent expansion, delete")
```

listing 242: set, read, percent-expand, delete, all through the one mapping (source: python/samples/src/Ch19/operational.py, lines 21-30)

Identity and secrecy are separate purchases and the docs separate them sharply. The secrets page: “The `secrets` module is used for generating cryptographically strong random numbers suitable for managing data such as passwords, account authentication, security tokens, and related secrets”, and in particular “secrets should be used in preference to the default pseudo-random number generator in the `random` module, which is designed for modelling and simulation, not security or cryptography.” The sample shows both faces: two `Random(1401)` objects repeat their sequence exactly, while `token_hex` never repeats and takes no seed at all:

```
    hexed = secrets.token_hex(8)
    if len(hexed) ≠ 16 or secrets.token_hex(8) == hexed:
        raise SystemExit("FAIL token_hex repeats or has the wrong length")
    if len(secrets.token_urlsafe(16)) ≠ 22:
        raise SystemExit("FAIL token_urlsafe length changed")
    if not 0 ≤ secrets.randbelow(100) < 100:
        raise SystemExit("FAIL randbelow left the range")
    print("ok 3 token_hex 16 chars, token_urlsafe 22, randbelow bounded")

    left = random.Random(1401)
    right = random.Random(1401)
    if left.random() ≠ right.random():
        raise SystemExit("FAIL the seeded random did not repeat")
    if left.sample(range(52), k=5) ≠ right.sample(range(52), k=5):
        raise SystemExit("FAIL the seeded sample did not repeat")
    print("ok 4 the seeded random repeats; secrets has no seed argument")
```

listing 243: tokens from secrets, repeatable draws from a seeded random (source: python/samples/src/Ch19/operational.py, lines 32-47)

Digests are the other identity axis. sha256 over empty input is a pinned constant, blake2b takes a `digest_size`, and since 3.11 `hashlib.file_digest` hashes an open file in one call, the same digest as the streaming version over the same bytes, the file-reading half being book 8, ch 15 (python 3.14, files and serialization) territory:

```
    if hashlib.sha256(b"").hexdigest() ≠ EMPTY_SHA256:
        raise SystemExit("FAIL the empty-input sha256 constant changed")
    if len(hashlib.blake2b(digest_size=16).hexdigest()) ≠ 32:
        raise SystemExit("FAIL blake2b digest_size did not hold")
    with tempfile.TemporaryDirectory() as tmp:
        path = os.path.join(tmp, "blob.bin")
        with open(path, "wb") as fh:
            fh.write(b"hello")
        with open(path, "rb") as fh:
            on_disk = hashlib.file_digest(fh, "sha256").hexdigest()
    if on_disk ≠ hashlib.sha256(b"hello").hexdigest():
        raise SystemExit("FAIL file_digest disagrees with the streaming hash")
    print("ok 5 sha256 constant, blake2b size, file_digest on disk")
```

listing 244: the empty-input constant, a sized blake2b, file_digest on disk (source: python/samples/src/Ch19/operational.py, lines 49-61)

Identifiers have kinds. `uuid4` is random and never repeats, `uuid5` is a deterministic hash of a namespace and a name, `uuid3` the same over md5, and `uuid1` stamps the host and clock, which is why its string leaks machine information into logs. The sample pins versions, stability, and the 36-character form. The last two tools cost one paragraph because they cost nothing to carry: `faulthandler.enable()` installs a handler that dumps native tracebacks on hard crashes, off by default and flipped in one check, and `python -m pdb script.py`, or `breakpoint()` mid-run, drops into the standard debugger when a print statement will not reach the failure.

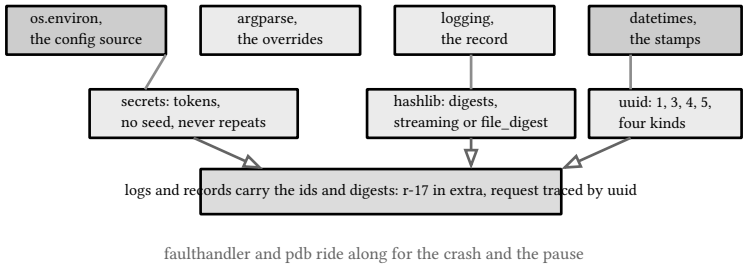


figure 83: the operational toolkit as one pipeline: config in, identity out

VERIFY: chapter 19 in the gate

The 4 samples under `python/samples/src/Ch19/` print 28 ok lines: `cli.py` 5, `logging_tree.py` 8, `datetimes.py` 8, `operational.py` 7. The argparse error paths capture stderr so the gate output stays clean, the logging sample attaches and removes its handlers instead of mutating the root permanently, and the zoneinfo check resolves its environment through `find_spec` and asserts the documented behavior of whichever side it lands on, every call raising on a bare interpreter, ho chi minh plus 7 with a utc round trip under tzdata, so it stays honest in both.

sources: docs.python.org/3.14/library/argparse.html (error exits with status 2, subcommands, the type-callable exception contract, choices after conversion, the automatic help action), docs.python.org/3.14/library/logging.html (getLogger identity, the name hierarchy, propagate, notset delegation, the root's warning level, the extra parameter and its key reservation), docs.python.org/3.14/library/datetime.html (aware versus naive, the UTC alias added 3.11, the utcnow deprecation since 3.12 and the recommended now(timezone.utc), fromisoformat's 3.11 expansion), docs.python.org/3.14/library/zoneinfo.html (data sources, the windows absence of an iana database, the tzdata recommendation, ZoneInfoNotFoundError), docs.python.org/3.14/library/secrets.html (cryptographically strong numbers, preference over the random module's default generator), all accessed 2026-09-12; the exit-0 help run, the percent expansion, the 22-character token_urlsafe, the bare interpreter's ZoneInfoNotFoundError, the venv's tzdata-backed ho chi minh offset, and pip's required-by pandas line probed on this machine the same day. Sample behavior verified by `make verify-py`, 28 checks in chapter 19.

20. testing with unittest

The whole book runs on the standard library, so the whole book tests on it too. unittest is the framework, `unittest.mock` is the seam it supplies for code that talks to things a test must not touch, and doctest is the third leg, examples that live in the documentation and fail the suite when the documentation lies. This chapter builds all three against one pinned interpreter and then shows the part that usually stays invisible: a suite run programmatically, its result object inspected, its exit code asserted. The 4 samples under `Ch20/` carry 27 checks, and the last one is a miniature of the verify gate this book itself runs under, the same machinery the capstone suite of book 8, ch 40 (python 3.14, capstone: ingest and analysis pipeline, part 1) rides.

20.1. unittest anatomy

The unit is `unittest.TestCase`, and the loader's contract picks the methods: "Return a suite of all test cases contained in the TestCase-derived testCaseClass", one instance per method whose name begins with `test`. The fixture protocol wraps each one. `setUp` "is called immediately before calling the test method", and "any exception raised by this method will be considered an error rather than a test failure", a distinction the result object keeps for the whole run. `tearDown` runs "even if the test method raised an exception", but only if setUp succeeded. And the instance is fresh every time: "A new TestCase instance is created as a unique test fixture used to execute each individual test method." The first sample makes the interleaving observable with an events list on the class:

```
class FixtureOrder(unittest.TestCase):
    events = []

    def setUp(self):
        self.events.append("setUp")

    def tearDown(self):
        self.events.append("tearDown")

    def test_alpha(self):
        self.events.append("alpha")

    def test_beta(self):
        self.events.append("beta")
```

listing 245: setUp, the two tests, tearDown, and the shared events list (source: python/samples/src/Ch20/anatomy.py, lines 18-31)

The check runs the class alone and requires the exact sequence, setup, alpha, teardown, setup, beta, teardown, so the per-method fixture is a measured fact here, not a convention:

```
    FixtureOrder.events.clear()
    result, _ = run_into_stream(FixtureOrder)
    if result.testsRun != 2 or result.failures or result.errors:
        raise SystemExit(f"FAIL fixture run {result.testsRun}")
    if FixtureOrder.events != [
        "setUp",
        "alpha",
        "tearDown",
        "setUp",
        "beta",
        "tearDown",
    ]:
        raise SystemExit(f"FAIL event order {FixtureOrder.events}")
    print("ok 2 a fresh fixture per test, setUp and tearDown interleaved")
```

listing 246: the recorded order asserted exactly: a fresh fixture per test (source: python/samples/src/Ch20/anatomy.py, lines 81-94)

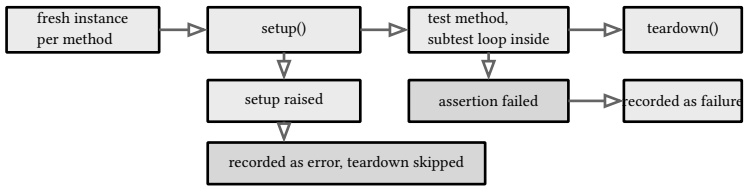
The assert inventory is large and honest about what it checks: `assertEqual` is ==, `assertTrue` is `bool(x) is True`, and the 3.14 release added to the family, probed here by presence and by use: `assertStartsWith`, `assertHasAttr`, `assertIsSubclass`. The exception side belongs to the context manager form, whose page promise is that "The context manager will store the caught exception object in its `exception` attribute", so the assertion can extend past the type into the message, the discipline book 8, ch 10 (python 3.14, exceptions and context managers) established for raise sites:

```
class Errors(unittest.TestCase):
    def test_message(self):
        with self.assertRaises(ValueError) as caught:
            halve(0)
        self.assertEqual(str(caught.exception), "cannot halve zero")


class MissingRaise(unittest.TestCase):
    def test_expected_a_raise(self):
        with self.assertRaises(ValueError):
            halve(4)
```

listing 247: assertraises as a context manager, and the failure when nothing raised (source: python/samples/src/Ch20/anatomy.py, lines 48-58)

The sample's fifth check runs the no-raise case and requires it recorded as exactly one failure, because the negative paths of a test framework are tests too. `subTest` covers the parametric grid, and the docs state its value in one sentence: "Without using a subtest, execution would stop after the first failure, and the error would be less easy to diagnose because the value of `i` wouldn't be displayed." A green grid counts as one test. A broken one names its parameters, which the third sample measures precisely.



failure means the test ran and its assertion said no, error means the fixture or the test broke

figure 84: one test method through the runner: the fixture cycle and the two red states

20.2. mocks

The module's own sentence is the whole charter: "unittest.mock is a library for testing in Python. It allows you to replace parts of your system under test with mock objects and make assertions about how they have been used." A `Mock` records calls and answers with what you configured, `return_value` for the value, `side_effect` for a sequence or a function, and the assertions are call assertions: `assert_called_once_with` "Assert that the mock was called exactly once and that call was with the specified arguments." The sample's system under test is four lines, a price function used by a total function, small enough that the seam is the only thing on stage:

```
class Session:
    def get(self, path):
        raise NotImplementedError


def unit_price(sku):
    return BASE_PRICE


def line_total(sku, quantity):
    return unit_price(sku) * quantity


@mock.patch(f"{__name__}.BASE_PRICE", 500)
def discounted_price():
    return unit_price("any")
```

listing 248: the seam: unit_price is looked up by line_total, and a patch decorator waits above (source: python/samples/src/Ch20/mocks.py, lines 12-27)

Discipline number one is spec. Given a spec, "Accessing any attribute not in this list will raise an `AttributeError`", which turns the classic mock failure, a green test exercising a method the real class never had, into a loud red one:

```
    strict = mock.Mock(spec=Session)
    strict.get("/health")
    strict.get.assert_called_once_with("/health")
    try:
        strict.post("/orders")
        raise SystemExit("FAIL the spec allowed an invented method")
    except AttributeError:
        pass
    print("ok 5 spec=session allowed get, refused an invented post")
```

listing 249: spec=session allowed get, refused the invented post (source: python/samples/src/Ch20/mocks.py, lines 57-65)

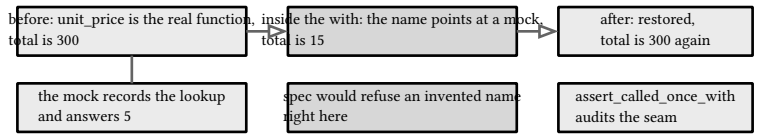
Discipline number two is aim. The patch page states the rule twice, once as mechanism, "patch() works by (temporarily) changing the object that a name points to with another one", and once as the aim: "The basic principle is that you patch where an object is looked up, which is not necessarily the same place as where it is defined." The sample patches this module's `unit_price` because that is the name `line_total` resolves at call time, and the check requires the restore: 300 before, 15 inside, 300 after:

```
    before = line_total("x", 3)
    with mock.patch(f"{__name__}.unit_price", return_value=5):
        patched = line_total("x", 3)
    after = line_total("x", 3)
    if (before, patched, after) != (300, 15, 300):
        raise SystemExit(f"FAIL patch results {(before, patched, after)}")
    print("ok 6 patch hit the name line_total looks up, then restored")

    if discounted_price() != 500 or unit_price("any") != 100:
        raise SystemExit("FAIL the decorator patch leaked")
    print("ok 7 patch as a decorator injected the value and cleaned up")
```

listing 250: patch where it is looked up, then the namespace is restored (source: python/samples/src/Ch20/mocks.py, lines 67-77)

When not to mock is the same chapter's answer as when: mock the seam you own, never the logic you are testing. A pure function like `line_total` gets a real assertion over real inputs, and the mock exists for the boundary calls, prices from a ledger, network from a transport, where the capstone of book 8, ch 40 (python 3.14, capstone: ingest and analysis pipeline, part 1) mocks its transport instead of opening a socket. `MagicMock` is the last stop, the variant that supports magic methods, so `len()` works when configured while a plain `Mock` refuses it with a TypeError, probed in the closing check.



the swap is temporary and scoped: the exit runs the restore, so the next test starts from the real namespace

figure 85: one namespace before, during, and after a patch

20.3. doctest and parametrics

doctest closes the gap between documentation and behavior by making the documentation run. The page's charter: "The `doctest` module searches for pieces of text that look like interactive Python sessions, and then executes those sessions to verify that they work exactly as shown." The examples are ordinary repl transcripts in the docstring, and `testmod` over the module "Return `(failure_count, test_count)`", two numbers a gate can count:

```
def add(a, b):
    """add two ints.

    >>> add(2, 3)
    5
    >>> add(-1, 1)
    0
    """
    return a + b


def shout(words):
    """uppercase with a bang.

    >>> shout("run")
    'RUN!'
    """
    return words.upper() + "!"
```

listing 251: three examples living in two docstrings, executed as tests (source: python/samples/src/Ch20/doctests.py, lines 14-31)

A wrong example is a failing test with the diff narrated, and the sample builds one on purpose from a fabricated string, asserting one failure, one try, and the words Expected and Got in the report:

```
    out = io.StringIO()
    runner = doctest.DocTestRunner()
    test = doctest.DocTestParser().get_doctest(
        ">>> 1 + 1\n3\n", {}, "fabricated", "fabricated", 0
    )
    with contextlib.redirect_stdout(out):
        runner.run(test)
    if runner.failures != 1 or runner.tries != 1:
        raise SystemExit(f"FAIL runner counted {(runner.failures, runner.tries)}")
    if "Expected" not in out.getvalue() or "Got" not in out.getvalue():
        raise SystemExit("FAIL the failure was not narrated")
    print("ok 3 a wrong example reports expected versus got, one failure")
```

listing 252: the fabricated wrong example: expected versus got, counted (source: python/samples/src/Ch20/doctests.py, lines 64-75)

The bridge into unittest is one call, `doctest.DocTestSuite(module)`, which turns the module's examples into a suite the standard runner executes, so doc examples and case classes ride the same gate. The parametric half of the section is the subTest grid, measured at both edges: a green 4-point grid is one green test, and a grid with 2 of 4 points failing produces exactly 2 failures that name `i=1` and `i=3` while the passing points stay silent:

```
    result, output = run_suite(unittest.TestLoader().loadTestsFromTestCase(HalfFail))
    if result.testsRun != 1 or len(result.failures) != 2:
        raise SystemExit(f"FAIL half-fail gave {len(result.failures)}")
    if "i=1" not in output or "i=3" not in output or "i=0" in output:
        raise SystemExit("FAIL the failing parameters were not named")
    print("ok 6 a grid with 2 of 4 failing named exactly those parameters")
```

listing 253: the half-broken grid: two failures, both parameters named (source: python/samples/src/Ch20/doctests.py, lines 90-95)

Skipping gets one sentence and no check: a skip is a claim that a test does not apply, and on this book's box the honest treatment is to investigate until it applies or delete it loudly, not to leave a silent green hole in the count. The gate treats a skipped test as absent from the denominator, and absence is not verification.

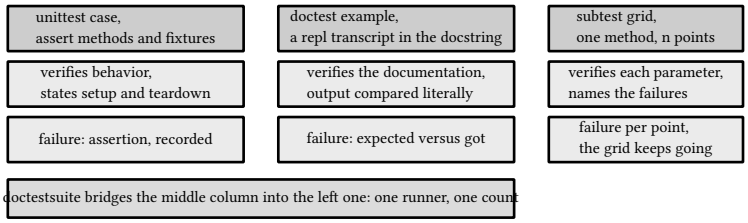


figure 86: three ways to state an expectation, and what each one verifies

20.4. the suite in the gate

The last sample is the gate in miniature, and its centerpiece is the entry point run programmatically. `unittest.main(argv=["gate", "Parsing", "Totals"], exit=False)` returns instead of exiting, the `TestProgram` keeps its result, and the counts are inspectable in the same process, 4 run, zero failures, zero errors, zero skips, with the runner's stream captured so the OK is an assertion and not decoration:

```
    suite = unittest.TestSuite([parsing, loader.loadTestsFromTestCase(Totals)])
    stream = io.StringIO()
    result = TextTestRunner(stream=stream).run(suite)
    if result.testsRun != 4 or result.failures or result.errors:
        raise SystemExit(f"FAIL suite run {result.testsRun}")
    if result.skipped:
        raise SystemExit("FAIL a test skipped silently")
    if "OK" not in stream.getvalue():
        raise SystemExit("FAIL the runner did not print OK")
    print("ok 3 four tests ran, zero failures, zero errors, zero skips")
```

listing 254: build the suite, run it, and assert what green means (source: python/samples/src/Ch20/gate.py, lines 49-58)

The red path goes through the same entry point with `exit=True` and a deliberately broken case: the process would exit with code 1 and the captured stream says FAILED, both asserted, so the gate's own failure semantics are themselves under test:

```
    quiet = io.StringIO()
    exited = "did-not-exit"
    try:
        unittest.main(
            module=sys.modules[__name__],
            argv=["gate", "Sabotage"],
            exit=True,
            testRunner=TextTestRunner(stream=quiet),
        )
    except SystemExit as exc:
        exited = exc.code
    if exited != 1:
        raise SystemExit(f"FAIL the red run exited {exited!r}")
    if "FAILED" not in quiet.getvalue():
        raise SystemExit("FAIL the red run did not say FAILED")
    print("ok 5 a red run exits 1 through the same entry point")
```

listing 255: the sabotaged run exits 1 and says FAILED, both checked (source: python/samples/src/Ch20/gate.py, lines 73-88)

The order of work this book follows everywhere is the order this chapter was built in, and the same order the capstone follows: the check exists first and is red, the implementation lands and turns it green, and the commit carries both. A green run then proves exactly three things, no more: every test method the loader found ran to the

end of its assertions, every fixture cleaned up after itself, and nothing was skipped. It does not prove the absence of untested paths, which is why the capstone pins its coverage story to its own test inventory rather than to a green banner. When the gate runs `make verify-py`, each sample's ok count is compared against the number printed in the chapter's closing callout, the capstone's unittest suite is executed through this same `main` machinery, and any disagreement is a red build, the drift alarm this whole corpus is built around.

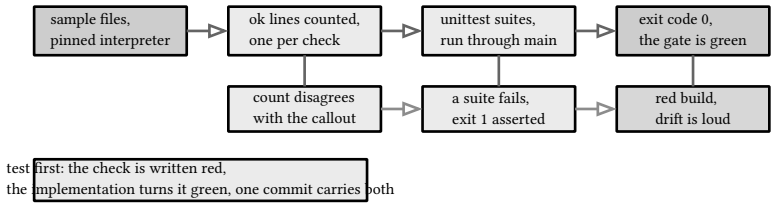


figure 87: the gate's path from sample to green, and where red comes from

VERIFY: chapter 20 in the gate

The 4 samples under `python/samples/src/Ch20/` print 27 ok lines: `anatomy.py` 7, `mocks.py` 8, `doctests.py` 6, `gate.py` 6. The deliberately broken cases, the missing raise, the wrong doctest example, the half-failing grid, and the sabotaged suite, run with their output captured into strings, so the failure accounting is asserted while the gate's own output stays clean.

sources: docs.python.org/3.14/library/unittest.html (loadTestsFrom TestCase, the setUp and tearDown contracts, the fresh instance per test, subTest's diagnostic sentence, the assert method inventory with the 3.14 additions, assertRaises storing the exception), docs.python.org/3.14/library/unittest.mock.html (the module charter, the spec AttributeError, patch's mechanism and the patch-where-it-is- looked-up principle, assert_called_once_with), docs.python.org/3.14/library/doctest.html (the module charter, testmod's failure and test counts), all accessed 2026-09-12; the exit-0 help path and the SystemExit(1) of the red run probed through the pinned interpreter on this machine the same day. Sample behavior verified by `make verify-py`, 27 checks in chapter 20.

21. asyncio

Chapter 17 bought io concurrency with threads and paid for each one, chapter 18 paid again in processes when the work turned cpu bound. asyncio buys the same io concurrency with zero extra execution vehicles: one thread, one loop, and coroutines that hand control back at every wait. The chapter assembles the machine in four steps, the loop itself and the windows fact about which loop you actually get, then tasks and the groups that make their failures honest, then queues as the backpressure layer between producers and workers, and finally the three escape hatches that leave the loop without losing it, threads, subprocesses, and streams. Every behavioral claim below is an ok line in the four samples, 54 in total, or a sentence quoted from a page at docs.python.org fetched 2026-09-12. The async web framework of book 8, ch 22 (python 3.14, fastapi) rides this loop unchanged, and the capstone pipeline of book 8, ch 40 (python 3.14, capstone: ingest and analysis pipeline, part 1) is this chapter's queue discipline at full size.

21.1. the event loop

A coroutine is a function that cannot run itself. The docs put the first hazard in one line, "Note that simply calling a coroutine will not schedule it to be executed", and the interpreter is not bluffing: calling the function builds a coroutine object and returns it, body untouched. The sample proves it the direct way, by checking the log the body would have written, then closes the object so the interpreter never has to warn about it:

```
    calls = []
    coro = work("idle", calls)
    if calls:
        raise SystemExit(f"FAIL calling a coroutine ran body: {calls}")
    if not asyncio.iscoroutine(coro):
        raise SystemExit("FAIL calling the def returned a non coroutine")
    coro.close()
    print("ok 2 calling a coroutine builds an object, runs nothing")
```

listing 256: calling a coroutine builds an object and runs nothing, `close` retires it (source: python/samples/src/Ch21/loop.py, lines 34-41)

The entry point is `asyncio.run`, and the runner page states the whole contract: "This function runs the awaitable, taking care of managing the asyncio event loop, **finalizing asynchronous generators**, and closing the executor", "The loop is closed at the end", it "should be used as a main entry point for asyncio programs, and should ideally only be called once", and "This function cannot be called when another asyncio event loop is running in the same thread". The last sentence has a runtime check with no loop of your own making, `asyncio.get_running_loop` outside a coroutine context is a `RuntimeError` by definition:

```
    try:
        asyncio.get_running_loop()
    except RuntimeError:
        print("ok 3 get_running_loop outside a loop raises RuntimeError")
    else:
        raise SystemExit("FAIL get_running_loop worked with no loop running")
```

listing 257: outside any loop, `get_running_loop` refuses rather than inventing one (source: python/samples/src/Ch21/loop.py, lines 43-48)

What the loop actually does to a running coroutine is measured, not asserted. One probe coroutine grabs the running loop, checks that the object identity survives an await, and times a sleep on `loop.time`, the loop's own monotonic clock:

```
async def nap(dur, result):
    loop = asyncio.get_running_loop()
    same = loop is asyncio.get_running_loop()
    before = loop.time()
    await asyncio.sleep(dur)
    return loop, same, loop.time() - before, result
```

listing 258: the probe: one loop object, identity checked across the suspension, duration measured on loop time (source: python/samples/src/Ch21/loop.py, lines 13-18)

`await` is that suspension. The docs guarantee the delay is a floor, and the sample holds the loop to it, `elapsed` ≥ `0.05` on a `sleep(0.05)`, while the value carried through the suspension comes back unchanged. The same probe returns the loop object to `main`, where two more of the runner page's sentences turn into checks, the loop is closed after `asyncio.run` returns, and on this platform the loop it built is the windows one:

```
    loop, same, elapsed, value = asyncio.run(nap(0.05, "kept"))
    if loop.is_closed() is not True:
        raise SystemExit("FAIL the loop was still open after run returned")
    print("ok 6 the loop is closed by the time asyncio.run returns")

    if sys.platform ≠ "win32":
        raise SystemExit(f"FAIL platform is {sys.platform}, this book runs on windows")
    if not isinstance(loop, asyncio.ProactorEventLoop):
        raise SystemExit(f"FAIL default loop is {type(loop).__name__}, not proactor")
    print("ok 7 the default loop on windows is the proactor event loop")
```

listing 259: after `run` returns the loop is closed, and on windows it was a `ProactorEventLoop` (source: python/samples/src/Ch21/loop.py, lines 59-68)

That is the platform fact of the chapter. The platforms page states it flatly, "Changed in version 3.8: On Windows, `ProactorEventLoop` is now the default event loop", and the sample asserts the class at runtime rather than trusting the default to persist. The runner page also carries the future of the old configuration route: "The asyncio policy system is deprecated and will be removed in Python 3.16; from there on, an explicit **loop_factory** is needed to configure the event loop". The fourth section's selector leg already uses `loop_factory` the modern way.

The last two facts of the section are ordering facts. Plain awaits, one after another with no tasks anywhere, stay strictly sequential, start and end nested per coroutine, because a coroutine that never yields to the scheduler cannot be interleaved with:

```
    async def two():
        await work("first", order)
        await work("second", order)

    asyncio.run(two())
    if order ≠ ["start first", "end first", "start second", "end second"]:
        raise SystemExit(f"FAIL sequential awaits interleaved: {order}")
    print("ok 11 plain awaits without tasks stay strictly sequential")
```

listing 260: sequential awaits without tasks stay sequential, the log is fully nested (source: python/samples/src/Ch21/loop.py, lines 84-91)

And the nesting check closes the loop on `run` itself: calling `asyncio.run` from inside a running loop is the `RuntimeError` the runner page promises, here caught with the inner coroutine explicitly closed so no warning escapes.

NOTE: one loop per run, and 3.14 hands you more eyes

`asyncio.run` builds, drives, and closes one loop per call, and the samples call it exactly once per program except `offload.py`, whose selector leg needs a second, different loop, which is the "ideally" in the docs' "ideally only be

called once". Python 3.14 also adds loop introspection to the module, `capture_call_graph` and `print_call_graph`, and a `python -m`

`asyncio ps PID` CLI that dumps a running program's task table with awaiter chains. This book verifies the primitives the CLI is built from, tasks, cancellation, and timeouts, and leaves the introspection tools as the stated boundary.

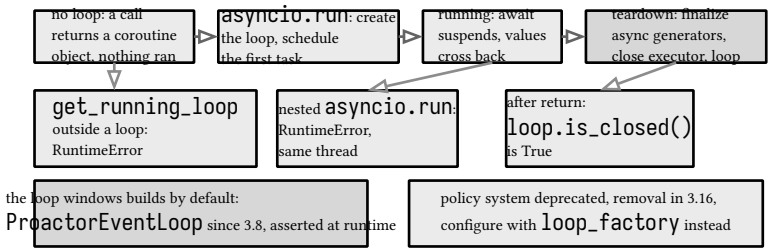


figure 88: the lifecycle of one `asyncio.run` as a state machine: no loop, loop created and driven, closed on return, the proactor default on windows

21.2. tasks and groups

A coroutine that someone awaits is a subroutine. A coroutine wrapped in a `Task` is a unit the loop owes turns to. `create_task` "Wrap[s] the **coro** into a Task and schedule[s] its execution", raises `RuntimeError` if no loop is running, and the scheduling half of that sentence is checkable: between the `create_task` call and the first `await`, the body has not run, because the loop cannot take a turn while the current coroutine keeps the thread:

```
    task = asyncio.create_task(slow("only", 0, calls, result="done"))
    if task.done() or calls:
        lines.append(f"FAIL task body ran before the first await: {calls}")
    else:
        lines.append("create_task schedules the body, it does not start it")

    if await task ≠ "done":
        lines.append("FAIL the result was lost")
    else:
        lines.append("awaiting the task returns its result")
```

listing 261: a fresh task has not started, and awaiting it returns the result (source: python/samples/src/Ch21/tasks.py, lines 48-57)

The task page also carries a warning worth internalizing: "Save a reference to the result of this function, to avoid a task disappearing mid-execution. The event loop only keeps weak references to tasks." `TaskGroup` keeps strong references, which is one of the reasons it is the default recommendation.

`gather` runs awaitables concurrently and hands back results in input order no matter the completion order, the sample's three sleeps finish in the order b, c, a while the result list still reads a, b, c:

```
    order = await asyncio.gather(
        slow("a", 0.03, calls),
        slow("b", 0.01, calls),
        slow("c", 0.02, calls),
    )
    if order == ["a", "b", "c"]:
        lines.append("gather results keep input order despite completion order")
    else:
        lines.append(f"FAIL gather reordered results: {order}")
    if calls[-3:] == ["end b", "end c", "end a"]:
        lines.append("completion order was b, c, a by delay")
    else:
        lines.append(f"FAIL completion order was {calls}")
```

listing 262: gather results keep input order while the calls log the true completion order (source: python/samples/src/Ch21/tasks.py, lines 59-71)

Failure is where `gather`'s contract gets interesting. The docs: "If **return_exceptions** is `False` (default), the first raised exception is immediately propagated to the task that awaits on `gather()`. Other awaitables in the **aws** sequence won't be cancelled and will continue to run." The sample proves both halves, the propagation, and the survivor still delivering its result after the `ValueError` passed through:

```
    calls.clear()
    survivor = asyncio.create_task(slow("q", 0.05, calls, result="q"))
    try:
        await asyncio.gather(boom(), survivor)
    except ValueError:
        lines.append("gather with the default propagates the first failure")
    else:
        lines.append("FAIL gather swallowed the failure")
    if await survivor == "q":
        lines.append("the unfailed sibling kept running, gather cancelled nothing")
    else:
        lines.append("FAIL the sibling was lost")
```

listing 263: gather propagates the first failure and abandons the rest to keep running (source: python/samples/src/Ch21/tasks.py, lines 85-96)

`TaskGroup`, added in 3.11, is the structured answer. The docs' failure rule is exact: "The first time any of the tasks belonging to the group fails with an exception other than `asyncio.CancelledError`, the remaining tasks in the group are cancelled", and the errors "are combined in an `ExceptionGroup` or `BaseExceptionGroup` (as appropriate) which is then raised". The sample runs both legs, two healthy children collected at context exit, then one failing child beside a slower healthy one:

```
    calls.clear()
    async with asyncio.TaskGroup() as tg:
        one = tg.create_task(slow("t1", 0.01, calls, result=1))
        two = tg.create_task(slow("t2", 0.02, calls, result=2))
    if (one.result(), two.result()) == (1, 2):
        lines.append("taskgroup awaits every child at context exit")
    else:
        lines.append("FAIL taskgroup results lost")

    calls.clear()
    try:
        async with asyncio.TaskGroup() as tg:
            tg.create_task(slow("s1", 0.02, calls))
            tg.create_task(boom(0.005))
    except* ValueError as eg:
        if len(eg.exceptions) == 1:
            lines.append("one failing child surfaces as a one error group")
        else:
            lines.append(f"FAIL group holds {len(eg.exceptions)} errors")
    else:
        lines.append("FAIL the taskgroup raised nothing")
    if "cancelled s1" in calls:
        lines.append("taskgroup cancelled the healthy sibling on failure")
    else:
        lines.append(f"FAIL sibling fate: {calls}")
```

listing 264: taskgroup, both legs: children awaited at exit, failure cancels the sibling and raises one error group (source: python/samples/src/Ch21/tasks.py, lines 98-122)

The task page states the comparison outright: "`TaskGroup` provides stronger safety guarantees than `gather` for scheduling a nesting of subtasks: if a task (or a subtask, a task scheduled by a task) raises an exception, `TaskGroup` will, while `gather` will not, cancel the remaining scheduled tasks." `except*` from book 8, ch 10 (python 3.14, exceptions and context managers) selects the group's payload, here exactly one `ValueError`.

Cancellation is a delivered exception, not a kill. "`CancelledError` will be raised in the task at the next opportunity", `cancel()` returns `True` on a pending task and `False` on a finished one, and the coroutine "has a chance to clean up or even deny the request by suppressing the exception", at the price of calling `Task.uncancel()` "in addition to catching the exception". The victim leg checks the whole discipline, the raise inside, the cleanup that runs first, the cancelled state, and the `False` on re-cancel:

```
    victim = asyncio.create_task(slow("v", 10, calls))
    await asyncio.sleep(0)
    if victim.cancel() is not True:
        lines.append("FAIL cancel of a pending task returned False")
    try:
        await victim
    except asyncio.CancelledError:
        lines.append("cancel raises CancelledError inside the coroutine")
    else:
        lines.append("FAIL the cancelled task returned normally")
    if victim.cancelled() and victim.cancel() is False:
        lines.append("the task is cancelled, and re cancelling a done task is False")
    else:
        lines.append("FAIL cancelled state wrong")
    if "cancelled v" in calls:
        lines.append("the cleanup path observed the cancellation before re raising")
    else:
        lines.append(f"FAIL cleanup missed: {calls}")
```

listing 265: cancel delivers `CancelledError` at the next await, cleanup runs, re cancelling a done task is False (source: python/samples/src/Ch21/tasks.py, lines 124-141)

The denial path exists and the sample walks it once, deliberately, because the counting it exposes is what `TaskGroup` and `asyncio.timeout` are built on. `cancelling()` reports "the number of calls to `cancel()` less the number of `uncancel()` calls":

```
async def deny_cancel(checks):
    task = asyncio.current_task()
    try:
        await asyncio.sleep(10)
    except asyncio.CancelledError:
        checks.append(("cancelling", task.cancelling()))
        task.uncancel()
        checks.append(("after uncancel", task.cancelling()))
        return "survived"
```

listing 266: the denier: catches the cancellation, records the count, uncancels, returns normally (source: python/samples/src/Ch21/tasks.py, lines 23-31)

```
    checks = []
    denier = asyncio.create_task(deny_cancel(checks))
    await asyncio.sleep(0.01)
    denier.cancel()
    if await denier == "survived":
        lines.append("a coroutine can deny cancellation with uncancel")
    else:
        lines.append("FAIL the denier did not survive")
    if checks == [("cancelling", 1), ("after uncancel", 0)]:
        lines.append("cancelling counts 1 pending request, uncancel drains it")
    else:
        lines.append(f"FAIL cancel counts: {checks}")
```

listing 267: the denier survives, and the count drains from 1 to exactly 0 (source: python/samples/src/Ch21/tasks.py, lines 143-154)

Timeouts are cancellation wearing a clock. `asyncio.timeout`, added in 3.11, "will cancel the current task and handle the resulting `asyncio.CancelledError` internally, transforming it into a `TimeoutError`", with the subtlety the docs flag in italics worth repeating plainly: "the `TimeoutError` can only be caught *outside* of the context manager". `wait_for` is the one-shot form, raising the same `TimeoutError` since 3.11:

```
    loop = asyncio.get_running_loop()
    before = loop.time()
    try:
        async with asyncio.timeout(0.05):
            await asyncio.sleep(10)
    except TimeoutError:
        if loop.time() - before ≥ 0.05:
            lines.append("asyncio.timeout converts cancellation into TimeoutError")
        else:
            lines.append("FAIL timeout fired early")
    else:
        lines.append("FAIL timeout never fired")

    try:
        await asyncio.wait_for(asyncio.sleep(10), 0.05)
    except TimeoutError:
        lines.append("wait_for raises the same TimeoutError past its deadline")
    else:
        lines.append("FAIL wait_for never fired")

    if await asyncio.wait_for(asyncio.sleep(0, result="quick"), 5) == "quick":
        lines.append("wait_for inside its deadline returns the result untouched")
    else:
        lines.append("FAIL wait_for lost the fast result")
```

listing 268: timeout as a context manager and wait_for as a call, both floors measured on loop time (source: python/samples/src/Ch21/tasks.py, lines 156-179)

PITFALL: fire and forget is a weak reference gamble

The tempting pattern, `asyncio.create_task(coro())` with the result discarded, is exactly what the task page's warning is about: the loop holds only a weak reference, so the task can be collected mid-execution and the exception it raised goes with it. Hold the task, or enter a `TaskGroup`, or add the done callback to a background set as the docs show. The samples keep every task in a variable and await or cancel each one, which is also why their failure accounting is exact.

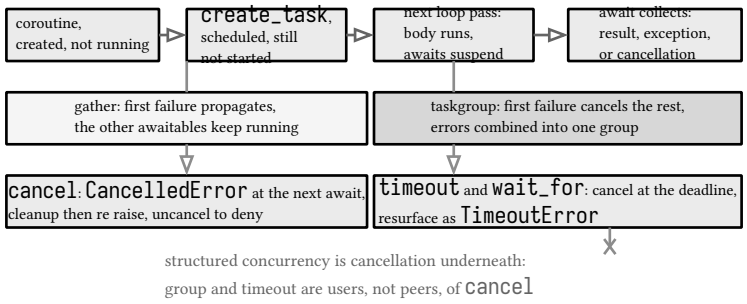


figure 89: tasks through the machine: scheduling, gather's abandoned siblings, taskgroup's cancelled ones, and the two clocks that end a task

21.3. queues and producers

The queue page opens with the two facts that drive everything else: `asyncio.Queue` "is a first in, first out (FIFO) queue", and "This class is not thread safe", it is the coroutine sibling of the `queue.Queue` that book 8, ch 17 (python 3.14, threads and the gil) used, with the same join and task_done protocol and none of the locks. `maxsize` is the whole backpressure story in one parameter: "If **maxsize** is less than or equal to zero, the queue size is infinite. If it is an integer greater than `0`, then `await put()` blocks when the queue reaches **maxsize** until an item is removed by

`get()`", and with the default `maxsize=0`, "`full()` never returns `True`". The backpressure leg pins all of it, a producer parked on its third `put` with exactly 2 items buffered, then drained to completion:

```
async def producer(q, items, events):
    for item in items:
        events.append(f"put {item}")
        await q.put(item)
    events.append("producer done")
```

listing 269: the producer: one `put` per item, completion logged after the last (source: python/samples/src/Ch21/queues.py, lines 7-11)

```
    events = []
    q3 = asyncio.Queue(maxsize=2)
    prod = asyncio.create_task(producer(q3, range(6), events))
    await asyncio.sleep(0.05)
    if prod.done() or q3.qsize() != 2 or events[-1] != "put 2":
        lines.append(f"FAIL backpressure state: {events} qsize {q3.qsize()}")
    else:
        lines.append("the third put suspends, the producer waits at maxsize")
    for _ in range(6):
        await q3.get()
    await prod
    if prod.done() and q3.qsize() == 0:
        lines.append("draining unblocks the producer, all 6 items delivered")
    else:
        lines.append("FAIL producer never finished")
```

listing 270: backpressure, asserted: the third put suspends, six gets release the producer (source: python/samples/src/Ch21/queues.py, lines 77-91)

The consumer side is the fan-out unit of the capstone pipeline, a `while True` loop over `await q.get()` ending in `q.task_done()`. The docs define the handshake precisely: "The count of unfinished tasks goes up whenever an item is added to the queue. The count goes down whenever a consumer coroutine calls `task_done()`", and "`join()`" blocks "until all items in the queue have been received and processed". Overcall it and the queue answers, "`task_done()` raises `ValueError` if called more times than there were items placed on the queue":

```
async def worker(name, q, done, calls):
    while True:
        item = await q.get()
        calls.append(f"{name} took {item}")
        await asyncio.sleep(0.01)
        done.add(item)
        q.task_done()
```

listing 271: the worker: get, process, task_done, forever (source: python/samples/src/Ch21/queues.py, lines 14-20)

```
    work_q = asyncio.Queue()
    done = set()
    calls.clear()
    workers = [
        asyncio.create_task(worker(f"w{i}", work_q, done, calls)) for i in range(3)
    ]
    for i in range(6):
        await work_q.put(i)
    await work_q.join()
    if done == set(range(6)):
        lines.append("join returns when every item got its task_done, 3 workers")
    else:
        lines.append(f"FAIL unfinished items: {set(range(6)) - done}")
    for w in workers:
        w.cancel()
    fates = await asyncio.gather(*workers, return_exceptions=True)
    if all(isinstance(f, asyncio.CancelledError) for f in fates) and work_q.empty():
        lines.append("graceful shutdown: join first, then cancel the idle workers")
    else:
        lines.append(f"FAIL worker shutdown: {fates!r}")
```

listing 272: three workers drain six items, join returns, then the idle workers are cancelled (source: python/samples/src/Ch21/queues.py, lines 93-112)

That closing sequence is the graceful shutdown pattern: `join` first, so every accepted item is finished, `cancel` after, so no worker dies holding one. The queue page also sets the timeout boundary, "methods of asyncio queues don't have a **timeout** parameter; use `asyncio.wait_for()` function to do queue operations with a timeout", which is the same `wait_for` the last section already proved.

Lazy production is the async generator, `yield` inside `async def`, consumed by `async for` one item per suspension. Fed into a queue it becomes a pipeline stage whose order survives, and whose pull timing is the consumer's, not the producer's:

```
async def numbers(count):
    for i in range(count):
        await asyncio.sleep(0)
        yield i
```

listing 273: an async generator yields one value per suspension (source: python/samples/src/Ch21/queues.py, lines 23-26)

```
    collected = []
    agen = numbers(3)
    async for value in agen:
        collected.append(value)
    if collected == [0, 1, 2]:
        lines.append("an async generator yields lazily under async for, in order")
    else:
        lines.append(f"FAIL async generator order {collected}")
```

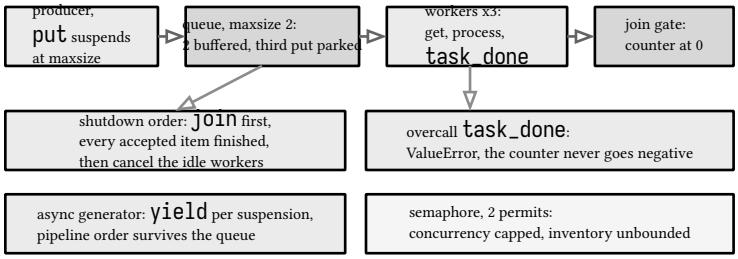
listing 274: `async for` consumes the generator lazily, order preserved (source: python/samples/src/Ch21/queues.py, lines 114-121)

When the bound must be on concurrency rather than inventory, `Semaphore` is the primitive. The sample tracks the high water mark of a 2 permit semaphore over 6 jobs and gets exactly 2, never 3, with all 6 results collected:

```
    q5 = asyncio.Queue()
    await q5.put("only")
    await q5.get()
    q5.task_done()
    try:
        q5.task_done()
    except ValueError:
        lines.append("task_done past the item count raises ValueError")
    else:
        lines.append("FAIL the extra task_done was allowed")

    permits = asyncio.Semaphore(2)
    peak = {"now": 0, "max": 0}
    out = await bounded(range(6), permits, peak)
    if peak["max"] == 2 and out == list(range(6)):
        lines.append("a semaphore bounds concurrency at 2, all 6 jobs complete")
    else:
        lines.append(f"FAIL semaphore peak {peak} out {out}")
```

listing 275: the overcalled `task_done` raises, and a 2 permit semaphore caps observed concurrency at exactly 2 (source: python/samples/src/Ch21/queues.py, lines 133-150)



backpressure lives in `maxsize`, concurrency control lives in the semaphore, shutdown lives in the join order

figure 90: the queue as a data structure: bounded inventory between producer and workers, the unfinished counter gating join, the semaphore gating concurrency

21.4. streams, subprocess, and threads

The loop is only worth having while everything on it awaits. The sample measures both sides of that sentence with a heartbeat task that ticks every 0.05 s of loop time. The same 0.3 s blocking sleep runs twice, once through `asyncio.to_thread`, where the heartbeat keeps counting, and once called inline, where the count is exactly zero for the whole block:

```
async def run():
    lines = []
    total = {"n": 0}

    async def heartbeat():
        while True:
            await asyncio.sleep(0.05)
            total["n"] += 1

    beat = asyncio.create_task(heartbeat())
    await asyncio.sleep(0.12)
    base = total["n"]
    await asyncio.to_thread(blocking_pause)
    during = total["n"] - base
    if during ≥ 3:
        lines.append("a blocking sleep in to_thread leaves the loop ticking")
    else:
        lines.append(f"FAIL only {during} heartbeats during to_thread")
```

listing 276: the heartbeat keeps ticking through `to_thread`, the blocking sleep runs on a pool thread (source: python/samples/src/Ch21/offload.py, lines 27-44)

```
    base = total["n"]
    blocking_pause()
    blocked = total["n"] - base
    if blocked == 0:
        lines.append("the same sleep inline blocks the loop, zero heartbeats")
    else:
        lines.append(f"FAIL {blocked} heartbeats leaked through an inline block")
```

listing 277: the identical sleep called inline starves the loop, zero heartbeats in 0.3 s (source: python/samples/src/Ch21/offload.py, lines 46-52)

That zero is the detection method: a blocked loop is not an error, it is a silence, and any periodic task will measure it. `to_thread` itself is transparent about arguments, return value, and thread identity:

```
    seen = []
    if await asyncio.to_thread(add_on_thread, 20, 22, seen) == 42:
        lines.append("to_thread passes arguments through and returns the value")
    else:
        lines.append("FAIL to_thread lost the return value")

    if seen and seen[0] ≠ threading.get_ident():
        lines.append("the work ran on a worker thread, not the loop thread")
    else:
        lines.append("FAIL the work stayed on the loop thread")
```

listing 278: to_thread passes arguments through, returns the value, and runs on a worker thread (source: python/samples/src/Ch21/offload.py, lines 54-63)

The streams api is the loop's io surface. The streams page: "Streams are high-level async/await-ready primitives to work with network connections. Streams allow sending and receiving data without using callbacks or low-level protocols and transports", and `open_connection` will "return a pair of `(reader, writer)` objects". This chapter never opens a socket, a loopback listener on windows is an interactive firewall event, not a deterministic check, so the reader half arrives the other documented way: the subprocess page states that "If **PIPE** is passed to **stdout** or **stderr** arguments, the `Process.stdout` and `Process.stderr` attributes will point to `StreamReader` instances". The child is a fresh `sys.executable` run with `-c`, under the same spawn realities book 8, ch 18 (python 3.14, multiprocessing

and process patterns) documented for process workers, and its pipe is exercised with the reader's real methods, `readline` for each line, `read` to drain, `at_eof` to confirm the end:

```
    script = "print('alpha')\nprint(2 + 3)\n"
    proc = await asyncio.create_subprocess_exec(
        sys.executable,
        "-c",
        script,
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.PIPE,
    )
    if isinstance(proc.stdout, asyncio.StreamReader):
        lines.append("a subprocess pipe arrives as a real stream reader")
    else:
        lines.append(f"FAIL stdout is {type(proc.stdout).__name__}")
```

listing 279: a subprocess on the default loop, its stdout asserted to be a real `StreamReader` (source: python/samples/src/Ch21/offload.py, lines 65-76)

```
    first = await proc.stdout.readline()
    if first == b"alpha\r\n":
        lines.append("readline delivers the first line, crlf and all")
    else:
        lines.append(f"FAIL first line was {first!r}")

    second = await proc.stdout.readline()
    if second.rstrip() == b"5":
        lines.append("the child computed 2 + 3, the stream carried the bytes")
    else:
        lines.append(f"FAIL second line was {second!r}")

    rest = await proc.stdout.read()
    code = await proc.wait()
    if rest == b"" and proc.stdout.at_eof() and code == 0:
        lines.append("read drains to eof, and wait reports exit code 0")
    else:
        lines.append(f"FAIL drain rest {rest!r} code {code}")
```

listing 280: readline delivers each line, crlf included, read drains to eof, wait reports the exit code (source: python/samples/src/Ch21/offload.py, lines 78-95)

Subprocess support is the second windows fact of the chapter, and the subprocess page states it without hedging: "On Windows subprocesses are provided by `ProactorEventLoop` only (default), `SelectorEventLoop` has no subprocess support." The platforms page itemizes what else the selector loop gives up on windows, `SelectSelector` "supports sockets and is limited to 512 sockets", `add_reader` and `add_writer` "only accept socket handles", and "Subprocesses are not supported, i.e. the `loop.subprocess_exec()` and `loop.subprocess_shell()` methods are not implemented". The mirror leg runs the identical `create_subprocess_exec` call on a selector loop built with `loop_factory`, the 3.16-proof configuration route, and it is a `NotImplementedError`, machine-checked:

```
def main():
    if sys.version_info[:3] != (3, 14, 7):
        raise SystemExit(f"FAIL pin: interpreter is {sys.version_info[:3]}, not 3.14.7")
    print("ok 1 interpreter pin 3.14.7")

    if sys.platform != "win32":
        raise SystemExit(f"FAIL platform is {sys.platform}, this book runs on windows")
    print("ok 2 platform is win32, the platform this chapter asserts")

    for n, line in enumerate(asyncio.run(run()), start=3):
        print(f"ok {n} {line}")

    coro = selector_subprocess()
    try:
        asyncio.run(coro, loop_factory=asyncio.SelectorEventLoop)
    except NotImplementedError:
        print("ok 11 the same call on a selector loop raises NotImplementedError")
    else:
        raise SystemExit("FAIL the selector loop claimed subprocess support")
    finally:
        coro.close()
```

listing 281: the same subprocess call on a selector loop: `NotImplementedError`, exactly as documented (source: python/samples/src/Ch21/offload.py, lines 102-122)

WARNING: why the book defaults to the proactor and never says its name in anger

On windows the proactor loop has no `add_reader` or `add_writer`, which strands libraries built on those callbacks, the documented reason people historically switched to the selector policy and then lost subprocesses, pipes beyond overlapped handles, and the 512 socket ceiling became their world instead. The default buys subprocess support and unbounded socket counts. The samples assert the default class at runtime, and the selector leg proves the tradeoff exists by tripping it, so neither fact is folklore.

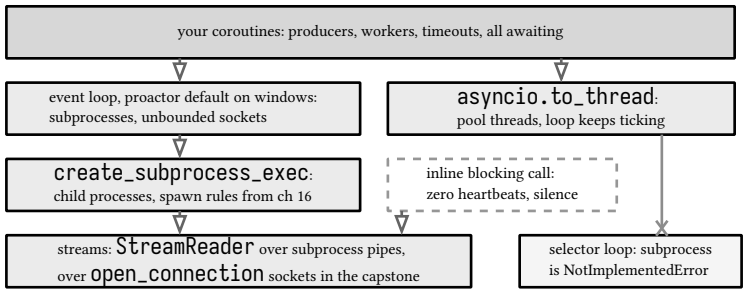


figure 91: the three ways off the loop as a layer stack: pool threads for blocking calls, child processes for real work, streams as the shared io surface, the selector alternative gated

VERIFY: 54 checks, every one deterministic on this box

`loop.py` pins the interpreter at 3.14.7 and contributes 12 checks, the coroutine that does not run, the runner contract, the closed loop, the proactor class, suspension timing, sequential order, and the nested run. `tasks.py` adds 19, scheduling, gather order and failure abandonment, taskgroup's exception group and sibling cancellation, the full cancel discipline with `uncancel` counts, and both timeout forms. `queues.py` adds 12, fifo order, maxsize backpressure, join and task_done accounting, the graceful shutdown order, the async generator pipeline, and the semaphore high water mark. `offload.py` adds 11, the heartbeat contrast, thread identity, the subprocess stream reader, and the selector loop's refusal. The timing checks use 5 to 20 fold margins between the measured and the competing values, so scheduling jitter cannot flip them.

sources: docs.python.org/3/library/asyncio-runner.html (asyncio.run's contract, the policy deprecation note), asyncio-task.html (calling a coroutine does not schedule it, create_task and the weak reference warning, gather semantics, TaskGroup failure and safety guarantees, cancel, uncancel, cancelling, timeout, wait_for), asyncio-queue.html (fifo, thread safety, maxsize, put, get, join and task_done, the wait_for note), asyncio-platforms.html (the proactor default since 3.8, the selector loop's 512 socket limit and missing add_reader, add_writer, and subprocess methods), asyncio-subprocess.html (proactor only on windows, PIPE producing StreamReader instances), asyncio-streams.html (the streams intro, open_connection's reader and writer pair, readline, read, at_eof), and docs.python.org/3/whatsnew/3.14.html (create_task kwargs pass-through, capture_call_graph and print_call_graph), all accessed 2026-09-12. Sample behavior verified by `make verify-py`, 54 checks in chapter 21.

22. fastapi

The web framework of this book is a pinned third party, taught in full here and used later: book 8, ch 40 (python 3.14, capstone: ingest and analysis pipeline, part 1) builds its fetch side on stdlib urllib, and book 8, ch 41 (python 3.14, capstone: ingest and analysis pipeline, part 2) serves the finished report over fastapi with pydantic response models and a TestClient suite. Nothing before this chapter imports the package. Every sample below asserts the exact pin, fastapi 0.141.1, the same way chapter 1 asserts the interpreter, and the chapter walks the machine in four steps: what a fastapi app actually is down at the protocol, the typed request pipeline with dependency injection, the async half plus lifespan and the in-process client, and the boundaries of what the pin installs and what it deliberately does not. Every behavioral claim is an `ok` line in the four samples, 45 in total, or a sentence quoted from a page at fastapi.tiangolo.com fetched 2026-09-12. The models are the book 8, ch 26 (python 3.14, pydantic) `BaseModel` in its serving role, and the event loop under the async endpoints is the one book 8, ch 21 (python 3.14, asyncio) documented.

22.1. the asgi model

The docs' first-steps page defines the object in one sentence, "`FastAPI` is a Python class that provides all the functionality for your API", and the technical note under it names the base: "`FastAPI` is a class that inherits directly from `Starlette`. You can use all the Starlette functionality with `FastAPI` too." Under this pin the base is starlette 1.6.0, and the inheritance is the whole architecture in one fact, because starlette speaks asgi, the python protocol with exactly three moving parts: an app is an async callable that takes a `scope` dict describing one request, a `receive` coroutine that yields incoming messages, and a `send` coroutine that accepts outgoing ones. The sample pins the stack from inside before doing anything else, the fastapi version, the starlette version, the subclass relation, and the callable's parameter names:

```
def main() -> None:
    if fastapi.__version__ != "0.141.1":
        raise SystemExit(f"FAIL pin: fastapi is {fastapi.__version__}, not 0.141.1")
    print("ok 1 fastapi pin 0.141.1")

    if starlette.__version__ != "1.6.0":
        raise SystemExit(f"FAIL starlette is {starlette.__version__}, not 1.6.0")
    if not issubclass(FastAPI, Starlette):
        raise SystemExit("FAIL FastAPI does not subclass Starlette")
    print("ok 2 starlette 1.6.0 is the base, FastAPI subclasses it")

    app = make_app()
    params = list(inspect.signature(app.__call__).parameters)
    if params != ["scope", "receive", "send"]:
        raise SystemExit(f"FAIL the app callable takes {params}")
    print("ok 3 the app is the asgi interface: scope, receive, send")
```

listing 282: the pin, the base class, and the interface: a fastapi app is a `scope, receive, send` callable (source: python/samples/src/Ch22/model.py, lines 57-72)

The signature check is the chapter's thesis as an assertion. A server never imports endpoint functions, it awaits the app callable and trades messages, which means the protocol can be exercised with no server and no client at all. The sample builds a scope dict by hand, `receive` answering an empty http request body, `send` recording into a list, and drives the app with plain `asyncio.run`:

```
    asyncio.run(app(scope_for("/items/7"), receive, send))
    if len(MESSAGES) != 2:
        raise SystemExit(f"FAIL raw call produced {len(MESSAGES)} messages")
    start = MESSAGES[0]
    if start["type"] != "http.response.start" or start["status"] != 200:
        raise SystemExit(f"FAIL start message is {start}")
    if dict(start["headers"]).get(b"content-type") != b"application/json":
        raise SystemExit(f"FAIL headers are {start['headers']}")
    print("ok 6 raw asgi answers http.response.start, 200, json content type")

    body = MESSAGES[1]
    if body["type"] != "http.response.body":
        raise SystemExit(f"FAIL body message is {body}")
    if json.loads(body["body"]) != {"item_id": 7, "double": 14, "q": None, "stock": 0}:
        raise SystemExit(f"FAIL body is {body['body']}")
    print("ok 7 raw asgi answers http.response.body with the json bytes")
```

listing 283: the raw asgi call: a hand built scope in, two protocol messages out, json bytes included (source: python/samples/src/Ch22/model.py, lines 85-100)

Exactly two messages come back, and their shapes are the protocol. The first is `http.response.start`, status 200 and the headers as byte pairs, the second `http.response.body` carrying `{"item_id": 7, "double": 14, ...}` as raw bytes. The `double` value is the quiet proof of typed routing: the path segment arrived as the string `"7"` and the endpoint multiplied it as the integer 7, because the annotation on the path parameter is `int` and fastapi converted before the function ran. The conversion is validation too, the docs' query-params page states the rule generally, parameters "are converted to that type and validated against it", and an unparseable path parameter is rejected with the pydantic error shape rather than reaching the function.

Routing is the layer above the protocol. The first-steps page again: "The `@app.get("/")` tells `FastAPI` that the function right below is in charge of handling requests that go to: the path `/`, using a `get` operation." Each decorator registers one `APIRoute` holding the path verbatim, and a fresh app already carries four routes it never wrote, `/openapi.json`, `/docs`, the docs' oauth redirect, and `/redoc`, the documentation surface the page describes as "the automatic interactive API documentation", powered by "the OpenAPI schema". The samples build every app inside a factory function:

```
def scope_for(path: str) -> dict:
    return {
        "type": "http",
        "asgi": {"version": "3.0"},
        "http_version": "1.1",
        "method": "GET",
        "scheme": "http",
        "path": path,
        "raw_path": path.encode(),
        "query_string": b"",
        "root_path": "",
        "headers": [(b"host", b"testserver")],
        "client": ("testclient", 0),
        "server": ("testserver", 80),
    }


def make_app() -> FastAPI:
    app = FastAPI(title="ch20 model")
    store: dict[str, int] = {}

    @app.get("/items/{item_id}")
    def read_item(item_id: int, q: str | None = None) -> dict:
        return {"item_id": item_id, "double": item_id * 2, "q": q, "stock": len(store)}

    @app.post("/items/{name}")
    def add_item(name: str) -> dict:
        store[name] = len(store) + 1
        return {"stored": name}

    return app
```

listing 284: the request as a dict, and the factory: routes registered by decorator, state closed over per build (source: python/samples/src/Ch22/model.py, lines 24-54)

NOTE: why the samples build apps with a factory, not a module global

The tutorial examples put `app = FastAPI()` at module level, which is fine for one app per process. A factory that closes over its own `store` gives every test a fresh app with fresh state, and the sample proves the isolation the direct way, two builds of the same factory serve different stock counts after a write to one of them. The capstone reuses this shape, one `build_app()` over the finished pipeline, so its suite can construct independent instances per test.

Through a client, the same callable answers as http. Unknown paths are 404, a known path under the wrong verb is 405, the query half appears only when the name is not bound by the path template:

```
    client = TestClient(app)
    page = client.get("/items/7")
    if page.json()["double"] != 14:
        raise SystemExit(f"FAIL client round trip is {page.json()}")
    if page.headers["content-type"] != "application/json":
        raise SystemExit(f"FAIL content type is {page.headers['content-type']}")
    print("ok 8 the client drives the same callable, path 7 arrives as int 7")

    if client.get("/items/7").json()["q"] is not None:
        raise SystemExit("FAIL the default query value was not none")
    if client.get("/items/7?q=bolt").json()["q"] != "bolt":
        raise SystemExit("FAIL the query value did not arrive")
    print("ok 9 a defaulted parameter is none, the same one after ?q= is a string")

    if client.get("/nope").status_code != 404:
        raise SystemExit("FAIL an unknown path was not 404")
    if client.put("/items/7").status_code != 405:
        raise SystemExit("FAIL a wrong method was not 405")
    print("ok 10 an unknown path is 404, a known path with the wrong verb is 405")
```

listing 285: client facts: int conversion, query defaulting, 404 against 405 (source: python/samples/src/Ch22/model.py, lines 102-120)

layer	owns	never sees	pinned by
uvicorn, the server	sockets, http parsing, worker processes	your endpoint code	prose only, not installed
the app callable	the asgi protocol: scope, receive, send	a socket	starlette 1.6.0, asserted
fastapi on top	routing, validation, dependencies, docs routes	the socket byte stream	fastapi 0.141.1, asserted
your endpoint	the work: models in, json out	protocol plumbing	the sample, in the factory

each layer calls down by awaiting, never by importing the one below

figure 92: what each layer of a served request owns, and what it never touches

22.2. validation and dependency injection

The request pipeline is typed end to end, and the types are pydantic models, the same `BaseModel` of book 8, ch 26 (python 3.14, pydantic) in its serving role. A parameter annotated with a model is the request body, parsed and validated before the endpoint runs. A parameter annotated `int` or `str | None` and not bound by the path template is a query parameter, the docs: “When you declare other function parameters that are not part of the path parameters, they are automatically interpreted as ‘query’ parameters.” The sample’s app is small and carries both models, a guarding dependency, and the status codes that make the contract explicit:

```
    def authorize(key: str = "denied") -> str:
        CALLS.append("authorize")
        if key != "s3cret":
            raise HTTPException(status_code=401, detail="bad key")
        return f"key:{key}"

    def stamp() -> str:
        CALLS.append("stamp")
        return "stamped"

    @app.post("/parts", status_code=201)
    def create(part: PartIn, auth: str = Depends(authorize)) -> dict:
        bins[part.name] = "A1"
        return {"name": part.name, "bin": "A1", "internal": auth}

    @app.get("/parts/{name}", response_model=PartOut)
    def read(name: str) -> dict:
        if name not in bins:
            raise HTTPException(status_code=404, detail=f"no bin for {name}")
        return {"name": name, "bin": bins[name], "internal": "hidden"}
```

listing 286: the pipeline endpoints: a body model in, a response model out, a dependency taking its own query parameter (source: python/samples/src/Ch22/validate.py, lines 24-43)

The dependency is the second half of the section. The docs define one from below: “It is just a function that can take all the same parameters that a *path operation function* can take”, and per request fastapi resolves the tree, “Calling your dependency (‘dependable’) function with the correct parameters”, getting the result and assigning it to the endpoint’s parameter. The `authorize` dependency reads a query `key` exactly the way an endpoint would, rejects with `HTTPException` before the endpoint is reached, and its return value travels into the payload:

```
    r = client.post(
        "/parts", params={"key": "s3cret"}, json={"name": "bolt", "grams": 1.5}
    )
    payload = r.json()
    if r.status_code ≠ 201 or payload["internal"] ≠ "key:s3cret":
        raise SystemExit(f"FAIL create is {r.status_code} {payload}")
    if r.headers["content-type"] ≠ "application/json":
        raise SystemExit(f"FAIL content type is {r.headers['content-type']}")
    print("ok 1 a body model in, 201 out, the dependency value injected")

    r = client.get("/parts/bolt")
    if r.json() ≠ {"name": "bolt", "bin": "A1"}:
        raise SystemExit(f"FAIL response model let {r.json()} through")
    print("ok 2 response_model strips every key the model does not declare")

    r = client.post(
        "/parts",
        params={"key": "s3cret"},
        json={"name": "nut", "grams": 2, "junk": True},
    )
    if r.status_code ≠ 201:
        raise SystemExit(f"FAIL an extra body key was rejected: {r.status_code}")
    print("ok 3 extra body keys are ignored by default")

    r = client.post("/parts", params={"key": "s3cret"}, json={"name": "bolt"})
    err = r.json()["detail"][0]
    if r.status_code ≠ 422 or err["type"] ≠ "missing":
        raise SystemExit(f"FAIL missing field: {r.status_code} {err}")
    if err["loc"] ≠ ["body", "grams"] or err["msg"] ≠ "Field required":
        raise SystemExit(f"FAIL missing shape: {err}")
    print("ok 4 a missing field is 422, loc names body, msg field required")
```

listing 287: the guarantees checked: 201 by declaration, the model stripped, extra keys ignored, the 422 shape pinned (source: python/samples/src/Ch22/validate.py, lines 65-95)

The 422 body is worth reading closely because it is a pydantic error list wearing a detail key. The docs' query page shows the same shape for a missing parameter, `{"type": "missing", "loc": ["query", "needy"], "msg": "Field required"}`, and the sample pins all three fields for a missing body member, type `missing`, `loc` naming `["body", "grams"]`, `msg` `Field required`. Wrong types get their own tag, `float_parsing`, and the `loc` list is the address of the failure, body or path or query, field name last. The other error surface is deliberately different: `HTTPException` answers with a string detail, so a 422 is always a list of validation findings and a raised exception is always prose:

```
    plain = client.get("/bins").json()
    tight = client.get("/bins", params={"limit": 2}).json()
    broken = client.get("/bins", params={"limit": "abc"})
    if plain["limit"] != 10 or tight["limit"] != 2:
        raise SystemExit(f"FAIL query limits are {plain} {tight}")
    err = broken.json()["detail"][0]
    if broken.status_code != 422 or err["loc"] != ["query", "limit"]:
        raise SystemExit(f"FAIL query shape: {err}")
    print("ok 6 a query int defaults, converts, and 422s on garbage")

    r = client.get("/parts/ghost")
    if r.status_code != 404 or r.json() != {"detail": "no bin for ghost"}:
        raise SystemExit(f"FAIL exception shape: {r.status_code} {r.json()}")
    print("ok 7 an httpexception answers 404 with a string detail")
```

listing 288: query validation and the exception surface: a list for 422, a string for a raised 404 (source: python/samples/src/Ch22/validate.py, lines 105-118)

`response_model` is a guarantee, not a comment. The `create` endpoint has no response model, so everything it returns ships, including the internal auth marker. The `read` endpoint declares `response_model=PartOut` and returns a dict carrying the same kind of internal field, and the filter removes it: only `name` and `bin` cross. The two checks side by side are the whole argument, the identical leak attempted twice, one filtered and one not:

```
    r = client.delete("/parts/bolt")
    if r.status_code != 204 or r.content != b"":
        raise SystemExit(f"FAIL delete is {r.status_code} {r.content!r}")
    if client.get("/parts/bolt").status_code != 404:
        raise SystemExit("FAIL the deleted row was still readable")
    print("ok 8 a 204 endpoint answers empty, and the row is gone")

    r = client.post("/parts", json={"name": "x", "grams": 1})
    if r.status_code != 401 or r.json() != {"detail": "bad key"}:
        raise SystemExit(f"FAIL auth leg: {r.status_code} {r.json()}")
    if client.get("/bins").json()["bins"] != ["nut"]:
        raise SystemExit("FAIL the rejected create left state behind")
    print("ok 9 a rejecting dependency answers 401, the endpoint never runs")
```

listing 289: 204 with an empty body, and a dependency that answers 401 leaving no state behind (source: python/samples/src/Ch22/validate.py, lines 120-132)

The cache rule comes from the sub-dependencies page: “If one of your dependencies is declared multiple times for the same *path operation*, for example, multiple dependencies have a common sub-dependency, `FastAPI` will know to call that sub-dependency only once per request”, saving the value “and pass it to all the ‘dependants’ that need it in that specific request”. The `stamped` endpoint takes the same dependency twice, and the call log says `stamp` once per request, once more for the next request, a per-request cache rather than a global one. The last check is an ordering fact the pipeline figure owes: on a request whose body is then rejected, the dependency log still shows `authorize`, the front half of the pipeline runs before the body verdict lands:

```
    before = CALLS.count("stamp")
    client.get("/stamped")
    if CALLS.count("stamp") - before != 1:
        raise SystemExit(f"FAIL stamp ran {CALLS.count('stamp') - before} times")
    client.get("/stamped")
    if CALLS.count("stamp") - before != 2:
        raise SystemExit("FAIL the dependency cache lasted across requests")
    print("ok 10 one shared dependency behind two parameters, once per request")

    CALLS.clear()
    TestClient(make_app()).post("/parts", params={"key": "s3cret"}, json={"name": "b"})
    if CALLS != ["authorize"]:
        raise SystemExit(f"FAIL call log is {CALLS}")
    print("ok 11 the dependency ran even though the body was rejected")
```

listing 290: the dependency cache per request, and the dependency that ran before a rejected body (source: python/samples/src/Ch22/validate.py, lines 134-147)

PITFALL: no response_model means no filter, and the docs will not warn you

An endpoint returning a dict of internal fields with no `response_model` ships every key, the sample's first check carries an `internal` auth marker straight to the client as json. The tutorial flow builds small dicts where that is harmless, so the habit transfers silently to the endpoint that returns a user record with a password hash inside. Declare the response model on every endpoint that returns structured data, and let the check pattern of this sample, assert the exact payload key set, fail loudly when a new field leaks.

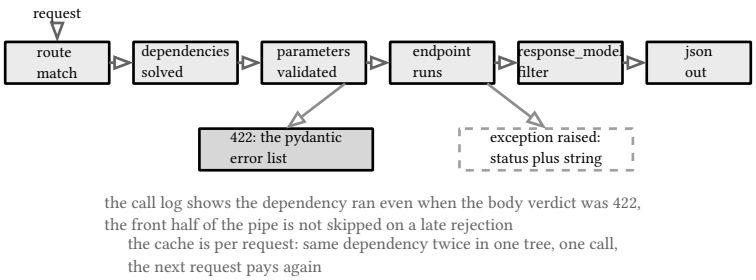


figure 93: the typed request pipeline with its two early exits

22.3. async endpoints, lifespan, and the testclient

An `async def` endpoint is a coroutine the framework awaits, on the same kind of loop book 8, ch 21 (python 3.14, asyncio) spent its chapter on. The sample's `ping` endpoint captures `asyncio.get_running_loop()` from inside, awaits a suspension, and returns, so the checks can see what actually carried it: a live loop, and on this machine the `ProactorEventLoop`, the windows default chapter 21 asserted for `asyncio.run`. Lifespan is the startup and shutdown contract, an async context manager passed as `FastAPI(lifespan=...)`, the docs: "The first part of the function, before the `yield`, will be executed before the application starts", and "the part after the `yield` will be executed after the application has finished", with the older `startup/shutdown` handlers set aside, "It's all `lifespan` or all events, not both":

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    EVENTS.append("startup")
    yield
    EVENTS.append("shutdown")


def make_app() -> FastAPI:
    app = FastAPI(lifespan=lifespan)

    @app.get("/ping")
    async def ping() -> dict:
        EVENTS.append("endpoint start")
        LOOPS.append(asyncio.get_running_loop())
        await asyncio.sleep(0)
        EVENTS.append("endpoint end")
        return {"pong": True}

    @app.get("/later")
    async def later(tasks: BackgroundTasks) -> dict:
        EVENTS.append("later endpoint")
        tasks.add_task(EVENTS.append, "background ran")
        return {"queued": True}

    return app
```

listing 291: the async half: a lifespan context manager and two async endpoints, one queuing a background task (source: python/samples/src/Ch22/serve.py, lines 14-38)

The client that drives all of it never opens a port. `TestClient` is starlette's, the testing page says so plainly, "`FastAPI` provides the same `starlette.testclient` as `fastapi.testclient` just as a convenience for you, the developer. But it comes directly from Starlette", and "It is based on HTTPX, which in turn is designed based on Requests". The sample checks both facts as identities, the re-export is the same class object, and the class's own method resolution order names `httpx2.Client` as a base, which is why httpx2 2.12.0 sits in the requirements file as a test-only pin that no sample imports:

```
    if TestClient is not starlette.testclient.TestClient:
        raise SystemExit("FAIL fastapi's TestClient is not starlette's class")
    print("ok 9 fastapi's TestClient is starlette's, a plain re-export")

    engine = [f"{base.__module__}.{base.__name__}" for base in TestClient.__mro__]
    if "httpx2.Client" not in engine:
        raise SystemExit(f"FAIL engine classes are {engine}")
    print("ok 10 the client subclasses httpx2's engine, the test-only pin")
```

listing 292: the two engine identities: the re-export is one class, and its base is the httpx2 engine (source: python/samples/src/Ch22/serve.py, lines 102-109)

The loop facts came out of the probes and are worth the ink. A bare request through a plain `TestClient(app)` builds a fresh portal loop for that one request and discards it, two requests, two distinct loop objects. Inside `with TestClient(app) as held` one portal lives for the whole session, the same loop object serving both requests, and that is also the only mode in which lifespan runs, startup on enter, shutdown on exit, exactly bracketing the session:

```
def main() -> None:
    app = make_app()
    client = TestClient(app)

    r = client.get("/ping")
    if r.status_code != 200 or r.json() != {"pong": True}:
        raise SystemExit(f"FAIL ping is {r.status_code} {r.json()}")
    print("ok 1 an async def endpoint round trips through the client")

    if not LOOPS or not isinstance(LOOPS[0], asyncio.ProactorEventLoop):
        seen = [type(lop).__name__ for lop in LOOPS]
        raise SystemExit(f"FAIL loop seen inside is {seen}")
    print("ok 2 the endpoint runs on a live loop, the windows proactor of ch 19")

    if EVENTS != ["endpoint start", "endpoint end"]:
        raise SystemExit(f"FAIL suspension order is {EVENTS}")
    print("ok 3 await suspends inside the endpoint and the order survives")

    client.get("/ping")
    if LOOPS[0] is LOOPS[1]:
        raise SystemExit("FAIL two bare requests shared one loop")
    print("ok 4 a bare client builds a fresh portal loop for every request")
```

listing 293: the async round trip, the loop class from inside, and the fresh portal a bare request builds (source: python/samples/src/Ch22/serve.py, lines 41-62)

```
    LOOPS.clear()
    EVENTS.clear()
    with TestClient(app) as held:
        held.get("/ping")
        held.get("/ping")
    if LOOPS[0] is not LOOPS[1]:
        raise SystemExit("FAIL the with block rotated loops")
    print("ok 5 the context manager keeps one portal loop for the session")

    want = [
        "startup",
        "endpoint start",
        "endpoint end",
        "endpoint start",
        "endpoint end",
        "shutdown",
    ]
    if EVENTS != want:
        raise SystemExit(f"FAIL lifespan order is {EVENTS}")
    print("ok 6 lifespan brackets the session, startup on enter, shutdown on exit")
```

listing 294: one loop per session under the context manager, and the exact lifespan bracket asserted (source: python/samples/src/Ch22/serve.py, lines 64-83)

Background tasks are the small end of async work. The docs: “You can define background tasks to be run **after** returning a response”, and the class “comes directly from `starlette.background`”. The endpoint queues `EVENTS.append` and returns; the event log then shows the endpoint body first, the task second, before the shutdown that ends the session. A bare client, the last check says, runs no lifespan at all, which is the default worth knowing, tests that never enter the context manager never pay startup and never see the state it builds:

```
    EVENTS.clear()
    with TestClient(app) as held:
        r = held.get("/later")
        if r.status_code != 200 or r.json() != {"queued": True}:
            raise SystemExit(f"FAIL later is {r.status_code} {r.json()}")
        if EVENTS != ["startup", "later endpoint", "background ran"]:
            raise SystemExit(f"FAIL background order is {EVENTS}")
    if EVENTS[-1] != "shutdown":
        raise SystemExit(f"FAIL shutdown missing: {EVENTS}")
    print("ok 7 a background task runs after the endpoint body, before shutdown")

    EVENTS.clear()
    TestClient(app).get("/ping")
    if EVENTS != ["endpoint start", "endpoint end"]:
        raise SystemExit(f"FAIL a bare client ran lifespan: {EVENTS}")
    print("ok 8 a bare client never runs lifespan, only the with block does")
```

listing 295: background task ordering, and the bare client that never runs lifespan (source: python/samples/src/Ch22/serve.py, lines 85-100)

PITFALL: in process does not mean no socket object, and a monkeypatch will find the difference

While proving the no-network claim, a draft sample replaced `socket.socket` with a raising stub and ran a request through the client. The request failed inside `asyncio` itself: the portal's fresh `ProactorEventLoop` builds its wakeup self-pipe as a socketpair, so constructing a loop touches the socket module without ever touching a network. The honest claim, and the one this chapter makes, is narrower and true: no listener is opened, no connection is made, the windows firewall never asks, and the request is served by an in-process call. Assert what the machine proves, not the slogan.

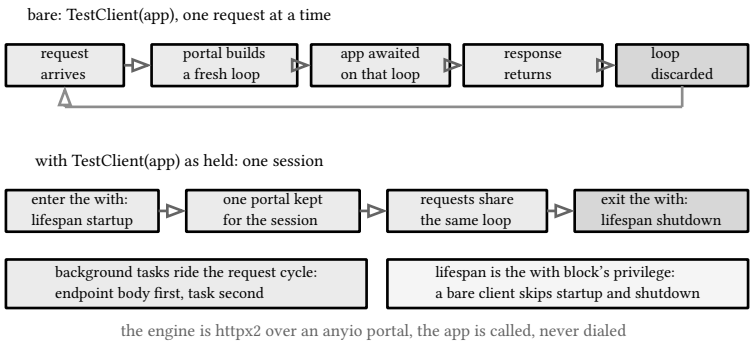


figure 94: the client's two modes as state machines: a fresh loop per request, or one held session with lifespan brackets

22.4. boundaries

The version line is 0.x, and the project's own numbering says so before any commentary does, the version string starts with 0, and the docs treat pre-1.0 as the reason to pin, the deployment pages open with the advice to "pin" the version to "the specific latest version that you know works correctly". The chapter does what the book always does, the exact pin in `books/python/requirements.txt` and the assert in-sample, and 0.141.1 sits above the release that made this book's interpreter legal, release notes 0.118.3, 2025-10-10, "Add support for Python 3.14":

```
def main() -> None:
    installed = md.version("fastapi")
    if installed ≠ "0.141.1" or fastapi.__version__ ≠ installed:
        raise SystemExit(
            f"FAIL metadata is {installed}, import says {fastapi.__version__}"
        )
    print("ok 1 installed metadata and the import agree on 0.141.1")

    if version_tuple(fastapi.__version__)[0] ≠ 0:
        raise SystemExit(f"FAIL the version left 0.x: {fastapi.__version__}")
    print("ok 2 the pin rides the 0.x line, pre 1.0 by the project's own numbering")

    base = [r for r in (md.requires("fastapi") or []) if ";" not in r]
    if "starlette≥0.46.0" not in base or "pydantic≥2.9.0" not in base:
        raise SystemExit(f"FAIL base requires are {base}")
    print("ok 3 the base install needs starlette, pydantic, and typing helpers")

    standard = [
        r.split(";")[0].strip()
        for r in (md.requires("fastapi") or [])
        if 'extra == "standard"' in r
    ]
    if len(standard) ≠ 9:
        raise SystemExit(f"FAIL the standard extra holds {len(standard)}: {standard}")
    print("ok 4 the standard extra drags nine packages behind it")
```

listing 296: the pin agreed between metadata and import, the 0.x line, and the standard extra's nine members counted (source: python/samples/src/Ch22/boundary.py, lines 17-41)

What the pin installs is deliberately thin. The base requirement list holds starlette, pydantic, and three typing helpers, nothing else, so a bare `pip install fastapi==0.141.1` ships no server and no client. The kitchen sink lives one bracket away, `pip install "fastapi[standard]"`, and the sample reads the installed metadata to count it, nine packages including `uvicorn[standard]`, `httpx`, jinja2, python-multipart, email-validator, pydantic-settings, pydantic-extra-types, the fastapi cli, and fastar. The docs' serving story rides that extra, the deployment pages describe "using the `fastapi` command, that runs Uvicorn, running a single process", with `--workers` for more, and this book stops at the prose. No sample serves a socket, and the venv makes that a checked fact rather than a promise:

```
    if not any(r.startswith("uvicorn[standard]") for r in standard):
        raise SystemExit(f"FAIL no uvicorn in {standard}")
    print("ok 5 the server, uvicorn, arrives only through the standard extra")

    if not any(r.startswith("httpx") for r in standard):
        raise SystemExit(f"FAIL no httpx in {standard}")
    print("ok 6 the docs' client, httpx, also rides the standard extra")

    if importlib.util.find_spec("uvicorn") is not None:
        raise SystemExit("FAIL uvicorn is installed in this venv")
    print("ok 7 uvicorn is absent from this venv, no sample can serve a socket")

    if importlib.util.find_spec("httpx2") is None:
        raise SystemExit("FAIL httpx2 is missing from this venv")
    if md.version("httpx2") != "2.12.0":
        raise SystemExit(f"FAIL httpx2 is {md.version('httpx2')}, not 2.12.0")
    if importlib.util.find_spec("httpx") is not None:
        raise SystemExit("FAIL the deprecated httpx is still installed")
    print("ok 8 the engine pin is httpx2 2.12.0, the deprecated httpx gone")

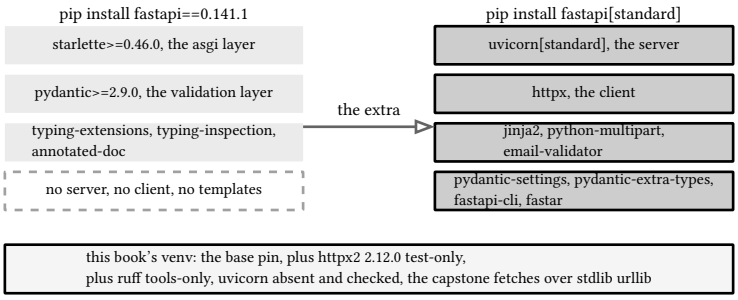
    with warnings.catch_warnings(record=True) as caught:
        warnings.simplefilter("always")
        importlib.reload(starlette.testclient)
    noted = [w for w in caught if issubclass(w.category, StarletteDeprecationWarning)]
    if noted:
        seen = [str(w.message) for w in noted]
        raise SystemExit(f"FAIL the engine load warned: {seen}")
    print("ok 9 the preferred engine loads with no deprecation warning")

    if sys.version_info[:2] != (3, 14):
        raise SystemExit(f"FAIL interpreter is {sys.version_info[:2]}")
    if version_tuple("0.141.1") < version_tuple("0.118.3"):
        raise SystemExit("FAIL the pin sits below the 3.14 support line")
    print("ok 10 python 3.14, above the 0.118.3 line that added its support")
```

listing 297: uvicorn absent, the engine pin agreed at httpx2 2.12.0, the warning-free load, and the 3.14 support line (source: python/samples/src/Ch22/boundary.py, lines 43-76)

The engine choice is the last boundary, and starlette 1.6.0 makes it in import order: the installed testclient tries `import httpx2 as httpx` first, and only when httpx2 is missing does it fall back to plain httpx, warning while it does, Using `httpx` with `starlette.testclient` is deprecated; install `httpx2` instead. This venv installs httpx2 2.12.0 as the test-only pin, so the first import wins, the deprecated name is gone from the venv entirely, and the sample proves the clean load the direct way, reloading the module under a recording filter and counting zero deprecation warnings. One wrinkle worth knowing before reading any stack trace: fastapi's own metadata still lists httpx in its standard extra, so the client the docs install for you and the engine starlette actually loads are two different names on this line. And when a future starlette drops the httpx fallback outright, nothing here changes, the pin already sits on the preferred side.

The client side of the capstone keeps its own boundary. book 8, ch 40 (python 3.14, capstone: ingest and analysis pipeline, part 1) fetches over stdlib urllib in worker threads, so the httpx2 pin never migrates from test-only to runtime, and book 8, ch 41 (python 3.14, capstone: ingest and analysis pipeline, part 2) serves the report api over fastapi with the TestClient suite from this chapter. What this chapter's api does not do is the rest of the list: no database, no authentication beyond a query key, no middleware, no websockets, no openapi customization, all of it fastapi surface the docs cover and the capstone does not need. The chapter's job was the machine under the decorators, and every layer of it is pinned.



serving under uvicorn stays prose: the deployment pages' fastapi command runs uvicorn

figure 95: the two installs, before and after the extra

VERIFY: 45 checks, every one deterministic on this box

A scoped run, `pwsh tools/run-py-samples.ps1 -Chapter Ch22`, resolves the toolchain and the hashed venv, then walks this chapter's four samples through the run, format, and dis legs and reports: 4 files, 45 checks, 0 dis assertions, format clean. `model.py` contributes 14, the pin, the subclass and signature facts, the raw asgi call, routing, the default documentation routes, and factory isolation. `validate.py` adds 11, the 201 and 204 contracts, the response model filter, the 422 shapes for body, path, and query, the two error surfaces, the rejecting dependency, the per-request cache, and the dependency that ran before a rejected body. `serve.py` adds 10, the async round trip, the proactor loop from inside, the portal contrast between bare and context managed clients, the lifespan bracket, background task ordering, and the two engine identities. `boundary.py` adds 10, the metadata agreement, the 0.x line, the standard extra counted at nine, uvicorn absent, the engine pin agreed at httpx2 2.12.0 with the deprecated name gone, the warning-free engine load, and the 3.14 support line. No sample opens a socket, installs a package, or runs a server. The scoped run leaves the capstone leg to the unscoped gate by design, and the dis leg is fully live with no bytecode rows pinned in this chapter.

sources: fastapi.tiangolo.com/tutorial/first-steps (the FastAPI class, the Starlette inheritance note, the path operation decorator, the docs and openapi routes), fastapi.tiangolo.com/tutorial/query-params (non path parameters read as query, conversion and validation, the 422 example shape), fastapi.tiangolo.com/tutorial/dependencies (a dependency takes the same parameters as a path operation, what fastapi does on each request), fastapi.tiangolo.com/tutorial/dependencies/sub-dependencies (the sub-dependency called once per request, the cache, use_cache), fastapi.tiangolo.com/tutorial/testing (the docs' httpx basis for TestClient, the starlette re-export), fastapi.tiangolo.com/advanced/events (the lifespan parameter, before and after the yield, all lifespan or all events), fastapi.tiangolo.com/tutorial/background-tasks (tasks after the response, the starlette origin of the class), fastapi.tiangolo.com/deployment/server-workers (the fastapi command running uvicorn, single process, workers), fastapi.tiangolo.com/deployment/versions (the advice to pin), fastapi.tiangolo.com/release-notes (0.118.3, add support for Python 3.14), and pypi.org/project/fastapi (the 0.141.1 pin and its metadata), accessed 2026-09-12. The engine facts are probed in the book venv the same day: starlette 1.6.0 beneath, the `import httpx2 as httpx` engine order read out of the installed starlette's testclient source with its httpx fallback warning, the standard extra's nine members, and the portal's `ProactorEventLoop` observed from inside an async endpoint. Sample behavior verified by `make verify-py`, 45 checks in chapter 22 of the samples suite.

23. stdlib numerics: math, statistics, decimal, random

Before book 8, ch 24 (python 3.14, numpy) puts numbers into arrays, the standard library already answers the four questions this chapter walks through in order. How do floats actually round, and why does `0.1 + 0.2` miss, the `math` and `cmath` answer, plus the integer boundary facts, 53 bit mantissas and the 4300 digit conversion limit. What does a one pass summary look like, the `statistics` answer, exact types in, exact types out, and a `NormalDist` object that inverts its own cdf. When must arithmetic be exact, the `decimal` answer, money from strings with a context that rounds on results only, and `fractions` for exact rationals. And where does randomness come from, the `random` answer, one mersenne twister behind module functions, instances you can own, and the hard line where `secrets` takes over. Every behavioral claim below is an ok line in the four samples, 74 in total, or a sentence quoted from a page at docs.python.org fetched 2026-09-12. The array and dataframe machinery of book 8, ch 24 (python 3.14, numpy) and book 8, ch 25 (python 3.14, pandas) starts from these semantics, it does not replace them.

23.1. math and cmath

Four different operations turn a float toward an integer, and they disagree in the negative half. `math.floor(-3.5)` is −4, “the largest integer less than or equal to **x**”, `math.ceil(-3.5)` is −3, “the smallest integer greater than or equal to **x**”, `int(-3.5)` is −3 because int conversion truncates toward zero, and `math.trunc(-3.5)` is documented as that truncation, “equivalent to `floor()` for positive **x**, and equivalent to `ceil()` for negative **x**”. The `//` operator sides with floor: `-7 // 2` is −4, not −3:

```
    (
        math.floor(-3.5) == -4
        and math.ceil(-3.5) == -3
        and isinstance(math.floor(-3.5), int),
        "floor and ceil return ints that bound the value, floor down at -3.5",
    ),
    (
        -7 // 2 == -4 and 7 // 2 == 3,
        "the // operator floors, so -7 // 2 is -4 not -3",
    ),
    (
        7.0 // 2 == 3.0 and isinstance(7.0 // 2, float),
        "float // float stays float, still floored",
    ),
    (
        int(-3.5) == -3 and math.trunc(-3.5) == -3 and math.floor(-3.5) == -4,
        "int and trunc cut toward zero, floor and // go more negative",
    ),
```

listing 298: the four integerings of −3.5 and −7, `floor` and `//` go down, `int` and `trunc` go toward zero (source: python/samples/src/Ch23/mathmod.py, lines 14-31)

Floats are binary64 and the sample shows the representation limits concretely rather than describing them. `0.1 + 0.2` is the neighbor double `0.30000000000000004`, `repr` picks the shortest string that round trips while 17 significant digits reveal what is stored, and past `2**53` the last mantissa bit is gone, so `float(2**63 + 1)` reads back as `2**63`. Accumulation drift is demonstrated the honest way, a manual `+=` loop lands on `0.9999999999999999` while `math.fsum`, which “avoids loss of precision by tracking multiple intermediate partial sums”, lands exactly on 1.0:

```
    (
        repr(0.1 + 0.2) == "0.30000000000000004" and 0.1 + 0.2 != 0.3,
        "0.1 + 0.2 is the neighbor double 0.30000000000000004",
    ),
    (
        repr(0.1) == "0.1" and format(0.1, ".17g") == "0.10000000000000001",
        "repr picks the shortest round trip, 17 digits show what is stored",
    ),
```

listing 299: the neighbor double, and the shortest repr versus the 17 digit truth (source: python/samples/src/Ch23/mathmod.py, lines 32-39)

```
    (
        float(2**53 + 1) == 2**53 and int(float(2**63 + 1)) == 2**63,
        "past 2**53 the last mantissa bit is gone, 2**63 + 1 reads back as 2**63",
    ),
    (
        accumulated() == 0.9999999999999999 and math.fsum([0.1] * 10) == 1.0,
        "manual += drifts to 0.9999999999999999, fsum lands exactly on 1.0",
    ),
```

listing 300: the lost bit past 2**53, and the += drift that fsum corrects (source: python/samples/src/Ch23/mathmod.py, lines 53-60)

Equality on top of that representation is `math.isclose`, with the contract printed as a formula: “the result will be: `abs(a-b) ≤ max(rel_tol * max(abs(a), abs(b)), abs_tol)`”, default `rel_tol=1e-09`, `abs_tol=0.0`. The default relative tolerance arbitrates the `0.1 + 0.2` case, but “near zero only `abs_tol` can say close, rel_tol of 0 is 0”, and the special values follow their own rules, “NaN is not considered close to any other value, including NaN. `inf` and `-inf` are only considered close to themselves”:

```
    (
        math.isclose(0.1 + 0.2, 0.3) and not math.isclose(1.0, 1.001),
        "isclose with the default rel_tol 1e-9 arbitrates float equality",
    ),
    (
        not math.isclose(1e-11, 0.0) and math.isclose(1e-11, 0.0, abs_tol=1e-9),
        "near zero only abs_tol can say close, rel_tol of 0 is 0",
    ),
    (
        math.isclose(math.inf, math.inf)
        and not math.isclose(float("nan"), float("nan")),
        "inf is close only to itself, nan to nothing, not even nan",
    ),
```

listing 301: isclose with the default, the abs_tol leg near zero, and the inf and nan rules (source: python/samples/src/Ch23/mathmod.py, lines 40-52)

The special values are citizens, not errors. Overflow “saturates”, the sample reaches `inf` both by literal, `float("1e400")`, and by arithmetic, `1e308 * 10`, nan never compares equal, `0.0` “is considered finite” while inf is not, and `-0.0` compares equal to `0.0` with `copysign(1.0, -0.0)` returning −1.0, the only witness to the sign bit:

```
        (
            float("1e400") == math.inf
            and 1e308 * 10 == math.inf
            and math.isinf(1e308 * 10),
            "overflow saturates to inf, by literal or by arithmetic",
        ),
        (
            not nans_compare_equal()
            and math.isnan(float("nan"))
            and not math.isfinite(math.inf)
            and math.isfinite(0.0),
            "no two nans compare equal, and 0.0 is finite while inf is not",
        ),
        (
            -0.0 == 0.0 and math.copysign(1.0, -0.0) == -1.0,
            "minus zero compares equal, only copysign exposes the sign bit",
        ),
```

listing 302: inf by two roads, nan unequal to itself and to any other nan, minus zero exposed only by copysign (source: python/samples/src/Ch23/mathmod.py, lines 61-77)

The module boundary is one sentence on the math page: “These functions cannot be used with complex numbers; use the functions of the same name from the `cmath` module if you require support for complex numbers.” `math.sqrt(-1)` is a `ValueError`, and 3.14 spells the domain out in the message, “expected a nonnegative input, got −1.0”, the improved error text this release added. `cmath.sqrt(-1)` is `1j` across the branch cut, `cmath.polar(3+4j)` is radius 5 and the angle, `cmath.rect` rebuilds `1j` from `pi/2` up to the 6.1e-17 error in `cos`:

```
        (
            sqrt_error() == "expected a nonnegative input, got -1.0",
            "math.sqrt(-1) is a ValueError, 3.14 spells out the domain",
        ),
        (
            cmath.sqrt(-1) == 1j and cmath.sqrt(4) == 2 + 0j,
            "cmath.sqrt crosses the branch, -1 gives 1j and 4 gives 2+0j",
        ),
        (
            cmath.polar(3 + 4j) == (5.0, 0.9272952180016122),
            "polar returns modulus and phase, 3+4j is radius 5",
        ),
        (
            abs(cmath.rect(1, cmath.pi / 2) - 1j) < 1e-15,
            "rect(1, pi/2) rebuilds 1j up to the 6.1e-17 error in cos",
        ),
```

listing 303: math refuses complex input with the new domain message, cmath crosses the branch and polar and rect round the plane (source: python/samples/src/Ch23/mathmod.py, lines 78-93)

Integers have no such ceiling, “Python floats typically carry no more than 53 bits of precision”, but ints are arbitrary precision and `2**200` carries all 61 digits. The one integer boundary is in base 10: converting between int and decimal string is quadratic, so the interpreter limits it, and the sample asserts the defaults, `sys.get_int_max_str_digits()` at 4300 with the check threshold at 640, then trips both directions, `int` of a 5000 digit string and `str` of a 5001 digit int are `ValueError`, while `hex` and back round trips freely since power of two bases are linear:

```
        (
            len(str(2**200)) == 61 and str(2**200).startswith("160693804425"),
            "ints are arbitrary precision, 2**200 carries all 61 digits",
        ),
        (
            int_limit(),
            "the str int limit defaults to 4300 digits, and int() refuses 5000",
        ),
        (
            str_limit(),
            "str of a 5001 digit int refuses too, but hex round trips freely",
        ),
```

listing 304: arbitrary precision ints, the 4300 digit default limit, and the hex path that ignores it (source: python/samples/src/Ch23/mathmod.py, lines 94-105)

```
def int_limit():
    if sys.get_int_max_str_digits() != 4300:
        return False
    if sys.int_info.default_max_str_digits != 4300:
        return False
    if sys.int_info.str_digits_check_threshold != 640:
        return False
    try:
        int("9" * 5000)
    except ValueError:
        return True
    return False


def str_limit():
    try:
        str(10**5000)
    except ValueError:
        return int(hex(2**200), 16) == 2**200
```

listing 305: the limit probed from both sides, int of a string and str of an int, hex unaffected (source: python/samples/src/Ch23/mathmod.py, lines 134-153)

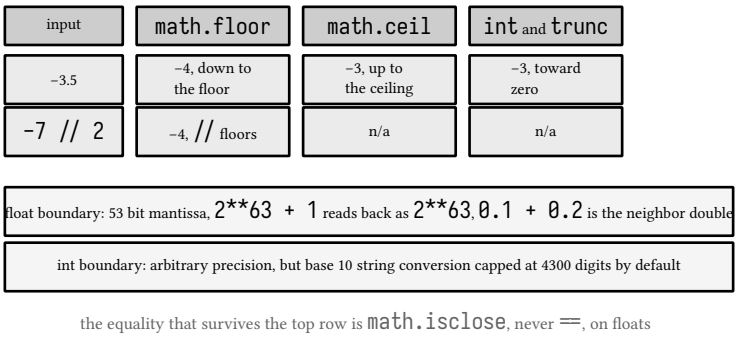


figure 96: the integerings as a matrix: one input, four rules, plus the two boundary rows where int and float stop being freely convertible

23.2. statistics

The module's type rule is the first sentence of the page: "Unless explicitly noted, these functions support `int`, `float`, `Decimal` and `Fraction`." The mean family shows what that buys. `mean` preserves the input's exactness, fractions in

give `Fraction(13, 21)` out, decimals give `Decimal("0.5625")`, while `fmean` is the float fast path, "Convert **data** to floats and compute the arithmetic mean. This runs faster than the `mean()` function and it always returns a `float`":

```
    (
        mean([1, 2, 3, 4]) == 2.5 and isinstance(mean([1, 2, 3, 4]), float),
        "mean of ints returns the exact float 2.5",
    ),
    (
        mean([Fraction(3, 7), Fraction(1, 21), Fraction(5, 3), Fraction(1, 3)])
        == Fraction(13, 21),
        "mean over fractions stays an exact fraction, 13/21",
    ),
    (
        mean([Decimal("0.5"), Decimal("0.75"), Decimal("0.625"), Decimal("0.375")])
        == Decimal("0.5625"),
        "mean over decimals stays an exact decimal, 0.5625",
    ),
    (
        fmean([1, 2, 3, 4]) == 2.5 and isinstance(fmean([1, 2, 3, 4]), float),
        "fmean converts to float first and always returns a float",
    ),
```

listing 306: mean keeps fractions and decimals exact, fmean converts to float and stays there (source: python/samples/src/Ch23/stats.py, lines 28-45)

Position and frequency need no arithmetic at all. `median` picks the middle value or the mean of the middle pair, `mode` returns the first most common value on ties, "if there are multiple modes with the same frequency, returns the first one encountered in the **data**", and works on names, and `multimode` lists every leader "in the order they were first encountered":

```
    (
        median([1, 3, 5]) == 3 and median([1, 2, 3, 4]) == 2.5,
        "median picks the middle, or the mean of the middle pair",
    ),
    (
        mode(["red", "blue", "blue", "red", "green", "red", "red"]) == "red",
        "mode returns the first most common value, works on names",
    ),
    (
        multimode("aabbbbccddddeeffffgg") == ["b", "d", "f"],
        "multimode lists every leader in first seen order",
    ),
```

listing 307: median, mode with first tie win on names, multimode listing every leader (source: python/samples/src/Ch23/stats.py, lines 46-57)

Spread comes in two denominators and the docs name them exactly: `pstdev` is "the population standard deviation", `stdev` "the sample standard deviation", n versus n-1, and the sample version is always the larger one. Both floats assert to their last digit, 0.986893273527251 against 1.0810874155219827 on the docs' own six value dataset:

```
    (
        pstdev([1.5, 2.5, 2.5, 2.75, 3.25, 4.75]) == 0.986893273527251
        and stdev([1.5, 2.5, 2.5, 2.75, 3.25, 4.75]) == 1.0810874155219827,
        "pstdev divides by n, stdev by n-1, both floats exact",
    ),
    (
        pstdev([2.0, 4.0]) < stdev([2.0, 4.0]),
        "the sample stdev always exceeds the population stdev",
    ),
```

listing 308: population versus sample stdev, exact floats, n-1 always the bigger number (source: python/samples/src/Ch23/stats.py, lines 58-66)

`quantiles` cuts data into equal probability intervals, "Returns a list of **n - 1** cut points", default `n=4` for quartiles, and the method default matters, `'exclusive'`, "used for data sampled from a population that can have more extreme values than found in the samples", versus `'inclusive'` which lands the cuts on data points. On 1 through 5 the two give `[1.5, 3.0, 4.5]` and `[2.0, 3.0, 4.0]`:

```
    (
        quantiles([1, 2, 3, 4, 5]) == [1.5, 3.0, 4.5],
        "quantiles defaults to n=4 and the exclusive method, 3 cut points",
    ),
    (
        quantiles([1, 2, 3, 4, 5], method="inclusive") == [2.0, 3.0, 4.0],
        "the inclusive method lands the cuts on data points instead",
    ),
```

listing 309: quartiles under both methods, defaults pinned to exclusive (source: python/samples/src/Ch23/stats.py, lines 67-74)

`NormalDist` is "a tool for creating and manipulating normal distributions of a random variable", and it is bidirectional: `cdf(0.0)` is exactly 0.5, `inv_cdf(0.975)` is 1.9599639845400536, the standard two sided 5 percent cut, and scaling to mean 100 sigma 15 moves it to 129.3994597681008. Its `samples` method takes a seed and "creates a new instance of the underlying random number generator", so the seeded sample list asserts exactly and repeats:

```
    (
        NormalDist().cdf(0.0) == 0.5
        and NormalDist().inv_cdf(0.975) == 1.9599639845400536,
        "the standard normal: cdf(0) is 0.5, inv_cdf(0.975) is 1.9599...",
    ),
    (
        NormalDist(100, 15).inv_cdf(0.975) == 129.3994597681008,
        "a scaled normal moves the 97.5th percentile to 129.4",
    ),
    (
        NormalDist().samples(3, seed=7)
        == [-0.45700767402282655, -1.0327981966610056, 0.38784457831235297]
        and NormalDist().samples(3, seed=7)
        == [-0.45700767402282655, -1.0327981966610056, 0.38784457831235297],
        "NormalDist samples with a seed are reproducible float lists",
    ),
```

listing 310: cdf and inv_cdf both exact, and seeded samples reproducible to the last float (source: python/samples/src/Ch23/stats.py, lines 75-90)

The mean family has two more members with their own domains: `harmonic_mean([40, 60])` is 48.0, "the reciprocal of the arithmetic `mean()` of the reciprocals", and the error paths are part of the contract, `mean` of nothing and `geometric_mean` of a negative are both `StatisticsError`:

```
    (
        harmonic_mean([40, 60]) == 48.0,
        "harmonic mean of 40 and 60 is 48, the reciprocal average",
    ),
    (
        error_kinds() == ("mean empty", "geometric negative"),
        "mean of nothing and geometric_mean of a negative both raise",
    ),
```

listing 311: harmonic mean exact, and the two empty or invalid datasets raising cleanly (source: python/samples/src/Ch23/stats.py, lines 91-98)

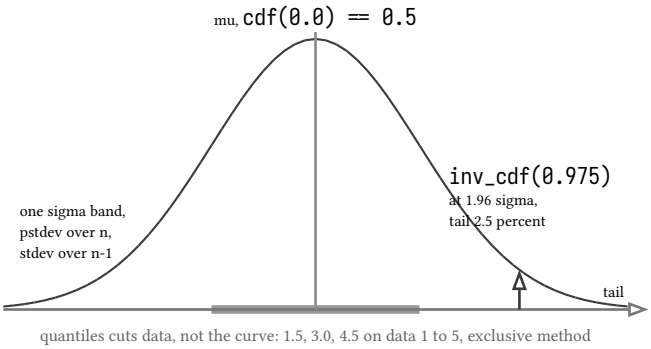


figure 97: the standard normal as a curve: cdf at zero, the 2.5 percent tail beyond 1.96 sigma, the one sigma band the two stdevs measure

23.3. decimal and fractions

The before and after is one expression. In binary floating point `0.1 + 0.1 + 0.1 - 0.3` drifts to `5.551115123125783e-17`, which the decimal page names as the reason "differences prevent reliable equality testing and differences can accumulate". The same expression over decimal literals is exactly zero, because each literal is the decimal it says it is. The everyday form agrees, `Decimal("0.1") + Decimal("0.2")` is `Decimal("0.3")`:

```
    (
        repr(0.1 + 0.1 + 0.1 - 0.3) == "5.551115123125783e-17"
        and Decimal("0.1") + Decimal("0.1") + Decimal("0.1") - Decimal("0.3")
        == Decimal(0),
        "the float drifts by 5.5e-17, the decimal version is exactly zero",
    ),
    (
        Decimal("0.1") + Decimal("0.2") == Decimal("0.3"),
        "decimal literals add to the literal answer, 0.3",
    ),
```

listing 312: the drift and the zero, same expression, two number systems (source: python/samples/src/Ch23/exact.py, lines 22-31)

Significance is part of the value. The module "incorporates a notion of significant places so that `1.30 + 1.20` is `2.50`. The trailing zero is kept to indicate significance. This is the customary presentation for monetary applications", and multiplication goes further, "`1.3 * 1.2` gives `1.56` while `1.30 * 1.20` gives `1.5600`", every input figure used:

```
    (
        str(Decimal("1.30") + Decimal("1.20")) == "2.50"
        and Decimal("1.3") * Decimal("1.2") == Decimal("1.56"),
        "1.30 + 1.20 keeps the trailing zero, significance is preserved",
    ),
    (
        str(Decimal("1.30") * Decimal("1.20")) == "1.5600",
        "schoolbook multiplication uses every input figure, 1.5600",
    ),
```

listing 313: trailing zeros survive the add, and multiplication uses every input figure (source: python/samples/src/Ch23/exact.py, lines 32-40)

The context is where rounding lives, and its defaults are assertable: "The default values are `Context.prec=28`, `Context.rounding= ROUND_HALF_EVEN`", with traps enabled for `Overflow`, `InvalidOperation`, and `DivisionByZero`. Its scope is narrower than newcomers expect, "The significance of a new Decimal is determined solely by the number of digits input. Context precision and rounding only come into play during arithmetic operations", and the FAQ's phrasing is the rule to remember: "It occurs **after** the computation", the inputs stay whole and the result rounds. An 11 digit pi constructs at all 11 digits, then `pi + 0` inside a `localcontext` of precision 5 rounds to `3.1416`, and `1/3` carries 28 threes at the default and 6 inside a scoped context:

```
    (
        getcontext().prec == 28
        and str(getcontext().rounding) == "ROUND_HALF_EVEN"
        and getcontext().traps[InvalidOperation],
        "the default context: precision 28, half even rounding, traps on",
    ),
    (
        str(Decimal("3.1415926535")) == "3.1415926535"
        and len(Decimal("3.1415926535").as_tuple().digits) == 11,
        "construction stores every input digit, the context has no say yet",
    ),
```

listing 314: the default context asserted: precision 28, half even, traps armed (source: python/samples/src/Ch23/exact.py, lines 41-51)

```
    (
        rounded_pi() == Decimal("3.1416"),
        "arithmetic rounds the result, pi + 0 at precision 5 is 3.1416",
    ),
    (
        Decimal(1) / Decimal(3) == Decimal("0.33333333333333333333333333333")
        and third_at(6) == Decimal("0.333333"),
        "1/3 carries 28 threes at the default, 6 inside localcontext",
    ),
```

listing 315: construction keeps every digit, arithmetic rounds the result, localcontext scopes the change (source: python/samples/src/Ch23/exact.py, lines 52-60)

Money is the application. Prices as decimal strings sum exactly, 19.99 and 0.01 and 20.00 and 0.01 total 40.01 with no drift anywhere, and `quantize` puts the cent back on a sum that wandered: `2.675` at two places rounds half even to `2.68` while the float path through `round(2.675, 2)` gives 2.67, because the float nearest 2.675 is below it. The traps are load bearing, quantize "would be greater than precision" signals `InvalidOperation` rather than silently mangling, and division by zero raises `DivisionByZero`:

```
    (
        sum(
            (
                Decimal("19.99"),
                Decimal("0.01"),
                Decimal("20.00"),
                Decimal("0.01"),
            ),
            Decimal(0),
        )
        == Decimal("40.01"),
        "money sums from string cents are exact, 40.01 with no drift",
    ),
```

listing 316: cents from strings, summed with zero drift (source: python/samples/src/Ch23/exact.py, lines 61-73)

```
    (
        Decimal("2.675").quantize(Decimal("0.01")) == Decimal("2.68")
        and round(2.675, 2) == 2.67,
        "quantize half even gives 2.68, the float path gives 2.67",
    ),
    (
        Decimal("7.325").quantize(Decimal("0.01"), rounding=ROUND_DOWN)
        == Decimal("7.32"),
        "quantize with round down truncates toward zero at the cent",
    ),
    (
        quantize_overflow() == "InvalidOperation",
        "quantize past context precision signals InvalidOperation, it is trapped",
    ),
    (
        divide_by_zero() == "DivisionByZero",
        "decimal division by zero raises instead of returning infinity",
    ),
```

listing 317: quantize half even versus the float path, round down, and both traps firing (source: python/samples/src/Ch23/exact.py, lines 74-91)

The float door is documented and exact in its own way: "If **value** is a `float`, the binary floating-point value is losslessly converted to its exact decimal equivalent. This conversion can often require 53 or more digits of precision", so `Decimal.from_float(0.1)` is the 55 digit expansion of the stored double and is not `Decimal("0.1")`. Python 3.14 adds `Decimal.from_number()` as "an alternative constructor", the same conversion by another door:

```
    (
        str(Decimal.from_float(0.1))
        == "0.1000000000000000055511151231257827021181583404541015625"
        and Decimal.from_number(0.1) == Decimal.from_float(0.1)
        and Decimal.from_number(0.1) != Decimal("0.1"),
        "decimal from a float converts the double exactly, 55 digits",
    ),
```

listing 318: from float and the new from_number, both the exact 55 digit double (source: python/samples/src/Ch23/exact.py, lines 92-98)

`fractions` is exactness without a context. `Fraction(1, 3) + Fraction(1, 6)` is `Fraction(1, 2)`, `Fraction(1, 3) * 3` is the int 1, and the constructors are the whole map: from string `Fraction("0.1")` is exactly 1/10, from float `Fraction(0.1)` is "the exact binary ratio", `3602879701896397 / 36028797018963968`, matching `(0.1).as_integer_ratio()` digit for digit. Python 3.14 widens the

door twice, "A `Fraction` object may now be constructed from any object with the `as_integer_ratio()` method", which is how `Fraction(Decimal("0.1"))` is 1/10, and `Fraction.from_number()` joins decimal's matching constructor. `limit_denominator` closes the section with the classic: the best rational approximation of pi with denominator under 1000 is 355/113:

```
        (
            Fraction(1, 3) + Fraction(1, 6) == Fraction(1, 2)
            and Fraction(1, 3) * 3 == 1,
            "fraction add and multiply stay exact, times 3 is the int 1",
        ),
        (
            Fraction("0.1") == Fraction(1, 10)
            and Fraction(0.1) == Fraction(3602879701896397, 36028797018963968)
            and (0.1).as_integer_ratio() == (3602879701896397, 36028797018963968),
            "fraction from string is 1/10, from float it is the exact binary ratio",
        ),
        (
            Fraction(Decimal("0.1")) == Fraction(1, 10)
            and Fraction.from_number(0.1) == Fraction(0.1),
            "3.14 builds fractions from any as_integer_ratio object and from_number",
        ),
```

listing 319: exact arithmetic, and the three construction doors including the two new in 3.14 (source: python/samples/src/Ch23/exact.py, lines 99-114)

```
        (
            Fraction(math.pi).limit_denominator(1000) == Fraction(355, 113),
            "limit_denominator finds 355/113, the famous approximation of pi",
        ),
        (
            float(Fraction(1, 3)) == 0.3333333333333333
            and Fraction(float(Fraction(1, 3))) != Fraction(1, 3),
            "the float door loses information, the round trip does not come back",
        ),
```

listing 320: limit_denominator finds 355/113, and the float door loses the round trip (source: python/samples/src/Ch23/exact.py, lines 115-123)

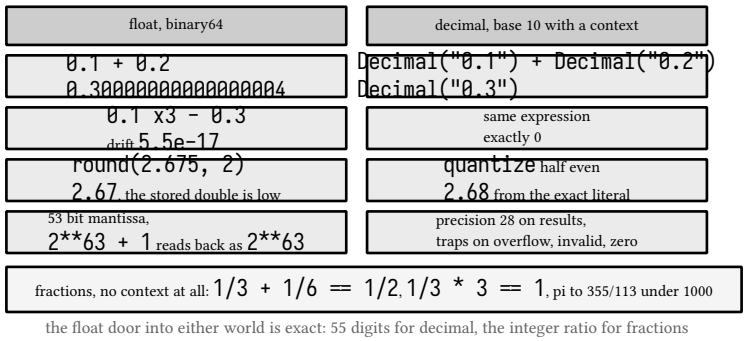


figure 98: before and after: the same four computations in float and in decimal, fractions exact off to the side

23.4. random

The module's own description is the state machine: "Python uses the Mersenne Twister as the core generator. It produces 53-bit precision floats and has a period of 2**19937-1. The underlying implementation in C is both fast and threadsafe", and it "is completely deterministic", which is the feature. Seeding is the entry point, `random.seed(4242)`

twice replays the same three float sequence while 4243 diverges, and a string seed uses "all of its bits" under the version 2 conversion:

```
def reseeded():
    random.seed(4242)
    first = [random.random() for _ in range(3)]
    random.seed(4242)
    second = [random.random() for _ in range(3)]
    random.seed(4243)
    other = [random.random() for _ in range(3)]
    return first == second and first != other
```

listing 321: reseeding replays the stream, a different seed diverges (source: python/samples/src/Ch23/randommod.py, lines 73-80)

The module functions are not free floating. "The functions supplied by this module are actually bound methods of a hidden instance of the `random.Random` class. You can instantiate your own instances of `Random` to get generators that don't share state." Both halves are checks: after `random.seed(1234)` the module's first draw equals `Random(1234)`'s first draw, and heavy module drawing never moves a seeded instance's stream, its next value still matches a fresh same seeded instance's second draw:

```
def module_matches_instance():
    random.seed(1234)
    module_value = random.random()
    instance_value = random.Random(1234).random()
    return module_value == instance_value


def instances_isolated():
    fresh = random.Random(777)
    fresh.random()
    expected_second = fresh.random()

    rng = random.Random(777)
    rng.random()
    random.seed(1)
    for _ in range(5):
        random.random()
    return rng.random() == expected_second
```

listing 322: the hidden instance and your own: same seed same draw, module draws never perturb an instance (source: python/samples/src/Ch23/randommod.py, lines 83-100)

The draws are contract bound, floats "uniformly in the half-open range `0.0 ≤ X < 1.0`", checked across 1000 seeded draws. The selection family splits by replacement: `sample` is "random sampling without replacement" whose members "need not be hashable or unique" and whose result is "a new list containing elements from the population while leaving the original population unchanged", asserted on the exact selection order, while `shuffle` permutes in place and sorted order recovers the original. `choices` draws with replacement and takes weights, the sample pins a zero weight never appearing, and the error contract is part of the api, "A `ValueError` is raised if all weights are zero" and "a `TypeError` to specify both **weights** and **cum_weights**":

```
def sampled():
    population = list(range(100))
    picked = random.Random(9).sample(population, 6)
    return (
        picked == [59, 78, 47, 34, 17, 23]
        and len(set(picked)) == 6
        and population == list(range(100))
    )
```

listing 323: sample without replacement, exact order, population untouched (source: python/samples/src/Ch23/randommod.py, lines 103-110)

```
        (
            random.Random(3).choices(["a", "b"], weights=[1, 0], k=5)
            == ["a", "a", "a", "a", "a"],
            "choices with replacement honors a zero weight, b never appears",
        ),
        (
            weight_errors() == ("zero total", "both kinds"),
            "all zero weights is a ValueError, both weight kinds is a TypeError",
        ),
        (
            bytes_check(),
            "randbytes returns n bytes, and consecutive draws differ",
        ),
        (state_replay(), "getstate plus setstate replays the same value twice"),
```

listing 324: choices honoring a zero weight, and randbytes length plus difference (source: python/samples/src/Ch23/randommod.py, lines 37-50)

State is inspectable and restorable, `getstate` before a draw and `setstate` after replays the same value, which is how a test can branch a random run without reseeding the world. And the boundary the module draws itself is quoted in its warning, verbatim: “The pseudo-random generators of this module should not be used for security purposes. For security or cryptographic uses, see the `secrets` module.” The sample checks the `secrets` surface, 16 hex characters from `token_hex(8)`, 4 bytes from `token_bytes`, neither ever repeating across calls, because that path rides `os.urandom`, not the twister:

```
def weight_errors():
    seen = []
    try:
        random.Random(1).choices(["a", "b"], weights=[0, 0], k=1)
    except ValueError:
        seen.append("zero total")
    try:
        random.Random(1).choices(["a"], weights=[1], cum_weights=[1], k=1)
    except TypeError:
        seen.append("both kinds")
    return tuple(seen)
```

listing 325: the weights error contract, both errors raised on demand (source: python/samples/src/Ch23/randommod.py, lines 126-136)

```
def state_replay():
    rng = random.Random(13)
    state = rng.getstate()
    drawn = rng.random()
    rng.setstate(state)
    return rng.random() == drawn
```

listing 326: getstate and setstate replay one value, the branch point for tests (source: python/samples/src/Ch23/randommod.py, lines 146-151)

WARNING: the line between random and secrets is about the attacker

The mersenne twister is deterministic by design, and reproducibility is what makes seeded simulations and golden tests possible. Deterministic is precisely what a token must never be: after observing a few hundred outputs the twister's state is recoverable and every future value is known. Anything an adversary could influence or observe, session ids, salts, tokens, reset links, belongs to `secrets`, which is why the module's own warning is the loudest sentence on the page. book 8, ch 19 (python 3.14, operational plumbing) places this same line in the operational context of hashing and secrets handling.

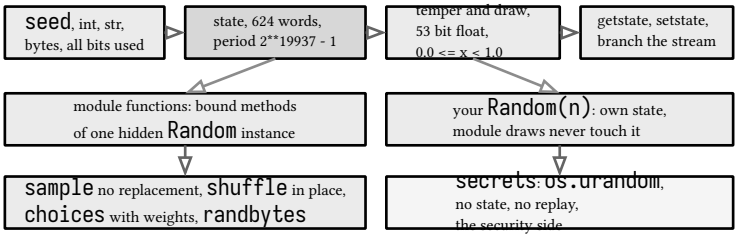


figure 99: random as a state machine: seed into 624 words of state, next per draw, module functions bound to the hidden instance, secrets on a different road entirely

VERIFY: 74 checks, machine pinned, no approximations in the asserts

`mathmod.py` pins the interpreter at 3.14.7 and contributes 22 checks, the four integerings, the neighbor double and the 53 bit boundary, the isclose contract with its special values, the accumulation drift against fsum, cmath across the branch, and both directions of the 4300 digit conversion limit. `stats.py` adds 17, exact types preserved through mean, the two denominators, both quantile methods, cdf and inv_cdf to the last digit, and the error paths. `exact.py` adds 20, the drift and the zero, significance arithmetic, the asserted default context, scoped precision, money with quantize and traps, the 55 digit float door, and fractions through all three construction doors. `randommod.py` adds 15, seeding, the hidden instance, isolation, the selection family with its weight errors, state replay, and the secrets boundary. Every expected value in these files was produced by this same interpreter, then pinned, so a change in any of them is a red, not a rounding dispute.

sources: docs.python.org/3/library/math.html (floor, ceil, trunc, isclose and its formula and special value rules, fsum, copysign, the 53 bit note, the complex boundary), cmath.html (sqrt, polar, rect), statistics.html (the type rule, fmean, mode and multimode tie order, pstdev and stdev, quantiles and both methods, NormalDist, cdf, inv_cdf, samples, harmonic mean), decimal.html (the exact arithmetic claims, significance and money, the default context values, construction exactness and from_float, the rounding after computation faq, quantize and its InvalidOperation, from_number), fractions.html (exact arithmetic, construction, the 3.14 as_integer_ratio rule and from_number, limit_denominator), random.html (the mersenne twister description and period, the hidden instance, seed semantics, the half open range, sample, shuffle, choices weights errors, randbytes, getstate, setstate, and the security warning), the stdtypes integer string conversion length limitation section (the 4300 digit default, the 640 threshold, the affected conversions), and docs.python.org/3/whatsnew/3.14.html (math domain error messages, `Decimal.from_number`, the fractions construction changes), all accessed 2026-09-12. Sample behavior verified by `make verify-py`, 74 checks in chapter 23.

24. numpy

This chapter teaches numpy before anything downstream uses it. The version is the first fact: numpy 2.5.3, pinned in `books/python/requirements.txt`, installed into the gate's venv, and asserted from inside the first sample, because search engines still surface numpy 2.x answers by volume and the behavior this chapter pins changed across that line. book 8, ch 41 (python 3.14, capstone: ingest and analysis pipeline, part 2) builds its analytics leg on the arrays introduced here, and book 8, ch 25 (python 3.14, pandas) builds its frames on the same arrays next chapter. The float semantics underneath are the ones book 8, ch 23 (python 3.14, stdlib numerics: math, statistics, decimal, random) already pinned: binary64, the neighbor double, and `math.isclose`. The stdlib chapter handles scalars one at a time; numpy handles a million of them as one buffer, and every claim below is an ok line in the four samples, 38 in total, or a sentence quoted from the numpy 2.5 manual fetched 2026-09-12.

24.1. the ndarray

An ndarray is one block of memory plus the metadata that walks it. The manual defines the walking triple in three lines: `ndarray.shape`, "Tuple of array dimensions", `ndarray.strides`, "Tuple of bytes to step in each dimension when traversing an array", and `ndarray.itemsize`, "Length of one array element in bytes". A 3 by 4 grid of int32 pins all three at once, strides `(16, 4)`, four bytes per element and sixteen to jump a row, and `asfortranarray` walks the same numbers the other way, `(4, 12)`:

```
def main():
    if sys.version_info[:3] ≠ (3, 14, 7):
        raise SystemExit(f"FAIL pin: interpreter is {sys.version_info[:3]}, not 3.14.7")
    print("ok 1 interpreter pin 3.14.7")
    if np.__version__ ≠ "2.5.3":
        raise SystemExit(f"FAIL pin: numpy is {np.__version__}, not 2.5.3")
    print("ok 2 numpy 2.5.3 exactly")
```

listing 327: the chapter contract: the interpreter pin, then the numpy 2.5.3 pin asserted in-sample (source: python/samples/src/Ch24/ndarray.py, lines 68-74)

```
def structure():
    grid = np.arange(12, dtype=np.int32).reshape(3, 4)
    return (
        grid.shape,
        str(grid.dtype),
        grid.strides,
        grid.itemsize,
        grid.nbytes,
        grid.size,
        grid.ndim,
    )
```

listing 328: shape, dtype, strides, itemsize, nbytes, size, ndim pinned as one tuple (source: python/samples/src/Ch24/ndarray.py, lines 8-18)

```
def fortran_strides():
    return np.asfortranarray(np.arange(12, dtype=np.int32).reshape(3, 4)).strides


def constructors():
    return (
        str(np.zeros((2, 3)).dtype),
        str(np.ones(3, dtype=np.float32).dtype),
        np.arange(1, 10, 3).tolist(),
        np.linspace(0, 1, 5).tolist(),
    )
```

listing 329: fortran order flips the strides, and the four everyday constructors with the dtype each was asked for (source: python/samples/src/Ch24/ndarray.py, lines 21-31)

The constructors are chosen by shape: `zeros` and `ones` take a shape, `arange` takes a step like `range`, and `linspace` takes a count and includes both endpoints, 0.0 through 1.0 in five values. The deeper half of the section is the view and copy split, and the manual states it as a rule, not a tendency: “All arrays generated by basic slicing are always views of the original array”, while “Advanced indexing always returns a copy of the data (contrast with basic slicing that returns a view)”. The sample asserts both halves on one owner:

```
def view_flags():
    own = np.array([[0, 1, 2, 3], [4, 5, 6, 7], [8, 9, 10, 11]], dtype=np.int32)
    views = (
        own[::2].base is own,
        own.reshape(4, 3).base is own,
        own.T.base is own,
        own.ravel().base is own,
        own[:, :2].base is own,
    )
    copies = (
        own.flatten().base is None,
        own[[0, 2]].base is None,
        own[own > 5].base is None,
        own.astype(np.float64).base is None,
        own.copy().base is None,
    )
    shares = (
        np.shares_memory(own, own.T),
        not np.shares_memory(own, own.copy()),
        not np.shares_memory(own, own[[0, 2]]),
    )
    return views, copies, shares
```

listing 330: five index expressions that stay views, five that own their memory, and shares_memory as the judge (source: python/samples/src/Ch24/ndarray.py, lines 34-55)

```
def mutation_sees_owner():
    own = np.array([[0, 1], [2, 3]], dtype=np.int64)
    view = own[:, 0]
    view[0] = 99
    saw_owner = own[0, 0] == 99
    copy = own.copy()
    copy[0, 0] = -5
    return saw_owner, own[0, 0] == 99
```

listing 331: a view write lands in the owner, a copy write never does (source: python/samples/src/Ch24/ndarray.py, lines 58-65)

The base attribute answers who owns the buffer, and the sample uses a literal np.array as its owner on purpose: base reports the memory owner, not the last expression. Slicing an array that is itself a reshape reports the original buffer, not the intermediate, so base is for ownership questions and np.shares_memory for relation questions.

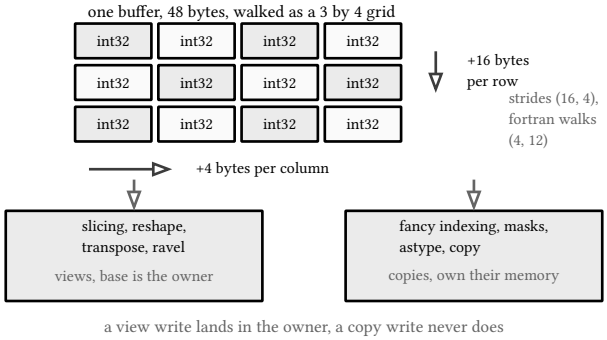


figure 100: the ndarray as memory: one buffer, a shape and strides that walk it, and the two doors out of it

24.2. vectorization and broadcasting

The before and after is one loop. The manual sells broadcasting as exactly that: “Broadcasting provides a means of vectorizing array operations so that looping occurs in C instead of Python.” The sample runs both forms on the same four temperatures and asserts they agree to the last bit, the python loop appending one division at a time, the ufunc expression making one pass over the buffer:

```
def loop_vs_ufunc():
    temps = np.array([270.1, 272.3, 268.9, 271.5])
    loop = []
    for c in temps:
        loop.append(c * 9.0 / 5.0 - 32.0)
    return loop == (temps * 9.0 / 5.0 - 32.0).tolist()
```

listing 332: the loop and the ufunc expression, asserted equal element for element (source: python/samples/src/Ch24/broadcast.py, lines 6-11)

The shape rules are three sentences. “When operating on two arrays, NumPy compares their shapes element-wise. It starts with the trailing (i.e. rightmost) dimension and works its way left.” Then the compatibility test: “Two dimensions are compatible when 1. they are equal, or 2. one of them is 1”, and when neither holds, “a ValueError: operands could not be broadcast together exception is thrown, indicating that the arrays have incompatible shapes”. The failing message quotes both shapes verbatim on this machine, (2,3) against (4,):

```
def broadcast_shapes():
    column = np.arange(4).reshape(4, 1)
    row = np.arange(3)
    grid = column + row
    return (
        grid.shape,
        grid[3, 2] == 5,
        (column + row).tolist()
        == [
            [0, 1, 2],
            [1, 2, 3],
            [2, 3, 4],
            [3, 4, 5],
        ],
    )
```

listing 333: a (4, 1) column plus a (3,) row stretches into the full (4, 3) grid (source: python/samples/src/Ch24/broadcast.py, lines 14-28)

```
def broadcast_errors():
    try:
        np.ones((2, 3)) + np.ones(4)
    except ValueError as exc:
        return (
            str(exc)
            == "operands could not be broadcast together with shapes (2,3) (4,) "
        )
    return False
```

listing 334: the mismatch quotes its shapes in the error text (source: python/samples/src/Ch24/broadcast.py, lines 31-39)

```
def read_only_view():
    stretched = np.broadcast_to(np.arange(3), (4, 3))
    try:
        stretched[0, 0] = 5
    except ValueError as exc:
        return (
            stretched.shape == (4, 3)
            and str(exc) == "assignment destination is read-only"
        )
    return False
```

listing 335: broadcast_to stretches without copying, and the result is read-only (source: python/samples/src/Ch24/broadcast.py, lines 42-51)

Reductions give the axis its meaning. `sum(axis=0)` folds down the columns, `sum(axis=1)` folds along the rows, `axis=None` flattens first, negative indices count from the last axis, and `keepdims` keeps the folded axis at size 1 so a downstream broadcast still lines up. The dtype rules come along for free and are worth pinning: an int32 array sums to int64 and means to float64, a float32 array stays float32 through its sum:

```
    grid = np.arange(12).reshape(3, 4)
    if grid.sum(axis=0).tolist() != [12, 15, 18, 21]:
        raise SystemExit("FAIL axis 0 reduction down the columns")
    print("ok 1 axis 0 reduces down the columns, [12, 15, 18, 21]")
    if grid.sum(axis=1).tolist() != [6, 22, 38]:
        raise SystemExit("FAIL axis 1 reduction along the rows")
    print("ok 2 axis 1 reduces along the rows, [6, 22, 38]")
    if grid.sum(axis=None) != 66 or grid.sum(axis=-1).tolist() != [6, 22, 38]:
        raise SystemExit("FAIL axis none flattens, axis -1 is the last axis")
    print("ok 3 axis None flattens first and axis -1 names the last axis")
    if grid.mean(axis=1, keepdims=True).shape != (3, 1):
        raise SystemExit("FAIL keepdims keeps the reduced axis at size 1")
    print("ok 4 keepdims keeps the reduced axis as a size 1 dimension")
```

listing 336: axis 0, axis 1, axis None, axis −1, and keepdims, each with its result pinned (source: python/samples/src/Ch24/broadcast.py, lines 105-117)

```
def reduction_dtypes():
    ints = np.array([[0, 1, 2, 3], [4, 5, 6, 7]], dtype=np.int32)
    return (
        str(ints.sum().dtype) == "int64",
        str(ints.mean().dtype) == "float64",
        str(np.ones(3, dtype=np.float32).sum().dtype) == "float32",
    )
```

listing 337: int32 sums to int64 and means to float64, float32 keeps its dtype (source: python/samples/src/Ch24/broadcast.py, lines 54-60)

The empty case splits by identity. An empty float array sums to 0.0, the identity for addition, and refuses a max with “zero-size array to reduction operation maximum which has no identity”:

```
def empty_reductions():
    empty = np.array([], dtype=np.float64)
    try:
        empty.max()
    except ValueError as exc:
        message = "zero-size array to reduction operation maximum which has no identity"
        return empty.sum() == 0.0 and str(exc) == message
    return False
```

listing 338: empty sums to the identity, empty max refuses with the identity message (source: python/samples/src/Ch24/broadcast.py, lines 63-70)

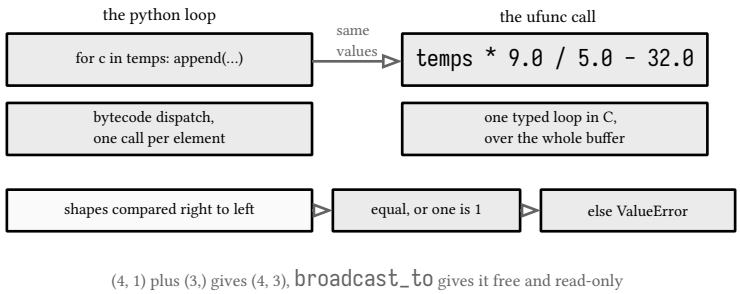


figure 101: before and after: one python loop per element against one ufunc pass, and the three sentence shape rule under it

24.3. fancy indexing and masks

Selection becomes data dependent here, and the indexer’s type decides the door. A boolean mask keeps the elements where it is true, masks combine with `&` and `|` elementwise, and an integer index array may repeat or

reorder indices freely. `np.where` plays two roles: three arguments make it the vector conditional, one argument makes it `nonzero`. The matrix case is worth seeing once, a boolean mask on a 3 by 4 grid returns the flat survivors, and paired index arrays select points, `(2, 1)` and `(0, 3)`:

```
def mask_basics():
    data = np.array([5, -2, 7, 0, -8, 3])
    keep = data > 0
    return (
        keep.tolist() == [True, False, True, False, False, True],
        data[keep].tolist() == [5, 7, 3],
        data[(data > 0) & (data < 6)].tolist() == [5, 3],
        data[data % 2 != 0].tolist() == [5, 7, 3],
    )
```

listing 339: masks select, combine with elementwise and, and invert (source: python/samples/src/Ch24/masks.py, lines 6-14)

```
def fancy_and_where():
    data = np.array([5, -2, 7, 0, -8, 3])
    idx = np.array([0, 0, 2])
    return (
        data[idx].tolist() == [5, 5, 7],
        np.where(data > 0, data, 0).tolist() == [5, 0, 7, 0, 0, 3],
        np.nonzero(data < 0)[0].tolist() == [1, 4],
        np.where(data < 0)[0].tolist() == [1, 4],
        int(np.argmax(data)) == 2 and int(np.argmin(data)) == 4,
        np.clip(data, -1, 1).tolist() == [1, -1, 1, 0, -1, 1],
    )
```

listing 340: integer arrays repeat indices, where picks values, nonzero and argmax locate (source: python/samples/src/Ch24/masks.py, lines 17-27)

```
    matrix = np.arange(12).reshape(3, 4)
    if matrix[matrix % 3 == 0].tolist() != [0, 3, 6, 9]:
        raise SystemExit("FAIL boolean mask on a matrix flattens the selection")
    print("ok 1 a boolean mask on a matrix returns the flat survivors, [0, 3, 6, 9]")
    rows = np.array([2, 0])
    cols = np.array([1, 3])
    if matrix[rows, cols].tolist() != [9, 3]:
        raise SystemExit("FAIL paired integer arrays select pointwise")
    print("ok 2 paired index arrays select points, (2, 1) and (0, 3)")
    if matrix[:, [1, 3]].tolist() != [[1, 3], [5, 7], [9, 11]]:
        raise SystemExit("FAIL column fancy indexing")
    print("ok 3 one fancy axis pairs with one basic slice axis")
```

listing 341: a mask on a matrix flattens, paired arrays select points, one axis may stay basic (source: python/samples/src/Ch24/masks.py, lines 80-91)

The assignment side has the one trap in this chapter. `a[idx] += 1` with a repeated index gathers, adds, and scatters buffered, so the second write to the same slot overwrites the first instead of accumulating: index 0 appears twice in `idx` and the count lands on 1, not 2. `np.add.at` is the unbuffered ufunc method made for exactly this, and it counts 2:

```
def scatter():
    plain = np.zeros(6, dtype=np.int64)
    plain[np.array([0, 0, 2])] += 1
    accumulated = np.zeros(6, dtype=np.int64)
    np.add.at(accumulated, np.array([0, 0, 2]), 1)
    return plain.tolist() == [1, 0, 1, 0, 0, 0], accumulated.tolist() == [
        2,
        0,
        1,
        0,
        0,
        0,
    ]
```

listing 342: buffered fancy scatter loses repeats, add.at accumulates them (source: python/samples/src/Ch24/masks.py, lines 30-42)

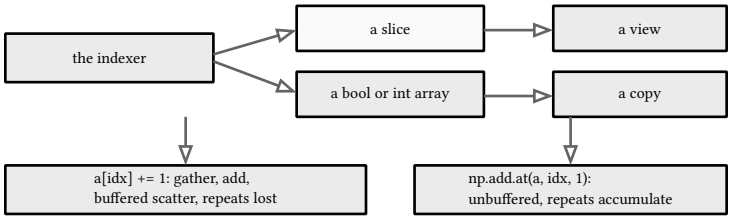
Missing data gets its own ufuncs rather than sentinel tricks: `isfinite` and `isnan` classify, `nansum` and `nanmean` skip the holes, and `clip` pins a range in one call:

```
def nan_aware():
    vals = np.array([1.0, np.nan, 3.0, np.inf])
    return (
        np.isfinite(vals).tolist() == [True, False, True, False],
        bool(np.nansum(vals) == np.inf),
        bool(np.nanmean(np.array([1.0, np.nan, 3.0])) == 2.0),
        np.isnan(vals).tolist() == [False, True, False, False],
    )
```

listing 343: isfinite and isnan classify, nanmean averages around the hole (source: python/samples/src/Ch24/masks.py, lines 45-52)

PITFALL: the buffered scatter loses repeats

`a[idx] += 1` is sugar for gather, add, scatter, and the scatter step writes each slot once per final value, not once per appearance in `idx`. When indices repeat and the operation must accumulate, `np.add.at(a, idx, 1)` is the documented door. The sample pins the divergent outcomes side by side, 1 against 2, so the trap is a red line in the gate rather than a war story.



where(cond, x, y) is the vector conditional, where(cond) is nonzero

nan aware ufuncs skip the holes instead of guarding around them

figure 102: what the indexer contains decides the door, and assignment through a fancy indexer decides the buffer

24.4. dtypes and promotion

Promotion is the numpy 2 line in the sand, specified by NEP 50 and enforced since 2.0: “We propose to remove all value-based logic and add special handling for Python scalars to preserve some convenient behaviors. Python scalars will be considered ‘weakly’ typed.” The principle behind it, “Values must never influence result type”, and the operating rule, “At no time is the value used to decide the result of this promotion. The value is only considered when it is converted to the new dtype; this may raise an error”, are both quotable from the nep. In practice: a python int or float adopts the array’s dtype, so an int32 array plus 1 stays int32 and a float32 array plus 1.5 stays

float32, while a kind change goes to the minimum float, float64. A numpy scalar or a second array is strong: int32 meets int64 and int64 wins, uint8 meets int8 and the signed result is int16, uint64 meets int64 and the result is float64, because no integer dtype holds both:

```
def weak_scalars():
    return (
        str((np.arange(3, dtype=np.int32) + 1).dtype),
        str((np.arange(3, dtype=np.float32) + 1.5).dtype),
        str((np.arange(3, dtype=np.uint8) + 1.5).dtype),
        str((np.arange(3, dtype=np.int32) + 1.5).dtype),
    )
```

listing 344: python scalars are weak, the array dtype survives, kind changes go float64 (source: python/samples/src/Ch24/dtypes.py, lines 6-12)

```
def strong_scalars():
    return (
        str((np.int32(1) + np.int64(2)).dtype),
        str((np.uint8(1) + np.int8(1)).dtype),
        str((np.uint64(1) + np.int64(2)).dtype),
        str((np.bool_(True) + np.int64(1)).dtype),
    )
```

listing 345: numpy scalars are strong, and uint64 with int64 lands in float64 (source: python/samples/src/Ch24/dtypes.py, lines 15-21)

The error side is the point of the design, errors over silent casting. An int8 array plus `256` raises `OverflowError: Python integer 256 out of bounds for int8`. A uint64 array plus `-1` refuses the same way. Writing `2**40` into an int32 slot raises "Python int too large to convert to C long", the windows flavor of the boundary, where a C long is four bytes. In place operations keep their dtype when they can, a float32 array absorbs `+= 1.5`, and refuse with the same_kind casting message when they cannot, an int32 array plus `1.5` in place:

```
def overflow_messages():
    outcomes = []
    try:
        np.arange(3, dtype=np.int8) + 256
    except OverflowError as exc:
        outcomes.append(str(exc) == "Python integer 256 out of bounds for int8")
    try:
        np.arange(3, dtype=np.uint64) + -1
    except OverflowError as exc:
        outcomes.append(str(exc) == "Python integer -1 out of bounds for uint64")
    slot = np.zeros(3, dtype=np.int32)
    try:
        slot[0] = 2**40
    except OverflowError as exc:
        outcomes.append(str(exc) == "Python int too large to convert to C long")
    return tuple(outcomes)
```

listing 346: three overflow refusals with the exact message text, never a silent wrap (source: python/samples/src/Ch24/dtypes.py, lines 24-39)

```
def inplace_rules():
    floats = np.ones(3, dtype=np.float32)
    floats += 1.5
    try:
        ints = np.ones(3, dtype=np.int32)
        ints += 1.5
        return False
    except TypeError as exc:
        return str(floats.dtype) == "float32" and "same_kind" in str(exc)
```

listing 347: in place keeps float32, int32 in place refuses float under same_kind (source: python/samples/src/Ch24/dtypes.py, lines 42-50)

One silent case survives, and it is worth knowing which one. A weak python scalar that fits the dtype is converted with no complaint, so a uint8 array holding 250 plus `10` wraps to 4 modulo 256, only the scalar-on-scalar form warns. The value is bounded by the dtype, not by the operation, which is the NEP 50 bargain: predictable types, and the programmer carries the range check.

Strings come in two dtypes. The historical `<U5` fixes the width at array creation, wasted bytes for short entries, truncation beyond it. `np.dtypes.StringDType()` is the variable width dtype with a real `None` for missing entries, and the `np.strings` module carries the ufuncs, `upper` on either dtype. Adding an int to either is a TypeError, "ufunc 'add' did not contain a loop with signature matching types", because there is no cast to fall back on:

```
def string_dtypes():
    fixed = np.array(["hi", "there"])
    new = np.array(["hi", None], dtype=np.dtypes.StringDType())
    return (
        str(fixed.dtype) == "<U5",
        np.strings.upper(fixed).tolist() == ["HI", "THERE"],
        str(new.dtype) == "StringDType()",
        repr(new[1]) == "'None'",
        np.strings.upper(new).tolist() == ["HI", "NONE"],
    )


def string_plus_int():
    try:
        np.array(["a"]) + 1
    except TypeError as exc:
        return "did not contain a loop" in str(exc)
    return False
```

listing 348: fixed U widths, StringDType and its None, np.strings on both, and the no-loop TypeError (source: python/samples/src/Ch24/dtypes.py, lines 53-70)

Dates are integer counts with units. The manual: "The date units are years ('Y'), months ('M'), weeks ('W'), and days ('D'), while the time units are hours ('h'), minutes ('m'), seconds ('s'), milliseconds ('ms'), and some additional SI-prefix seconds-based units", and for calendars, "NumPy includes a set of 'busday' (business day) functions". A week of days builds with `arange`, subtraction yields a timedelta64, adding a one day timedelta shifts the week, 90 minutes cast to hours truncates to 1, and monday through friday of the book's reference week counts 5:

```python
def datetime_facts():
    week = np.arange("2026-09-07", "2026-09-14", dtype="datetime64[D]")
    span = np.datetime64("2026-09-12") - np.datetime64("2026-09-07")
    return (
        str(week.dtype) == "datetime64[D]",
        week.size == 7 and str(week[0]) == "2026-09-07",
        repr(span) == "np.timedelta64(5,'D')",
        str((week + np.timedelta64(1, "D"))[1]) == "2026-09-09",
        str(np.timedelta64(90, "m").astype("timedelta64[h]")) == "1 hours",
        int(np.busday_count("2026-09-07", "2026-09-12")) == 5,
    )
```

listing 349: datetime64 units, timedelta arithmetic, and the business day count (source: python/samples/src/Ch24/dtypes.py, lines 73-83)

The pandas handoff closes the chapter. book 8, ch 25 (python 3.14, pandas) stores its text in the str dtype and its numbers in these dtypes, and `to_numpy` hands the buffer over, which is why the two libraries compose without copying in the common path.

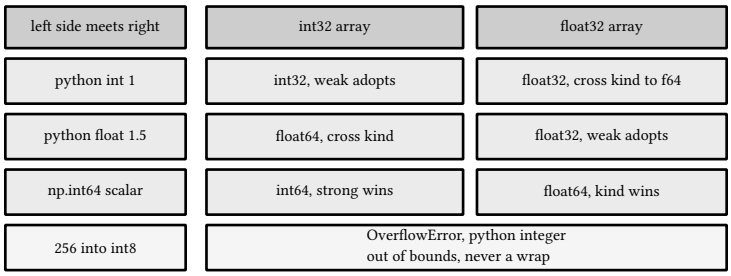


figure 103: promotion as a matrix: weak python scalars adopt, strong numpy operands negotiate, out of range raises

VERIFY: 38 checks, the pin asserted first

A scoped run, `pwsh tools/run-py-samples.ps1 -Chapter Ch24`, walks the four samples through the run and format legs and reports: 4 files, 38 checks, format clean. `ndarray.py` carries the interpreter and numpy pins and contributes 9, the stride triple, both stride orders, the constructors, and the view and copy split with its mutation proof. `broadcast.py` adds 13, the loop equivalence, the (4, 3) broadcast, the quoted mismatch message, the read-only stretch, the axis family, and the reduction dtypes. `masks.py` adds 9, masks and integer arrays, the two roles of `where`, the scatter trap against `add.at`, and the nan aware ufuncs. `dtypes.py` adds 7, the weak and strong columns, three exact overflow messages, the in place rule, both string dtypes, and the datetime facts. Every expected value was produced by this venv, then pinned.

sources: numpy.org/doc/2.5/reference/arrays.ndarray.html (the memory layout attribute definitions, shape, strides, itemsize, base), numpy.org/doc/2.5/user/basics.indexing.html (basic slicing always views, advanced indexing always copies), numpy.org/doc/2.5/user/basics.broadcasting.html (element-wise shape comparison, the two compatibility conditions, the ValueError wording, looping in C), numpy.org/neps/nep-0050-promotion_rules.html (weakly typed python scalars, values never influence result type, the error on conversion), numpy.org/doc/2.5/reference/arrays.datetime.html (date and time unit strings, the busday functions), all accessed 2026-09-12. Overflow message texts, the uint8 wrap to 4, the chained base behavior, and every expected value probed on this machine under the pinned venv the same day. Sample behavior verified by `make verify-py`, 38 checks in chapter 24.

25. pandas

This chapter teaches pandas before book 8, ch 41 (python 3.14, capstone: ingest and analysis pipeline, part 2) builds its report leg on it, and the version matters more here than anywhere else in the book: pandas 3.0.5, a patch of the 3.0 line released 2026-01-21, where copy-on-write became the only behavior and a dedicated string dtype became the default. Search results still surface 2.x answers by volume, and half of them describe a library that silently mutated through views, which is precisely what 3.0 removed. The frames are built on the arrays of book 8, ch 24 (python 3.14, numpy), they carry the same dtypes underneath, and every behavioral claim below is an ok line in the four samples, 33 in total, or a sentence quoted from the pandas 3.0 documentation fetched 2026-09-12.

25.1. series and dataframe

A Series is one column of values plus an Index, and a DataFrame is columns sharing one Index. The index is first class, not row numbering: it travels with the data, slicing by label includes both endpoints, and arithmetic aligns two Series on the union of their indexes, filling the gaps with nan. The sample pins all three, and the alignment check is the one that surprises newcomers, `a + b` on indexes that partially overlap answers at the union with holes where either side is missing:

```
    if sys.version_info[:3] != (3, 14, 7):
        raise SystemExit(f"FAIL pin: interpreter is {sys.version_info[:3]}, not 3.14.7")
    print("ok 1 interpreter pin 3.14.7")
    if pd.__version__ != "3.0.5":
        raise SystemExit(f"FAIL pin: pandas is {pd.__version__}, not 3.0.5")
    print("ok 2 pandas 3.0.5 exactly")
```

listing 350: the chapter contract: the interpreter pin, then the pandas 3.0.5 pin asserted in-sample (source: python/samples/src/Ch25/frames.py, lines 20-25)

```
def alignment():
    left = pd.Series([1, 2], index=["a", "b"])
    right = pd.Series([10, 20], index=["b", "c"])
    summed = left + right
    return (
        np.isnan(summed["a"]) and summed["b"] == 12 and np.isnan(summed["c"]),
        summed.index.tolist() == ["a", "b", "c"],
    )
```

listing 351: arithmetic aligns on the union of the two indexes, gaps become nan (source: python/samples/src/Ch25/frames.py, lines 9-16)

Construction takes dicts for frames, lists plus an index for series, and the dtype inference has one 3.0 rule to know, quoted from the release notes: “Starting with pandas 3.0, a dedicated string data type is enabled by default (backed by PyArrow under the hood, if installed, otherwise falling back to being backed by NumPy object-dtype).” This book does not pin pyarrow, so the fallback storage serves, `str(dtype)` reads “str”, and a missing entry is nan. The two addressing doors are `loc` by label and `iloc` by position, and the endpoint rule is the difference between them:

```
    temps = pd.Series([18.5, 21.0, 19.5], index=["mon", "tue", "wed"], name="temps")
    frame = pd.DataFrame({"k": ["x", "y", "x"], "v": [1, 2, 3]})
    labeled = pd.Series([1, 2, 3], index=[10, 20, 30])
    writeable = pd.DataFrame({"k": ["x", "y", "x"], "v": [1, 2, 3]})
    writeable.loc[1, "v"] = 20
    if writeable["v"].tolist() != [1, 20, 3]:
        raise SystemExit("FAIL loc assignment must write the cell")
    print("ok 3 loc writes a single cell in one statement")
```

listing 352: series and frame construction, and a loc write in one statement (source: python/samples/src/Ch25/frames.py, lines 27-34)

```
    checks = [
        (
            temps.index.tolist() == ["mon", "tue", "wed"] and temps.name == "temps",
            "a series carries its index and its name",
        ),
        (
            str(frame.dtypes.iloc[0]) == "str" and str(frame.dtypes.iloc[1]) == "int64",
            "text columns infer the str dtype, integers infer int64",
        ),
        (
            str(pd.Series(["a", None]).dtype) == "str",
            "the str dtype is the pandas 3.0 default for text",
        ),
```

listing 353: the index and name travel, text infers str, integers infer int64 (source: python/samples/src/Ch25/frames.py, lines 36-48)

```
        (
            frame.loc[1, "k"] == "y" and frame.iloc[1, 0] == "y",
            "loc addresses by label, iloc by position",
        ),
        (
            labeled.loc[10:20].tolist() == [1, 2],
            "label slicing includes both endpoints, unlike positions",
        ),
        (
            labeled.iloc[0:2].tolist() == [1, 2],
            "positional slicing with iloc excludes the endpoint as usual",
        ),
        (
            frame[frame["v"] > 2].shape == (1, 2),
            "a boolean mask keeps whole rows",
```

listing 354: loc by label with both endpoints, iloc by position without, a boolean mask keeps rows (source: python/samples/src/Ch25/frames.py, lines 49-63)

The numpy handoff goes both ways and keeps the buffer: `to_numpy` returns the ndarray underneath, and an ndarray plus column names builds a frame directly:

```
        (
            alignment() == (True, True),
            "arithmetic aligns on the union of indexes, gaps become nan",
        ),
        (
            isinstance(temps.to_numpy(), np.ndarray)
            and pd.DataFrame(np.arange(6).reshape(2, 3)).shape == (2, 3),
            "to_numpy hands over arrays, arrays build frames",
        ),
```

listing 355: alignment on the union, then to_numpy and back (source: python/samples/src/Ch25/frames.py, lines 65-73)

NOTE: label slices close, position slices do not

`s.loc[10:20]` on an index of 10, 20, 30 returns two rows, both endpoints included, because labels identify rows rather than counting them. `s.iloc[0:2]` over the same series returns two rows by the python convention, endpoint excluded. The sample pins both answers on the same index so the contrast is one red line if either changes.

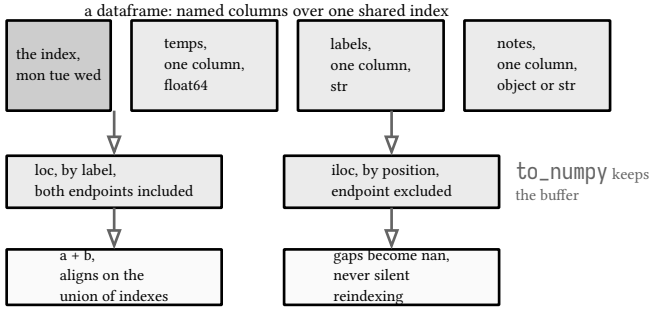


figure 104: a frame as columns over one shared index, the two addressing doors, and the union rule

25.2. io and copy-on-write

The io doors round trip. `to_csv` and `read_csv` come back equal with the text column re-inferred as str, `to_json` and `read_json` with `orient="records"` come back equal the same way, and the sample does both inside a `tempfile.mkdtemp` sandbox, no scratch files anywhere else:

```
def io_round_trips(tmp):
    frame = pd.DataFrame({"k": ["x", "y", "z"], "n": [1, 2, 3], "f": [0.5, 1.5, 2.5]})
    frame.to_csv(tmp / "rows.csv", index=False)
    csv = pd.read_csv(tmp / "rows.csv")
    frame.to_json(tmp / "rows.json", orient="records")
    jsn = pd.read_json(tmp / "rows.json", orient="records")
    return (
        csv.equals(frame)
        and jsn.equals(frame)
        and str(csv.dtypes.iloc[0]) == "str"
        and str(csv.dtypes.iloc[1]) == "int64"
    )
```

listing 356: csv and json round trips, dtypes re-inferred, equality asserted (source: python/samples/src/Ch25/cowio.py, lines 11-22)

The 3.0 behavior is copy-on-write, and the release notes state it as two rules: “The result of *any* indexing operation (subsetting a DataFrame or Series in any way, i.e. including accessing a DataFrame column as a Series) or any method returning a new DataFrame or Series, always *behaves as if* it were a copy in terms of user API”, and as the consequence, “if you want to modify an object (DataFrame or Series), the only way to do this is to directly modify that object itself.” The user guide names the mechanism underneath: “A new lazy copy mechanism that

defers the copy until the object in question is modified and only if this object shares data with another object." What that means in checks: extracting a column and filling it in place changes the extracted Series only, a slice write changes the slice only, and even `copy(deep=False)` no longer aliases:

```
def cow_isolation():
    frame = pd.DataFrame({"a": [1.0, np.nan, 3.0], "b": [4.0, 5.0, 6.0]})
    column = frame["a"]
    column.fillna(0.0, inplace=True)
    column_isolated = frame["a"].isna().tolist() == [False, True, False]
    view = frame.iloc[0:2]
    view.iloc[0, 0] = 99.0
    slice_isolated = frame.iloc[0, 0] == 1.0
    lazy = frame.copy(deep=False)
    lazy.iloc[0, 0] = 77.0
    shallow_isolated = frame.iloc[0, 0] == 1.0
    return column_isolated, slice_isolated, shallow_isolated
```

listing 357: column, slice, and shallow copy writes all stay isolated under copy-on-write (source: python/samples/src/Ch25/cowio.py, lines 25-36)

Chained assignment is the headline removal. The release notes: chained assignment "will stop working. Because this now consistently never works, the `SettingWithCopyWarning` is removed, and defensive `.copy()` calls to silence the warning are no longer needed", and the user guide adds that it will "raise a `ChainedAssignmentError` warning with CoW enabled". The sample asserts the 3.0 semantics directly, the write does not land and the warning fires, and then does the same edit the supported way, one `loc` statement, which the guide states plainly: "`loc` should be used as an alternative."

```
def chained_assignment():
    frame = pd.DataFrame({"a": [1, 2, 3]})
    with warnings.catch_warnings(record=True) as caught:
        warnings.simplefilter("always")
        frame["a"][1] = 12345
        kinds = [w.category.__name__ for w in caught]
    return frame["a"].tolist() == [1, 2, 3] and "ChainedAssignmentError" in kinds


def loc_writes():
    frame = pd.DataFrame({"a": [1, 2, 3]})
    frame.loc[frame["a"] > 1, "a"] = 0
    return frame["a"].tolist() == [1, 0, 0]
```

listing 358: chained assignment never lands and warns, one loc statement lands (source: python/samples/src/Ch25/cowio.py, lines 39-51)

The option that used to toggle all this is itself deprecated: "Setting the option `mode.copy_on_write` no longer has any impact. The option is deprecated and will be removed in pandas 4.0." Reading it still answers True while warning. One keyword did disappear outright, `fillna(method=...)` is gone in 3.0 and the method calls `ffill` and `bfill` replace it:

```
def option_is_deprecated():
    with warnings.catch_warnings(record=True) as caught:
        warnings.simplefilter("always")
        value = pd.options.mode.copy_on_write
    text = str(caught[0].message) if caught else ""
    return value is True and "can no longer be disabled" in text


def removed_method_keyword():
    try:
        pd.Series([1.0, np.nan]).fillna(method="ffill")
    except TypeError as exc:
        return "unexpected keyword argument 'method'" in str(exc)
    return False
```

listing 359: the deprecated option still reads True, and fillna(method=) raises TypeError (source: python/samples/src/Ch25/cowio.py, lines 54-67)

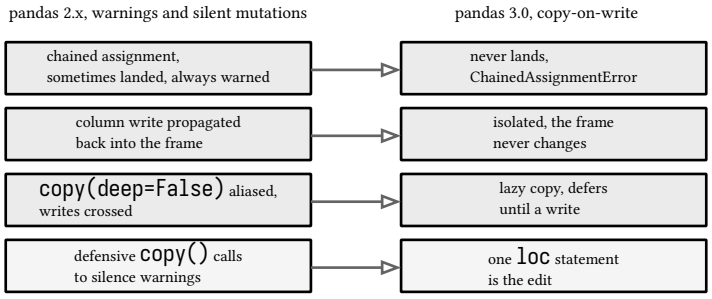


figure 105: before and after: the same four edits in the 2.x world and the 3.0 copy-on-write world

WARNING: old tutorials will hand you 2.x code

The classic 2.x idiom `df["a"][0] = 5` looks executable and does nothing in 3.0 beyond raising a `ChainedAssignmentError` warning, so code copied from an older answer fails silently in the exact way the removal was designed to make loud. The supported forms are one statement each: `df.loc[0, "a"] = 5` for a cell, and `df.loc[mask, "a"] = 5` for a masked set. The sample pins that the chained form leaves `[1, 2, 3]` untouched while the `loc` form writes `[1, 0, 0]`.

25.3. reshape and join

The working set for this section is one small sales frame, regions, kinds, units, and prices, small enough to verify every reshape by hand. `groupby` splits on keys, aggregates each group, and returns the keys sorted, which is default behavior worth knowing because the pinned expectations encode the sorted order, region e before n before s. The named aggregation spells output columns directly, and a list of keys builds a MultiIndex whose tuples are the groups:

```
def sales():
    return pd.DataFrame(
        {
            "region": ["n", "s", "n", "s", "e"],
            "kind": ["a", "a", "b", "b", "a"],
            "units": [10, 20, 30, 40, 50],
            "price": [1.0, 2.0, 3.0, 4.0, 5.0],
        }
    )
```

listing 360: the sales fixture, five rows, three keys (source: python/samples/src/Ch25/reshape.py, lines 6-14)

```python
def grouped():
    frame = sales()
    totals = frame.groupby("region", as_index=False)["units"].sum()
    named = frame.groupby("region").agg(total=("units", "sum"), avg=("price", "mean"))
    multi = frame.groupby(["region", "kind"])["units"].sum()
    return (
        totals.to_dict(orient="records")
        == [
            {"region": "e", "units": 50},
            {"region": "n", "units": 40},
            {"region": "s", "units": 60},
        ],
        named["total"].tolist() == [50, 40, 60]
        and named["avg"].tolist() == [5.0, 2.0, 3.0],
        multi.index.tolist()
        == [("e", "a"), ("n", "a"), ("n", "b"), ("s", "a"), ("s", "b")]
        and multi.tolist() == [50, 10, 30, 20, 40],
    )
```

listing 361: groupby sum with as_index, named agg with total and avg, multi key tuples (source: python/samples/src/Ch25/reshape.py, lines 17-34)

Joins come in two shapes. `merge` is the sql door, `how=` picks left, inner, or outer, the outer form gains an `_merge` column through `indicator=True` reporting left_only, both, or right_only per row, and the output arrives sorted on the key. `join` aligns on the indexes. `validate=` turns a relational assumption into an assertion, duplicate keys under `one_to_one` raise `MergeError` instead of producing a quietly exploded cross product:

```python
def merged():
    left = pd.DataFrame({"k": ["x", "y", "z"], "lv": [1, 2, 3]})
    right = pd.DataFrame({"k": ["y", "z", "w"], "rv": [4, 5, 6]})
    left_merge = pd.merge(left, right, on="k", how="left")["rv"]
    outer = pd.merge(left, right, on="k", how="outer", indicator=True)
    return (
        pd.isna(left_merge.iloc[0]) and left_merge.tolist()[1:] == [4.0, 5.0],
        pd.merge(left, right, on="k", how="inner").shape == (2, 3),
        outer["_merge"].tolist() == ["right_only", "left_only", "both", "both"]
        and outer["k"].tolist() == ["w", "x", "y", "z"],
    )
```

listing 362: left keeps rows with nan gaps, inner intersects, outer reports provenance (source: python/samples/src/Ch25/reshape.py, lines 37-47)

```python
def joins():
    left = pd.DataFrame({"lv": [1, 2]}, index=["x", "y"])
    right = pd.DataFrame({"rv": [4, 5]}, index=["y", "z"])
    joined = left.join(right, how="left")
    duplicates = pd.DataFrame({"k": ["x", "x"], "lv": [1, 2]})
    same = pd.DataFrame({"k": ["x", "x"], "rv": [3, 4]})
    try:
        pd.merge(duplicates, same, on="k", validate="one_to_one")
        raised = False
    except pd.errors.MergeError:
        raised = True
    return joined["lv"].tolist() == [1, 2] and pd.isna(joined["rv"].iloc[0]), raised
```

listing 363: join on the index, and validate refusing duplicate merge keys (source: python/samples/src/Ch25/reshape.py, lines 50-61)

Wide and long are two layouts of one table. `pivot` widens, one column per distinct key value, and refuses duplicate index pairs, `melt` stacks back to long, and `pivot_table` aggregates where `pivot` would refuse. The index algebra underneath is `set_index` promoting a column and `reset_index` demoting it back, `concat` stacks frames with `ignore_index` rebuilding positions, and the counting helpers, `value_counts`, `nlargest`, and `sort_values`, answer the everyday questions:

```
def pivoted():
    wide = sales().pivot(index="region", columns="kind", values="units")
    melted = wide.reset_index().melt(id_vars="region", value_name="units")
    table = pd.pivot_table(
        sales(), index="region", values="units", aggfunc=["sum", "mean"]
    )
    return (
        wide["a"].to_dict() == {"e": 50.0, "n": 10.0, "s": 20.0}
        and pd.isna(wide["b"]["e"])
        and wide["b"].dropna().to_dict() == {"n": 30.0, "s": 40.0},
        melted.columns.tolist() == ["region", "kind", "units"]
        and int(melted["units"].isna().sum()) == 1,
        table.columns.tolist() == [("sum", "units"), ("mean", "units")],
    )
```

listing 364: pivot widens with a hole where no data exists, melt stacks back, pivot_table aggregates (source: python/samples/src/Ch25/reshape.py, lines 64-77)

```
def index_algebra():
    frame = sales()
    indexed = frame.set_index("region")
    reset = indexed.reset_index()
    stacked = pd.concat([frame.iloc[:2], frame.iloc[3:]], ignore_index=True)
    return (
        indexed.index.tolist() == ["n", "s", "n", "s", "e"],
        reset.columns.tolist()[0] == "region",
        stacked.index.tolist() == [0, 1, 2, 3]
        and stacked["units"].tolist() == [10, 20, 40, 50],
        frame["region"].value_counts().to_dict() == {"n": 2, "s": 2, "e": 1}
        and frame.nlargest(2, "units")["units"].tolist() == [50, 40]
        and frame.sort_values("units", ascending=False)["units"].tolist()
        == [50, 40, 30, 20, 10],
    )
```

listing 365: set_index and reset_index, concat with ignore_index, counts and nlargest (source: python/samples/src/Ch25/reshape.py, lines 80-94)

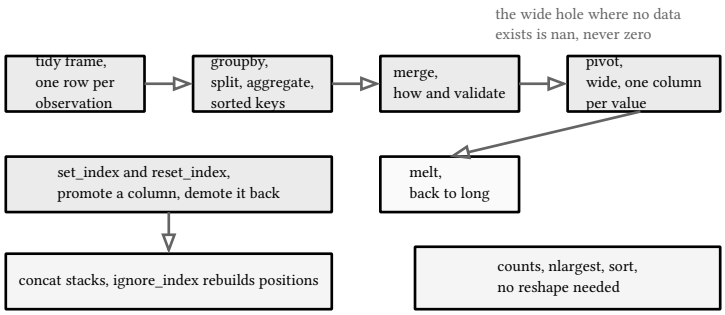


figure 106: the reshape pipeline: tidy frame to grouped totals to merged lookup to wide pivot and back

25.4. time series and missing

A datetime index turns the frame into a time series, and 3.0 stamps it at microsecond resolution by default, `datetime64[us]`, naive unless a tz is attached. Forty-eight hourly stamps build with `date_range`, `resample("D").sum()` folds them into two day buckets summing 276 and 852, and `rolling(3).mean()` slides a three point window that leads with two nan because the first two windows are short:

```
def hourly():
    index = pd.date_range("2026-09-07", periods=48, freq="h")
    return pd.Series(np.arange(48, dtype=float), index=index)
```

listing 366: forty-eight hourly stamps, the working series (source: python/samples/src/Ch25/timeseries.py, lines 7-10)

```
    daily = ts.resample("D").sum()
    rolled = ts.rolling(3).mean()
    checks = [
        (
            str(ts.index.dtype) == "datetime64[us]" and ts.index.tz is None,
            "48 hourly stamps infer datetime64 microseconds, naive by default",
        ),
        (
            daily.index.tolist()
            == pd.to_datetime(["2026-09-07", "2026-09-08"]).tolist()
            and daily.tolist() == [276.0, 852.0],
            "resample D buckets the two days, 276 and 852",
        ),
        (
            int(rolled.iloc[:2].isna().sum()) == 2 and bool(rolled.iloc[2] == 1.0),
            "a 3 point rolling mean leads with two nan, first value 1.0",
        ),
        (
            ts.loc["2026-09-08"].shape == (24,)
            and bool(ts.loc["2026-09-07 02:00:00"] == 2.0),
            "partial string indexing picks a day, exact string picks a stamp",
        ),
```

listing 367: microsecond index, resample D, the rolling lead, partial and exact string indexing (source: python/samples/src/Ch25/timeseries.py, lines 28-49)

String indexing is the ergonomic win, `ts.loc["2026-09-08"]` selects the whole day as a partial match, and a full timestamp string selects one row. `shift` moves values down a slot, `diff` subtracts the previous row, and the index itself answers calendar questions, `dayofweek` is 0 for the monday this series starts on, with `day_name` spelling it out.

```
        (
            bool(ts.shift(1).iloc[1] == 0.0) and bool(ts.diff().iloc[1] == 1.0),
            "shift moves values down, diff subtracts the previous",
        ),
        (
            gap_work() == (True, True, True, True),
            "ffill, bfill, and interpolate, linear and time aware",
        ),
        (
            ts.index[0].dayofweek == 0 and ts.index[0].day_name() == "Monday",
            "the index answers calendar questions, monday is day 0",
        ),
```

listing 368: shift and diff against the previous row, and the calendar accessors (source: python/samples/src/Ch25/timeseries.py, lines 50-61)

Missing data has its own small toolbox. `isna` counts the holes, `fillna` plugs them with a constant, `dropna` removes the rows that carry them, and the three directional fills each mean something different: `ffill` carries the last value forward, `bfill` borrows the next value backward, and `interpolate` walks between the neighbors. The method matters on irregular stamps, linear interpolation by position would put 2.0 in the middle of a 1.0 and 3.0 gap, but `method="time"` weighs the stamps, one day then three days, and lands 1.5:

```
def gap_work():
    gap = pd.Series([1.0, np.nan, 3.0], index=pd.date_range("2026-09-07", periods=3))
    irregular = pd.Series(
        [1.0, np.nan, 3.0],
        index=pd.to_datetime(["2026-09-07", "2026-09-08", "2026-09-11"]),
    )
    return (
        gap.ffill().tolist() == [1.0, 1.0, 3.0],
        gap.bfill().tolist() == [1.0, 3.0, 3.0],
        gap.interpolate().tolist() == [1.0, 2.0, 3.0],
        irregular.interpolate(method="time").tolist() == [1.0, 1.5, 3.0],
    )
```

listing 369: ffill, bfill, and interpolate, linear and time weighted (source: python/samples/src/Ch25/timeseries.py, lines 12-23)

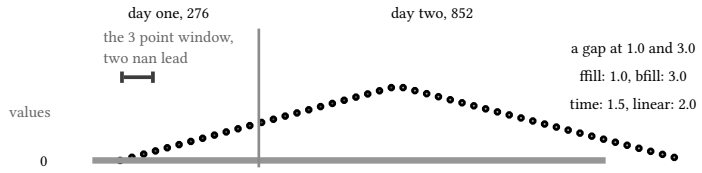
```
    missing = pd.Series([1.0, np.nan, 3.0, np.nan])
    if int(missing.isna().sum()) != 2:
        raise SystemExit("FAIL isna count")
    print("ok 1 isna counts the two holes")
    if missing.fillna(-1.0).tolist() != [1.0, -1.0, 3.0, -1.0]:
        raise SystemExit("FAIL fillna values")
    print("ok 2 fillna plugs holes without touching the rest")
    if len(missing.dropna()) != 2:
        raise SystemExit("FAIL dropna length")
    print("ok 3 dropna drops the rows that carry holes")

    for n, (passed, label) in enumerate(checks, start=4):
        if passed is not True:
            raise SystemExit(f"FAIL check {n}: {label}")
        print(f"ok {n} {label}")
```

listing 370: isna counts, fillna plugs, dropna removes (source: python/samples/src/Ch25/timeseries.py, lines 64-78)

PITFALL: interpolate linear and time disagree on uneven stamps

On an index stamped one day, then three days apart, the gap value sits closer to the first stamp in time. Position-based interpolation answers 2.0, the midpoint of two positions, while `interpolate(method="time")` answers 1.5, the midpoint of the interval. The sample pins both numbers on the same data so the choice is visible as two different ok lines rather than a preference.



resample D folds 48 stamps into 2 buckets, rolling keeps the shape and nan leads

figure 107: forty-eight hourly points folded into two day buckets, a three point window with its two nan lead, and a gap filled three ways

VERIFY: 33 checks, the pin asserted first

> A scoped run, `pwsh tools/run-py-samples.ps1 -Chapter Ch25`, walks the four samples through the run and format legs and reports: 4 files, 33 checks, format clean. `frames.py` carries the interpreter and pandas pins and contributes 12, construction, the str default, both addressing doors with their endpoint rules, the union alignment, and the numpy handoff. `cowio.py` adds 6, both io round trips, the three isolation proofs, the chained assignment refusal with its warning, the deprecated option, and the removed keyword. `reshape.py` adds 5, groupby in its three forms, merge in its three joins with validation, pivot and melt, and the index algebra. `timeseries.py` adds 10, the resample and rolling numbers, string indexing, shift and diff, the four fills with the 1.5 against 2.0 pair, and the calendar accessors. Every expected value was produced by this venv, then pinned.

sources: pandas.pydata.org/docs/whatsnew/v3.0.0.html (the dedicated str dtype default and its pyarrow fallback sentence, the copy-on-write summary rules, the chained assignment stop and SettingWithCopyWarning removal, the mode.copy_on_write deprecation note), pandas.pydata.org/docs/user_guide/copy_on_write.html (the ChainedAssignmentError warning, the lazy copy mechanism, loc as the alternative), both accessed 2026-09-12. The groupby key sort, the outer merge sort order, the microsecond index resolution, the fillna method removal, and every expected value probed on this machine under the pinned venv the same day. Sample behavior verified by `make verify-py`, 33 checks in chapter 25.

26. pydantic

This chapter teaches pydantic before book 8, ch 22 (python 3.14, fastapi) and book 8, ch 40 (python 3.14, capstone: ingest and analysis pipeline, part 1) build on it, because in both places it plays the same role: the boundary where untrusted dictionaries become typed objects or loud failures. The version is 2.13.4, pinned in the book's requirements file and asserted from inside the first sample, the second major line of the library, the one with the rust core under the `pydantic-core` package. The stdlib shapes of book 8, ch 9 (python 3.14, dataclasses, enums, and pattern matching) and book 8, ch 8 (python 3.14, the typing system) are the raw material, annotations on a class, and pydantic turns those annotations into a validator and a serializer. Every behavioral claim below is an ok line in the four samples, 25 in total, or a sentence quoted from the pydantic documentation fetched 2026-09-12.

26.1. basemodel and validation

The manual defines the object in one sentence: "Models are simply classes which inherit from BaseModel and define fields as annotated attributes", and the behavior in two more: "Initialization of the object will perform all parsing and validation", and "Pydantic will raise a ValidationError exception whenever it finds an error in the data it's validating." The error is collected, not thrown at the first offense: "A single exception will be raised regardless of the number of errors found", and that exception "will contain information about all of the errors and how they happened". The sample feeds one bad id and one bad email in the same constructor and gets both back, with the error surface pinned key for key:

```
def main():
    if sys.version_info[:3] != (3, 14, 7):
        raise SystemExit(f"FAIL pin: interpreter is {sys.version_info[:3]}, not 3.14.7")
    print("ok 1 interpreter pin 3.14.7")
    if pydantic.__version__ != "2.13.4":
        raise SystemExit(f"FAIL pin: pydantic is {pydantic.__version__}, not 2.13.4")
    print("ok 2 pydantic 2.13.4 exactly")
```

listing 371: the chapter contract: the interpreter pin, then the pydantic 2.13.4 pin asserted in-sample (source: python/samples/src/Ch26/models.py, lines 77-83)

```
class User(BaseModel):
    id: int
    name: str
    age: int = Field(default=1, ge=0, le=150)
    email: Optional[str] = None


class Strict(BaseModel):
    model_config = ConfigDict(strict=True)

    n: int


class Locked(BaseModel):
    model_config = ConfigDict(extra="forbid")

    n: int
```

listing 372: three models: plain fields, Field constraints, strict mode, extra=forbid (source: python/samples/src/Ch26/models.py, lines 10-26)

```
def error_surface():
    try:
        User(id="notanint", name="x", age=5, email=123)
    except ValidationError as exc:
        errors = exc.errors()
        return (
            exc.title == "User" and exc.error_count() == 2,
            [e["type"] for e in errors] == ["int_parsing", "string_type"],
            [e["loc"] for e in errors] == [("id",), ("email",)],
            sorted(errors[0]) == ["input", "loc", "msg", "type", "url"],
            "errors.pydantic.dev" in errors[0]["url"],
        )
    return (False, False, False, False, False)
```

listing 373: two errors collected at once, each with type, loc, msg, input, and a docs url (source: python/samples/src/Ch26/models.py, lines 29-41)

Each error row is a dict with five keys on this version, `input`, `loc`, `msg`, `type`, and `url`, where `loc` paths into the model and `url` points at the error's page under errors.pydantic.dev. Some types add a sixth `ctx` key; `int_parsing` does not. The exception itself carries `title`, the class name, and `error_count()`. Lax mode is the default and coerces on the way in, the string "42" becomes the int 42, while `ConfigDict(strict=True)` flips the same input to an `int_type` error. Constraints live in `Field`: `ge` and `le` bound a number, an absent required field reports type `missing`, and `extra="forbid"` turns unknown keys into `extra_forbidden` errors. Dict entry points run the same machinery, `model_validate` "Validates the given object against the Pydantic model" and `model_validate_json` "Validates the given JSON data against the Pydantic model":

```
def missing_field():
    try:
        User(name="x")
    except ValidationError as exc:
        first = exc.errors()[0]
        return first["type"] == "missing" and first["loc"] == ("id",)
    return False


def constraint_blocked():
    try:
        User(id=1, name="x", age=200)
    except ValidationError as exc:
        return exc.errors()[0]["type"] == "less_than_equal"
    return False
```

listing 374: a missing field reports missing, a bounds violation reports less_than_equal (source: python/samples/src/Ch26/models.py, lines 44-58)

```
def strict_rejects():
    try:
        Strict(n="5")
    except ValidationError as exc:
        return exc.errors()[0]["type"] == "int_type"
    return False


def forbids_extra():
    try:
        Locked(n=1, zzz=2)
    except ValidationError as exc:
        return exc.errors()[0]["type"] == "extra_forbidden"
    return False
```

listing 375: strict rejects the string 5, extra keys are forbidden (source: python/samples/src/Ch26/models.py, lines 61-74)

```
    coerced = User(id="42", name="ada")
    checks = [
        (
            error_surface() == (True, True, True, True, True),
            "one ValidationError carries every field's error with loc and type",
        ),
        (missing_field(), "an absent required field reports type missing"),
        (
            coerced.id == 42 and isinstance(coerced.id, int) and coerced.email is None,
            "lax mode coerces the string 42 to the int 42",
        ),
        (
            constraint_blocked(),
            "Field(ge=0, le=150) blocks 200 with less_than_equal",
        ),
        (strict_rejects(), "strict mode rejects the string 5 with int_type"),
        (
            forbids_extra(),
            "extra=forbid rejects unknown keys with extra_forbidden",
        ),
        (
            User.model_validate({"id": 1, "name": "x", "age": "7"}).age == 7,
            "model_validate runs the same validation on a dict",
        ),
```

listing 376: coercion, constraint, strict, forbid, and model_validate, as the run sees them (source: python/samples/src/Ch26/models.py, lines 85-108)

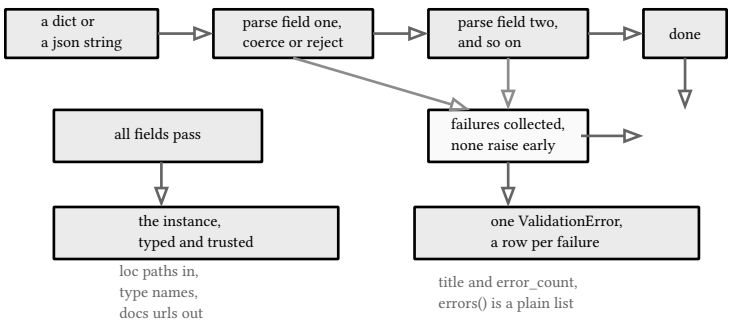


figure 108: validation as a state machine: parse each field, collect every failure, hand back one instance or one error with all rows

26.2. serializers and json

The dump side mirrors the validate side. The manual: “Pydantic allows models (and any other type using type adapters) to be serialized in two modes: Python and JSON”, and the difference is that “The Python output may contain non-JSON serializable data”, a datetime stays a datetime, while `mode="json"` and `model_dump_json()` produce json-safe values, the datetime becoming its iso string. The knobs are part of the daily vocabulary: `exclude_none` drops the None fields, `Field(exclude=True)` removes a field from every dump, and `TypeAdapter` extends the same machinery to types that are not models, `TypeAdapter(list[int]).validate_python(["1", "2"])` answering `[1, 2]`:

```
def event_dumps():
    event = Event(when=dt.datetime(2026, 9, 12, 8, 30), secret="s3cret")
    return (
        event.model_dump(exclude_none=True)
        == {"when": dt.datetime(2026, 9, 12, 8, 30)},
        isinstance(event.model_dump(mode="json")["when"], str),
        event.model_dump_json() == '{"when":"2026-09-12T08:30:00","note":null}',
        "secret" not in event.model_dump(),
    )
```

listing 377: python mode keeps the datetime, json mode strings it, the excluded field never appears (source: python/samples/src/Ch26/serial.py, lines 50-58)

```
def round_trips():
    event = Event(when=dt.datetime(2026, 9, 12, 8, 30))
    back = Event.model_validate_json(event.model_dump_json())
    return (
        back == event,
        TypeAdapter(list[int]).validate_python(["1", "2"]) == [1, 2],
        TypeAdapter(dt.datetime).validate_json('"2026-09-12T08:30:00"')
        == dt.datetime(2026, 9, 12, 8, 30),
    )
```

listing 378: model and TypeAdapter json round trips land exactly (source: python/samples/src/Ch26/serial.py, lines 61-69)

Custom checks hang off annotations. The modern form composes: `Annotated[str, AfterValidator(strip_lower)]` is a type with a function attached, reusable anywhere a type goes. The model-scoped forms are `field_validator`, which sees one field plus the already validated ones through `info.data`, and `model_validator(mode="after")`, which sees the whole instance and is the only place a cross field rule can be checked honestly:

```
class Event(BaseModel):
    when: dt.datetime
    note: str | None = None
    secret: str = Field(default="", exclude=True)


def strip_lower(value: str) -> str:
    return value.strip().lower()


class Tagged(BaseModel):
    tag: Annotated[str, AfterValidator(strip_lower)]
    count: Annotated[int, Field(gt=0)] = 1
```

listing 379: a model with an excluded field, and an annotated validator as a reusable type (source: python/samples/src/Ch26/serial.py, lines 17-30)

```
class Pair(BaseModel):
    lo: int
    hi: int

    @field_validator("hi")
    @classmethod
    def hi_nonnegative(cls, value):
        if value < 0:
            raise ValueError("hi must be ≥ 0")
        return value

    @model_validator(mode="after")
    def ordered(self):
        if self.lo > self.hi:
            raise ValueError("lo must be ≤ hi")
        return self
```

listing 380: field_validator for one field, model_validator after for the pair (source: python/samples/src/Ch26/serial.py, lines 32-47)

The schema is a free byproduct: `model_json_schema()` emits plain json schema, `gt` surfacing as `exclusiveMinimum`, `ge` and `le` as `minimum` and `maximum`, and a plain int field spelling nothing more than type integer with a title. Two escape hatches complete the picture: `model_copy(update=...)` rebinds fields without revalidating, and `model_construct` builds an instance with no validation at all, useful for trusted fast paths and dangerous everywhere else:

```
def schema_bounds():
    count = Tagged.model_json_schema()["properties"]["count"]
    lo = Pair.model_json_schema()["properties"]["lo"]
    return (
        count["exclusiveMinimum"] == 0 and count["default"] == 1,
        lo == {"title": "Lo", "type": "integer"},
    )


def copy_and_construct():
    base = Tagged(tag="a")
    updated = base.model_copy(update={"tag": "b"})
    raw = Tagged.model_construct(tag="  NOT CLEANED  ")
    return updated.tag == "b" and raw.tag == "  NOT CLEANED  "
```

listing 381: schema bounds as json schema keys, model_copy updating, model_construct skipping validation (source: python/samples/src/Ch26/serial.py, lines 85-99)

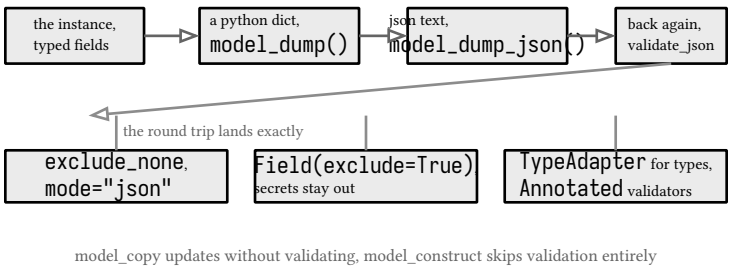


figure 109: the serializer pipeline: instance to python dict to json text and back, with the knobs on the side

26.3. composed models

Fields annotated with other models compose by recursion, and the error locs show the path: an Outer holding one Inner and a list of them reports failures at `('inner', 'x')` and `('items', 1, 'x')`, the list index sitting in the path

between the field and the inner field. Defaults stay per instance, `Field(default_factory=list)` builds a fresh list each time, and `model_fields_set` records exactly which fields the input provided, empty for an all defaults construction:

```
class Inner(BaseModel):
    x: int


class Outer(BaseModel):
    inner: Inner
    items: list[Inner] = []
```

listing 382: a model nested in a model, and a list of them (source: python/samples/src/Ch26/composed.py, lines 8-15)

```
def nested_error_locs():
    try:
        Outer.model_validate({"inner": {"x": "bad"}, "items": [{"x": 2}, {"x": "no"}]})
    except ValidationError as exc:
        locs = [e["loc"] for e in exc.errors()]
        return (
            locs == [("inner", "x"), ("items", 1, "x")],
            [e["type"] for e in exc.errors()] == ["int_parsing", "int_parsing"],
        )
    return (False, False)
```

listing 383: error locs path into the tree, the list index in the middle (source: python/samples/src/Ch26/composed.py, lines 55-65)

```
def defaults_and_fields_set():
    plan = Plan()
    return (
        plan.model_dump() == {"tags": [], "note": None},
        plan.model_fields_set == set(),
        Plan(tags=["a"]).model_fields_set == {"tags"},
    )
```

listing 384: default_factory builds fresh lists, model_fields_set records what was set (source: python/samples/src/Ch26/composed.py, lines 87-94)

Unions of models have two dispatch strategies. The plain union tries members left to right, first match wins, which works until two members share a shape. The discriminated union is the routing answer, and the manual introduces it under the name it also carries: “sometimes referred to as ‘Tagged unions’”, where the union is “validated more efficiently using a discriminator, by specifically choosing which member of the union”, which “makes validation more efficient and also avoids a proliferation of errors when validation fails”. The tag is a Literal field common to every member, `Field(discriminator="kind")` names it, and the failures get their own error types, `union_tag_not_found` when the tag is absent and `union_tag_invalid` when it names no member:

```
class Cat(BaseModel):
    kind: Literal["cat"] = "cat"
    meows: int = 0


class Dog(BaseModel):
    kind: Literal["dog"] = "dog"
    barks: int = 0


class Pet(BaseModel):
    animal: Annotated[Union[Cat, Dog], Field(discriminator="kind")]


class Plain(BaseModel):
    animal: Union[Cat, Dog]


class Plan(BaseModel):
    tags: list[str] = Field(default_factory=list)
    note: Optional[str] = None
```

listing 385: cat and dog share the literal tag kind, the discriminator routes, the plain union does not (source: python/samples/src/Ch26/composed.py, lines 17-38)

```
def discriminated():
    pet = Pet.model_validate({"animal": {"kind": "cat", "meows": 2}})
    outcomes = [type(pet.animal).__name__ == "Cat" and pet.animal.meows == 2]
    try:
        Pet.model_validate({"animal": {"kind": "fish"}})
    except ValidationError as exc:
        outcomes.append(exc.errors()[0]["type"] == "union_tag_invalid")
    try:
        Pet.model_validate({"animal": {"meows": 1}})
    except ValidationError as exc:
        outcomes.append(exc.errors()[0]["type"] == "union_tag_not_found")
    return tuple(outcomes)


def plain_union():
    parsed = Plain.model_validate({"animal": {"kind": "dog", "barks": 1}})
    left = Plain.model_validate({"animal": {"meows": 3}})
    return type(parsed.animal).__name__ == "Dog" and type(left.animal).__name__ == "Cat"
```

listing 386: routing by tag, union_tag_invalid for fish, union_tag_not_found for a missing tag (source: python/samples/src/Ch26/composed.py, lines 67-84)

Computed state can join the dump. A property alone is invisible to serialization, decorating it with `computed_field` puts it in `model_dump()` output, the rectangle's area riding along with its width and height:

```python
class Screen(BaseModel):
    w: float
    h: float

    @computed_field
    @property
    def area(self) -> float:
        return self.w * self.h
```

listing 387: a computed_field lands in the dump, a plain property does not (source: python/samples/src/Ch26/composed.py, lines 40-47)

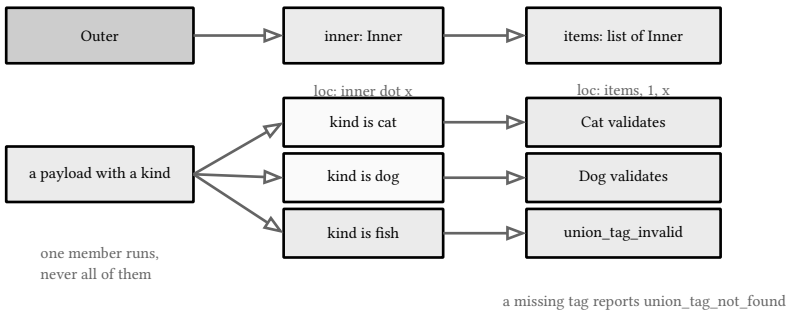


figure 110: the composed tree with its error paths, and the discriminator routing one payload to exactly one member

26.4. boundaries and pins

Three boundaries close the chapter, each stated rather than sampled past its edge. The first is the v1 history. Pydantic v1 was the 2018 to 2022 line, single implementation in python, and v2 arrived in mid 2023 rewritten over a rust core with the `model_` method names this chapter uses. The 2.13 package still ships a frozen copy of v1 as the `pydantic.v1` subpackage, reached by importing it, `from pydantic import v1`, not by attribute access, and a v1 BaseModel defined from it still coerces the way v1 did, the string "7" becomes 7 in both lines. Migration errors point at errors.pydantic.dev/2.13/migration, the same host the validation error urls use:

```python
def v1_namespace():
    modern = pydantic.BaseModel

    class V1Point(legacy.BaseModel):
        x: int

    return (
        legacy.BaseModel is not modern,
        V1Point(x="7").x == 7,
        V2Point(x="7").x == 7,
    )
```

listing 388: the v1 namespace imports as a subpackage and still coerces like v1 (source: python/samples/src/Ch26/bounds.py, lines 19-30)

The second boundary is settings. `pydantic-settings` is a separate package, env var and secrets loading over a BaseModel, and it is not in this book's pins, so the chapter states it and stops. The book 8, ch 19 (python 3.14, operational plumbing) chapter covers environment handling with the stdlib, which is the pattern the capstone actually uses.

```
def settings_absent():
    try:
        import pydantic_settings  # noqa: F401
    except ImportError:
        return True
    return False


def dataclass_trade_off():
    dc = DcPoint(x="42")
    vd = V2Point(x="42")
    return dc.x == "42" and isinstance(dc.x, str) and vd.x == 42
```

listing 389: settings absent by pin, and the dataclass storing what the model would reject (source: python/samples/src/Ch26/bounds.py, lines 32-44)

The third boundary is the dataclass trade-off. A dataclass with `x: int` stores whatever arrives, the string "42" stays a string, while a BaseModel validates and coerces on construction. book 8, ch 9 (python 3.14, dataclasses, enums, and pattern matching) is the right tool for trusted internal records where the annotation is documentation, pydantic is the right tool at boundaries where the data has not been earned. The schema is the tiebreaker that usually decides it: json schema for free, or none:

```
def main():
    checks = [
        (
            v1_namespace() == (True, True, True),
            "pydantic.v1 is a separate frozen line that still coerces like v1",
        ),
        (
            settings_absent(),
            "pydantic_settings is not in this book's pins, a stated boundary",
        ),
        (
            dataclass_trade_off(),
            "a dataclass stores the string, a model validates the int",
        ),
        (
            sorted(V2Point.model_json_schema())
            == ["properties", "required", "title", "type"],
            "the schema is json schema, type, properties, required, title",
        ),
    ]
```

listing 390: the schema is type, properties, required, title, and the checks as the run sees them (source: python/samples/src/Ch26/bounds.py, lines 46-65)

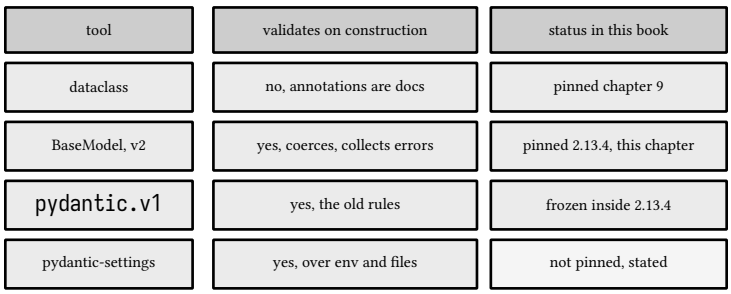


figure 111: the tool matrix: dataclasses, pydantic v2, the v1 namespace, and pydantic-settings against what validates and what this book pins

VERIFY: 25 checks, the pin asserted first

A scoped run, `pwsh tools/run-py-samples.ps1 -Chapter Ch26`, walks the four samples through the run and format legs and reports: 4 files, 25 checks, format clean. `models.py` carries the interpreter and pydantic pins and contributes 9, the collected error surface with its keys and urls, missing and bounds error types, strict and forbid modes, and `model_validate`. `serial.py` adds 6, both dump modes with the exact json string, round trips through the adapter, the annotated and scoped validators, schema bounds, and the two escape hatches. `composed.py` adds 6, nested locs, discriminator routing with both tag error types, the plain union order, defaults with `model_fields_set`, and the computed field in the dump. `bounds.py` adds 4, the v1 namespace, the settings absence, the dataclass contrast, and the schema shape. Every expected value was produced by this venv, then pinned.

sources: pydantic.dev/docs/validation/latest/concepts/models/ (the BaseModel definition, initialization performing parsing and validation, the single collected ValidationError, model_validate and model_validate_json), pydantic.dev/docs/validation/latest/concepts/unions/ (tagged unions, the discriminator choosing one member, the efficiency and error claims), pydantic.dev/docs/validation/latest/concepts/serialization/ (the two serialization modes, python output holding non-json data), all accessed 2026-09-12. The error row keys, the v1 import path and its coercion, the pydantic-settings absence, and every expected value probed on this machine under the pinned venv the same day. Sample behavior verified by `make verify-py`, 25 checks in chapter 26.

27. http kernel

Everything else in this service part sits on one small module: the http kernel. Python's standard library ships no router, no error envelope, no middleware, and that lack is why this part exists: `http.server` parses one request off a socket and calls a method named after the verb, everything above that is ours to build in the open. This chapter builds `pyapi/kernel.py`: value shaped requests and responses, the hand rolled route table as the spine, the contract envelope with exactly one success writer and one failure writer, the json codec configured so the frozen vector bytes match, the request id on its contextvar, and a socket adapter thin enough that everything above it tests as function calls.

27.1. the handler shape on http.server

The platform's server is a composition of small classes, one job each: `socketserver.BaseServer` drives the serve loop, `TCPServer` owns the listening socket and the accept, `ThreadingHTTPServer` is `socketserver.ThreadingMixIn` layered over `HTTPServer` so one handler instance per request runs on its own daemon thread, and the handler parses the request line and headers into an `email.message.Message`. That last fact is worth pinning: the platform's own header parsing is the email module, and this chapter reuses the same machinery for media types instead of hand parsing.

`BaseHTTPRequestHandler` dispatches by name, it looks up `do_` plus the verb, and a missing method answers a plain text 501 page. Both defaults break the contract, which puts every non-2xx in the envelope. The adapter owns them by shape rather than enumeration: `__getattr__` returns the one `_handle` method for any `do_` name, so TRACE reaches the router and gets the 405 envelope, and a test pins that against a live socket. Above the adapter sits the kernel's own contract: `dispatch` takes a `Request` of plain values and returns a `Response` of plain values, handlers raise `APIError` instead of writing a failure, and the adapter is the only code that touches `rfile` or `wfile`:

```
    def _handle(self):
        split = urllib.parse.urlsplit(self.path)
        raw_length = self.headers.get("Content-Length") or "0"
        try:
            length = int(raw_length)
        except ValueError:
            length = -1
        if length < 0:
            self.close_connection = True
            self.send_response(400)
            self.send_header("Content-Length", "0")
            self.send_header("Connection", "close")
            self.end_headers()
            return
        body = self.rfile.read(min(length, MAX_BODY_BYTES + 1)) if length else b""
        request = Request(
            method=self.command,
            path=urllib.parse.unquote(split.path),
            query=dict(urllib.parse.parse_qsl(split.query, keep_blank_values=True)),
            headers={name.lower(): value for name, value in self.headers.items()},
            body=body,
            client=self.client_address,
        )
        if length > MAX_BODY_BYTES or "content-length" not in self.headers:
            self.close_connection = True
        response = self.server.app.dispatch(request)
        self.send_response(response.status)
        for name, value in response.headers.items():
            self.send_header(name, value)
        bodyless = response.status in (204, 304)
        if response.body and not bodyless:
            self.send_header("Content-Length", str(len(response.body)))
        if self.close_connection:
            self.send_header("Connection", "close")
        self.end_headers()
        if response.body and not bodyless:
            self.wfile.write(response.body)
```

listing 391: the adapter reads the wire into values and writes the response back (source: python/api/pyapi/kernel.py, lines 381-417)

The adapter caps its body read at one megabyte plus one byte, so a lying content length never buffers past the cap before the decode ladder rejects the body. The wire's own garbage gets its own answers: a content length that does not parse, or parses negative, is a 400 with the connection closed rather than a blocked read or a thread parked on `read(-1)`, the socket carries a five second read timeout so an idle connection cannot hold its handler forever, and any request that leaves input unread, an oversize body or a request without a content length at all, closes the connection after its answer, because a keep-alive socket with bytes still in it desyncs the next request line. Responses carry `Content-Length` on every body, which keeps the `HTTP/1.1` keep alive framing honest, and 204 with 304 send neither body nor length because the protocol forbids both. The platform's `log_message` default writes plain lines to stderr, and the adapter silences it: request logging belongs to the access log layer in the next chapter.

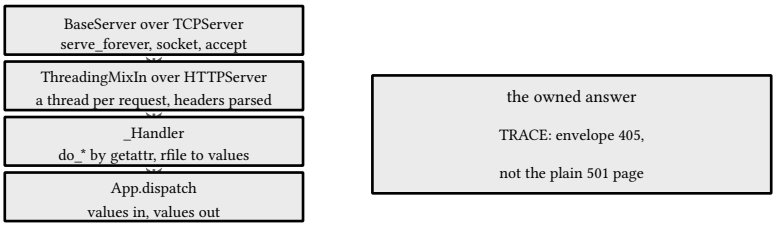


figure 112: the platform stack, and the adapter owns the answers the platform would get wrong

27.2. the route table, hand rolled

Go's standard mux grew method patterns and precedence in 1.22, and python's `http.server` never had a router at all, so the table is hand rolled and its decomposition is the chapter's spine. A pattern is a path with whole segment parameters, `/api/users/{id}`. Compile splits it into a tuple of segments, each a literal or a parameter name, and rejects a brace anywhere else in a segment, so parameters are segments or nothing. Matching walks the compiled tuple against the path's segments: lengths agree, a literal equals its part, and a parameter takes exactly one non empty segment, never an empty one, which is why `/api/users/` with its trailing slash is a 404 rather than a user lookup with a blank id.

Precedence is one rule: when two same method patterns fit one path, the one with more literal segments wins. Two patterns that tie, equal shape with equal literal counts, can never disambiguate a request, so `route` raises `ValueError` at registration and an ambiguous table never ships, the ruling the go lane gets from its mux panicking at registration. The 404 and 405 split falls out of the same lookup: a path no pattern fits is `not_found` at 404, a path other methods fit is `not_found` at 405 plus `Allow`, because the code set is closed at eleven and the message plus `Allow` carry the distinction:

```
    def _match(self, method, path):
        """the table lookup: all patterns that fit the path, the most
        specific same-method one wins, a path only reachable under
        other methods yields the allow set, nothing yields none."""
        parts = segments_of(path)
        fitted = []
        for entry in self.routes:
            values = _match_segments(entry.compiled, parts)
            if values is not None:
                fitted.append((entry, values))
        mine = [(e, v) for e, v in fitted if e.method == method]
        if mine:
            best = max(
                mine, key=lambda ev: sum(seg is not None for seg, _ in ev[0].compiled)
            )
            return best
        if fitted:
            allow = ", ".join(sorted({e.method for e, _ in fitted}))
            return allow
        return None
```

listing 392: the lookup: most specific wins, other methods yield the allow set (source: python/api/pyapi/kernel.py, lines 295-314)

The table has one property worth more than any example, matching never depends on registration order. The suite pins it with a seeded `random.Random(2026)` loop, two hundred shuffles of one five pattern table, asserting the same handler answers the same paths every time, and the tie check is what makes the property hold.

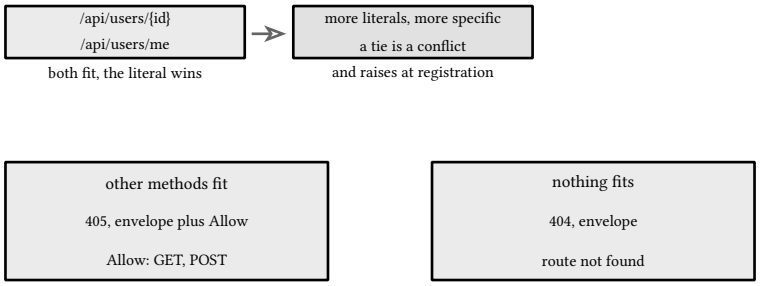


figure 113: literals beat parameters, ties raise at registration, leftovers split into 405 and 404

27.3. the error envelope

The contract freezes one failure shape for the whole api: an error object carrying a code from a closed set of eleven, one human sentence, an optional `details` array on `validation_failed` only, and the request id. Every non-2xx rides it, including the 404 and 405 the lookup owns, the 422 the user chapter adds, the 429 and 503 the rate and load chapters add. The codes are declared once as module constants and `CODE_STATUS` is the single table mapping each to the status it carries everywhere, so the day a status needs to move the code does not.

Two codes are split across statuses and each split site passes its status explicitly: `invalid_json` rides 400 on a malformed body and 415 on a wrong media type, `not_found` rides 404 and 405. `at` is the one sanctioned override, it returns the same error carrying the explicit status. `fail` also enforces the details rule at construction, it raises `ValueError` when details ride any code but `validation_failed`, so the one exception to their absence cannot happen by accident. One failure writer builds the envelope, and the dict literal orders `code`, `message`, then `details` when present, then `request_id`, the member order the go lane's struct fields freeze:

```
def failure(request, error):
    """the one failure writer: the contract envelope, nested two
    levels, details between message and request_id the way the go
    struct order freezes them."""
    body = {"code": error.code, "message": error.message}
    if error.details is not None:
        body["details"] = error.details
    body["request_id"] = resolve_request_id(request.headers)
    head = {"Content-Type": "application/json"}
    head.update(error.headers)
    return Response(error.status, head, _dumps({"error": body}))
```

listing 393: the one failure writer: nested two levels, details between message and request_id (source: python/api/pyapi/kernel.py, lines 155-165)

Handlers raise `APIError` rather than writing a failure, and `dispatch` catches it at the innermost layer, which is where go had two channels python has one: a returned error and a panic are both exceptions here, so the mapping is by class. `APIError` is the handled failure with contract shape, any other exception is a bug, and converting bugs into envelopes is the recover layer's job in the next chapter. The kernel tests pin the boundary: an `APIError` becomes the envelope, a `RuntimeError` propagates untouched.

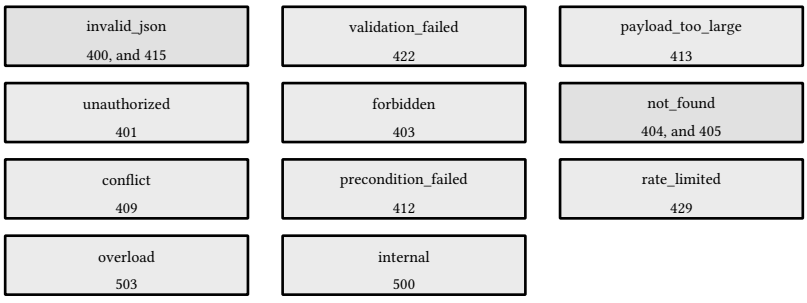


figure 114: eleven codes, eleven statuses in the table, thirteen on the wire

27.4. json in and out

Bodies go out through one success writer whose three settings are the whole byte contract. `json.dumps` with `separators` of comma and colon emits no spaces, `ensure_ascii=False` lets non ascii text ride as utf8 instead of `\\u` escapes, and `.encode()` fixes the wire as bytes. Dict literal order is insertion order, so the literal mirrors the go struct field order the vectors were cut from, and the vectors are byte authoritative: this chapter's tests pin the healthz body and a non ascii payload as exact bytes, so any drift in writer settings is a red test here.

Reading is stricter than writing, because request bodies are input. `read_json` is the one decode path, three rules in a fixed order. The media type comes from `email.message.Message` with the header value installed on it, the same parser the platform's own header reading uses, and `get_content_type` strips parameters, so `application/json; charset=utf-8` passes while anything else, including a missing header which falls to the parser's text/plain default, is 415 `invalid_json`. The body is then checked against the one megabyte cap, and oversize is always `payload_too_large` at 413, never `invalid_json`, the binding ruling. What survives goes to `json.loads`, which accepts bytes and detects the encoding itself, and a malformed body is 400 `invalid_json`. Field validation is the user chapter's concern, this ladder only decides whether the body is json at all:

```
def read_json(request):
    """the decode ladder every write route inherits: the media type
    must be application/json (415), the body capped at 1 MiB (413,
    always payload_too_large), the json must parse (400)."""
    if media_type(request.headers.get("content-type")) ≠ "application/json":
        raise fail(INVALID_JSON, "content type must be application/json").at(415)
    if len(request.body) > MAX_BODY_BYTES:
        raise fail(PAYLOAD_TOO_LARGE, "request body exceeds 1 MiB")
    try:
        return json.loads(request.body)
    except ValueError, UnicodeDecodeError:
        raise fail(INVALID_JSON, "malformed json body") from None
```

listing 394: the decode ladder: 415 wrong type, 413 past one mib, 400 malformed (source: python/api/pyapi/kernel.py, lines 177-188)

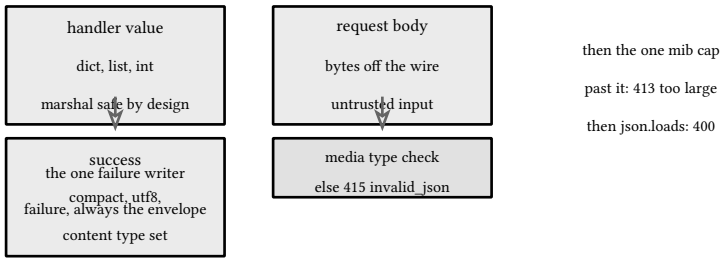


figure 115: two writers, one decode path, four contract statuses

27.5. the request id

Every request needs one id that every artifact agrees on: the access log line, the error envelope, the panic log. The ruling carrier is `contextvars`, one `ContextVar` with a default of `None`, set and reset through its token by the request id layer in the next chapter. A thread local would not do, because the deadline layer runs the handler on a worker thread, and a thread local set on the serving thread would read empty there, where a context copied into the thread carries the value through.

Resolution happens once, in a fixed order: the contextvar when the middleware has run, else the client's `X-Request-ID` when the value parses as a uuid, else a fresh id from the injected source. Parsing is contract, not politeness: the envelope promises `request_id` is a uuid, and a free text header echoed into logs is log injection. `uuid.UUID` accepts the braced, urn, and bare hex spellings and `str` normalizes them to the dashed form, and garbage raises `ValueError` and is replaced by the mint, never echoed. The mint is `uuid.uuid7`, time ordered the way the store chapter's ids are, and it is a parameter, so the vector replays in later chapters inject their fixed ids through the request id layer rather

than reaching into the kernel. The token pair is the hygiene rule: `reset_request_id` puts the context back exactly as it was, the layer's `finally` owns the reset, and a test pins the leak, after a dispatched request `current_request_id` is `None` again:

```
def resolve_request_id(headers, new_id=mint_request_id):
    """the resolution order: the contextvar when the middleware has
    run, else a client header that parses as a uuid, canonicalized,
    else a fresh uuid v7 from the injected source. garbage headers
    are replaced, never echoed."""
    current = _request_id_var.get()
    if current is not None:
        return current
    raw = headers.get("x-request-id", "")
    if raw:
        try:
            return str(uuid.UUID(raw))
        except ValueError:
            pass
    return new_id()
```

listing 395: resolve once: contextvar, then a parseable header, then the mint (source: python/api/pyapi/kernel.py, lines 138-152)

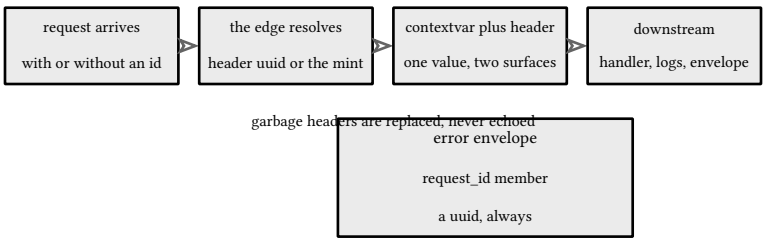


figure 116: one id, born at the edge, quoted by everything downstream

27.6. testing the kernel without a socket

Go has `httptest` and this lane has something better for most of the suite: responses are values. A test builds a `Request` the way the adapter would, calls `app.dispatch`, and asserts on the `Response` it gets back, status, headers, exact body bytes. Nothing listens, so there is no port to collide with and no timing, the round trip is a function call in the test's own thread.

The adapter is the only code that touches a socket, and it earns six tests over a real loopback listener: `make_server` binds `127.0.0.1` on port 0, the operating system picks a free port, a thread runs `serve_forever`, and the test's `urllib` call blocks on the response, never on a sleep. The round trip proves the HTTP/1.1 framing, the post test the read path, the 404 and 405 tests the envelope on the wire with `Allow`, and the TRACE test the every method rule. Cleanup is ordered, stop the loop, close the listener, join the thread, and cleanups run last in first out:

```
    def test_healthz_freezes_its_bytes(self):
        app = kernel.App(kernel.Config(version="dev"))
        response = app.dispatch(make_request(path="/healthz"))
        self.assertEqual(response.status, 200)
        self.assertEqual(response.body, b'{"status":"ok","version":"dev"}')

    def test_the_version_comes_from_config(self):
        app = kernel.App(kernel.Config(version="9f3a1"))
        response = app.dispatch(make_request(path="/healthz"))
        self.assertEqual(json.loads(response.body)["version"], "9f3a1")

    def test_readyz_answers_ready_while_the_probe_agrees(self):
        app = kernel.App(kernel.Config(version="t"))
        response = app.dispatch(make_request(path="/readyz"))
        self.assertEqual(response.status, 200)
        self.assertEqual(response.body, b'{"status":"ready"}')

    def test_readyz_flips_to_overload_when_the_probe_disagrees(self):
        app = kernel.App(kernel.Config(version="t", ready=lambda: False))
        response = app.dispatch(make_request(path="/readyz"))
        self.assertEqual(response.status, 503)
        self.assertEqual(envelope_of(response)["code"], "overload")
```

listing 396: values in, values out: exact bytes asserted, no listener involved (source: python/api/tests/test_kernel.py, lines 312-333)

The property loop is the suite's own mechanism, a seeded `random.Random` over a named invariant, no property library. The router's invariant is order independence, two hundred shuffled registrations of the same five patterns answer the same paths with the same handlers. Zero tests are skipped and zero sleep, the two rules the whole lane runs under.

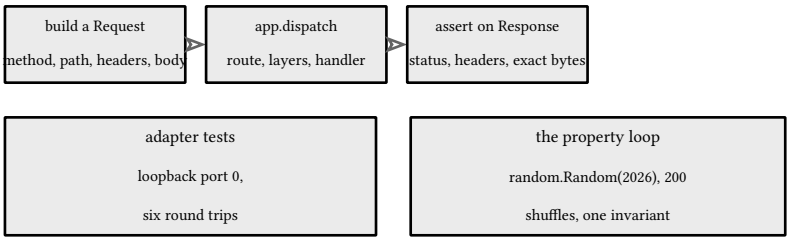


figure 117: build, dispatch, assert: one function call, and the adapter's five socket tests

27.7. the app skeleton and its wiring table

The kernel also ships the program skeleton. `Config` is the config hub, listen address, build version, and a readiness probe, and `from_env` reads `GOAPI_ADDR` with `GOAPI_VERSION`, names kept for compose parity with the go and c# lanes running the same contract. The two kernel routes are the deployment contract: `GET /healthz` answers liveness with the version from config so a deployment can prove what it is running, and `GET /readyz` answers readiness, 200 while the probe agrees and the envelope's `overload` code at 503 the moment it does not, the hook the load chapter's drain wiring flips. `route` registers with the tie check every family inherits, `use` appends a middleware layer first used outermost, and `serve` runs `serve_forever` on a `ThreadingHTTPServer` subclass that carries the app to its handler instances:

```
    def __init__(self, config=None):
        self.config = config or Config()
        self.routes = []
        self.layers = []
        self.route("GET", "/healthz", self._healthz)
        self.route("GET", "/readyz", self._readyz)

    def route(self, method, pattern, handler):
        compiled = _compile(pattern)
        for existing in self.routes:
            if existing.method ≠ method:
                continue
            if self._conflicts(existing.compiled, compiled):
                raise ValueError(
                    f"{method} {pattern} ties an existing pattern, "
                    "an ambiguous table never ships"
                )
        entry = _Route(method, pattern, compiled, handler)
        self.routes.append(entry)
        return entry
```

listing 397: the app registers its two routes with the tie check every family inherits (source: python/api/pyapi/kernel.py, lines 253-272)

`main.py` is the composition root, one `build_app` function the integrator owns and every family appends its wiring to, mirroring the go lane's `main.go` and `wire.go` split: a construction, the middleware stack in wiring order, then one registration block per family. Each later chapter lands as a few lines inside `build_app` and nothing in this module changes.

The framework story is one sentence, said the way every chapter of this part says it: the stdlib primitive comes first, the framework family is composition over those primitives plus dated meta, book 8, ch 22 (python 3.14, fastapi) being the teaching home for fastapi with flask and django beside it. Nothing here imports any of them, the maintenance risk rationale runs in the suite chapter, and the platform surface already carries the weight: the router, the envelope, and the codec above are the primitives those frameworks would wrap.

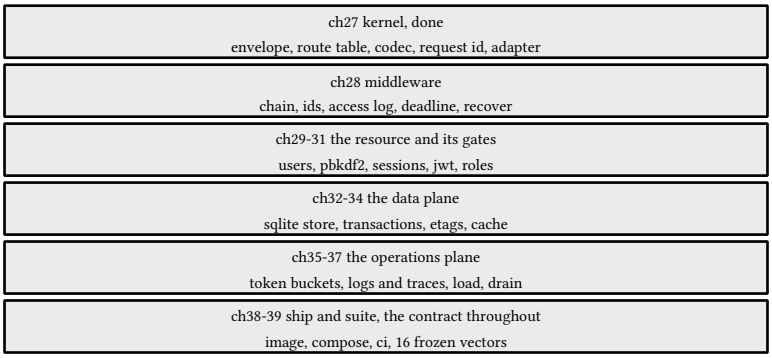


figure 118: the part as six rows: every chapter adds routes or layers, none rewrites the kernel

sources: docs.python.org for http.server, socketserver, contextvars, email.message, json, uuid, urllib.parse, and unittest, accessed 2026-09-27, with the `ThreadingHTTPServer` composition read off its method resolution order in the pinned 3.14 interpreter and the `get_content_type` default probed there the same day. Verified by `pyapi` tests, 42 of them over `kernel.py` in `tests/test_kernel.py`, plus the verify-pyapi gates: unittest discovery, compileall over the vehicle, and ruff under the book's frozen config.

28. middleware

Cross-cutting behavior does not belong inside the handlers, and the stdlib does not make anyone put it there. This chapter composes the service's middleware stack over the previous chapter's kernel, in `pyapi/middleware.py`: the chain fold over the kernel's handler shape, request ids born once at the edge, the access log over the injected clock with its hand rolled json log handler, the deadline as a worker thread plus an event wait, recover into the contract envelope, and the wiring order that makes the stack correct rather than merely present.

28.1. the chain over the kernel's shape

The kernel's handler shape is one function, a `Request` in and a `Response` out, and one layer is one function over exactly that: given the next handler, return the handler that wraps it. `chain` folds a list of layers into one by wrapping right to left, so the first argument ends up outermost, it sees the request first and the response last, the onion picture every middleware chapter draws because it is the true one. A chain is itself a layer, so chains compose into bigger chains, and the kernel's `App.use` appends onto the same fold, first used outermost:

```
def chain(*layers):
    """folds layers into one: chain(a, b, c)(h) serves a, then b,
    then c, then h, so the first argument is outermost and sees the
    request first and the response last."""

    def layer(next):
        for wrap in reversed(layers):
            next = wrap(next)
        return next

    return layer
```

listing 398: one layer is a function over the next handler, the fold serves the first argument outermost (source: python/api/pyapi/middleware.py, lines 50-60)

The loop runs backwards for a reason worth saying out loud: folding left to right would make the last layer outermost, and the wiring order in this chapter's sixth section is a contract. A test pins the fold directly, three recording layers around a terminal handler assert the pass order `outer`, `inner`, `handler`, and a second test nests one chain inside another to show composition:

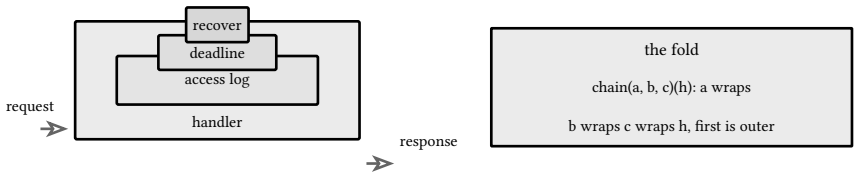


figure 119: the onion: first argument outermost, request in, response out

28.2. recover

An exception out of a handler must not take the process down, and the client must still get the contract envelope. The recover layer is one `try` around the inner call: an `Exception` becomes one error log line with the request id, the exception repr, and the full traceback, then the internal envelope through the kernel's one failure writer, with a generic message, because a leaked `sqlite3.OperationalError` tells a client more than it should know. The go lane had two failure channels, returned errors and panics, where python has one, the exception, and the class does the split: `APIError` is the handled failure the kernel already renders, everything else is a bug, and this layer is where bugs become answers:

```
    def layer(next):
        def handler(request):
            try:
                return next(request)
            except KeyboardInterrupt, SystemExit:
                raise
            except Exception as exc:
                logger.error(
                    "panic recovered",
                    extra={
                        "request_id": kernel.current_request_id() or "-",
                        "exception": repr(exc),
                        "stack": traceback.format_exc(),
                    },
                )
                return kernel.failure(
                    request, kernel.fail(kernel.INTERNAL, "internal error")
                )
```

listing 399: recover: signals rethrow, bugs become the envelope with the detail kept for the log (source: python/api/pyapi/middleware.py, lines 163-180)

Two exceptions are deliberately let through. `KeyboardInterrupt` is the interpreter's own signal and `SystemExit` is a request to exit the process, the closest stdlib kin of the go lane's `ErrAbortHandler` sentinel, and middleware that converts either into a 500 changes what the platform was told to do. The `except` clause lists both without parentheses, a spelling python 3.14 allows for the first time when the clause carries no `as`, and the pinned interpreter accepts it while the format gate enforces it. Placement is the one real rule: recover sits innermost, inside the deadline, so a bug becomes a response on the worker thread and flows back through the deadline's box as a value.

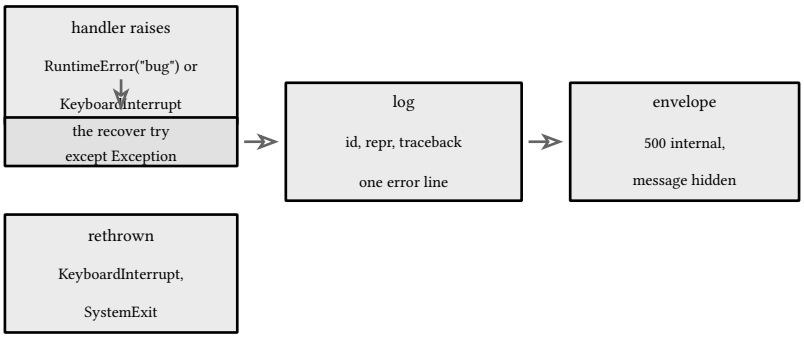


figure 120: bug unwind: one path to the log, one to the envelope, signals rethrown

28.3. request id at the edge

Every request needs one id that every artifact agrees on, and this layer is where it is born. Resolution is the kernel's order from the previous chapter, the contextvar when set, else a client header that parses as a uuid, else the mint, and the layer is the only writer: it resolves once, echoes the id on the response header so a client can correlate its own logs with the server's, and installs it in the contextvar so everything downstream reads one value without touching headers again. The reset is the hygiene half, the `finally` block returns the token before the layer returns, so no id leaks onto the next request the serving thread handles:

```
def request_id(new_id=None):
    """resolves one id per request at the edge, echoes it on the
    response, and installs it in the contextvar for everything
    downstream: the envelope, the access log, the handler."""
    mint = new_id or kernel.mint_request_id

    def layer(next):
        def handler(request):
            rid = kernel.resolve_request_id(request.headers, mint)
            token = kernel.set_request_id(rid)
            try:
                response = next(request)
            finally:
                kernel.reset_request_id(token)
            response.headers[kernel.REQUEST_ID_HEADER] = rid
            return response

        return handler

    return layer
```

listing 400: resolve once at the edge, echo it, set the contextvar, reset in finally (source: python/api/pyapi/middleware.py, lines 63-82)

The mint is a factory parameter, and that is the vector seam: the replay chapters inject their fixed ids by passing a callable here, never by reaching into the kernel. The tests check all three births, a client uuid echoes, garbage is replaced by the injected mint rather than passed through, an absent header mints, and the downstream envelope quotes whatever the edge resolved, which is the property the whole stack depends on.

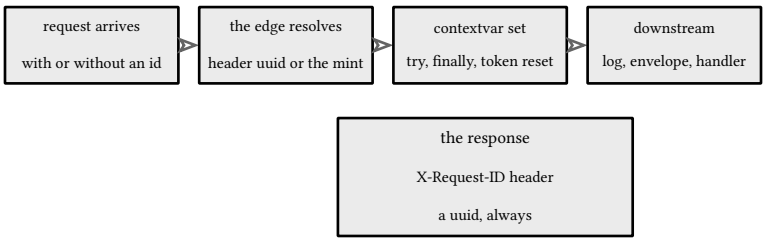


figure 121: one id, born at the edge, quoted by everything downstream

28.4. the access log

Go's access log had to wrap the `ResponseWriter` and capture the first status, because a go handler writes a response to a stream and returns nothing. This kernel's handler returns the response as a value, and that difference deletes the whole problem class: the log layer calls the inner stack, holds the `Response` it gets back, and reads `status` and the body length straight off it, so the line says exactly what went on the wire, there is no first-write rule to get wrong, and no optional interface silently lost to a wrapper. The one cost is ordering, the line is written after the inner stack returns, which is precisely what makes a recovered 500 loggable as a 500:

```
    def layer(next):
        def handler(request):
            start = clock()
            response = next(request)
            logger.info(
                "request",
                extra={
                    "method": request.method,
                    "path": request.path,
                    "status": response.status,
                    "bytes": len(response.body),
                    "duration": round(clock() - start, 6),
                    "request_id": kernel.current_request_id() or "-",
                },
            )
            return response
```

listing 401: the response is a value: the log reads status and bytes straight off it (source: python/api/pyapi/middleware.py, lines 93-108)

The duration comes from the injected clock, `monotonic` by default, called once before and once after, so a test with a two value clock asserts the exact `0.25` and no test ever reads a wall clock. The line itself is json from the hand rolled `JsonLogHandler`, a subclass of `logging.Handler` whose `emit` builds one dict per record, level, message, and whichever request attrs the record carries, compact separators and utf8 out, one object per line. It is the same handler seam the observability chapter builds its full story on, introduced here at the size the access log needs.

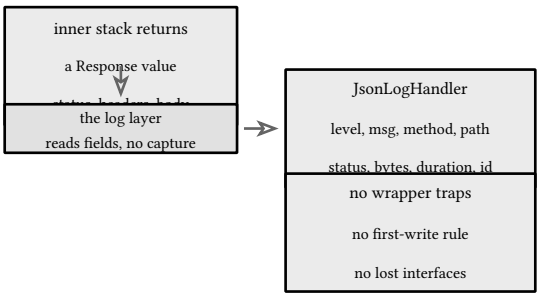


figure 122: fields off the response value, one json line, duration over the injected clock

28.5. the deadline over the injected clock

Python cannot preempt a running handler, and the honest deadline admits it: the handler runs on a worker thread, the serving thread waits on an `Event` for whatever remains of the budget, computed from the injected clock, and the two never touch the same response. `Event.wait` with a zero or negative remainder returns immediately, so an expired budget answers at once with the `overload` envelope while the abandoned handler keeps running on its daemon thread, writing into a box nobody reads, the detached writer idea restated as a dict. The worker is a copy of the caller's context running `run`, which is how the request id reaches the handler thread, a plain thread local set on the serving thread would read empty there. An exception out of the handler is caught as `BaseException` and delivered to the serving thread verbatim, where recover or the platform sees it:

```
            def run():
                try:
                    box["response"] = next(request)
                except BaseException as exc:
                    box["error"] = exc
                finally:
                    done.set()

            worker = threading.Thread(
                target=context.run, args=(run,), daemon=True, name="pyapi-deadline"
            )
            worker.start()
            if not done.wait(max(0.0, expires - clock())):
                return kernel.failure(
                    request, kernel.fail(kernel.OVERLOAD, "request timed out")
                )
            if "error" in box:
                raise box["error"]
            return box["response"]
```

listing 402: the worker fills a box, the wait decides, expiry answers overload and drops the late response (source: python/api/pyapi/middleware.py, lines 131-149)

The expiry test is the lane's whole testing discipline in one place. The handler gates on an `Event` the test holds closed, the budget is zero over a constant clock, so the wait returns false without any sleeping, the 503 envelope is asserted, the late response is proven absent from the wire bytes, and only then does the test open the gate and join the abandoned thread by waiting on its own produced event. Nothing measures time, everything waits on a gate.

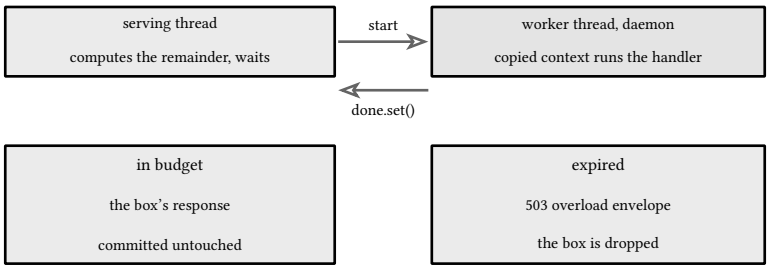


figure 123: two threads, one box: the wait decides who answers

28.6. wiring order and why

The order of the four layers is contract, and each position is a sentence. Request id outermost, because every other artifact quotes it: the access log field, the panic log, the envelope, all of it. Access log second, outside recovery, so a recovered 500 is logged as the 500 the client saw rather than swallowed inside the layer that produced it. Deadline third. Recover innermost, and in this lane the reason is data flow rather than go's goroutine rule: the bug becomes a response on the worker thread, flows back through the deadline's box as a value, and the whole stack sees one answer. The wiring is one `chain` call in the composition root, reading top to bottom as the request flows, and a test pins it as the stack builder every other test reuses:

```
    def stack(self, log, clock=lambda: 0.0):
        return middleware.chain(
            middleware.request_id(lambda: "01a0f5d5-6c80-7ca2-8d7f-652c7031c304"),
            middleware.access_log(log.logger, clock=clock),
            middleware.deadline(5.0, clock=clock),
            middleware.recover(log.logger),
        )
```

listing 403: the stack, in wiring order, one chain call in the composition root (source: python/api/tests/test_middleware.py, lines 240-247)

The stack test is the one that fails when anyone reorders the wiring. A handler that raises `RuntimeError` through the full stack produces one answer everywhere: the client sees the 500 envelope, the response header and the envelope's `request_id` are the same value, and the log holds two lines that agree, the panic line with the stack, then the request line with status 500 quoting the same id. Two answers for one request is the failure mode the order exists to prevent, and the go lane's wrong-order diagram shows exactly how it happens there when the log sits inside recover:

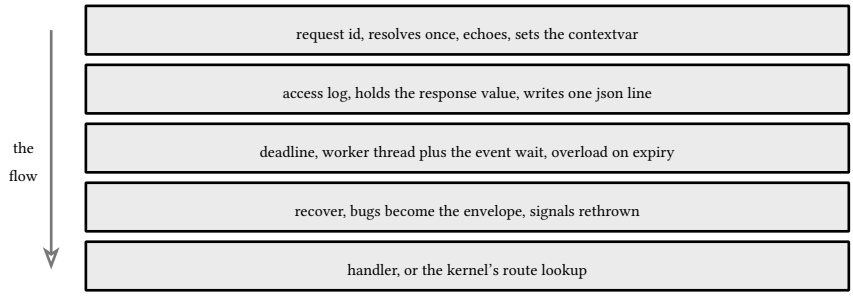


figure 124: right order: the log outside recover sees the answer the client saw

28.7. testing the stack

The kernel's value shape pays its second dividend here: the whole stack tests as plain function calls, no listener, no client, no socket at all, because every layer is a function over a function over a `Request`. The only real thread in the suite is the deadline's own worker, and it joins through events, never through time. The logging assertions read whole json lines back through a logger wired to the json handler over a `StringIO`, one buffer per test, so a wrong field is a failed assert on a parsed object rather than a grep over prose. The clock is a two value iterable where the duration matters and a constant where only the ordering matters:

```
    def test_expiry_answers_overload_and_drops_the_late_response(self):
        gate = threading.Event()
        produced = threading.Event()

        def slow(request):
            gate.wait()
            produced.set()
            return kernel.success({"late": True})

        response = middleware.deadline(0.0, clock=lambda: 100.0)(slow)(make_request())
        self.assertEqual(response.status, 503)
        self.assertEqual(json.loads(response.body)["error"]["code"], "overload")
        self.assertNotIn("late", response.body.decode())
        gate.set()
        self.assertTrue(produced.wait(5), "the abandoned handler never finished")
```

listing 404: the expiry test: a held gate, a zero budget, a join, no sleeps (source: python/api/tests/test_middleware.py, lines 200-215)

The last test wraps the stack around a real `App` and dispatches through it, 200 on the healthz route, 404 on an unknown path, and asserts the log shows both statuses in order, which is the integration proof that the layers and the kernel agree on one request lifecycle. Eighteen tests cover the module, zero are skipped, and none of them sleeps.

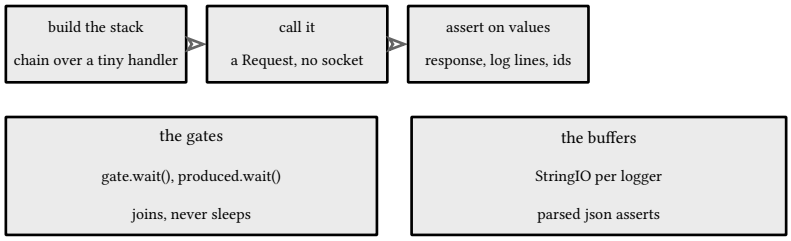


figure 125: function calls with gates: the only thread in the suite is the deadline's own

sources: docs.python.org for logging, threading, contextvars, and traceback, accessed 2026-09-27, with the paren-free except spelling checked against the what's new for 3.14 and `Event.wait` semantics against the threading page the same day. Verified by `pyapi` tests, 18 of them over `middleware.py` in `tests/test_middleware.py`, plus the verify-pyapi gates over the whole vehicle.

29. the user resource

The service's first real resource is the user: a record with an id, an email, a display name, roles, a creation time, and a version. The chapter builds the resource layer on the chapter 27 kernel: the representation and its two projections, a strict decode over the stdlib `json` module, validation from first principles, merge-patch updates, keyset pagination, the status contract, and the golden handler tests that pin all of it against the frozen vectors, all in the `pyapi/users/` package.

29.1. the representation

A resource is three things at once: the bytes on the wire, the fields each reader may see, and the strong etag that names one exact version of the body. The stored record is one frozen dataclass and the wire shape is a dict in a fixed member order, the struct-with-tags discipline stated in python: dict literal order is wire order, and the frozen vectors are byte-authoritative:

```
    def to_wire(self) -> dict:
        return {
            "id": self.id,
            "email": self.email,
            "display_name": self.display_name,
            "roles": list(self.roles),
            "created_at": rfc3339(self.created_at),
            "version": self.version,
        }

    def public_wire(self) -> dict:
        return {
            "id": self.id,
            "display_name": self.display_name,
            "created_at": rfc3339(self.created_at),
            "version": self.version,
        }

    def project(self, admin: bool) -> dict:
        """the representation a reader may see: the full record for
        admins, the public projection for everyone else."""
        return self.to_wire() if admin else self.public_wire()
```

listing 405: the record, its wire members, and the public projection (source: python/api/pyapi/users/record.py, lines 60-82)

The projection is where field-level authorization lives: email and roles are admin-only data, the list handler calls one method and marshals whatever comes back, and a rule enforced in one function cannot be forgotten in a second handler. The version member bumps once per successful write. The password hash rides the dataclass, and because `to_wire` never mentions it, no marshal can leak it.

The etag is not stored anywhere, it is computed: marshal the representation once, sha256 those exact bytes, quote the hex. The etag is a function of the body, so the two can never disagree, and the kernel's one success writer keeps it that way: handlers hash `response.body`, the bytes about to go on the wire, never a second marshal that could drift:

```
def etag_body(body: bytes) -> str:
    return f'"{hashlib.sha256(body).hexdigest()}"'
```

fragment, illustrative, not executed

python field	json member
id: str	id
email: str	email
display_name: str	display_name
roles: tuple	roles
created_at: datetime	created_at
version: int	version
password_hash: str	never sent

figure 126: the record fields against the wire members, one field kept off the wire

29.2. decoding with strictness

Input is hostile until proven otherwise, and the kernel's decode ladder answers three questions in a fixed order: is the media type json at all, is the body parseable inside the 1 MiB cap, and does it carry only members the contract knows. The kernel's `read_json` owns the first two, answering 415 and 413 and 400. The third belongs to this package, because the stdlib `json.loads` answers any shape at all, so strictness is a walk over the parsed object:

```
def bind_strings(request, names):
    """the typed decode: unknown members refuse with 400, every kept
    member a string, null reading as the empty string the way the go
    lane's typed decode reads its zero value, so a null member reaches
    validation and answers 422, and wrong types answer 400."""
    payload = kernel.read_json(request)
    if not isinstance(payload, dict):
        raise kernel.fail(kernel.INVALID_JSON, "body must be a json object")
    unknown = sorted(set(payload).difference(names))
    if unknown:
        raise kernel.fail(kernel.INVALID_JSON, f"unknown member {unknown[0]}")
    bound = {}
    for name in names:
        value = payload.get(name, "")
        if value is None:
            value = ""
        elif not isinstance(value, str):
            raise kernel.fail(kernel.INVALID_JSON, f"member {name} must be a string")
        bound[name] = value
    return bound
```

listing 406: unknown members refuse with 400, every kept member a string (source: python/api/pyapi/users/routes.py, lines 75-95)

Rejecting unknown members is the strictness that pays rent. A client that sends a pet member against a contract with no pet gets a 400, not a silently ignored field, so a typo'd member name surfaces at the border instead of corrupting data six layers in. The accepted set is the name tuple the caller passes, decode's business and not validation's. A null member reads as the empty string, the zero value the go lane's typed decode produced, so a null reaches validation and answers 422 with the presence rule's own words, while a number where a string belongs answers 400 at the border.

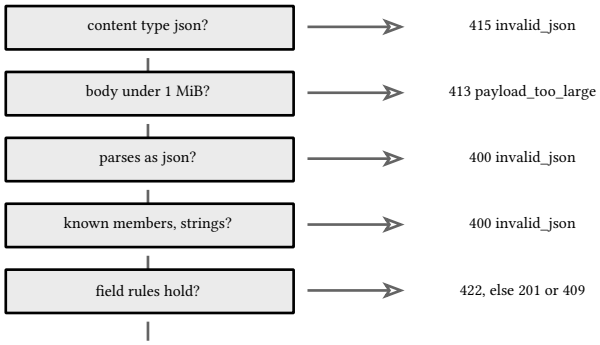


figure 127: the decode ladder: each rung rejects a class of bad input

29.3. validation from first principles

Validation rules come from the contract, not from a library: passwords at least 12 characters and at most 128, one email shape, display names 1 to 64 characters. Stating them as plain code keeps every number greppable and every message owned. The function returns one detail per violated rule rather than failing on the first, so the 422 body's details array tells the whole story:

```
def validate_register(email: str, password: str, display_name: str) -> list[str]:
    """one detail per violated rule, in field order, so the 422
    details array tells the whole story in one round trip."""
    details = []
    if not email:
        details.append("email is required")
    elif not valid_email(email):
        details.append("email is not a valid address")
    if not password:
        details.append("password is required")
    elif len(password) < MIN_PASSWORD_LENGTH:
        details.append(f"password must be at least {MIN_PASSWORD_LENGTH} characters")
    elif len(password) > MAX_PASSWORD_LENGTH:
        details.append(f"password must be at most {MAX_PASSWORD_LENGTH} characters")
    if not display_name:
        details.append("display_name is required")
    elif len(display_name) > MAX_DISPLAY_NAME:
        details.append(f"display_name must be at most {MAX_DISPLAY_NAME} characters")
    return details
```

listing 407: one detail per broken rule, in field order, off the named constants (source: python/api/pyapi/users/validate.py, lines 17-36)

Three classes of rule run here. Presence rules catch missing members. Shape rules catch an email that parses as nothing, where the standard library's `email.utils.parseaddr` plus a one-at-sign and dotted-domain check approximates the real grammar without rejecting addresses people actually hold. Range rules catch lengths, with the bounds in named constants the detail strings interpolate rather than restate, so a message can never drift from the rule it reports.

The 422 and the 409 are different animals. A rule violation is about the request alone, checkable without touching state, so it is 422 validation. The email conflict is about the store's current contents, so it is 409 conflict, and it surfaces only after validation passed and the create call raised its conflict error. The normalize step closes the last dodge: emails are lower-cased and trimmed before they reach the store, so `Ada@Example.ORG` cannot register a second account beside `ada@example.org`.

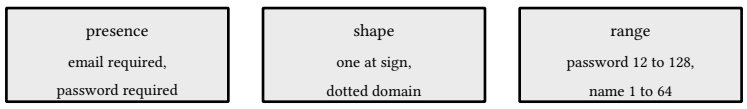


figure 128: three rule classes, one details array

29.4. patch merge semantics

RFC 7386 merge-patch gives updates three-way semantics a typed decode cannot express: absent leaves the field alone, null deletes it, a value replaces it. So the patch decodes into whatever `json` produces, a dict, and the handler walks the members in sorted order so details are deterministic:

```
    for key in sorted(payload):
        value = payload[key]
        if key == "display_name":
            if value is None:
                details.append("display_name cannot be removed")
            elif not isinstance(value, str):
                details.append("display_name must be a string")
            else:
                broken = validate_display_name(value)
                details.extend(broken)
                if not broken:
                    name = value
        elif key in _IMMUTABLE_FIELDS:
            details.append(f"{key} is immutable")
        else:
```

listing 408: the member walk: null on the display name too, immutable members refuse, unknown members refuse (source: python/api/pyapi/users/routes.py, lines 188-202)

The contract makes one member writable, the display name, and the walk shows the full policy in one branching. A null on it answers 422, because deleting the display name would leave the record invalid. The immutable members, id, email, created_at, version, roles, refuse the moment they appear, null or not: patching them is not a validation typo, it is an attempt to write a field the contract closed. Unknown members refuse too, here by the walk because a parsed dict cannot carry the register route's name tuple.

After the walk, the precondition and the write run in order. The client's If-Match must equal the current body's etag, else 412, and the update carries the expected version into the store so a write that lost a race answers 412 rather than silently clobbering. The no-op patch answers 200 with the current representation, nothing bumped.

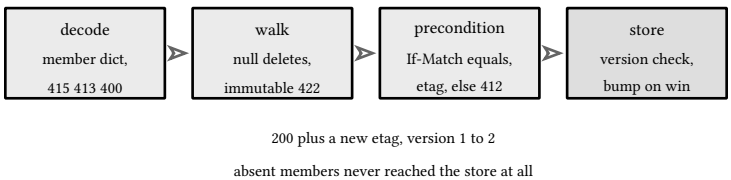


figure 129: a patch request walks the members, then the precondition, then the store

29.5. keyset pagination

Offset pagination asks for row 86041 onward, and it fails twice. The database still walks and discards every skipped row, so page 4000 costs what a table scan costs. And offsets are positions in a moving file: one registration between requests and a row appears twice or vanishes. Keyset pagination fixes both by remembering where it stopped. The order is the pair (created_at, id), the cursor encodes the last row's position, and the next page is everything strictly after it:

```python
def key_of(user: User) -> CursorKey:
    return CursorKey(t=unix_milli(user.created_at), i=user.id)


def sorts_after(user: User, cursor: CursorKey) -> bool:
    """whether user sorts strictly after the cursor position."""
    key = key_of(user)
    if key.t != cursor.t:
        return cursor.t < key.t
    return cursor.i < key.i


def encode_cursor(key: CursorKey) -> str:
    """base64url without padding of the two-member payload, the opaque
    string clients pass back verbatim."""
    payload = json.dumps({"t": key.t, "i": key.i}, separators=_WIRE_SEPARATORS)
    return base64.urlsafe_b64encode(payload.encode("utf-8")).decode("ascii").rstrip("=")
```

listing 409: the position, the composite order, and the base64url unpadded encoding (source: python/api/pyapi/users/pagination.py, lines 35-52)

The payload is deliberately tiny, two members, because the cursor is a position, not a page number. Time is milliseconds since the epoch, computed with pure integer arithmetic off a `datetime` because float timestamps drift at this magnitude. The id breaks ties when two users share a creation millisecond, and uuid v7 ids sort by their own embedded time, so the composite order is total. The encoding is `base64.urlsafe_b64encode` with the padding stripped: opaque to clients, trivially decodable in a test, and the frozen vectors carry its exact bytes. A cursor the service never issued is garbage, not a missing page, so decode failures answer 422, and the limit must be an integer in 1..100 with a default of 20. The decode is strict about the alphabet, the member set, and the member types, because anything looser would let a client smuggle a hand-built position past the contract.

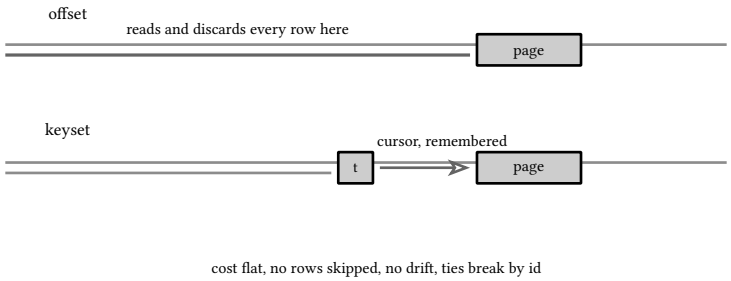


figure 130: offset walks the whole prefix, keyset jumps to the remembered position

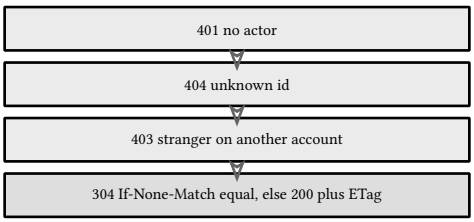
29.6. the status contract

Every failure this layer can produce already has a code in the chapter 27 table, so this layer states outcomes and the table states statuses. The get handler is the whole ladder in one screen: 401 without an actor, 404 for an unknown id, 403 for a stranger touching someone else's record, then the etag work:

```python
def _get(family, request):
    actor = require_actor()
    uid = request.values["id"]
    user = _get_or_404(family, uid)
    require_owner(actor, uid)
    response = kernel.success(user.to_wire())
    etag = etag_body(response.body)
    if conc.if_none_match_hits(request.headers.get("if-none-match", ""), etag):
        return kernel.Response(304, {"ETag": etag}, b"")
    response.headers["ETag"] = etag
    return response
```

listing 410: the failure ladder before any body is written, 304 after it (source: python/api/pyapi/users/routes.py, lines 133-143)

After the ladder comes the one status that is not a failure, 304. If the client's If-None-Match names the computed etag, the body it already holds is current, and the service answers the header and nothing else. The order inside the ladder is a policy choice this chapter owns: the 404 answers before the 403, so an unknown id and a stranger's id are distinguishable, and APIs that prefer to hide existence answer 404 for both, a tradeoff the authorization chapter walks. What the contract forbids is a third path: every failure raises one kernel `APIError` the failure writer renders, exactly one writer. The owner check is two lines, `user_id` equal or admin, shared by every object verb.



patch adds 412 above the ladder, one envelope for every failure

figure 131: one request climbs the ladder, each rung one contract code

29.7. golden handler tests

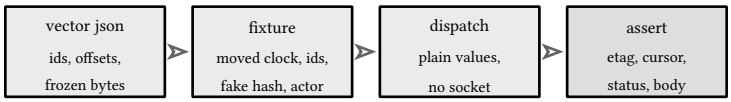
The frozen vectors under `contract/testdata` are the fixture source of truth, and the tests read them as data: ids, timestamps, etags, and cursor bytes come from the json files, never re-derived by the test:

```python
    def test_register_201_matches_the_frozen_vector(self):
        vector = json.loads((VECTORS / "01-register-201.json").read_text())
        response = self.seed_ada()
        self.assertEqual(response.status, 201)
        self.assertEqual(
            response.headers["Location"], vector["response"]["headers"]["Location"]
        )
        self.assertEqual(
            response.headers["ETag"], vector["response"]["headers"]["ETag"]
        )
        self.assertEqual(response.body, compact(vector["response"]["body"]))
```

listing 411: the vector drives the fixture and judges the response, bytes exact (source: python/api/tests/test_users_routes.py, lines 104-114)

Exactness is the point. The etag assertion proves the body bytes byte for byte, and the response body is compared against the vector's compact form directly. The harness injects everything time-dependent: a clock moved by hand between requests, an id source with the vector's uuids, a fake hash for the authn chapter's pbkdf2, and the actor installed into the contextvar the identity middleware will own. Nothing sleeps, nothing dials.

The same discipline covers the rest of the slice: register 201 and 409, the 422 details array, unknown and immutable patch members, the stale If-Match 412, the walk's exact cursors, and the ownership grid. A seeded `random.Random` property loop walks shuffled insertions at page sizes 1, 3, and 100 and asserts the invariant: the pages cover every user exactly once, in (created_at, id) order, whatever order the rows arrived in, and when a vector changes the code is wrong.



drift means the code is wrong, never the vector

figure 132: a golden test: the vector file drives the fixture and judges the dispatch

sources: docs.python.org/3/library/json.html for dumps separators and loads, docs.python.org/3/library/email.utils.html for parseaddr, docs.python.org/3/library/base64.html for urlsafe_b64encode and b64decode validate, and docs.python.org/3/library/hashlib.html for sha256, all accessed 2026-09-27 against the pinned cpython 3.14.7. RFC 7386 and RFC 9110 sections 8.8.3 and 13.1.1 at rfc-editor.org, the offset argument from use-the-index-luke.com. Verified by `tests/test_users.py` and `tests/test_users_routes.py` under `make verify-pyapi`, 39 tests green, ruff clean at 88 columns.

30. authn

Authentication answers one question, who is calling, and this chapter builds every piece of the answer in `pyapi/authn/`, one package mirroring the go lane's `internal/authn`: pbkdf2 password storage with parameters carried in the hash string, a login path flattened so unknown emails and wrong passwords cost the same, sessions with rotation and reuse detection, the cookie line the vectors freeze, and the HS256 jwt hand-rolled over `hmac` plus `hashlib`.

30.1. password storage with pbkdf2

Passwords are never stored, only proofs that cost an attacker real time to reproduce. The platform's answer is `hashlib.pbkdf2_hmac`, salted sha256 iterated 210000 times to derive a 32 byte key from a 16 byte salt, the frozen contract's tier, with `ISSUE_ITERATIONS` at 210000 asserted structurally and never run in tests, `TEST_ITERATIONS` at 2000 exercising the identical path, and the salt source a seam:

```
    def hash(self, password: str) -> str:
        salt = self._salt_source()
        key = hashlib.pbkdf2_hmac(
            "sha256", password.encode("utf-8"), salt, self._iterations, KEY_LEN
        )
        return (
            "$pbkdf2-sha256"
            + f"$i={self._iterations},l={KEY_LEN}"
            + "$"
            + base64.b64encode(salt).decode("ascii")
            + "$"
            + base64.b64encode(key).decode("ascii")
        )
```

listing 412: one hash: fresh salt, contract iterations, phc-style rendering (source: python/api/pyapi/authn/password.py, lines 37-50)

Iteration count is the whole defense. A single sha256 over a password is microseconds on a gpu, while two hundred ten thousand sequential rounds move each guess into the tens of milliseconds, and the store thief pays the same because the count travels inside the string.

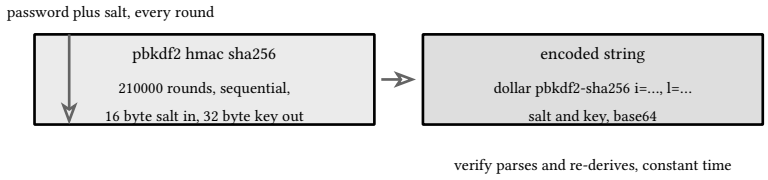


figure 133: pbkdf2: the salt and iteration count ride beside the derived key

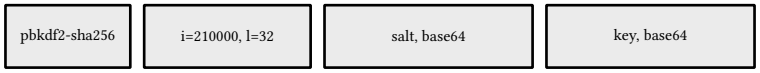
30.2. the phc string and the memory-hard alternative

The encoded string is the contract between hashing today and verifying years later. This service uses the phc style, `$pbkdf2-sha256$i=210000,l=32$salt$key` with salt and key in standard base64, so verification replays the exact parameters the hash was written with, never today's defaults. the parameters ride the string rather than the code's opinion, which is what a future tier change would turn: a new hash carries its own count and verifies beside the old ones, at the cost of a rehash on login this vehicle deliberately does not build:

```
    def verify(self, encoded: str, password: str) -> bool:
        """re-derives the key under the parameters carried in the
        encoded string and compares in constant time. malformed strings
        and wrong passwords both answer false: verification failure is
        a boolean fact, never a reason to distinguish the two paths by
        shape."""
        parsed = _parse_phc(encoded)
        if parsed is None:
            return False
        iterations, salt, key = parsed
        derived = hashlib.pbkdf2_hmac(
            "sha256", password.encode("utf-8"), salt, iterations, len(key)
        )
        return hmac.compare_digest(derived, key)
```

listing 413: verify re-derives under the string's parameters and compares in constant time (source: python/api/pyapi/authn/password.py, lines 51-65)

The parse refuses what an attacker would plant in a stolen row, absurd iterations, absurd key lengths, base64 that fails `validate=True`, a tiny salt, all answered with the same false a wrong password gets: verification failure is a boolean fact, never a signal about which check failed. Argon2id is the construction the industry moved to, worth decomposing though this vehicle never imports it. Parameters, salt, and output length compress into a first block, lanes fill a large memory array by compressing the previous block with a referenced block from an earlier lane, each extra pass re-reads the array in a different order, and shrinking the array breaks the compression chain, so a gpu pays the memory traffic a cpu pays. The standard library ships none of it, so argon2 arrives through `argon2-cffi` and its kin, dated meta.



the string is the contract: verify replays these parameters, never defaults

figure 134: the phc string: parameters, salt, and key all travel together

30.3. login and timing flattening

A login endpoint that answers fast for unknown emails and slowly for wrong passwords hands out an account census to anyone with a stopwatch. The flattening fix sits in the unknown-email branch: run the same pbkdf2 verification the wrong-password path runs, against a dummy hash computed once at construction, discard the result, and answer the identical bytes:

```python
def _login(family, request):
    bound = users.bind_strings(request, ("email", "password"))
    if not bound["email"] or not bound["password"]:
        raise kernel.fail(
            kernel.VALIDATION_FAILED,
            "validation failed",
            ["email is required", "password is required"],
        )
    try:
        user = family.users_store.get_by_email(users.normalize_email(bound["email"]))
    except users.NotFoundError:
        # unknown email: verify against the dummy hash so the work,
        # and therefore the latency profile, matches the wrong
        # password path, then answer the identical bytes
        family.hasher.verify(family.dummy_hash, bound["password"])
        raise kernel.fail(kernel.UNAUTHORIZED, "invalid credentials") from None
    if not family.hasher.verify(user.password_hash, bound["password"]):
        raise kernel.fail(kernel.UNAUTHORIZED, "invalid credentials")
    return _issue(family, user.id)
```

listing 414: both failure branches answer the same 401 after the same work (source: python/api/pyapi/authn/routes.py, lines 139-158)

Both branches raise the same envelope, so the only observable difference left is network jitter. The success path runs on through `issue`, which mints the session, refresh token, cookie, and access token in one place: prove, then issue.

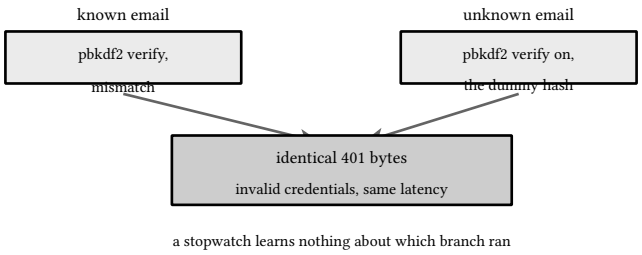


figure 135: the timing-equalized login: unknown email pays what a wrong password pays

30.4. server-side sessions

A session is the server's own record of one login: an id, the user, the expiry, a revoked flag, and the sha256 of the current refresh token, never the token itself, so a stolen store dump cannot be replayed live:

```python
class Session:
    """one login's server-side state. the refresh token itself never
    lives here or anywhere, only its sha256."""

    id: str
    user_id: str
    refresh_hash: bytes
    expires_at: datetime
    revoked: bool = False

    def alive(self, now: datetime) -> bool:
        """the shared time policy: a session counts while not revoked
        and not expired at now. the store knows structural state, the
        service knows the time, and this is the only place they
        meet."""
        return not self.revoked and now < self.expires_at
```

listing 415: the session record and the one time policy both channels share (source: python/api/pyapi/authn/session.py, lines 52-69)

The lifetime rules stay with the caller, on purpose: the store knows which token is current and which session is revoked, the service applies the time policy through `alive`, so every expiry test is pure arithmetic on an injected clock. The store is one lock over a session map and a token map, and it keeps superseded records because presenting one is the theft signal rotation builds on.

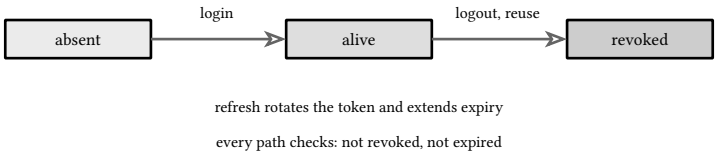


figure 136: the session lifecycle: login creates, refresh extends, logout or reuse revokes

30.5. cookie mechanics

The browser carries the refresh token in a cookie, and the cookie's attributes are its entire protection: Path scopes it to the api tree, HttpOnly keeps scripts from reading it, Secure keeps it off plain http, SameSite Lax blocks cross-site posts while allowing top-level navigation.

```python
def session_cookie(token: str) -> str:
    """the Set-Cookie value for one refresh token."""
    return f"{COOKIE_NAME}={token}; Path=/api; HttpOnly; Secure; SameSite=Lax"


def clear_session_cookie() -> str:
    """expires the cookie: Max-Age 0 rendered exactly, attributes kept
    so the deletion overwrites the exact cookie it replaces."""
    return f"{COOKIE_NAME}=; Path=/api; Max-Age=0; HttpOnly; Secure; SameSite=Lax"


def read_cookie(header: str | None, name: str) -> str | None:
    """pulls one cookie's value out of an incoming Cookie header:
    pairs split on the first equals sign, the first occurrence of a
    name wins, and a longer name sharing a prefix must not match."""
    if not isinstance(header, str):
        return None
    for part in header.split(";"):
        eq = part.find("=")
        if eq == -1:
            continue
        if part[:eq].strip() == name:
            return part[eq + 1 :].strip()
```

listing 416: the line builders and the incoming reader, plain templates (source: python/api/pyapi/authn/cookie.py, lines 13-35)

The attribute line is frozen by the vectors, order included, and the stdlib `http.cookies` is the one place the platform and the contract disagree: it prints a fixed order of its own, alphabetical, `HttpOnly; Path; SameSite; Secure`, identical in meaning and different in bytes from the vector's `Path; HttpOnly; Secure; SameSite`, so the vectors win and the builders are plain templates. Logout clears with Max-Age 0 rendered exactly.

attribute	stops	scope
Path /api	leak on sibling paths	one tree
HttpOnly	script exfiltration	read denied
Secure	plaintext transit	https only
SameSite Lax	cross-site post	top nav only

four attributes, four attacks, zero javascript visible

figure 137: the cookie attribute matrix: what each one stops

30.6. jwt from scratch with hmac

A json web token is three base64url parts, header, claims, signature, and the whole format fits in two functions of primitives the standard library owns. HMAC-SHA256 signs the first two segments with a shared secret, verification recomputes and compares in constant time, and the claim rules run after the signature, never before:

```python
def sign_hs256(secret: bytes, claims: dict) -> str:
    """signs claims with hmac-sha256. dict order is wire order, so the
    same facts serialize to the same bytes and the golden test
    reproduces the frozen vector's token exactly."""
    header = _b64url(
        json.dumps({"alg": "HS256", "typ": "JWT"}, separators=_JSON_SEPARATORS).encode()
    )
    payload = _b64url(
        json.dumps(claims, separators=_JSON_SEPARATORS, ensure_ascii=False).encode()
    )
    signature = hmac.new(
        secret, f"{header}.{payload}".encode(), hashlib.sha256
    ).digest()
    return f"{header}.{payload}.{_b64url(signature)}"
```

listing 417: sign HS256, no library, hmac plus hashlib plus base64 (source: python/api/pyapi/authn/jwt.py, lines 64-79)

The claims dict is built in a fixed order, exp, iat, iss, sid, sub, so the serialized form is deterministic and the golden test reproduces the frozen vector's token exactly, signature included. There is no roles claim on purpose: authorization reads the store, and the sid claim makes every token revocable by killing one session row. The parser kills the classics: the alg pin runs before anything trusts a payload byte, tampering and wrong secrets fail the constant-time compare, and the caller always gets one 401.

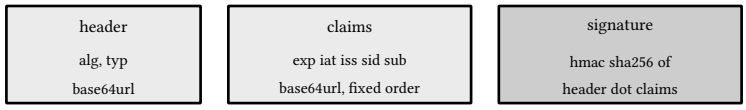


figure 138: three segments, the signature covers exactly the first two

30.7. the asymmetric variant in prose

HMAC has one secret and both sides hold it, so whoever can verify tokens can also mint them, fine in one process and the reason the issued lane stays HS256. When verification spreads, EdDSA over ed25519 earns its complexity: the private key signs, the public key verifies, and the verification side provably cannot forge. The standard library ships no ed25519, so this vehicle signs nothing with it and the `cryptography` package is dated meta. The algorithm is the only part that changes, the envelope and habits are shared, and this is the entire surface that would fork:

```python
def check_claims(claims: dict, now: datetime) -> dict:
    """the claim rules both algorithms share: our issuer, and exp
    strictly in the future relative to now."""
    if claims.get("iss") ≠ ISSUER:
        raise WrongIssuer("issuer is not this service")
    exp = claims.get("exp")
    if type(exp) is not int:
        raise MalformedToken("exp claim is not an integer")
    if unix_seconds(now) ≥ exp:
        raise ExpiredToken("token expired")
    return claims
```

listing 418: the claim rules both algorithms share, issuer then expiry (source: python/api/pyapi/authn/jwt.py, lines 108-120)

The header still pins the algorithm, so the alg confusion attack dies before any payload byte is read.

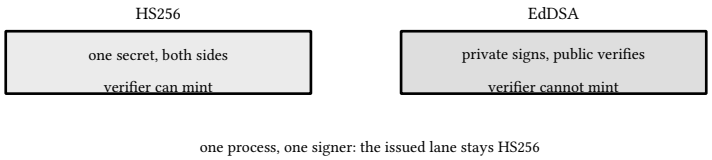


figure 139: shared secret versus split key: who can mint, who can check

30.8. the refresh rotation and reuse detection

Access tokens live fifteen minutes, the refresh token is the long-lived credential, and it rotates on every use. Rotation means the store remembers the token it just retired, marked superseded, because presenting a superseded token is the strongest theft signal the design gets: the holder moved on, so someone else is replaying. The answer is family revocation, the replay and its successor die together:

```
    def rotate(self, current: bytes, following: bytes, expires_at: datetime) -> Session:
        """swaps the session's current refresh token for following and
        extends its expiry. presenting a superseded token revokes the
        whole family and answers SessionRevokedError: the rotated
        token dies with the replayed one. unknown tokens miss."""
        with self._lock:
            rec = self._tokens.get(current)
            if rec is None:
                raise UnknownRefreshError("unknown refresh token")
            session = self._sessions[rec.sid]
            if rec.superseded or session.revoked:
                self._sessions[rec.sid] = replace(session, revoked=True)
                raise SessionRevokedError("session revoked")
            self._tokens[current] = replace(rec, superseded=True)
            self._tokens[following] = _TokenRec(sid=session.id)
            rotated = replace(session, refresh_hash=following, expires_at=expires_at)
            self._sessions[rec.sid] = rotated
            return rotated
```

listing 419: rotate: superseded tokens revoke the family, unknown ones miss (source: python/api/pyapi/authn/session.py, lines 108-125)

The vector pair pins the whole story. Refresh at plus twenty returns a new cookie and a new token with the same session id. Replay the old cookie at plus twenty-one and both die: the replay and the successor both answer 401 session revoked, a second apart. A thief gets one use, then loses the family with the victim.

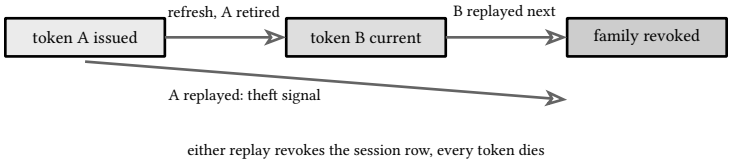


figure 140: the refresh family: rotation forks, reuse revokes everything

30.9. testing auth without walls

Every time-dependent behavior here is tested as math: the clock is one attribute moved by hand, the id source hands out the vector's uuids, and the random source is a re-armed iterator over the vector's 32 byte tokens, so the sid claim, the refresh token, and the Set-Cookie line reproduce the frozen vector byte for byte:

```
class CountingHasher(authn.Pbkdf2Hasher):
    """a strict double that counts verifies so the timing flatten is a
    measured fact, not a comment."""

    def __init__(self, **kwargs):
        super().__init__(**kwargs)
        self.verifies = 0

    def verify(self, encoded, password):
        self.verifies += 1
        return super().verify(encoded, password)
```

listing 420: a strict double that counts verifies, the flatten as a measured fact (source: python/api/tests/test_authn_routes.py, lines 37-47)

No test sleeps, no test opens a socket, and the test tier keeps the suite under a second. The request_id column of the 401 bodies is pinned through the kernel's client header resolution, so even failure envelopes compare byte for byte.

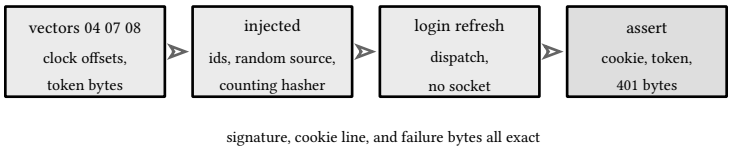


figure 141: an auth golden test: fixed inputs in, frozen bytes out

sources: docs.python.org/3/library/hashlib.html for pbkdf2_hmac, docs.python.org/3/library/hmac.html for compare_digest, docs.python.org/3/library/base64.html for urlsafe_b64encode and validate, docs.python.org/3/library/secrets.html for token_bytes, and docs.python.org/3/library/http.cookies.html for the attribute order, all accessed 2026-09-27 against the pinned cpython 3.14.7, with RFC 8018, RFC 7519, the phc format at github.com/P-H-C/phc-string-format, RFC 9106, and cheatsheetseries.owasp.com. Verified by `tests/test_authn.py` and `tests/test_authn_routes.py` under `make verify-pyapi`, 29 tests green, ruff clean at 88 columns.

31. authz and rbac

Authentication settled who is calling. Authorization decides what that caller may do, and this chapter builds the whole answer in `pyapi/authz.py`: the distinction between the two questions, roles as stored data with a grant and revoke surface, policies as plain functions a table names per route, the 401 versus 403 discipline, object-level authorization on the routes that carry ids, a deny-by-default registrar that makes an unaudited route impossible to register, and the role by endpoint matrix test that pins the whole grid in one table.

31.1. authentication versus authorization

Two questions, two layers, two failure codes. Authentication runs first and installs an actor or does not, and authorization reads that actor against the route and the object. The wiring order makes the sequence physical: the identity middleware wraps the dispatch, the policies run inside it on the actor the contextvar carries, so by the time a policy runs, the actor question is settled and a policy never parses a token, checks a signature, or reads a cookie:

```
def anyone(_actor, _values) -> Denial | None:
    """the open route policy: registration and login, the routes whose
    whole job is taking credentials from strangers."""
    return None


def authenticated(actor, _values) -> Denial | None:
    """requires a resolved actor and nothing more: the caller proved
    who they are, and the route holds no object."""
    if actor is None:
        return Denial(CODE_UNAUTHORIZED, "authentication required")
    return None
```

listing 421: the two cheapest policies: open, and any-actor (source: python/api/pyapi/authz.py, lines 34-45)

Conflating the two questions produces the classic broken answer, a 403 for an anonymous request. A caller that never proved anything has no authorization to deny, only an authentication demand, and the distinction is observable: 401 means try again with credentials, 403 means do not bother, this identity will never pass. Every layer in this chapter keeps that boundary, and the denial is data, a code and one sentence the kernel renders through the one failure writer, never a status the policy owns.

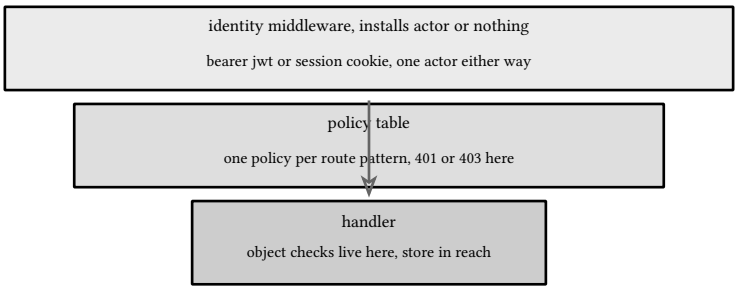


figure 142: two layers in wiring order: identity outside, policy inside, handler last

31.2. roles and the grant table

Roles are rows, not magic. Each account carries a set, the bootstrap rule grants the first registration admin, and the admin routes add and remove one role at a time, answering the resulting set. The invariant that matters is never zero admins, and it lives in the store where both mutating verbs run it, grant and revoke alike:

```
    def _admins_remain(self, candidate: User) -> None:
        """the never-zero-admins invariant on the post state, the scan
        substituting the candidate for the stored record so a rejected
        revoke leaves no trace. both verbs run it by construction."""
        for uid, user in self._by_id.items():
            if uid == candidate.id:
                user = candidate
            if ROLE_ADMIN in user.roles:
                return
        raise LastAdminError("last admin")
```

listing 422: the never-zero-admins invariant on the post state, both verbs run it (source: python/api/pyapi/users/store.py, lines 131-140)

A revoke computes the post state, checks that some admin still stands across the whole table, and only then persists, so the rejected demotion leaves no trace. The frozen vector walks the story: demoting the only admin answers 409 conflict, last admin, granting admin to a second user succeeds, and revoking that second admin then succeeds too, because the first one is standing again. The bootstrap rule makes the lockout unreachable in practice, the invariant makes it unreachable in theory. The grant route itself validates the closed set, admin or user, answering 422 for anything else.

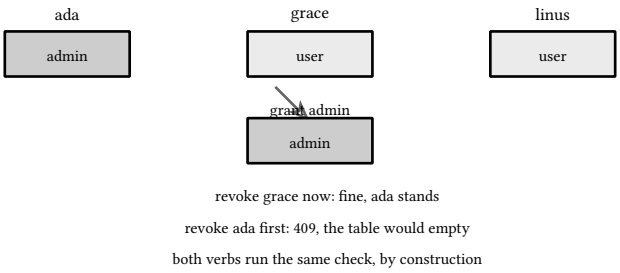


figure 143: the grant table: one column per user, the admin row must never empty

31.3. policies as functions

A policy is one function from actor plus path values to verdict: none allows, a denial denies. The whole surface composes from four of them, open, any-actor, a required role, an owner-or-admin check, and each is short enough to read in one breath:

```
def require_role(role: str) -> Policy:
    """builds the policy that demands one role. the 401 versus 403
    discipline lives here: no actor is an authentication failure, a
    present actor without the role is an authorization failure."""

    def policy(actor, _values) -> Denial | None:
        if actor is None:
            return Denial(CODE_UNAUTHORIZED, "authentication required")
        if not actor.has_role(role):
            return Denial(CODE_FORBIDDEN, "insufficient role")
        return None

    return policy
```

listing 423: the role policy: no actor is 401, the wrong actor is 403 (source: python/api/pyapi/authz.py, lines 48-61)

Plain functions beat a rules dsl or annotation mining at this size because the module type-checks the table, the audit test can walk it, and a new rule is a function next to its three siblings. The path values ride in as a dict the

router already extracted, so an object policy reads the id it needs without re-parsing anything, and the policy type alias states the whole contract in one line, actor or none and values in, denial or none out.

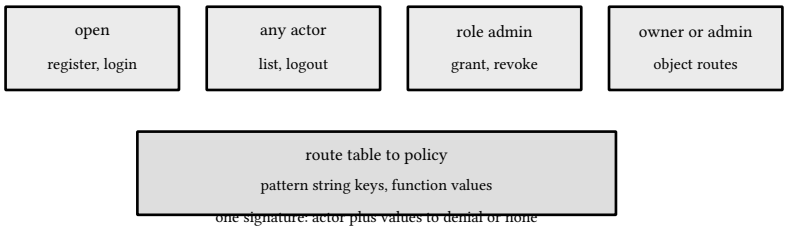


figure 144: four policies, one signature, the table maps routes onto them

31.4. 401 versus 403

The boundary lives in one place per policy: the actor check answers 401, the rule check answers 403, and nothing in between exists:

```python
def owner_or_admin(param: str = "id") -> Policy:
    """builds the object-level policy: the actor owns the path's id,
    or holds admin. this is the check whose absence the industry names
    IDOR, one route pattern at a time."""

    def policy(actor, values) -> Denial | None:
        if actor is None:
            return Denial(CODE_UNAUTHORIZED, "authentication required")
        uid = values.get(param)
        if uid is None or (actor.user_id ≠ uid and not actor.admin):
            return Denial(CODE_FORBIDDEN, "not the owner")
        return None

    return policy
```

listing 424: the object policy: own the id or hold admin, 401 before 403 (source: python/api/pyapi/authz.py, lines 63-77)

Read it as a state machine. Anonymous gets 401 with the same message every time, authentication required, no hint about the route's existence or its rules. A present actor that fails the rule gets 403 with the rule's own words, not the owner, a statement about the actor, never about the resource. The one deliberate blur is existential: this service answers 404 for an unknown id before 403 for a stranger's id, which reveals that an id exists to anyone authenticated. Hiding it instead means answering 404 for both, and the tradeoff is real, disclosure against debuggability. What is never acceptable is mixing the codes by accident, a 403 for a missing Authorization header, a 401 for a demoted admin.

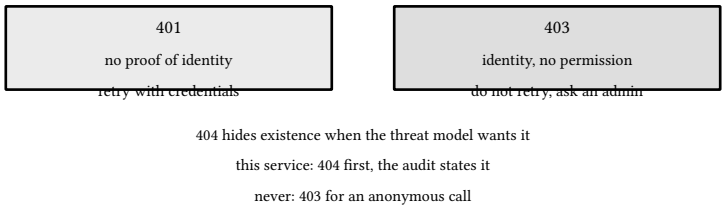


figure 145: the two failure codes, and the one deliberate blur

31.5. object-level authorization

Route rules cannot guard a route whose danger lives in the path parameter. A get on someone else's id is the IDOR class, insecure direct object reference, and it survives every route-level check because the route is the same for everyone. The check must happen after the fetch, where the code knows whether the object exists and who owns

it, which is why the object routes carry only the any-actor policy in the table and the ownership rule runs in the handler:

```
def path_param(path: str, pattern: str, param: str) -> str | None:
    """extracts the value under one {wildcard} by walking path and
    pattern in lockstep, the same segment match the router performs at
    dispatch. method-prefixed patterns are handled by dropping the
    method word."""
    if " " in pattern and not pattern.startswith("/"):
        pattern = pattern.split(" ", 1)[1]
    segments = path.split("/")
    molds = pattern.split("/")
    if len(segments) != len(molds):
        return None
    for segment, mold in zip(segments, molds):
        if mold.startswith("{") and mold.endswith("}") and len(mold) > 2:
            if mold[1:-1] == param:
                return segment
    return None
```

listing 425: the lockstep walk: path and pattern split together, the wildcard under its name (source: python/api/pyapi/authz.py, lines 79-95)

The lockstep walk is the router's own segment match restated as a function, dropping the method word and comparing segment by segment, a literal equal or a name captured. The kernel's router fills the values dict at dispatch, so this walk is the belt-and-suspenders form the tests exercise directly, and the handler-side ownership check plus the 404-before-403 ordering answer all three questions correctly: unknown id 404, stranger 403, owner or admin through. Putting the owner check in the table instead would flatten unknown and forbidden into one 403, leak existence, and duplicate the store read the handler needs anyway, so the layering is not style, it is the only arrangement that works.

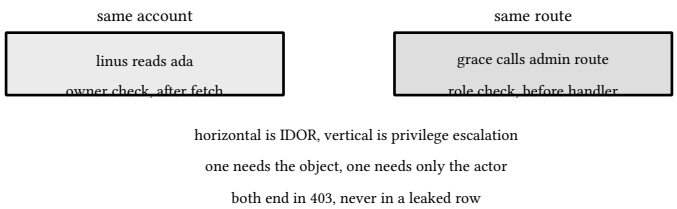


figure 146: horizontal versus vertical: two axes of privilege, two different checks

31.6. deny by default with a route audit

One table maps every registered pattern to its policy. The go lane enforced it with a middleware that re-matched every request against the router, answering 403 when a route lacked a table row. Python's dispatch already extracts the path values at routing time, so the same guarantee lands earlier and louder here: the registrar every family mounts through refuses a route with no policy at wiring time, and wraps every registered handler with its policy once the router has filled the values:

```
class Guarded:
    """the registrar every family mounts through: a route missing from
    the audit table cannot be registered at all, and every registered
    handler runs its policy once the router has filled the path
    values. an unaudited route becomes loudly unreachable at wiring
    time instead of quietly open at request time, and 404 and 405 stay
    routing answers the kernel owns, never authorization verdicts."""

    def __init__(self, app, table=None):
        self.app = app
        self.table = ROUTE_POLICIES if table is None else table

    def route(self, method, pattern, handler):
        policy = self.table.get(f"{method} {pattern}")
        if policy is None:
            raise ValueError(f"{method} {pattern} has no policy row, deny by default")
        self.app.route(method, pattern, partial(_serve_policy, policy, handler))


def _serve_policy(policy, handler, request):
    denial = policy(users.current_actor(), request.values)
    if denial is not None:
        raise kernel.fail(denial.code, denial.message)
    return handler(request)
```

listing 426: the guarded registrar: no table row, no route, the policy runs at dispatch (source: python/api/pyapi/authz.py, lines 127-150)

The audit test closes the loop from the other side: every table row must resolve to a registered route with exactly that method and pattern, and every registered route must carry a row, so a route cannot exist unaudited, a row cannot point at nothing, and a new route that forgets its policy becomes a wiring error instead of quietly shipping open. Requests that match no pattern at all fall through to the kernel fallback, because 404 and 405 are routing answers, not authorization verdicts.

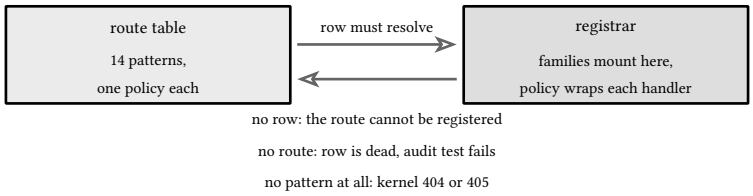


figure 147: the audit: table and registrar face each other, both directions checked

31.7. the rbac matrix test

The grid is the proof: every route times every actor kind, anonymous, regular user, admin, stranger on someone else's object, one expected status per cell, asserted in one table-driven test through the full guarded stack, the same path production traffic takes:

```
        rows = [
            ("GET", "/api/users", None, 401),
            ("GET", "/api/users", owner, 200),
            ("GET", "/api/users", stranger, 200),
            ("GET", "/api/users", admin, 200),
            ("GET", f"/api/users/{GRACE_ID}", None, 401),
            ("GET", f"/api/users/{GRACE_ID}", owner, 200),
            ("GET", f"/api/users/{GRACE_ID}", stranger, 403),
            ("GET", f"/api/users/{GRACE_ID}", admin, 200),
            ("PATCH", f"/api/users/{GRACE_ID}", None, 401),
            ("PATCH", f"/api/users/{GRACE_ID}", owner, 200),
            ("PATCH", f"/api/users/{GRACE_ID}", stranger, 403),
            ("PATCH", f"/api/users/{GRACE_ID}", admin, 200),
            ("POST", f"/api/admin/users/{GRACE_ID}/roles", None, 401),
            ("POST", f"/api/admin/users/{GRACE_ID}/roles", owner, 403),
            ("POST", f"/api/admin/users/{GRACE_ID}/roles", stranger, 403),
            ("POST", f"/api/admin/users/{GRACE_ID}/roles", admin, 200),
            ("DELETE", f"/api/users/{LINUS_ID}", None, 401),
            ("DELETE", f"/api/users/{LINUS_ID}", owner, 403),
            ("DELETE", f"/api/users/{LINUS_ID}", stranger, 204),
            ("DELETE", f"/api/users/{GRACE_ID}", admin, 204),
```

listing 427: the matrix rows: actor times route, one status each (source: python/api/tests/test_authz.py, lines 438-458)

Each row is a cell with a name that fails loudly. The grid runs the vector's last-admin chain alongside, so the 409 and the two 200s around it are pinned by frozen bytes, not just by intent, and the seeded property loop adds the population check: two hundred random actor draws against every policy, with the invariant that an admin is never denied, an anonymous caller passes only the open routes, and no actor kind ever sees a code outside the 401 and 403 pair. Adding a route costs three lines, one row per interesting actor, and the audit plus the matrix together answer the two questions reviewers actually ask: is every route covered, and does every covered route behave for every kind of caller.

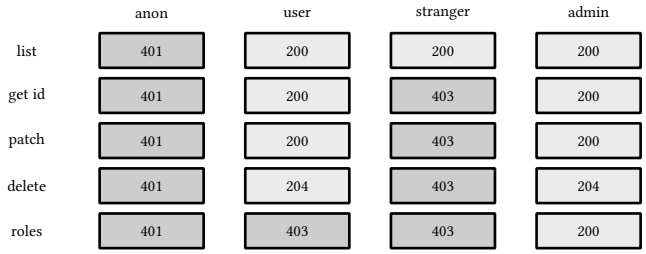


figure 148: the matrix: routes one way, actors the other, statuses in the cells

sources: OWASP, Insecure Direct Object Reference Prevention Cheat Sheet and Access Control Cheat Sheet, at cheatsheetseries.owasp.com, for the IDOR class and the deny-by-default guidance, and RFC 9110 sections 15.5.1 to 15.5.4 on 401, 403, and 404 semantics, both accessed 2026-09-27. The lockstep walk and the values dict mirror the str.split and dict pages at docs.python.org/3/library/ stdtypes.html, verified against the pinned cpython 3.14.7. Verified by `tests/test_authz.py` under `make verify-pyapi`, 22 tests green, ruff clean at 88 columns.

32. store

The service has run on in-memory dicts, which tests love and restarts hate. This lane's answer is its named infrastructure: the sqlite3 module as the pinned interpreter bundles it, the engine reporting version 3.50.4 on this machine's cpython 3.14.7, a real delta beside the corpus 3.53.4 pin for the c-built engines and the 3.53.1 node bundles, and the platform test freezes the measured number so an interpreter change is a red test before it is a surprise here. No object relational mapper sits in the graph. SQLAlchemy is dated meta in the suite chapter and nothing here needs it, because the whole job is schema migrations that append and never edit, parameterized queries over one connection per operation, the transaction idiom in its with-block form over explicit begin statements, the isolation a wal-mode sqlite actually gives, the version guard that makes lost updates visible, keyset pagination as one row-value comparison, and the window-function report the go book's engine era walked in loops instead.

32.1. db-api, the bundled engine, and the connection budget

DB-API 2.0 is a protocol, not a database. A program opens a connection by path, asks it for cursors, binds values as parameters, and reads rows back, and any module that implements the shape can sit underneath. sqlite3 is that module in the standard library, and the engine arrives inside the interpreter build, no driver to install and no third party to trust. Version 3.14 removed the module's old `version` and `version_info` constants, so `sqlite3.sqlite_version` is the one true answer about the engine, the string the platform test pins.

Three platform facts shape every line of this store. First, the legacy default opens transactions implicitly: with the default `autocommit=LEGACY_TRANSACTION_CONTROL` and a non-None `isolation_level`, a transaction is implicitly opened before any insert, update, delete, or replace, and commit is the caller's job to remember. The vehicle refuses the folklore and passes `isolation_level=None`, which leaves transaction control to explicit begin statements, so every boundary in this chapter is a line of code. The docs recommend `autocommit=False` for PEP 249 behavior and note the legacy default will change in a future release, so code that relies on implicit begins is waiting for its interpreter to break it. Second, `timeout=5.0` is the busy budget, five seconds the connection spends retrying a locked table before raising, which is why no retry loop appears anywhere in this store: the platform already wrote it. Third, a connection is not thread-safe by default and the module enforces it the hard way, `check_same_thread=True` raises ProgrammingError the moment a connection is used from the thread that did not create it, so the store borrows one fresh connection per operation and opening is cheap.

```
    def connect(self) -> sqlite3.Connection:
        """a fresh connection with the pragmas armed. wal is persistent
        in the file but re-issued idempotently, foreign keys bite per
        connection only, and isolation_level=None leaves transaction
        control to the explicit begin statements instead of the legacy
        implicit transactions python opens before dml by default: the
        boundaries stay code, not folklore."""
        conn = sqlite3.connect(
            self.path,
            timeout=self.timeout_s,
            isolation_level=None,
        )
        conn.row_factory = sqlite3.Row
        conn.execute("PRAGMA journal_mode=WAL")
        conn.execute("PRAGMA foreign_keys=ON")
        return conn
```

listing 428: the connect hub: pragmas armed, row factory set, transactions left to explicit begins (source: python/api/pyapi/store/db.py, lines 131-146)

The with-block is the trap that bites every beginner. Used as a context manager, a connection commits on success and rolls back on exception, and closes nothing: the docs say it plainly, the context manager neither implicitly opens a new transaction nor closes the connection, and point at `contextlib.closing()` for the closing scope. A test that leaves a connection open keeps the file handle alive, and on Windows the temp directory holding it fails to delete. The first drafts of this suite hit exactly that, three tests red on PermissionError until every helper wrapped its connections in closing.

The module ships no pool, unlike the ado.net provider the c# lane had to switch off. What pooling would be is a dozen lines over `queue.Queue`: build a fixed set once, hand one out under a context manager, take it back at the end, and replace a connection that broke mid-use instead of returning it to the next borrower. The pool's connections are built with `check_same_thread=False` because borrowing crosses threads by design, the docs' price for that is that write operations may need to be serialized by the user, and the answer is begin immediate plus the busy budget, the same two mechanisms every other writer here already passes through.

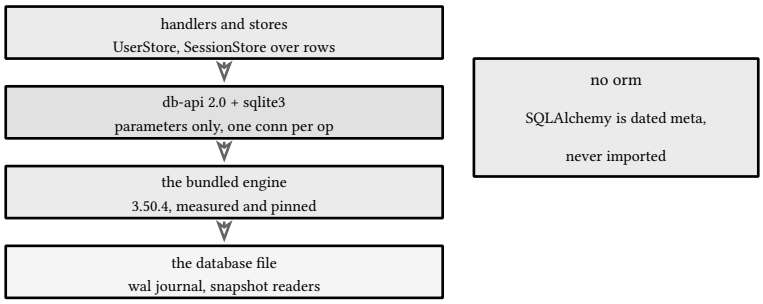


figure 149: the stack the store sits on, one borrowed connection at a time

32.2. migrations test driven

Schema changes are code changes, so they ship as records, and the history is append-only by contract: a shipped version is frozen, a correction is a new numbered version, and no record is ever edited. The registry is the user_version pragma, an integer sqlite keeps in the database header itself, so no table the database keeps about itself can drift from the schema it describes.

This wave ships four versions. Version 1 is the whole vocabulary in one record, users with their email uniqueness index, roles with the check constraint, sessions with their refresh token history, events as one presence row per user per day, reports with their idempotency claim. Version 2 adds the user version column, the substrate the optimistic concurrency sections stand on, and the keyset index beside it. Version 3 adds the idempotency keys table the concurrency chapter fills. Version 4 conforms the session vocabulary to the port the authn chapter froze, the expiry column in, the creation stamp the port never carried out, the token hash a blob, and it is its own record because a new fact is a new record even mid wave. Each applies inside one explicit transaction with the version bump in the same group, so a version that fails halfway leaves nothing durable and can simply be retried. The test drives the runner from an empty directory, proves all four records applied in order, re-runs and proves nothing applies twice, reopens the database and proves the registry came back with it, then reads sqlite_master and asserts every table and index the contract's vocabulary names.

```
    def migrate(self, conn: sqlite3.Connection) -> list[int]:
        """apply pending versions in order, each inside one explicit
        transaction with the user_version bump in the same group, so a
        version that fails halfway leaves nothing durable and can
        simply be retried. the registry lives in the database header
        itself, so no table the database keeps about itself can drift
        from the schema it describes. returns the versions applied."""
        applied: list[int] = []
        current = conn.execute("PRAGMA user_version").fetchone()[0]
        for version, statements in enumerate(MIGRATIONS, start=1):
            if version ≤ current:
                continue
            with write_tx(conn):
                for statement in statements:
                    conn.execute(statement)
                conn.execute(f"PRAGMA user_version={version}")
            applied.append(version)
        return applied
```

listing 429: one version, one transaction, the ddl and the header bump land together, applied at most once (source: python/api/pyapi/store/db.py, lines 186-203)

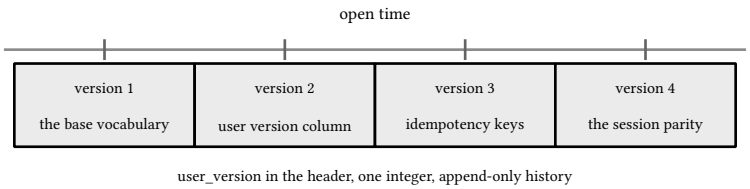


figure 150: the version list only grows right

32.3. queries with parameters every time

Values ride parameters, never string concatenation, and the module enforces the shape in this release: using a sequence of parameters with named placeholders now raises ProgrammingError, deprecated since 3.12, so the two binding styles cannot be mixed by accident anymore. Every statement in the store uses positional ? marks with a tuple, and the one f-string that appears builds a column list from a module constant, never a value. The read path pins one snapshot per group: `read_tx` begins deferred, the wal snapshot is pinned at the group's first statement and kept for its whole life, so a user and its roles are read as one state, never a torn view of an insert in flight.

The keyset walk is one row-value comparison. The index covers `(created_at_ms, id)`, the cursor decodes to that pair, and the where clause compares both columns in one tuple expression, `(created_at_ms, id) > (?, ?)`, which sqlite has served as one seek into the index since 3.15. An offset scan would pay for every skipped row on every page, the classic pagination trap, and the cursor never asks for position, only for direction:

```
    def list(self, after: CursorKey, limit: int) -> list[User]:
        """the keyset walk: at most limit users strictly after the
        cursor position, one row-value comparison against the
        (created_at, id) pair the pagination module's cursor carries,
        one seek into the index instead of an offset scan whose cost
        grows with every page. the zero cursor reads from the start."""
        with closing(self._db.connect()) as conn:
            with read_tx(conn):
                rows = conn.execute(
                    f"SELECT {_USER_COLUMNS} FROM users"
                    " WHERE (created_at_ms, id) > (?, ?)"
                    " ORDER BY created_at_ms, id LIMIT ?",
                    (after.t, after.i, limit),
                ).fetchall()
                users = [_user(row, self._roles(conn, row["id"])) for row in rows]
        return users
```

listing 430: the keyset walk: one row-value comparison against the cursor key, ascending, bounded (source: python/api/pyapi/store/users.py, lines 110-125)

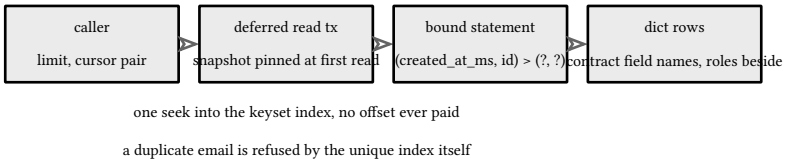


figure 151: one read's path from caller to dict rows

32.4. the transaction idiom

One shape covers every write in the service: begin immediate, run the body, and let the platform's with-block be the deferred rollback. `BEGIN IMMEDIATE` takes the writer lock up front rather than at the first write, so the busy budget is spent where the caller chose to write. The body stages whatever it likes. Landing inside `with conn:` commits the group on success, rolls it back on any exception unwinding through, and after a commit the same exit is a no-op because there is nothing left to undo. There is no finally dance and no manual rollback call on the error path, the dispose lesson of the c# lane restated as a with-block. The tests pin all three exits: a committed group lands, a body that inserts and then raises leaves nothing, and a group whose second statement violates the email index rolls back its first statement too, because groups are atomic or they are nothing. The session adapter carries the hardest case: presenting a superseded refresh token revokes the whole family inside the same write transaction and the SessionRevokedError answer is raised after the commit, so the family cannot be rolled back to life.

```
def write_tx(conn: sqlite3.Connection) -> Iterator[sqlite3.Connection]:
    """the write shape: begin immediate takes the writer lock up front
    so the busy budget is spent where the caller chose to write, and
    the platform's with-block is the deferred rollback: commit on
    success, rollback on any exception unwinding through, a no-op
    after commit because there is nothing left to undo."""
    conn.execute("BEGIN IMMEDIATE")
    with conn:
        yield conn
```

listing 431: the write shape: begin immediate, body, and the with-block is the deferred rollback (source: python/api/pyapi/store/db.py, lines 207-216)

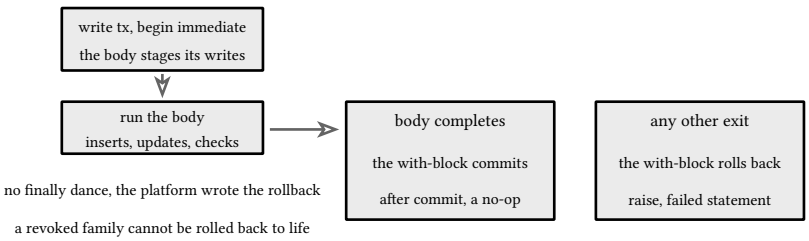


figure 152: a transaction's exits, every one of them clean

32.5. isolation levels and what sqlite actually gives you

The textbook menu, read uncommitted through serializable, is a menu of guarantees a general engine picks among, and the sqlite menu is short: everything is serializable by default, and read uncommitted only exists over a shared cache, exactly the dirty-read mode nobody wants beside a wal. There is no configuration file standing between the claim and the mechanism here, the mechanism is wal plus one writer. What that buys is snapshot isolation. A deferred transaction pins the snapshot its first statement reads and keeps it for its whole life. No dirty reads, because readers only ever see committed states. No non-repeatable reads inside one transaction, because the snapshot never moves. One writer at a time, because begin immediate takes the write lock before the first write. Write skew stays possible, two writers can act on disjoint stale snapshots and each commit cleanly, and that boundary is a fact about the locks, stated where the code states it.

The demo stays deterministic and single threaded, which is the whole trick. The reader's connection begins deferred and reads, capturing its view. A second connection commits past the pinned snapshot, and wal is what lets it, the reader holds no lock the writer needs. The reader reads again inside its transaction and still sees the old value, and a fresh read outside sees the new one. No thread, no barrier, no sleep, the interleave is the test's own statement order:

```
    def test_the_readers_view_holds_through_a_commit(self):
        users = UserStore(self.store.db)
        made = seed(users)
        query = "SELECT display_name, version FROM users WHERE id = ?"
        with (
            closing(self.store.db.connect()) as reader,
            closing(self.store.db.connect()) as writer,
        ):
            with read_tx(reader):
                first = reader.execute(query, (made.id,)).fetchone()
                with write_tx(writer):
                    writer.execute(
                        "UPDATE users SET display_name = 'renamed', version = 2"
                        " WHERE id = ?",
                        (made.id,),
                    )
                again = reader.execute(query, (made.id,)).fetchone()
            self.assertEqual(first["display_name"], "ada lovelace")
            self.assertEqual(again["version"], first["version"])
        with closing(self.store.db.connect()) as fresh:
            row = fresh.execute(query, (made.id,)).fetchone()
            self.assertEqual((row["display_name"], row["version"]), ("renamed", 2))
```

listing 432: pin a view, commit past it, read again: the interleave is the statement order (source: python/api/tests/test_store_isolation.py, lines 43-64)

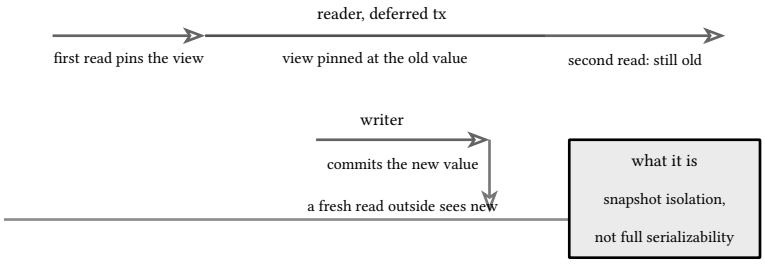


figure 153: one timeline: the reader's view holds

32.6. the lost update demo and the cas answer

Two writers read the same user, both compute, both write, and one update silently vanishes. The store reproduces it with two blind writes, update with no version guard: both report one row changed, both succeed, the second name is all that survives, and the version column never moved, so nothing anywhere records that anything was lost. Serialization does not fix this, and owning the transaction shape makes the reason visible: begin immediate guarantees each write lands whole and says nothing about the stale read the write was computed from, so the serialized blind writers merely lose the update in a tidy order.

The version column is the detector and the guarded update is the fix, the strong discipline in SQL form. The check, the bump, and the write share one statement inside one write transaction, so no second writer can slip between the read and the write. Zero rows changed is the refusal, and the classifier decides which refusal it is, missing or stale, against the known id. Of two racers holding version 1 exactly one changes a row, and the loser gets the VersionError it must handle, which over http is exactly the 412 the next chapter's duel stages with two released patchers:

```
    def update_display_name(self, uid: str, expect_version: int, name: str) -> User:
        """the guarded update, the strong discipline in sql form: the
        check, the bump, and the write share one statement inside one
        write transaction, so no second writer can slip between the
        read and the write. zero rows affected is the refusal."""
        with closing(self._db.connect()) as conn:
            with write_tx(conn):
                changed = conn.execute(
                    "UPDATE users SET display_name = ?, version = version + 1"
                    " WHERE id = ? AND version = ?",
                    (name, uid, expect_version),
                ).rowcount
                if changed != 1:
                    self._classify(conn, uid, expect_version)
                row, roles = self._read(conn, uid)
        return _user(row, roles)
```

listing 433: the guarded update: check, bump, and write in one statement, zero rows is the refusal (source: python/api/pyapi/store/users.py, lines 141-156)

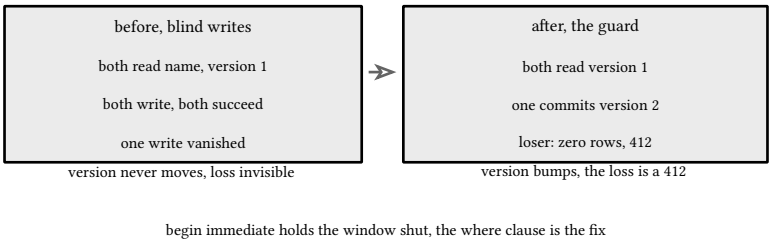


figure 154: same race, two outcomes: silent loss versus a refused writer

32.7. analytics with window functions

Reports are sql's home turf, and this lane keeps them there. Presence collapses events to one row per user per day before any counting happens, because actives count users, not actions: the events primary key is the pair (day, user_id) and the insert rides on conflict do nothing, so a duplicate event is absorbed at write time. From there the windows do the work. Row number partitioned by user in day order marks each user's first day in the window as rn = 1, which is the newcomer split. Lag carries yesterday's count forward with today's as its default, so the first day's delta is zero by construction, the same arithmetic without the self-join. Leaders rank by count descending then user id ascending, a total order, so a replayed report cannot differ run to run. Days are iso text and text comparison is chronological for iso dates, so the window bounds are two string parameters, and one report reads inside one deferred transaction, so it always sees one snapshot of the events. Over the frozen vector's seeded events, (2026-09-25, ada), (2026-09-26, ada), (2026-09-26, grace), the series is exactly the golden rows, 1 then 2, asserted against the vector file itself:

```
ROWS_SQL = """
    WITH presence AS (
        SELECT day, user_id FROM events WHERE day BETWEEN ? AND ?
    ),
    per_day AS (
        SELECT day, COUNT(*) AS active,
               SUM(CASE WHEN rn = 1 THEN 1 ELSE 0 END) AS new
        FROM (
            SELECT day, user_id,
                   ROW_NUMBER() OVER (
                       PARTITION BY user_id ORDER BY day
                   ) AS rn
            FROM presence
        )
        GROUP BY day
    )
    SELECT day, active, new,
           active - LAG(active, 1, active) OVER (ORDER BY day) AS delta
    FROM per_day
    ORDER BY day
"""
```

listing 434: the report query: presence, row number for the newcomer split, lag for the delta (source: python/api/pyapi/store/analytics.py, lines 19-39)

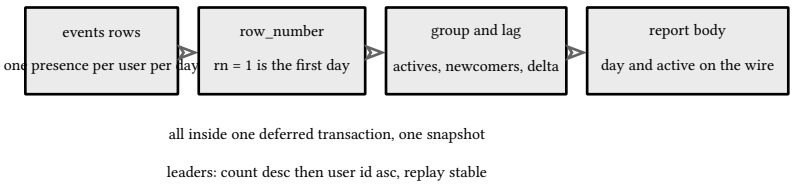


figure 155: events rows to windows to the report body

32.8. store tests without docker

Every test in the family starts the same way: a fresh temp directory, a store at a file inside it, migrated once in the fixture's constructor, act, close. No container, no port, no shared server, no ordering between tests, and the directory deletes clean because every connection is closed, the lesson the with-block section states and the first drafts of this very suite relearned. Nothing in the family sleeps: the snapshot interleave is statement order, the guarded-update duel needs no barrier at the store layer because begin immediate serializes the racers itself, and the keyset property walks seeded random users through the sql and a python-sorted twin and asserts the same pages, the two implementations that must agree. The fixtures seed through the schema itself, straight sql with bound

parameters, and the foreign keys bite the fixture that forgets its user row, which is the constraint doing its job. Fifty-two store tests across six files cover the migrations and the registry, the pragmas, the pool, the transaction exits, the isolation pair, the blind-write demo beside the guard, the user vocabulary with the cursor codec and the keyset walk, the session family, the analytics windows against the vector's own rows, and the idempotency backend's round trip and lazy expiry:

```
class TempStore:
    """the whole test story: a fresh directory, a store at a file
    inside it, migrated once, torn down with the directory."""

    def __init__(self) -> None:
        self.tmp = tempfile.TemporaryDirectory()
        self.db = StoreDb(str(Path(self.tmp.name) / "service.db"))
        with closing(self.db.connect()) as conn:
            self.db.migrate(conn)
```

listing 435: the whole test story: a temp dir, a store at a file inside it, migrated once, per test (source: python/api/tests/test_store_db.py, lines 27-35)

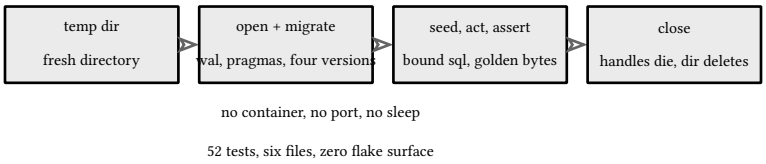


figure 156: a store test's lifecycle, nothing shared between runs

sources: docs.python.org for the sqlite3 module page, accessed 2026-09-27, the connect signature and defaults (timeout 5.0, isolation_level deferred by default, check_same_thread true, autocommit at LEGACY_TRANSACTION_CONTROL with the promised future change to False), the connection context manager section quoting that it "neither implicitly opens a new transaction nor closes the connection" and pointing at contextlib.closing, the legacy implicit begin before insert, update, delete, and replace, the autocommit attribute added in 3.12 with False recommended, sqlite3.Row as the recommended row factory, the threadsafety levels, and the user_version pragma example, the 3.14 what's new page for the removal of sqlite3.version and version_info and the ProgrammingError on sequences with named placeholders, and sqlite.org for the write-ahead logging page, all at the same date. Verified by the store family's 52 tests across the six files named above under the pyapi gate, with the engine string 3.50.4 read from `SELECT sqlite_version()` on the pinned interpreter and frozen by tests/test_platform.py before this chapter was written.

33. concurrency control

The store chapter's guarded update gave one writer the win: one commit lands, the loser learns the fact. This chapter moves that fact to the http edge and builds everything around it: strong etags over the exact response body bytes, the If-Match and If-None-Match duel two released patchers settle, idempotency keys with stored response snapshots replayed byte for byte, singleflight as a lock plus a handoff event, the synchronization primitives revisited as shapes with jobs, bounded fan-out that latches the first error, and the race canary python runs without a detector. The mechanisms live in pyapi/conc.py with injected clocks and duck-typed backends, the composition root wires them to the routes, and the frozen vectors 09 through 12 pin the etag lane against bytes this code reproduces.

33.1. the lost update prevented

Two clients read the same user, both patch, and without protection one write silently vanishes. The stack answers in three layers. The precondition decision fails fast: an If-Match that is missing, the contract chose 412 over the 428 the RFC suggests, or stale against the current representation refuses the request before the handler burns its decode. The etag itself is the sha256 of the exact body bytes, quoted the way RFC 9110 spells an entity tag, computed over the same wire representation the handlers stamp, so the decision layer and the handler can never disagree about a version. And the store's guarded update inside one write transaction is the guarantee itself, the only layer that is atomic. Fast fail, invariant, atomicity, in that order.

The duel stages the whole chain over the real store: two patchers read the row, build the etag over the body they read, both carry the same valid tag and both pass the verdict, released together by one close of a barrier, and the guarded update decides. Exactly one 200, exactly one VersionError, version 2 standing after, every run, no sleep anywhere, the barrier is the schedule:

```
        def patcher(name: str):
            row = self.users.get(self.made.id)
            etag = etag_of(row)
            # the wrapper's fast fail: both carry the same valid tag,
            # both pass, exactly as the vectors' patch pair does
            verdict = conc.if_match_verdict(etag, etag)
            barrier.wait()
            if verdict is not conc.MatchVerdict.PROCEED:
                errors.append(verdict)
                return
            try:
                updated = self.users.update_display_name(row.id, row.version, name)
                results.append((updated.version, etag_of(updated)))
            except VersionError as exc:
                errors.append(exc)
```

listing 436: the duel: one barrier releases both writers, the guarded update crowns one (source: python/api/tests/test_duel.py, lines 50-64)

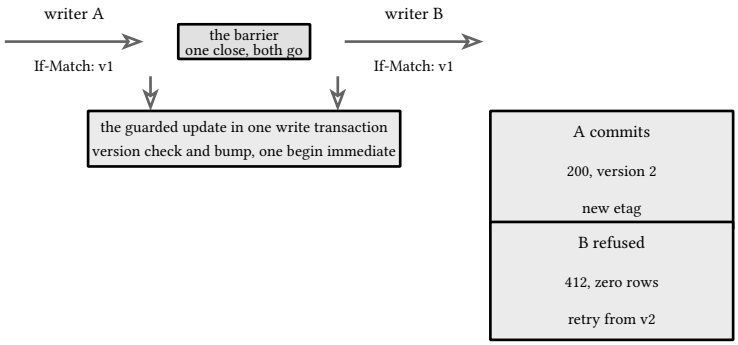


figure 157: two patchers, one barrier, one winner

33.2. if-none-match and 304

The read side of the same coin costs nothing when nothing changed. A GET carrying an If-None-Match that names the current representation answers 304 Not Modified, and the decision answers it before the handler runs: the store read and the marshal are both skipped, and the client, which already holds the body the etag names, keeps it. The response carries the etag and nothing else, because a 304 with a body would make the client cache the wrong thing for the wrong reason, the body it was told it already has.

The check is four decisions in order: no header means an ordinary read, a missing resource passes through so the handler's own 404 answers and the wrapper never invents statuses, a header that does not name the current etag is a normal read too because the client's copy is stale, and only the match answers 304. The comparison accepts the header's list form and its star form, because RFC 9110 defines both, and the frozen vector 10 pins the single-tag case byte for byte: its If-None-Match is the digest of the body vector 09 carries, which the test computes rather than copies:

```
def if_none_match_hits(header: str, current: str | None) -> bool:
    """the conditional get decision. true answers 304: the header names
    the current representation, the client already holds the body, and
    the store read and the marshal are both skipped. a missing header
    is an ordinary read, a missing resource passes through so the
    handler's own 404 answers, and a stale header is a fresh read."""
    if not header or current is None:
        return False
    return "*" in _tags(header) or current in _tags(header)
```

listing 437: the conditional get decision: no header, no resource, no match, then the 304 (source: python/api/pyapi/conc.py, lines 45-53)

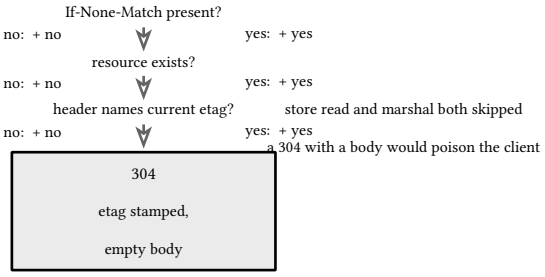


figure 158: the conditional GET decision

33.3. idempotency keys

A client that times out retries, and a register route that recreates on retry is a double-charge machine. The contract's answer is the Idempotency-Key header. The exact request bytes are hashed under sha256, whitespace drift is a different request, and the key is scoped by route and caller identity, so two clients may use the same key freely. The first successful run is stored as a snapshot, status, header set, body, and a later request with the

same key and the same hash replays those bytes at the stored status: the retried register returns the original 201, Location and ETag included, byte for byte.

The two failure paths are as deliberate. The same key under a different hash raises the mismatch, the client is misusing the contract, the caller maps it to a 422 with details, and the stored snapshot is untouched. And a failing first run is never stored, a retry after a validation error deserves a fresh run, not a replayed refusal, so only outcomes under 400 become snapshots. The backend is a protocol the guard never imports, the in-memory dict under a lock for tests, the sqlite table from the store chapter for the wired service, and both answer the same three states.

The half that is hard is concurrency. A lookup that misses, a handler that runs, a save that lands only after the handler returns: two callers under the same key inside that window both miss, both run, and the loser answers the handler's own conflict for an operation that succeeded and is recorded under its key. The guard closes the window with an in-flight table keyed by the same route, identity, and key triple the rows live under: the first caller claims the slot and answers a release on the way in, a later caller waits on that event and then reads the settled row again, so a concurrent double submit replays the 201 the first caller stored. A waiter whose row is still absent, because the first run failed or its save failed, runs fresh in the freed slot, exactly the no-op release the settle rule allows:

```
    def run(
        self,
        key: str,
        route: str,
        identity: str,
        body: bytes,
        handler: Callable[[], Snapshot],
    ) -> Snapshot:
        digest = hashlib.sha256(body).digest()
        snap = self._replay(key, route, identity, digest)
        if snap is not None:
            return snap
        triple = (key, route, identity)
        wait = self._claim(triple)
        if wait is not None:
            wait.wait()
            snap = self._replay(key, route, identity, digest)
            if snap is not None:
                return snap
            return self._first_run(key, route, identity, digest, handler)
        try:
            return self._first_run(key, route, identity, digest, handler)
        finally:
            self._release(triple)
```

listing 438: run: replay a match, wait on the in-flight handoff, settle and release (source: python/api/pyapi/conc.py, lines 180-203)

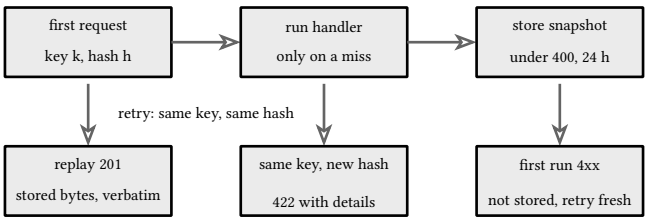


figure 159: one key's life: first run, snapshot, replay

33.4. singleflight

When a hot key expires, every request for it computes the same expensive thing at once, the thundering herd, and the origin pays for n identical reads. The loader collapses the herd: the first caller for a key runs the function, every concurrent caller with the same key parks on that run's handoff event and receives its settled outcome. The entry is forgotten the moment the run settles, which is the property that keeps this honest: the herd that arrives after the lull fills again. A collapse tool, not a cache, and the distinction is a design decision, not a limitation, the next chapter puts a cache in front of this loader and gets both behaviors explicitly.

The whole mechanism is one lock plus one dict plus one event per flight, built from the platform's own primitives the way the go lane decomposed the x/sync group it wrapped. Registration is synchronous under the lock: once claim returns, the caller has provably joined an in-flight run or started the only one, because the map lookup and the entry creation share one critical section, so joining has no scheduling window, and the collapse test leans on exactly that. The settle path removes the entry from the map before it sets the event, so a caller arriving after the settle starts a new flight, never joins a dead one:

```
    def claim(self, key: str) -> tuple[Flight, bool]:
        """register synchronously. once claim returns, the caller has
        provably joined an in-flight run or started the only one: the
        map lookup and the entry creation share one critical section,
        so joining has no scheduling window, which is the property the
        collapse tests lean on."""
        with self._lock:
            flight = self._flights.get(key)
            if flight is not None:
                flight._waiters += 1
                return flight, False
            flight = Flight()
            self._flights[key] = flight
            return flight, True
```

listing 439: claim: the registration seam, one critical section for lookup and creation (source: python/api/pyapi/conc.py, lines 290-303)

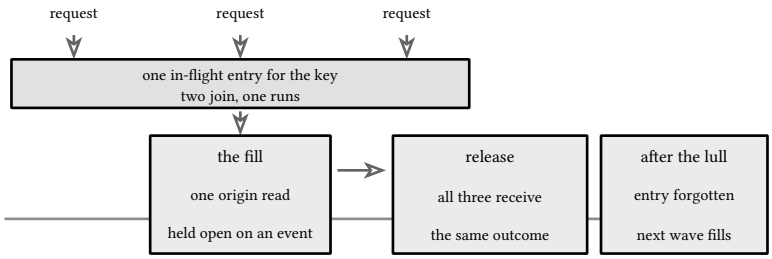


figure 160: three requests arrive together, one origin read happens

33.5. the primitives revisited

The threading family is small and each shape has one job. Lock is the whole story in two methods, shared mutable state, one lock, nothing clever between them, and Counter is that story in code. RLock is the reentrant twin for when one lock-holder must re-enter, the price is a count the unlock must drain. Event is a one-shot signal any number of waiters can park on, the loader's handoff and the canary's gates are both it. Barrier is the rendezvous, all parties arrive or nobody proceeds, and the duel releases through one. Semaphore and its bounded twin are admission tokens, capacity waiting slots, release gives one back, and the bounded form refuses to release what was never taken. Condition is the shape the others compose into when a state change must wake selected waiters, lock plus wait plus notify, and it is also the recipe for the one lock the stdlib does not ship, there is no read-write lock in threading, unlike the go and c# platforms this book mirrors, and the honest statement is that cpython's own dict does not need one because the interpreter's lock is the reader lock.

Under all of them the same honesty the go chapter stated: a lock at the wrong altitude loses updates while holding everything tidy, and free-threaded builds, officially supported in 3.14 under PEP 779, make the question sharper, not new. The canary at the end of this chapter exists to keep the claim tested:

```python
class Counter:
    """the whole lock story in two methods: shared mutable state, one
    lock, nothing clever between them. the canary's guarded twin is
    this shape with the interleave forced around it."""

    def __init__(self, start: int = 0) -> None:
        self._lock = threading.Lock()
        self._value = start

    def add(self, n: int) -> None:
        with self._lock:
            self._value += n

    @property
    def value(self) -> int:
        with self._lock:
            return self._value
```

listing 440: the lock story in two methods: shared state, one lock (source: python/api/pyapi/conc.py, lines 336-354)

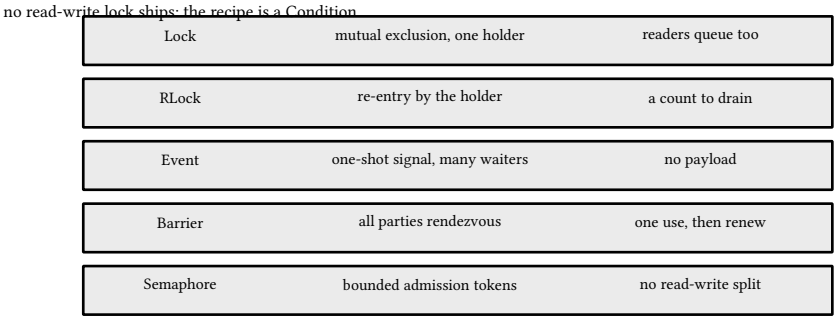


figure 161: each primitive, the guarantee it buys

33.6. bounded fan-out with first error

Spawn a thread per item and the size of your input becomes the size of your origin's worst day: the queue you built to protect it becomes the stampede you meant to prevent. The valve is a worker pool over `queue.Queue`, at most limit workers, each pulling items until it draws a sentinel. The first exception is latched under a lock, later errors cannot overwrite it, the latch trips a cancel event every worker checks before starting a task, so work still queued when the first error lands is skipped, and the pool raises the latched error after it drains, the one worth acting on, never the last cancel noise to land.

This is the errgroup of the go lane rebuilt from the platform's own parts, a pool plus a latched first error plus a cancel, and the platform's packaged spelling of the same thing is `concurrent.futures.ThreadPoolExecutor` with `max_workers=limit`, stdlib surface that adds futures and joins on top of exactly this loop. The chapter builds it from the primitives it teaches because reading the loop is the lesson. The tests pin both promises deterministically: the bound holds, the tasks in flight observed together at a barrier of size limit exactly limit, and with a single worker the first error is the raised one and every item still queued is provably skipped:

```
    def worker() -> None:
        while True:
            item = pending.get()
            try:
                if item is _STOP:
                    return
                if cancel.is_set():
                    continue
                try:
                    work(item)
                except BaseException as exc:
                    with latch:
                        if not first:
                            first.append(exc)
                    cancel.set()
            finally:
                pending.task_done()
```

listing 441: the worker: pull, skip when cancelled, latch the first error, always task done (source: python/api/pyapi/conc.py, lines 377-393)

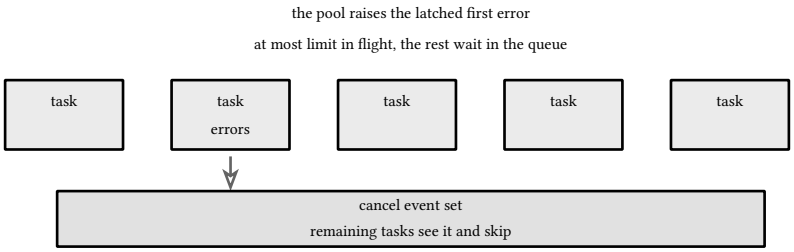


figure 162: the first error travels left through the cancel event

33.7. the race canary without a detector

The go lane's canary leaned on a race detector that cpython does not ship, so the python canary manufactures the interleaving instead. Its broken half is a read-modify-write with no lock, two writers forced into the losing order by a barrier: both read the cell, both meet at the barrier, both write what they read plus one, and the final value is old plus one, one increment gone. The interleave is not a timing hope, the barrier is the schedule, so the lost update is a deterministic fact about the code, exactly what a detector would report and what a test can assert. The guarded twin is the same two writers through Counter's lock and it keeps both.

The trap the canary guards against is the comfortable one: the gil makes byte-granular tears rare, so an unlocked counter can pass a thousand runs and fail the week it meets a free-threaded build or an interleave that lands between two bytecodes. Rare is not absent, and 3.14 ships free-threading as officially supported, so the discipline is the lock, not the luck. Delete the lock in the guarded twin and the deterministic broken half still passes its assertions while the twin fails, which is the demo the canary exists to keep honest:

```
    def unsynchronized_writers(self) -> int:
        cell = [0]
        both_read = threading.Barrier(2)

        def writer():
            current = cell[0]
            both_read.wait()  # both writers now hold the same stale read
            cell[0] = current + 1

        threads = [threading.Thread(target=writer) for _ in range(2)]
        for thread in threads:
            thread.start()
        for thread in threads:
            thread.join()
        return cell[0]
```

listing 442: the deliberately broken half: both read, both wait, both write what they read (source: python/api/tests/test_conc.py, lines 468-482)

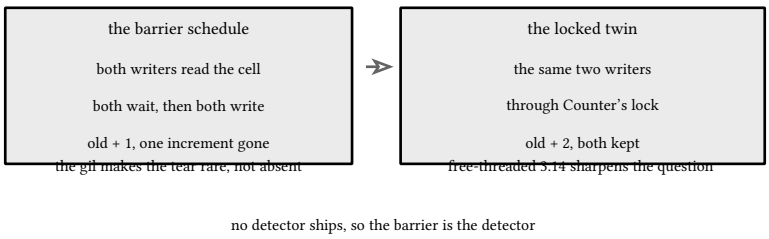


figure 163: the canary's two verdicts, one schedule

sources: docs.python.org for the threading module page, accessed 2026-09-27, the Lock, RLock, Event, Barrier, Semaphore, BoundedSemaphore, and Condition entries read for their guarantees, the queue module page for Queue's FIFO and sentinel idioms, the hashlib page for sha256 and hexdigest, and the 3.14 what's new page for threading.Thread.start setting the operating system thread name (gh-59705) and PEP 779 making free-threaded builds officially supported, the If-Match and If-None-Match semantics read from RFC 9110 sections 13.1.1 and 13.1.2 at rfc-editor.org at the same date. Verified by the conc family's 25 tests in tests/test_conc.py plus the two duel tests in tests/test_duel.py, under the pyapi gate, with the etag lane pinned against the frozen vectors 09 through 12 by computing their digests rather than copying them.

34. caching

A cache is a promise to answer from a copy, and every promise has a price. This chapter prices it honestly: what caching promises and what it silently trades away, the http semantics a private api owes its clients, the LRU from first principles over a dict and a list with the platform's fused OrderedDict spelling proven equivalent beside it, TTL beside LRU so age evicts as well as crowding, the stampede guard that rides the previous chapter's loader, invalidation by delete after write, and the discipline of what never gets cached at all. Everything lives in pyapi/cache.py over an injected clock, and the stdlib ships no cache module, functools's memoization being the nearest kin and not one, so every line of the structure is owned and stated.

34.1. what caching promises

A cache answers from a copy to save the origin the work, and both sides of the bargain matter: the copy must be cheap to find, and the answer must be one the origin would still give. Every caching decision is a bet that the read will be asked again before it changes, and the three terms of the bet are capacity, how many copies, lifetime, how long each stays, and invalidation, what a write owes the copies it made wrong. Miss any term and the cache stops being an optimization and becomes a second source of truth, which is the failure mode every caching chapter exists to prevent.

This service's copy lives in one layer, a bounded store of responses in front of the route table, constructed with its three terms up front and an injected clock so every one of them is testable as arithmetic rather than waiting. The clock is a callable returning epoch milliseconds, the same convention the token bucket and the idempotency window use, and no test in the family sleeps:

```
class LruCache:
    """least recently used over one dict and one list, from scratch:
    the dict finds any key in constant time, the list carries the
    recency order with the coldest key at index 0, and the two agree
    because every entry lives in both. a read moves its key to the hot
    end with a list scan, and an eviction pops index 0, both linear in
    the recency dimension: the honest cost of the from-scratch
    spelling, stated where the code pays it. fine at book scale, and
    the fused spelling beside this class is the upgrade path."""

    def __init__(self, max_entries: int, ttl_ms: int, now_ms: Callable[[], int]):
        self._lock = threading.Lock()
        self._max = max_entries
        self._ttl_ms = ttl_ms
        self._now_ms = now_ms
        self._items: dict[str, tuple[Any, int]] = {}
        self._order: list[str] = []
```

listing 443: the layer's substrate: capacity, lifetime, and a clock, stated at construction (source: python/api/pyapi/cache.py, lines 18-34)

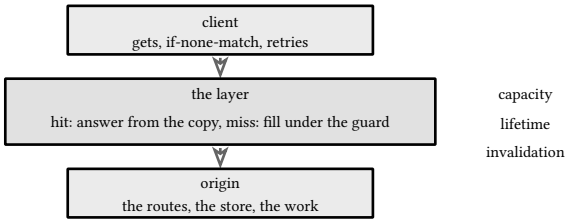


figure 164: the layer sits between client and origin, answering what it can

34.2. http caching semantics for private apis

A private api's responses are personalized and credentialed, so the shared-cache machinery of the web mostly does not apply, but the headers still owe clients a contract. Cache-Control states what may be stored and for how long, and private keeps shared proxies out of it, these bodies vary by who is asking. Vary names the request headers the representation depends on, here the bearer token and the session cookie, which is precisely the pair the cache key carries so two identities never share an entry. no-store means never, not even for a moment, and it is the default.

The reads are revalidation business. max-age=0 with must-revalidate tells the client nothing is fresh for long, every reuse round-trips its etag through the 304 lane the previous chapter built, and the layer's own copy is short-lived and keyed by the vary values, a latency optimization behind the protocol, never a replacement for it. The table is deny by default: a route is cacheable only when a row says so, and no other file in the service can make one cacheable by accident. The two cacheable rows are the user reads, the collection and the single resource, and everything else, every write route and every credential route, carries the no-store posture the exclusion section enforces:

```
POLICY: dict[str, Policy] = {
    "GET /api/users": Policy(
        "private, max-age=0, must-revalidate", ("Authorization", "Cookie"), True
    ),
    "GET /api/users/{id}": Policy(
        "private, max-age=0, must-revalidate", ("Authorization", "Cookie"), True
    ),
}

# the default posture for every route the table does not name: a
# private api caches nothing unless a row says otherwise
NO_STORE = "private, no-store"
```

listing 444: the per-route policy: two cacheable reads, everything else no-store by default (source: python/api/pyapi/cache.py, lines 198-209)

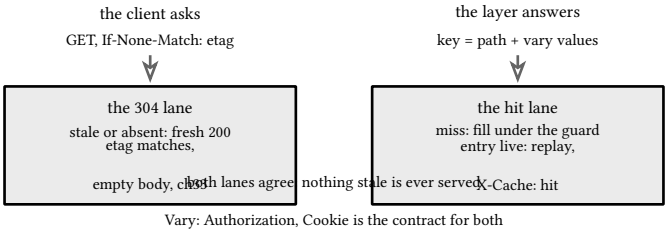


figure 165: two freshness decisions: the client's and the layer's

34.3. lru from first principles

Least recently used is one dict and one list, from scratch. The dict finds any key in constant time, the list carries the recency order with the coldest key at index 0, and the two agree because every entry lives in both. A read moves its key to the hot end, a write inserts at the hot end, and when capacity is exceeded the key at index 0 is the eviction, removed from both structures.

The honest cost of the from-scratch spelling is linear time in the recency dimension: `list.remove` scans to find the key and `pop(0)` shifts the tail, both fine at book scale and both stated where the code pays them. The platform's fused spelling exists because of exactly this: the collections documentation says OrderedDict "was designed to be good at reordering operations" while the plain dict "was designed to be very good at mapping operations", that its algorithm "can handle frequent reordering operations better than dict", which "makes it suitable for implementing various kinds of LRU caches", and that a plain dict "does not have an efficient equivalent" for moving a key to the cold end. `move_to_end` is the read's reorder in constant time and `popitem(last=False)` is the coldest eviction,

so FusedLru is the same structure in one object. The parity test proves the two interchangeable over seeded op streams, the same answers, the same count, the same recency order, step after step, which is what makes either one safe to grow from. The proof of eviction is the scramble: fill to capacity, touch a key from the middle, and the next insert evicts the true least recently used entry, not the oldest insert:

```
    def get(self, key: str) -> tuple[Any, bool]:
        """the live value and true, or anything and false. an expired
        entry is removed on the spot and answers a miss: expiry is
        lazy, every read is its own eviction decision."""
        with self._lock:
            entry = self._items.get(key)
            if entry is None:
                return None, False
            value, expires_ms = entry
            if self._now_ms() ≥ expires_ms:
                self._order.remove(key)
                del self._items[key]
                return None, False
            self._order.remove(key)
            self._order.append(key)
            return value, True
```

listing 445: the pairing in the from-scratch spelling: the dict finds, the list orders, the scan is the stated cost (source: python/api/pyapi/cache.py, lines 36-51)

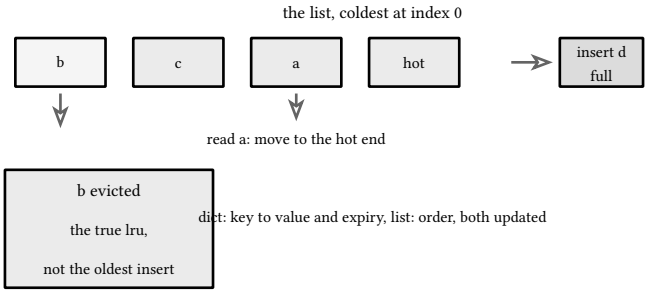


figure 166: a read moves its key to the hot end, an insert evicts index 0

34.4. ttl beside lru

LRU answers crowding, not age. An entry nobody reads can be least recently used forever and still hold its bytes, and an entry everybody reads can be hot long after the origin moved on. The TTL gives every entry an expiry stamped from the injected clock at write time, and reads check it lazily: an expired entry evicts itself on contact, so every read is its own eviction decision and no sweep is needed for correctness. The lazy branch lives in both spellings, and the fused one shows it in the least code:

The sweep is for memory, not correctness. It walks the entries once and drops whatever aged out unread, so a key the traffic forgot cannot squat on its bytes forever, and because it is nothing but the injected clock and a delete loop, the test for a whole simulated hour is a loop that steps the clock by hand and counts what fell out, no thread, no timer, no sleep. The wiring may call it from a daemon loop if it wants a janitor, and the chapter states the trade plainly: the janitor saves memory between reads, it never saves correctness, because the lazy branch already owns that:

```
    def get(self, key: str) -> tuple[Any, bool]:
        with self._lock:
            entry = self._items.get(key)
            if entry is None:
                return None, False
            value, expires_ms = entry
            if self._now_ms() ≥ expires_ms:
                del self._items[key]
                return None, False
            self._items.move_to_end(key)
            return value, True
```

listing 446: the fused read: lazy expiry on contact, then move_to_end (source: python/api/pyapi/cache.py, lines 117-127)

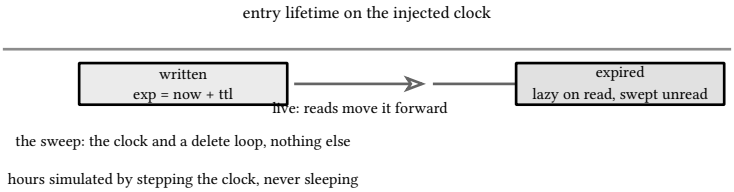


figure 167: two clocks push entries out: age and crowding

34.5. the stampede and its guard

Every entry expires eventually, and the moment a hot one does, the herd arrives together: a hundred concurrent misses, a hundred identical origin reads, the exact load spike the cache existed to prevent, arriving at the worst moment, the cold one. The guard is the previous chapter's loader worn one layer down: a miss fills through the loader, the herd collapses into one origin read, every waiter receives the same settled outcome, and that outcome is the only thing that ever writes the cache entry.

The collapse test holds the loader's synchronous registration exactly the way the conc chapter's does: three callers claim the key, one owns the fill and holds it open on an event, two join, one fill happens, and all three answers are the same value. And errors do not land in the cache, a failed fill raises having stored nothing, so an origin outage cannot be memoized into a sticky failure, and the next caller after the outage fills again:

```
def get_or_fill(
    cache: LruCache | FusedLru,
    loader: conc.Loader,
    key: str,
    fill: Callable[[], Any],
) -> Any:
    """the read-through form: a live cache entry answers directly, a
    miss fills under the loader so the herd collapses into one origin
    read, and only the settled value writes the entry. a failed fill
    raises and lands nothing, so an origin outage cannot be memoized
    into a sticky failure."""
    value, found = cache.get(key)
    if found:
        return value
    value, _shared = loader.run(key, fill)
    cache.set(key, value)
    return value
```

listing 447: the read-through form: a live entry answers, a miss fills under the loader (source: python/api/pyapi/cache.py, lines 162-178)

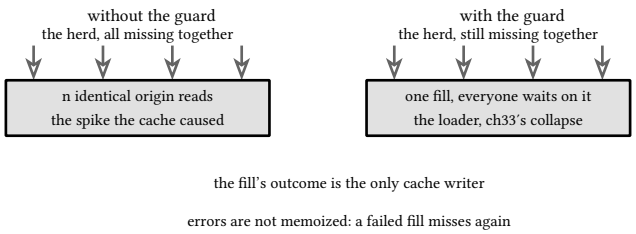


figure 168: the same expiry, with and without the guard

34.6. invalidation on write

A write changes the origin, and the copies of what it changed are now wrong. The discipline is delete after write, never update: the writer drops the affected entries and lets the next read refill them through the guard. Deleting is idempotent and always safe, and the refill is the single writer the cache already trusts. Updating instead, pushing the writer's view into the entry, sounds cheaper, one write instead of a miss, but the writer does not know what a concurrent reader cached mid-flight, and an update that lands just before a stale read's own store leaves the wrong value with a fresh timestamp, the worst of both worlds. The staged test runs two layers through the same write, one updated by push, one deleted and refilled, pushes the origin twice, and only the deleted one converges on the truth both times.

The rule is path plus collection: a write to a row invalidates the row and the list that reads it, a write to a collection invalidates the list and any children under it, and nothing else. The key carries the path first, so the drop is one filtered pass, and a failed write invalidates nothing because the origin never moved:

```
    def invalidate(self, path: str) -> int:
        """delete after write, never update: a successful write to a
        path drops the entries for that path, for its parent
        collection, and for any child under it, and nothing else. the
        writer does not know what a concurrent reader cached
        mid-flight, deleting is idempotent and always safe, and the
        refill is the single writer the cache already trusts."""
        parent = path.rsplit("/", 1)[0] if "/" in path[1:] else ""
        return self._cache.delete_where(
            lambda key: (
                _key_path(key) not in (path, parent)
                and not _key_path(key).startswith(path + "/")
            )
        )
```

listing 448: delete after write: the path, its collection, its children, nothing else (source: python/api/pyapi/cache.py, lines 308-321)

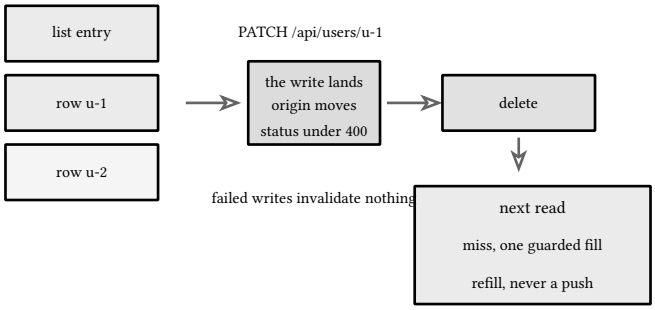


figure 169: write to a row: the row and the list go, the refill is the only writer

34.7. what not to cache

The last decision is the one that prevents the incidents. Credential routes, login and refresh, carry secrets and must never be replayable from a copy. Rate-limited responses, the 429 lane, must always reach the client, because a cached budget answer is a lie about money. Personalized bodies without a vary key to separate identities are a cross-user leak waiting for traffic. Anything the contract marks no-store is a legal promise, not a hint. The layer's policy table is deny by default, cacheable only where a row says so, which makes the exclusion list the empty set of accidents: a new route opts in or it does not get cached, and no change elsewhere in the service can flip that. The test pins the posture route by route, every write route and every credential route absent from the table, and the layer refuses to store a snapshot for a route with no row, so the day someone adds a cacheable row to the wrong route, this is the test that says which lane they put it in:

```
    def test_the_table_is_deny_by_default(self):
        for route in (
            "POST /api/users",
            "POST /api/auth/login",
            "POST /api/auth/refresh",
            "POST /api/auth/logout",
            "PATCH /api/users/{id}",
            "DELETE /api/users/{id}",
            "GET /metrics",
        ):
            with self.subTest(route=route):
                row = cache.policy_for(route)
                self.assertFalse(row.cacheable)
                self.assertEqual(row.control, cache.NO_STORE)
```

listing 449: the exclusion posture: deny by default, route by route (source: python/api/tests/test_cache.py, lines 328-341)

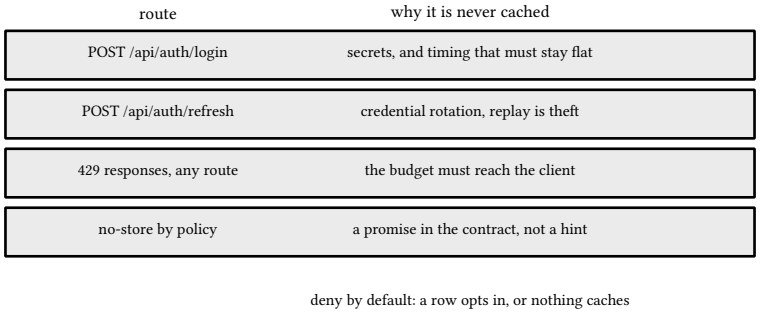


figure 170: the exclusion list, each row with its reason

sources: docs.python.org for the collections page, accessed 2026-09-27, the OrderedDict entry quoting that it "was designed to be good at reordering operations" beside the plain dict "designed to be very good at mapping operations", that its algorithm "can handle frequent reordering operations better than dict" and "makes it suitable for implementing various kinds of LRU caches", with move_to_end and popitem(last=False) read from the same entry, the threading page for the lock the structures share, the 3.14 library index for the absence of any cache module in the standard library, and the Cache-Control, Vary, and no-store semantics read from RFC 9111 section 5.2 at rfc-editor.org at the same date. Verified by the cache family's 19 tests in tests/test_cache.py under the pyapi gate, with the parity loop running 20 seeded trials of 400 operations each over both spellings and the stampede collapse pinned through the loader's synchronous claim seam from the previous chapter.

35. rate limiting

A service that answers every request it receives eventually answers none of them: password guessing arrives as a flood of logins, a runaway client loop arrives as the same request ten thousand times. This chapter builds the service's rate limiting in `pyapi/limit.py`. The standard library ships no rate limiter, so the token bucket built here over an injected clock is the production gate, not a teaching mock, with `threading.BoundedSemaphore` and `queue.Queue` decomposed along the way as the admission and handoff primitives they are, and the 429 contract, the keying, and the sleepless test discipline close the chapter.

35.1. why limits exist

Capacity is finite and failure is nonlinear. Past some offered load a service does not degrade gently, it falls off a cliff: queues grow, latency climbs past every client timeout, memory fills with requests waiting on answers. A limit is the refusal to stand on that cliff. It converts an overload that would take the whole service down into a fast, honest rejection of the excess, and the client that honors it gets a working service at one request per second instead of a dead one at zero.

The two limited routes carry numbers chosen for what they protect. Login allows a burst of five per address, then one per second, because every login attempt pays the pbkdf2 verification budget, 210000 iterations of hmac-sha256, and a credential guesser does not need more than five tries to be dangerous. Register allows ten per client ip for the same reason plus its idempotency bookkeeping. Both refill one token per second, so a tripped client is back to full capacity five and ten seconds later, the difference between a limit and a lockout:

```
# the contract's two limited routes, verbatim numbers: login bursts 5
# per lower(email), register bursts 10 per client ip, both refill one
# token per second
LOGIN_BURST = 5
REGISTER_BURST = 10
REFILL_PER_SECOND = 1.0
LOGIN_MESSAGE = "too many login attempts"
REGISTER_MESSAGE = "too many registration attempts"
# a key's bucket retires after this much disuse, in seconds
DEFAULT_IDLE_TTL = 600.0
# the kernel's body cap from the wire contract, 1 MiB. the login key
# reads at most this much of a body, mirroring the go lane's bounded
# reader; this copy stands until the composition shares one source
MAX_KEY_BODY_BYTES = 1024 * 1024
```

listing 450: the contract's two rules as one constant block, numbers and messages (source: python/api/pyapi/limit.py, lines 29-42)

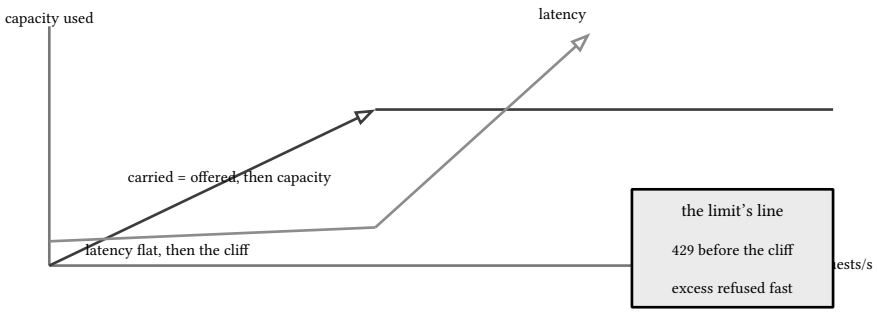


figure 171: offered load versus what the service can carry, the cliff between them

35.2. the token bucket from first principles

A token bucket holds a float, a capacity, and a refill rate. It starts full, every allowed request removes one token, and time adds tokens continuously at the configured rate until the capacity caps them. The two knobs split the two jobs cleanly: capacity is the burst allowance, how much may arrive at once, and the refill rate is the sustained allowance, how much may arrive per second forever. The login rule, capacity five at one per second, permits five in the first instant, a sixth at the one second mark, then one per second forever.

The arithmetic of a denial is the part worth deriving rather than reciting. After the sixth same-second draw the level is exactly zero, so the next token exists one full second in the future, and that instant is both the reset header and the retry hint. Half a second later the level is 0.5, still not spendable, and the next whole token only half a second away: a whole level waits a full period, a fraction waits its complement, and a denial spends nothing:

```
    def allow(self, now):
        """draw one token at now. allowed costs exactly one token; a
        denial costs nothing and reports the wait for the level to
        reach the next whole token, the earliest the caller may
        retry."""
        self._refill(now)
        allowed = self._level ≥ 1
        if allowed:
            self._level -= 1
        remaining = int(self._level)
        wait = self._next_token_in()
        retry_after = 0.0 if allowed else wait
        return Decision(
            allowed, int(self._capacity), remaining, now + wait, retry_after
        )

    def _next_token_in(self):
        # a whole level waits a full period, a fraction waits its
        # complement: 1 - frac(level) over the refill rate
        frac = self._level - int(self._level)
        return (1 - frac) / self._per_second
```

listing 451: Allow: one draw, one decision, and the wait computed from the fractional level (source: python/api/pyapi/limit.py, lines 87-107)

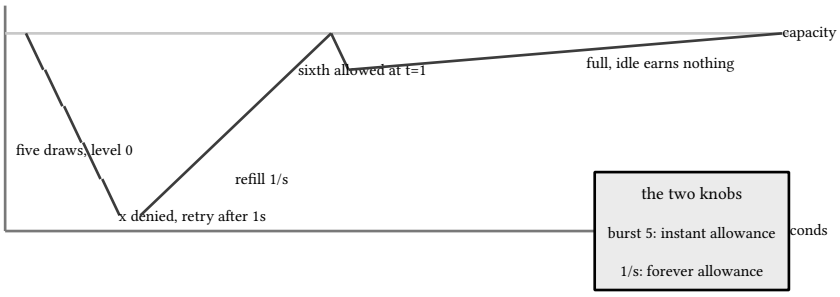


figure 172: bucket level over one draw timeline, the refill climbing between bursts

35.3. the stdlib has no rate limiter

Said plainly: the python standard library has no rate limiting anywhere in it, no token bucket, no sliding window, no throttling helper. The go lane reached for `golang.org/x/time/rate` and the c sharp lane for `System.Threading.RateLimiting`'s `TokenBucketRateLimiter`, both platform-official, and this lane has no counterpart to reach for, so the bucket from the previous section ships. What the stdlib does ship is the two primitives a limiter is built from.

`threading.BoundedSemaphore` is capacity without time: whole slots, a fixed count, and `acquire(blocking=False)` is an immediate yes or no, the exact admission move a gate needs. The docs give the bound check in one line, releasing past the initial value raises `ValueError`, and that loudness is a feature: an unbalanced release is a bug, not a slot to invent. `queue.Queue` is a handoff without a rate: the docs open with "The `queue` module implements multi-producer, multi-consumer queues", locking semantics included, and its `maxsize` blocks the producer when full, which is backpressure, not refusal. A semaphore counts what is inside, a queue moves work between threads, and neither knows what the clock said. The token bucket is the third structure: the semaphore's capacity plus the clock, as a float level time refills.

The production gate wraps that bucket in a map policy: one bucket per key, created on first use, retired after an idle ttl, with the sweep amortized into the request path, every `idle_ttl` of clock time a request sweeps inside the lock it already holds, which deletes the sweeper thread entirely. The lock is one `threading.Lock` held for the lookup and the draw, because ThreadingHTTPServer dispatches each request on its own thread and python threads interleave at every bytecode, so the map and the levels share one mutex the way the go lane's limiter shares one:

```
    def _entry_for(self, rule, key, now):
        if now - self._last_sweep ≥ self._idle_ttl:
            self._sweep_locked(now)
        entry = self._buckets.get(key)
        if entry is None:
            entry = _Entry(Bucket(rule.burst, rule.per_second, now), now)
            self._buckets[key] = entry
        entry.last = now
        return entry

    def _sweep_locked(self, now):
        retired = 0
        for key in list(self._buckets):
            if now - self._buckets[key].last ≥ self._idle_ttl:
                del self._buckets[key]
                retired += 1
        self._last_sweep = now
        return retired
```

listing 452: per-key buckets, created on first use, swept on the lock the path already holds (source: python/api/pyapi/limit.py, lines 235-252)

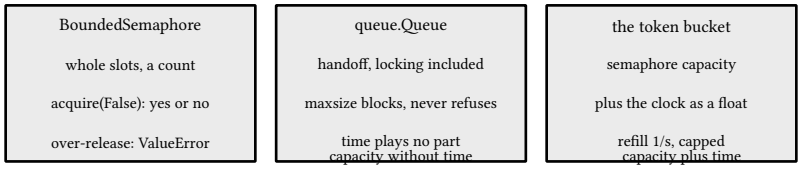


figure 173: semaphore, queue, bucket: what each primitive actually carries

35.4. the 429 contract

The wire shape is pinned by the contract and the frozen vector, and the layer writes it in one place. Every response from a limited route carries three headers: `X-RateLimit-Limit` is the burst capacity, `X-RateLimit-Remaining` is whole tokens left after this draw, `X-RateLimit-Reset` is the unix second at which the next token arrives. A denial adds `Retry-After` in seconds, the ceiling of the wait, and answers the standard envelope with code `rate_limited` through the kernel's one failure writer, never a bare status or a bespoke body, so the request id in the denial matches every other signal of the request.

The sixth login vector pins every byte. The clock sits at the frozen `t0` plus twelve seconds, five earlier logins at the same instant left the bucket reporting remaining 4, 3, 2, 1, 0, and the sixth draw finds an empty bucket: status 429, limit 5, remaining 0, reset 1790830813, exactly `t0` plus thirteen seconds, retry after 1, and the envelope's message

the frozen `too many login attempts`. The layer writes the three headers before the inner handler runs, so they ride on successes too:

```
def limit_layer(gate):
    """the gate as one middleware layer: limited routes carry the
    three rate headers on every response, allowed or denied, and a
    denial answers the contract's rate_limited envelope with
    Retry-After through the kernel's one failure writer, the same
    request id every other signal of this request carries."""

    def layer(next_handler):
        def handler(request):
            route = f"{request.method} {request.path}"
            rule = gate.rule_for(route)
            if rule is None:
                return next_handler(request)
            decision = gate.allow(route, rule.key(request))
            headers = rate_headers(decision)
            if not decision.allowed:
                error = fail(RATE_LIMITED, rule.message)
                error.headers.update(headers)
                return failure(request, error)
            response = next_handler(request)
            response.headers.update(headers)
            return response

        return handler

    return layer
```

listing 453: the layer: three headers always, Retry-After and the envelope on denial (source: python/api/pyapi/limit.py, lines 255-280)

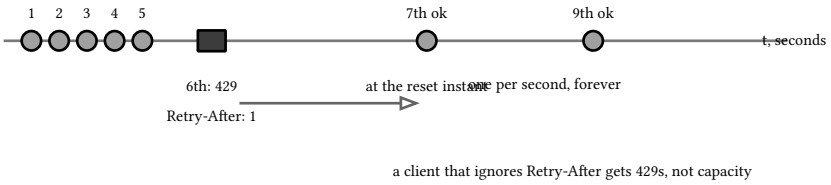


figure 174: a client that honors Retry-After: five through, one refused, the next at the reset instant

35.5. what to key on

A limit is only as good as its key, and the two routes key on different things for opposite reasons. Login keys on `lower(email)`: the attack the rule exists for is guessing one account's password, so the bucket belongs to the account, and the casing normalization means `ADA@Example.ORG` and `ada@example.org` are one bucket. The key lives in the request body, which the kernel has already read into `Request.body` as bytes, so the key function parses what is there, bounded at one mebibyte. Register keys on the client address the socket reported, carried into the request by the kernel's adapter, and the proxy headers stay unread on purpose: `X-Forwarded-For` is a header any client can write, and a limiter that keys on it verbatim sells a fresh bucket of ten registrations per forged address, one per request, forever.

The shared bucket closes the dodge hole. A body the email key cannot parse lands in one `login:unparsed` bucket, so garbage cannot orbit the limit by being invalid, and the suite drives that path with four shapes of unusable body, non-json, a list, an empty or non-string email:

```
def key_email(request):
    """login keys on lower(email). the key lives in the body, so at
    most MAX_KEY_BODY_BYTES of it is read, mirroring the go lane's
    bounded reader, and whatever past the cap is ignored. a body
    without a parseable email shares one bucket: garbage must not
    sidestep the limit."""
    body = request.body
    if len(body) > MAX_KEY_BODY_BYTES:
        body = body[:MAX_KEY_BODY_BYTES]
    try:
        parsed = json.loads(body)
    except ValueError, UnicodeDecodeError:
        return "login:unparsed"
    if not isinstance(parsed, dict):
        return "login:unparsed"
    email = parsed.get("email")
    if not isinstance(email, str) or not email:
        return "login:unparsed"
    return "login:" + email.lower()
```

listing 454: KeyEmail: a bounded parse, garbage into one shared bucket (source: python/api/pyapi/limit.py, lines 147-165)

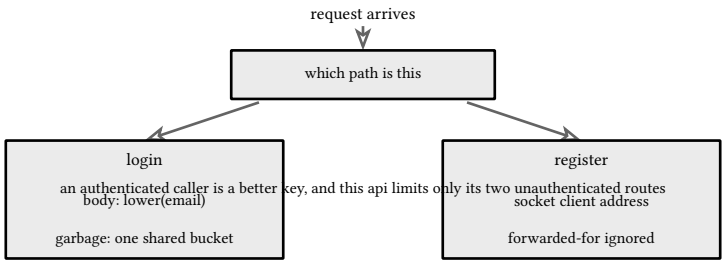


figure 175: key choice: the route decides what the bucket belongs to

35.6. limits under test

Every time-dependent behavior in this chapter is arithmetic on a fixed clock, because a rate limiter tested against the wall clock is tested against a scheduler. The injected clock is a two-line fake whose `advance` is the only way time moves, and the fixed-draw tests walk the bucket through its whole grammar: burst, cap, sustained rate, half-token denial, denial that spends nothing. The property loop is a seeded `random.Random` over the one invariant a bucket must hold, across two hundred trials of five hundred draws at random gaps, the allowed count never exceeds capacity plus elapsed seconds times the refill rate. The vector replay runs the layer against the frozen file over a real `App`: five setup dispatches, then the sixth asserts the vector's status, every header byte, the envelope's code and message, and the request id checked for uuid v7 shape only, since this layer mints it fresh. No test sleeps, and the twenty tests run in milliseconds:

```
    def test_the_sixth_login_vector_replays_over_the_stack(self):
        vector = json.loads((VECTORS / "06-login-429.json").read_text(encoding="utf-8"))
        clock = FakeClock(T0 + vector["when_offset_seconds"])
        app = self.build_app(clock)
        request = self.vector_request(vector)
        for _ in range(5):
            app.dispatch(request)
        response = app.dispatch(request)
        self.assertEqual(response.status, vector["response"]["status"])
        for name, value in vector["response"]["headers"].items():
            self.assertEqual(response.headers.get(name), value, name)
        envelope = json.loads(response.body)["error"]
        self.assertEqual(envelope["code"], "rate_limited")
        self.assertEqual(
            envelope["message"], vector["response"]["body"]["error"]["message"]
        )
        # this layer mints the id fresh, so only its shape is this
        # module's to claim
        self.assertEqual(uuid.UUID(envelope["request_id"]).version, 7)
```

listing 455: the vector replay: frozen clock at t0+12s, five draws, then the frozen sixth over the app (source: python/api/tests/test_limit.py, lines 242-260)

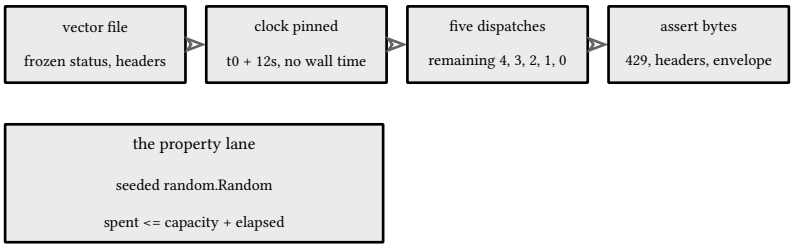


figure 176: the fixed-vector pipeline: file in, draws through the layer, bytes out

35.7. admission preview

A rate limit refuses requests over time, an admission gate refuses them over concurrency, and the two compose because they bound different axes of the same cliff. Little's law states the trade in one line: the average number of requests inside the system equals the arrival rate times the time each spends there, `L = lambda W`. Past the point where every arriving request finds all workers busy, latency rises steeply while throughput stops moving, the hockey stick chapter 37 measures. A rate limit caps lambda at the edge. An admission gate caps L directly with a `BoundedSemaphore`, holds one slot per admitted request through the `try_acquire` move the docs describe, returning false immediately rather than blocking, and answers the contract's overload envelope the instant the slots run out. The semaphore and its 503 are chapter 37's to build, along with the drain it protects:

```
    def try_acquire(self):
        """take one slot without waiting: the whole admission policy.
        false means the gate is full and the caller answers 503. the
        semaphore take and the counter step share one lock so a drain
        reading in_flight can never observe zero with a slot already
        claimed but not yet counted."""
        with self._mu:
            if not self._slots.acquire(blocking=False):
                return False
            self._in_flight += 1
        return True

    def release(self):
        """give the slot back. the semaphore's bound check runs after
        the counter, so an unbalanced release fails loudly on the
        release the semaphore refuses."""
        with self._mu:
            self._in_flight -= 1
            self._empty.notify_all()
        self._slots.release()
```

listing 456: the gate's take and give back: nonblocking acquire, bounded release (source: python/api/pyapi/load.py, lines 50-69)

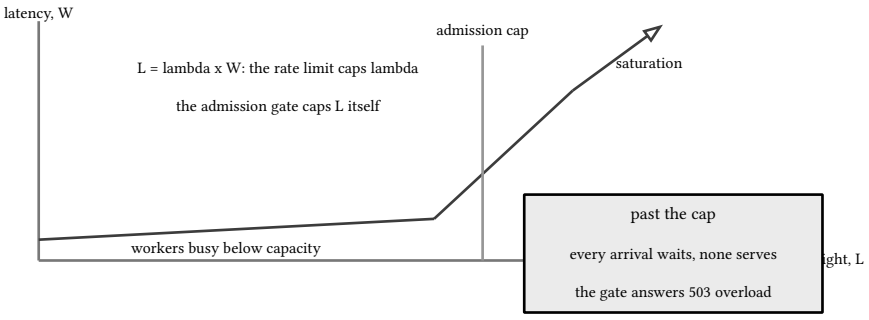


figure 177: latency versus concurrency: the hockey stick the admission cap cuts off

sources: docs.python.org, the 3.14 threading page for BoundedSemaphore's bound check, acquire(blocking=False), and the absence of any rate limiting utility, and the 3.14 queue page for the multi-producer multi-consumer description, plus RFC 6585 and RFC 9110 for 429 and Retry-After, all accessed 2026-09-27. Verified by `python/api/tests/test_limit.py`, 21 tests under unittest discover on the pinned cpython 3.14.7, the sixth-login vector replayed byte for byte from `python/api/contract/testdata/06-login-429.json`.

36. observability

A service no one can see is a service no one can fix, and a service that logs prose no machine can query is seen only by whoever is reading along live. This chapter builds the service's observability in `pyapi/obs.py`: `logging` with a hand rolled json handler that redacts a closed key set and takes its clock by injection, the request context carried in `contextvars`, the W3C traceparent header parsed and formatted from scratch, the metrics registry and its prometheus text exposition with the contract's series and buckets, p99 done honestly by nearest rank, the exposition on its own listener per the contract, and one request followed through every signal.

36.1. logging the real setup

The production logger is one constructor plus one handler subclass, not a pattern study: json lines to a stream, a level, and redaction installed once so every line the service ever writes passes through it. The handler is a `logging.Handler` whose `emit` writes one line built by `format_line`: the time from the injected clock rendered as RFC 3339 UTC, the level lower cased, the message, the request id when the context carries one, then every `extra` field the call passed, in call order, with the redact set masked. The extras fall out of `record.__dict__` by subtraction: every LogRecord carries the same closed set of standard attributes, 21 measured on the pinned 3.14.7, and whatever remains is exactly what the caller passed, in the order it was written. A handler that raises goes to `handleError`, the platform's contract, never into the request:

```
    def emit(self, record):
        try:
            self._stream.write(self.format_line(record) + "\n")
        except Exception:
            self.handleError(record)

    def format_line(self, record):
        fields = {
            "time": _iso_utc(self._now()),
            "level": record.levelname.lower(),
            "msg": record.getMessage(),
        }
        if request_id() is not None:
            fields["request_id"] = request_id()
        for key, value in record.__dict__.items():
            if key in _STANDARD_ATTRS or key == "message":
                continue
            if key.lower() in self._redact and isinstance(value, str):
                value = REDACTED
            fields[key] = value
        return json.dumps(fields, ensure_ascii=False, separators=(",", ":"))
```

listing 457: Emit and format_line: one json line, extras by subtraction, redaction on the way out (source: python/api/pyapi/obs.py, lines 83-103)

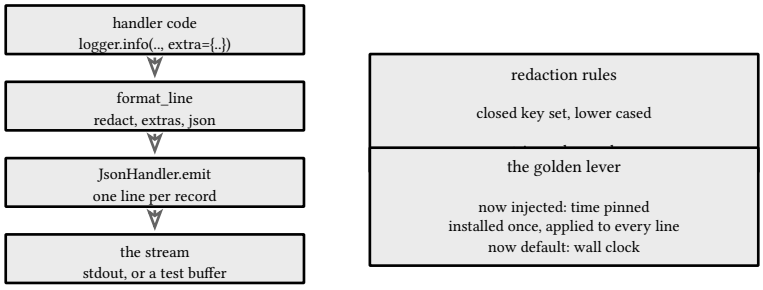


figure 178: the handler's own stack: call site through the handler to the stream

36.2. structured beats interpolated

An interpolated message packs data into prose, and prose is lossy: `login failed for ada, attempt 3` cannot be grouped by email, counted by attempt, or joined to its request id without a regex that breaks the day the message rewords itself. The structured line carries the same information as fields, and every field keeps its name, its type, and its place in an index. The service's rule: messages name the event, `extra` carries the data, and no handler interpolates a value it could pass as a field.

The golden test pins the whole line: the time member is the injected clock, the level and msg come from the call, the fields land in call order, and the password field is the masked value, never the secret. With the clock injected, a log line replays in a test the way a response body can, which is why the handler takes `now` at construction:

```
    def test_the_golden_line_pins_every_member(self):
        logger, stream = make_logger()
        logger.info(
            "login failed",
            extra={
                "email": "ada@example.org",
                "password": "correct horse",
                "attempts": 3,
            },
        )
        self.assertEqual(
            stream.getvalue(),
            '{"time":"2026-10-01T05:00:00Z","level":"info","msg":"login failed",'
            '"email":"ada@example.org","password":"[redacted]","attempts":3}\n',
        )

    def test_redaction_folds_case_and_masks_strings_only(self):
```

listing 458: the golden line pinned byte for byte, password masked, time from the injected clock (source: python/api/tests/test_obs.py, lines 28-44)

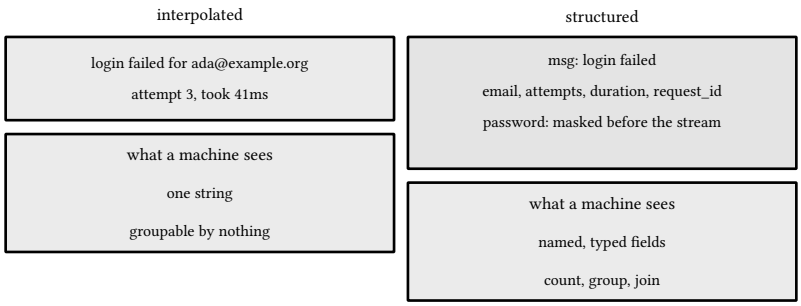


figure 179: the same failure, line oriented and structured

36.3. the request context in contextvars

Every signal of one request needs to agree on whose request it was, and the carrier is `contextvars`. The module's page states the thread behavior in one sentence: "Since each thread has its own context stack, `ContextVar` objects behave in a similar fashion to `threading.local()` when values are assigned in different threads." Measured on the pinned interpreter, a thread started after the spawner set a value reads the variable's default, which is exactly what ThreadingHTTPServer wants: each request arrives on its own thread that begins clean, the middleware sets the id and the trace at request start, and isolation is free.

One value has one carrier, and the request id is the kernel's, so this module reads it through the kernel's getter and never mints a second variable for the same fact. The trace context is observability's own, set by the span layer and read by anything that wants the current trace:

```
# the request context carrier. the request id is the kernel's var, one
# value one carrier: this module reads it and never mints a second
# copy. the trace context is observability's own. a new thread starts
# without the spawner's values, so each request thread under
# ThreadingHTTPServer begins clean and the request middleware sets
# these at request start
_TRACE = contextvars.ContextVar("trace", default=None)


def request_id():
    """the kernel's bound request id, or None outside a request."""
    return current_request_id()


def set_trace(trace):
    return _TRACE.set(trace)


def trace():
    return _TRACE.get()
```

listing 459: the carrier: the kernel's request id read, never duplicated, and the trace var beside it (source: python/api/pyapi/obs.py, lines 44-63)

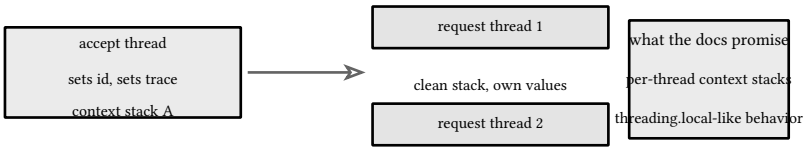


figure 180: one variable per request thread, the fresh thread starts clean

36.4. tracing from scratch

A trace is the shape of one request's time: nested spans, each with a name, an id, a parent, and a start and end, joined across services by one shared header. The W3C trace context format is deliberately small, four lowercase hex fields, `version-traceid-spanid-flags`, 2, 32, 16, and 2 digits, and small enough to parse and format by hand, which is what this module does: no sdk, no collector, no span leaves the process, the point is to know the format and the tree.

The parser is strict where the spec allows a choice. Four fields of the exact lengths, version `ff` reserved and rejected, all-zero trace or span ids rejected, uppercase hex rejected because lowercase is the canonical rendering, and only the sampled bit of the flags byte kept, the bit the spec says propagates, everything else masked away exactly as the wire writes it back. On the server side one call continues an inbound trace: parse, then a context whose trace id is the inbound trace and whose span is a fresh id, sampling inherited. A header that is absent or malformed answes None, and the caller roots fresh, because one bad header must never kill a request:

```
def parse_traceparent(header):
    """parse the traceparent header strictly: four fields of the exact
    lengths 2, 32, 16, and 2, lowercase hex, version ff reserved,
    all-zero trace or span ids rejected, only the sampled bit of the
    flags byte kept. any failure answes None so one bad header can
    never kill a request."""
    parts = header.split("-")
    if len(parts) != 4:
        return None
    version, trace_id, span_id, flags = parts
    if version == "ff" or not _is_hex(version, 2):
        return None
    if not _is_hex(trace_id, 32) or trace_id == "0" * 32:
        return None
    if not _is_hex(span_id, 16) or span_id == "0" * 16:
        return None
    if not _is_hex(flags, 2):
        return None
    return TraceContext(trace_id, span_id, bool(int(flags, 16) & 1))
```

listing 460: parse: four fields, exact lengths, reserved version and case rejected (source: python/api/pyapi/obs.py, lines 136-154)

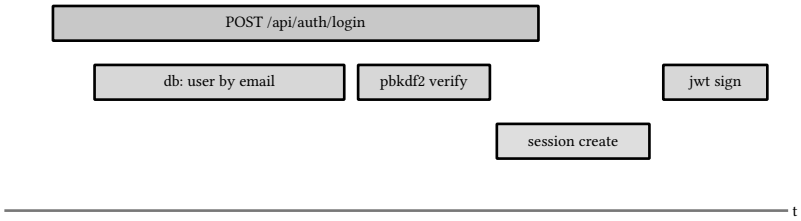


figure 181: one login request's spans over time, the tree the recorder prints

36.5. metrics and the prometheus text format

Two series carry the load story, and both are registry types, not libraries. The counter is one series per route and status code, `goapi_requests_total{route,code}`, incremented once per request by the recording layer. The histogram is one per route, `goapi_request_duration_seconds`, with the contract's bucket ladder tight at the low end where service latencies live and geometric through the tail: one, two and a half, five, ten, twenty five, fifty milliseconds, then the hundreds, seconds, and `+Inf`. Each observation increments every bucket whose bound it fits under, which is what makes the counts cumulative, plus the sum and the count. The exposition rules are the format's own: a histogram must carry a `+Inf` bucket whose value equals the count, and label values escape backslash, quote, and newline.

The renderer writes the prometheus text directly, help and type lines, counter family first then histogram, series sorted by label tuple, bucket lines ascending with `+Inf` last. Zero buckets are elided, the one deliberate divergence from client_golang, which prints every finite bucket at zero as well, and the frozen metrics vector pins the consequence byte for byte: one deliberately slow healthz observation, past every finite bound, renders exactly the eight lines the contract froze. `render()` is the one writer of exposition bytes, and every listener that serves them delegates here. The endpoint itself binds on its own listener in production, `ThreadingHTTPServer` on the internal address with the socket bound before the factory returns, and the same bytes serve on the main mux in tests through the wiring's route member:

```
# HELP goapi_requests_total requests by route and code.
# TYPE goapi_requests_total counter
goapi_requests_total{route="/healthz",code="200"} 1
# HELP goapi_request_duration_seconds request latency.
# TYPE goapi_request_duration_seconds histogram
goapi_request_duration_seconds_bucket{route="/healthz",le="+Inf"} 1
goapi_request_duration_seconds_sum{route="/healthz"} <observed>
goapi_request_duration_seconds_count{route="/healthz"} 1
```

listing 461: the frozen vector the exposition is checked against, byte for byte (source: python/api/contract/testdata/15-metrics.txt, lines 1-8)

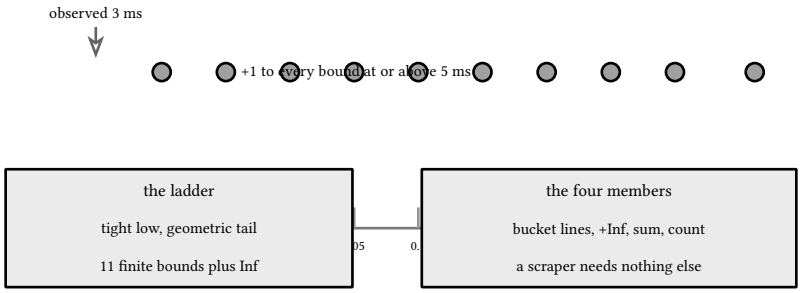


figure 182: cumulative buckets: each observation fills every bound above it

36.6. p99 done honestly

The percentile definition is nearest rank: sort the latencies, take the ceil of p over 100 times n as the rank, answer that rank's value. One function over the sorted list is the whole definition, the same one the profiling chapter pinned and the load chapter reports from, and its honesty is in what it refuses to hide: p99 of one hundred values is the ninety-ninth smallest, not an interpolation between the ninety-ninth and hundredth, and p99.9 of ten samples clamps to the largest, because a percentile past the data is a bound, not a measurement.

The bucket approximation is the second half of honesty. A scraped histogram cannot answer nearest rank, it can only say which bucket the rank lands in, so `from_cumulative` answers the smallest bound whose cumulative count reaches the rank, an upper bound on the true value, never the value itself, and not knowable when the rank falls into `+Inf`. The test pins the contrast: ninety-nine samples at 0.9 milliseconds and one at 1.1 give an exact p99 of 0.9 milliseconds and a bucket p99 of 1, the bound, and a rank past 2.5 seconds renders as a dash in the load chapter's table:

```
def nearest_rank(sorted_values, p):
    """the p-th percentile of sorted values by the nearest-rank
    method: the ceil(p/100 x n)-th smallest value. p is honored as
    given, so p99 of one hundred values is the ninety-ninth smallest,
    and a rank that runs past the end clamps to the largest, which is
    what p99.9 of ten samples must mean."""
    if not sorted_values:
        return None
    rank = math.ceil(p / 100 * len(sorted_values))
    rank = min(max(rank, 1), len(sorted_values))
    return sorted_values[rank - 1]
```

listing 462: nearest rank: ceil the rank, clamp the ends, no interpolation anywhere (source: python/api/pyapi/obs.py, lines 338-348)

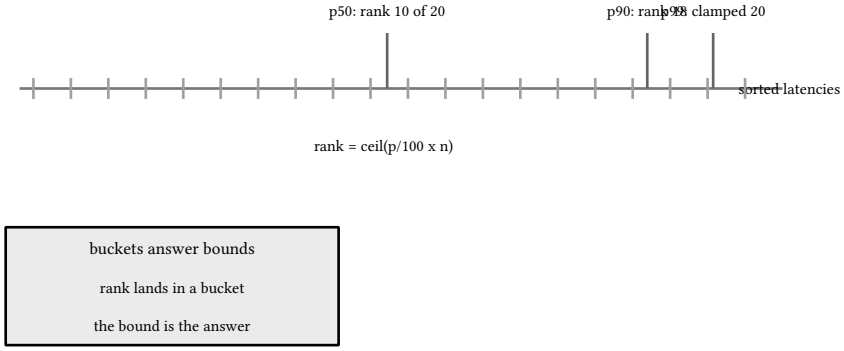


figure 183: the tail ladder: where p50, p90, p99 sit in one sorted sample

36.7. one request, every signal

Wire the signals in the order the request meets them and one login tells the whole story. The recording layer is mounted outermost, so every response including the kernel backstop's lands in both series, and it never sniffs a socket for the status: the kernel hands middleware a `Response` value, so the layer reads the status straight off it and the latency is the injected clock's arithmetic around the inner handler. The span layer continues the inbound traceparent or roots fresh, holds the trace in the contextvar, and records one root span whose start and end come from the same clock. The route label comes from the app's own table, the matched pattern rather than the raw path, so param routes share one series and an unmatched request lands in one bounded `unmatched` series instead of the path's unbounded cardinality.

Reading it back is what the tests do, separately and without ceremony: the golden line test asserts the id on the log line, the traceparent tests pin the grammar's strict side, the exposition test replays the frozen vector, and the listener test serves the exposition over a loopback socket and asserts the body equals `render()`. Grep the id and every artifact answers:

```
def metrics_layer(metrics, route_for, now=None):
    """the recording layer, mounted outermost so every response
    including the kernel backstop's lands in both series. the latency
    is the injected clock's arithmetic around the inner handler and
    the status is read straight off the Response value: the kernel
    hands middleware values, not a socket to sniff."""
    clock = now if now is not None else time.time

    def layer(next_handler):
        def handler(request):
            start = clock()
            response = next_handler(request)
            route = route_for(request)
            metrics.count_request(route, response.status)
            metrics.observe(route, clock() - start)
            return response

        return handler
```

listing 463: the recording layer: status off the Response value, both series ticked, the clock injected (source: python/api/pyapi/obs.py, lines 432-450)

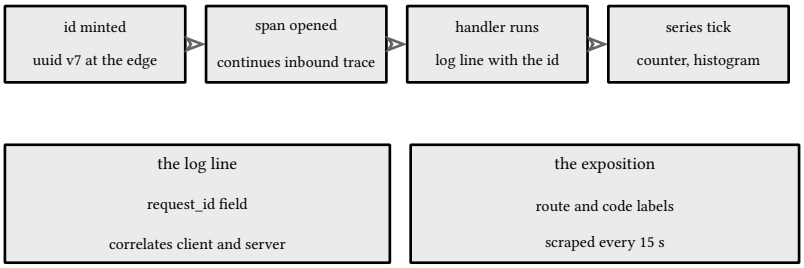


figure 184: one request id through every signal

sources: docs.python.org, the 3.14 logging page for the Handler contract and the LogRecord attribute table, and the 3.14 contextvars page for the per-thread context stack and the threading.local comparison, both accessed 2026-09-27, the W3C trace context format at w3c.github.io/trace-context for the traceparent grammar and the sampled flag, and the prometheus exposition format documentation for the histogram members, the mandatory +Inf bucket, and the escaping rules, both accessed 2026-09-27. Verified by `python/api/tests/test_obs.py`, 21 tests under unittest discover on the pinned cpython 3.14.7, with the exposition replayed line for line from `python/api/contract/testdata/15-metrics.txt` and the LogRecord attribute set measured on the interpreter itself.

37. load and p99

A service that has never been loaded is a service whose p99 is a guess, and a deploy that has never been drained is a deploy that drops requests every time it ships. This chapter closes the family with the tools that make both claims checkable, all in `pyapi/load.py`: the admission gate that answers overload at the cap, the graceful drain whose order is the contract and whose exit code reports what the drain earned, the two harness loops over injected client, clock, and pacing seams, the report table with its honest percentiles, and the thread census that holds the harness itself honest after a run. Chapter 35 capped lambda, this chapter caps `L` and measures what both bought.

37.1. admission control

The rate limit bounds arrivals over time. The admission gate bounds requests inside the service at once, and the mechanism is a counting semaphore: one `threading.BoundedSemaphore` of the deployment's honest concurrency, `acquire(blocking=False)` to take a slot, the release riding a `finally` so a handler that raises still gives the slot back. The nonblocking acquire is the whole policy: a slot is taken when one is free, and when the gate is full the answer is immediate, the contract's overload envelope at 503, never a queue, because a queued request holds a thread and memory the service just proved it cannot spend.

Two details make the gate trustworthy rather than merely present. The semaphore is bounded, so releasing past the initial count raises `ValueError`, and the suite pins that loudness with its own test: an unbalanced release is a bug this vehicle wants named, not a slot invented from nothing. And the counter beside the semaphore is observation, not enforcement: `in_flight` exists so the drain and the tests can see the occupancy, while the semaphore alone decides who enters. The gate as one middleware layer refuses through the kernel's failure writer, so a 503 carries the standard envelope with the request id like every other failure:

```
def admission_layer(gate):
    """the gate as one middleware layer: nonblocking take, release in
    a finally so a handler that raises still gives the slot back, and
    the overload envelope through the kernel's one failure writer when
    the gate is full."""

    def layer(next_handler):
        def handler(request):
            if not gate.try_acquire():
                return failure(request, fail(OVERLOAD, "server at capacity"))
            try:
                return next_handler(request)
            finally:
                gate.release()

        return handler

    return layer
```

listing 464: the gate as a layer: nonblocking take, finally release, overload envelope at the cap (source: python/api/pyapi/load.py, lines 79-96)

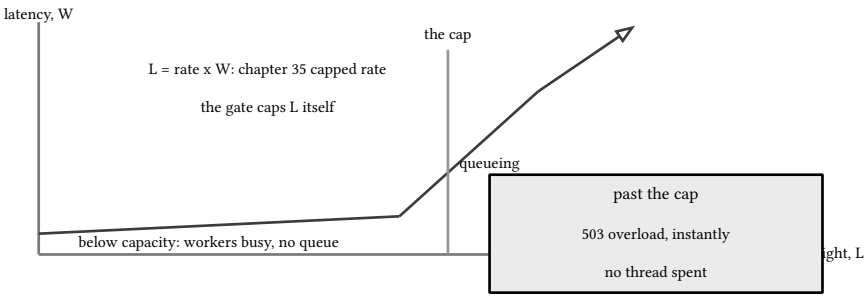


figure 185: latency versus in-flight: the hockey stick the gate cuts

37.2. the drain contract

Shutdown has an order, and the order is the contract. Readiness flips first: the drainer's state moves to draining, `/readyz` answers its 503 overload envelope through the kernel's config hook, and every load balancer watching the service stops routing new requests here while the old ones are still being answered well. Then the listener closes, so new connections are refused at the tcp level. Then the in-flight requests finish under a grace deadline, and the helper returns what happened instead of pretending the drain succeeded.

The platform's own behavior explains why the accounting is the gate's job. The docs are explicit: "`ThreadingMixIn.server_close` waits until all non-daemon threads complete", and the kernel's server runs its request threads as daemons precisely so the platform's close does not join them. That makes `shutdown` plus `server_close` stop the accept loop and close the socket without waiting on handlers, and the gate's condition, `wait_empty`, is what actually waits, releasing the caller the moment the last slot goes back or the grace runs out:

```
def drain(server, gate, drainer, grace):
    """the contract's order, stated once: readiness flips first so
    every load balancer watching stops routing here, the listener
    stops accepting, then in-flight requests finish under the grace
    deadline. the result is the caller's to judge, never swallowed.
    a listener close that raises, a socket already gone, must not
    steal the verdict: the emptiness wait still runs."""
    drainer.begin_drain()
    server.shutdown()
    try:
        server.server_close()
    except OSError:
        pass
    emptied = gate.wait_empty(grace)
    return DrainResult(emptied, gate.in_flight, grace)
```

listing 465: Drain: readiness flips, listener closes, in-flight finish under the grace (source: python/api/pyapi/load.py, lines 138-152)

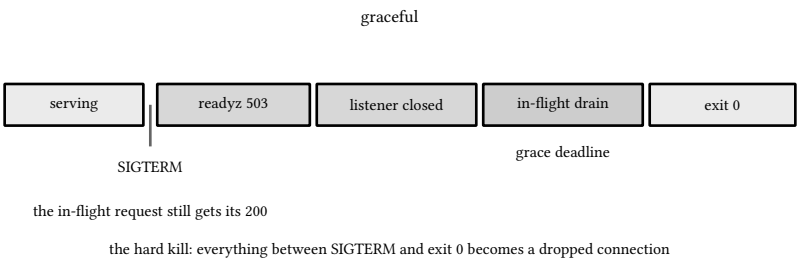


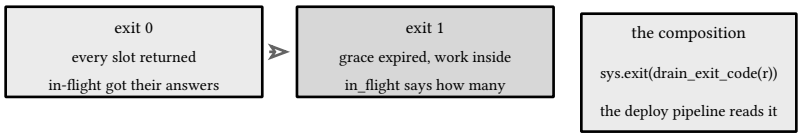
figure 186: SIGTERM to drain to exit, against the hard kill

37.3. the exit code reports the drain

A drain that returns nothing is a drain that cannot be alerted on. The helper answers a `DrainResult` carrying three facts: whether everything in flight finished inside the grace, how many requests were still holding slots when the grace ran out, and the grace that was on the clock. The composition turns that into the process exit code, zero when the last slot went back in time, one when the grace expired with work still inside, and an operator watching a deploy sees the difference in the container's exit status without reading a single log line. The result is the caller's to judge, never swallowed by the helper, which is the same discipline the kernel applies to failures: report, do not absorb:

```
def drain_exit_code(result):
    """the exit code reports what the drain earned: zero when every
    in-flight request finished, one when the grace expired with work
    still inside."""
    return 0 if result.drained else 1
```

listing 466: the exit code: zero when drained, one when the grace expired with work inside (source: python/api/pyapi/load.py, lines 155-161)



the helper reports, the caller judges

figure 187: the two exits and what each one claims

37.4. testing the drain without sleeps

The drain test is observations on real state, and none of them wait on the clock. The full test holds two slots through a real ThreadingHTTPServer, starts the drain on its own thread, and gates on the drainer's `flipped` Event, a `threading.Event` the flip sets, so the assertion that readiness moved costs nothing. Fresh dials are polled, not slept on: the probe opens new connections until the refused one arrives, the poll of real socket state, and the held slots are released by the test so the drain's condition fires and the join completes. The expired-grace leg is a zero-second look: `Condition.wait_for` with a timeout of zero checks the predicate once and returns its falsity, so the deadline path is exercised with no waiting at all, which is the honest python analog of the go lane's synctest bubble:

```
    def test_an_expired_grace_reports_the_deadline(self):
        server = make_server()
        self.addCleanup(server.server_close)
        gate = load.AdmissionGate(1)
        drainer = load.Drainer()
        gate.try_acquire()
        result = load.drain(server, gate, drainer, 0.0)
        self.assertFalse(result.drained)
        self.assertEqual(result.in_flight, 1)
        self.assertEqual(load.drain_exit_code(result), 1)
        gate.release()

    def test_readyz_flips_to_overload_under_the_kernel_config_hook(self):
        gate, drainer, _layer = load.wire_load(2)
        app = kernel.App(kernel.Config(ready=drainer.ready))
```

listing 467: the expired grace: a zero-second look at a held slot, the deadline reported (source: python/api/tests/test_load.py, lines 121-135)

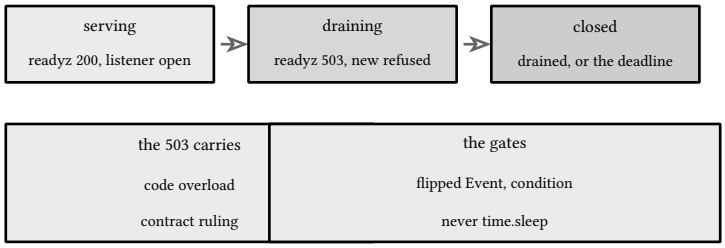


figure 188: the drain lifecycle: serving, draining, closed

37.5. the load harness

The two loops differ in what they hold constant. The closed loop holds concurrency constant: worker threads, each firing one request at a time and starting the next only after the previous answer lands, so offered load never exceeds the worker count and the loop measures a system at a chosen concurrency. The open loop holds arrival rate constant: request `i` is scheduled at `i` over rate seconds past the start and fired whether or not anything has come back, so when the service slows, requests pile up exactly the way they do in production, and the cliff becomes visible. Little's law predicts the pile: in-flight equals arrival rate times service time, and an open loop at fixed rate turns rising latency directly into rising in-flight count.

Three seams are injected, which is what keeps every number in a test a planned value rather than scheduling. The client answers one `(status, seconds)` pair per request index and owns its own timing, so latency is reported by the seam, not measured around whatever the thread scheduler did. The clock reads now for the elapsed span. The pacing seam receives target instants, the real one sleeping the delta, the test one recording the schedule and returning instantly, so the open loop's arithmetic is asserted exactly. Warmup fires first on every loop and never reaches the result, because the first request pays for the dial, the dns lookup, and the cold caches, and none of that is the number the run exists to measure:

```
class ScriptedClient:
    """the strict fake: one planned (status, seconds) per request
    index, no clock reads, no sockets."""

    def __init__(self, plan):
        self.plan = plan
        self.calls = []

    def __call__(self, index):
        self.calls.append(index)
        return self.plan[index]
```

listing 468: the strict fake client: one planned pair per index, no clock reads, no sockets (source: python/api/tests/test_load.py, lines 159-169)

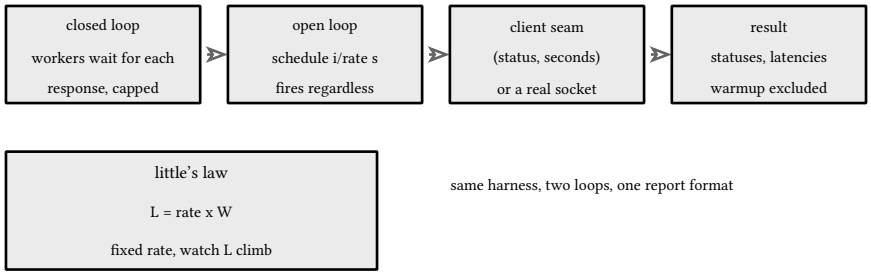


figure 189: workers, client seam, result: one harness, two loops

37.6. the report and honest percentiles

One run folds into one row, and the percentiles are nearest rank, the definition chapter 36 pinned, computed over the sorted latencies the client seam reported. The exact row and the bucketed row sit in the same table over the same samples: the bucketed row feeds the latencies through a histogram and answers each percentile as the bound the rank lands in, which is what a scraper sees. Six latencies of 1, 1, 2, 3, 5, and 8 milliseconds give an exact p50 of 2 and p99 of 8, while the bucketed row reads p50 as 2.5 and p99 as 10, the bounds, and a rank that falls past the largest finite bound renders as a dash, because inventing a digit the samples never had is the one sin the definition forbids. Two rows, same run, the difference is the resolution a histogram throws away, stated in the table's source column instead of hidden in a footnote, and the units live in the header where a latency column cannot be misread:

```
def summarize(latencies):
    """one run's honest percentiles, exact by nearest rank over the
    sorted samples."""
    ordered = sorted(latencies)
    return {f"p{p}": nearest_rank(ordered, p) for p in PERCENTILES}


def bucket_summarize(latencies):
    """the same samples as a scraper sees them: fed through a
    histogram, every percentile answered as the bound the rank lands
    in, None where the rank falls past the largest finite bound."""
    metrics = Metrics()
    for latency in latencies:
        metrics.observe("run", latency)
    cum, bounds, _, _ = metrics.histogram("run")
    return {f"p{p}": from_cumulative(cum, bounds, p) for p in PERCENTILES}
```

listing 469: summarize exact by nearest rank, bucket_summarize as the scraper sees it (source: python/api/pyapi/load.py, lines 284-300)

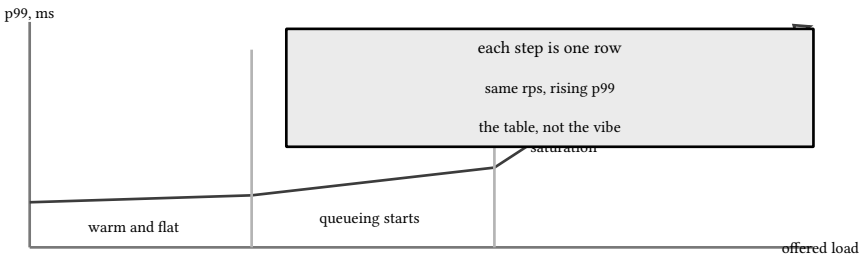


figure 190: p99 under offered-load steps: where the cliff shows in the table

37.7. the thread census

A load tool that leaks a thread per run silently slows every later run and lies about the service it measures, and python names the leak directly: `threading.enumerate()` lists every live thread, so the census is a set difference against a snapshot taken before the run, and the harness test asserts the difference is empty after both loops have finished. The snapshot is taken in the same process the harness ran in, which is also its honesty: whatever the loops started and did not join appears by name, no profile required. The go lane needed a runtime profile to say this, python says it with one function over the thread list, and the discipline is the same in both: leak nothing per run, or the numbers are fiction:

```
def thread_census(before):
    """threads alive now that were not in the snapshot: the leak this
    check names, because a load tool that leaks a thread per run
    silently slows every later run and lies about the service it
    measures."""
    known = set(before)
    return [thread for thread in threading.enumerate() if thread not in known]
```

listing 470: the census: threads alive now that were not in the snapshot (source: python/api/pyapi/load.py, lines 272-279)

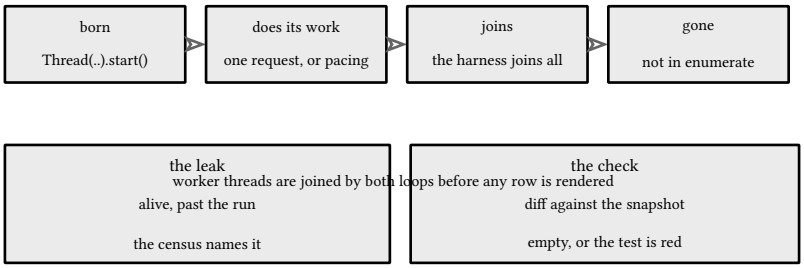


figure 191: a thread lifecycle: born, joined, or leaked

sources: docs.python.org, the 3.14 socketserver page for ThreadingMixIn's daemon_threads and the server_close joining rule, the 3.14 threading page for BoundedSemaphore, Event, and Condition.wait_for, accessed 2026-09-27. Verified by `python/api/tests/test_load.py`, 17 tests under unittest discover on the pinned cpython 3.14.7, with the report table pinned from fixed latencies and the drain observed over a real loopback listener without a single sleep.

38. ship it

The module ends as a directory of tests unless it also ships. This chapter turns the service into an artifact and a lane: an image built from one stage with no install step at all, one compose command that brings it up healthy, and a make gate that replays the frozen contract over that stack. Everything leans on machinery the earlier chapters built: the kernel routes chapter 27 owns, the pbkdf2 cost chapter 30 pins, the bucket chapter 35 refills, the exposition chapter 36 binds, the admission gate and drain chapter 37 wires. The platform envelopes around a shipped service, the ci workflow, the helm chart, and the terraform slice, are taught in full in book 4, ch 29 (go 1.27, ship it) and mirrored in book 6, ch 33 (c# 15, f# 11, and .net 11, ship it) and book 7, ch 34 (javascript es2026 and typescript 7, ship it), and the corpus keeps one executing lane per vehicle instead of re-deriving them. What this chapter owns is the python truth of each layer: what the image is when the interpreter is the whole platform and nothing installs, and what the probe runs when the only executable in the image is the runtime the service is made of.

38.1. the image

One stage, because python has nothing to stage. The go lane's chapter crossed from a builder into `scratch` with one static file, legal because its store engine was pure go. The `c#` lane needed the aspnet base because its sqlite bundle is a native library the runtime must carry. The js lane's image is the runtime alone, and its install step exists as a proof: `npm ci` over the committed lockfile resolves zero packages, so the day a dependency sneaks into the manifest the build fails loudly. This vehicle's dependency list is empty because the platform and the dependency are the same object: cpython ships with `sqlite3`, `json`, and `http.server` compiled in, and the vehicle imports nothing outside that. There is no install step to make a no-op of. The source is the dependency, `COPY` is the only content step, and the image states that plainly:

```
# single stage, stdlib honesty: the vehicle imports nothing outside
# the standard library, so the image has no pip layer at all, the
# source tree is the dependency, and a builder stage would carry a
# toolchain the image never runs. the tag is the exact version the
# book builds under, cpython 3.14.7 on debian trixie slim.
FROM python:3.14.7-slim
WORKDIR /app
# the python image ships as root, unlike node's, so this lane carries
# its own user: uid 1000, no home, no login shell, just an identity to
# own files with. /data is created with an owner because a named
# volume inherits the image content's ownership on first mount, and a
# volume over a root-owned directory would deny the container user its
# database.
RUN useradd --uid 1000 pyapi && mkdir /data && chown pyapi:pyapi /data
COPY main.py .
# a directory source copies its contents, so the destination names the
# package directory itself the way the js lane's COPY src src does
COPY pyapi pyapi
ARG VERSION=dev
ENV GOAPI_VERSION=${VERSION}
# unbuffered so the json access log streams to docker logs instead of
# block-buffering; no bytecode because the runtime user cannot write
# under /app anyway and the source is the shipped artifact; utf8 mode
# is the same mode the plain gate runs under with -X utf8
ENV PYTHONUNBUFFERED=1 PYTHONDONTWRITEBYTECODE=1 PYTHONUTF8=1
USER pyapi
EXPOSE 8080
ENTRYPOINT ["python", "main.py"]
```

listing 471: the whole image: one stage, the runtime, no install layer at all, the user the lane creates, the source (source: python/api/Dockerfile, lines 1-28)

The tag is the exact version the book builds under: the machine's pinned interpreter is cpython 3.14.7, the same interpreter the plain gate runs under, and `python:3.14.7-slim` was verified present against the registry's tags api the day this chapter shipped, with the image's own `python -V` answering 3.14.7. The `slim` variant is debian trixie minus the common packages, and two absentees matter later: neither `curl` nor `wget` is in it.

The user is this lane's own, and that is a python fact the other lanes did not have. Node's image ships as uid 1000 already, so the js lane only names the number. The python image ships as root, `id` answers uid 0, so the lane creates the user itself: `useradd --uid 1000 pyapi`, no home, no login shell, just an identity to own files with. `/data` is created and chowned at build time because a named volume inherits the image content's ownership on first mount, and a volume over a root-owned directory would deny the container user its database, the ruling the js lane made carried across.

Three environment lines earn their place. `PYTHONUNBUFFERED` keeps the json access log streaming to `docker logs`, one line per request the moment it is written, verified by reading the lane's own log off the composed stack. `PYTHONDONTWRITEBYTECODE` states a fact the file permissions already enforce: the source tree ships root-owned and read-only under `/app`, the runtime user cannot write a `__pycache__` into it, and the source is the shipped artifact. `PYTHONUTF8` puts the process in the same mode the plain gate runs under with `-X utf8`. The whole service adds 388,740 bytes over its base, measured off `docker image inspect`, under half a megabyte and every byte of it source.

The lane caught its own bug on the first build, and the catch is the argument for the lane. `COPY main.py pyapi ./` flattened the package: a directory source copies its contents, not itself, so the modules landed loose in `/app` and the container exited 1 naming `ModuleNotFoundError: No module named 'pyapi'`. Loud at boot, before any request, because the entrypoint runs the composition and the composition imports the package. The fix names the directory, `COPY pyapi pyapi`, the same shape as the js lane's `COPY src src`:

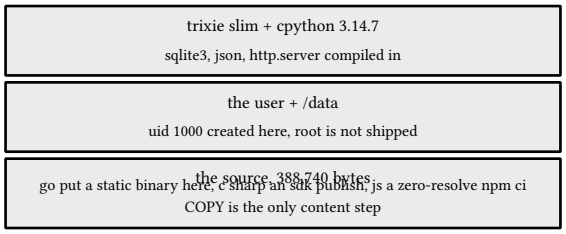


figure 192: one stage, four layers, and the empty slot where the others install

38.2. compose, one command

The compose file is one service and its volume, and the contract it enforces is the return value of a single command. `docker compose up -d --build --wait` builds the image, creates the container, starts it, and blocks until the healthcheck passes, so a green up means the image built, the store applied its migrations against a fresh volume, and the listener answers probes. `down -v` removes the volume in the same breath, so every lane run starts from migration zero, which is what keeps the suite behind it deterministic:

```
services:
  api:
    build: .
    environment:
      GOAPI_JWT_SECRET: dev-only-not-a-secret
      GOAPI_DB: /data/goapi.db
      # the kernel parses its addr as one host:port string, so the
      # container side binds every interface explicitly and the host
      # port mapping below is what keeps the lane off 8080 on the
      # shared machine. the metrics listener reads its own name, the
      # obs module's GOAPI_DEBUG_ADDR with its :9091 default
      GOAPI_ADDR: 0.0.0.0:8080
      GOAPI_DEBUG_ADDR: 0.0.0.0:9091
    # host ports stay off 8080 and off every sibling lane: go holds
    # 8080/6060, c sharp 18080/16090, js 19080/19090 and 19180/19190,
    # so a lane must be the only thing under its ports
    ports:
      - "19280:8080"
      - "19290:9091"
    volumes:
      - data:/data
    # the entry runs the load family's drain on SIGTERM with a
    # 30-second budget, and the orchestrator's default kill lands 10
    # seconds after the term, so the grace is widened or the drain the
    # entry runs would be truncated before it could report its exit
    stop_grace_period: 40s
    healthcheck:
      # no curl, no wget in the slim image: the runtime probes its own
      # healthz on loopback with the stdlib, 200 and only 200
      test: ["CMD", "python", "-c", "import sys,urllib.request;
sys.exit(urllib.request.urlopen('http://127.0.0.1:8080/healthz', timeout=1).status != 200)"]
      <<: *hc
```

listing 472: the whole system: both listeners, the volume, the stdlib probe, the widened grace, host ports off 8080 (source: python/api/compose.yml, lines 14-44)

The healthcheck has no `curl` to call, and that is the python shape of the probe. The slim image carries neither `curl` nor `wget`, both probed absent, so the runtime probes its own `healthz` on loopback with `urllib.request`, the same

stdlib the server is made of, and the assertion is 200 and only 200. The kernel's route answers from the route table the composition registered, so a passing probe also proves the composition ran to its last line.

The environment block carries exactly what construction demands, and one honesty belongs here rather than in a footnote. The secret is read at wiring with an empty default, `os.environ.get("GOAPI_JWT_SECRET", "")`, and this chapter ran the image without the name to see what happens: the service starts, `healthz` answers 200, register answers 201, and login mints a real bearer signed with the empty key. The js lane's entry fails closed, refusing to start without the secret, and that refusal is the right owner for it, so this vehicle's entry carries it too: `main()` exits 2 before the listener binds when the name is unset, `build_app` keeps the empty default so tests wire families explicitly, and the composition suite pins both directions. The compose file always supplies one, and the platform envelopes in the go lane exist partly to replace it. The measured difference this chapter opened with is history now, kept here because the probe that found it is the same probe that closed it.

Both listeners bind every interface inside the container because both defaults bind loopback, and a published port cannot reach a loopback bind. The metrics listener reads its own name, chapter 36's `GOAPI_DEBUG_ADDR` with its `:9091` default, and the kernel's `GOAPI_ADDR` carries the api listener. The grace is widened to 40 seconds for a reason the load chapter earned: the entry runs the drain on `SIGTERM` with a 30-second budget, the orchestrator's default kill lands 10 seconds after the term, and an unwidened grace would truncate the drain before it could report its exit code.

The host side of both mappings sits on 19280 and 19290, and the sibling lanes own the rest of the table: go holds 8080 and 6060, c sharp 18080 and 16090, js 19080 and 19090 with its typed lane on 19180 and 19190. A lane is the only thing under its ports, because two lanes racing for one port on one shared machine is a flake factory neither lane can debug from its own logs.

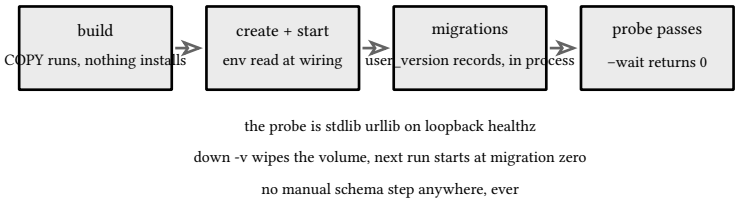


figure 193: what a green up proves, in order

38.3. the docker lane as a gate

The lane is one make target, and its shape mirrors the two-gate rule book 13, ch 4 (infrastructure: docker, databases, and testing, compose and one click local systems) owns: the plain module gate never needs a docker daemon, and the docker lane never runs silently, a missing daemon a loud red with the instruction to start it, never a skip. The integration suite lives under `integration/`, outside the `tests/` directory the plain gate discovers with `unittest discover -s tests`, so the daemon-free gate never even imports it, the faithful mirror of go's build tag and the `c#` lane's project outside the solution:

```
verify-pyapi-docker:
  @docker info >/dev/null 2>&1 || { \
    echo "docker daemon not reachable." >&2; \
    echo "start Docker Desktop (or the engine), then rerun: make verify-pyapi-docker" >&2; \
    exit 1; \
  }
  @cd books/python/api && docker compose down -v >/dev/null 2>&1 || true && \
    trap 'docker compose down -v' EXIT && \
    docker compose up -d --build --wait && \
    PYAPI_URL=http://127.0.0.1:19280 \
```

```
    PYAPI_METRICS_URL=http://127.0.0.1:19290 \
    python -m unittest discover -s integration; rc=$$?; exit $$rc
```

fragment, illustrative, not executed

Two environment families live in this lane and never mix. The `GOAPI_` names are the vehicle's configuration surface, the same keys the compose file sets, read once at wiring. The `PYAPI_URL` and `PYAPI_METRICS_URL` names are the lane's own probe coordinates, read by the integration suite and set by the make target, never by the service. The split guards a failure mode the `c#` lane named: renaming one family without the other makes the lane silently probe its defaults, a green suite pointed at nothing. The suite also refuses a target that carries a path, because the probe routes are root-relative and a prefix would be silently dropped, and the suite would stay green while probing the wrong namespace.

Five tests cover the contract's scenarios: both probes and the two routing envelopes, including the ruling that a 405 carries the `not_found` code plus an `Allow` header; the register with its `Location` and strong ETag, and the replay answered 409 on the email conflict; the login pair with its cookie attribute line, its bearer token, and the identical 401 bytes for a wrong password and an unknown account; the bucket drained to a 429 with `Retry-After` and a remaining count of zero; and the metrics exposition answered on the internal listener. The ETag rule is the sharpest, a strong ETag is the sha256 of the exact response body, so the suite recomputes the hash over the wire bytes and compares:

```
def want_strong_etag(test, result):
    """the strong etag rule: the header is the sha256 hex of the exact
    response body, recomputed over the live wire bytes rather than
    read off any frozen vector."""
    etag = (header(result, "ETag") or "").strip('"')
    test.assertTrue(etag, f"no ETag header, body {result[2]!r}")
    want = hashlib.sha256(result[2]).hexdigest()
    test.assertEqual(etag, want, "the ETag is not the sha256 of the body")
    return etag
```

listing 473: the etag check recomputed over live bytes, not frozen ones (source: python/api/integration/test_lane.py, lines 86-94)

The drain is where the lane tells a python truth the in-process suites cannot. A wrong-password login costs about 27 milliseconds over the wire, read off the lane's own access log, the pbkdf2 verify chapter 30 pins running inside the 401, while the bucket refills one token per second, so a counted drain of exactly five draws races the refill and flakes. The suite draws until the 429 is a fact, bounded at twelve draws, and asserts the denial's shape rather than the interleaving. The clock under test is real, refill is part of the behavior under it.

The lane earned its keep twice during this chapter's own landing. The first was the flatten, the image layer's `COPY` catching a bug no in-process suite could see because no in-process suite imports from an image's `/app`. The second came while the store family was still mid-landing: the lane went red on a login that answered 500, and the container's own log named `SessionStore.create() takes 1 positional argument but 2 were given`, a signature drift between the authn routes and a store edit that no family's own suite had walked, because no family's suite registers through one family and logs in through another over the composition. Both were the same argument: the composed service over a real socket is a different test object than any family's stack, and the contract only holds when the composition holds.

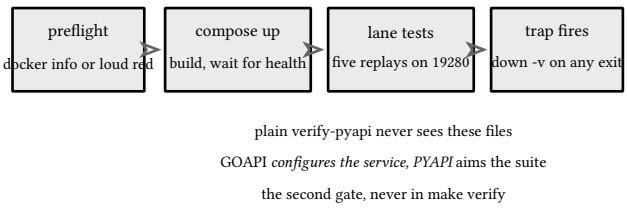


figure 194: the gate: one lane, one trap, no skips

38.4. the shipped checklist

The lane ends where the family began, with the property each chapter contributed, and the composition root is the document that carries them all: the stack reads trace, metrics, request id, access log, limit, deadline, recover, then identity installs the actor and the admission gate rides innermost, so a refusal at the cap still carries the request id and lands in the access log like every other answer. The process entry below the composition owns the two listeners and the ordered shutdown: `SIGTERM`, which is what a container stop sends, raises into the join, the drain runs over the real server with readiness flipped first so the probe stops passing while in-flight requests finish, the metrics listener closes after the drain, and the exit code reports what the drain earned, the load chapter's rule carried to the process edge:

```
def require_secret():
    """the entry's own refusal: the vehicle never starts with the
    signing secret unset, because a service that mints bearers under
    an empty key is running a lock it never installed. build_app
    keeps the empty default so tests wire families explicitly, the
    refusal belongs to the entry alone."""
    if not os.environ.get("GOAPI_JWT_SECRET"):
        print("GOAPI_JWT_SECRET is required to start", file=sys.stderr)
        raise SystemExit(2)


def main():
    require_secret()
    app = build_app()
    server = app.make_server()
    host, _, port = (os.environ.get("GOAPI_DEBUG_ADDR") or METRICS_ADDR).rpartition(":")
    metrics_server, _ = app.metrics_listen((host, int(port)))
    serve = threading.Thread(
        target=server.serve_forever, name="pyapi-serve", daemon=True
    )
    serve.start()

    def term(signum, frame):
        raise KeyboardInterrupt

    signal.signal(signal.SIGTERM, term)
    try:
        serve.join()
    except KeyboardInterrupt:
        result = load.drain(server, app.gate, app.drainer, DRAIN_GRACE)
        try:
            metrics_server.server_close()
        except OSError:
            pass
        raise SystemExit(load.drain_exit_code(result))


if __name__ == "__main__":
    main()
```

listing 474: the process entry: the secret refusal, both listeners, the term raise, the drain with its reported exit code (source: python/api/main.py, lines 134-172)

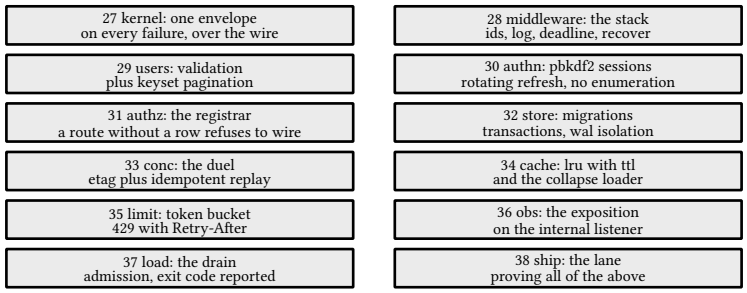


figure 195: the readiness table: twelve chapters, twelve properties, all proven

The parameter family is what transfers. `GOAPI_ADDR`, `GOAPI_VERSION`, `GOAPI_DB`, and `GOAPI_JWT_SECRET` are the same names in every lane's compose file, and `GOAPI_DEBUG_ADDR` is this book's own name for the metrics listener, the same role go's 6060 and js's 6090 play: the image reference and the host ports are the only edits between one lane's file and another's. The chapter declares what it does not carry: the ci workflow, the helm chart, and the terraform slice

are the go lane's teaching and the c# mirror, and the corpus keeps one executing lane per vehicle. Docker's storage model, the container runtime, and compose internals belong to book 13, ch 3 (infrastructure: docker, databases, and testing, containers: processes, signals, pid 1) and book 13, ch 4 (infrastructure: docker, databases, and testing, compose and one click local systems). What this chapter owns is the narrow band between a green module and a running service, and the suite chapter that follows hardens the whole set.

sources: the base image tag verified against the registry at hub.docker.com/v2/repositories/library/python/tags, `3.14.7-slim` present, accessed 2026-09-27, and the image facts read off the pulled image the same day, `python -V` answering 3.14.7, `id` answering uid 0 so the user is the lane's own, `curl` and `wget` both absent, the service measured at 388,740 bytes over the base by `docker image inspect`. Docker docs on copy semantics, healthchecks, `stop_grace_period`, and `docker compose up --wait` semantics at docs.docker.com, accessed 2026-09-27. The wrong-password figure is this lane's own access log over the composed stack. Verified by `make verify-pyapi` with its 313 daemon-free tests, `make verify-pyapi-docker` end to end with its five scenarios, and a run of the image without the secret whose behavior this chapter states.

39. the test suite

The service part ends with the service tested, and this chapter turns the suite around and examines it as its own artifact. book 8, ch 20 (python 3.14, testing with unittest) taught unittest's anatomy on samples, the fixture protocol, the assert inventory, subTest, and the exit code a programmatic run asserts, and every service chapter applied one technique to its own component. What is left is the discipline the whole suite runs on: hand-rolled doubles beside exactly one `mock.patch`, every deadline on an injected clock with `Event` and join gates instead of sleeps, invariants held by seeded `random.Random` loops because the stdlib ships no fuzz engine, a coverage lane over `trace` that stays opt-in, and the ecosystem's pytest, hypothesis, and time-machine taught as dated facts and declined on stated criteria. One rule governs everything: no third party is adopted, and every library the ecosystem reaches for by habit is taught as a dated fact instead.

39.1. what the suite holds

The inventory first, stated from the tree, not memory: 313 tests in 22 modules under `tests/`, the kernel and middleware suites, the users and authn families with their route files, authz, the store's seven, the duel over it, conc, cache, limit, obs, load, the platform pins, the vector spine, and the composition proof, the one file that pulls `main.build_app` into the process and answers every route through the full stack over a real sqlite file. The spine comes first because every family hangs from it: the 16 frozen golden vectors, byte-copied from the go lane's contract testdata and pinned by sha256, so drift in any lane's copy is a loud red in every lane. The platform pins hold the ground under all of it, the interpreter is 3.14 and the bundled sqlite is 3.50.4, measured facts, so a silent engine change fails a test instead of surprising a chapter's numbers. Two boundaries shape the count: the docker lane's wire replay sits in `integration/`, deliberately outside the discover glob, and is book 8, ch 38 (python 3.14, ship it)'s subject, and zero tests sleep and zero tests skip, both greppable facts about the tree:

```python
class VectorPinTests(unittest.TestCase):
    def test_the_directory_holds_exactly_the_frozen_sixteen(self):
        present = sorted(p.name for p in VECTORS.iterdir() if p.is_file())
        self.assertEqual(present, sorted(EXPECTED))

    def test_every_vector_answers_its_frozen_digest(self):
        for name, digest in sorted(EXPECTED.items()):
            with self.subTest(vector=name):
                data = (VECTORS / name).read_bytes()
                self.assertEqual(
                    hashlib.sha256(data).hexdigest(),
                    digest,
                    f"{name} drifted, the code or the copy is wrong",
                )
```

listing 475: the spine: sixteen vectors, each failing under its own name when a byte drifts (source: python/api/tests/test_contract_vectors.py, lines 37-50)

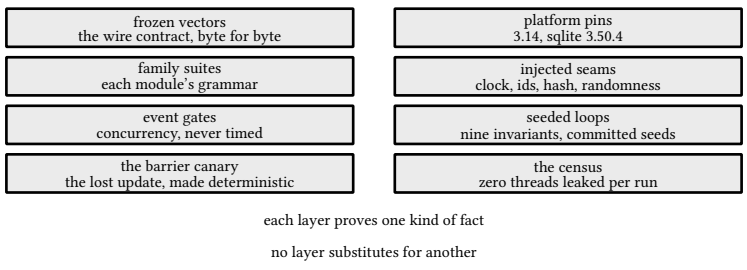


figure 196: the suite's layers, each against the one thing it proves

39.2. the unittest surface

The unit is `unittest.TestCase`, the loader collects one fresh instance per `test_` method, and book 8, ch 20 (python 3.14, testing with unittest) measured that interleaving, so this section counts what the vehicle does with the surface. The assert inventory as used: 16 distinct methods plus `self.fail` across 730 call sites, `assertEqual` alone two thirds of them, the context manager form of `assertRaises` extended with `as got` into the caught error's own status and code, `assertAlmostEqual` with `delta=` where float accumulation is the honest tolerance, the identity family `assertIs` and `assertIsNone` for sentinel answers where == would accept a falsy stand-in, and `self.fail` in the `else` branch of a `try` that should have raised. `subTest` carries the parametric grids, 27 of them, a green grid counts once and a broken one names the input that broke it. Fixtures pair 17 `setUp` with 13 `tearDown`, and `addCleanup` appears 12 times where order is the contract: cleanups run LIFO and run even when `setUp` fails partway through, which is why the kernel's socket fixture arms its three teardown steps in reverse the moment the thread starts, not in a `finally` a later line might never reach:

```
class SocketAdapterTests(unittest.TestCase):
    def serve(self, app):
        server = app.make_server()
        thread = threading.Thread(target=server.serve_forever, daemon=True)
        thread.start()
        # cleanups run LIFO: stop the loop, close the listener, join
        self.addCleanup(thread.join)
        self.addCleanup(server.server_close)
        self.addCleanup(server.shutdown)
        return server.server_address
```

listing 476: the listener fixture: three cleanups armed at birth, LIFO at death (source: python/api/tests/test_kernel.py, lines 365-374)

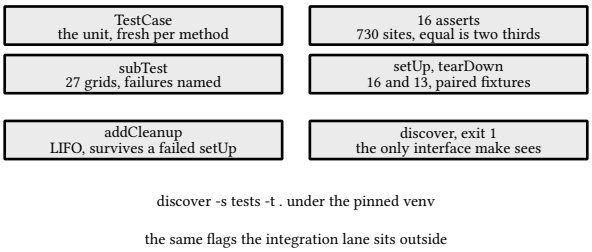


figure 197: the surface against what the vehicle uses it for

39.3. doubles and unittest.mock

The doubles are hand-rolled and live beside the tests they serve. `make_request` and `envelope_of` build and read the kernel's plain shapes, `recorded` and `terminal` stamp an ordered box for chain tests, and `CapturedLog` wires the production `JsonLogHandler` over a `StringIO`, so a test reads whole structured lines back through the real formatter rather than a fake that would only test itself. The strict shape is `ScriptedClient` in the load tests: one planned pair per request index, every call recorded, an unplanned index an immediate failure. `unittest.mock` appears at exactly one site in the whole suite, `mock.patch.dict` over `os.environ` for the `GOAPI_DB` name in the store's wire test, the one dependency python code reads from process ambient instead of an argument, and `patch.dict` is the honest tool there because it swaps a mapping's contents and restores them on exit, no attribute spec to keep honest. The composition test needs the same seam and spells it by hand, setting the variable and restoring it in an `addCleanup`, which is the pair worth seeing side by side: one ambient dependency, two restorations, no mocks anywhere else. `MagicMock` and `patch.object`, the ecosystem's reflex, are unused for a structural reason: every seam this service defines is an injected argument, `users.Family(now=, new_id=, hash_password=)`, `Limiter(now=)`, and `access_log(logger, clock=)`, so a double is a value the test builds whole, never a loader patch, and the spec discipline `Mock(spec=...)` sells against attribute typos is the interpreter's own `AttributeError` when the double is a real class, because `FakeClock` is its own spec:

```
def recorded(name, box):
    return lambda next: lambda request: (box.append(name), next(request))[1]


def terminal(box, response=None):
    def handler(request):
        box.append("handler")
        return response or kernel.success({"ok": True})

    return handler


class CapturedLog(io.StringIO):
    """a logger wired to the hand rolled json handler over a buffer,
    so a test reads whole structured lines back."""

    def __init__(self):
        super().__init__()
        handler = middleware.JsonLogHandler(self)
        self.logger = logging.getLogger(f"pyapi.test.{id(self)}")
        self.logger.handlers = [handler]
        self.logger.setLevel(logging.INFO)
        self.logger.propagate = False

    def lines(self):
        self.seek(0)
        return [json.loads(line) for line in self.read().splitlines()]
```

listing 477: the hand-rolled half: a recorder, a terminal, and the production log handler over a buffer (source: python/api/tests/test_middleware.py, lines 26-52)

the job	the vehicle's answer	the ecosystem's habit
counting, recording	a box, a calls list	MagicMock, call_count
the ambient seam	patch.dict(os.environ)	monkeypatch, pytest-mock
spec discipline	real classes, own spec	Mock(spec=...)
strictness	the scripted fake	expectation apis

figure 198: four double shapes against the job each does

39.4. clocks, events, zero sleeps

The rule is absolute and greppable: no test anywhere sleeps, the string `.sleep(` appears nowhere under `tests/`, and the word itself lives only in docstrings, the limiter's own stating the rule in one line, "a rate limiter tested against time.sleep is tested against a scheduler", and in the load harness's injected `sleep_until` seam, which its test replaces with a recorder that answers instantly. Every deadline runs on an injected clock: `FakeClock` moves by arithmetic, `SteppedClock` steps by whole ticks, and the middleware tests drive the access log's duration from an iterator of two planned reads, so 0.25 is a stated fact about the seam, not a measurement of the scheduler. Concurrency is gated, never timed. The deadline expiry test parks the handler on a `gate Event`, the layer answers 503 and drops the late response, and `produced.wait(5)` then proves the abandoned handler finished anyway, a fence with a bound, loud if the fact never happens. The drain test gates on the drainer's `flipped Event`, polls fresh dials until the listener refuses, which is real socket state, and joins with a bound. The expired grace is `wait_for` at a zero timeout, one predicate look, no waiting at all. The conc runner asserts the second caller is alive while it is parked on the first caller's handoff event, in that file's words "a fact about blocking, not a timing probe", and the `Barrier` rendezvous in the pool, duel, and canary tests makes both writers hold the same stale read before either writes:

```
    def test_expiry_answers_overload_and_drops_the_late_response(self):
        gate = threading.Event()
        produced = threading.Event()

        def slow(request):
            gate.wait()
            produced.set()
            return kernel.success({"late": True})

        response = middleware.deadline(0.0, clock=lambda: 100.0)(slow)(make_request())
        self.assertEqual(response.status, 503)
        self.assertEqual(json.loads(response.body)["error"]["code"], "overload")
        self.assertNotIn("late", response.body.decode())
        gate.set()
        self.assertTrue(produced.wait(5), "the abandoned handler never finished")
```

listing 478: the expiry: gated on an event, the late answer dropped, the handler proven finished (source: python/api/tests/test_middleware.py, lines 200-214)

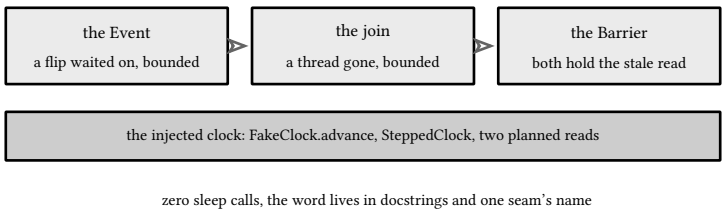


figure 199: the three gates, none of them a clock

39.5. properties as seeded loops

The go lane has a fuzz engine and ships seeds that ride the plain gate, node has neither, and python is in node's camp: the stdlib has no mutation search, no corpus directory, no coverage-guided anything. The vehicle's answer is the seeded loop. A property is a sentence that must hold for every input, `random.Random` with a committed seed generates the inputs, the invariant is named in the file beside the seed, and a failure replays exactly, because the seed is the case. The suite holds nine loops: route order irrelevance on seed 2026, the bucket spend bound on 20261001, the authz verdict grid on 3131, the cache capacity bound on 5501 and the hand-rolled versus fused parity on 5502, etag determinism on 3301 and the conditional round trip on 3302, the sql keyset walk against a sorted walk on 4402, and the cursor walk that covers the table exactly once over shuffled insertions on 2925. The bucket loop is the cleanest demonstration, its invariant is its arithmetic, the draws allowed can never exceed capacity plus refill, held over 200 rounds of 500 draws with the clock advanced by random fractions, and the honest boundary is stated with it: a seeded loop is a fixed stream, not a search, which is why the seeds are committed like the vector bytes, the replay is the contract:

```
    def test_spent_tokens_never_exceed_capacity_plus_refill(self):
        rng = random.Random(20261001)
        for _ in range(200):
            clock = FakeClock()
            bucket = limit.Bucket(5, 1.0, clock())
            allowed = 0
            for _ in range(500):
                if bucket.allow(clock()).allowed:
                    allowed += 1
                clock.advance(rng.random() * 0.25)
            elapsed = clock() - T0
            self.assertLessEqual(allowed, 5 + elapsed + 1e-9)
```

listing 479: the spend bound as a property: allowed never exceeds capacity plus refill, seed committed (source: python/api/tests/test_limit.py, lines 93-104)

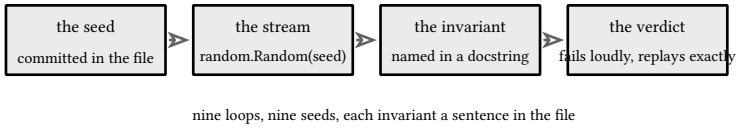


figure 200: seed to stream to invariant to verdict, the python answer to fuzzing

39.6. dated and declined

The ecosystem's testing stack is dated here, named, and declined, and the criteria are stated so the choice reads as a decision rather than a habit. pytest, at 9.1.1 on PyPI the day of this writing, offers fixtures, `parametrize`, assert rewriting through a bytecode hook, and a plugin graph. hypothesis, at 6.168.1, offers composable strategies, shrinking that walks a failure down to its minimal form, and a database of failures. time-machine, at 3.5.1, patches `time.time` and `datetime.now` at the C level so a global clock becomes fake-able. Each fails the vehicle's adoption criteria on its own ground: pytest's runner would add a dependency to a gate whose whole design is that the interpreter alone runs it, and the surface it sells is already in use, `TestCase` is the fixture, `subTest` the parametrize, and the named assert methods already report their operands, which is all assert rewriting would improve. hypothesis earns its keep against generator complexity a hand-rolled loop cannot express, and these shapes are flat, `randbytes` over a length, `randrange` over a dozen keys, `choice` over four actor kinds. time-machine exists for code that reads a global clock, and this vehicle's modules take `now` as an injected callable, so the fake clock is an argument and there is nothing global to patch. The same accounting covers the framework lane: fastapi's `TestClient` story belongs to book 8, ch 22 (python 3.14, fastapi) and is declined there on the same rule. The verdict grid below is the loop hypothesis would own, 200 seeded actor picks against every policy, admin passes everywhere, anonymous passes only the open set, no code outside the 401/403 pair, and an admin denial fails the test by name:

```
    def test_the_grid_is_stable_under_seeded_actor_populations(self):
        # invariant: an admin passes every route, an anonymous caller
        # passes only the open ones, and no actor kind ever sees a code
        # outside the 401/403 pair.
        rng = random.Random(3131)
        codes = set()
        for _ in range(200):
            kind = rng.choice(["anon", "user", "admin", "owner"])
            a = {
                "anon": None,
                "user": actor(("user",)),
                "admin": actor(("admin", "user")),
                "owner": actor(("user",), uid=ADA_ID),
            }[kind]
            for pattern, policy in authz.ROUTE_POLICIES.items():
                values = {"id": ADA_ID, "role": "admin"}
                denial = policy(a, values)
                if denial is None:
                    if kind == "anon":
                        self.assertIn(pattern, self.OPEN_ROUTES)
                    continue
                codes.add(denial.code)
                self.assertIn(denial.code, ("unauthorized", "forbidden"))
                if kind == "admin":
                    self.fail(f"admin denied on {pattern}: {denial}")
        self.assertEqual(codes, {"unauthorized", "forbidden"})
```

listing 480: the verdict grid: seeded actor populations against every policy, seed 3131 (source: python/api/tests/test_authz.py, lines 154-179)

the axis	what it offers	why the vehicle declines
pytest 9.1.1	fixtures, parametrize	the surface is already in use
hypothesis 6.168.1	strategies, shrinking	flat shapes, the seed replays
time-machine 3.5.1	the c-level clock patch	now is an argument
dependency cost	one module, or three	zero, the interpreter runs the gate

figure 201: the declined column against what each would have owned

39.7. coverage, opt in

Coverage answers what the suite executes, and the honest lane is opt-in exactly like the go and `c#` lanes, never inside `make verify`, because a percentage measures the machine and the tree, not the code. The mechanism is the stdlib `trace` module and one invocation: the lane runs `unittest discover` itself under `--module`, counting executed lines while the suite passes, and `trace`'s runctx arms `threading.settrace` beside `sys.settrace`, read from the module's own source, so worker threads the suite spawns count too. `'$prefix'` is `trace`'s spelling for the stdlib directory, the one ignore that scopes the report to the vehicle tree, `--file` accumulates counts under `.cover/` with one `.cover` file per module, and `--summary` prints the table once. Two boundaries keep it honest, both probed on the pinned 3.14.7. The composition test pulls `build_app` into the process, but the serving loop, the SIGTERM-to-drain chain, and the exit code run only as a real process in the docker lane, so the report covers the suite's own execution and the entrypoint's runtime half stays on the wire. And `trace`'s own main swallows the traced program's `SystemExit`, a red suite would still exit 0, so the lane requires the run's verdict line instead of the exit code, coverage of a red run is not a report. No percentage appears in this book: the number moves with every edit, and a pinned number would be a lie with a date on it:

```
# verdict line is required below instead of the exit code. main.py's
# wiring and the docker lane's wire replay stay outside this report,
# it covers what the in-process suite executes. run from
# books/python/api, mirroring the go lane's api/cover.sh.
set -e
hash=$(sha256sum ../requirements.txt | cut -c1-12)
py=../../../tools/python/build/venv-$hash/Scripts/python.exe
if [ ! -x "$py" ]; then
  echo "pyapi-cover: shared venv venv-$hash not built under tools/python/build." >&2
  echo "run make verify-pyapi first (it builds the venv), then rerun: make pyapi-cover" >&2
  exit 1
fi
rm -rf .cover
mkdir -p .cover
"$py" -X utf8 -m trace --count --summary --file=.cover/counts \
  --coverdir=.cover --ignore-dir '$prefix' \
  --module unittest discover -s tests -t . 2>&1 | tee .cover/run.log
```

listing 481: the lane: venv resolved, suite traced, the verdict line required (source: python/api/cover.sh, lines 13-29)

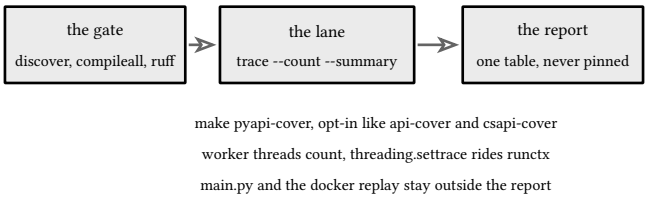


figure 202: the gate against the opt-in lane

39.8. what the suite proves

The closer maps each layer to the fact it establishes, the service part's readiness table one chapter later. The vectors prove the wire contract byte for byte across every lane, the platform pins prove the interpreter and the bundled engine, the family suites prove each module's grammar, the injected seams prove determinism, ids and hashes and clocks supplied by the test, the event gates prove the concurrency windows close without one millisecond of waiting, the seeded loops prove nine invariants under their committed streams, and the census proves the harness leaks no thread per run. The canary is python's honest answer to a missing tool: cpython ships no race detector, so the suite forces the interleave with a barrier and turns the lost update into a deterministic fact, the unsynchronized twin answers 1 and the locked twin answers 2, which proves the hazard the lock exists for rather than the detector go's lane runs. The suite's last proof is its own exit: the process ends, daemon threads cannot hide inside it, the census already named anything left, and `make` reads 0 or 1, the one number the whole chain exists to produce. Zero sleeps, zero skips, contract minimums, not preferences:

```
    def test_a_run_leaks_no_worker_threads(self):
        before = list(threading.enumerate())
        client = ScriptedClient([(200, 0.001)] * 12)
        load.Harness(client, FakeClock()).closed_loop(3, 10, warmup=2)
        self.assertEqual(load.thread_census(before), [])
```

listing 482: the census: the thread list before and after, the difference must be empty (source: python/api/tests/test_load.py, lines 213-217)

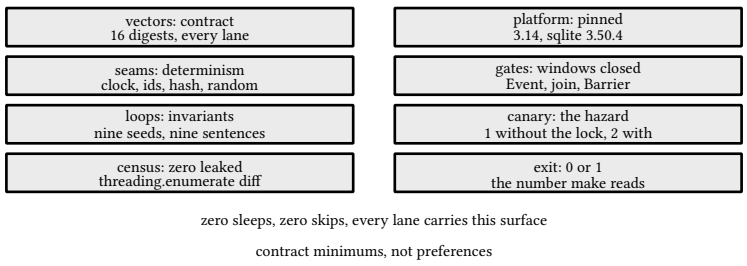


figure 203: the layer to property to where proven grid, closing the part

sources: docs.python.org, the 3.14 library pages for unittest (TestCase loading, subTest, the assert methods, and discover), unittest.mock (patch.dict and the spec argument), and trace (the command line, the `'$prefix'` spelling, and the runctx hooks), accessed 2026-09-27, the trace facts verified against the pinned cpython 3.14.7 locally: the lane run over the whole suite, the worker-thread hook read from the module source, and the swallowed SystemExit probed with a deliberately red run. pypi.org/project for pytest at 9.1.1, hypothesis at 6.168.1, and time-machine at 3.5.1, accessed 2026-09-27, deliberately not adopted. Verified by the vehicle's suite, 313 tests under `make verify-pyapi` with zero sleeps and zero skips, and by `make pyapi-cover` green over the same tree.

40. capstone: ingest and analysis pipeline, part 1

Chapters 1 through 26 taught pieces. This chapter and the next assemble them into one working program under `books/python/capstone/`: 9 modules and a unittest suite of 78 tests that the verify gate runs as its capstone leg, the same gate that has counted ok lines for every sample since chapter 1. The program ingests sensor readings in jsonl form. Named async line iterators go in, and two lists come out: records validated at a pydantic boundary, and dead letters carrying a reason from a closed three-word vocabulary. Part 1 owns ingest and validation, `sources.py`, `fetch.py`, `models.py`, `pipeline.py`, with 48 of the 78 tests. Part 2, book 8, ch 41 (python 3.14, capstone: ingest and analysis pipeline, part 2), owns aggregation, the canonical report, the cli, and the served api.

40.1. the capstone contract

The contract is one sentence: every raw line that enters the pipeline leaves as either a validated `Reading` in `records` or a `DeadLetter` in `dead`, and the only other outcome is `PipelineTimeout`, which is the pipeline refusing to hang. A malformed line is a payload with a destination, never a crash. The entry point states the whole surface:

```
async def run(
    sources: Mapping[str, AsyncIterator[str]],
    queue_size: int = 8,
    consumers: int = 2,
    join_timeout: float = 30.0,
) -> PipelineResult:
    # drain every source to completion and return the split of good
    # records against dead letters. records is appended to only inside
    # consumers, all on the one event-loop thread, so a plain list is
    # safe without a lock. join_timeout bounds the graceful shutdown:
    # exceeding it cancels every task and raises PipelineTimeout.
    raw: asyncio.Queue = asyncio.Queue(maxsize=queue_size)
    dlq: asyncio.Queue = asyncio.Queue()
    result = PipelineResult()
    producers = [
        asyncio.create_task(_produce(name, lines, raw, dlq))
        for name, lines in sources.items()
    ]
    consumer_tasks = [
        asyncio.create_task(_consume(raw, dlq, result.records))
        for _ in range(consumers)
    ]
```

listing 483: the run signature: sources in, defaults for the queue, the consumers, and the join budget (source: python/capstone/pipeline.py, lines 52-73)

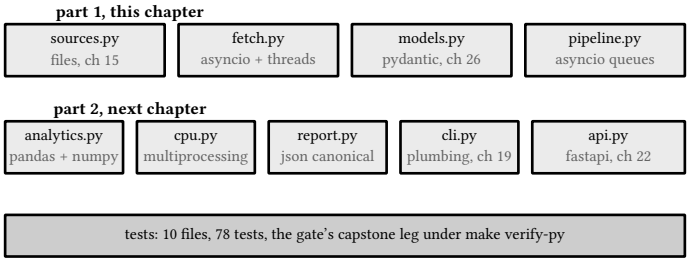
Three numbers are load bearing. `queue_size` bounds the raw queue at 8 by default, so memory is bounded by construction and a slow consumer slows producers down instead of being buried. `consumers` sets 2 validation workers. `join_timeout` caps the graceful drain at 30 seconds, and exceeding it cancels every task and raises. The records list needs no lock because only consumers append to it, and every consumer runs on the one event-loop thread.

Every module is a pillar chapter assembled. The adapters read files the way book 8, ch 15 (python 3.14, files and serialization) taught, `encoding=` named on every open. The fetch adapter runs blocking stdlib urllib inside worker threads through `asyncio.to_thread`, the offload of book 8, ch 21 (python 3.14, asyncio) meeting the thread discipline of book 8, ch 17 (python 3.14, threads and the gil). The boundary models are the pydantic of book 8, ch 26 (python 3.14, pydantic). The queue choreography is the asyncio queue machinery from the same asyncio chapter, with the sentinel arithmetic this section of the capstone had to get right on its own. Part 2 aggregates with the pandas of book 8, ch 25 (python 3.14, pandas) and the numpy of book 8, ch 24 (python 3.14, numpy), offloads through the

process pool patterns of book 8, ch 18 (python 3.14, multiprocessing and process patterns), parses its command line with the argparse and logging of book 8, ch 19 (python 3.14, operational plumbing), and serves with book 8, ch 22 (python 3.14, fastapi).

NOTE: built test first, module by module

The order of work followed book 8, ch 20 (python 3.14, testing with unittest): the failing test existed before the module it names, and each commit carried the red test and the green implementation together. The closing callout of this chapter is the resulting ledger, module by module with its count, so a reader can check the inventory against the suite output the way every chapter's ok count is checked against the gate.



the toolchain of chapter 1 underneath everything: pinned interpreter, venv, -X utf8, red on any failure

figure 204: the nine modules and the pillar chapter each one assembles

40.2. sources and fetch

Two adapters turn the outside world into lines, and both keep their promises small enough to test. `tail` is a snapshot: it yields every complete line present and stops, and a trailing partial line is held back rather than completed with an invented newline. `follow` is watch mode: catch up, then poll for appends until the stop event is set, with one final read draining lines that landed before stop won. A partial line stays buffered across polls until more bytes finish it. Rotation is out of scope, stated in the module header, and the shrink bound stands in for it: if the file becomes shorter than what was already consumed, the stream ends instead of reading from a stale offset.

```
def _complete_lines(buffer: str) -> tuple[list[str], str]:
    # split on \n, keep the tail (no newline yet) as the new buffer.
    # a lone trailing \r is stripped so crlf logs and lf logs agree.
    lines = buffer.split("\n")
    partial = lines.pop()
    return [line.rstrip("\r") for line in lines], partial


async def tail(path) -> AsyncIterator[str]:
    # snapshot: everything complete, then stop. short reads on a local
    # file are cheap; the to_thread offload is reserved for network io
    # in fetch.py.
    buffer = ""
    with open(path, "r", encoding="utf-8", newline="") as handle:
        while chunk := handle.read(CHUNK):
            buffer += chunk
            lines, buffer = _complete_lines(buffer)
            for line in lines:
                yield line
            await asyncio.sleep(0)  # stay cooperative on large files
```

listing 484: complete-line splitting and the tail snapshot, a partial line never delivered (source: python/capstone/sources.py, lines 19-38)

The splitting helper is the whole line discipline: split on \n, keep the tail without a newline as the new buffer, strip a lone trailing \r so crlf logs and lf logs agree. The open uses `newline=""` so the \r survives to be stripped here rather than being translated underneath, the chapter 15 habit. Reads run in 4096-character chunks with `asyncio.sleep(0)` between them, so a large file cannot monopolize the loop.

The network sibling is `fetch.py`, and it starts from a boundary the fastapi chapter stated: the book teaches no http client library, so the client side is stdlib `urllib.request`. Blocking socket work belongs off the event loop, so the actual call runs inside a worker thread via `asyncio.to_thread`, and every failure crosses back as one of four typed exceptions:

```
class FetchError(Exception):
    # base: this url did not turn into text
    def __init__(self, detail: str):
        super().__init__(detail)
        self.detail = detail


class FetchTimeout(FetchError):
    """the socket timeout fired before the body arrived"""


class FetchStatus(FetchError):
    """the server answered, but not with 200"""

    def __init__(self, status: int):
        super().__init__(f"http status {status}")
        self.status = status


class FetchDecode(FetchError):
    """the body is not utf-8"""


class FetchUnreachable(FetchError):
    """no answer at all: refused, no dns, no route"""
```

listing 485: the typed exception taxonomy: timeout, status, decode, unreachable, over one base (source: python/capstone/fetch.py, lines 13-37)

```
async def fetch_text(url: str, timeout: float = FETCH_TIMEOUT) -> str:
    try:
        status, payload = await asyncio.to_thread(_open, url, timeout)
    except urllib.error.HTTPError as exc:
        exc.close()  # it is response-like: release it before rewriting
        raise FetchStatus(exc.code) from exc
    except urllib.error.URLError as exc:
        if isinstance(exc.reason, TimeoutError):
            raise FetchTimeout(str(exc.reason)) from exc
        raise FetchUnreachable(str(exc.reason)) from exc
    except TimeoutError as exc:
        # read timeouts surface bare; connect timeouts arrive wrapped
        raise FetchTimeout(str(exc) or "timed out") from exc
    if status != 200:
        raise FetchStatus(status)
    try:
        return payload.decode("utf-8")
    except UnicodeDecodeError as exc:
        raise FetchDecode(str(exc)) from exc
```

listing 486: the adapter body: to_thread for the blocking call, every urllib error rewritten (source: python/capstone/fetch.py, lines 48-66)

The mapping has one subtlety per branch. `HTTPError` is response-like, so it is closed before being rewritten, and its status code rides into `FetchStatus`. Connect timeouts arrive wrapped in `URLError` while read timeouts surface bare, and both become `FetchTimeout`. A non-200 status that reached the return without raising still fails, and a body that is not utf-8 becomes `FetchDecode` instead of a decode traceback. Nothing in this module ever leaks a raw urllib error into the pipeline.

The proof that the adapter works against real network io is exactly one suite in the capstone, and it says so in its first line:

```
class RealSocketTest(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        cls.server = ThreadingHTTPServer(("127.0.0.1", 0), Handler)
        cls.port = cls.server.server_address[1]
        cls.thread = threading.Thread(target=cls.server.serve_forever, daemon=True)
        cls.thread.start()

    @classmethod
    def tearDownClass(cls):
        cls.server.shutdown()
        cls.server.server_close()

    def url(self, path="/"):
        return f"http://127.0.0.1:{self.port}{path}"

    def test_fixture_bytes_round_trip(self):
        text = run(fetch.fetch_text(self.url()))
        self.assertEqual(text, REMOTE.decode("utf-8"))

    def test_lines_split_the_document(self):
        lines = run(fetch.fetch_lines(self.url()))
        self.assertEqual(len(lines), 2)
        self.assertEqual(lines[0], '{"sensor": "zeta", "ts": 3000, "value": 6.0}')

    def test_source_adapter_yields_lines(self):
        async def collect():
            return [line async for line in fetch.fetch_source(self.url())]

        self.assertEqual(len(run(collect())), 2)

    def test_read_timeout_is_fetchtimeout(self):
        with self.assertRaises(fetch.FetchTimeout) as caught:
            run(fetch.fetch_text(self.url("/slow"), timeout=0.3))
        self.assertIsInstance(caught.exception, fetch.FetchError)

    def test_404_is_fetchstatus(self):
        with self.assertRaises(fetch.FetchStatus) as caught:
            run(fetch.fetch_text(self.url("/missing")))
        self.assertEqual(caught.exception.status, 404)

    def test_bad_utf8_is_fetchdecode(self):
        with self.assertRaises(fetch.FetchDecode):
            run(fetch.fetch_text(self.url("/binary")))
```

listing 487: the real-socket proof: a threaded http.server on an ephemeral loopback port, six checks (source: python/capstone/tests/test_fetch.py, lines 54-97)

The server is a `ThreadingHTTPServer` bound to `127.0.0.1` on port 0 in a daemon thread, started once per class in `setUpClass` and shut down in `tearDownClass`. Six checks run against it: the fixture bytes round trip as text, the document splits into lines, the async source adapter yields them, a handler that sleeps a full second against a 0.3 second client timeout proves `FetchTimeout` on a genuine stalled socket, a 404 proves `FetchStatus`, and a body of raw non-utf-8 bytes proves `FetchDecode`. The suite's other six checks never open a socket at all: they monkeypatch `urllib.request.urlopen` with side effects and assert the same taxonomy, the mock seam of chapter 20 aimed at the one boundary worth faking.

PITFALL: a loopback listener still pops the windows firewall dialog

> Measured on this box and recorded in this repo's `tools/net-quiet.ps1`: with the firewall's notify-on-listen setting on, even a listener bound to 127.0.0.1 on an ephemeral port raises the allow-access dialog, and a popped dialog stalls an unattended gate run until someone clicks it. `make verify` therefore runs its network preflight before any suite, `tools/test-net-preflight.ps1`, asserting `NotifyOnListen` is off on every profile, the setting microsoft documents as `Set-NetFirewallProfile -NotifyOnListen False`. Loopback traffic was never filtered, so the suite keeps working with the setting off, and the one real-socket test runs unattended or the gate reports the drift loudly.

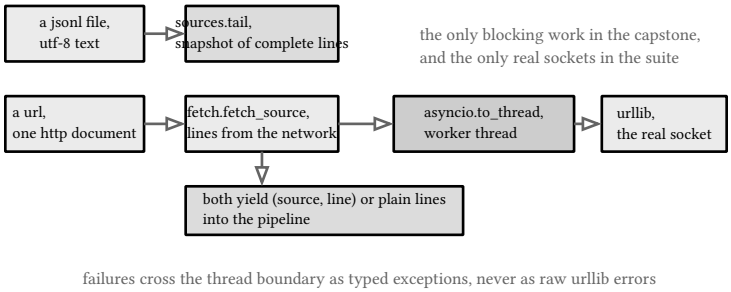


figure 205: two adapters, one contract: lines tagged with a source name, and where the thread hop happens

40.3. pydantic at the boundary

Every raw line meets pydantic exactly once, in `parse_line`, and the good payload that survives is one small model:

```
class Reading(BaseModel):
    sensor: str = Field(pattern=r"^[a-z][a-z0-9_]{0,31}$")
    ts: int = Field(ge=0, le=4102444800)  # unix seconds, bounded at 2100
    value: float = Field(allow_inf_nan=False)


# what a reject looks like on the dead-letter queue
class DeadLetter(BaseModel):
    source: str
    reason: str  # one of REASONS
    detail: str
    line: str
```

listing 488: the Reading contract and the DeadLetter shape, the pipeline's two currencies (source: python/capstone/models.py, lines 16-27)

The constraints are the pipeline's data contract. A sensor name matches `^[a-z][a-z0-9_]{0,31}$`, so names are lowercase, bounded, and safe as report keys. A timestamp is an integer between 0 and 4102444800, unix seconds bounded at the start of 2100, so a garbage clock cannot smuggle in an arbitrary int. A value is a float with `allow_inf_nan=False`, and that last flag carries weight: `json.loads` happily parses the `NaN` literal as valid json, so the poison float can arrive through a perfectly legal document, and the boundary is where it stops. Extra fields are ignored, an int value coerces to float, both pinned by tests. A reject is data too: a `DeadLetter` carries the source name, one reason from the closed tuple `("schema", "field", "source")`, a detail string, and the offending line verbatim. The vocabulary is closed by an explicit test, so adding a fourth reason means changing the contract test first, never silently.

```python
def parse_line(source: str, line: str) -> Reading:
    # turn one raw line into a Reading, or raise RecordRejected
    text = line.strip()
    if not text:
        raise _reject(source, line, "schema", "empty")
    try:
        payload = json.loads(text)
    except json.JSONDecodeError as exc:
        raise _reject(source, line, "schema", "JSONDecodeError") from exc
    if not isinstance(payload, dict):
        raise _reject(source, line, "schema", "not-object")
    try:
        return Reading.model_validate(payload)
    except ValidationError as exc:
        # loc:type pairs only: pydantic's prose messages embed input
        # reprs, which would make the golden report unstable
        detail = "|".join(
            "{}:{}".format(".".join(str(part) for part in err["loc"]), err["type"])
            for err in exc.errors()
        )
        raise _reject(source, line, "field", detail) from exc
```

listing 489: parse_line: three schema rejects, then pydantic, then one field reject (source: python/capstone/models.py, lines 43-63)

Three ways to fail before pydantic even runs: an empty line, a line that is not json, and json that is not an object, all reason `schema`. Then `Reading.model_validate` gets its chance, and a `ValidationError` becomes reason `field` with a detail built from `loc:type` pairs joined by bars, `ts:int_parsing`, `value:missing`. The consumers see none of this machinery: rejection travels as `RecordRejected`, an exception carrying the `DeadLetter`, so a raw pydantic error never escapes the boundary.

PITFALL: pydantic's prose error messages are not stable data

A `ValidationError` message embeds the input value's repr, so the dead-letter detail for a bad timestamp would contain whatever the broken line happened to say. Part 2 commits the report to a golden file diffed byte for byte, and input reprs would make that diff unstable against fixture edits. The `loc:type` pairs keep the detail a closed-form summary of what was rejected and why, stable enough to pin, informative enough to debug from.

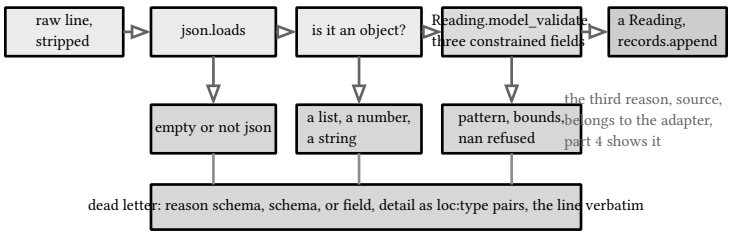


figure 206: one line through the boundary: the checks in order and the three reject slots

40.4. the queue pipeline

The middle of the capstone is the part asyncio chapters warn about: producers, consumers, a bounded queue between them, and a shutdown that must drain everything exactly once.

```
async def _produce(name, lines, raw, dlq):
    try:
        async for line in lines:
            await raw.put((name, line))
    except Exception as exc:
        # the adapter itself failed (fetch error, unreadable file):
        # one dead letter for the whole source, the producer ends, the
        # pipeline lives on. nothing escapes as a crash.
        await dlq.put(
            models.DeadLetter(
                source=name, reason="source", detail=type(exc).__name__, line=""
            )
        )


async def _consume(raw, dlq, records):
    while True:
        item = await raw.get()
        if item is _DONE:
            return
        source, line = item
        try:
            records.append(models.parse_line(source, line))
        except models.RecordRejected as rejected:
            await dlq.put(rejected.dead)
```

listing 490: a producer per source and the consumer loop, rejects routed to the dead-letter queue (source: python/capstone/pipeline.py, lines 25-49)

A producer iterates its adapter and puts `(name, line)` pairs on the raw queue. When the adapter itself fails, a fetch timeout, an unreadable file, the failure becomes one dead letter with reason `source` and the exception's type name as its detail, the producer ends, and the pipeline lives on. A consumer takes an item, runs `parse_line`, and appends either a `Reading` or, when `RecordRejected` arrives, its dead letter to the dlq. The bounded queue is backpressure: `await raw.put(...)` suspends the producer when the queue is full, so a slow consumer slows ingestion down, and no line is ever dropped or buffered without bound. The tight-queue test pins the extreme:

```
    def test_records_survive_a_tight_queue(self):
        # queue_size=1 is maximal backpressure: the producer's put
        # awaits on every full queue, and every line still arrives
        lines = (FIXTURES / "events-a.jsonl").read_text(encoding="utf-8").splitlines()
        result = run_pipeline({"tight": pipeline.entry(lines)}, queue_size=1)
        self.assertEqual(len(result.records), 8)
        self.assertEqual(result.dead, [])
```

listing 491: queue_size 1 is maximal backpressure, and all 8 lines still arrive (source: python/capstone/tests/test_pipeline.py, lines 56-63)

Shutdown is where the naive design dies, so the capstone does it in two phases with sentinels counted against consumers:

```
    async def phases():
        # two-phase graceful shutdown: sentinels are counted against
        # consumers, not producers, so no consumer can starve another
        # by swallowing two sentinels. producers join first (every line
        # enqueued), then exactly one stop per consumer, then consumers
        # drain what precedes their stop and exit.
        await asyncio.gather(*producers)
        for _ in consumer_tasks:
            await raw.put(_DONE)
        await asyncio.gather(*consumer_tasks)

    tasks = producers + consumer_tasks
    try:
        await asyncio.wait_for(phases(), timeout=join_timeout)
    except TimeoutError:
        for task in tasks:
            task.cancel()
        await asyncio.gather(*tasks, return_exceptions=True)
        raise PipelineTimeout(
            f"pipeline did not drain within {join_timeout}s"
        ) from None
    while not dlq.empty():
        result.dead.append(dlq.get_nowait())
    # stable order for the report and the golden test, whatever order
    # the consumers happened to reject in
    result.dead.sort(key=lambda d: (d.source, d.reason, d.detail, d.line))
    return result
```

listing 492: the two-phase drain, the bounded join, and the stable dead-letter order (source: python/capstone/pipeline.py, lines 75-101)

Phase one joins the producers, which means every line is already on the queue or the producers are dead. Only then does the main task put exactly one `_DONE` sentinel per consumer, and each consumer drains what precedes its own stop and exits. The whole `phases()` coroutine runs under `asyncio.wait_for` with the join budget, and a timeout cancels every task, gathers them with `return_exceptions=True` so cancellation does not itself raise, and raises `PipelineTimeout` from `None`, a clean failure with no misleading cause chain. After a clean drain, the dlq is emptied into the result and the dead letters are sorted by `(source, reason, detail, line)`, the same key part 2's report will use, so the output order is a contract rather than whichever consumer happened to reject first.

PITFALL: one sentinel per producer starves a consumer

The common shutdown puts a sentinel on the queue for each producer and lets each consumer exit on the first sentinel it draws. With 2 producers and 2 consumers, nothing stops consumer A from drawing both sentinels: A exits, B blocks on `get()` forever, and the join on B never returns. The bug is arithmetic, not timing, so no test interleaving reliably catches it. Counting sentinels against consumers instead, after the producers have joined, makes it structurally impossible: every consumer is guaranteed exactly one stop, and every item ahead of that stop is real work.

```
    def test_join_timeout_cancels_and_raises(self):
        # a producer that never ends: the bounded join cancels every
        # task and raises PipelineTimeout instead of hanging forever
        async def stuck():
            while True:
                await asyncio.sleep(10)
                yield "never"  # pragma: no cover

        with self.assertRaises(pipeline.PipelineTimeout):
            run_pipeline({"stuck": stuck()}, join_timeout=0.2)
```

listing 493: a producer that never ends: the bounded join cancels and raises instead of hanging (source: python/capstone/tests/test_pipeline.py, lines 141-150)

The stuck generator never yields, the join budget of 0.2 seconds expires, and the test asserts `PipelineTimeout` rather than a hang. Cancellation is the last resort behind the graceful path, and the watch-mode shutdown test exercises the graceful path end to end: a `follow` source drains its late append, the stop event ends the producer, and `run()` returns with both records.

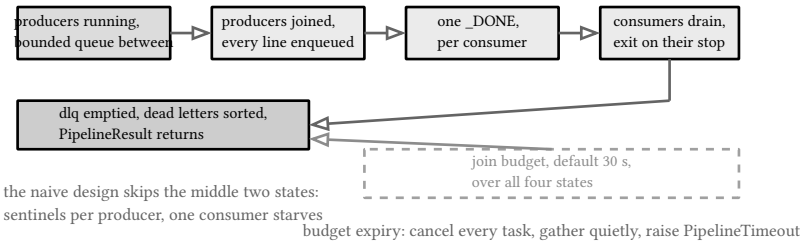


figure 207: the drain as a state machine: join, sentinel per consumer, exit, and the budget over all of it

40.5. profiled: the ingest pipeline

The pipeline now measures itself. The in-suite check runs 3000 synthetic lines through `run()` under `cProfile` and asserts the shape of the profile rather than any timing: the top capstone-owned row by `tottime` is `_consume`, the consumer loop every line passes through, and `parse_line`'s `cumtime` is more than twice its own `tottime`, because the boundary function's time is really json decoding and pydantic validation underneath it. The tracemalloc check holds the same fixed input to a bounded peak, between 1 and 6 MB, tight enough that a leak would break it and loose enough that a scheduling hiccup cannot.

```
class ProfiledIngest(unittest.TestCase):
    def test_top_of_tottime_is_the_consume_loop(self):
        lines = synthetic_lines(LINES)
        prof = cProfile.Profile()
        prof.enable()
        result, summary = run_phases(lines)
        prof.disable()
        self.assertEqual(len(result.records), LINES)
        self.assertEqual(len(summary.sensors), 20)

        rows = [
            (tt, name, str(fn))
            for (fn, _, name), (_, _, tt, _, _) in pstats.Stats(prof).stats.items()
        ]
        rows.sort(reverse=True)
        top_capstone = [
```

listing 494: the deterministic legs: top of tottime by name, the boundary names present, cumtime against tottime (source: python/capstone/tests/test_profiled.py, lines 34-50)

The opt-in bench, `samples/bench/capstone_profile.py --n 30000`, carries the dated capture the two capstone chapters quote, and part 2's closing section reads it through the profiling chapters' lens.

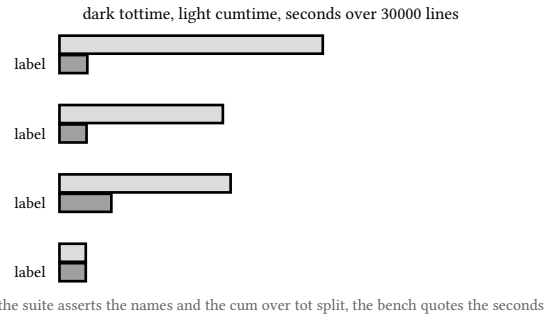


figure 208: the ingest profile as three owners: the consumer loop, the json decode under every line, the pydantic validation under every line

40.6. what part 1 proves

The end-to-end tests run the real pipeline over committed fixtures and check the split by hand:

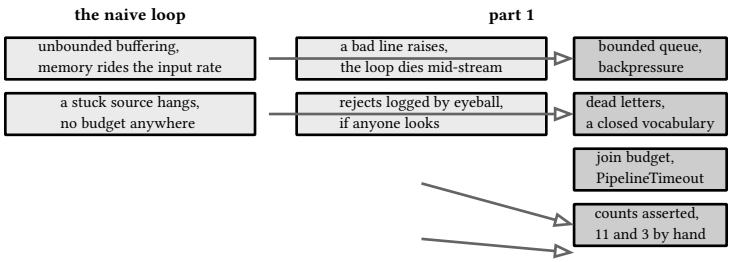
```
    def test_counts_split_between_records_and_dead_letters(self):
        result = run_pipeline(fixture_entries())
        self.assertEqual(len(result.records), 11)
        self.assertEqual(len(result.dead), 3)
        for reading in result.records:
            self.assertIsInstance(reading, models.Reading)

    def test_dead_letter_reasons_and_sources(self):
        result = run_pipeline(fixture_entries())
        by_reason = {}
        for dead in result.dead:
            by_reason.setdefault(dead.reason, []).append(dead)
        self.assertEqual(len(by_reason["schema"]), 1)
        self.assertEqual(by_reason["schema"][0].source, "events-b")
        self.assertEqual(by_reason["schema"][0].detail, "JSONDecodeError")
        self.assertEqual(len(by_reason["field"]), 2)
        self.assertEqual({d.source for d in by_reason["field"]}, {"events-b"})
        details = {d.detail for d in by_reason["field"]}
        self.assertIn("value:missing", details)
        self.assertIn("ts:int_parsing", details)
```

listing 495: the fixture arithmetic: 11 records, 3 dead letters, exact reasons and sources (source: python/capstone/tests/test_pipeline.py, lines 30-49)

`events-a.jsonl` holds 8 valid lines. `events-b.jsonl` holds 3 valid lines and 3 rejects, one not json at all, one missing its value, one with a string timestamp. The pipeline returns 11 records and 3 dead letters, the schema reject carrying `JSONDecodeError` as its detail and the two field rejects carrying `value:missing` and `ts:int_parsing`, all from `events-b`. The sort test then requires the dead letters in `(source, reason, detail, line)` order, whatever order the consumers rejected in. The failure tests close the loop: an adapter that dies mid-stream after two good lines leaves those records intact and files one `source` dead letter, a dead neighbor does not take the good source down with it, and a stuck producer ends in `PipelineTimeout`, never a hang.

The part 1 inventory, from the suite output: `test_sources` 7 tests, 4 over the snapshot adapter and 3 over watch mode, `test_fetch` 12, the 6 real-socket proofs and the 6 monkeypatched taxonomy checks, `test_models` 15, the accept table, the reject table, and the closed vocabulary, `test_pipeline` 10, 5 end to end, 3 source-failure, and 2 shutdown, and `test_profiled` 4, the profile-shape and bounded peak checks of the section above. That is 48 of the suite's 78. The boundaries are the ones stated along the way: records leave at full precision as pydantic objects with no aggregation yet, the only real network in the entire capstone is one loopback listener in one suite, one event loop and a worker thread per blocking call carry all the concurrency, and nothing is persisted anywhere.



each right-hand box is a test, and the tests are the difference

figure 209: what a naive ingest loop does and what part 1 does, side by side

VERIFY: part 1 in the gate

From `books/python/capstone`, the gate's own invocation under the book venv, `python -X utf8 -m unittest discover -s tests -t .`, reports 78 tests and OK. Part 1 owns 48 of them: `test_sources` 7, `test_fetch` 12, `test_models` 15, `test_pipeline` 10, `test_profiled` 4. The remaining 30 belong to part 2, and that chapter quotes its own inventory.

> The fixture arithmetic is pinned by hand: 14 committed input lines split into 11 records and 3 dead letters across 4 sensors, and the real-socket suite binds, serves, and shuts down one loopback listener without the gate ever stalling on a firewall dialog.

sources: docs.python.org/3.14/library/asyncio-queue.html (the Queue class, maxsize, put and get), docs.python.org/3.14/library/asyncio- task.html (to_thread for blocking calls), docs.python.org/3.14/library/ urllib.request.html (urlopen, HTTPError as a response-like exception, URLError wrapping reasons), docs.python.org/3.14/library/http.server (ThreadingHTTPServer), docs.python.org/3.14/library/json.html (the NaN literal json.loads accepts), pydantic.dev/docs/validation/latest/ concepts/fields (Field constraints), pydantic.dev/docs/validation/ latest/errors/errors (ErrorDetails dictionaries with loc and type), learn.microsoft.com/windows/security/operating-system-security/ network-security/windows-firewall/configure-with-command-line (Set-NetFirewallProfile, NotifyOnListen), all accessed 2026-09-12. The 78-test count, the 11-and-3 fixture split, and the loopback firewall popup behavior probed on this machine the same day, the profiled legs and their dated capture added 2026-09-13. Verified by the capstone leg of `make verify-py`, 48 of 78 tests in this chapter's modules.

41. capstone: ingest and analysis pipeline, part 2

Part 1, book 8, ch 40 (python 3.14, capstone: ingest and analysis pipeline, part 1), ended with a list of validated `Reading` objects and a sorted list of dead letters. This chapter turns records into numbers, numbers into one canonical document, the document into a cli, and the cli's payload into a served api. The modules are `analytics.py`, `cpu.py`, `report.py`, `cli.py`, and `api.py`, and their share of the suite is 30 of the 78 tests: `test_analytics` 8, `test_cpu` 3, `test_golden` 5, `test_api` 10, `test_versions` 4. The through-line is the one part 1 set up: every artifact downstream is byte-stable, typed, and pinned, so the golden file, the response models, and the version table can all be checked by machine.

41.1. analytics and the pool

Aggregation is one function with a typed return:

```
def summarize(records) -> Summary:
    # records: iterable of models.Reading; materialized once so both
    # lists and generators are welcome
    rows = [record.model_dump() for record in records]
    frame = pandas.DataFrame(rows)
    if frame.empty:
        return Summary(total=0, sensors={})
    grouped = frame.groupby("sensor")["value"].agg(
        count="size",
        mean=numpy.mean,
        min=numpy.min,
        max=numpy.max,
        median=numpy.median,
    )
    sensors = {
        str(name): SensorStats(
            count=int(row["count"]),
            mean=float(row["mean"]),
            min=float(row["min"]),
            max=float(row["max"]),
            median=float(row["median"]),
        )
        for name, row in grouped.iterrows()
    }
    # groupby sorts already; sorting again pins the contract for the
    # report and the golden diff
    return Summary(total=len(rows), sensors=dict(sorted(sensors.items())))
```

listing 496: summarize: pandas groupby feeding numpy callables, results cast back to plain python numbers (source: python/capstone/analytics.py, lines 23-49)

Records are materialized once through `model_dump`, so lists and generators are equally welcome, and the frame goes through the `groupby`/`agg` core that book 8, ch 25 (python 3.14, pandas) taught. The aggregators are explicit numpy callables, `numpy.mean`, `numpy.min`, `numpy.max`, `numpy.median`, the numeric engine of book 8, ch 24 (python 3.14, numpy) named rather than implied. Two details are contract rather than style. The stats are cast with `int()` and `float()` on the way out, because a numpy scalar that survived into the json report would poison it, and a test asserts every mean and median is a plain `float`. And the sensors mapping is sorted after the `groupby`, which sorts already, so the report order is pinned by the code itself instead of by pandas' current defaults. The module keeps full precision, rounding to 3 decimals belongs to `report.py` alone, and the analytics tests choose values as binary fractions, halves and quarters, so every mean and median is exact in float arithmetic and the assertions carry no rounding story at all.

The pool variant exists for the reason book 8, ch 18 (python 3.14, multiprocessing and process patterns) gave: pandas work on the loop thread freezes every other coroutine, and the escape is a process.

```
# capstone cpu: the heavy aggregation offloaded to a process pool, so
# pandas work never freezes the event loop thread. spawn-safe by
# construction: the worker is a module-level function, data crosses as
# plain dicts, and importing this module executes nothing.
# one submit, not sharding: count/mean/min/max would merge across
# shards, but a median of medians is a lie, so the whole frame goes to
# one worker and the pool buys the loop its freedom, not a speedup.
from concurrent.futures import ProcessPoolExecutor

import analytics
import models


def _summarize_rows(rows: list) -> dict:
    # worker body, unpickled in the child: rebuild readings, aggregate,
    # hand back plain json data
    return analytics.summarize(models.Reading(**row) for row in rows).model_dump()


def run_pooled(records: list, workers: int = 2):
    # the pool path over the same contract as analytics.summarize
    rows = [record.model_dump() for record in records]
    if not rows:
        return analytics.summarize([])
    with ProcessPoolExecutor(max_workers=workers) as pool:
        payload = pool.submit(_summarize_rows, rows).result()
    return analytics.Summary.model_validate(payload)
```

listing 497: the whole pool module: a module-level worker, plain dict payloads, one submit (source: python/capstone/cpu.py, lines 1-27)

`run_pooled` is spawn-safe by construction, which on windows is the only construction that works: the worker is a module-level function, the payload and the result are plain dicts, and importing this module executes nothing. The `__main__` guard the spawn rules require lives where an entry point belongs, at the bottom of `cli.py`. The single submit is a documented choice, and the module's own header states the arithmetic: count, mean, min, and max would merge cleanly across shards, but a median of medians is a lie, so the whole frame goes to one worker and the pool buys the event loop its freedom rather than a speedup. The parity tests pin the contract on spawned windows workers: the pooled summary equals the inline summary to the last float, the return is a typed `Summary`, and empty input skips the pool entirely. The data boundary is pickle: rows cross as dicts, and on this interpreter the default protocol is 5, probed, the protocol PEP 574 introduced for out-of-band buffers. Payloads here are json-shaped and small, so the buffer machinery never enters the picture, and returning `model_dump` data rather than numpy structures keeps it that way.

NOTE: the pool is freedom, not throughput

With one submit, the aggregation is not parallel at all: a single spawned worker does the whole `groupby`. What the pool buys is the loop thread back, so a slow frame delays heartbeats and queue traffic by a pickle round trip instead of by the full aggregation time. The cli uses the inline path, because at fixture scale serializing the rows costs more than aggregating them, and `run_pooled` stands as the tested offload pattern for the frames where that trade flips.

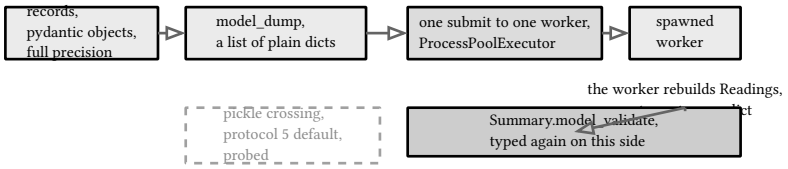


figure 210: the pool crossing: what is pickled, what is spawned, and what comes back

41.2. the cli and logging

The command line is argparse subcommands, the pattern of book 8, ch 19 (python 3.14, operational plumbing), and the exit-code table is the module's first comment: 0 ok, 1 runtime failure, 2 usage from argparse itself, 3 serve without uvicorn. Logging owns stderr and nothing else:

```
def setup_logging(verbose: bool) -> None:
    logging.basicConfig(
        stream=sys.stderr,  # stdout belongs to the report document
        level=logging.DEBUG if verbose else logging.INFO,
        format="%(levelname)s %(name)s %(message)s",
        force=True,  # repeated main() calls rebind to the live stderr
    )
```

listing 498: setup_logging: stderr for logs, stdout reserved for the report document (source: python/capstone/cli.py, lines 22-28)

`force=True` is there for the tests: `main()` is called repeatedly in one process, and basicConfig would otherwise silently no-op after the first call, leaving later runs logging into a stale stderr. The verbosity flag flips between INFO and DEBUG, and the format is level, logger name, message, with no timestamps because the gate compares output, not vibes.

```
def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        prog="capstone", description="async ingest pipeline over jsonl sources"
    )
    parser.add_argument("-v", "--verbose", action="store_true", help="debug logging")
    sub = parser.add_subparsers(dest="command", required=True)

    report_cmd = sub.add_parser("report", help="one-shot ingest, print the report")
    report_cmd.add_argument("sources", nargs="+", help="jsonl files to ingest")
    report_cmd.add_argument(
        "-o", "--output", type=Path, default=None, help="write the report here"
    )

    serve_cmd = sub.add_parser("serve", help="serve the report over http (uvicorn)")
    serve_cmd.add_argument("sources", nargs="+", help="jsonl files to ingest")
    serve_cmd.add_argument("--host", default="127.0.0.1", help="bind address")
    serve_cmd.add_argument("--port", type=int, default=8000, help="bind port")
    return parser
```

listing 499: the parser: required subcommands, a Path-valued output, host and port defaults (source: python/capstone/cli.py, lines 86-103)

The `serve` subcommand carries the capstone's most explicit boundary:

```
def command_serve(args) -> int:
    try:
        import uvicorn
    except ImportError:
        log.error(
            "serve needs uvicorn, which is not installed: add the fastapi "
            "standard extra (pip install 'fastapi[standard]') and retry"
        )
        return 3
    try:
        document = make_report(args.sources)
    except (OSError, pipeline.PipelineTimeout) as exc:
        log.error("serve failed: %s", exc)
        return 1
    log.info("serving the report on %s:%d", args.host, args.port)
    uvicorn.run(api.create_app(document), host=args.host, port=args.port)
    return 0
```

listing 500: command_serve: the uvicorn import is the boundary, exit 3 explains itself (source: python/capstone/cli.py, lines 67-83)

`uvicorn` is imported inside the function, and an `ImportError` is a stated condition with an exit code of its own, a logged hint naming the fastapi standard extra, and no traceback. The pins suite asserts the venv really is without uvicorn, so the branch is tested behavior, not dead code. Before serving, `make_report` checks every path with `is_file()`, so a missing source is a clean exit 1 with the name on stderr, and source names are file stems, which is why the committed report says `events-a` and never an absolute path. The written output uses `newline=""` so windows does not translate the canonical document's `\n` into `\r\n` on disk.

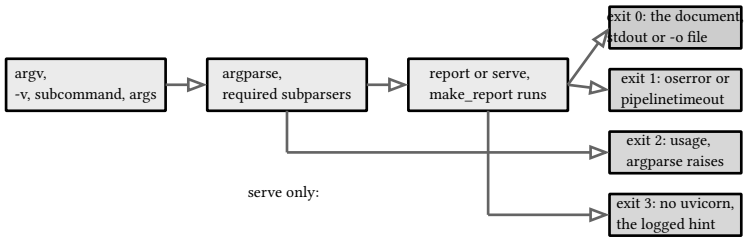
```
class CliEdgesTest(unittest.TestCase):
    def test_usage_error_exits_2(self):
        with contextlib.redirect_stderr(io.StringIO()):
            with self.assertRaises(SystemExit) as caught:
                cli.main(["report"])
        self.assertEqual(caught.exception.code, 2)

    def test_missing_source_exits_1(self):
        stderr = io.StringIO()
        with (
            contextlib.redirect_stderr(stderr),
            contextlib.redirect_stdout(io.StringIO()),
        ):
            code = cli.main(["report", str(FIXTURES / "nope.jsonl")])
        self.assertEqual(code, 1)
        self.assertIn("no such source file", stderr.getvalue())

    def test_serve_without_uvicorn_exits_3(self):
        # honest boundary: uvicorn is not in the venv (the pins assert
        # that), so serve explains the fastapi standard extra and fails
        # loudly rather than silently skipping
        stderr = io.StringIO()
        with contextlib.redirect_stderr(stderr):
            code = cli.main(["serve", *SOURCES])
        self.assertEqual(code, 3)
        self.assertIn("fastapi[standard]", stderr.getvalue())
```

listing 501: the cli edges: usage exits 2, a missing source exits 1, serve without uvicorn exits 3 (source: python/capstone/tests/test_golden.py, lines 60-86)

All three edges are asserted with stderr captured, including the `SystemExit` argparse raises for the usage error, so the exit-code table is test output rather than a comment.



logs ride stderr at every exit, the document alone rides stdout

figure 211: one argv through the cli, and every exit code it can produce

41.3. the golden report

The report module is 44 lines and owns two decisions: the document shape and its byte-exact rendering.

```
# capstone report: the canonical json document. one schema version,
# sorted keys, every float rounded to 3 decimals (ROUNDING below), dead
# letters in a stable sort, sensors name-sorted. canonical() renders
# the exact text the golden test, the cli, and the api agree on.
import json

SCHEMA_VERSION = 1
ROUNDING = 3


def build_report(result, summary) -> dict:
    # result: pipeline.PipelineResult, summary: analytics.Summary
    return {
        "schema": SCHEMA_VERSION,
        "records": summary.total,
        "dead_letters": len(result.dead),
        "dead": [
            {
                "source": dead.source,
                "reason": dead.reason,
                "detail": dead.detail,
                "line": dead.line,
            }
            for dead in sorted(
                result.dead, key=lambda d: (d.source, d.reason, d.detail, d.line)
            )
        ],
        "sensors": {
            name: {
                "count": stats.count,
                "mean": round(stats.mean, ROUNDING),
                "min": round(stats.min, ROUNDING),
                "max": round(stats.max, ROUNDING),
                "median": round(stats.median, ROUNDING),
            }
            for name, stats in sorted(summary.sensors.items())
        },
    }


def canonical(report: dict) -> str:
    # the byte-exact rendering: sorted keys, indent 2, one trailing
    # newline so the file is posix-friendly
    return json.dumps(report, sort_keys=True, indent=2) + "\n"
```

listing 502: the whole module: schema version, rounding, the stable sorts, and the canonical renderer (source: python/capstone/report.py, lines 1-44)

`build_report` takes the pipeline result and the summary and produces a dict with one schema version, the record and dead-letter counts, the dead letters under the same sort key the pipeline already applied, and the sensors name-sorted with every float rounded to 3 decimals. `canonical` renders it: `json.dumps` with sorted keys, indent 2, and one trailing newline so the file is posix-friendly. Those choices, plus the `loc:type` detail format part 1 chose for exactly this moment, are what make the document deterministic from fixed inputs.

The golden test then pins the whole chain, cli included:

```
class GoldenReportTest(unittest.TestCase):
    def run_cli(self, *extra):
        captured = io.StringIO()
        with contextlib.redirect_stdout(captured):
            code = cli.main(["report", *SOURCES, *extra])
        return code, captured.getvalue()

    def assert_matches_golden(self, text):
        # a unified diff rides in the failure message, so golden drift
        # reads straight out of the gate log
        diff = "".join(
            difflib.unified_diff(
                read_golden().splitlines(keepends=True),
                text.splitlines(keepends=True),
                fromfile="tests/golden/report.json",
                tofile="cli report",
            )
        )
        self.assertEqual(text, read_golden(), "golden drift:\n" + diff)

    def test_stdout_matches_golden(self):
        code, printed = self.run_cli()
        self.assertEqual(code, 0)
        self.assert_matches_golden(printed)

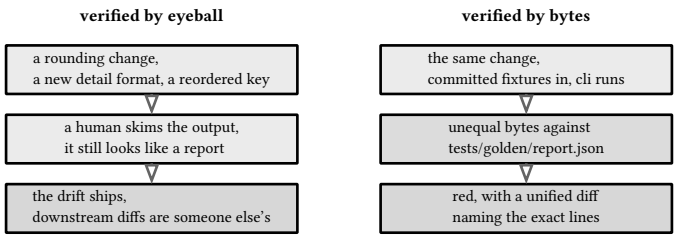
    def test_output_file_matches_golden(self):
        with tempfile.TemporaryDirectory() as tmp:
            target = Path(tmp) / "report.json"
            code, _ = self.run_cli("-o", str(target))
            self.assertEqual(code, 0)
            written = target.read_text(encoding="utf-8")
        self.assert_matches_golden(written)
```

listing 503: the golden suite: the real cli over committed fixtures, diffed byte for byte (source: python/capstone/tests/test_golden.py, lines 26-58)

The test calls `cli.main` with the two fixture paths and stdout captured, and requires the printed text to equal `tests/golden/report.json` byte for byte, a file committed alongside the fixtures. The assertion's failure message carries a unified diff between the golden file and the new output, so drift reads straight out of the gate log with line numbers. The second check writes through `-o` into a temp file and requires the same bytes on disk. The committed document says 11 records, 3 dead letters, and 4 sensors: alpha with a count of 4, beta 3, gamma 3, epsilon 1. Golden, not eyeball: any change to a detail format, a rounding step, a sort order, or the canonical renderer reddens this diff, and the number in this paragraph is checkable against the file.

PITFALL: two newline traps sit between green and a false red

The cli writes the canonical document with `newline=""` because a default text-mode write on windows translates every `\n` to `\r\n`, and the byte comparison would fail on an artifact of the platform rather than of the code. The golden reader uses ordinary `read_text`, whose universal newlines fold any `\r\n` back to `\n`, so an editor flipping the committed file's line endings cannot fake drift either. Both behaviors are load bearing and neither is visible in the diff when they go wrong, only in the test that catches them.



golden files turn formatting into a contract: the eyeball never sees the file at all

figure 212: the same behavior change, verified by eyeball and verified by bytes

41.4. the served api

The api module is a factory over the finished artifact:

```python
class ReportResponse(BaseModel):
    # "schema" is reserved vocabulary on BaseModel, so the field is
    # declared under its python name and aliased back to the report key
    model_config = ConfigDict(populate_by_name=True)

    schema_version: int = Field(alias="schema")
    records: int
    dead_letters: int
    dead: list[DeadItemResponse]
    sensors: dict[str, SensorStatsResponse]
```

listing 504: the response model: schema is reserved vocabulary on BaseModel, so the field carries an alias (source: python/capstone/api.py, lines 27-36)

`schema` is a pydantic reserved name on model classes, so the python field is `schema_version` with `alias="schema"` and `populate_by_name=True`, and the wire format stays the canonical document's key. The factory wires the report in through a lifespan, the pattern book 8, ch 22 (python 3.14, fastapi) taught:

```python
def create_app(report: dict) -> FastAPI:
    @asynccontextmanager
    async def lifespan(app: FastAPI):
        app.state.report = report
        try:
            yield
        finally:
            app.state.report = None

    app = FastAPI(title="capstone ingest pipeline", version="1", lifespan=lifespan)

    def current_report():
        # 503, not a traceback, if a request slips outside the lifespan
        loaded = getattr(app.state, "report", None)
        if loaded is None:
            raise HTTPException(status_code=503, detail="report not loaded")
        return loaded

    @app.get("/report", response_model=ReportResponse)
    async def get_report():
        return current_report()
```

listing 505: create_app: lifespan state, the 503 guard, and the report endpoint (source: python/capstone/api.py, lines 46-66)

`create_app` never imports `cli`, so the app constructs without any argparse side effects, and the lifespan is what puts the report on `app.state` between startup and shutdown, clearing it in a `finally`. A request that slips outside the lifespan meets `current_report`, which answers 503 rather than raising an `AttributeError` through the framework. The two query-shaped endpoints finish the surface:

```
    @app.get("/stats", response_model=StatsResponse)
    async def get_stats(min_count: int = Query(default=1, ge=1)):
        data = current_report()
        counts = [entry["count"] for entry in data["sensors"].values()]
        return StatsResponse(
            records=data["records"],
            dead_letters=data["dead_letters"],
            sensors_total=len(counts),
            sensors_shown=sum(1 for count in counts if count ≥ min_count),
        )

    @app.get("/sensors/{name}", response_model=SensorStatsResponse)
    async def get_sensor(name: str):
        data = current_report()
        try:
            return data["sensors"][name]
        except KeyError:
            raise HTTPException(
                status_code=404, detail=f"unknown sensor: {name}"
            ) from None
```

listing 506: stats with a validated query parameter, and the sensor lookup with an honest 404 (source: python/capstone/api.py, lines 68-88)

`min_count` arrives through `Query(default=1, ge=1)`, so a zero is a 422 from the framework before any handler code runs, and the unknown sensor name is a 404 with the name in the detail. Every response rides a pydantic response model, so the serialized shape is a contract even though the handler returns plain dicts.

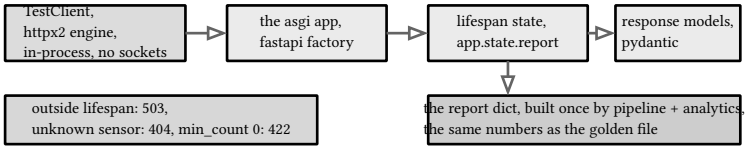
The suite drives the app in-process with fastapi's `TestClient`, no sockets anywhere, the engine under it on this starlette line being httpx2, the test-only pin the versions section pins from the other side. The report under test is not a fixture dict: `setUpClass` builds it by running the real pipeline and analytics over the committed fixtures, so the api numbers are the golden numbers. The lifespan contract has its own pair of tests:

```
    def test_lifespan_clears_state_on_shutdown(self):
        with self.client():
            self.assertIsNotNone(self.app.state.report)
        self.assertIsNone(self.app.state.report)

    def test_request_outside_lifespan_is_503(self):
        # never entering the lifespan leaves no report loaded: an honest
        # 503, not an attribute error
        client = TestClient(self.app)
        response = client.get("/report")
        self.assertEqual(response.status_code, 503)
```

listing 507: entering the TestClient context runs the lifespan, and never entering it answers 503 (source: python/capstone/tests/test_api.py, lines 102-112)

uvicorn appears in prose only, here and in the cli's serve branch: the book's venv carries no server, the exit-3 contract says so out loud, and installing the fastapi standard extra is the documented path from prose to a running process.



serve in prose: uvicorn.run(api.create_app(document)) when the standard extra is installed, exit 3 with the hint when it is not, and the pins assert this venv has none

figure 213: the served report's layer stack, and the status codes at the edges

41.5. version pins asserted

The capstone runs on one toolchain, and the suite says which one before any behavioral test trusts the api shapes underneath it:

```
class PinsTest(unittest.TestCase):
    def test_interpreter(self):
        # the gate resolves the venv built from pinned cpython 3.14
        self.assertEqual(platform.python_version(), "3.14.7")

    def test_library_pins(self):
        # requirements.txt, verbatim. httpx2 is test-only: the engine
        # under fastapi's TestClient on the current starlette line,
        # never imported by the package.
        expected = {
            "fastapi": "0.141.1",
            "starlette": "1.6.0",
            "httpx2": "2.12.0",
            "pydantic": "2.13.4",
            "pandas": "3.0.5",
            "numpy": "2.5.3",
        }
        for name, pinned in expected.items():
            with self.subTest(name=name):
                self.assertEqual(version(name), pinned)

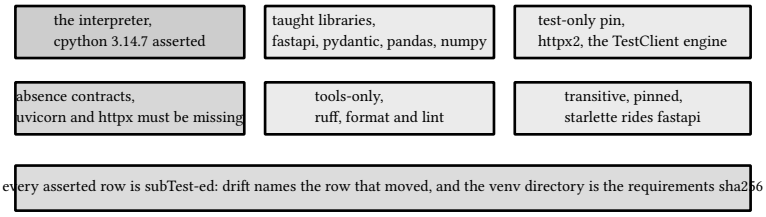
    def test_uvicorn_absent(self):
        # serve is an honest stub: it explains uvicorn is missing and
        # exits 3. if uvicorn ever lands in the venv, that contract and
        # this assert change together.
        with self.assertRaises(ModuleNotFoundError):
            __import__("uvicorn")

    def test_httpx_absent(self):
        # starlette deprecates httpx for TestClient in favor of
        # httpx2; if httpx ever sneaks back in, the engine under the
        # api tests silently changes and this assert fires
        with self.assertRaises(ModuleNotFoundError):
            __import__("httpx")
```

listing 508: the pins: the interpreter, the six library versions, and the two absence contracts (source: python/capstone/tests/test_versions.py, lines 9-42)

The interpreter assert is the same 3.14.7 every sample pins since book 8, ch 1 (python 3.14, toolchain: cpython 3.14 on windows). The six library pins are `requirements.txt` verbatim, each under its own `subTest` so drift names the library that moved: fastapi 0.141.1, starlette 1.6.0, httpx2 2.12.0, pydantic 2.13.4, pandas 3.0.5, numpy 2.5.3. The two absence contracts are the interesting half. uvicorn must be missing, because the cli's exit 3 is a tested promise that a quietly installed server would break, and the comment says the assert and the contract change

together. httpx must be missing because starlette moved its TestClient engine to httpx2, and if httpx ever sneaks back into the venv, the machinery under the api tests silently changes with no test failing, which is precisely when an assert should fire. The gate closes the loop from the other side: the venv's directory name is derived from the requirements sha256, so a changed pin builds a fresh venv, and this table is what describes the inside of it.



a green row here is what lets every other test trust the shapes it is asserting against

figure 214: the pin families: what is taught, what is test-only, what is absent, what is transitive

41.6. profiled, and why

The bench behind part 1's profiled section runs the whole pipeline at 30000 lines and prints the deterministic profiler's table, committed as the dated capture:

```
$ python capstone_profile.py --n 30000
python 3.14.7, pydantic boundary over 30000 lines, 20 sensors, 0 dead letters
         1531497 function calls (1531202 primitive calls) in 0.639 seconds

   Ordered by: internal time
   List reduced from 817 to 10 due to restriction <10>

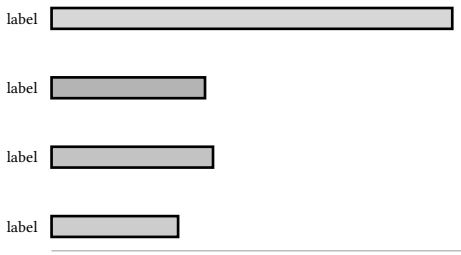
   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
    30000    0.038    0.000    0.097    0.000 decoder.py:340(decode)
     7502    0.036    0.000    0.340    0.000 pipeline.py:40(_consume)
    30000    0.035    0.000    0.211    0.000 models.py:43(parse_line)
    30021    0.034    0.000    0.034    0.000 {method 'validate_python' of
'pydantic_core._pydantic_core.SchemaValidator' objects}
    30000    0.029    0.000    0.029    0.000 decoder.py:351(raw_decode)
    30002    0.027    0.000    0.086    0.000 queues.py:156(put_nowait)
    37502    0.025    0.000    0.088    0.000 queues.py:172(get)
    30000    0.024    0.000    0.024    0.000 {method 'to_python' of
'pydantic_core._pydantic_core.SchemaSerializer' objects}
    30000    0.021    0.000    0.125    0.000 __init__.py:304(loads)
     7509    0.020    0.000    0.551    0.000 base_events.py:1977(_run_once)
```

listing 509: the capstone under cProfile: 1.5 million calls, 0.639 seconds, the boundary and the queue machinery on top (source: python/samples/bench/captures/2026-09-13-capstone-profile.txt, lines 1-18)

Read through book 8, ch 14 (python 3.14, profiling: cprofile, sys.monitoring, and p99)'s lens, the first thing the table says is which times are facts: the ranking is stable rerun after rerun because the profiler is deterministic, so "json decode and `parse_line` and `validate_python` own the top" is a statement about code, not about a lucky afternoon. The second is what cumtime is for: `parse_line` carries 0.211 s of cumulative time against 0.035 s of its own, so the boundary function is a toll booth, the toll is paid underneath it in json and pydantic core, and optimizing the Python body of `parse_line` would optimize the smallest number on its row. `_run_once`, the event loop's own step function, carries 0.551 s cumulative of the 0.639 s total, which is the honest price of the async design: most of the wall time is suspension and resumption machinery, visible only because the profiler counts calls, not because any of it is slow.

Read through book 8, ch 13 (python 3.14, pymalloc: arenas, freelists, and tracemalloc)'s lens, the same run is an allocator story. Thirty thousand lines each allocate a decoded dict, a `Reading`, floats and strings that die on the spot,

and the freelists absorb the churn, which is why the tracemalloc column reads 18154289 current against 26640564 peak: the surviving 18 MB is the materialized records list the analytics phase needs, and the 8 MB above it at the crest is pandas building its frame, released when `summarize` returns. Nothing grows per line; the peak tracks the frame build, not the ingest.



fractions of the 0.639 s total, cumtime over 30000 lines, capture of 2026-09-13

figure 215: the 0.639 s capstone run split by owner: the event loop's cumulative shadow over everything, the boundary's third, the queue plumbing between

41.7. honest boundaries

What this capstone is not is part of its design, so it gets stated with the same care as the pins. It is not a production ingestion system: nothing is persisted beyond an optional file write, there are no retries or backoff for a flaky source, no continuous metrics or telemetry, no horizontal scale, and one machine carries all of it. The served api is a snapshot, not live queries: the report is built once before the server starts, so a reading ingested after startup never appears in a response. The analytics is one-shot batch over a materialized list, not streaming, and the aggregation cost grows with the record count by design. The security posture is a loopback bind with no auth, no tls, and no rate limits, which is what a teaching artifact should be and what a deployment is not.

The scale limits are equally concrete, and the metrics story split when the profiled sections were added. The fixtures are 14 lines and the golden numbers are hand-counted, so the arithmetic is checkable, and what the capstone measures about itself is now real and named: the in-suite profile-shape and bounded-peak checks of the two profiled sections, the dated 30000-line capture above, and the kernel scales in book 8, ch 14 (python 3.14, profiling: cprofile, sys.monitoring, and p99), 1m and 10m measured, 100m as deleted one-offs, 1b by math. What stays unmeasured is production throughput: no number here claims how fast this pipeline ingests a flaky real source over a real network, because the fixtures cannot answer that and the book will not invent it. The raw queue bounds at 8 because backpressure is the lesson, not the tuning. The pool is one submit for the median's sake, and the join budget is 30 seconds because a stuck source must end in a raise, not because 30 was measured. The next steps in a real system follow from the same boundaries: rotation-aware source adapters where the shrink bound now stands, a persistent dead-letter sink with replay, fetch retries with exponential backoff and jitter, sharded aggregation where medians need raw values or sketches on the workers, uvicorn or equivalent under the factory with real process supervision, and a schema-versioning story for the golden document the day its shape changes.

the capstone, as built	what production would add
sources: snapshot, follow, shrink bound	rotation, checkpoints, a durable log
fetch: one attempt, typed failures	retries with jitter, circuit breaking
pipeline: one loop, in-memory queues	a broker, persistence, horizontal scale
analytics: one-shot batch, one submit	streaming shards, sketches for medians
api: a snapshot, loopback, no auth	a supervised server, tls, auth, limits

every left box is tested and pinned, every right box is deliberately absent and said so

figure 216: each capstone layer, and what a production system would add beside it

VERIFY: the capstone in the gate

The full suite, run the way the gate runs it from `books/python/capstone`, `python -X utf8 -m unittest discover -s tests -t .` under the book venv, reports 78 tests and OK. Part 2 owns 30 of them: `test_analytics` 8, `test_cpu` 3, `test_golden` 5, `test_api` 10, `test_versions` 4, the profiled legs living in part 1's inventory. The golden contract holds byte for byte on both paths, stdout and the `-o` file, and the pins table asserts the exact six library versions and the two absences the exit-3 contract depends on. Verified by `make verify-py`, 1103 checks and 78 capstone tests in chapters 1 through 28.

41.8. the service part after the capstone

The capstone is this book's teaching artifact, and the next evidence layer was built after it, on the same manual: the service part under `books/python/api`, a stdlib-exclusive vehicle on http.server with a hand-rolled route table, seven layers reading trace through recover, and a sqlite store. The findings register dates the service-part wave 2026-09-27 and records the suite at 313 tests over 22 modules with the 16 frozen vectors as the spine, and the adversarial closeout grew it to the 314 that stands. `make verify-pyapi` gates it, with the docker lane and the opt-in pyapi-cover lane as the second and third gates outside the plain verify chain.

sources: fastapi.tiangolo.com/advanced/events (lifespan as an async context manager with yield), fastapi.tiangolo.com/tutorial/testing (TestClient), pydantic.dev/docs/validation/latest/concepts/fields (Field constraints), pydantic.dev/docs/validation/latest/errors/errors (ErrorDetails with loc and type), pandas.pydata.org/docs/user_guide/ groupby.html (groupby aggregation), numpy.org/doc/stable/reference/ generated/ numpy.mean.html (the ufunc used as an aggregator), peps.python.org/pep-0574 (pickle protocol 5 and out-of-band buffers), docs.python.org/3.14/library/concurrent.futures.html (ProcessPoolExecutor), docs.python.org/3.14/library/argparse.html (subparsers, usage errors exiting 2), docs.python.org/3.14/library/ logging.html (basicConfig, force), all accessed 2026-09-12. The 78-test count, the pooled-versus-inline parity, the default pickle protocol of 5, and the exit codes 0 through 3 probed on this machine the same day, the profiled capture and its lens taken 2026-09-13. Verified by `make verify-py`, 30 of 78 capstone tests in this chapter's modules.

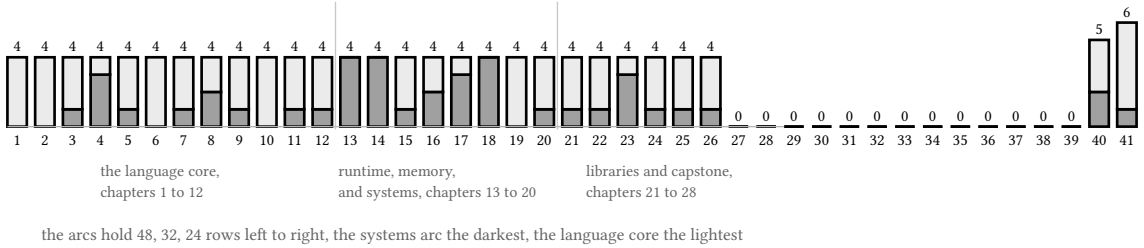
42. appendices: coverage and sources

Twenty-six content chapters and the two capstone parts stand behind this book and this chapter adds no claim to any of them. It computes two views over what those chapters left behind, the topic matrix at `coverage/topics.typ`, one row per verified topic with its cost and its nearest stdlib or library counterpart, and the pinned source list at `coverage/sources.typ`, one row per canonical page a chapter actually consulted, then freezes the version pins those chapters asserted. Every number on this page is computed by this page from those arrays, and the asserts below fail the build when the matrix and the manifest disagree, so the appendix cannot drift from the corpus it describes.

42.1. topic matrix

The matrix holds 115 topic rows across the 41 chapters and none unmapped, and that zero is the assert above, not prose: each row's chapter field is looked up against the manifest ids and a row naming an id no chapter carries stops the build instead of rendering a stray bar. The distribution is flat by design, 26 chapters carry 4 rows, and the two capstone parts carry the only exceptions, 1 rows for part 1 and 6 for part 2, the chapters whose opening and closing sections the plan let grow past the four-section grammar. The sparsest count is 0 and no arc owns the ledger: chapters 1 through 12 hold 48 rows for the language core, 13 through 20 hold 32 for the runtime, memory, and systems chapters, and 21 through 28 hold 24 for the libraries and the capstone.

The strip splits every bar by cost. Dark is the 41 rows that carry a measured or probed fact, a probe count, a byte count, a version census, a pinned divergence. Light is the 74 semantics and documentation rows, the honest boundary the book refuses to dress up as a measurement. The runtime and systems arc is the darkest, 19 of 32 rows measured, because its chapters assert behavior in running samples, pipe deadlocks, worker caps, race outcomes, thread-to-process ratios, and a later pass added its two most measured chapters, pymalloc and profiling, with not one n/a row between them. The language core is the lightest by share, 11 of 48, semantics rows for the most part with the gc thresholds and the growth ladders the exceptions. The library arc sits between, 8 of 24, the probed promotion and interpolation facts against quoted documentation.



the arcs hold 48, 32, 24 rows left to right, the systems arc the darkest, the language core the lightest

figure 217: the distribution strip: one bar per chapter in manifest order, the number above a bar is its topic count, dark is a row with a measured or probed cost, light is an n/a row

42.2. sources ledger

Every chapter ends with its own sources line and this section is all of them rolled up: 79 rows over 79 distinct urls, no page consulted twice, every row carrying its own access date. The rows name 2 verification passes, #date-counts.at(p.at(0)) rows on #p.at(1), #date-counts.at(p.at(0)) rows on #p.at(1), the 2026-09-12 pass that verified the book and the later pass that added the memory and measurement chapters, each row dated by its own fetch [the single date because the book's verification ran as one pass so far]. The host census is computed the same way and its counts are rows per family, not fetches. The python software foundation carries the load, 47 rows on docs.python.org for the library and reference pages plus 12 on peps.python.org, the numbered proposals the language chapters quote. The third-party houses follow: fastapi.tiangolo.com carries 5 for the taught web chapter and its release notes, numpy.org 6 including the nep 50 promotion proposal, the pydantic family 4 across two host forms, docs.pydantic.dev for the fields page and pydantic.dev for the validation concepts, the current canonical home, and pandas.pydata.org 3. The last two rows sit outside the language entirely: github.com carries the ruff

changelog row that dates the formatter pin, and learn.microsoft.com carries the netsh command reference behind the firewall preflight that keeps the capstone's real-socket test from popping a dialog on this machine.

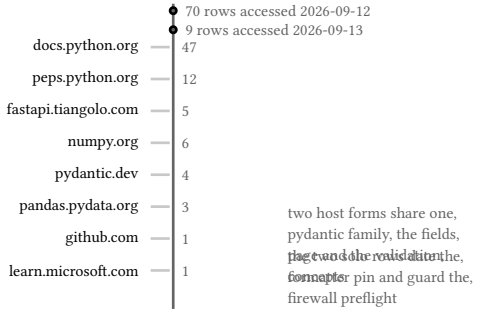


figure 218: the pin set on its access timeline: one circle per verification pass, one row per host family, the count beside each label computed from the array

42.3. the version drift rule

Four moves happened under this book while it was written, and each is recorded rather than silently repaired. The register's environment facts paragraph carries the first two. The taught web library switched from httpx to fastapi 0.141.1 before chapter 22 was written, one user decision mid-build, with 3.14 support verified back to 0.118.3, and the capstone's client leg settled on stdlib urllib so the taught set stayed honest about what a boundary fetch needs. Then the gate surfaced starlette 1.6.0 deprecating httpx inside TestClient with the message "install httpx2 instead", so the test-only pin moved to httpx2 2.12.0 and the deprecation itself became a taught boundary fact in chapter 22. The third move is ruff: 0.16.0 grew its default rule set from 59 rules to 413, so the book freezes its selection at E4, E7, E9, and F with line length 88, because a format gate that changes meaning between patch releases is not a gate. The fourth is environment drift rather than package drift: pandas hard-depends on tzdata, so the gate venv resolves timezones while the bare per-user 3.14.7 raises ZoneInfoNotFoundError even for utc, which chapter 19 teaches two-sided rather than papering over. Row pyt-002 is the arithmetic cousin of the family: the figure plan header said 109 rows while the table carried 110 after the fastapi switch gave the capstone its sixth section, and the re-total is the fix.

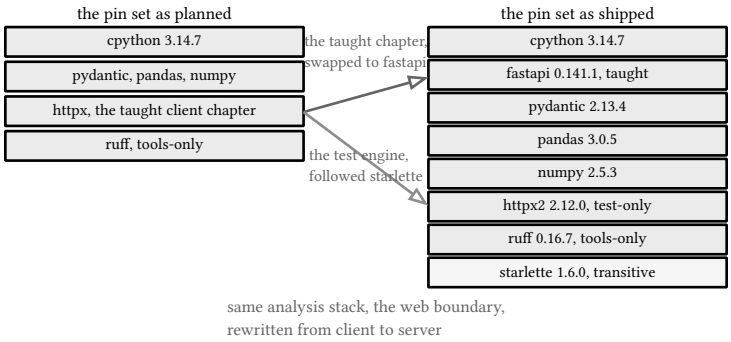


figure 219: before and after: the pin set as planned against the pin set as shipped, one user decision and two gate discoveries apart

NOTE: version drift rule

The pins this book froze: cpython 3.14.7 for every sample and the in-sample patch assert, fastapi 0.141.1 as the taught web framework, pydantic 2.13.4, pandas 3.0.5, numpy 2.5.3, httpx2 2.12.0 as the test-only TestClient engine, ruff 0.16.7 as the tools-only formatter with its rule selection frozen, and starlette 1.6.0 riding along transitive and pinned. Packages and pages will move again: a deprecation lands, a promotion rule changes, a default grows. Cite what you fetched, assert the exact version from inside the sample, and treat the pins and access dates as the only durable facts. The 79 rows behind the timeline are that record: when a future fetch disagrees with a row here, the row is what this book claimed and the date is when it was true.

VERIFY: the gate behind the matrix

> `make verify-py` over the tree this chapter lands in reports: 107 files, 1103 checks, 18 dis assertions, format clean, 78 capstone tests. This chapter ships no samples, so its own checks are the three asserts above, the unmapped zero, the family and date censuses, and the distinct-url count, plus the compile that turns the arrays into the numbers on this page. Every count quoted in the verify callouts of the written chapters rolls up into that one gate line, and the line is reproducible from a clean venv whose name is the hash of the requirements file that built it.

sources: `coverage/topics.typ` and `coverage/sources.typ`, the two arrays this page computes every number from, and docs/audit/python.md, register rows pyt-001 and pyt-002 plus the environment facts paragraph, the drift record this chapter generalizes. All 79 source rows accessed 2026-09-12 and 2026-09-13, no new pages fetched for this appendix, it cites the record the chapters already left behind. Sample behavior throughout the book verified by `make verify-py`, 1103 checks behind the matrix this chapter computes.